

Processor Programming Reference (PPR) for AMD Family 19h Model 11h, Revision B2 Processors Volume 2 of 9

Legal Notices

© 2018-2025 Advanced Micro Devices, Inc. AMD Confidential. All rights reserved.

The information contained herein is for informational purposes only, and is subject to change without notice. While every precaution has been taken in the preparation of these materials, it may contain technical inaccuracies, omissions and typographical errors, and AMD is under no obligation to update or otherwise correct this information. Advanced Micro Devices, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of these materials, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of AMD hardware, software or other products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by these materials. Terms and limitations applicable to the purchase or use of AMD's products are as set forth in a signed agreement between the parties or in AMD's Standard Terms and Conditions of Sale.

Trademarks:

AMD, the AMD Arrow logo, and combinations thereof are trademarks of Advanced Micro Devices, Inc.

AGESA is a trademark of Advanced Micro Devices, Inc.

AMD Virtualization is a trademark of Advanced Micro Devices, Inc.

AMD-V is a trademark of Advanced Micro Devices, Inc.

Adobe is a registered trademark of Adobe.

Arm is a registered trademark of Arm Limited.

CXL is a trademark of Compute Express Link Consortium, Inc.

Compute Express Link is a trademark of Compute Express Link Consortium, Inc.

DirectX is a registered trademark of Microsoft Corporation.

EPYC is a trademark of Advanced Micro Devices, Inc.

HyperTransport is a trademark of HyperTransport Consortium.

Infinity Fabric is a trademark of Advanced Micro Devices, Inc.

Linux is a registered trademark of Linus Torvalds.

MIPI I3C is a registered trademark of MIPI Alliance.

Microsoft is a registered trademark of Microsoft Corporation.

PCI Express is a registered trademark of PCI-SIG Corporation.

PCIe is a registered trademark of PCI-SIG Corporation.

SoundWire is a registered trademark of MIPI Alliance, Inc.

Windows is a registered trademark of Microsoft Corporation.

Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

Reverse engineering or disassembly is prohibited.

USE OF THIS PRODUCT IN ANY MANNER THAT COMPLIES WITH THE MPEG ACTUAL OR DE FACTO VIDEO AND/OR AUDIO STANDARDS IS EXPRESSLY PROHIBITED WITHOUT ALL NECESSARY LICENSES UNDER APPLICABLE PATENTS. SUCH LICENSES MAY BE ACQUIRED FROM VARIOUS THIRD PARTIES INCLUDING, BUT NOT LIMITED TO, IN THE MPEG PATENT PORTFOLIO, WHICH LICENSE IS AVAILABLE FROM MPEG LA, L.L.C., 6312 S. FIDDLERS GREEN CIRCLE, SUITE 400E, GREENWOOD VILLAGE, COLORADO 80111.

List of Chapters

Volume 1:

- 1** [Overview](#)
- 2** [Core Complex \(CCX\)](#)

Volume 2:

- 3** **Reliability, Availability, and Serviceability (RAS) Features**
- 4** **System Management Network (SMN)**

Volume 3:

- 5** [Remote System Management Unit \(RSMU\)](#)

Volume 4:

- 6** [System Management Unit \(SMU\)](#)

Volume 5:

- 7** [Security](#)
- 8** [Advanced Platform Management Link \(APML\)](#)
- 9** [SB Temperature Sensor Interface \(SB-TSI\)](#)
- 10** [Host System Management Port \(HSMP\)](#)
- 11** [Data Fabric \(DF\)](#)
- 12** [Unified Memory Controller \(UMC\)](#)

Volume 6:

- 13** [Northbridge IO \(NBIO\)](#)

Volume 7:

- 14** [Fusion Controller Hub \(FCH\)](#)

Volume 8:

- 15** [System Hub \(SYSHUB\)](#)

Volume 9:

- 16** [DXIO](#)

[List of Namespaces](#)

[List of Definitions](#)

Table of Contents

3 Reliability, Availability, and Serviceability (RAS) Features

- 3.1 Machine Check Architecture
 - 3.1.1 Overview
 - 3.1.1.1 Legacy Machine Check Architecture
 - 3.1.1.2 Machine Check Architecture Extensions
 - 3.1.1.3 Use of MCA Information
 - 3.1.1.3.1 Error Management
 - 3.1.1.3.2 Fault Management
 - 3.1.2 Machine Check Registers
 - 3.1.2.1 Global Registers
 - 3.1.2.2 Machine Check Banks
 - 3.1.2.2.1 Legacy MCA Registers
 - 3.1.2.2.2 Legacy MCA MSRs
 - 3.1.2.2.3 MCAX Registers
 - 3.1.2.2.4 MCAX MSRs
 - 3.1.2.3 Access Permissions
 - 3.1.3 Machine Check Errors
 - 3.1.3.1 Error Severities
 - 3.1.3.2 Exceptions and Interrupts
 - 3.1.3.3 Error Codes
 - 3.1.3.4 Extended Error Codes
 - 3.1.3.5 DOER and SEER State
 - 3.1.3.6 MCA Overflow Recovery
 - 3.1.3.7 MCA Recovery
 - 3.1.4 Machine Check Features
 - 3.1.4.1 Error Thresholding
 - 3.1.4.2 Error Simulation
 - 3.1.5 Software Guidelines
 - 3.1.5.1 Recognizing MCAX Support
 - 3.1.5.2 Communicating MCAX Support
 - 3.1.5.3 Machine Check Initialization
 - 3.1.5.4 Determining Bank Count
 - 3.1.5.5 Determining Bank Type
 - 3.1.5.6 Recognizing Error Type
 - 3.1.5.7 Machine Check Error Handling
- 3.2 Machine Check Architecture Implementation
 - 3.2.1 Implemented Machine Check Banks
 - 3.2.2 Implemented Machine Check Bank Registers
 - 3.2.3 Mapping of Banks to Blocks
 - 3.2.4 Decoding Error Type
 - 3.2.5 MCA Banks
 - 3.2.5.1 LS
 - 3.2.5.2 IF
 - 3.2.5.3 L2
 - 3.2.5.4 DE
 - 3.2.5.5 EX
 - 3.2.5.6 FP
 - 3.2.5.7 L3
 - 3.2.5.8 CS
 - 3.2.5.9 PIE

- 3.2.5.10 UMC
- 3.2.5.11 PB
- 3.2.5.12 PSP
- 3.2.5.13 SMU
- 3.2.5.14 MP5
- 3.2.5.15 NBIO
- 3.2.5.16 PCIE
- 3.2.5.17 KPX SERDES
- 3.2.5.18 KPX GMI
- 3.2.5.19 PCS GMI
- 3.2.5.20 KPX WAFL
- 3.2.5.21 PCS XGMI
- 3.2.5.22 NBIF
- 3.2.5.23 SHUB
- 3.2.5.24 USB
- 3.2.5.25 SATA
- 3.2.5.26 MPDMA
- 3.3 Data Poisoning
 - 3.3.1 Deferred Errors
 - 3.3.2 Clearing Poison
 - 3.3.3 Enabling and Disabling
- 3.4 Platform First Error Handling
 - 3.4.1 Overview
 - 3.4.2 Feature Indicator
 - 3.4.3 Error Cloaking
 - 3.4.4 Interrupt Redirection
 - 3.4.5 Event Percolation
 - 3.4.6 Enabling and Disabling
- 3.5 CPU Core RAS Features
 - 3.5.1 L2 and L3 Data Array DEC-TED ECC
 - 3.5.2 L2 Tag and State Array SEC-DED ECC
 - 3.5.3 L3 Tag Array SEC-DED ECC
 - 3.5.4 Shadow Tag Array SEC-DED ECC
 - 3.5.5 CPU Watchdog Timer (CPU WDT)
- 3.6 Data Fabric RAS Features
 - 3.6.1 Data Fabric Watchdog Timers (DF WDT)
 - 3.6.2 Bad Data Containment
 - 3.6.3 Data Fabric Links
 - 3.6.4 Probe Filter ECC
- 3.7 DRAM RAS Features
 - 3.7.1 Error Correcting Codes
 - 3.7.1.1 ECC Mapping
 - 3.7.1.2 DRAM Device to Symbol Mapping
 - 3.7.1.3 Hardware Managed ECC History Mechanism
 - 3.7.1.4 ECC Syndromes
 - 3.7.1.4.1 X16 ECC
 - 3.7.1.4.2 X4 ECC
 - 3.7.2 Bad Symbol ID
 - 3.7.3 DRAM Scrubbers
 - 3.7.3.1 Periodic Scrubber
 - 3.7.3.2 Redirect and Poison Scrubbers
 - 3.7.4 Corrected ECC Error Counters
 - 3.7.5 DRAM ECC Error Injection

- 3.7.5.1 Explicitly Addressed Error Injection
 - 3.7.5.2 Persistent Error Injection
 - 3.7.5.3 One-shot Error Injection
 - 3.7.6 Uncorrected ECC Error Retry
 - 3.7.6.1 Uncorrected ECC Error Injection
 - 3.7.7 Address/Command Parity Errors (CAPEs)
 - 3.7.7.1 Address/Command Parity Error Injection
 - 3.7.8 Read and Write Data CRC Errors
 - 3.7.8.1 Write Data CRC (WRCRC)
 - 3.7.8.2 Read Data CRC (RDCRC)
 - 3.7.8.3 CRC and UECC Error Injection
 - 3.7.9 DDR5 Error Check and Scrub (ECS)
 - 3.7.10 Post Package Repair
 - 3.8 PCI Express® RAS Features
 - 3.8.1 PCI Express® Advanced Error Reporting (AER)
 - 3.8.1.1 Recommended PCI Express® AER Severity
 - 3.8.2 PCIe® Error Actions
 - 3.8.3 PCIe® Error Status
 - 3.8.4 PCI Express® eDPC
 - 3.8.5 PCI Express® ECRC
 - 3.8.6 PCI Express® Error Injection
 - 3.9 NBIO RAS Features
 - 3.9.1 SRAM ECC and Parity
 - 3.9.1.1 SRAM ECC and Parity Enablement
 - 3.9.1.2 SRAM ECC and Parity Error Severity
 - 3.9.1.3 SRAM ECC and Parity Error Actions
 - 3.9.1.4 SRAM ECC and Parity Error Status
 - 3.9.1.5 SRAM ECC and Parity Correctable Error Thresholding
 - 3.9.1.6 SRAM ECC and Parity Error Injection
 - 3.9.2 System Fatal Error Event
 - 3.9.2.1 System Fatal Error Event Severity
 - 3.9.2.2 System Fatal Error Event Actions
 - 3.9.2.3 System Fatal Error Event Status
 - 3.9.3 Data Poisoning
 - 3.9.3.1 Egress Port Poisoned Data Severity
 - 3.9.3.2 Egress Port Poisoned Data Actions
 - 3.9.3.3 Egress Port Poisoned Data Status
 - 3.9.3.4 Internal Port Poisoned Data Severity
 - 3.9.3.5 Internal Port Poisoned Data Actions
 - 3.9.3.6 Internal Port Poisoned Data Status
 - 3.9.4 Global Error Status
 - 3.9.5 Multifunction NBIO RAS pin
 - 3.10 SATA, USB, and FCH RAS Features
- 4 System Management Network (SMN)**
- 4.1 Definitions
 - 4.2 SMN Network Overview

List of Figures

List of Tables

Table 41:	Machine Check Terms and Acronyms
Table 42:	Legacy MCA MSR Layout
Table 43:	MCAX MSR Layout
Table 44:	MCAX Implementation-Specific Register Layout
Table 45:	Error Overwrite Priorities
Table 46:	Error Scope Hierarchy
Table 47:	Error Code Types
Table 48:	Error code: transaction type (TT)
Table 49:	Error codes: cache level (LL)
Table 50:	Error codes: memory transaction type (RRRR)
Table 51:	Blocks Capable of Supporting MCA Banks
Table 52:	Mapping of Blocks to MCA_IPID[HwId] and MCA_IPID[McaType]
Table 53:	Legacy MCA Registers
Table 54:	MCAX Registers
Table 55:	Core MCA Bank to Block Mapping
Table 56:	Non-core MCA Bank to Block Mapping
Table 57:	Non-core MCA Bank to Block Mapping for SDP-wide products
Table 58:	MCA_STATUS_LS
Table 59:	MCA_ADDR_LS
Table 60:	MCA_SYND_LS
Table 61:	MCA_STATUS_IF
Table 62:	MCA_ADDR_IF
Table 63:	MCA_SYND_IF
Table 64:	MCA_STATUS_L2
Table 65:	MCA_ADDR_L2
Table 66:	MCA_SYND_L2
Table 67:	MCA_STATUS_DE
Table 68:	MCA_ADDR_DE
Table 69:	MCA_SYND_DE
Table 70:	MCA_STATUS_EX
Table 71:	MCA_ADDR_EX
Table 72:	MCA_SYND_EX
Table 73:	MCA_STATUS_FP
Table 74:	MCA_ADDR_FP
Table 75:	MCA_SYND_FP
Table 76:	MCA_STATUS_L3
Table 77:	MCA_ADDR_L3
Table 78:	MCA_SYND_L3
Table 79:	MCA_STATUS_CS
Table 80:	MCA_ADDR_CS
Table 81:	MCA_SYND_CS
Table 82:	MCA_STATUS_PIE
Table 83:	MCA_ADDR_PIE
Table 84:	MCA_SYND_PIE
Table 85:	MCA_STATUS_UMC
Table 86:	MCA_ADDR_UMC
Table 87:	MCA_SYND_UMC
Table 88:	MCA_STATUS_PB
Table 89:	MCA_ADDR_PB
Table 90:	MCA_SYND_PB

Table 91:	MCA_STATUS_PSP
Table 92:	MCA_ADDR_PSP
Table 93:	MCA_SYND_PSP
Table 94:	MCA_STATUS_SMU
Table 95:	MCA_ADDR_SMU
Table 96:	MCA_SYND_SMU
Table 97:	MCA_STATUS_MP5
Table 98:	MCA_ADDR_MP5
Table 99:	MCA_SYND_MP5
Table 100:	MCA_STATUS_NBIO
Table 101:	MCA_ADDR_NBIO
Table 102:	MCA_SYND_NBIO
Table 103:	MCA_STATUS_PCIE
Table 104:	MCA_ADDR_PCIE
Table 105:	MCA_SYND_PCIE
Table 106:	MCA_STATUS_KPX_SERDES
Table 107:	MCA_ADDR_KPX_SERDES
Table 108:	MCA_SYND_KPX_SERDES
Table 109:	MCA_STATUS_KPX_GMI
Table 110:	MCA_ADDR_KPX_GMI
Table 111:	MCA_SYND_KPX_GMI
Table 112:	MCA_STATUS_PCS_GMI
Table 113:	MCA_ADDR_PCS_GMI
Table 114:	MCA_SYND_PCS_GMI
Table 115:	MCA_STATUS_KPX_WAFL
Table 116:	MCA_ADDR_KPX_WAFL
Table 117:	MCA_SYND_KPX_WAFL
Table 118:	MCA_STATUS_PCS_XGMI
Table 119:	MCA_ADDR_PCS_XGMI
Table 120:	MCA_SYND_PCS_XGMI
Table 121:	MCA_STATUS_NBIF
Table 122:	MCA_ADDR_NBIF
Table 123:	MCA_SYND_NBIF
Table 124:	MCA_STATUS_SHUB
Table 125:	MCA_ADDR_SHUB
Table 126:	MCA_SYND_SHUB
Table 127:	MCA_STATUS_USB
Table 128:	MCA_ADDR_USB
Table 129:	MCA_SYND_USB
Table 130:	MCA_STATUS_SATA
Table 131:	MCA_ADDR_SATA
Table 132:	MCA_SYND_SATA
Table 133:	MCA_STATUS_MPDMA
Table 134:	MCA_ADDR_MPDMA
Table 135:	MCA_SYND_MPDMA
Table 136:	L2 and L3 DEC-TED ECC Syndromes
Table 137:	x16 ECC Single-Bit Correctable Syndrome Bits[31:15]
Table 138:	x4 ECC Correctable Syndromes
Table 139:	AER Severity for Root Complex and Switch Downstream Ports
Table 140:	AER Severity for Endpoint Devices and Switch Upstream Ports
Table 141:	NBIO Correctable Parity Structures
Table 142:	NBIO Uncorrectable Parity Structures
Table 143:	NBIO ECC Structures

Table 144: NBIO DDP+UCP Structures

Table 145: Error Status TypeSel Selection

Table 146: IOHC Port Numbering and System Fatal Error Event Status

Table 147: Definitions

3 Reliability, Availability, and Serviceability (RAS) Features

A full implementation of [RAS](#) involves capabilities and support from the processor design, board hardware design, BIOS, firmware, and software.

3.1 Machine Check Architecture

Table 41: Machine Check Terms and Acronyms

Term	Description
MCA	Machine Check Architecture.
MCAX	Machine Check Architecture eXtensions.
WRIG	Writes Ignored.

3.1.1 Overview

The processor contains logic and registers to detect, log, and correct errors in the data or control paths. The Machine Check Architecture (MCA) defines facilities by which processor and system hardware errors are logged and reported to system software. This allows system software to perform a strategic role in recovery from and diagnosis of hardware errors.

3.1.1.1 Legacy Machine Check Architecture

The legacy x86 Machine Check Architecture (MCA) refers to the standard x86 facilities for error logging and reporting. Refer to the AMD64 Architecture Programmer's Manual for an architectural overview of the Machine Check Architecture.

Support for the MCA is indicated by Core::X86::Cpuid::FeatureIdEdx[MCA] or Core::X86::Cpuid::FeatureExtIdEdx[MCA].

3.1.1.2 Machine Check Architecture Extensions

Machine Check Architecture Extensions (MCAX) is AMD's x86-64 extension to the Machine Check Architecture.

Goals of MCAX include:

- Accommodate a variety of implementations, where each implementation may have a different assignment of MCA bank to block.
 - For example, one implementation may have 1 memory channel with an MCA bank, and another otherwise identical implementation may have 2 memory channels, each with their own MCA bank. Therefore, MCA bank allocation will appear different between these two implementations. MCAX is designed to require no assumptions about which MCA banks access which blocks.
 - Provide granular information for error logging, to improve error handling and diagnosability.
 - Preserve compatibility with system software which is not MCAX-aware.

Features of the MCA Extensions include:

- Increased MCA Bank Count: Features to support an expansion of the number of MCA banks supported by AMD processors.
- MCA Extension Registers: Expanded information logged in MCA banks to allow for improved error handling, better diagnosability, and future scalability.
- MCA DOER/SEER Roles: Separation of MCA information to take advantage of emerging software roles, namely

Error Management (Dynamic Operational Error Handling, or DOER) for managing running programs, and Fault Management (Symptom Elaboration of Errors, or SEER) for hardware diagnosability and reconfiguration. This clearer separation is accompanied by the assurances of architectural state (vs. implementation dependent state), so that operating systems can rely on the state and exploit new functionality.

Support for Machine Check Architecture Extensions (MCAX) is indicated by Core::X86::Cpuid::RasCap[ScalableMca].

3.1.1.3 Use of MCA Information

The MCA registers contain information that can be used for multiple purposes. Some of this information is architecturally specified, and remains consistent from generation to generation, enabling portable, stable code. Some of this information is implementation specific; it is vital for diagnosis and other software functions, but may change with new implementations. It is important to understand how this information is categorized, and how it should be used. This section describes a framework for that.

There are two fundamental roles to be carried out after an error occurs; Error Management and Fault Management. All information required for Error Management is architectural and stable; some information required for Fault Management is also architectural.

3.1.1.3.1 Error Management

Error Management describes actions necessary by operational software (e.g., the operating system or the hypervisor) to manage running programs that are affected by the error. The list of possible actions for operational error management is generally fairly short: take no action; terminate a single affected process, program, or virtual machine; terminate system operation. The Error Management role is defined as the DOER role (Dynamic Operational Error Handling). The name is intended to indicate an active role in managing running programs. Information used by the DOER is fairly limited and straightforward. It includes only those status fields needed to make decisions about the scope and severity of the error, and to determine what immediate action is to be taken.

3.1.1.3.2 Fault Management

Fault Management describes optional actions for purposes of diagnosis, repair, and reconfiguration of the underlying hardware. The Fault Management role is described as SEER (Symptom Elaboration of Errors) because it peers further into hardware behavior and may try to influence future behavior via Predictive Fault Analysis, reconfiguration, service actions, etc. Because the SEER depends on understanding specifics of hardware configuration, it necessarily requires implementation specific knowledge and may not be portable across implementations.

Fields that are not explicitly specified as DOER are SEER. By separating error handling software into DOER and SEER roles, programmers can create both simpler and more functional code. The terms DOER and SEER appear in other sections of this document as an aid to reasoning about error handling and understanding actions to be taken.

3.1.2 Machine Check Registers

Host software references MCA registers via MSRs. MSRs are accessed through x86 WRMSR and RDMSR instructions. MSR addresses are private to a logical core; a given MSR referenced by two different cores results in references to two different MCA registers.

3.1.2.1 Global Registers

Core::X86::Cpuid::FeatureIdEdx[MCA] or Core::X86::Cpuid::FeatureExtIdEdx[MCA] indicates the presence of the following machine check registers:

- Core::X86::Msr::MCG_CAP
 - Reports how many machine check register banks are supported. This value reflects the number of MCA banks visible to that logical core. Some banks may be RAZ/WRIG either due to the bank being reserved or unused on this processor or because the block's MCA bank is controlled by another logical core.
- Core::X86::Msr::MCG_STAT
 - Provides basic information about processor state after the occurrence of a machine check error.
- Core::X86::Msr::MCG_CTL
 - Used by software to enable or disable the logging and reporting of machine check errors in the error reporting banks. Some bits may be RAZ/WRIG either due to the bank being reserved or unused on this processor or because the block's MCA bank is controlled by another logical core.
- Core::X86::Msr::McaIntrCfg
 - Used by software to configure certain machine check interrupts.

3.1.2.2 Machine Check Banks

A processor contains multiple blocks, and some of them have banks of machine check architecture registers (MCA banks). An MCA bank logs and reports errors to software.

The legacy MCA supports up to 32 MCA banks per logical core. MCAX supports up to 64 MCA banks per logical core.

The processor ensures that non-zero error status in an MCA bank is visible to exactly one logical core in a system, and that error notifications are directed to that logical core. Hardware also makes MCA bank configuration and control registers available to exactly one logical core. Banks associated with a CPU core are controlled by that logical core. Banks associated with other blocks are controlled by an implementation-specific logical core.

3.1.2.2.1 Legacy MCA Registers

Each legacy MCA bank allocates address space for 4 legacy MCA registers.

The legacy MCA registers include:

- MCA_CTL
 - Enables error reporting via machine check exception.
- MCA_STATUS
 - Logs information associated with errors.
- MCA_ADDR
 - Logs address information associated with errors. The use of AMD Secure Memory Encryption may change the information logged in the address register. See 2.1.3 [Memory Encryption] for more details.
- MCA_MISC0
 - Logs miscellaneous information associated with errors.

3.1.2.2.2 Legacy MCA MSRs

The legacy MCA MSRs are MSR0000_04[7F:00]. The legacy MCA MSR space contains 32 banks of 4 registers per bank. The layout of the legacy MCA MSR space is given in Table 42 [Legacy MCA MSR Layout].

Table 42: Legacy MCA MSR Layout

MCA bank (decimal)	MCA_CTL (MSR0000_0xxx)	MCA_STATUS	MCA_ADDR	MCA_MISC0
0	400	401	402	403
1	404	405	406	407
2	408	409	40A	40B

3	40C	40D	40E	40F
4	410	411	412	413
5	414	415	416	417
6	418	419	41A	41B
...				
31	47C	47D	47E	47F

Features and registers associated with the MCA Extensions are not available in this legacy MSR address range. AMD recommends that operating systems use the MCAX MSR address range, rather than rely on the legacy MCA MSR address range.

All unimplemented or unused registers in the legacy MCA MSR address range are RAZ/WRIG. MC4 registers (MSR0000_0410:0000_0413) are RAZ/WRIG.

MSR0000_0000 is aliased to the MCAX MSR address for MC0_ADDR, and MSR0000_0001 is aliased to the MCAX MSR address of MC0_STATUS.

3.1.2.2.3 MCAX Registers

Each MCAX bank allocates address space for 16 MCA registers. All unimplemented registers in the MCA [MSR](#) space are RAZ/WRIG. MCAX bank registers include the legacy MCA registers as well as registers associated with the MCA Extensions.

The MCA Extension registers include:

- MCA_CONFIG
 - Provide configuration capabilities for this MCA bank.
- MCA_IPID
 - Provides information on the block associated with this MCA bank.
- MCA_SYND
 - Logs physical location information associated with a logged error.
- MCA_DESTATUS
 - Logs status information associated with a deferred error.
- MCA_DEADDR
 - Logs address information associated with a deferred error.
- MCA_MISC[1:4]
 - Provides additional threshold counters within an MCA bank.
- MCA_TRANSYND
 - Logs physical location information associated with a transparent error.
- MCA_TRANSADDR
 - Logs address associated with a transparent error.
- MCA_SYND1 & MCA_SYND2
 - Log information associated with a logged error, such as FruText.

3.1.2.2.4 MCAX MSRs

MCAX MSRs are present at MSRC000_2[3FF:000]. This [MSR](#) address range contains space for 64 banks of 16 registers each. MSRC000_2[FFF:400] are Reserved for future use. The MCAX MSR address range allows access to both legacy MCA registers and MCAX registers in each MCA bank.

The x86 MCAX MSR address format is SSSS_SBBR (hex). S=MCA register space (i.e., MSRC000_2XXX). B=MCA bank. R=Register offset within MCA bank. The layout of the MCAX MSR space is given in Table 43 [MCAX MSR

Layout].

Access to unused MCAX MSRs is RAZ/WRIG. MCA Bank 4 is always Read-as-zero (RAZ/WRIG).

Table 43: MCAX MSR Layout

MCA bank	MCAX MSR (MSRC000_2xxx)												
	Legacy MCA Bank registers				MCAX Bank registers								
	CTL	STATUS	ADDR	MISC0	CONFIG	IPID	SYND	Reserved	DESTAT	DEADDR	MISC[4:1]	SYND1	SYND2
0	000	001	002	003	004	005	006	007	008	009	00D:00A	00E	00F
1	010	011	012	013	014	015	016	017	018	019	01D:01A	01E	01F
2	020	021	022	023	024	025	026	027	028	029	02D:02A	02E	02F
...													
63	3F0	3F1	3F2	3F3	3F4	3F5	3F6	3F7	3F8	3F9	3FD:3FA	3FE	3FF

All processors maintain the same mapping of MSR to MCA bank number (MSRC000_2000 for the beginning of MCA Bank 0, MSRC000_2010 for the beginning of MCA Bank 1, etc.), regardless of what block the bank represents (see 3.1.5.5 [Determining Bank Type]).

MCA_CTL_MASK MSRs are present at MSRC001_04[3F:00]. MSRC001_04[FF:40] are Reserved for future use. MCA_TRANSYND MSRs are present at MSRC001_05[3F:00]. MCA_TRANSADDR MSRs are present at MSRC001_06[3F:00]. MSRC001_05[FF:40] and MSRC001_06[FF:40] are Reserved for future use. The layout of these registers is given in Table 44 [MCAX Implementation-Specific Register Layout].

Table 44: MCAX Implementation-Specific Register Layout

MCA bank	MCA_CTL_MASK (MSRC001_04xx)	MCA_TRANSYND (MSRC001_05xx)	MCA_TRANSADDR (MSRC001_06xx)
0	00	00	00
1	01	01	01
2	02	02	02
...			
63	3F	3F	3F

3.1.2.3 Access Permissions

When `McStatusWrEn == 0`, a Write to an implemented `MCA_STATUS` register causes a General Protection Fault (`#GP`) unless the value being written is zero. When `McStatusWrEn == 1`, a Write to an implemented `MCA_STATUS` register does not cause a `#GP` regardless of data value.

Access to legacy `MCA_CTL_MASK` (`MSRC001_00xx`) causes a General Protection Fault (`#GP`).

Access to legacy `MC4_MISC1-8` (`MSRC000_0408:C000_040F`) is `RAZ/WRIG`.

3.1.3 Machine Check Errors

3.1.3.1 Error Severities

The classes of machine check errors are, in priority order from highest to lowest:

- Uncorrected
- Deferred
- Corrected
- Transparent

Uncorrected errors cannot be corrected by hardware. Uncorrected errors update the status and address registers if not masked from logging in `MCA_CTL_MASK`. Information in the status and address registers from a previously logged lower priority error is overwritten. Previously logged errors of the same priority are not overwritten. Uncorrected errors that are enabled for reporting in `MCA_CTL` result in reporting to software via machine check exceptions. If an uncorrected error is masked from logging, the error is ignored by hardware (exceptions are noted in the register definitions). If an uncorrected error is disabled from reporting, containment of the error and logging/reporting of subsequent errors may be affected. Therefore, enable reporting of unmasked uncorrected errors for normal operation. Disable reporting of uncorrected errors only for debug purposes.

Deferred errors are errors that cannot be corrected by hardware, but do not cause an immediate interruption in program flow, loss of data integrity, or corruption of processor state. These errors indicate that data has been corrupted but not consumed; no exception is generated because the data has not been referenced by a core or an IO link. Hardware writes information to the status and address registers in the corresponding bank that identifies the source of the error if deferred errors are enabled for logging. If there is information in the status and address registers from a previously logged lower priority error, it is overwritten. Previously logged errors of the same or higher priority are not overwritten. Deferred errors are not reported via machine check exceptions; they can optionally be reported via [LVT](#) or [SMI](#).

Corrected errors are those which have been corrected by hardware and cause no loss of data or corruption of processor state. Hardware writes the status and address registers in the corresponding register bank with information that identifies the source of the error if they are enabled for logging. Corrected errors are not reported via machine check exceptions. Some corrected errors may optionally be reported to software via LVT or SMI if the number of errors exceeds a configurable threshold.

Transparent errors are those which have been corrected by hardware and cause no loss of data or corruption of processor state, and leave the hardware able to correct a second error in the same structure or device (e.g., a single-bit error in a cache with double-bit error correction capability). When enabled for logging in the MCA, transparent errors are treated identically to corrected errors.

An error to be logged when the status register contains valid data can result in an overflow condition. During error overflow conditions, the new error may not be logged or an error which has already been logged in the

status register may be overwritten.

Table 45 [Error Overwrite Priorities] indicates which errors are overwritten in the error status registers.

Table 45: Error Overwrite Priorities

		Older Error		
		Uncorrected	Deferred	Corrected
Newer Error	Uncorrected	-	Overwrite	Overwrite
	Deferred	-	-	Overwrite
	Corrected	-	-	-

Table 46 [Error Scope Hierarchy] provides a hierarchy of error scopes that determine the potential ability to recover the system based on fields in MCA_STATUS when MCA_STATUS[Val] == 1.

Table 46: Error Scope Hierarchy

PCC	UC	TCC	Deferred	Comments
1	X	X	X	Uncorrected system fatal error. Action required. A hardware-uncorrected error has corrupted system state. The error is fatal to the system and the system processing must be terminated.
0	1	1	X	Uncorrected thread fatal error. Action required. A hardware-uncorrected error has corrupted state for the process thread executing on the interrupted logical core. State for other process threads is unaffected.
0	1	0	X	Uncorrected recoverable error. Action required. A hardware-uncorrected error has not corrupted state of the process thread. Recovery of the process thread is possible if the uncorrected error is corrected by software.
0	0	0	1	Deferred error. Action optional. A hardware-uncorrected error has been discovered but not yet consumed. Error handling software may attempt to correct this error, or prevent access by processes which map the data, or make the physical resource containing the data inaccessible.
0	0	0	0	Corrected error. Action optional. A hardware-corrected error has been corrected. No action is required by error handling software.

3.1.3.2 Exceptions and Interrupts

Some or all errors logged in the MCA may require an interrupt or exception to be signaled.

The processor supports the following x86 interrupt/exception types to be communicated to the x86 core in response to an error:

- Machine Check Exception (MCE)
- System Management Interrupt ([SMI](#))
- APIC based interrupt ([LVT](#))

MCEs can be architecturally precise, context-synchronous, or asynchronous. An MCE that sets Core::X86::Msrr::MCG_STAT[RIPV] = 1 and Core::X86::Msrr::MCG_STAT[EIPV] = 1 is precise and the program can be restarted reliably. Other interrupts are architecturally asynchronous.

The ability of hardware to generate a machine check exception upon an error is indicated by

Core::X86::Cpuid::FeatureIdEdx[MCE] or Core::X86::Cpuid::FeatureExtIdEdx[MCE].

3.1.3.3 Error Codes

The MCA_STATUS[ErrorCode] field contains information used to identify the logged error. This section identifies how to decode the ErrorCode field.

Table 47: Error Code Types

Error Code	Error Code Type	Description
0000 0000 0001 TTLL	TLB	TT = Transaction Type LL = Cache Level
0000 0001 RRRR TTLL	Memory	RRRR = Memory Transaction Type TT = Transaction Type LL = Cache Level
0000 1XXT RRRR XXLL	Bus	XX = Reserved T = Timeout RRRR = Memory Transaction Type LL = Cache Level
0000 01UU 0000 0000	Internal Unclassified	UU = Internal Error Type

Table 48: Error code: transaction type (TT)

TT	Transaction Type
00	Instruction
01	Data
10	Generic
11	Reserved

Table 49: Error codes: cache level (LL)

LL	Cache Level
00	L0: Core
01	L1: Level 1
10	L2: Level 2
11	LG: Generic

Table 50: Error codes: memory transaction type (RRRR)

RRRR	Memory Transaction Type
0000	Generic
0001	Generic Read
0010	Generic Write
0011	Data Read
0100	Data Write
0101	Instruction Fetch
0110	Prefetch
0111	Evict
1000	Snoop (Probe)

Errors can also be identified by the MCA_STATUS[ErrorCodeExt] field. MCA_STATUS[ErrorCodeExt] indicates which bit position in the corresponding MCA_CTL register enables error reporting for the logged error. For instance,

MCA_STATUS[ErrorCodeExt] == 0x9 means that the logged error is enabled by MCA_CTL[9], and the description of MCA_CTL[9] contains information on decoding the error log. Specific ErrorCodeExt values are implementation dependent, and should not be used by architectural or portable code.

3.1.3.4 Extended Error Codes

The MCA_STATUS[ErrorCodeExt] field contains additional information used to identify the logged error. Error positions in MCA_CTL and MCA_CTL_MASK and Extended Error Codes are fixed within a given bank type. That is, for an MCA bank with a given MCA_IPID[HwId, McaType] value, the processor ensures that the same error is reported in a given bit position of MCA_CTL regardless of the product in which that bank appears. Similarly, for an MCA bank with a given MCA_IPID[HwId, McaType] value, hardware ensures that the mapping of errors to Extended Error Codes is consistent across products.

3.1.3.5 DOER and SEER State

The DOER fields are:

- MCG_STAT
 - Count
 - MCIP
 - RIPV
 - EIPV
- MCA_STATUS
 - Val
 - PCC
 - TCC
 - UC
 - MiscV
 - AddrV

The MCA_STATUS[Deferred] bit is used for SEER functionality but is architectural.

3.1.3.6 MCA Overflow Recovery

MCA Overflow Recovery is a feature allowing recovery of the system when the overflow bit is set. MCA Overflow Recovery is supported when Core::X86::Cpuid::RasCap[McaOverflowRecov] == 1.

When MCA Overflow Recovery is supported, software may rely on MCA_STATUS[PCC] == 1 to indicate all system-fatal conditions. When MCA Overflow Recovery is not supported, an uncorrected error logged with MCA_STATUS[Overflow] = 1 may indicate the system-fatal condition that an error requiring software intervention was not logged. Therefore, software must terminate system processing whenever an uncorrected error is logged with MCA_STATUS[Overflow] = 1.

3.1.3.7 MCA Recovery

MCA Recovery is a feature allowing recovery of the system when the hardware cannot correct an error. MCA Recovery is supported when Core::X86::Cpuid::RasCap[SUCCOR] == 1.

When MCA Recovery is supported and an uncorrected error has been detected that the hardware can contain to the task or process to which the machine check has been delivered, it logs a context-synchronous uncorrectable error (MCA_STATUS[UC] = 1, MCA_STATUS[PCC] = 0). The rest of the system is unaffected and may continue running if

supervisory software can terminate only the affected process or VM.

3.1.4 Machine Check Features

3.1.4.1 Error Thresholding

For some types of errors, the hardware maintains counts of the number of errors. When the counter reaches a programmable threshold, an event may optionally be triggered to signal system software. This is known as error thresholding. The primary purpose of error thresholding is to help software recognize an excessive rate of errors, which may indicate marginal or failing hardware. This information can be used to make decisions about deconfiguring hardware or scheduling service actions. The error count is incremented for corrected, deferred, and uncorrected errors.

The MCA_MISCL registers contain the architectural interface for error thresholding. The registers contain a 12-bit error counter that can be initialized to any value except FFFh, with the option to interrupt when the counter reaches FFFh.

MCA_MISCL[ThresholdIntType] determines the type of interrupt to be generated for threshold overflow errors in that counter. This can be set to None, [LVT](#), or [SMI](#). If this is set to LVT, Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset] specifies the LVT offset that is used. Only one LVT offset is used per socket and the interrupt is routed to the APIC of the logical core from which the MCA bank is visible.

3.1.4.2 Error Simulation

Error simulation involves creating the appearance to software that an error occurred, and can be used to debug machine check interrupt handlers. See Core::X86::Msr::HWCR[McStatusWrEn] for making MCA registers writable for non-zero values. When McStatusWrEn is set, privileged software can write non-zero values to the specified registers without generating exceptions, and then simulate a machine check using the INT18 instruction (INTn instruction with an operand of 18). Setting a reserved bit in these registers does not generate an exception when this mode is enabled. However, setting a reserved bit may result in undefined behavior.

3.1.5 Software Guidelines

3.1.5.1 Recognizing MCAX Support

Software which reads the MCA registers must recognize whether an implementation uses the legacy format or the MCAX format. This is accomplished by starting with [CPLD](#) Fn8000_0007_EBX[ScalableMca]. If ScalableMca == 1, then the implementation supports the MCAX indicator (MCA_CONFIG[Mcac]). An MCA bank is an MCAX bank if MCA_CONFIG[Mcac] == 1 in that bank.

3.1.5.2 Communicating MCAX Support

Software which supports MCAX must set MCA_CONFIG[McacEn] = 1 in each MCA bank.

Software that supports MCAX should use the MCAX MSRs to access both legacy and MCAX registers.

3.1.5.3 Machine Check Initialization

The following initialization sequence must be followed:

- Platform firmware must initialize the MCA_CTL_MASK registers prior to the initialization of the MCA_CTL registers and Core::X86::Msr::MCG_CTL. Platform firmware and the operating system must not clear MCA_CTL_MASK bits that are set to 1. MCA_CTL_MASK registers must be set the same across all cores.
- The operating system and platform firmware must initialize the MCA_CONFIG registers prior to initialization of the MCA_CTL registers.
- The MCA_CTL registers must be initialized prior to enabling the error reporting banks in MCG_CTL.
- The Core::X86::Msr::MCG_CTL register must be programmed identically for all cores in a processor, although the Read-write bits may differ per core.
- CR4.MCE must be set to enable machine check exceptions.

Platform firmware should configure the MCA_CONFIG register in each bank as follows:

- MCA_CONFIG[McaEn] = 1 if MCG_CAP[Count] <= 32, 0 otherwise.
- MCA_CONFIG[LogDeferredInMcaStat] = 1
- MCA_CONFIG[TransparentErrorLoggingEnable] = 0

The operating system should configure the MCA_CONFIG registers as follows:

- MCA_CONFIG[McaEn] = 1 if the operating system has been updated to use the MCA Extension MSR addresses. Otherwise, the operating system should preserve the platform firmware-programmed value of this field.
- MCA_CONFIG[LogDeferredInMcaStat] and MCA_CONFIG[DeferredIntType] to appropriate values based on OS support for deferred errors.

MCA_STATUS MSRs are cleared by hardware after a cold reset. If initializing after a warm reset, then platform firmware should check for valid MCA errors and if present save the status for later diagnostic use.

Platform firmware may initialize the MCA without setting CR4.MCE; this results in a shutdown on any machine check which would have caused a machine check exception (followed by a reboot if configured). Alternatively, platform firmware that wishes to ensure continued operation in the event that a machine check occurs during boot may write MCG_CTL with all ones and write zeros into each MCA_CTL register. With these settings, a machine check error results in MCA_STATUS being written without generating a machine check exception or a shutdown. Platform firmware may then poll MCA_STATUS registers during critical sections of boot to ensure system integrity. Note that the system may be operating with corrupt data before polling MCA_STATUS registers. Before passing control to the operating system, platform firmware should restore the values of those registers to what the operating system is expecting.

After MCA initialization, system software should check the Val bit on each MCA_STATUS register. It is possible that valid error status information has already been logged in the MCA_STATUS registers at the time software is attempting to initialize them. The status can reflect errors logged prior to a warm reset or errors recorded during the system power-up and boot process. Before clearing the MCA_STATUS registers, software should examine their contents and log any errors found.

3.1.5.4 Determining Bank Count

System software should read Core::X86::Msr::MCG_CAP[Count] to determine the number of machine check banks visible to a logical core. The banks are numbered from 0 to one less than the value found in Core::X86::Msr::MCG_CAP[Count]. For example, if the Count field indicates five banks are supported, they are numbered MC0 through MC4.

3.1.5.5 Determining Bank Type

To determine which type of block is mapped to an MCA bank, software can query the MCA_IPID register within that bank. This register exists when MCA_CONFIG[McaX] == 1 in a given bank.

MCA_IPID[HardwareID] provides the block type for the block that contains this MCA bank. For blocks that contain

multiple MCA bank types (e.g., CPU cores), MCA_IPID[McaType] provides an identifier for the type of MCA bank. MCA_IPID[McaType] values are specific to a given MCA_IPID[HardwareID]. Therefore, an MCA bank type can be identified by the value of {MCA_IPID[Hwid], MCA_IPID[McaType]}. For instance, the CPU core's LS bank is identified by MCA::LS::MCA_IPID_LS[HardwareID] == 176 and MCA::LS::MCA_IPID_LS[McaType] == 0. An MCA_IPID[HardwareID] value of 0 indicates an unpopulated MCA bank that is ensured to be RAZ/WRIG.

MCA_IPID[InstanceId] provides a unique instance number to allow software to differentiate blocks with multiple identical instances within a processor. MCA_IPID[InstanceId] values are processor-specific and are not ensured to be stable across different processor generations.

3.1.5.6 Recognizing Error Type

Software can use the combination of MCA_IPID[Hwid, McaType] and MCA_STATUS[ErrorCodeExt] to recognize a specific error type.

3.1.5.7 Machine Check Error Handling

A machine check handler is invoked to handle an exception for a particular thread. The information needed by the machine check handler is not shared with other threads, so no cross-thread coordination or special handling is required. Specifically, all MCA banks are only visible from a single thread, so software on a single thread can access each bank through [MSR](#) space without contention from other threads.

At a minimum, the machine check handler must be capable of logging error information for later examination. The handler should log as much information as is needed to diagnose the error. More thorough exception handler implementations can analyze errors to determine if each error is recoverable by software. If a recoverable error is identified, the exception handler can attempt to correct the error and restart the interrupted program. An error may not be recoverable for the process or virtual machine it directly affects, but may be containable, so that other processes or virtual machines in the system are unaffected and system operation is recovered.

Machine check exception handlers that attempt to recover must be thorough in their analysis and the corrective actions they take. The following guidelines should be used when writing such a handler:

- Data collection:
 - Read Core::X86::Msr::MCG_CAP[Count] to determine the number of status registers visible to the logical core.
 - All status registers in all error reporting banks must be examined to identify the cause of the machine check exception.
 - Check the valid bit in each status register (MCA_STATUS[Val]). The remainder of the status register should be examined only when its valid bit is set.
 - When identifying the error condition and determining how to handle the error, portable exception handlers should examine only DOER fields in machine check registers.
 - Error handlers should collect all available MCA information, but should only interrogate details to the level which affects their actions. Lower level details may be useful for diagnosis and root cause analysis, but not for error handling.
 - Error handlers should save the values in MCA_ADDR, MCA_MISC0, and MCA_SYND even if MCA_STATUS[AddrV], MCA_STATUS[MiscV], and MCA_STATUS[SyndV] are zero. Error handlers should save the values in MCA_MISC[4:1] if the registers exist.
- DOER Error Management:
 - Check MCA_STATUS[PCC].
 - If PCC is set, error recovery is not possible. The handler should log the error information and terminate the system. If PCC is clear, the handler may continue with the following recovery steps.
 - Check MCA_STATUS[UC].
 - If UC is set, the processor did not correct the error. Continue with the following recovery steps.

- If MCA Overflow Recovery is not supported, and MCA_STATUS[Overflow] == 1, error recovery is not possible; follow the steps for PCC = 1. See 3.1.3.6 [MCA Overflow Recovery].
- If MCA Recovery is not supported, error recovery is not possible; follow the steps for PCC = 1. See 3.1.3.7 [MCA Recovery].
- If MCA Recovery is supported:
 - Check MCA_STATUS[TCC].
 - If TCC is set, the context of the process thread executing on the interrupted logical core may be corrupt and the thread cannot be recovered. The rest of the system is unaffected; it is possible to terminate only the affected process thread.
 - If TCC is clear, the context of the process thread executing on the interrupted logical core is not corrupt. Recovery of the process thread may be possible, but only if the uncorrected error condition is first corrected by software. Otherwise, the interrupted process thread must be terminated.
 - Legacy exception handlers can check Core::X86::Msr::MCG_STAT[RIPV] and Core::X86::Msr::MCG_STAT[EIPV] in place of MCA_STATUS[TCC]. If RIPV == EIPV == 1, the interrupted program can be restarted reliably. Otherwise, the program cannot be restarted reliably.
 - If UC is clear, the processor either corrected or deferred the error and no software action is needed. The handler can log the error information and continue process execution.
- Exit:
 - When an exception handler is able to successfully log an error condition, clear the MCA_STATUS registers prior to exiting the machine check handler.
 - Prior to exiting the machine check handler, clear Core::X86::Msr::MCG_STAT[MCIP]. MCIP indicates that a machine check exception is in progress. If this bit is set when another machine check exception occurs, the processor enters the shutdown state.

3.2 Machine Check Architecture Implementation

3.2.1 Implemented Machine Check Banks

Table 51: Blocks Capable of Supporting MCA Banks

Acronym	Block Function
LS	Load-Store Unit
IF	Instruction Fetch Unit
L2	L2 Cache Unit
DE	Decode Unit
EX	Execution Unit
FP	Floating-Point Unit
L3	L3 Cache Unit
PIE	Power Management, Interrupts, Etc.
CS	Coherent Station
UMC	Unified Memory Controller
NBIO	Northbridge IO Unit
PB	Parameter Block
PSP	Platform Security Processor (now known as ASP)

SMU	System Management Unit (including MP1 and MPIO processors)
PCIE	PCIe® Root Port
MP5	System and Power Management Microprocessor (for the CCD)
NBIF	On-chip PCIe Bus Interface
SHUB	System Hub
USB	USB Controller
SATA	SATA Controller
PCS_GMI	GMI Controller
PCS_XGMI	XGMI Controller
KPX_SERDES	High Speed Interface Unit
MPIO	IO Microprocessor
KPX_GMI	High Speed Interface Unit (GMI)
MPDMA	DMA Engine Controller
KPX_WAFL	High Speed Interface Unit (WAFL)

Table 52: Mapping of Blocks to MCA_IPID[HwId] and MCA_IPID[McaType]

Block	Hardware ID	MCA Type
LS	0xb0	0x10
IF	0xb0	0x1
L2	0xb0	0x2
L3	0xb0	0x7
MP5	0x1	0x2
PB	0x5	0x0
PCS_GMI	0x241	0x0
KPX_GMI	0x269	0x0
UMC	0x96	0x0
PCIE	0x46	0x1
NBIO	0x18	0x0
SHUB	0x80	0x0
SATA	0xa8	0x0
NBIF	0x6c	0x0
PIE	0x2e	0x1
PSP	0xff	0x1
KPX_WAFL	0x267	0x0
USB	0xaa	0x0
SMU	0x1	0x1
MPDMA	0x1	0x3
PCS_XGMI	0x50	0x0
KPX_SERDES	0x259	0x0
CS	0x2e	0x2
EX	0xb0	0x5
FP	0xb0	0x6
DE	0xb0	0x3

3.2.2 Implemented Machine Check Bank Registers

Table 53 [Legacy MCA Registers] provides links to the description of each block's Legacy MCA registers. Table 54

[MCAX Registers] provides links to the description of each block's MCA Extension Registers.

Table 53: Legacy MCA Registers

Block	MCA Register				
	CTL	STATUS	ADDR	MISC	CTL_MASK
LS	MCA::LS::MCA_CTL_LS	MCA::LS::MCA_STATUS_LS	MCA::LS::MCA_ADDR_LS	MCA::LS::MCA_MISC0_LS	MCA::LS::MCA_CTL_MASK_LS
IF	MCA::IF::MCA_CTL_IF	MCA::IF::MCA_STATUS_IF	MCA::IF::MCA_ADDR_IF	MCA::IF::MCA_MISC0_IF	MCA::IF::MCA_CTL_MASK_IF
L2	MCA::L2::MCA_CTL_L2	MCA::L2::MCA_STATUS_L2	MCA::L2::MCA_ADDR_L2	MCA::L2::MCA_MISC0_L2	MCA::L2::MCA_CTL_MASK_L2
DE	MCA::DE::MCA_CTL_DE	MCA::DE::MCA_STATUS_DE	MCA::DE::MCA_ADDR_DE	MCA::DE::MCA_MISC0_DE	MCA::DE::MCA_CTL_MASK_DE
EX	MCA::EX::MCA_CTL_EX	MCA::EX::MCA_STATUS_EX	MCA::EX::MCA_ADDR_EX	MCA::EX::MCA_MISC0_EX	MCA::EX::MCA_CTL_MASK_EX
FP	MCA::FP::MCA_CTL_FP	MCA::FP::MCA_STATUS_FP	MCA::FP::MCA_ADDR_FP	MCA::FP::MCA_MISC0_FP	MCA::FP::MCA_CTL_MASK_FP
L3	MCA::L3::MCA_CTL_L3	MCA::L3::MCA_STATUS_L3	MCA::L3::MCA_ADDR_L3	MCA::L3::MCA_MISC0_L3	MCA::L3::MCA_CTL_MASK_L3
PIE	MCA::PIE::MCA_CTL_PIE	MCA::PIE::MCA_STATUS_PIE	MCA::PIE::MCA_ADDR_PIE	MCA::PIE::MCA_MISC0_PIE	MCA::PIE::MCA_CTL_MASK_PIE
CS	MCA::CS::MCA_CTL_CS	MCA::CS::MCA_STATUS_CS	MCA::CS::MCA_ADDR_CS	MCA::CS::MCA_MISC0_CS	MCA::CS::MCA_CTL_MASK_CS
UMC	MCA::UMC::MCA_CTL_UMC	MCA::UMC::MCA_STATUS_UMC	MCA::UMC::MCA_ADDR_UMC	MCA::UMC::MCA_MISC0_UMC MCA::UMC::MCA_MISC1_UMC	MCA::UMC::MCA_CTL_MASK_UMC
PB	MCA::PB::MCA_CTL_PB	MCA::PB::MCA_STATUS_PB	MCA::PB::MCA_ADDR_PB	MCA::PB::MCA_MISC0_PB	MCA::PB::MCA_CTL_MASK_PB
PSP	MCA::PSP::MCA_CTL_PSP	MCA::PSP::MCA_STATUS_PSP	MCA::PSP::MCA_ADDR_PSP	MCA::PSP::MCA_MISC0_PSP	MCA::PSP::MCA_CTL_MASK_PSP
MP5	MCA::MP5::MCA_CTL_MP5	MCA::MP5::MCA_STATUS_MP5	MCA::MP5::MCA_ADDR_MP5	MCA::MP5::MCA_MISC0_MP5	MCA::MP5::MCA_CTL_MASK_MP5
SMU	MCA::SMU::MCA_CTL_SMU_MP1 MCA::SMU::MCA_CTL_SMU_MPIO	MCA::SMU::MCA_STATUS_SMU	MCA::SMU::MCA_ADDR_SMU	MCA::SMU::MCA_MISC0_SMU	MCA::SMU::MCA_CTL_MASK_SMU_MP1 MCA::SMU::MCA_CTL_MASK_SMU_MPIO
NBIO	MCA::NBIO::MCA_CTL_NBIO	MCA::NBIO::MCA_STATUS_NBIO	MCA::NBIO::MCA_ADDR_NBIO	MCA::NBIO::MCA_MISC0_NBIO	MCA::NBIO::MCA_CTL_MASK_NBIO
PCIE	MCA::PCIE::MCA_CTL_PCIE	MCA::PCIE::MCA_STATUS_PCIE	MCA::PCIE::MCA_ADDR_PCIE	MCA::PCIE::MCA_MISC0_PCIE	MCA::PCIE::MCA_CTL_MASK_PCIE
NBIF	MCA::NBIF::MCA_CTL_NBIF	MCA::NBIF::MCA_STATUS_NBIF	MCA::NBIF::MCA_ADDR_NBIF	MCA::NBIF::MCA_MISC0_NBIF	MCA::NBIF::MCA_CTL_MASK_NBIF
SHUB	MCA::SHUB::MCA_CTL_SHUB	MCA::SHUB::MCA_STATUS_SHUB	MCA::SHUB::MCA_ADDR_SHUB	MCA::SHUB::MCA_MISC0_SHUB	MCA::SHUB::MCA_CTL_MASK_SHUB
KPX_SERDES	MCA::KPX::SERDES::MCA_CTL_KPX_SERDES	MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES	MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES	MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES	MCA::KPX::SERDES::MCA_CTL_MASK_KPX_SERDES
KPX_GMI	MCA::KPX::GMI::MCA_CTL_KPX_GMI	MCA::KPX::GMI::MCA_STATUS_KPX_GMI	MCA::KPX::GMI::MCA_ADDR_KPX_GMI	MCA::KPX::GMI::MCA_MISC0_KPX_GMI	MCA::KPX::GMI::MCA_CTL_MASK_KPX_GMI
KPX_WAFL	MCA::KPX::WAFL::MCA_CTL_KPX_WAFL	MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL	MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL	MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL	MCA::KPX::WAFL::MCA_CTL_MASK_KPX_WAFL
PCS_GMI	MCA::PCS::GMI::MCA_CTL_PCS_GMI	MCA::PCS::GMI::MCA_STATUS_PCS_GMI	MCA::PCS::GMI::MCA_ADDR_PCS_GMI	MCA::PCS::GMI::MCA_MISC0_PCS_GMI	MCA::PCS::GMI::MCA_CTL_MASK_PCS_GMI
PCS_XGMI	MCA::PCS::XGMI::MCA_CTL_PCS_XGMI	MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI	MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI	MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI	MCA::PCS::XGMI::MCA_CTL_MASK_PCS_XGMI
MPDMA	MCA::MPDMA::MCA_CTL_MPDMA	MCA::MPDMA::MCA_STATUS_MPDMA	MCA::MPDMA::MCA_ADDR_MPDMA	MCA::MPDMA::MCA_MISC0_MPDMA	MCA::MPDMA::MCA_CTL_MASK_MPDMA
SATA	MCA::SATA::MCA_CTL_SATA	MCA::SATA::MCA_STATUS_SATA	MCA::SATA::MCA_ADDR_SATA	MCA::SATA::MCA_MISC0_SATA	MCA::SATA::MCA_CTL_MASK_SATA
USB	MCA::USB::MCA_CTL_USB	MCA::USB::MCA_STATUS_USB	MCA::USB::MCA_ADDR_USB	MCA::USB::MCA_MISC0_USB	MCA::USB::MCA_CTL_MASK_USB

Table 54: MCAX Registers

Block	MCA Register				
	CONFIG	IPID	SYND	DESTAT	DEADDR
LS	MCA::LS::MCA_CONFIG_LS	MCA::LS::MCA_IPID_LS	MCA::LS::MCA_SYND_LS	MCA::LS::MCA_DESTAT_LS	MCA::LS::MCA_DEADDR_LS
IF	MCA::IF::MCA_CONFIG_IF	MCA::IF::MCA_IPID_IF	MCA::IF::MCA_SYND_IF	--	--
L2	MCA::L2::MCA_CONFIG_L2	MCA::L2::MCA_IPID_L2	MCA::L2::MCA_SYND_L2	MCA::L2::MCA_DESTAT_L2	MCA::L2::MCA_DEADDR_L2
DE	MCA::DE::MCA_CONFIG_DE	MCA::DE::MCA_IPID_DE	MCA::DE::MCA_SYND_DE	--	--
EX	MCA::EX::MCA_CONFIG_EX	MCA::EX::MCA_IPID_EX	MCA::EX::MCA_SYND_EX	--	--
FP	MCA::FP::MCA_CONFIG_FP	MCA::FP::MCA_IPID_FP	MCA::FP::MCA_SYND_FP	--	--
L3	MCA::L3::MCA_CONFIG_L3	MCA::L3::MCA_IPID_L3	MCA::L3::MCA_SYND_L3	MCA::L3::MCA_DESTAT_L3	MCA::L3::MCA_DEADDR_L3
PIE	MCA::PIE::MCA_CONFIG_PIE	MCA::PIE::MCA_IPID_PIE	MCA::PIE::MCA_SYND_PIE	MCA::PIE::MCA_DESTAT_PIE	MCA::PIE::MCA_DEADDR_PIE
CS	MCA::CS::MCA_CONFIG_CS	MCA::CS::MCA_IPID_CS	MCA::CS::MCA_SYND_CS	MCA::CS::MCA_DESTAT_CS	MCA::CS::MCA_DEADDR_CS
UMC	MCA::UMC::MCA_CONFIG_UMC	MCA::UMC::MCA_IPID_UMC	MCA::UMC::MCA_SYND_UMC MCA::UMC::MCA_SYND1_UMC MCA::UMC::MCA_SYND2_UMC	MCA::UMC::MCA_DESTAT_UMC	MCA::UMC::MCA_DEADDR_UMC
PB	MCA::PB::MCA_CONFIG_PB	MCA::PB::MCA_IPID_PB	MCA::PB::MCA_SYND_PB	--	--
PSP	MCA::PSP::MCA_CONFIG_PSP	MCA::PSP::MCA_IPID_PSP	MCA::PSP::MCA_SYND_PSP	--	--
SMU	MCA::SMU::MCA_CONFIG_SMU	MCA::SMU::MCA_IPID_SMU	MCA::SMU::MCA_SYND_SMU	--	--
MP5	MCA::MP5::MCA_CONFIG_MP5	MCA::MP5::MCA_IPID_MP5	MCA::MP5::MCA_SYND_MP5	--	--
NBIO	MCA::NBIO::MCA_CONFIG_NBIO	MCA::NBIO::MCA_IPID_NBIO	MCA::NBIO::MCA_SYND_NBIO	MCA::NBIO::MCA_DESTAT_NBIO	
PCIE	MCA::PCIE::MCA_CONFIG_PCIE	MCA::PCIE::MCA_IPID_PCIE	MCA::PCIE::MCA_SYND_PCIE	MCA::PCIE::MCA_DESTAT_PCIE	MCA::PCIE::MCA_DEADDR_PCIE
NBIF	MCA::NBIF::MCA_CONFIG_NBIF	MCA::NBIF::MCA_IPID_NBIF	MCA::NBIF::MCA_SYND_NBIF	MCA::NBIF::MCA_DESTAT_NBIF	MCA::NBIF::MCA_DEADDR_NBIF
SHUB	MCA::SHUB::MCA_CONFIG_SHUB	MCA::SHUB::MCA_IPID_SHUB	MCA::SHUB::MCA_SYND_SHUB	MCA::SHUB::MCA_DESTAT_SHUB	MCA::SHUB::MCA_DEADDR_SHUB
KPX_SERDES	MCA::KPX::SERDES::MCA_CONFIG_KPX_SERDES	MCA::KPX::SERDES::MCA_IPID_KPX_SERDES	MCA::KPX::SERDES::MCA_SYND_KPX_SERDES	--	--
KPX_GMI	MCA::KPX::GMI::MCA_CONFIG_KPX_GMI	MCA::KPX::GMI::MCA_IPID_KPX_GMI	MCA::KPX::GMI::MCA_SYND_KPX_GMI	--	--
KPX_WAFL	MCA::KPX::WAFL::MCA_CONFIG_KPX_WAFL	MCA::KPX::WAFL::MCA_IPID_KPX_WAFL	MCA::KPX::WAFL::MCA_SYND_KPX_WAFL	--	--
PCS_GMI	MCA::PCS::GMI::MCA_CONFIG_PCS_GMI	MCA::PCS::GMI::MCA_IPID_PCS_GMI	MCA::PCS::GMI::MCA_SYND_PCS_GMI	--	--
PCS_XGMI	MCA::PCS::XGMI::MCA_CONFIG_PCS_XGMI	MCA::PCS::XGMI::MCA_IPID_PCS_XGMI	MCA::PCS::XGMI::MCA_SYND_PCS_XGMI	--	--
MPDMA	MCA::MPDMA::MCA_CONFIG_MPDMA	MCA::MPDMA::MCA_IPID_MPDMA	MCA::MPDMA::MCA_SYND_MPDMA	--	--
SATA	MCA::SATA::MCA_CONFIG_SATA	MCA::SATA::MCA_IPID_SATA	MCA::SATA::MCA_SYND_SATA	--	--
USB	MCA::USB::MCA_CONFIG_USB	MCA::USB::MCA_IPID_USB	MCA::USB::MCA_SYND_USB	--	--

3.2.3 Mapping of Banks to Blocks

Table 55 [Core MCA Bank to Block Mapping] shows MCA banks that are present in the address space of every logical

core.

Table 55: Core MCA Bank to Block Mapping

Bank	Block
0	LS
1	IF
2	L2
3	DE
4	RAZ
5	EX
6	FP

Table 56 [Non-core MCA Bank to Block Mapping] shows MCA banks that are present in the address space of specific logical cores.

Table 56: Non-core MCA Bank to Block Mapping

Bank	Thread 0	Thread 2	Thread 4	Thread 6	Thread 8	Thread 10	Thread 12	Thread 14	Thread 16	Thread 18	Thread 20	Thread 22
0	LS	LS	LS	LS	LS	LS	LS	LS	LS	LS	LS	LS
1	IF	IF	IF	IF	IF	IF	IF	IF	IF	IF	IF	IF
2	L2	L2	L2	L2	L2	L2	L2	L2	L2	L2	L2	L2
3	DE	DE	DE	DE	DE	DE	DE	DE	DE	DE	DE	DE
4	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
5	EX	EX	EX	EX	EX	EX	EX	EX	EX	EX	EX	EX
6	FP	FP	FP	FP	FP	FP	FP	FP	FP	FP	FP	FP
7	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
8	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
9	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
10	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
11	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
12	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
13	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
14	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3	L3
15	MP5	MP5	MP5	MP5	MP5	MP5	MP5	MP5	MP5	MP5	MP5	MP5
16	PB	PB	PB	PB	PB	PB	PB	PB	PB	PB	PB	PB
17	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI
18	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI
19	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI
20	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI
21	UMC	UMC	UMC	UMC	UMC	UMC	PB	RAZ	RAZ	RAZ	RAZ	RAZ
22	UMC	UMC	UMC	UMC	UMC	UMC	PCIE	PCIE	RAZ	RAZ	RAZ	RAZ
23	CS	CS	CS	CS	CS	CS	CS	CS	RAZ	RAZ	RAZ	RAZ
24	CS	CS	CS	CS	CS	CS	CS	CS	RAZ	RAZ	RAZ	RAZ
25	NBIO	NBIO	NBIO	NBIO	SHUB	SHUB	SHUB	SHUB	RAZ	RAZ	RAZ	RAZ
26	PCIE	PCIE	PCIE	PCIE	SATA	SATA	SATA	SATA	RAZ	RAZ	RAZ	RAZ
27	PCIE	PCIE	PCIE	PCIE	NBIF	NBIF	NBIF	NBIF	RAZ	RAZ	RAZ	RAZ
28	PIE	PSP	KPX_WA FL	KPX_WA FL	NBIF	NBIF	USB	USB	RAZ	RAZ	RAZ	RAZ
29	SMU	SMU	MPDMA	MPDMA	MPDMA	MPDMA	MPDMA	MPDMA	RAZ	RAZ	RAZ	RAZ
30	PCS_XG MI	PCS_XG MI	PCS_XG MI	PCS_XG MI	PCS_XG MI	PCS_XG MI	PCS_XG MI	PCS_XG MI	RAZ	RAZ	RAZ	RAZ
31	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	RAZ	RAZ	RAZ	RAZ
32	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ

33	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
34	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
35	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
36	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
37	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
38	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
39	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
40	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
41	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
42	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
43	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
44	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
45	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
46	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
47	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
48	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
49	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
50	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
51	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
52	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
53	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
54	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
55	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
56	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
57	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
58	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
59	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
60	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
61	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
62	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
63	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ

Table 57 [Non-core MCA Bank to Block Mapping for SDP-wide products]

Table 57: Non-core MCA Bank to Block Mapping for SDP-wide products

Bank	Thread 0	Thread 2	Thread 4	Thread 6	Thread 8	Thread 10	Thread 12	Thread 14
0	LS	LS	LS	LS	LS	LS	LS	LS
1	IF	IF	IF	IF	IF	IF	IF	IF
2	L2	L2	L2	L2	L2	L2	L2	L2
3	DE	DE	DE	DE	DE	DE	DE	DE
4	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
5	EX	EX	EX	EX	EX	EX	EX	EX
6	FP	FP	FP	FP	FP	FP	FP	FP
7	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
8	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
9	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
10	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
11	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
12	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
13	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
14	L3	L3	L3	L3	RAZ	RAZ	RAZ	RAZ
15	MP5	MP5	MP5	MP5	RAZ	RAZ	RAZ	RAZ

16	PB	PB	PB	PB	RAZ	RAZ	RAZ	RAZ
17	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI
18	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI	PCS_GMI
19	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI
20	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI	KPX_GMI
21	UMC	UMC	UMC	UMC	UMC	UMC	PB	RAZ
22	UMC	UMC	UMC	UMC	UMC	UMC	PCIE	PCIE
23	CS	CS	CS	CS	CS	CS	CS	CS
24	CS	CS	CS	CS	CS	CS	CS	CS
25	NBIO	NBIO	NBIO	NBIO	SHUB	SHUB	SHUB	SHUB
26	PCIE	PCIE	PCIE	PCIE	SATA	SATA	SATA	SATA
27	PCIE	PCIE	PCIE	PCIE	NBIF	NBIF	NBIF	NBIF
28	PIE	PSP	KPX_WAF L	KPX_WAF L	NBIF	NBIF	USB	USB
29	SMU	SMU	MPDMA	MPDMA	MPDMA	MPDMA	MPDMA	MPDMA
30	PCS_XGM I	PCS_XGM I	PCS_XGM I	PCS_XGM I	PCS_XGM I	PCS_XGM I	PCS_XGM I	PCS_XGM I
31	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES	KPX_SER DES
32	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
33	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
34	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
35	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
36	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
37	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
38	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
39	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
40	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
41	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
42	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
43	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
44	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
45	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
46	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
47	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
48	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
49	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
50	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
51	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
52	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
53	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
54	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
55	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
56	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
57	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
58	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
59	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
60	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
61	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ

62	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ
63	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ	RAZ

3.2.4 Decoding Error Type

If a valid error is logged in MCA_STATUS or MCA_DESTAT of an MCA bank:

1. Read the values of this bank's MCA_IPID and MCA_STATUS registers.
2. Use Table 51 [Blocks Capable of Supporting MCA Banks] to look up the block associated with the values of MCA_IPID[HwId] and MCA_IPID[McaType].
3. In 3.2.5 [MCA Banks], find the sub-section associated with the block in error.
4. In this sub-section, find the MCA_STATUS table.
5. In the table, look up the row associated with the MCA_STATUS[ErrorCodeExt] value.
6. The error type in this row is the logged error. The MCA_STATUS, MCA_ADDR and MCA_SYND tables contain information associated with this error.
7. If there is an error in both MCA_STATUS and MCA_DESTAT, the registers contain the same error if MCA_STATUS[Deferred] is set. If MCA_STATUS[Deferred] is not set, MCA_DESTAT contains information for a different error than MCA_STATUS.

3.2.5 MCA Banks

3.2.5.1 LS

MSR0000_0400...MSRC000_2000 [LS Machine Check Control Thread 0] (MCA::LS::MCA_CTL_LS)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::LS::MCA_CTL_LS register must be enabled by the corresponding enable bit in Core::X86::Msrr:[MCG_CTL](#). Does not affect error detection, correction, or logging.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSRLEGACY; MSR0000_0400
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2000
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSRLEGACY; MSR0000_0400
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2000
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSRLEGACY; MSR0000_0400

_ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2000
_ccd10_lthree0_core6_thread1_inst0_n173_aliasMSRLEGACY; MSR0000_0400
_ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2000
_ccd10_lthree0_core7_thread0_inst0_n174_aliasMSRLEGACY; MSR0000_0400
_ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2000
_ccd10_lthree0_core7_thread1_inst0_n175_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2000
_ccd11_lthree0_core0_thread0_inst0_n176_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2000
_ccd11_lthree0_core0_thread1_inst0_n177_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2000
_ccd11_lthree0_core1_thread0_inst0_n178_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2000
_ccd11_lthree0_core1_thread1_inst0_n179_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2000
_ccd11_lthree0_core2_thread0_inst0_n180_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2000
_ccd11_lthree0_core2_thread1_inst0_n181_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2000
_ccd11_lthree0_core3_thread0_inst0_n182_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2000
_ccd11_lthree0_core3_thread1_inst0_n183_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2000
_ccd11_lthree0_core4_thread0_inst0_n184_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2000
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2000
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2000
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2000
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2000
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2000
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSRLEGACY; MSR0000_0400
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2000
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2000
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2000
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2000
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2000
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2000
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2000
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2000
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2000
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2000
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2000
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2000
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2000
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2000
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2000

_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2000
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSRLEGACY; MSR0000_0400
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2000
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2000
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2000
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2000
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2000
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2000
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2000
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2000
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2000
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2000
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2000
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2000
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2000
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2000
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2000
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2000
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSRLEGACY; MSR0000_0400
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2000
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2000
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2000
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2000
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2000
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2000
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2000
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2000
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2000
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2000
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2000
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2000
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2000
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2000
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2000
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSRLEGACY; MSR0000_0400

_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2000
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSRLEGACY; MSR0000_0400
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2000
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2000
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2000
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2000
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2000
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2000
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2000
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2000
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2000
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2000
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2000
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2000
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2000
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2000
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2000
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2000
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSRLEGACY; MSR0000_0400
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2000
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2000
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2000
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2000
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2000
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2000
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2000
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2000
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2000
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2000
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2000
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2000
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2000
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2000
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2000
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2000

_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSRLEGACY; MSR0000_0400
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2000
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2000
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2000
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2000
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2000
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2000
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2000
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2000
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2000
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2000
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2000
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2000
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2000
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2000
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2000
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2000
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSRLEGACY; MSR0000_0400
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2000
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2000
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2000
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2000
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2000
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2000
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2000
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2000
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2000
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2000
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2000
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2000
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2000
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2000
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2000
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSRLEGACY; MSR0000_0400
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2000
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSRLEGACY; MSR0000_0400

_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2000
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2000
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2000
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2000
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2000
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2000
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2000
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2000
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2000
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2000
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2000
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2000
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2000
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2000
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2000
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2000
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSRLEGACY; MSR0000_0400
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2000
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2000
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2000
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2000
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2000
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2000
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2000
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2000
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2000
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2000
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2000
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2000
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2000
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2000
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2000
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2000
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSRLEGACY; MSR0000_0400
_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2000

_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSRLEGACY; MSR0000_0400	
Bits	Description
63:24	Reserved.
23	STORE_DATA_OTHER. Read-write. Reset: 0. A parity error was detected in an STLF, SCB EMEM entry, store data mask or SRB store data by any access.
22	HWA. Read-write. Reset: 0. A hardware assertion error was reported
21	SystemReadDataErrorWcb. Read-write. Reset: 0. A SystemReadDataError error was reported on read data returned from L2 for a WCB store
20	SystemReadDataErrorScb. Read-write. Reset: 0. A SystemReadDataError error was reported on read data returned from L2 for an SCB store
19	SystemReadDataErrorLoad. Read-write. Reset: 0. A SystemReadDataError error was reported on read data returned from L2 for a load
18	SCB_POISON. Read-write. Reset: 0. A poisoned line was detected in an SCB entry by any access
17	WCB. Read-write. Reset: 0. A parity error was detected in a WCB entry by any access
16	SCB_DATA. Read-write. Reset: 0. A parity error was detected in an SCB entry data field by any access
15	SCB_ADDR. Read-write. Reset: 0. A parity error was detected in an SCB entry address field by any access
14	SCB_STATE. Read-write. Reset: 0. A parity error was detected in an SCB entry state field by any access
13	MAB. Read-write. Reset: 0. A parity error was detected in a MAB entry by any access
12	LDQ. Read-write. Reset: 0. A parity error was detected in an LDQ entry by any access
11	STQ. Read-write. Reset: 0. A parity error was detected in an STQ entry by any access
10	PWC. Read-write. Reset: 0. A parity error was detected in a PWC entry by any access
9	L2DTLB. Read-write. Reset: 0. A parity error was detected in an L2 TLB entry by any access
8	L1DTLB. Read-write. Reset: 0. A parity error was detected in an L1 TLB entry by any access. This error only logs a valid address down through bit [12], in spite of the AddrLsbCnt value of 0.
7	EMEM_RMW. Read-write. Reset: 0. An ECC error was detected on an EMEM read-modify-write by a store
6	EMEM_LOAD. Read-write. Reset: 0. An ECC error was detected on an EMEM read by a load.
5	DC_TAG_STORE. Read-write. Reset: 0. An ECC error or poison bit mismatch was detected on a tag read by a store.
4	DC_TAG_LOAD. Read-write. Reset: 0. An ECC error or poison bit mismatch was detected on a tag read by a load.
3	DC_TAG_VICTIM. Read-write. Reset: 0. An ECC error or poison bit mismatch was detected on a tag read by a probe or victimization.
2	DC_DATA_RMW. Read-write. Reset: 0. An ECC error was detected on a data cache read-modify-write by a store
1	DC_DATA_LOAD. Read-write. Reset: 0. An ECC error or L2 poison was detected on a data cache read by a load.
0	DC_DATA_VICTIM. Read-write. Reset: 0. An ECC error was detected on a data cache read by a probe or victimization

MSR0000_0001...MSRC000_2001 [LS Machine Check Status Thread 0] (MCA::LS::MCA_STATUS_LS)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSRLSLEGACY; MSR0000_0001
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2001
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSRLEGACY; MSR0000_0401
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSRLSLEGACY; MSR0000_0001
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2001
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSRLEGACY; MSR0000_0401
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSRLSLEGACY; MSR0000_0001
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2001
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSRLEGACY; MSR0000_0401
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSRLSLEGACY; MSR0000_0001
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2001
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSRLEGACY; MSR0000_0401
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSRLSLEGACY; MSR0000_0001
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2001
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSRLEGACY; MSR0000_0401
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSRLSLEGACY; MSR0000_0001
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2001

[illegible]

_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2001
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSRLEGACY; MSR0000_0401
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSRSLLEGACY; MSR0000_0001
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2001
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSRLEGACY; MSR0000_0401
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSRSLLEGACY; MSR0000_0001
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2001
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSRLEGACY; MSR0000_0401
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSRSLLEGACY; MSR0000_0001
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2001
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSRLEGACY; MSR0000_0401
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSRSLLEGACY; MSR0000_0001
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2001
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSRLEGACY; MSR0000_0401
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSRSLLEGACY; MSR0000_0001
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2001
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSRLEGACY; MSR0000_0401
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2001
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2001
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2001
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2001
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2001
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2001
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2001
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2001
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2001
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2001
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2001
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2001
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2001
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2001
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2001
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSRLEGACY; MSR0000_0401
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSRSLLEGACY; MSR0000_0001
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2001
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSRLEGACY; MSR0000_0401

[illegible]

[illegible]

[illegible]

[illegible]

_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSRSLLEGACY; MSR0000_0001	
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2001	
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSRLEGACY; MSR0000_0401	
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2001	
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSRLEGACY; MSR0000_0401	
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2001	
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSRLEGACY; MSR0000_0401	
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2001	
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSRLEGACY; MSR0000_0401	
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2001	
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSRLEGACY; MSR0000_0401	
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2001	
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSRLEGACY; MSR0000_0401	
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2001	
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSRLEGACY; MSR0000_0401	
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSRSLLEGACY; MSR0000_0001	
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2001	
... (skipped 76 rows)	
_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSRSLLEGACY; MSR0000_0001	
Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::LS::MCA_CTL_LS. This bit is a copy of bit in MCA::LS::MCA_CTL_LS for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::LS::MCA_MISC0_LS. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::LS::MCA_ADDR_LS contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::LS::MCA_STATUS_LS[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::LS::MCA_SYND_LS. If MCA::LS::MCA_SYND_LS[ErrorPriority] is the same as the priority of the error in MCA::LS::MCA_STATUS_LS, then the information in MCA::LS::MCA_SYND_LS is associated with the error in MCA::LS::MCA_STATUS_LS. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::LS::MCA_STATUS_LS[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::LS::MCA_ADDR_LS[ErrorAddr]. A value of 0 indicates that MCA::LS::MCA_ADDR_LS[56:0] contains a valid byte address. A value of 6 indicates that MCA::LS::MCA_ADDR_LS[56:6] contains a valid cache line address and that MCA::LS::MCA_ADDR_LS[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::LS::MCA_ADDR_LS[56:12] contain a valid 4KB memory page and that MCA::LS::MCA_ADDR_LS[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::LS::MCA_CTL_LS enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 58: MCA_STATUS_LS

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
DC_DATA_VICTIM	0x0	0	0	0	0/1	0	1
DC_DATA_LOAD	0x1	0/1	0	0/1	0	0/1	1
DC_DATA_RMW	0x2	0	0	0	0/1	0	1
DC_TAG_VICTIM	0x3	0/1	0/1	0/1	0/1	0	0
DC_TAG_LOAD	0x4	0/1	0/1	0/1	0/1	0	0
DC_TAG_STORE	0x5	0/1	0/1	0/1	0/1	0	0
EMEM_LOAD	0x6	1	1	1	0	0	1
EMEM_RMW	0x7	0/1	0/1	0/1	0	0	0
L1DTLB	0x8	0	0	0	0	0	0
L2DTLB	0x9	0	0	0	0	0	1
PWC	0xa	0	0	0	0	0	1
STQ	0xb	1	1	1	0	0	0
LDQ	0xc	1	1	1	0	0	0
MAB	0xd	1	1	1	0	0	0
SCB_STATE	0xe	1	1	1	0	0	0
SCB_ADDR	0xf	1	1	1	0	0	0
SCB_DATA	0x10	1	1	1	0	0	0

WCB	0x11	1	1	1	0	0	0
SCB_POISON	0x12	0	0	0	1	0	0
SystemReadDataErrorLoad	0x13	1	0	1	0	0	1
SystemReadDataErrorSnb	0x14	1	1	1	0	0	1
SystemReadDataErrorWcb	0x15	1	1	1	0	0	0
HWA	0x16	1	1	1	0	0	0
STORE_DATA_OTHER	0x17	1	1	1	0	0	0

MSR0000_0000...MSRC000_2002 [LS Machine Check Address Thread 0] (MCA::LS::MCA_ADDR_LS)

Reset: Cold,0000_0000_0000_0000h.

MCA::LS::MCA_ADDR_LS stores an address and other information associated with the error in MCA::LS::MCA_STATUS_LS. The register is only meaningful if MCA::LS::MCA_STATUS_LS[Val]=1 and MCA::LS::MCA_STATUS_LS[AddrV]=1.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSRLSLEGACY; MSR0000_0000
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2002
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSRLEGACY; MSR0000_0402
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSRLSLEGACY; MSR0000_0000
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2002
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSRLEGACY; MSR0000_0402
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSRLSLEGACY; MSR0000_0000
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2002
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSRLEGACY; MSR0000_0402
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSRLSLEGACY; MSR0000_0000
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2002
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSRLEGACY; MSR0000_0402
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSRLSLEGACY; MSR0000_0000
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2002

[illegible]

_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2002
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSRSLLEGACY; MSR0000_0000
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2002
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSRSLLEGACY; MSR0000_0000
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2002
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSRSLLEGACY; MSR0000_0000
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2002
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSRSLLEGACY; MSR0000_0000
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2002
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSRSLLEGACY; MSR0000_0000
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2002
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSRSLLEGACY; MSR0000_0000
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2002
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSRLEGACY; MSR0000_0402
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2002
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2002
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2002
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2002
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2002
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2002
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2002
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2002
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2002
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2002
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2002
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2002
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2002
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2002
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSRLEGACY; MSR0000_0402
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSRSLLEGACY; MSR0000_0000
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2002
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSRLEGACY; MSR0000_0402

[illegible]

[illegible]

[illegible]

_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2002
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2002
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2002
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2002
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2002
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2002
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2002
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2002
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2002
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2002
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2002
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2002
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2002
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2002
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2002
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSRSLLEGACY; MSR0000_0000
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2002
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSRLEGACY; MSR0000_0402
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSRSLLEGACY; MSR0000_0000
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2002
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSRLEGACY; MSR0000_0402
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSRSLLEGACY; MSR0000_0000
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2002
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSRLEGACY; MSR0000_0402
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSRSLLEGACY; MSR0000_0000
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2002
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSRLEGACY; MSR0000_0402
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSRSLLEGACY; MSR0000_0000
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2002
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSRLEGACY; MSR0000_0402
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSRSLLEGACY; MSR0000_0000
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2002
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSRLEGACY; MSR0000_0402
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSRSLLEGACY; MSR0000_0000
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2002

<u>_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2002</u>	
<u>_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2002</u>	
<u>_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2002</u>	
<u>_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2002</u>	
<u>_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2002</u>	
<u>_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2002</u>	
<u>_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2002</u>	
<u>_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSRLEGACY; MSR0000_0402</u>	
<u>_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSRSLLEGACY; MSR0000_0000</u>	
<u>_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2002</u>	
... (skipped 76 rows)	
<u>_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSRSLLEGACY; MSR0000_0000</u>	
Bits	Description
63:57	Reserved.
56:0	ErrorAddr. Read-write,Volatile. Reset: Cold,000_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::LS::MCA_STATUS_LS. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 59: MCA_ADDR_LS

Error Type	Bits	Description
------------	------	-------------

DC_DATA_VICTIM	[56:52] [51:6] [5] [4:0]	Reserved Physical address Reserved Which data word had the error
DC_DATA_LOAD	[56:52] [51:1] [0]	Reserved Physical address Reserved
DC_DATA_RMW	[56:52] [51:6] [5:0]	Reserved Physical address Reserved
DC_TAG_VICTIM	[57:0]	Reserved
DC_TAG_LOAD	[57:0]	Reserved
DC_TAG_STORE	[57:0]	Reserved
EMEM_LOAD	[56:52] [51:1] [0]	Reserved Physical address Reserved
EMEM_RMW	[57:0]	Reserved
L1DTLB	[57:0]	Reserved
L2DTLB	[56:52] [51:12] [11:0]	Reserved Physical address Reserved
PWC	[56:52] [51:12] [11:0]	Reserved Physical address Reserved
STQ	[57:0]	Reserved
LDQ	[57:0]	Reserved
MAB	[57:0]	Reserved
SCB_STATE	[57:0]	Reserved
SCB_ADDR	[57:0]	Reserved
SCB_DATA	[57:0]	Reserved
WCB	[57:0]	Reserved
SCB_POISON	[57:0]	Reserved
SystemReadDataErrorLoad	[56:52] [51:1] [0]	Reserved Physical address Reserved
SystemReadDataErrorScb	[56:52] [51:6] [5:0]	Reserved Physical address Reserved
SystemReadDataErrorWcb	[57:0]	Reserved
HWA	[57:0]	Reserved
STORE_DATA_OTHER	[57:0]	Reserved

MSR0000_0403...MSRC000_2003 [LS Machine Check Miscellaneous 0 Thread 0] (MCA::LS::MCA_MISC0_LS)

Log miscellaneous information associated with errors.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSRLEGACY; MSR0000_0403
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2003
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSRLEGACY; MSR0000_0403
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2003
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSRLEGACY; MSR0000_0403

_ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2003
_ccd10_lthree0_core7_thread1_inst0_n175_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2003
_ccd11_lthree0_core0_thread0_inst0_n176_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2003
_ccd11_lthree0_core0_thread1_inst0_n177_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2003
_ccd11_lthree0_core1_thread0_inst0_n178_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2003
_ccd11_lthree0_core1_thread1_inst0_n179_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2003
_ccd11_lthree0_core2_thread0_inst0_n180_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2003
_ccd11_lthree0_core2_thread1_inst0_n181_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2003
_ccd11_lthree0_core3_thread0_inst0_n182_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2003
_ccd11_lthree0_core3_thread1_inst0_n183_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2003
_ccd11_lthree0_core4_thread0_inst0_n184_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2003
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2003
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2003
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2003
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2003
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2003
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSRLEGACY; MSR0000_0403
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2003
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2003
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2003
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2003
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2003
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2003
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2003
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2003
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2003
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2003
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2003
_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2003
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2003
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2003
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2003
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2003
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSRLEGACY; MSR0000_0403
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2003

_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2003
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2003
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2003
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2003
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2003
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2003
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2003
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2003
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2003
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2003
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2003
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2003
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2003
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2003
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2003
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSRLEGACY; MSR0000_0403
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2003
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2003
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2003
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2003
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2003
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2003
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2003
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2003
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2003
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2003
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2003
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2003
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2003
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2003
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2003
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2003
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSRLEGACY; MSR0000_0403
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2003
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSRLEGACY; MSR0000_0403

_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2003
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2003
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2003
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2003
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2003
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2003
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2003
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2003
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2003
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2003
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2003
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2003
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2003
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2003
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2003
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSRLEGACY; MSR0000_0403
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2003
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2003
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2003
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2003
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2003
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2003
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2003
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2003
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2003
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2003
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2003
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2003
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2003
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2003
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2003
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2003
_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSRLEGACY; MSR0000_0403
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2003
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2003

_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2003
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2003
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2003
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2003
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2003
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2003
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2003
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2003
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2003
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2003
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2003
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2003
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2003
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2003
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSRLEGACY; MSR0000_0403
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2003
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2003
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2003
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2003
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2003
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2003
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2003
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2003
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2003
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2003
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2003
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2003
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2003
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2003
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2003
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2003
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSRLEGACY; MSR0000_0403
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2003
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSRLEGACY; MSR0000_0403
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2003
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSRLEGACY; MSR0000_0403

<u>_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2003</u>	
<u>_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSRLEGACY; MSR0000_0403</u>	
<u>_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2003</u>	
<u>_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSRLEGACY; MSR0000_0403</u>	
Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP . Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked . Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP . Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::LS::MCA_MISC0_LS[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset . Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::LS::MCA_MISC0_LS[Locked]) ? Read-write : Read-only.
51	CntEn . Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::LS::MCA_MISC0_LS[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType . Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrlw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::LS::MCA_MISC0_LS[Locked]) ? Read-write : Read-only.
48	Ovrlw . Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::LS::MCA_MISC0_LS[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt . Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::LS::MCA_MISC0_LS[Locked]) ? Read-write : Read-only.
31:24	BlkPtr . Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2004 [LS Machine Check Configuration Thread 0] (MCA::LS::MCA_CONFIG_LS)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2004
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2004
 _ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2004
 _ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2004
 _ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2004

_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2004
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2004
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2004
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2004
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2004
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2004
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2004
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2004
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2004
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2004
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2004
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2004
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2004
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2004
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2004
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2004
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2004
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2004
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2004
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2004
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2004
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2004
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2004
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2004
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2004
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2004
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2004
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2004
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2004
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2004
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2004
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2004
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2004
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2004
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2004
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2004
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2004
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2004
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2004
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2004
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2004
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2004
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2004
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2004
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2004
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2004
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2004
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2004
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2004
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2004
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2004
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2004
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2004
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2004
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2004
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2004
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2004
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2004
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2004
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2004
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2004
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2004
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2004
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2004
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2004

_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2004
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2004
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2004
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2004
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2004
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2004
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2004
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2004
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2004
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2004
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2004
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2004
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2004
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2004
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2004
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2004
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2004
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2004
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2004
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2004
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2004
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2004
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2004
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2004
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2004
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2004
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2004
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2004
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2004
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2004
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2004
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2004
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2004
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2004
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2004
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2004
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2004
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2004
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2004
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2004
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2004
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2004
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2004
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2004
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2004
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2004
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2004
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2004
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2004
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2004
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2004
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2004
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2004
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2004
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2004
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2004
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2004
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2004
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2004
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2004
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2004
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2004
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2004
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2004
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2004

_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2004	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat. Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::LS::MCA_STATUS_LS and MCA::LS::MCA_ADDR_LS in addition to MCA::LS::MCA_DESTAT_LS and MCA::LS::MCA_DEADDR_LS. 0=Only log deferred errors in MCA::LS::MCA_DESTAT_LS and MCA::LS::MCA_DEADDR_LS. This bit does not affect logging of deferred errors in MCA::LS::MCA_SYND_LS, MCA::LS::MCA_MISC0_LS.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::LS::MCA_STATUS_LS, MCA::LS::MCA_ADDR_LS, MCA::LS::MCA_MISC0_LS, and MCA::LS::MCA_SYND_LS.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::LS::MCA_CONFIG_LS[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::LS::MCA_CONFIG_LS[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::LS::MCA_CONFIG_LS[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::LS::MCA_CONFIG_LS[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::LS::MCA_DESTAT_LS and MCA::LS::MCA_DEADDR_LS are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::LS::MCA_MISC0_LS[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::LS::MCA_STATUS_LS[TCC] is present.

MSRC000_2005 [LS IP Identification Thread 0] (MCA::LS::MCA_IPID_LS)

Reset: 0010_00B0_0000_0000h.

The MCA::LS::MCA_IPID_LS register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2005
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2005
 _ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2005
 _ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2005
 _ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2005

_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2005
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2005
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2005
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2005
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2005
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2005
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2005
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2005
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2005
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2005
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2005
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2005
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2005
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2005
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2005
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2005
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2005
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2005
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2005
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2005
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2005
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2005
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2005
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2005
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2005
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2005
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2005
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2005
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2005
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2005
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2005
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2005
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2005
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2005
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2005
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2005
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2005
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2005
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2005
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2005
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2005
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2005
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2005
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2005
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2005
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2005
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2005
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2005
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2005
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2005
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2005
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2005
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2005
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2005
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2005
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2005
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2005
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2005
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2005
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2005
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2005
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2005
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2005
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2005
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2005

_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2005
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2005
_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2005
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2005
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2005
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2005
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2005
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2005
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2005
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2005
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2005
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2005
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2005
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2005
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2005
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2005
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2005
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2005
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2005
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2005
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2005
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2005
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2005
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2005
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2005
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2005
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2005
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2005
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2005
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2005
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2005
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2005
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2005
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2005
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2005
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2005
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2005
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2005
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2005
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2005
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2005
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2005
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2005
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2005
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2005
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2005
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2005
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2005
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2005
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2005
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2005
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2005
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2005
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2005
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2005
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2005
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2005
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2005
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2005
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2005
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2005
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2005
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2005
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2005
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2005

_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2005	
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2005	
_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2005	
Bits	Description
63:48	McaType . Read-only. Reset: 0010h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR: 2000_AE00h Init: _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR: 2000_AF00h Init: _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR: 2002_AE00h Init: _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR: 2002_AF00h Init: _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR: 2004_AE00h Init: _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR: 2004_AF00h Init: _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR: 2006_AE00h Init: _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR: 2006_AF00h Init: _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR: 2008_AE00h Init: _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR: 2008_AF00h Init: _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR: 200A_AE00h Init: _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR: 200A_AF00h Init: _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR: 200C_AE00h Init: _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR: 200C_AF00h Init: _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR: 200E_AE00h Init: _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR: 200E_AF00h Init: _ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR: 2080_AE00h Init: _ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR: 2080_AF00h Init: _ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR: 2082_AE00h Init: _ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR: 2082_AF00h Init: _ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR: 2084_AE00h Init: _ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR: 2084_AF00h Init: _ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR: 2086_AE00h Init: _ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR: 2086_AF00h Init: _ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR: 2088_AE00h Init: _ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR: 2088_AF00h Init: _ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR: 208A_AE00h Init: _ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR: 208A_AF00h Init: _ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR: 208C_AE00h Init: _ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR: 208C_AF00h Init: _ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR: 208E_AE00h Init: _ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR: 208E_AF00h Init: _ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR: 2100_AE00h Init: _ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR: 2100_AF00h Init: _ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR: 2102_AE00h Init: _ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR: 2102_AF00h Init: _ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR: 2104_AE00h Init: _ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR: 2104_AF00h Init: _ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR: 2106_AE00h Init: _ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR: 2106_AF00h Init: _ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR: 2108_AE00h Init: _ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR: 2108_AF00h Init: _ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR: 210A_AE00h Init: _ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR: 210A_AF00h Init: _ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR: 210C_AE00h Init: _ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR: 210C_AF00h Init: _ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR: 210E_AE00h Init: _ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR: 210E_AF00h Init: _ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR: 2180_AE00h Init: _ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR: 2180_AF00h Init: _ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR: 2182_AE00h Init: _ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR: 2182_AF00h Init: _ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR: 2184_AE00h

Init: _ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR: 2184_AE00h
Init: _ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR: 2186_AE00h
Init: _ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR: 2186_AF00h
Init: _ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR: 2188_AE00h
Init: _ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR: 2188_AF00h
Init: _ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR: 218A_AE00h
Init: _ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR: 218A_AF00h
Init: _ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR: 218C_AE00h
Init: _ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR: 218C_AF00h
Init: _ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR: 218E_AE00h
Init: _ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR: 218E_AF00h
Init: _ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR: 2200_AE00h
Init: _ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR: 2200_AF00h
Init: _ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR: 2202_AE00h
Init: _ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR: 2202_AF00h
Init: _ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR: 2204_AE00h
Init: _ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR: 2204_AF00h
Init: _ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR: 2206_AE00h
Init: _ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR: 2206_AF00h
Init: _ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR: 2208_AE00h
Init: _ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR: 2208_AF00h
Init: _ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR: 220A_AE00h
Init: _ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR: 220A_AF00h
Init: _ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR: 220C_AE00h
Init: _ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR: 220C_AF00h
Init: _ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR: 220E_AE00h
Init: _ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR: 220E_AF00h
Init: _ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR: 2280_AE00h
Init: _ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR: 2280_AF00h
Init: _ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR: 2282_AE00h
Init: _ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR: 2282_AF00h
Init: _ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR: 2284_AE00h
Init: _ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR: 2284_AF00h
Init: _ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR: 2286_AE00h
Init: _ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR: 2286_AF00h
Init: _ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR: 2288_AE00h
Init: _ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR: 2288_AF00h
Init: _ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR: 228A_AE00h
Init: _ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR: 228A_AF00h
Init: _ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR: 228C_AE00h
Init: _ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR: 228C_AF00h
Init: _ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR: 228E_AE00h
Init: _ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR: 228E_AF00h
Init: _ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR: 2300_AE00h
Init: _ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR: 2300_AF00h
Init: _ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR: 2302_AE00h
Init: _ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR: 2302_AF00h
Init: _ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR: 2304_AE00h
Init: _ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR: 2304_AF00h
Init: _ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR: 2306_AE00h
Init: _ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR: 2306_AF00h
Init: _ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR: 2308_AE00h
Init: _ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR: 2308_AF00h
Init: _ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR: 230A_AE00h
Init: _ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR: 230A_AF00h
Init: _ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR: 230C_AE00h
Init: _ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR: 230C_AF00h
Init: _ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR: 230E_AE00h
Init: _ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR: 230E_AF00h
Init: _ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR: 2380_AE00h
Init: _ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR: 2380_AF00h
Init: _ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR: 2382_AE00h
Init: _ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR: 2382_AF00h
Init: _ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR: 2384_AE00h
Init: _ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR: 2384_AF00h

Init: _ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR: 2386_AE00h
Init: _ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR: 2386_AF00h
Init: _ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR: 2388_AE00h
Init: _ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR: 2388_AF00h
Init: _ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR: 238A_AE00h
Init: _ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR: 238A_AF00h
Init: _ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR: 238C_AE00h
Init: _ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR: 238C_AF00h
Init: _ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR: 238E_AE00h
Init: _ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR: 238E_AF00h
Init: _ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR: 2400_AE00h
Init: _ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR: 2400_AF00h
Init: _ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR: 2402_AE00h
Init: _ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR: 2402_AF00h
Init: _ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR: 2404_AE00h
Init: _ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR: 2404_AF00h
Init: _ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR: 2406_AE00h
Init: _ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR: 2406_AF00h
Init: _ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR: 2408_AE00h
Init: _ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR: 2408_AF00h
Init: _ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR: 240A_AE00h
Init: _ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR: 240A_AF00h
Init: _ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR: 240C_AE00h
Init: _ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR: 240C_AF00h
Init: _ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR: 240E_AE00h
Init: _ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR: 240E_AF00h
Init: _ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR: 2480_AE00h
Init: _ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR: 2480_AF00h
Init: _ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR: 2482_AE00h
Init: _ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR: 2482_AF00h
Init: _ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR: 2484_AE00h
Init: _ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR: 2484_AF00h
Init: _ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR: 2486_AE00h
Init: _ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR: 2486_AF00h
Init: _ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR: 2488_AE00h
Init: _ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR: 2488_AF00h
Init: _ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR: 248A_AE00h
Init: _ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR: 248A_AF00h
Init: _ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR: 248C_AE00h
Init: _ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR: 248C_AF00h
Init: _ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR: 248E_AE00h
Init: _ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR: 248E_AF00h
Init: _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR: 2500_AE00h
Init: _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR: 2500_AF00h
Init: _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR: 2502_AE00h
Init: _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR: 2502_AF00h
Init: _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR: 2504_AE00h
Init: _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR: 2504_AF00h
Init: _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR: 2506_AE00h
Init: _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR: 2506_AF00h
Init: _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR: 2508_AE00h
Init: _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR: 2508_AF00h
Init: _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR: 250A_AE00h
Init: _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR: 250A_AF00h
Init: _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR: 250C_AE00h
Init: _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR: 250C_AF00h
Init: _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR: 250E_AE00h
Init: _ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR: 250E_AF00h
Init: _ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR: 2580_AE00h
Init: _ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR: 2580_AF00h
Init: _ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR: 2582_AE00h
Init: _ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR: 2582_AF00h
Init: _ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR: 2584_AE00h
Init: _ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR: 2584_AF00h
Init: _ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR: 2586_AE00h

Init: _ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR: 2586_AF00h
Init: _ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR: 2588_AE00h
Init: _ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR: 2588_AF00h
Init: _ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR: 258A_AE00h
Init: _ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR: 258A_AF00h
Init: _ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR: 258C_AE00h
Init: _ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR: 258C_AF00h
Init: _ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR: 258E_AE00h
Init: _ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR: 258E_AF00h

MSRC000_2006 [LS Machine Check Syndrome Thread 0] (MCA::LS::MCA_SYND_LS)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::LS::MCA_STATUS_LS [Thread 0](#)

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2006
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2006
 _ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2006
 _ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2006
 _ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2006

_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2006
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2006
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2006
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2006
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2006
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2006
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2006
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2006
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2006
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2006
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2006
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2006
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2006
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2006
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2006
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2006
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2006
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2006
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2006
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2006
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2006
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2006
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2006
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2006
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2006
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2006
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2006
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2006
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2006
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2006
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2006
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2006
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2006
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2006
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2006
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2006
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2006
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2006
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2006
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2006
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2006
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2006
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2006
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2006
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2006
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2006
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2006
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2006
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2006
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2006
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2006
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2006
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2006
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2006
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2006
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2006
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2006
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2006
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2006
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2006
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2006
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2006
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2006
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2006
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2006

_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2006
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2006
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2006
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2006
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2006
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2006
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2006
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2006
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2006
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2006
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2006
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2006
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2006
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2006
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2006
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2006
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2006
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2006
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2006
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2006
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2006
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2006
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2006
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2006
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2006
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2006
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2006
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2006
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2006
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2006
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2006
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2006
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2006
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2006
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2006
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2006
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2006
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2006
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2006
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2006
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2006
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2006
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2006
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2006
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2006
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2006
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2006
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2006
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2006
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2006
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2006
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2006
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2006
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2006
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2006
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2006
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2006
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2006
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2006
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2006
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2006
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2006
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2006
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2006
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2006

_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2006	
Bits	Description
63:39	Reserved.
38:32	Syndrome. Read-write, Volatile. Reset: Cold, 00h. Contains the syndrome, if any, associated with the error logged in MCA::LS::MCA_STATUS_LS. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::LS::MCA_SYND_LS[Length]. The Syndrome field is only valid when MCA::LS::MCA_SYND_LS[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::LS::MCA_SYND_LS. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::LS::MCA_SYND_LS[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::LS::MCA_SYND_LS. For example, a syndrome length of 9 means that MCA::LS::MCA_SYND_LS[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 60 [MCA_SYND_LS].

Table 60: MCA_SYND_LS

Error Type	Bits	Description
DC_DATA_VICTIM	[17]	1
	[16]	1
	[15:14]	Reserved
	[13:8]	Cache line index
	[7:3]	Reserved
	[2:0]	Cache line way
DC_DATA_LOAD	[17]	1
	[16]	1
	[15:14]	Reserved
	[13:8]	Cache line index
	[7:3]	Reserved
	[2:0]	Cache line way
DC_DATA_RMW	[17]	0
	[16]	1
	[15:5]	Reserved
	[4:0]	Which data word had the error
DC_TAG_VICTIM	[17]	1
	[16]	1
	[15:14]	Reserved
	[13:8]	Cache line index
	[7:3]	Reserved
	[2:0]	Cache line way
DC_TAG_LOAD	[17]	1
	[16]	1
	[15:14]	Reserved
	[13:8]	Cache line index
	[7:3]	Reserved
	[2:0]	Cache line way
DC_TAG_STORE	[17]	1
	[16]	1

	[15:14]	Reserved
	[13:8]	Cache line index
	[7:3]	Reserved
	[2:0]	Cache line way
EMEM_LOAD	[17]	1
	[16]	0
	[15:14]	Reserved
	[13:8]	Cache line index
	[7:0]	Reserved
EMEM_RMW	[17]	0
	[16]	1
	[15:5]	Reserved
	[4:0]	Which data word had the error
L1DTLB	[17]	0
	[16]	1
	[15:7]	Reserved
	[6:0]	TLB entry index
L2DTLB	[17]	1
	[16]	1
	[15:12]	Reserved
	[11:7]	TLB way
	[6:0]	TLB entry index
PWC	[17]	0
	[16]	1
	[15:6]	Reserved
	[5:0]	PWC entry index
STQ	[17]	0
	[16]	1
	[15:6]	Reserved
	[5:0]	STQ entry index
LDQ	[17]	0
	[16]	1
	[15:6]	Reserved
	[5:0]	LDQ entry index
MAB	[17]	0
	[16]	1
	[15:5]	Reserved
	[4:0]	MAB entry index
SCB_STATE	[17]	0
	[16]	1
	[15:4]	Reserved
	[3:0]	SCB entry index
SCB_ADDR	[17]	0
	[16]	1
	[15:4]	Reserved
	[3:0]	SCB entry index
SCB_DATA	[17]	0
	[16]	1
	[15:4]	Reserved
	[3:0]	SCB entry index

WCB	[17] [16] [15:3] [2:0]	0 1 Reserved WCB entry index
SCB_POISON	[17] [16] [15:4] [3:0]	0 1 Reserved SCB entry index
SystemReadDataErrorLoad	[17] [16] [15:2] [1:0]	0 0 Reserved SystemReadDataError response error type
SystemReadDataErrorScb	[17] [16] [15:2] [1:0]	0 0 Reserved SystemReadDataError response error type
SystemReadDataErrorWcb	[17] [16] [15:2] [1:0]	0 0 Reserved SystemReadDataError response error type
HWA	[17] [16] [15:8] [7] [6] [5:0]	0 0 Reserved MCA was signaled Reserved assertion type
STORE_DATA_OTHER	[17] [16] [15:9] [8:7] [6] [5:0]	0 0 Reserved Error Type, 11 = STLF, 10 = SCB EMEM, 01 = Store Data Mask, 00 = SRB Store Data if 1 then [5:0] are STQ index, if 0 then [3:0] are SCB index if bit 6 is 1 then STQ index if bit 6 is 0 then [5:4] are 2'b0 and [3:0] are STQ index

MSRC000_2008 [LS Machine Check Deferred Error Status Thread 0] (MCA::LS::MCA_DESTAT_LS)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2008
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2008
 _ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2008
 _ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2008
 _ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2008

_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2008
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2008
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2008
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2008
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2008
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2008
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2008
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2008
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2008
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2008
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2008
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2008
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2008
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2008
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2008
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2008
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2008
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2008
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2008
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2008
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2008
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2008
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2008
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2008
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2008
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2008
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2008
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2008
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2008
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2008
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2008
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2008
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2008
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2008
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2008
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2008
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2008
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2008
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2008
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2008
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2008
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2008
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2008
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2008
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2008
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2008
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2008
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2008
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2008
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2008
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2008
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2008
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2008
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2008
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2008
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2008
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2008
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2008
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2008
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2008
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2008
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2008
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2008
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2008
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2008

_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2008
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2008
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2008
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2008
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2008
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2008
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2008
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2008
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2008
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2008
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2008
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2008
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2008
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2008
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2008
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2008
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2008
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2008
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2008
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2008
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2008
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2008
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2008
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2008
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2008
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2008
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2008
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2008
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2008
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2008
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2008
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2008
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2008
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2008
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2008
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2008
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2008
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2008
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2008
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2008
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2008
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2008
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2008
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2008
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2008
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2008
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2008
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2008
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2008
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2008
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2008
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2008
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2008
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2008
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2008
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2008
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2008
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2008
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2008
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2008
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2008
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2008
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2008
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2008
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2008

_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2008	
Bits	Description
63	Val. Read-write, Volatile. Reset: Cold, 0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold, 0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold, 0h.
58	AddrV. Read-write, Volatile. Reset: Cold, 0. 1=MCA::LS::MCA_DEADDR_LS contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold, 0h.
53	SyndV. Read-write, Volatile. Reset: Cold, 0. 1=This error logged information in MCA::LS::MCA_SYND_LS. If MCA::LS::MCA_SYND_LS[ErrorPriority] is the same as the priority of the error in MCA::LS::MCA_STATUS_LS, then the information in MCA::LS::MCA_SYND_LS is associated with the error in MCA::LS::MCA_DESTAT_LS.
52:45	RESERV2. Read-write. Reset: Cold, 00h.
44	Deferred. Read-write, Volatile. Reset: Cold, 0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold, 0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold, 00h. Specifies the least significant valid bit of the address contained in MCA::LS::MCA_ADDR_LS[ErrorAddr]. A value of 0 indicates that MCA::LS::MCA_ADDR_LS[56:0] contains a valid byte address. A value of 6 indicates that MCA::LS::MCA_ADDR_LS[56:6] contains a valid cache line address and that MCA::LS::MCA_ADDR_LS[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::LS::MCA_ADDR_LS[56:12] contain a valid 4KB memory page and that MCA::LS::MCA_ADDR_LS[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold, 0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold, 00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold, 0000h. Error code for this error.

MSRC000_2009 [LS Deferred Error Address Thread 0] (MCA::LS::MCA_DEADDR_LS)

Reset: Cold,0000_0000_0000_0000h.

The MCA::LS::MCA_DEADDR_LS register stores the address associated with the error in MCA::LS::MCA_DESTAT_LS. The register is only meaningful if MCA::LS::MCA_DESTAT_LS[Val]=1 and MCA::LS::MCA_DESTAT_LS[AddrV]=1. The lowest valid bit of the address is defined by MCA::LS::MCA_DESTAT_LS[AddrLsb].

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC000_2009
_ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC000_2009
_ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC000_2009
_ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC000_2009
_ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC000_2009
_ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC000_2009
_ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC000_2009
_ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC000_2009
_ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC000_2009
_ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC000_2009
_ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC000_2009
_ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC000_2009
_ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC000_2009
_ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC000_2009
_ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC000_2009
_ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC000_2009
_ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC000_2009
_ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC000_2009
_ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC000_2009
_ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC000_2009
_ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC000_2009
_ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC000_2009
_ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC000_2009
_ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC000_2009
_ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC000_2009
_ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC000_2009
_ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC000_2009
_ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC000_2009
_ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC000_2009
_ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC000_2009
_ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC000_2009
_ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC000_2009
_ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC000_2009
_ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC000_2009
_ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC000_2009
_ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC000_2009
_ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC000_2009
_ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC000_2009
_ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC000_2009
_ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC000_2009
_ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC000_2009
_ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC000_2009
_ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC000_2009
_ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC000_2009
_ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC000_2009
_ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC000_2009
_ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC000_2009
_ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC000_2009
_ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC000_2009
_ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC000_2009
_ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC000_2009
_ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC000_2009
_ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC000_2009
_ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC000_2009
_ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC000_2009
_ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC000_2009
_ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC000_2009

_ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC000_2009
_ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC000_2009
_ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC000_2009
_ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC000_2009
_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC000_2009
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC000_2009
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC000_2009
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC000_2009
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC000_2009
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC000_2009
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC000_2009
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC000_2009
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC000_2009
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC000_2009
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC000_2009
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC000_2009
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC000_2009
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC000_2009
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC000_2009
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC000_2009
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC000_2009
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC000_2009
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC000_2009
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC000_2009
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC000_2009
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC000_2009
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC000_2009
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC000_2009
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC000_2009
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC000_2009
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC000_2009
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC000_2009
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC000_2009
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC000_2009
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC000_2009
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC000_2009
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC000_2009
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC000_2009
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC000_2009
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC000_2009
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC000_2009
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC000_2009
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC000_2009
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC000_2009
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC000_2009
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC000_2009
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC000_2009
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC000_2009
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC000_2009
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC000_2009
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC000_2009
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC000_2009
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC000_2009
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC000_2009
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC000_2009
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC000_2009
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC000_2009
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC000_2009
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC000_2009
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC000_2009
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC000_2009
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC000_2009
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC000_2009
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC000_2009
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC000_2009

_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC000_2009
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC000_2009
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC000_2009
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC000_2009
_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC000_2009
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC000_2009
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC000_2009
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC000_2009
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC000_2009
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC000_2009
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC000_2009
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC000_2009
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC000_2009
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC000_2009
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC000_2009
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC000_2009
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC000_2009
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC000_2009
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC000_2009
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC000_2009
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC000_2009
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC000_2009
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC000_2009
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC000_2009
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC000_2009
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC000_2009
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC000_2009
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC000_2009
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC000_2009
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC000_2009
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC000_2009
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC000_2009
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC000_2009
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC000_2009
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC000_2009
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC000_2009
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC000_2009
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC000_2009
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC000_2009
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC000_2009
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC000_2009
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC000_2009
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC000_2009
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC000_2009
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC000_2009
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC000_2009
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC000_2009
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC000_2009
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC000_2009
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC000_2009
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC000_2009
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC000_2009
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC000_2009
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC000_2009
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC000_2009
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC000_2009
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC000_2009
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC000_2009
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC000_2009
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC000_2009
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC000_2009
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC000_2009
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC000_2009
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC000_2009
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC000_2009

_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC000_2009	
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC000_2009	
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC000_2009	
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC000_2009	
_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC000_2009	
Bits	Description
63:57	Reserved.
56:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::LS::MCA_DESTAT_LS. The lowest-order valid bit of the address is specified in MCA::LS::MCA_DESTAT_LS[AddrLsb].

MSRC001_0400 [LS Machine Check Control Mask Thread 0] (MCA::LS::MCA_CTL_MASK_LS)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_core0_thread0_inst0_n0_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core0_thread1_inst0_n1_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core1_thread0_inst0_n2_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core1_thread1_inst0_n3_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core2_thread0_inst0_n4_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core2_thread1_inst0_n5_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core3_thread0_inst0_n6_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core3_thread1_inst0_n7_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core4_thread0_inst0_n8_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core4_thread1_inst0_n9_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core5_thread0_inst0_n10_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core5_thread1_inst0_n11_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core6_thread0_inst0_n12_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core6_thread1_inst0_n13_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core7_thread0_inst0_n14_aliasMSR; MSRC001_0400
 _ccd0_lthree0_core7_thread1_inst0_n15_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core0_thread0_inst0_n160_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core0_thread1_inst0_n161_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core1_thread0_inst0_n162_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core1_thread1_inst0_n163_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core2_thread0_inst0_n164_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core2_thread1_inst0_n165_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core3_thread0_inst0_n166_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core3_thread1_inst0_n167_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core4_thread0_inst0_n168_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core4_thread1_inst0_n169_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core5_thread0_inst0_n170_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core5_thread1_inst0_n171_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core6_thread0_inst0_n172_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core6_thread1_inst0_n173_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core7_thread0_inst0_n174_aliasMSR; MSRC001_0400
 _ccd10_lthree0_core7_thread1_inst0_n175_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core0_thread0_inst0_n176_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core0_thread1_inst0_n177_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core1_thread0_inst0_n178_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core1_thread1_inst0_n179_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core2_thread0_inst0_n180_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core2_thread1_inst0_n181_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core3_thread0_inst0_n182_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core3_thread1_inst0_n183_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core4_thread0_inst0_n184_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core4_thread1_inst0_n185_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core5_thread0_inst0_n186_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core5_thread1_inst0_n187_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core6_thread0_inst0_n188_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core6_thread1_inst0_n189_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core7_thread0_inst0_n190_aliasMSR; MSRC001_0400
 _ccd11_lthree0_core7_thread1_inst0_n191_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core0_thread0_inst0_n16_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core0_thread1_inst0_n17_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core1_thread0_inst0_n18_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core1_thread1_inst0_n19_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core2_thread0_inst0_n20_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core2_thread1_inst0_n21_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core3_thread0_inst0_n22_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core3_thread1_inst0_n23_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core4_thread0_inst0_n24_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core4_thread1_inst0_n25_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core5_thread0_inst0_n26_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core5_thread1_inst0_n27_aliasMSR; MSRC001_0400
 _ccd1_lthree0_core6_thread0_inst0_n28_aliasMSR; MSRC001_0400

_ccd1_lthree0_core6_thread1_inst0_n29_aliasMSR; MSRC001_0400
_ccd1_lthree0_core7_thread0_inst0_n30_aliasMSR; MSRC001_0400
_ccd1_lthree0_core7_thread1_inst0_n31_aliasMSR; MSRC001_0400
_ccd2_lthree0_core0_thread0_inst0_n32_aliasMSR; MSRC001_0400
_ccd2_lthree0_core0_thread1_inst0_n33_aliasMSR; MSRC001_0400
_ccd2_lthree0_core1_thread0_inst0_n34_aliasMSR; MSRC001_0400
_ccd2_lthree0_core1_thread1_inst0_n35_aliasMSR; MSRC001_0400
_ccd2_lthree0_core2_thread0_inst0_n36_aliasMSR; MSRC001_0400
_ccd2_lthree0_core2_thread1_inst0_n37_aliasMSR; MSRC001_0400
_ccd2_lthree0_core3_thread0_inst0_n38_aliasMSR; MSRC001_0400
_ccd2_lthree0_core3_thread1_inst0_n39_aliasMSR; MSRC001_0400
_ccd2_lthree0_core4_thread0_inst0_n40_aliasMSR; MSRC001_0400
_ccd2_lthree0_core4_thread1_inst0_n41_aliasMSR; MSRC001_0400
_ccd2_lthree0_core5_thread0_inst0_n42_aliasMSR; MSRC001_0400
_ccd2_lthree0_core5_thread1_inst0_n43_aliasMSR; MSRC001_0400
_ccd2_lthree0_core6_thread0_inst0_n44_aliasMSR; MSRC001_0400
_ccd2_lthree0_core6_thread1_inst0_n45_aliasMSR; MSRC001_0400
_ccd2_lthree0_core7_thread0_inst0_n46_aliasMSR; MSRC001_0400
_ccd2_lthree0_core7_thread1_inst0_n47_aliasMSR; MSRC001_0400
_ccd3_lthree0_core0_thread0_inst0_n48_aliasMSR; MSRC001_0400
_ccd3_lthree0_core0_thread1_inst0_n49_aliasMSR; MSRC001_0400
_ccd3_lthree0_core1_thread0_inst0_n50_aliasMSR; MSRC001_0400
_ccd3_lthree0_core1_thread1_inst0_n51_aliasMSR; MSRC001_0400
_ccd3_lthree0_core2_thread0_inst0_n52_aliasMSR; MSRC001_0400
_ccd3_lthree0_core2_thread1_inst0_n53_aliasMSR; MSRC001_0400
_ccd3_lthree0_core3_thread0_inst0_n54_aliasMSR; MSRC001_0400
_ccd3_lthree0_core3_thread1_inst0_n55_aliasMSR; MSRC001_0400
_ccd3_lthree0_core4_thread0_inst0_n56_aliasMSR; MSRC001_0400
_ccd3_lthree0_core4_thread1_inst0_n57_aliasMSR; MSRC001_0400
_ccd3_lthree0_core5_thread0_inst0_n58_aliasMSR; MSRC001_0400
_ccd3_lthree0_core5_thread1_inst0_n59_aliasMSR; MSRC001_0400
_ccd3_lthree0_core6_thread0_inst0_n60_aliasMSR; MSRC001_0400
_ccd3_lthree0_core6_thread1_inst0_n61_aliasMSR; MSRC001_0400
_ccd3_lthree0_core7_thread0_inst0_n62_aliasMSR; MSRC001_0400
_ccd3_lthree0_core7_thread1_inst0_n63_aliasMSR; MSRC001_0400
_ccd4_lthree0_core0_thread0_inst0_n64_aliasMSR; MSRC001_0400
_ccd4_lthree0_core0_thread1_inst0_n65_aliasMSR; MSRC001_0400
_ccd4_lthree0_core1_thread0_inst0_n66_aliasMSR; MSRC001_0400
_ccd4_lthree0_core1_thread1_inst0_n67_aliasMSR; MSRC001_0400
_ccd4_lthree0_core2_thread0_inst0_n68_aliasMSR; MSRC001_0400
_ccd4_lthree0_core2_thread1_inst0_n69_aliasMSR; MSRC001_0400
_ccd4_lthree0_core3_thread0_inst0_n70_aliasMSR; MSRC001_0400
_ccd4_lthree0_core3_thread1_inst0_n71_aliasMSR; MSRC001_0400
_ccd4_lthree0_core4_thread0_inst0_n72_aliasMSR; MSRC001_0400
_ccd4_lthree0_core4_thread1_inst0_n73_aliasMSR; MSRC001_0400
_ccd4_lthree0_core5_thread0_inst0_n74_aliasMSR; MSRC001_0400
_ccd4_lthree0_core5_thread1_inst0_n75_aliasMSR; MSRC001_0400
_ccd4_lthree0_core6_thread0_inst0_n76_aliasMSR; MSRC001_0400
_ccd4_lthree0_core6_thread1_inst0_n77_aliasMSR; MSRC001_0400
_ccd4_lthree0_core7_thread0_inst0_n78_aliasMSR; MSRC001_0400
_ccd4_lthree0_core7_thread1_inst0_n79_aliasMSR; MSRC001_0400
_ccd5_lthree0_core0_thread0_inst0_n80_aliasMSR; MSRC001_0400
_ccd5_lthree0_core0_thread1_inst0_n81_aliasMSR; MSRC001_0400
_ccd5_lthree0_core1_thread0_inst0_n82_aliasMSR; MSRC001_0400
_ccd5_lthree0_core1_thread1_inst0_n83_aliasMSR; MSRC001_0400
_ccd5_lthree0_core2_thread0_inst0_n84_aliasMSR; MSRC001_0400
_ccd5_lthree0_core2_thread1_inst0_n85_aliasMSR; MSRC001_0400
_ccd5_lthree0_core3_thread0_inst0_n86_aliasMSR; MSRC001_0400
_ccd5_lthree0_core3_thread1_inst0_n87_aliasMSR; MSRC001_0400
_ccd5_lthree0_core4_thread0_inst0_n88_aliasMSR; MSRC001_0400
_ccd5_lthree0_core4_thread1_inst0_n89_aliasMSR; MSRC001_0400
_ccd5_lthree0_core5_thread0_inst0_n90_aliasMSR; MSRC001_0400
_ccd5_lthree0_core5_thread1_inst0_n91_aliasMSR; MSRC001_0400
_ccd5_lthree0_core6_thread0_inst0_n92_aliasMSR; MSRC001_0400
_ccd5_lthree0_core6_thread1_inst0_n93_aliasMSR; MSRC001_0400

_ccd5_lthree0_core7_thread0_inst0_n94_aliasMSR; MSRC001_0400
_ccd5_lthree0_core7_thread1_inst0_n95_aliasMSR; MSRC001_0400
_ccd6_lthree0_core0_thread0_inst0_n96_aliasMSR; MSRC001_0400
_ccd6_lthree0_core0_thread1_inst0_n97_aliasMSR; MSRC001_0400
_ccd6_lthree0_core1_thread0_inst0_n98_aliasMSR; MSRC001_0400
_ccd6_lthree0_core1_thread1_inst0_n99_aliasMSR; MSRC001_0400
_ccd6_lthree0_core2_thread0_inst0_n100_aliasMSR; MSRC001_0400
_ccd6_lthree0_core2_thread1_inst0_n101_aliasMSR; MSRC001_0400
_ccd6_lthree0_core3_thread0_inst0_n102_aliasMSR; MSRC001_0400
_ccd6_lthree0_core3_thread1_inst0_n103_aliasMSR; MSRC001_0400
_ccd6_lthree0_core4_thread0_inst0_n104_aliasMSR; MSRC001_0400
_ccd6_lthree0_core4_thread1_inst0_n105_aliasMSR; MSRC001_0400
_ccd6_lthree0_core5_thread0_inst0_n106_aliasMSR; MSRC001_0400
_ccd6_lthree0_core5_thread1_inst0_n107_aliasMSR; MSRC001_0400
_ccd6_lthree0_core6_thread0_inst0_n108_aliasMSR; MSRC001_0400
_ccd6_lthree0_core6_thread1_inst0_n109_aliasMSR; MSRC001_0400
_ccd6_lthree0_core7_thread0_inst0_n110_aliasMSR; MSRC001_0400
_ccd6_lthree0_core7_thread1_inst0_n111_aliasMSR; MSRC001_0400
_ccd7_lthree0_core0_thread0_inst0_n112_aliasMSR; MSRC001_0400
_ccd7_lthree0_core0_thread1_inst0_n113_aliasMSR; MSRC001_0400
_ccd7_lthree0_core1_thread0_inst0_n114_aliasMSR; MSRC001_0400
_ccd7_lthree0_core1_thread1_inst0_n115_aliasMSR; MSRC001_0400
_ccd7_lthree0_core2_thread0_inst0_n116_aliasMSR; MSRC001_0400
_ccd7_lthree0_core2_thread1_inst0_n117_aliasMSR; MSRC001_0400
_ccd7_lthree0_core3_thread0_inst0_n118_aliasMSR; MSRC001_0400
_ccd7_lthree0_core3_thread1_inst0_n119_aliasMSR; MSRC001_0400
_ccd7_lthree0_core4_thread0_inst0_n120_aliasMSR; MSRC001_0400
_ccd7_lthree0_core4_thread1_inst0_n121_aliasMSR; MSRC001_0400
_ccd7_lthree0_core5_thread0_inst0_n122_aliasMSR; MSRC001_0400
_ccd7_lthree0_core5_thread1_inst0_n123_aliasMSR; MSRC001_0400
_ccd7_lthree0_core6_thread0_inst0_n124_aliasMSR; MSRC001_0400
_ccd7_lthree0_core6_thread1_inst0_n125_aliasMSR; MSRC001_0400
_ccd7_lthree0_core7_thread0_inst0_n126_aliasMSR; MSRC001_0400
_ccd7_lthree0_core7_thread1_inst0_n127_aliasMSR; MSRC001_0400
_ccd8_lthree0_core0_thread0_inst0_n128_aliasMSR; MSRC001_0400
_ccd8_lthree0_core0_thread1_inst0_n129_aliasMSR; MSRC001_0400
_ccd8_lthree0_core1_thread0_inst0_n130_aliasMSR; MSRC001_0400
_ccd8_lthree0_core1_thread1_inst0_n131_aliasMSR; MSRC001_0400
_ccd8_lthree0_core2_thread0_inst0_n132_aliasMSR; MSRC001_0400
_ccd8_lthree0_core2_thread1_inst0_n133_aliasMSR; MSRC001_0400
_ccd8_lthree0_core3_thread0_inst0_n134_aliasMSR; MSRC001_0400
_ccd8_lthree0_core3_thread1_inst0_n135_aliasMSR; MSRC001_0400
_ccd8_lthree0_core4_thread0_inst0_n136_aliasMSR; MSRC001_0400
_ccd8_lthree0_core4_thread1_inst0_n137_aliasMSR; MSRC001_0400
_ccd8_lthree0_core5_thread0_inst0_n138_aliasMSR; MSRC001_0400
_ccd8_lthree0_core5_thread1_inst0_n139_aliasMSR; MSRC001_0400
_ccd8_lthree0_core6_thread0_inst0_n140_aliasMSR; MSRC001_0400
_ccd8_lthree0_core6_thread1_inst0_n141_aliasMSR; MSRC001_0400
_ccd8_lthree0_core7_thread0_inst0_n142_aliasMSR; MSRC001_0400
_ccd8_lthree0_core7_thread1_inst0_n143_aliasMSR; MSRC001_0400
_ccd9_lthree0_core0_thread0_inst0_n144_aliasMSR; MSRC001_0400
_ccd9_lthree0_core0_thread1_inst0_n145_aliasMSR; MSRC001_0400
_ccd9_lthree0_core1_thread0_inst0_n146_aliasMSR; MSRC001_0400
_ccd9_lthree0_core1_thread1_inst0_n147_aliasMSR; MSRC001_0400
_ccd9_lthree0_core2_thread0_inst0_n148_aliasMSR; MSRC001_0400
_ccd9_lthree0_core2_thread1_inst0_n149_aliasMSR; MSRC001_0400
_ccd9_lthree0_core3_thread0_inst0_n150_aliasMSR; MSRC001_0400
_ccd9_lthree0_core3_thread1_inst0_n151_aliasMSR; MSRC001_0400
_ccd9_lthree0_core4_thread0_inst0_n152_aliasMSR; MSRC001_0400
_ccd9_lthree0_core4_thread1_inst0_n153_aliasMSR; MSRC001_0400
_ccd9_lthree0_core5_thread0_inst0_n154_aliasMSR; MSRC001_0400
_ccd9_lthree0_core5_thread1_inst0_n155_aliasMSR; MSRC001_0400
_ccd9_lthree0_core6_thread0_inst0_n156_aliasMSR; MSRC001_0400
_ccd9_lthree0_core6_thread1_inst0_n157_aliasMSR; MSRC001_0400
_ccd9_lthree0_core7_thread0_inst0_n158_aliasMSR; MSRC001_0400

_ccd9_lthree0_core7_thread1_inst0_n159_aliasMSR; MSRC001_0400	
Bits	Description
63:24	Reserved.
23	STORE_DATA_OTHER. Read-write. Reset: 0. A parity error was detected in an STLF, SCB EMEM entry, store data mask or SRB store data by any access.
22	HWA. Read-write. Reset: 0. A hardware assertion error was reported
21	SystemReadDataErrorWcb. Read-write. Reset: 0. Init: BIOS,1. A SystemReadDataError error was reported on read data returned from L2 for a WCB store
20	SystemReadDataErrorScb. Read-write. Reset: 0. Init: BIOS,1. A SystemReadDataError error was reported on read data returned from L2 for an SCB store
19	SystemReadDataErrorLoad. Read-write. Reset: 0. Init: BIOS,1. A SystemReadDataError error was reported on read data returned from L2 for a load
18	SCB_POISON. Read-write. Reset: 0. A poisoned line was detected in an SCB entry by any access
17	WCB. Read-write. Reset: 0. A parity error was detected in a WCB entry by any access
16	SCB_DATA. Read-write. Reset: 0. A parity error was detected in an SCB entry data field by any access
15	SCB_ADDR. Read-write. Reset: 0. A parity error was detected in an SCB entry address field by any access
14	SCB_STATE. Read-write. Reset: 0. A parity error was detected in an SCB entry state field by any access
13	MAB. Read-write. Reset: 0. A parity error was detected in a MAB entry by any access
12	LDQ. Read-write. Reset: 0. A parity error was detected in an LDQ entry by any access
11	STQ. Read-write. Reset: 0. A parity error was detected in an STQ entry by any access
10	PWC. Read-write. Reset: 0. A parity error was detected in a PWC entry by any access
9	L2DTLB. Read-write. Reset: 0. A parity error was detected in an L2 TLB entry by any access
8	L1DTLB. Read-write. Reset: 0. A parity error was detected in an L1 TLB entry by any access. This error only logs a valid address down through bit [12], in spite of the AddrLsbCnt value of 0.
7	EMEM_RMW. Read-write. Reset: 0. An ECC error was detected on an EMEM read-modify-write by a store
6	EMEM_LOAD. Read-write. Reset: 0. An ECC error was detected on an EMEM read by a load.
5	DC_TAG_STORE. Read-write. Reset: 0. An ECC error or poison bit mismatch was detected on a tag read by a store.
4	DC_TAG_LOAD. Read-write. Reset: 0. An ECC error or poison bit mismatch was detected on a tag read by a load.
3	DC_TAG_VICTIM. Read-write. Reset: 0. An ECC error or poison bit mismatch was detected on a tag read by a probe or victimization.
2	DC_DATA_RMW. Read-write. Reset: 0. An ECC error was detected on a data cache read-modify-write by a store
1	DC_DATA_LOAD. Read-write. Reset: 0. An ECC error or L2 poison was detected on a data cache read by a load.
0	DC_DATA_VICTIM. Read-write. Reset: 0. An ECC error was detected on a data cache read by a probe or victimization

3.2.5.2 IF

MSR0000_0404...MSRC000_2010 [IF Machine Check Control Thread 0] (MCA::IF::MCA_CTL_IF)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::IF::MCA_CTL_IF register must be enabled by the corresponding enable bit in Core::X86::Msrr:[MCG_CTL](#). Does not affect error detection, correction, or logging.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2010
_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2010
_ccd0_lthree0_core0_thread1_inst1_n1_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2010
_ccd0_lthree0_core1_thread0_inst1_n2_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2010
_ccd0_lthree0_core1_thread1_inst1_n3_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2010
_ccd0_lthree0_core2_thread0_inst1_n4_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2010
_ccd0_lthree0_core2_thread1_inst1_n5_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2010
_ccd0_lthree0_core3_thread0_inst1_n6_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2010
_ccd0_lthree0_core3_thread1_inst1_n7_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2010
_ccd0_lthree0_core4_thread0_inst1_n8_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2010
_ccd0_lthree0_core4_thread1_inst1_n9_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2010
_ccd0_lthree0_core5_thread0_inst1_n10_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2010
_ccd0_lthree0_core5_thread1_inst1_n11_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2010
_ccd0_lthree0_core6_thread0_inst1_n12_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2010
_ccd0_lthree0_core6_thread1_inst1_n13_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2010
_ccd0_lthree0_core7_thread0_inst1_n14_aliasMSRLEGACY; MSR0000_0404
_ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2010
_ccd0_lthree0_core7_thread1_inst1_n15_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2010
_ccd10_lthree0_core0_thread0_inst1_n160_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2010
_ccd10_lthree0_core0_thread1_inst1_n161_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2010
_ccd10_lthree0_core1_thread0_inst1_n162_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2010
_ccd10_lthree0_core1_thread1_inst1_n163_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2010
_ccd10_lthree0_core2_thread0_inst1_n164_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2010
_ccd10_lthree0_core2_thread1_inst1_n165_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2010
_ccd10_lthree0_core3_thread0_inst1_n166_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2010
_ccd10_lthree0_core3_thread1_inst1_n167_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2010
_ccd10_lthree0_core4_thread0_inst1_n168_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2010
_ccd10_lthree0_core4_thread1_inst1_n169_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2010
_ccd10_lthree0_core5_thread0_inst1_n170_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2010
_ccd10_lthree0_core5_thread1_inst1_n171_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2010
_ccd10_lthree0_core6_thread0_inst1_n172_aliasMSRLEGACY; MSR0000_0404

_ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2010
_ccd10_lthree0_core6_thread1_inst1_n173_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2010
_ccd10_lthree0_core7_thread0_inst1_n174_aliasMSRLEGACY; MSR0000_0404
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2010
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2010
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2010
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2010
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2010
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2010
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2010
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2010
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2010
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2010
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2010
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2010
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2010
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2010
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2010
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2010
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSRLEGACY; MSR0000_0404
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2010
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2010
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2010
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2010
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2010
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2010
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2010
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2010
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2010
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2010
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2010
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2010
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2010
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2010
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2010

_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2010
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSRLEGACY; MSR0000_0404
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2010
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2010
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2010
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2010
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2010
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2010
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2010
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2010
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2010
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2010
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2010
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2010
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2010
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2010
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2010
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2010
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSRLEGACY; MSR0000_0404
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2010
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2010
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2010
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2010
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2010
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2010
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2010
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2010
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2010
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2010
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2010
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2010
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2010
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2010
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2010
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSRLEGACY; MSR0000_0404

_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2010
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSRLEGACY; MSR0000_0404
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2010
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2010
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2010
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2010
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2010
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2010
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2010
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2010
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2010
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2010
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2010
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2010
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2010
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2010
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2010
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2010
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSRLEGACY; MSR0000_0404
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2010
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2010
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2010
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2010
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2010
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2010
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2010
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2010
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2010
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2010
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2010
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2010
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2010
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2010
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2010
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2010

_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSRLEGACY; MSR0000_0404
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2010
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2010
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2010
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2010
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2010
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2010
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2010
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2010
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2010
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2010
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2010
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2010
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2010
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2010
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2010
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2010
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSRLEGACY; MSR0000_0404
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2010
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2010
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2010
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2010
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2010
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2010
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2010
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2010
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2010
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2010
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2010
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2010
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2010
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2010
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2010
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSRLEGACY; MSR0000_0404
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2010
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSRLEGACY; MSR0000_0404

_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2010
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2010
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2010
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2010
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2010
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2010
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2010
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2010
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2010
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2010
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2010
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2010
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2010
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2010
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2010
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2010
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSRLEGACY; MSR0000_0404
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2010
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2010
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2010
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2010
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2010
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2010
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2010
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2010
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2010
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2010
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2010
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2010
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2010
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2010
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2010
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2010
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSRLEGACY; MSR0000_0404
_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2010

_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSRLEGACY; MSR0000_0404	
Bits	Description
63:19	Reserved.
18	CtMceError . Read-write. Reset: 0. CT MCE.
17	BsrParity . Read-write. Reset: 0. BSR Parity Error.
16	L2TlbMultiHit . Read-write. Reset: 0. L2-TLB Multi-Hit
15	L1TlbMultiHit . Read-write. Reset: 0. L1-TLB Multi-Hit.
14	HwAssert . Read-write. Reset: 0. Hardware Assertion Error.
13	SystemReadDataError . Read-write. Reset: 0. L2 Cache Error Response.
12	L2RespPoison . Read-write. Reset: 0. L2 Cache Response Poison Error. Error is the result of consuming poison data.
11	L2BtbMultiHit . Read-write. Reset: 0. BP L2-BTB Multi-Hit Error.
10	L1BtbMultiHit . Read-write. Reset: 0. BP L1-BTB Multi-Hit Error.
9	BpqSnpParT1 . Read-write. Reset: 0. BPQ Thread 1 Snoop Parity Error.
8	RSVD_8 . Read-write. Reset: 0. Reserved.
7	L2ItlbParity . Read-write. Reset: 0. L2-TLB Parity Error.
6	L1ItlbParity . Read-write. Reset: 0. L1-TLB Parity Error.
5	RSVD_5 . Read-write. Reset: 0. Reserved.
4	DqParity . Read-write. Reset: 0. PRQ Parity Error.
3	DataParity . Read-write. Reset: 0. IC Data Array Parity Error.
2	TagParity . Read-write. Reset: 0. IC Full Tag Parity Error.
1	TagMultiHit . Read-write. Reset: 0. IC Microtag or Full Tag Multi-hit Error.
0	OcUtagParity . Read-write. Reset: 0. Op Cache Microtag Parity Error. Parity errors on PA and other relevant uTag fields are reported, independent of any utag probing. The parity error way and index are logged.

MSR0000_0405...MSRC000_2011 [IF Machine Check Status Thread 1] (MCA::IF::MCA_STATUS_IF)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core0_thread0_inst1_n0_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSRLEGACY; MSR0000_0405
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2011
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2011
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSRLEGACY; MSR0000_0405
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2011

_ccd10_lthree0_core7_thread0_inst1_n174_aliasMSRLEGACY; MSR0000_0405
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2011
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2011
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2011
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2011
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2011
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2011
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2011
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2011
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2011
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2011
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2011
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2011
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2011
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2011
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2011
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2011
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSRLEGACY; MSR0000_0405
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2011
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2011
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2011
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2011
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2011
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2011
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2011
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2011
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2011
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2011
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2011
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2011
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2011
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2011
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2011
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSRLEGACY; MSR0000_0405
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2011
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSRLEGACY; MSR0000_0405

_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2011
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2011
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2011
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2011
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2011
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2011
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2011
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2011
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2011
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2011
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2011
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2011
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2011
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2011
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2011
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2011
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSRLEGACY; MSR0000_0405
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2011
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2011
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2011
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2011
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2011
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2011
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2011
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2011
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2011
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2011
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2011
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2011
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2011
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2011
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2011
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2011
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSRLEGACY; MSR0000_0405
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2011

_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2011
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2011
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2011
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2011
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2011
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2011
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2011
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2011
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2011
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2011
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2011
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2011
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2011
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2011
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2011
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSRLEGACY; MSR0000_0405
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2011
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2011
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2011
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2011
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2011
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2011
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2011
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2011
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2011
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2011
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2011
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2011
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2011
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2011
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2011
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2011
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSRLEGACY; MSR0000_0405
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2011
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSRLEGACY; MSR0000_0405

_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2011
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2011
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2011
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2011
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2011
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2011
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2011
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2011
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2011
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2011
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2011
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2011
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2011
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2011
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2011
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSRLEGACY; MSR0000_0405
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2011
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2011
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2011
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2011
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2011
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2011
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2011
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2011
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2011
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2011
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2011
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2011
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2011
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2011
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2011
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2011
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSRLEGACY; MSR0000_0405
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2011
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSRLEGACY; MSR0000_0405
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2011

_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSRLEGACY; MSR0000_0405	
ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSRLEGACY; MSR0000_0405	
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2011	
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSRLEGACY; MSR0000_0405	
_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2011	
_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSRLEGACY; MSR0000_0405	
Bits	Description

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::IF::MCA_CTL_IF. This bit is a copy of bit in MCA::IF::MCA_CTL_IF for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::IF::MCA_MISC0_IF. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::IF::MCA_ADDR_IF contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::IF::MCA_STATUS_IF[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::IF::MCA_SYND_IF. If MCA::IF::MCA_SYND_IF[ErrorPriority] is the same as the priority of the error in MCA::IF::MCA_STATUS_IF, then the information in MCA::IF::MCA_SYND_IF is associated with the error in MCA::IF::MCA_STATUS_IF. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::IF::MCA_STATUS_IF[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::IF::MCA_ADDR_IF[ErrorAddr]. A value of 0 indicates that MCA::IF::MCA_ADDR_IF[56:0] contains a valid byte address. A value of 6 indicates that MCA::IF::MCA_ADDR_IF[56:6] contains a valid cache line address and that MCA::IF::MCA_ADDR_IF[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::IF::MCA_ADDR_IF[56:12] contain a valid 4KB memory page and that MCA::IF::MCA_ADDR_IF[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::IF::MCA_CTL_IF enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 61: MCA_STATUS_IF

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
OcUtagParity	0x0	1	1	1	0	0	0
TagMultiHit	0x1	0	0	0	0	0	1
TagParity	0x2	0	0	0	0	0	1
DataParity	0x3	0	0	0	0	0	1
DqParity	0x4	1	1	1	0	0	0
RSVD_5	0x5	1	1	1	0	0	0
L1ItlbParity	0x6	0	0	0	0	0	1
L2ItlbParity	0x7	0	0	0	0	0	1
RSVD_8	0x8	1	1	1	0	0	0
BpqSnpParT1	0x9	0	0	0	0	0	0
L1BtbMultiHit	0xa	0	0	0	0	0	0
L2BtbMultiHit	0xb	0	0	0	0	0	0

L2RespPois on	0xc	1	0	1	0	1	1
SystemRead DataError	0xd	1	0	1	0	0	1
HwAssert	0xe	1	1	1	0	0	0
L1TlbMulti Hit	0xf	0	0	0	0	0	1
L2TlbMulti Hit	0x10	0	0	0	0	0	1
BsrParity	0x11	1	1	1	0	0	0
CtMceError	0x12	1	1	1	0	0	0

MSR0000_0406...MSRC000_2012 [IF Machine Check Address Thread 0] (MCA::IF::MCA_ADDR_IF)

Reset: Cold,0000_0000_0000_0000h.

MCA::IF::MCA_ADDR_IF stores an address and other information associated with the error in MCA::IF::MCA_STATUS_IF. The register is only meaningful if MCA::IF::MCA_STATUS_IF[Val]=1 and MCA::IF::MCA_STATUS_IF[AddrV]=1.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core0_thread0_inst1_n0_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSRLEGACY; MSR0000_0406
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2012
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSRLEGACY; MSR0000_0406
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2012
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSRLEGACY; MSR0000_0406

_ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2012
_ccd10_lthree0_core6_thread1_inst1_n173_aliasMSRLEGACY; MSR0000_0406
_ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2012
_ccd10_lthree0_core7_thread0_inst1_n174_aliasMSRLEGACY; MSR0000_0406
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2012
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2012
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2012
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2012
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2012
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2012
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2012
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2012
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2012
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2012
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2012
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2012
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2012
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2012
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2012
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2012
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSRLEGACY; MSR0000_0406
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2012
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2012
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2012
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2012
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2012
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2012
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2012
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2012
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2012
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2012
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2012
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2012
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2012
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2012
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2012

_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2012
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSRLEGACY; MSR0000_0406
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2012
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2012
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2012
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2012
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2012
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2012
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2012
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2012
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2012
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2012
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2012
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2012
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2012
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2012
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2012
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2012
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSRLEGACY; MSR0000_0406
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2012
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2012
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2012
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2012
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2012
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2012
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2012
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2012
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2012
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2012
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2012
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2012
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2012
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2012
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2012
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSRLEGACY; MSR0000_0406

_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2012
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSRLEGACY; MSR0000_0406
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2012
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2012
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2012
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2012
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2012
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2012
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2012
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2012
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2012
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2012
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2012
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2012
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2012
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2012
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2012
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2012
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSRLEGACY; MSR0000_0406
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2012
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2012
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2012
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2012
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2012
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2012
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2012
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2012
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2012
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2012
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2012
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2012
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2012
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2012
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2012
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2012

_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSRLEGACY; MSR0000_0406
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2012
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2012
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2012
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2012
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2012
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2012
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2012
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2012
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2012
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2012
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2012
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2012
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2012
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2012
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2012
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2012
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSRLEGACY; MSR0000_0406
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2012
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2012
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2012
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2012
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2012
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2012
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2012
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2012
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2012
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2012
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2012
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2012
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2012
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2012
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2012
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSRLEGACY; MSR0000_0406
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2012
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSRLEGACY; MSR0000_0406

_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2012
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2012
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2012
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2012
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2012
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2012
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2012
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2012
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2012
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2012
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2012
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2012
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2012
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2012
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2012
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2012
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSRLEGACY; MSR0000_0406
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2012
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2012
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2012
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2012
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2012
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2012
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2012
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2012
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2012
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2012
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2012
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2012
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2012
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2012
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2012
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2012
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSRLEGACY; MSR0000_0406
_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2012

_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSRLEGACY; MSR0000_0406	
Bits	Description
63:57	Reserved.
56:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 000_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::IF::MCA_STATUS_IF. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 62: MCA_ADDR_IF

Error Type	Bits	Description
OcUtagParity	[56:0]	Reserved
TagMultiHit	[56:52] [51:0]	Reserved Physical Address
TagParity	[56:52] [51:0]	Reserved Physical Address
DataParity	[56:0]	VA
DqParity	[56:0]	Reserved
RSVD_5	[56:0]	Reserved
L1ItlbParity	[56:0]	VA
L2ItlbParity	[56:0]	VA
RSVD_8	[56:0]	Reserved
BpqSnpParT1	[56:0]	Reserved
L1BtbMultiHit	[56:0]	Reserved
L2BtbMultiHit	[56:0]	Reserved
L2RespPoison	[56:52] [51:0]	Reserved Physical Address
SystemReadDataError	[56:52] [51:0]	Reserved Physical Address
HwAssert	[56:0]	Reserved
L1TlbMultiHit	[56:0]	VA
L2TlbMultiHit	[56:0]	VA
BsrParity	[56:0]	Reserved
CtMceError	[56:0]	Reserved

MSR0000_0407...MSRC000_2013 [IF Machine Check Miscellaneous 0 Thread 0] (MCA::IF::MCA_MISC0_IF)

Log miscellaneous information associated with errors.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core0_thread0_inst1_n0_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSRLEGACY; MSR0000_0407
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2013
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSRLEGACY; MSR0000_0407
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2013
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSRLEGACY; MSR0000_0407

_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2013
_ccd10_lthree0_core7_thread1_inst1_n175_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2013
_ccd11_lthree0_core0_thread0_inst1_n176_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2013
_ccd11_lthree0_core0_thread1_inst1_n177_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2013
_ccd11_lthree0_core1_thread0_inst1_n178_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2013
_ccd11_lthree0_core1_thread1_inst1_n179_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2013
_ccd11_lthree0_core2_thread0_inst1_n180_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2013
_ccd11_lthree0_core2_thread1_inst1_n181_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2013
_ccd11_lthree0_core3_thread0_inst1_n182_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2013
_ccd11_lthree0_core3_thread1_inst1_n183_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2013
_ccd11_lthree0_core4_thread0_inst1_n184_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2013
_ccd11_lthree0_core4_thread1_inst1_n185_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2013
_ccd11_lthree0_core5_thread0_inst1_n186_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2013
_ccd11_lthree0_core5_thread1_inst1_n187_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2013
_ccd11_lthree0_core6_thread0_inst1_n188_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2013
_ccd11_lthree0_core6_thread1_inst1_n189_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2013
_ccd11_lthree0_core7_thread0_inst1_n190_aliasMSRLEGACY; MSR0000_0407
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2013
_ccd11_lthree0_core7_thread1_inst1_n191_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2013
_ccd1_lthree0_core0_thread0_inst1_n16_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2013
_ccd1_lthree0_core0_thread1_inst1_n17_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2013
_ccd1_lthree0_core1_thread0_inst1_n18_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2013
_ccd1_lthree0_core1_thread1_inst1_n19_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2013
_ccd1_lthree0_core2_thread0_inst1_n20_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2013
_ccd1_lthree0_core2_thread1_inst1_n21_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2013
_ccd1_lthree0_core3_thread0_inst1_n22_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2013
_ccd1_lthree0_core3_thread1_inst1_n23_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2013
_ccd1_lthree0_core4_thread0_inst1_n24_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2013
_ccd1_lthree0_core4_thread1_inst1_n25_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2013
_ccd1_lthree0_core5_thread0_inst1_n26_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2013
_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2013
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2013
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2013
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSRLEGACY; MSR0000_0407
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2013

_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2013
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2013
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2013
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2013
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2013
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2013
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2013
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2013
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2013
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2013
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2013
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2013
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2013
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2013
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2013
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSRLEGACY; MSR0000_0407
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2013
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2013
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2013
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2013
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2013
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2013
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2013
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2013
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2013
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2013
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2013
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2013
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2013
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2013
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2013
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2013
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSRLEGACY; MSR0000_0407
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2013
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSRLEGACY; MSR0000_0407

_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2013
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2013
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2013
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2013
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2013
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2013
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2013
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2013
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2013
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2013
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2013
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2013
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2013
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2013
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2013
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSRLEGACY; MSR0000_0407
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2013
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2013
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2013
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2013
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2013
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2013
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2013
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2013
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2013
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2013
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2013
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2013
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2013
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2013
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2013
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2013
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSRLEGACY; MSR0000_0407
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2013
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2013

_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2013
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2013
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2013
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2013
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2013
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2013
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2013
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2013
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2013
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2013
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2013
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2013
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2013
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2013
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSRLEGACY; MSR0000_0407
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2013
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2013
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2013
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2013
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2013
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2013
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2013
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2013
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2013
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2013
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2013
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2013
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2013
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2013
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2013
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2013
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSRLEGACY; MSR0000_0407
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2013
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSRLEGACY; MSR0000_0407
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2013
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSRLEGACY; MSR0000_0407

<u>_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2013</u>	
<u>_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSRLEGACY; MSR0000_0407</u>	
<u>_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2013</u>	
<u>_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSRLEGACY; MSR0000_0407</u>	
Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP . Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked . Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP . Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::IF::MCA_MISC0_IF[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset . Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::IF::MCA_MISC0_IF[Locked]) ? Read-write : Read-only.
51	CntEn . Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::IF::MCA_MISC0_IF[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType . Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrlw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::IF::MCA_MISC0_IF[Locked]) ? Read-write : Read-only.
48	Ovrlw . Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::IF::MCA_MISC0_IF[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt . Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::IF::MCA_MISC0_IF[Locked]) ? Read-write : Read-only.
31:24	BlkPtr . Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2014 [IF Machine Check Configuration Thread 1] (MCA::IF::MCA_CONFIG_IF)

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2014
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2014
 _ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2014
 _ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2014
 _ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2014

_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2014
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2014
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2014
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2014
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2014
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2014
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2014
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2014
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2014
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2014
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2014
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2014
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2014
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2014
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2014
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2014
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2014
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2014
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2014
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2014
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2014
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2014
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2014
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2014
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2014
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2014
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2014
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2014
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2014
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2014
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2014
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2014
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2014
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2014
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2014
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2014
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2014
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2014
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2014
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2014
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2014
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2014
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2014
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2014
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2014
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2014
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2014
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2014
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2014
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2014
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2014
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2014
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2014
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2014
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2014
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2014
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2014
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2014
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2014
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2014
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2014
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2014
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2014
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2014
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2014

_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2014
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2014
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2014
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2014
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2014
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2014
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2014
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2014
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2014
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2014
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2014
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2014
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2014
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2014
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2014
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2014
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2014
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2014
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2014
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2014
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2014
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2014
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2014
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2014
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2014
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2014
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2014
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2014
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2014
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2014
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2014
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2014
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2014
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2014
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2014
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2014
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2014
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2014
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2014
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2014
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2014
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2014
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2014
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2014
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2014
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2014
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2014
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2014
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2014
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2014
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2014
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2014
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2014
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2014
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2014
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2014
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2014
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2014
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2014
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2014
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2014
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2014
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2014
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2014
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2014

_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2014	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::IF::MCA_STATUS_IF, MCA::IF::MCA_ADDR_IF, MCA::IF::MCA_MISC0_IF, and MCA::IF::MCA_SYND_IF.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::IF::MCA_CONFIG_IF[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::IF::MCA_CONFIG_IF[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::IF::MCA_CONFIG_IF[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::IF::MCA_MISC0_IF[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::IF::MCA_STATUS_IF[TCC] is present.

MSRC000_2015 [IF IP Identification Thread 0] (MCA::IF::MCA_IPID_IF)

Reset: 0001_00B0_0000_0000h.

The MCA::IF::MCA_IPID_IF register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2015
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2015
 _ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2015
 _ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2015
 _ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2015

_ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2015
_ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2015
_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2015
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2015
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2015
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2015
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2015
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2015
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2015
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2015
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2015
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2015
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2015
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2015
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2015
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2015
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2015
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2015
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2015
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2015
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2015
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2015
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2015
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2015
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2015
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2015
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2015
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2015
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2015
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2015
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2015
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2015
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2015
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2015
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2015
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2015
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2015
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2015
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2015
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2015
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2015
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2015
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2015
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2015
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2015
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2015
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2015
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2015
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2015
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2015
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2015
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2015
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2015
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2015
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2015
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2015
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2015
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2015
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2015
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2015
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2015
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2015
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2015
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2015
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2015

_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2015
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2015
_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2015
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2015
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2015
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2015
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2015
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2015
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2015
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2015
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2015
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2015
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2015
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2015
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2015
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2015
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2015
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2015
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2015
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2015
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2015
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2015
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2015
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2015
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2015
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2015
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2015
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2015
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2015
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2015
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2015
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2015
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2015
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2015
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2015
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2015
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2015
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2015
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2015
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2015
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2015
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2015
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2015
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2015
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2015
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2015
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2015
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2015
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2015
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2015
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2015
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2015
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2015
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2015
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2015
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2015
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2015
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2015
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2015
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2015
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2015
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2015
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2015
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2015
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2015

_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2015	
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2015	
_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2015	
Bits	Description
63:48	McaType . Read-only. Reset: 0001h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR: 2000_AA00h Init: _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR: 2000_AB00h Init: _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR: 2002_AA00h Init: _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR: 2002_AB00h Init: _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR: 2004_AA00h Init: _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR: 2004_AB00h Init: _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR: 2006_AA00h Init: _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR: 2006_AB00h Init: _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR: 2008_AA00h Init: _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR: 2008_AB00h Init: _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR: 200A_AA00h Init: _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR: 200A_AB00h Init: _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR: 200C_AA00h Init: _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR: 200C_AB00h Init: _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR: 200E_AA00h Init: _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR: 200E_AB00h Init: _ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR: 2080_AA00h Init: _ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR: 2080_AB00h Init: _ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR: 2082_AA00h Init: _ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR: 2082_AB00h Init: _ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR: 2084_AA00h Init: _ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR: 2084_AB00h Init: _ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR: 2086_AA00h Init: _ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR: 2086_AB00h Init: _ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR: 2088_AA00h Init: _ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR: 2088_AB00h Init: _ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR: 208A_AA00h Init: _ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR: 208A_AB00h Init: _ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR: 208C_AA00h Init: _ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR: 208C_AB00h Init: _ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR: 208E_AA00h Init: _ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR: 208E_AB00h Init: _ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR: 2100_AA00h Init: _ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR: 2100_AB00h Init: _ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR: 2102_AA00h Init: _ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR: 2102_AB00h Init: _ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR: 2104_AA00h Init: _ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR: 2104_AB00h Init: _ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR: 2106_AA00h Init: _ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR: 2106_AB00h Init: _ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR: 2108_AA00h Init: _ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR: 2108_AB00h Init: _ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR: 210A_AA00h Init: _ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR: 210A_AB00h Init: _ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR: 210C_AA00h Init: _ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR: 210C_AB00h Init: _ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR: 210E_AA00h Init: _ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR: 210E_AB00h Init: _ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR: 2180_AA00h Init: _ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR: 2180_AB00h Init: _ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR: 2182_AA00h Init: _ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR: 2182_AB00h Init: _ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR: 2184_AA00h

Init: _ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR: 2184_AB00h
Init: _ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR: 2186_AA00h
Init: _ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR: 2186_AB00h
Init: _ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR: 2188_AA00h
Init: _ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR: 2188_AB00h
Init: _ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR: 218A_AA00h
Init: _ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR: 218A_AB00h
Init: _ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR: 218C_AA00h
Init: _ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR: 218C_AB00h
Init: _ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR: 218E_AA00h
Init: _ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR: 218E_AB00h
Init: _ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR: 2200_AA00h
Init: _ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR: 2200_AB00h
Init: _ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR: 2202_AA00h
Init: _ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR: 2202_AB00h
Init: _ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR: 2204_AA00h
Init: _ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR: 2204_AB00h
Init: _ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR: 2206_AA00h
Init: _ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR: 2206_AB00h
Init: _ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR: 2208_AA00h
Init: _ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR: 2208_AB00h
Init: _ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR: 220A_AA00h
Init: _ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR: 220A_AB00h
Init: _ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR: 220C_AA00h
Init: _ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR: 220C_AB00h
Init: _ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR: 220E_AA00h
Init: _ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR: 220E_AB00h
Init: _ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR: 2280_AA00h
Init: _ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR: 2280_AB00h
Init: _ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR: 2282_AA00h
Init: _ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR: 2282_AB00h
Init: _ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR: 2284_AA00h
Init: _ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR: 2284_AB00h
Init: _ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR: 2286_AA00h
Init: _ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR: 2286_AB00h
Init: _ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR: 2288_AA00h
Init: _ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR: 2288_AB00h
Init: _ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR: 228A_AA00h
Init: _ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR: 228A_AB00h
Init: _ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR: 228C_AA00h
Init: _ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR: 228C_AB00h
Init: _ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR: 228E_AA00h
Init: _ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR: 228E_AB00h
Init: _ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR: 2300_AA00h
Init: _ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR: 2300_AB00h
Init: _ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR: 2302_AA00h
Init: _ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR: 2302_AB00h
Init: _ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR: 2304_AA00h
Init: _ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR: 2304_AB00h
Init: _ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR: 2306_AA00h
Init: _ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR: 2306_AB00h
Init: _ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR: 2308_AA00h
Init: _ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR: 2308_AB00h
Init: _ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR: 230A_AA00h
Init: _ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR: 230A_AB00h
Init: _ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR: 230C_AA00h
Init: _ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR: 230C_AB00h
Init: _ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR: 230E_AA00h
Init: _ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR: 230E_AB00h
Init: _ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR: 2380_AA00h
Init: _ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR: 2380_AB00h
Init: _ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR: 2382_AA00h
Init: _ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR: 2382_AB00h
Init: _ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR: 2384_AA00h
Init: _ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR: 2384_AB00h

Init: _ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR: 2386_AA00h
Init: _ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR: 2386_AB00h
Init: _ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR: 2388_AA00h
Init: _ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR: 2388_AB00h
Init: _ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR: 238A_AA00h
Init: _ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR: 238A_AB00h
Init: _ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR: 238C_AA00h
Init: _ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR: 238C_AB00h
Init: _ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR: 238E_AA00h
Init: _ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR: 238E_AB00h
Init: _ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR: 2400_AA00h
Init: _ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR: 2400_AB00h
Init: _ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR: 2402_AA00h
Init: _ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR: 2402_AB00h
Init: _ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR: 2404_AA00h
Init: _ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR: 2404_AB00h
Init: _ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR: 2406_AA00h
Init: _ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR: 2406_AB00h
Init: _ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR: 2408_AA00h
Init: _ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR: 2408_AB00h
Init: _ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR: 240A_AA00h
Init: _ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR: 240A_AB00h
Init: _ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR: 240C_AA00h
Init: _ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR: 240C_AB00h
Init: _ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR: 240E_AA00h
Init: _ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR: 240E_AB00h
Init: _ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR: 2480_AA00h
Init: _ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR: 2480_AB00h
Init: _ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR: 2482_AA00h
Init: _ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR: 2482_AB00h
Init: _ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR: 2484_AA00h
Init: _ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR: 2484_AB00h
Init: _ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR: 2486_AA00h
Init: _ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR: 2486_AB00h
Init: _ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR: 2488_AA00h
Init: _ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR: 2488_AB00h
Init: _ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR: 248A_AA00h
Init: _ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR: 248A_AB00h
Init: _ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR: 248C_AA00h
Init: _ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR: 248C_AB00h
Init: _ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR: 248E_AA00h
Init: _ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR: 248E_AB00h
Init: _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR: 2500_AA00h
Init: _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR: 2500_AB00h
Init: _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR: 2502_AA00h
Init: _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR: 2502_AB00h
Init: _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR: 2504_AA00h
Init: _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR: 2504_AB00h
Init: _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR: 2506_AA00h
Init: _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR: 2506_AB00h
Init: _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR: 2508_AA00h
Init: _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR: 2508_AB00h
Init: _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR: 250A_AA00h
Init: _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR: 250A_AB00h
Init: _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR: 250C_AA00h
Init: _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR: 250C_AB00h
Init: _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR: 250E_AA00h
Init: _ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR: 250E_AB00h
Init: _ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR: 2580_AA00h
Init: _ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR: 2580_AB00h
Init: _ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR: 2582_AA00h
Init: _ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR: 2582_AB00h
Init: _ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR: 2584_AA00h
Init: _ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR: 2584_AB00h
Init: _ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR: 2586_AA00h

Init: _ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR: 2586_AB00h
Init: _ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR: 2588_AA00h
Init: _ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR: 2588_AB00h
Init: _ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR: 258A_AA00h
Init: _ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR: 258A_AB00h
Init: _ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR: 258C_AA00h
Init: _ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR: 258C_AB00h
Init: _ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR: 258E_AA00h
Init: _ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR: 258E_AB00h

MSRC000_2016 [IF Machine Check Syndrome Thread 0] (MCA::IF::MCA_SYND_IF)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::IF::MCA_STATUS_IF [Thread 0](#)

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC000_2016
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC000_2016
 _ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC000_2016
 _ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC000_2016
 _ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC000_2016

_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC000_2016
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC000_2016
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC000_2016
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC000_2016
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC000_2016
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC000_2016
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC000_2016
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC000_2016
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC000_2016
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC000_2016
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC000_2016
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC000_2016
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC000_2016
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC000_2016
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC000_2016
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC000_2016
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC000_2016
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC000_2016
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC000_2016
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC000_2016
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC000_2016
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC000_2016
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC000_2016
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC000_2016
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC000_2016
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC000_2016
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC000_2016
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC000_2016
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC000_2016
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC000_2016
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC000_2016
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC000_2016
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC000_2016
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC000_2016
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC000_2016
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC000_2016
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC000_2016
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC000_2016
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC000_2016
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC000_2016
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC000_2016
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC000_2016
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC000_2016
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC000_2016
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC000_2016
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC000_2016
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC000_2016
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC000_2016
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC000_2016
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC000_2016
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC000_2016
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC000_2016
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC000_2016
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC000_2016
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC000_2016
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC000_2016
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC000_2016
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC000_2016
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC000_2016
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC000_2016
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC000_2016
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC000_2016
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC000_2016
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC000_2016
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC000_2016

_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC000_2016
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC000_2016
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC000_2016
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC000_2016
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC000_2016
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC000_2016
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC000_2016
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC000_2016
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC000_2016
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC000_2016
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC000_2016
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC000_2016
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC000_2016
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC000_2016
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC000_2016
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC000_2016
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC000_2016
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC000_2016
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC000_2016
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC000_2016
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC000_2016
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC000_2016
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC000_2016
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC000_2016
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC000_2016
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC000_2016
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC000_2016
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC000_2016
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC000_2016
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC000_2016
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC000_2016
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC000_2016
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC000_2016
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC000_2016
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC000_2016
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC000_2016
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC000_2016
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC000_2016
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC000_2016
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC000_2016
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC000_2016
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC000_2016
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC000_2016
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC000_2016
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC000_2016
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC000_2016
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC000_2016
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC000_2016
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC000_2016
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC000_2016
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC000_2016
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC000_2016
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC000_2016
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC000_2016
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC000_2016
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC000_2016
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC000_2016
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC000_2016
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC000_2016
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC000_2016
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC000_2016
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC000_2016
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC000_2016
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC000_2016
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC000_2016

_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC000_2016	
Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::IF::MCA_STATUS_IF. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::IF::MCA_SYND_IF[Length]. The Syndrome field is only valid when MCA::IF::MCA_SYND_IF[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::IF::MCA_SYND_IF. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::IF::MCA_SYND_IF[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::IF::MCA_SYND_IF. For example, a syndrome length of 9 means that MCA::IF::MCA_SYND_IF[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 63 [MCA_SYND_IF].

Table 63: MCA_SYND_IF

Error Type	Bits	Description
OcUtagParity	[17:16]	Reserved
	[15:8]	Way bit vector
	[7:6]	Reserved
	[5:0]	Index
TagMultiHit	[17:16]	Reserved
	[15:8]	Way bit vector
	[7:6]	Reserved
	[5:0]	Index
TagParity	[17]	Reserved
	[16]	Way Valid
	[15:8]	Reserved
	[7:0]	Way bit vector
DataParity	[17:15]	Reserved
	[14:12]	Data Bank
	[11:8]	Sub-Bank
	[7:0]	Index
DqParity	[17]	Reserved
	[16]	PA Error Valid
	[15:0]	Way bit vector
RSVD_5	[17:0]	Reserved
L1ItlbParity	[17:6]	Reserved
	[5:0]	Index
L2ItlbParity	[17:16]	Reserved
	[15:8]	Way bit vector
	[7:6]	Reserved
	[5:0]	Index
RSVD_8	[17:0]	Reserved
BpqSnPParT1	[17:16]	Reserved
	[15:8]	Way bit vector
	[7:6]	Reserved

	[5:0]	Index
L1BtbMultiHit	[17:12] [11:8] [7:0]	Reserved Way bit vector Index
L2BtbMultiHit	[17:12] [11:9] [8:0]	Reserved Table Index
L2RespPoison	[17:0]	Reserved
SystemReadDataError	[17:4] [3] [2] [1] [0]	Reserved Protection Violation Transaction Error Target Abort Master Abort
HwAssert	[17:5] [4:0]	Assertion log Code
L1TlbMultiHit	[17:6] [5:0]	Reserved Index
L2TlbMultiHit	[17:16] [15:8] [7:6] [5:0]	Reserved Way bit vector Reserved Index
BsrParity	[17:8] [7:0]	Reserved Index
CtMceError	[17:2] [1:0]	Reserved Thread bit vector

MSRC001_0401 [IF Machine Check Control Mask Thread 0] (MCA::IF::MCA_CTL_MASK_IF)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_core0_thread0_inst1_n0_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core0_thread1_inst1_n1_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core1_thread0_inst1_n2_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core1_thread1_inst1_n3_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core2_thread0_inst1_n4_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core2_thread1_inst1_n5_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core3_thread0_inst1_n6_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core3_thread1_inst1_n7_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core4_thread0_inst1_n8_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core4_thread1_inst1_n9_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core5_thread0_inst1_n10_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core5_thread1_inst1_n11_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core6_thread0_inst1_n12_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core6_thread1_inst1_n13_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core7_thread0_inst1_n14_aliasMSR; MSRC001_0401
 _ccd0_lthree0_core7_thread1_inst1_n15_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core0_thread0_inst1_n160_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core0_thread1_inst1_n161_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core1_thread0_inst1_n162_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core1_thread1_inst1_n163_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core2_thread0_inst1_n164_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core2_thread1_inst1_n165_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core3_thread0_inst1_n166_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core3_thread1_inst1_n167_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core4_thread0_inst1_n168_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core4_thread1_inst1_n169_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core5_thread0_inst1_n170_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core5_thread1_inst1_n171_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core6_thread0_inst1_n172_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core6_thread1_inst1_n173_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core7_thread0_inst1_n174_aliasMSR; MSRC001_0401
 _ccd10_lthree0_core7_thread1_inst1_n175_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core0_thread0_inst1_n176_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core0_thread1_inst1_n177_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core1_thread0_inst1_n178_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core1_thread1_inst1_n179_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core2_thread0_inst1_n180_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core2_thread1_inst1_n181_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core3_thread0_inst1_n182_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core3_thread1_inst1_n183_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core4_thread0_inst1_n184_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core4_thread1_inst1_n185_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core5_thread0_inst1_n186_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core5_thread1_inst1_n187_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core6_thread0_inst1_n188_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core6_thread1_inst1_n189_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core7_thread0_inst1_n190_aliasMSR; MSRC001_0401
 _ccd11_lthree0_core7_thread1_inst1_n191_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core0_thread0_inst1_n16_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core0_thread1_inst1_n17_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core1_thread0_inst1_n18_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core1_thread1_inst1_n19_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core2_thread0_inst1_n20_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core2_thread1_inst1_n21_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core3_thread0_inst1_n22_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core3_thread1_inst1_n23_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core4_thread0_inst1_n24_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core4_thread1_inst1_n25_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core5_thread0_inst1_n26_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core5_thread1_inst1_n27_aliasMSR; MSRC001_0401
 _ccd1_lthree0_core6_thread0_inst1_n28_aliasMSR; MSRC001_0401

_ccd1_lthree0_core6_thread1_inst1_n29_aliasMSR; MSRC001_0401
_ccd1_lthree0_core7_thread0_inst1_n30_aliasMSR; MSRC001_0401
_ccd1_lthree0_core7_thread1_inst1_n31_aliasMSR; MSRC001_0401
_ccd2_lthree0_core0_thread0_inst1_n32_aliasMSR; MSRC001_0401
_ccd2_lthree0_core0_thread1_inst1_n33_aliasMSR; MSRC001_0401
_ccd2_lthree0_core1_thread0_inst1_n34_aliasMSR; MSRC001_0401
_ccd2_lthree0_core1_thread1_inst1_n35_aliasMSR; MSRC001_0401
_ccd2_lthree0_core2_thread0_inst1_n36_aliasMSR; MSRC001_0401
_ccd2_lthree0_core2_thread1_inst1_n37_aliasMSR; MSRC001_0401
_ccd2_lthree0_core3_thread0_inst1_n38_aliasMSR; MSRC001_0401
_ccd2_lthree0_core3_thread1_inst1_n39_aliasMSR; MSRC001_0401
_ccd2_lthree0_core4_thread0_inst1_n40_aliasMSR; MSRC001_0401
_ccd2_lthree0_core4_thread1_inst1_n41_aliasMSR; MSRC001_0401
_ccd2_lthree0_core5_thread0_inst1_n42_aliasMSR; MSRC001_0401
_ccd2_lthree0_core5_thread1_inst1_n43_aliasMSR; MSRC001_0401
_ccd2_lthree0_core6_thread0_inst1_n44_aliasMSR; MSRC001_0401
_ccd2_lthree0_core6_thread1_inst1_n45_aliasMSR; MSRC001_0401
_ccd2_lthree0_core7_thread0_inst1_n46_aliasMSR; MSRC001_0401
_ccd2_lthree0_core7_thread1_inst1_n47_aliasMSR; MSRC001_0401
_ccd3_lthree0_core0_thread0_inst1_n48_aliasMSR; MSRC001_0401
_ccd3_lthree0_core0_thread1_inst1_n49_aliasMSR; MSRC001_0401
_ccd3_lthree0_core1_thread0_inst1_n50_aliasMSR; MSRC001_0401
_ccd3_lthree0_core1_thread1_inst1_n51_aliasMSR; MSRC001_0401
_ccd3_lthree0_core2_thread0_inst1_n52_aliasMSR; MSRC001_0401
_ccd3_lthree0_core2_thread1_inst1_n53_aliasMSR; MSRC001_0401
_ccd3_lthree0_core3_thread0_inst1_n54_aliasMSR; MSRC001_0401
_ccd3_lthree0_core3_thread1_inst1_n55_aliasMSR; MSRC001_0401
_ccd3_lthree0_core4_thread0_inst1_n56_aliasMSR; MSRC001_0401
_ccd3_lthree0_core4_thread1_inst1_n57_aliasMSR; MSRC001_0401
_ccd3_lthree0_core5_thread0_inst1_n58_aliasMSR; MSRC001_0401
_ccd3_lthree0_core5_thread1_inst1_n59_aliasMSR; MSRC001_0401
_ccd3_lthree0_core6_thread0_inst1_n60_aliasMSR; MSRC001_0401
_ccd3_lthree0_core6_thread1_inst1_n61_aliasMSR; MSRC001_0401
_ccd3_lthree0_core7_thread0_inst1_n62_aliasMSR; MSRC001_0401
_ccd3_lthree0_core7_thread1_inst1_n63_aliasMSR; MSRC001_0401
_ccd4_lthree0_core0_thread0_inst1_n64_aliasMSR; MSRC001_0401
_ccd4_lthree0_core0_thread1_inst1_n65_aliasMSR; MSRC001_0401
_ccd4_lthree0_core1_thread0_inst1_n66_aliasMSR; MSRC001_0401
_ccd4_lthree0_core1_thread1_inst1_n67_aliasMSR; MSRC001_0401
_ccd4_lthree0_core2_thread0_inst1_n68_aliasMSR; MSRC001_0401
_ccd4_lthree0_core2_thread1_inst1_n69_aliasMSR; MSRC001_0401
_ccd4_lthree0_core3_thread0_inst1_n70_aliasMSR; MSRC001_0401
_ccd4_lthree0_core3_thread1_inst1_n71_aliasMSR; MSRC001_0401
_ccd4_lthree0_core4_thread0_inst1_n72_aliasMSR; MSRC001_0401
_ccd4_lthree0_core4_thread1_inst1_n73_aliasMSR; MSRC001_0401
_ccd4_lthree0_core5_thread0_inst1_n74_aliasMSR; MSRC001_0401
_ccd4_lthree0_core5_thread1_inst1_n75_aliasMSR; MSRC001_0401
_ccd4_lthree0_core6_thread0_inst1_n76_aliasMSR; MSRC001_0401
_ccd4_lthree0_core6_thread1_inst1_n77_aliasMSR; MSRC001_0401
_ccd4_lthree0_core7_thread0_inst1_n78_aliasMSR; MSRC001_0401
_ccd4_lthree0_core7_thread1_inst1_n79_aliasMSR; MSRC001_0401
_ccd5_lthree0_core0_thread0_inst1_n80_aliasMSR; MSRC001_0401
_ccd5_lthree0_core0_thread1_inst1_n81_aliasMSR; MSRC001_0401
_ccd5_lthree0_core1_thread0_inst1_n82_aliasMSR; MSRC001_0401
_ccd5_lthree0_core1_thread1_inst1_n83_aliasMSR; MSRC001_0401
_ccd5_lthree0_core2_thread0_inst1_n84_aliasMSR; MSRC001_0401
_ccd5_lthree0_core2_thread1_inst1_n85_aliasMSR; MSRC001_0401
_ccd5_lthree0_core3_thread0_inst1_n86_aliasMSR; MSRC001_0401
_ccd5_lthree0_core3_thread1_inst1_n87_aliasMSR; MSRC001_0401
_ccd5_lthree0_core4_thread0_inst1_n88_aliasMSR; MSRC001_0401
_ccd5_lthree0_core4_thread1_inst1_n89_aliasMSR; MSRC001_0401
_ccd5_lthree0_core5_thread0_inst1_n90_aliasMSR; MSRC001_0401
_ccd5_lthree0_core5_thread1_inst1_n91_aliasMSR; MSRC001_0401
_ccd5_lthree0_core6_thread0_inst1_n92_aliasMSR; MSRC001_0401
_ccd5_lthree0_core6_thread1_inst1_n93_aliasMSR; MSRC001_0401

_ccd5_lthree0_core7_thread0_inst1_n94_aliasMSR; MSRC001_0401
_ccd5_lthree0_core7_thread1_inst1_n95_aliasMSR; MSRC001_0401
_ccd6_lthree0_core0_thread0_inst1_n96_aliasMSR; MSRC001_0401
_ccd6_lthree0_core0_thread1_inst1_n97_aliasMSR; MSRC001_0401
_ccd6_lthree0_core1_thread0_inst1_n98_aliasMSR; MSRC001_0401
_ccd6_lthree0_core1_thread1_inst1_n99_aliasMSR; MSRC001_0401
_ccd6_lthree0_core2_thread0_inst1_n100_aliasMSR; MSRC001_0401
_ccd6_lthree0_core2_thread1_inst1_n101_aliasMSR; MSRC001_0401
_ccd6_lthree0_core3_thread0_inst1_n102_aliasMSR; MSRC001_0401
_ccd6_lthree0_core3_thread1_inst1_n103_aliasMSR; MSRC001_0401
_ccd6_lthree0_core4_thread0_inst1_n104_aliasMSR; MSRC001_0401
_ccd6_lthree0_core4_thread1_inst1_n105_aliasMSR; MSRC001_0401
_ccd6_lthree0_core5_thread0_inst1_n106_aliasMSR; MSRC001_0401
_ccd6_lthree0_core5_thread1_inst1_n107_aliasMSR; MSRC001_0401
_ccd6_lthree0_core6_thread0_inst1_n108_aliasMSR; MSRC001_0401
_ccd6_lthree0_core6_thread1_inst1_n109_aliasMSR; MSRC001_0401
_ccd6_lthree0_core7_thread0_inst1_n110_aliasMSR; MSRC001_0401
_ccd6_lthree0_core7_thread1_inst1_n111_aliasMSR; MSRC001_0401
_ccd7_lthree0_core0_thread0_inst1_n112_aliasMSR; MSRC001_0401
_ccd7_lthree0_core0_thread1_inst1_n113_aliasMSR; MSRC001_0401
_ccd7_lthree0_core1_thread0_inst1_n114_aliasMSR; MSRC001_0401
_ccd7_lthree0_core1_thread1_inst1_n115_aliasMSR; MSRC001_0401
_ccd7_lthree0_core2_thread0_inst1_n116_aliasMSR; MSRC001_0401
_ccd7_lthree0_core2_thread1_inst1_n117_aliasMSR; MSRC001_0401
_ccd7_lthree0_core3_thread0_inst1_n118_aliasMSR; MSRC001_0401
_ccd7_lthree0_core3_thread1_inst1_n119_aliasMSR; MSRC001_0401
_ccd7_lthree0_core4_thread0_inst1_n120_aliasMSR; MSRC001_0401
_ccd7_lthree0_core4_thread1_inst1_n121_aliasMSR; MSRC001_0401
_ccd7_lthree0_core5_thread0_inst1_n122_aliasMSR; MSRC001_0401
_ccd7_lthree0_core5_thread1_inst1_n123_aliasMSR; MSRC001_0401
_ccd7_lthree0_core6_thread0_inst1_n124_aliasMSR; MSRC001_0401
_ccd7_lthree0_core6_thread1_inst1_n125_aliasMSR; MSRC001_0401
_ccd7_lthree0_core7_thread0_inst1_n126_aliasMSR; MSRC001_0401
_ccd7_lthree0_core7_thread1_inst1_n127_aliasMSR; MSRC001_0401
_ccd8_lthree0_core0_thread0_inst1_n128_aliasMSR; MSRC001_0401
_ccd8_lthree0_core0_thread1_inst1_n129_aliasMSR; MSRC001_0401
_ccd8_lthree0_core1_thread0_inst1_n130_aliasMSR; MSRC001_0401
_ccd8_lthree0_core1_thread1_inst1_n131_aliasMSR; MSRC001_0401
_ccd8_lthree0_core2_thread0_inst1_n132_aliasMSR; MSRC001_0401
_ccd8_lthree0_core2_thread1_inst1_n133_aliasMSR; MSRC001_0401
_ccd8_lthree0_core3_thread0_inst1_n134_aliasMSR; MSRC001_0401
_ccd8_lthree0_core3_thread1_inst1_n135_aliasMSR; MSRC001_0401
_ccd8_lthree0_core4_thread0_inst1_n136_aliasMSR; MSRC001_0401
_ccd8_lthree0_core4_thread1_inst1_n137_aliasMSR; MSRC001_0401
_ccd8_lthree0_core5_thread0_inst1_n138_aliasMSR; MSRC001_0401
_ccd8_lthree0_core5_thread1_inst1_n139_aliasMSR; MSRC001_0401
_ccd8_lthree0_core6_thread0_inst1_n140_aliasMSR; MSRC001_0401
_ccd8_lthree0_core6_thread1_inst1_n141_aliasMSR; MSRC001_0401
_ccd8_lthree0_core7_thread0_inst1_n142_aliasMSR; MSRC001_0401
_ccd8_lthree0_core7_thread1_inst1_n143_aliasMSR; MSRC001_0401
_ccd9_lthree0_core0_thread0_inst1_n144_aliasMSR; MSRC001_0401
_ccd9_lthree0_core0_thread1_inst1_n145_aliasMSR; MSRC001_0401
_ccd9_lthree0_core1_thread0_inst1_n146_aliasMSR; MSRC001_0401
_ccd9_lthree0_core1_thread1_inst1_n147_aliasMSR; MSRC001_0401
_ccd9_lthree0_core2_thread0_inst1_n148_aliasMSR; MSRC001_0401
_ccd9_lthree0_core2_thread1_inst1_n149_aliasMSR; MSRC001_0401
_ccd9_lthree0_core3_thread0_inst1_n150_aliasMSR; MSRC001_0401
_ccd9_lthree0_core3_thread1_inst1_n151_aliasMSR; MSRC001_0401
_ccd9_lthree0_core4_thread0_inst1_n152_aliasMSR; MSRC001_0401
_ccd9_lthree0_core4_thread1_inst1_n153_aliasMSR; MSRC001_0401
_ccd9_lthree0_core5_thread0_inst1_n154_aliasMSR; MSRC001_0401
_ccd9_lthree0_core5_thread1_inst1_n155_aliasMSR; MSRC001_0401
_ccd9_lthree0_core6_thread0_inst1_n156_aliasMSR; MSRC001_0401
_ccd9_lthree0_core6_thread1_inst1_n157_aliasMSR; MSRC001_0401
_ccd9_lthree0_core7_thread0_inst1_n158_aliasMSR; MSRC001_0401

_ccd9_lthree0_core7_thread1_inst1_n159_aliasMSR; MSRC001_0401	
Bits	Description
63:19	Reserved.
18	CtMceError . Read-write. Reset: 0. CT MCE.
17	BsrParity . Read-write. Reset: 0. BSR Parity Error.
16	L2TlbMultiHit . Read-write. Reset: 0. Init: BIOS,1. L2-TLB Multi-Hit
15	L1TlbMultiHit . Read-write. Reset: 0. L1-TLB Multi-Hit.
14	HwAssert . Read-write. Reset: 0. Hardware Assertion Error.
13	SystemReadDataError . Read-write. Reset: 0. Init: BIOS,1. L2 Cache Error Response.
12	L2RespPoison . Read-write. Reset: 0. L2 Cache Response Poison Error. Error is the result of consuming poison data.
11	L2BtbMultiHit . Read-write. Reset: 0. Init: BIOS,1. BP L2-BTB Multi-Hit Error.
10	L1BtbMultiHit . Read-write. Reset: 0. BP L1-BTB Multi-Hit Error.
9	BpqSnpParT1 . Read-write. Reset: 0. BPQ Thread 1 Snoop Parity Error.
8	RSVD_8 . Read-write. Reset: 0. Reserved.
7	L2ItlbParity . Read-write. Reset: 0. L2-TLB Parity Error.
6	L1ItlbParity . Read-write. Reset: 0. L1-TLB Parity Error.
5	RSVD_5 . Read-write. Reset: 0. Reserved.
4	DqParity . Read-write. Reset: 0. PRQ Parity Error.
3	DataParity . Read-write. Reset: 0. IC Data Array Parity Error.
2	TagParity . Read-write. Reset: 0. IC Full Tag Parity Error.
1	TagMultiHit . Read-write. Reset: 0. IC Microtag or Full Tag Multi-hit Error.
0	OcUtagParity . Read-write. Reset: 0. Op Cache Microtag Parity Error. Parity errors on PA and other relevant uTag fields are reported, independent of any utag probing. The parity error way and index are logged.

3.2.5.3 L2

MSR0000_0408...MSRC000_2020 [L2 Machine Check Control Thread 0] (MCA::L2::MCA_CTL_L2)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::L2::MCA_CTL_L2 register must be enabled by the corresponding enable bit in Core::X86::Msrr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2020
_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2020
_ccd0_lthree0_core0_thread1_inst2_n1_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2020
_ccd0_lthree0_core1_thread0_inst2_n2_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2020
_ccd0_lthree0_core1_thread1_inst2_n3_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2020
_ccd0_lthree0_core2_thread0_inst2_n4_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2020
_ccd0_lthree0_core2_thread1_inst2_n5_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2020
_ccd0_lthree0_core3_thread0_inst2_n6_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2020
_ccd0_lthree0_core3_thread1_inst2_n7_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2020
_ccd0_lthree0_core4_thread0_inst2_n8_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2020
_ccd0_lthree0_core4_thread1_inst2_n9_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2020
_ccd0_lthree0_core5_thread0_inst2_n10_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2020
_ccd0_lthree0_core5_thread1_inst2_n11_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2020
_ccd0_lthree0_core6_thread0_inst2_n12_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2020
_ccd0_lthree0_core6_thread1_inst2_n13_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2020
_ccd0_lthree0_core7_thread0_inst2_n14_aliasMSRLEGACY; MSR0000_0408
_ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2020
_ccd0_lthree0_core7_thread1_inst2_n15_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2020
_ccd10_lthree0_core0_thread0_inst2_n160_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2020
_ccd10_lthree0_core0_thread1_inst2_n161_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2020
_ccd10_lthree0_core1_thread0_inst2_n162_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2020
_ccd10_lthree0_core1_thread1_inst2_n163_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2020
_ccd10_lthree0_core2_thread0_inst2_n164_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2020
_ccd10_lthree0_core2_thread1_inst2_n165_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2020
_ccd10_lthree0_core3_thread0_inst2_n166_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2020
_ccd10_lthree0_core3_thread1_inst2_n167_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2020
_ccd10_lthree0_core4_thread0_inst2_n168_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2020
_ccd10_lthree0_core4_thread1_inst2_n169_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2020
_ccd10_lthree0_core5_thread0_inst2_n170_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2020
_ccd10_lthree0_core5_thread1_inst2_n171_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2020
_ccd10_lthree0_core6_thread0_inst2_n172_aliasMSRLEGACY; MSR0000_0408

_ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2020
_ccd10_lthree0_core6_thread1_inst2_n173_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2020
_ccd10_lthree0_core7_thread0_inst2_n174_aliasMSRLEGACY; MSR0000_0408
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2020
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2020
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2020
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2020
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2020
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2020
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2020
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2020
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2020
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2020
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2020
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2020
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2020
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2020
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2020
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2020
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSRLEGACY; MSR0000_0408
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2020
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2020
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2020
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2020
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2020
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2020
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2020
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2020
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2020
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2020
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2020
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2020
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2020
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2020
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2020

_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2020
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSRLEGACY; MSR0000_0408
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2020
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2020
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2020
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2020
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2020
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2020
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2020
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2020
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2020
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2020
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2020
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2020
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2020
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2020
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2020
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2020
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSRLEGACY; MSR0000_0408
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2020
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2020
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2020
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2020
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2020
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2020
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2020
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2020
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2020
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2020
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2020
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2020
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2020
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2020
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2020
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSRLEGACY; MSR0000_0408

_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2020
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSRLEGACY; MSR0000_0408
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2020
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2020
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2020
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2020
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2020
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2020
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2020
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2020
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2020
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2020
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2020
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2020
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2020
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2020
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2020
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2020
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSRLEGACY; MSR0000_0408
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2020
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2020
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2020
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2020
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2020
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2020
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2020
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2020
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2020
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2020
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2020
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2020
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2020
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2020
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2020
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2020

_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSRLEGACY; MSR0000_0408
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2020
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2020
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2020
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2020
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2020
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2020
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2020
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2020
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2020
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2020
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2020
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2020
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2020
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2020
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2020
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2020
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSRLEGACY; MSR0000_0408
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2020
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2020
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2020
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2020
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2020
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2020
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2020
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2020
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2020
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2020
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2020
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2020
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2020
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2020
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2020
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSRLEGACY; MSR0000_0408
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2020
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSRLEGACY; MSR0000_0408

_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2020
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2020
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2020
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2020
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2020
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2020
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2020
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2020
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2020
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2020
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2020
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2020
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2020
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2020
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2020
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2020
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSRLEGACY; MSR0000_0408
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2020
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2020
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2020
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2020
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2020
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2020
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2020
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2020
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2020
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2020
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2020
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2020
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2020
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2020
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2020
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2020
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSRLEGACY; MSR0000_0408
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2020

_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSRLEGACY; MSR0000_0408	
Bits	Description
63:5	Reserved.
4	Sdp. Read-write. Reset: 0. SDP Read Response Parity Error
3	Hwa. Read-write. Reset: 0. Hardware Assert Error.
2	Data. Read-write. Reset: 0. L2M Data Array ECC Error.
1	Tag. Read-write. Reset: 0. L2M Tag or State Array ECC Error. MCA_STATUS_L2[CECC]=1 and MCA_STATUS_L2[UC]=1 indicates a hard error. MCA_SYND_L2[Syndrome]=17'b{1'b{tag error}, 1'b{state error}, 7'b{tag syndrome}, 1'b{reserved}, 4'b{state syndrome}, 3'b{DCValid syndrome}}.
0	MultiHit. Read-write. Reset: 0. L2M Tag Multiple-Way-Hit error.

MSR0000_0409...MSRC000_2021 [L2 Machine Check Status Thread 0] (MCA::L2::MCA_STATUS_L2)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core0_thread0_inst2_n0_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSRLEGACY; MSR0000_0409
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2021
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2021
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSRLEGACY; MSR0000_0409
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2021

_ccd10_lthree0_core7_thread0_inst2_n174_aliasMSRLEGACY; MSR0000_0409
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2021
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2021
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2021
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2021
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2021
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2021
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2021
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2021
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2021
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2021
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2021
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2021
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2021
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2021
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2021
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2021
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSRLEGACY; MSR0000_0409
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2021
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2021
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2021
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2021
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2021
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2021
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2021
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2021
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2021
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2021
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2021
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2021
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2021
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2021
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2021
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSRLEGACY; MSR0000_0409
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2021
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSRLEGACY; MSR0000_0409

_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2021
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2021
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2021
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2021
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2021
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2021
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2021
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2021
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2021
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2021
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2021
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2021
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2021
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2021
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2021
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2021
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSRLEGACY; MSR0000_0409
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2021
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2021
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2021
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2021
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2021
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2021
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2021
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2021
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2021
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2021
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2021
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2021
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2021
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2021
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2021
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2021
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSRLEGACY; MSR0000_0409
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2021

_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2021
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2021
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2021
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2021
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2021
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2021
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2021
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2021
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2021
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2021
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2021
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2021
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2021
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2021
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2021
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSRLEGACY; MSR0000_0409
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2021
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2021
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2021
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2021
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2021
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2021
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2021
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2021
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2021
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2021
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2021
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2021
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2021
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2021
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2021
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2021
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSRLEGACY; MSR0000_0409
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2021
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSRLEGACY; MSR0000_0409

_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2021
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2021
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2021
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2021
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2021
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2021
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2021
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2021
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2021
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2021
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2021
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2021
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2021
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2021
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2021
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSRLEGACY; MSR0000_0409
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2021
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2021
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2021
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2021
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2021
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2021
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2021
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2021
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2021
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2021
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2021
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2021
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2021
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2021
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2021
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2021
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSRLEGACY; MSR0000_0409
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2021
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSRLEGACY; MSR0000_0409
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2021

_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSRLEGACY; MSR0000_0409	
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2021	
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSRLEGACY; MSR0000_0409	
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2021	
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSRLEGACY; MSR0000_0409	
Bits	Description

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::L2::MCA_CTL_L2. This bit is a copy of bit in MCA::L2::MCA_CTL_L2 for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::L2::MCA_MISC0_L2. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::L2::MCA_ADDR_L2 contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::L2::MCA_STATUS_L2[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::L2::MCA_SYND_L2. If MCA::L2::MCA_SYND_L2[ErrorPriority] is the same as the priority of the error in MCA::L2::MCA_STATUS_L2, then the information in MCA::L2::MCA_SYND_L2 is associated with the error in MCA::L2::MCA_STATUS_L2. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::L2::MCA_STATUS_L2[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::L2::MCA_ADDR_L2[ErrorAddr]. A value of 0 indicates that MCA::L2::MCA_ADDR_L2[55:0] contains a valid byte address. A value of 6 indicates that MCA::L2::MCA_ADDR_L2[55:6] contains a valid cache line address and that MCA::L2::MCA_ADDR_L2[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::L2::MCA_ADDR_L2[55:12] contain a valid 4KB memory page and that MCA::L2::MCA_ADDR_L2[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::L2::MCA_CTL_L2 enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 64: MCA_STATUS_L2

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
MultiHit	0x0	1	1	1	0	0	1
Tag	0x1	0/1	0/1	0/1	0	0	1
Data	0x2	0/1	0/1	0/1	0/1	0	1
Hwa	0x3	1	1	1	0	0	1
Sdp	0x4	0/1	0/1	0/1	0/1	0	0

MSR0000_040A...MSRC000_2022 [L2 Machine Check Address Thread 0] (MCA::L2::MCA_ADDR_L2)

Reset: Cold,0000_0000_0000_0000h.

MCA::L2::MCA_ADDR_L2 stores an address and other information associated with the error in MCA::L2::MCA_STATUS_L2. The register is only meaningful if MCA::L2::MCA_STATUS_L2[Val]=1 and MCA::L2::MCA_STATUS_L2[AddrV]=1.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core0_thread0_inst2_n0_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSRLEGACY; MSR0000_040A
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2022
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSRLEGACY; MSR0000_040A
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2022
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSRLEGACY; MSR0000_040A

_ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2022
_ccd10_lthree0_core6_thread1_inst2_n173_aliasMSRLEGACY; MSR0000_040A
_ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2022
_ccd10_lthree0_core7_thread0_inst2_n174_aliasMSRLEGACY; MSR0000_040A
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2022
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2022
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2022
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2022
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2022
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2022
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2022
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2022
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2022
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2022
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2022
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2022
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2022
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2022
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2022
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2022
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSRLEGACY; MSR0000_040A
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2022
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2022
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2022
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2022
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2022
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2022
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2022
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2022
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2022
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2022
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2022
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2022
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2022
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2022
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2022

_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2022
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSRLEGACY; MSR0000_040A
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2022
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2022
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2022
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2022
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2022
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2022
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2022
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2022
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2022
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2022
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2022
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2022
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2022
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2022
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2022
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2022
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSRLEGACY; MSR0000_040A
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2022
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2022
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2022
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2022
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2022
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2022
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2022
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2022
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2022
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2022
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2022
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2022
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2022
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2022
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2022
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSRLEGACY; MSR0000_040A

_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2022
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSRLEGACY; MSR0000_040A
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2022
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2022
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2022
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2022
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2022
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2022
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2022
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2022
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2022
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2022
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2022
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2022
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2022
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2022
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2022
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2022
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSRLEGACY; MSR0000_040A
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2022
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2022
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2022
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2022
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2022
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2022
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2022
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2022
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2022
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2022
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2022
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2022
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2022
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2022
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2022
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2022

_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSRLEGACY; MSR0000_040A
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2022
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2022
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2022
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2022
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2022
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2022
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2022
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2022
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2022
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2022
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2022
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2022
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2022
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2022
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2022
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2022
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSRLEGACY; MSR0000_040A
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2022
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2022
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2022
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2022
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2022
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2022
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2022
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2022
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2022
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2022
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2022
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2022
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2022
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2022
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2022
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSRLEGACY; MSR0000_040A
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2022
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSRLEGACY; MSR0000_040A

_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2022
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2022
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2022
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2022
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2022
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2022
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2022
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2022
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2022
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2022
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2022
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2022
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2022
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2022
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2022
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2022
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSRLEGACY; MSR0000_040A
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2022
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2022
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2022
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2022
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2022
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2022
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2022
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2022
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2022
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2022
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2022
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2022
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2022
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2022
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2022
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2022
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSRLEGACY; MSR0000_040A
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2022

_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSRLEGACY; MSR0000_040A	
Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::L2::MCA_STATUS_L2. For physical addresses, the most significant bit is given by Core::X86::CpuId::LongModeInfo[PhysAddrSize].

Table 65: MCA_ADDR_L2

Error Type	Bits	Description
MultiHit	[55:52]	Reserved
	[51:6]	Physical Address
	[5:0]	Reserved
Tag	[55:52]	Reserved
	[51:6]	Physical Address
	[5:0]	Reserved
Data	[55:52]	Reserved
	[51:6]	Physical Address
	[5:0]	Reserved
Hwa	[31:0]	Reserved
Sdp	[55:0]	Reserved

MSR0000_040B...MSRC000_2023 [L2 Machine Check Miscellaneous 0 Thread 0] (MCA::L2::MCA_MISC0_L2)

Log miscellaneous information associated with errors.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core0_thread0_inst2_n0_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSRLEGACY; MSR0000_040B
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2023
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSRLEGACY; MSR0000_040B
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2023
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSRLEGACY; MSR0000_040B

_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2023
_ccd10_lthree0_core7_thread1_inst2_n175_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2023
_ccd11_lthree0_core0_thread0_inst2_n176_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2023
_ccd11_lthree0_core0_thread1_inst2_n177_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2023
_ccd11_lthree0_core1_thread0_inst2_n178_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2023
_ccd11_lthree0_core1_thread1_inst2_n179_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2023
_ccd11_lthree0_core2_thread0_inst2_n180_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2023
_ccd11_lthree0_core2_thread1_inst2_n181_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2023
_ccd11_lthree0_core3_thread0_inst2_n182_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2023
_ccd11_lthree0_core3_thread1_inst2_n183_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2023
_ccd11_lthree0_core4_thread0_inst2_n184_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2023
_ccd11_lthree0_core4_thread1_inst2_n185_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2023
_ccd11_lthree0_core5_thread0_inst2_n186_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2023
_ccd11_lthree0_core5_thread1_inst2_n187_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2023
_ccd11_lthree0_core6_thread0_inst2_n188_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2023
_ccd11_lthree0_core6_thread1_inst2_n189_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2023
_ccd11_lthree0_core7_thread0_inst2_n190_aliasMSRLEGACY; MSR0000_040B
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2023
_ccd11_lthree0_core7_thread1_inst2_n191_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2023
_ccd1_lthree0_core0_thread0_inst2_n16_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2023
_ccd1_lthree0_core0_thread1_inst2_n17_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2023
_ccd1_lthree0_core1_thread0_inst2_n18_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2023
_ccd1_lthree0_core1_thread1_inst2_n19_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2023
_ccd1_lthree0_core2_thread0_inst2_n20_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2023
_ccd1_lthree0_core2_thread1_inst2_n21_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2023
_ccd1_lthree0_core3_thread0_inst2_n22_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2023
_ccd1_lthree0_core3_thread1_inst2_n23_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2023
_ccd1_lthree0_core4_thread0_inst2_n24_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2023
_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2023
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2023
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2023
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2023
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2023
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSRLEGACY; MSR0000_040B
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2023

_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2023
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2023
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2023
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2023
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2023
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2023
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2023
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2023
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2023
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2023
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2023
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2023
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2023
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2023
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2023
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSRLEGACY; MSR0000_040B
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2023
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2023
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2023
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2023
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2023
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2023
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2023
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2023
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2023
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2023
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2023
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2023
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2023
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2023
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2023
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2023
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSRLEGACY; MSR0000_040B
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2023
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSRLEGACY; MSR0000_040B

_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2023
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2023
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2023
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2023
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2023
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2023
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2023
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2023
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2023
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2023
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2023
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2023
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2023
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2023
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2023
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSRLEGACY; MSR0000_040B
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2023
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2023
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2023
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2023
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2023
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2023
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2023
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2023
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2023
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2023
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2023
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2023
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2023
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2023
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2023
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2023
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSRLEGACY; MSR0000_040B
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2023
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2023

_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2023
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2023
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2023
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2023
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2023
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2023
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2023
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2023
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2023
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2023
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2023
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2023
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2023
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2023
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSRLEGACY; MSR0000_040B
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2023
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2023
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2023
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2023
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2023
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2023
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2023
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2023
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2023
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2023
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2023
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2023
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2023
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2023
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2023
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2023
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSRLEGACY; MSR0000_040B
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2023
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSRLEGACY; MSR0000_040B
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2023
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSRLEGACY; MSR0000_040B

<u>_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2023</u>	
<u>_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSRLEGACY; MSR0000_040B</u>	
<u>_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2023</u>	
<u>_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSRLEGACY; MSR0000_040B</u>	
Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP . Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked . Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP . Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::L2::MCA_MISC0_L2[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset . Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::L2::MCA_MISC0_L2[Locked]) ? Read-write : Read-only.
51	CntEn . Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::L2::MCA_MISC0_L2[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType . Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrlw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::L2::MCA_MISC0_L2[Locked]) ? Read-write : Read-only.
48	Ovrlw . Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::L2::MCA_MISC0_L2[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt . Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::L2::MCA_MISC0_L2[Locked]) ? Read-write : Read-only.
31:24	BlkPtr . Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2024 [L2 Machine Check Configuration Thread 0] (MCA::L2::MCA_CONFIG_L2)

Reset: 0000_0002_0000_0127h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2024
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2024
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2024
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2024
 _ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2024

_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2024
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2024
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2024
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2024
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2024
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2024
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2024
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2024
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2024
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2024
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2024
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2024
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2024
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2024
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2024
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2024
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2024
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2024
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2024
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2024
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2024
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2024
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2024
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2024
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2024
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2024
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2024
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2024
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2024
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2024
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2024
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2024
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2024
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2024
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2024
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2024
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2024
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2024
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2024
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2024
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2024
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2024
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2024
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2024
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2024
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2024
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2024
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2024
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2024
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2024
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2024
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2024
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2024
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2024
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2024
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2024
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2024
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2024
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2024
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2024
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2024
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2024
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2024
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2024
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2024

_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2024
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2024
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2024
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2024
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2024
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2024
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2024
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2024
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2024
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2024
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2024
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2024
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2024
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2024
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2024
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2024
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2024
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2024
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2024
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2024
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2024
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2024
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2024
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2024
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2024
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2024
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2024
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2024
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2024
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2024
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2024
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2024
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2024
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2024
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2024
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2024
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2024
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2024
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2024
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2024
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2024
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2024
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2024
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2024
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2024
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2024
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2024
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2024
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2024
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2024
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2024
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2024
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2024
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2024
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2024
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2024
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2024
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2024
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2024
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2024
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2024
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2024
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2024
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2024
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2024

_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2024	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat. Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::L2::MCA_STATUS_L2 and MCA::L2::MCA_ADDR_L2 in addition to MCA::L2::MCA_DESTAT_L2 and MCA::L2::MCA_DEADDR_L2. 0=Only log deferred errors in MCA::L2::MCA_DESTAT_L2 and MCA::L2::MCA_DEADDR_L2. This bit does not affect logging of deferred errors in MCA::L2::MCA_SYND_L2, MCA::L2::MCA_MISC0_L2.
33	TransparentErrorLoggingEnable. Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::L2::MCA_STATUS_L2, MCA::L2::MCA_ADDR_L2, MCA::L2::MCA_MISC0_L2, and MCA::L2::MCA_SYND_L2 in addition to MCA::L2::MCA_TRANSYND_L2 and MCA::L2::MCA_TRANSADDR_L2. 0=Only log transparent errors in MCA::L2::MCA_TRANSYND_L2 and MCA::L2::MCA_TRANSADDR_L2.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::L2::MCA_CONFIG_L2[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::L2::MCA_CONFIG_L2[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::L2::MCA_CONFIG_L2[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::L2::MCA_CONFIG_L2[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::L2::MCA_DESTAT_L2 and MCA::L2::MCA_DEADDR_L2 are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::L2::MCA_CONFIG_L2[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::L2::MCA_MISC0_L2[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::L2::MCA_STATUS_L2[TCC] is present.

MSRC000_2025 [L2 IP Identification Thread 0] (MCA::L2::MCA_IPID_L2)

Reset: 0002_00B0_0000_0000h.

The MCA::L2::MCA_IPID_L2 register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2025
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2025
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2025
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2025
 _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2025

_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2025
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2025
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2025
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2025
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2025
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2025
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2025
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2025
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2025
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2025
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2025
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2025
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2025
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2025
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2025
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2025
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2025
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2025
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2025
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2025
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2025
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2025
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2025
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2025
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2025
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2025
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2025
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2025
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2025
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2025
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2025
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2025
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2025
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2025
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2025
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2025
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2025
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2025
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2025
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2025
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2025
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2025
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2025
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2025
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2025
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2025
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2025
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2025
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2025
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2025
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2025
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2025
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2025
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2025
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2025
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2025
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2025
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2025
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2025
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2025
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2025
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2025
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2025
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2025
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2025

_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2025
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2025
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2025
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2025
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2025
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2025
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2025
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2025
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2025
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2025
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2025
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2025
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2025
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2025
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2025
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2025
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2025
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2025
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2025
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2025
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2025
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2025
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2025
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2025
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2025
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2025
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2025
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2025
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2025
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2025
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2025
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2025
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2025
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2025
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2025
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2025
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2025
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2025
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2025
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2025
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2025
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2025
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2025
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2025
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2025
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2025
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2025
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2025
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2025
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2025
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2025
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2025
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2025
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2025
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2025
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2025
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2025
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2025
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2025
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2025
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2025
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2025
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2025
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2025
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2025

_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2025	
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2025	
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2025	
Bits	Description
63:48	McaType . Read-only. Reset: 0002h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR: 2000_8A00h Init: _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR: 2000_8B00h Init: _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR: 2002_8A00h Init: _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR: 2002_8B00h Init: _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR: 2004_8A00h Init: _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR: 2004_8B00h Init: _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR: 2006_8A00h Init: _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR: 2006_8B00h Init: _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR: 2008_8A00h Init: _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR: 2008_8B00h Init: _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR: 200A_8A00h Init: _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR: 200A_8B00h Init: _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR: 200C_8A00h Init: _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR: 200C_8B00h Init: _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR: 200E_8A00h Init: _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR: 200E_8B00h Init: _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR: 2080_8A00h Init: _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR: 2080_8B00h Init: _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR: 2082_8A00h Init: _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR: 2082_8B00h Init: _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR: 2084_8A00h Init: _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR: 2084_8B00h Init: _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR: 2086_8A00h Init: _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR: 2086_8B00h Init: _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR: 2088_8A00h Init: _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR: 2088_8B00h Init: _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR: 208A_8A00h Init: _ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR: 208A_8B00h Init: _ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR: 208C_8A00h Init: _ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR: 208C_8B00h Init: _ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR: 208E_8A00h Init: _ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR: 208E_8B00h Init: _ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR: 2100_8A00h Init: _ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR: 2100_8B00h Init: _ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR: 2102_8A00h Init: _ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR: 2102_8B00h Init: _ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR: 2104_8A00h Init: _ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR: 2104_8B00h Init: _ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR: 2106_8A00h Init: _ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR: 2106_8B00h Init: _ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR: 2108_8A00h Init: _ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR: 2108_8B00h Init: _ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR: 210A_8A00h Init: _ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR: 210A_8B00h Init: _ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR: 210C_8A00h Init: _ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR: 210C_8B00h Init: _ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR: 210E_8A00h Init: _ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR: 210E_8B00h Init: _ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR: 2180_8A00h Init: _ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR: 2180_8B00h Init: _ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR: 2182_8A00h Init: _ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR: 2182_8B00h Init: _ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR: 2184_8A00h

Init: _ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR: 2184_8B00h
Init: _ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR: 2186_8A00h
Init: _ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR: 2186_8B00h
Init: _ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR: 2188_8A00h
Init: _ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR: 2188_8B00h
Init: _ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR: 218A_8A00h
Init: _ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR: 218A_8B00h
Init: _ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR: 218C_8A00h
Init: _ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR: 218C_8B00h
Init: _ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR: 218E_8A00h
Init: _ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR: 218E_8B00h
Init: _ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR: 2200_8A00h
Init: _ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR: 2200_8B00h
Init: _ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR: 2202_8A00h
Init: _ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR: 2202_8B00h
Init: _ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR: 2204_8A00h
Init: _ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR: 2204_8B00h
Init: _ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR: 2206_8A00h
Init: _ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR: 2206_8B00h
Init: _ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR: 2208_8A00h
Init: _ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR: 2208_8B00h
Init: _ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR: 220A_8A00h
Init: _ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR: 220A_8B00h
Init: _ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR: 220C_8A00h
Init: _ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR: 220C_8B00h
Init: _ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR: 220E_8A00h
Init: _ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR: 220E_8B00h
Init: _ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR: 2280_8A00h
Init: _ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR: 2280_8B00h
Init: _ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR: 2282_8A00h
Init: _ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR: 2282_8B00h
Init: _ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR: 2284_8A00h
Init: _ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR: 2284_8B00h
Init: _ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR: 2286_8A00h
Init: _ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR: 2286_8B00h
Init: _ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR: 2288_8A00h
Init: _ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR: 2288_8B00h
Init: _ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR: 228A_8A00h
Init: _ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR: 228A_8B00h
Init: _ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR: 228C_8A00h
Init: _ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR: 228C_8B00h
Init: _ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR: 228E_8A00h
Init: _ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR: 228E_8B00h
Init: _ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR: 2300_8A00h
Init: _ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR: 2300_8B00h
Init: _ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR: 2302_8A00h
Init: _ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR: 2302_8B00h
Init: _ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR: 2304_8A00h
Init: _ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR: 2304_8B00h
Init: _ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR: 2306_8A00h
Init: _ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR: 2306_8B00h
Init: _ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR: 2308_8A00h
Init: _ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR: 2308_8B00h
Init: _ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR: 230A_8A00h
Init: _ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR: 230A_8B00h
Init: _ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR: 230C_8A00h
Init: _ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR: 230C_8B00h
Init: _ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR: 230E_8A00h
Init: _ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR: 230E_8B00h
Init: _ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR: 2380_8A00h
Init: _ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR: 2380_8B00h
Init: _ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR: 2382_8A00h
Init: _ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR: 2382_8B00h
Init: _ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR: 2384_8A00h
Init: _ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR: 2384_8B00h

Init: _ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR: 2386_8A00h
Init: _ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR: 2386_8B00h
Init: _ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR: 2388_8A00h
Init: _ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR: 2388_8B00h
Init: _ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR: 238A_8A00h
Init: _ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR: 238A_8B00h
Init: _ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR: 238C_8A00h
Init: _ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR: 238C_8B00h
Init: _ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR: 238E_8A00h
Init: _ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR: 238E_8B00h
Init: _ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR: 2400_8A00h
Init: _ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR: 2400_8B00h
Init: _ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR: 2402_8A00h
Init: _ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR: 2402_8B00h
Init: _ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR: 2404_8A00h
Init: _ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR: 2404_8B00h
Init: _ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR: 2406_8A00h
Init: _ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR: 2406_8B00h
Init: _ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR: 2408_8A00h
Init: _ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR: 2408_8B00h
Init: _ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR: 240A_8A00h
Init: _ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR: 240A_8B00h
Init: _ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR: 240C_8A00h
Init: _ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR: 240C_8B00h
Init: _ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR: 240E_8A00h
Init: _ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR: 240E_8B00h
Init: _ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR: 2480_8A00h
Init: _ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR: 2480_8B00h
Init: _ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR: 2482_8A00h
Init: _ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR: 2482_8B00h
Init: _ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR: 2484_8A00h
Init: _ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR: 2484_8B00h
Init: _ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR: 2486_8A00h
Init: _ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR: 2486_8B00h
Init: _ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR: 2488_8A00h
Init: _ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR: 2488_8B00h
Init: _ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR: 248A_8A00h
Init: _ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR: 248A_8B00h
Init: _ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR: 248C_8A00h
Init: _ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR: 248C_8B00h
Init: _ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR: 248E_8A00h
Init: _ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR: 248E_8B00h
Init: _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR: 2500_8A00h
Init: _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR: 2500_8B00h
Init: _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR: 2502_8A00h
Init: _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR: 2502_8B00h
Init: _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR: 2504_8A00h
Init: _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR: 2504_8B00h
Init: _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR: 2506_8A00h
Init: _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR: 2506_8B00h
Init: _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR: 2508_8A00h
Init: _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR: 2508_8B00h
Init: _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR: 250A_8A00h
Init: _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR: 250A_8B00h
Init: _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR: 250C_8A00h
Init: _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR: 250C_8B00h
Init: _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR: 250E_8A00h
Init: _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR: 250E_8B00h
Init: _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR: 2580_8A00h
Init: _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR: 2580_8B00h
Init: _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR: 2582_8A00h
Init: _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR: 2582_8B00h
Init: _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR: 2584_8A00h
Init: _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR: 2584_8B00h
Init: _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR: 2586_8A00h

Init: _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR: 2586_8B00h
Init: _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR: 2588_8A00h
Init: _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR: 2588_8B00h
Init: _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR: 258A_8A00h
Init: _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR: 258A_8B00h
Init: _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR: 258C_8A00h
Init: _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR: 258C_8B00h
Init: _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR: 258E_8A00h
Init: _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR: 258E_8B00h

MSRC000_2026 [L2 Machine Check Syndrome Thread 0] (MCA::L2::MCA_SYND_L2)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::L2::MCA_STATUS_L2 [Thread 0](#)

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2026
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2026
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2026
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2026
 _ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2026

_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2026
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2026
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2026
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2026
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2026
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2026
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2026
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2026
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2026
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2026
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2026
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2026
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2026
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2026
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2026
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2026
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2026
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2026
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2026
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2026
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2026
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2026
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2026
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2026
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2026
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2026
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2026
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2026
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2026
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2026
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2026
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2026
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2026
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2026
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2026
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2026
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2026
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2026
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2026
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2026
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2026
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2026
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2026
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2026
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2026
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2026
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2026
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2026
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2026
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2026
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2026
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2026
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2026
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2026
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2026
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2026
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2026
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2026
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2026
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2026
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2026
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2026
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2026
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2026
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2026

_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2026
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2026
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2026
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2026
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2026
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2026
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2026
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2026
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2026
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2026
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2026
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2026
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2026
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2026
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2026
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2026
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2026
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2026
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2026
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2026
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2026
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2026
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2026
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2026
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2026
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2026
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2026
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2026
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2026
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2026
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2026
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2026
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2026
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2026
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2026
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2026
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2026
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2026
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2026
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2026
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2026
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2026
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2026
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2026
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2026
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2026
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2026
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2026
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2026
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2026
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2026
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2026
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2026
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2026
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2026
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2026
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2026
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2026
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2026
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2026
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2026
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2026
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2026
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2026
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2026

_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2026	
Bits	Description
63:51	Reserved.
50:32	Syndrome. Read-write, Volatile. Reset: Cold, 0_0000h. Contains the syndrome, if any, associated with the error logged in MCA::L2::MCA_STATUS_L2. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::L2::MCA_SYND_L2[Length]. The Syndrome field is only valid when MCA::L2::MCA_SYND_L2[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::L2::MCA_SYND_L2. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::L2::MCA_SYND_L2[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::L2::MCA_SYND_L2. For example, a syndrome length of 9 means that MCA::L2::MCA_SYND_L2[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 66 [MCA_SYND_L2].

Table 66: MCA_SYND_L2

Error Type	Bits	Description
MultiHit	[18:8] [7:0]	Index One-hot way vector
Tag	[17:14] [13:3] [2:0]	Reserved Index Way
Data	[17:17] [16] [15:5] [4] [3] [2:0]	Reserved Poison Index TopBotLoc Reserved Way
Hwa	[17:0]	Reserved
Sdp	[17:17] [16:6] [5:3] [2:0]	Reserved Addr[16:6] Addr[5:3] L2Way

MSRC000_2028 [L2 Machine Check Deferred Error Status Thread 0] (MCA::L2::MCA_DESTAT_L2)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2028
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2028
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2028
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2028
 _ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2028

_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2028
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2028
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2028
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2028
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2028
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2028
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2028
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2028
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2028
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2028
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2028
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2028
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2028
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2028
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2028
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2028
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2028
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2028
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2028
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2028
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2028
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2028
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2028
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2028
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2028
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2028
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2028
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2028
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2028
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2028
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2028
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2028
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2028
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2028
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2028
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2028
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2028
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2028
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2028
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2028
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2028
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2028
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2028
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2028
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2028
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2028
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2028
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2028
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2028
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2028
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2028
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2028
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2028
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2028
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2028
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2028
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2028
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2028
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2028
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2028
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2028
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2028
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2028
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2028
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2028

_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2028
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2028
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2028
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2028
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2028
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2028
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2028
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2028
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2028
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2028
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2028
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2028
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2028
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2028
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2028
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2028
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2028
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2028
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2028
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2028
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2028
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2028
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2028
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2028
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2028
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2028
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2028
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2028
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2028
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2028
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2028
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2028
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2028
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2028
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2028
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2028
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2028
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2028
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2028
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2028
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2028
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2028
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2028
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2028
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2028
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2028
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2028
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2028
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2028
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2028
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2028
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2028
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2028
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2028
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2028
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2028
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2028
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2028
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2028
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2028
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2028
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2028
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2028
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2028
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2028

_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2028	
Bits	Description
63	Val. Read-write, Volatile. Reset: Cold, 0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold, 0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold, 0h.
58	AddrV. Read-write, Volatile. Reset: Cold, 0. 1=MCA::L2::MCA_DEADDR_L2 contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold, 0h.
53	SyndV. Read-write, Volatile. Reset: Cold, 0. 1=This error logged information in MCA::L2::MCA_SYND_L2. If MCA::L2::MCA_SYND_L2[ErrorPriority] is the same as the priority of the error in MCA::L2::MCA_STATUS_L2, then the information in MCA::L2::MCA_SYND_L2 is associated with the error in MCA::L2::MCA_DESTAT_L2.
52:45	RESERV2. Read-write. Reset: Cold, 00h.
44	Deferred. Read-write, Volatile. Reset: Cold, 0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold, 0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold, 00h. Specifies the least significant valid bit of the address contained in MCA::L2::MCA_ADDR_L2[ErrorAddr]. A value of 0 indicates that MCA::L2::MCA_ADDR_L2[55:0] contains a valid byte address. A value of 6 indicates that MCA::L2::MCA_ADDR_L2[55:6] contains a valid cache line address and that MCA::L2::MCA_ADDR_L2[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::L2::MCA_ADDR_L2[55:12] contain a valid 4KB memory page and that MCA::L2::MCA_ADDR_L2[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold, 0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold, 00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold, 0000h. Error code for this error.

MSRC000_2029 [L2 Deferred Error Address Thread 0] (MCA::L2::MCA_DEADDR_L2)

Reset: Cold,0000_0000_0000_0000h.

The MCA::L2::MCA_DEADDR_L2 register stores the address associated with the error in MCA::L2::MCA_DESTAT_L2. The register is only meaningful if MCA::L2::MCA_DESTAT_L2[Val]=1 and MCA::L2::MCA_DESTAT_L2[AddrV]=1. The lowest valid bit of the address is defined by MCA::L2::MCA_DESTAT_L2[AddrLsb].

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC000_2029
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC000_2029
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC000_2029
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC000_2029
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC000_2029

_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC000_2029
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC000_2029
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC000_2029
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC000_2029
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC000_2029
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC000_2029
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC000_2029
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC000_2029
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC000_2029
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC000_2029
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC000_2029
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC000_2029
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC000_2029
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC000_2029
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC000_2029
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC000_2029
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC000_2029
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC000_2029
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC000_2029
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC000_2029
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC000_2029
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC000_2029
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC000_2029
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC000_2029
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC000_2029
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC000_2029
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC000_2029
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC000_2029
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC000_2029
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC000_2029
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC000_2029
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC000_2029
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC000_2029
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC000_2029
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC000_2029
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC000_2029
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC000_2029
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC000_2029
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC000_2029
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC000_2029
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC000_2029
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC000_2029
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC000_2029
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC000_2029
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC000_2029
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC000_2029
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC000_2029
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC000_2029
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC000_2029
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC000_2029
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC000_2029
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC000_2029
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC000_2029
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC000_2029
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC000_2029
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC000_2029
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC000_2029
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC000_2029
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC000_2029
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC000_2029
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC000_2029
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC000_2029
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC000_2029
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC000_2029
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC000_2029

_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC000_2029
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC000_2029
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC000_2029
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC000_2029
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC000_2029
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC000_2029
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC000_2029
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC000_2029
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC000_2029
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC000_2029
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC000_2029
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC000_2029
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC000_2029
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC000_2029
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC000_2029
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC000_2029
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC000_2029
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC000_2029
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC000_2029
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC000_2029
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC000_2029
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC000_2029
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC000_2029
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC000_2029
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC000_2029
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC000_2029
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC000_2029
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC000_2029
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC000_2029
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC000_2029
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC000_2029
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC000_2029
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC000_2029
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC000_2029
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC000_2029
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC000_2029
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC000_2029
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC000_2029
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC000_2029
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC000_2029
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC000_2029
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC000_2029
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC000_2029
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC000_2029
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC000_2029
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC000_2029
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC000_2029
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC000_2029
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC000_2029
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC000_2029
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC000_2029
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC000_2029
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC000_2029
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC000_2029
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC000_2029
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC000_2029
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC000_2029
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC000_2029
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC000_2029
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC000_2029
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC000_2029
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC000_2029
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC000_2029
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC000_2029
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC000_2029

_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC000_2029	
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC000_2029	
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC000_2029	
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC000_2029	
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC000_2029	
Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::L2::MCA_DESTAT_L2. The lowest-order valid bit of the address is specified in MCA::L2::MCA_DESTAT_L2[AddrLsb].

MSRC001_0402 [L2 Machine Check Control Mask Thread 0] (MCA::L2::MCA_CTL_MASK_L2)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC001_0402
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC001_0402
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC001_0402
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC001_0402
 _ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC001_0402

_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC001_0402
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC001_0402
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC001_0402
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC001_0402
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC001_0402
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC001_0402
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC001_0402
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC001_0402
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC001_0402
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC001_0402
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC001_0402
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC001_0402
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC001_0402
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC001_0402
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC001_0402
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC001_0402
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC001_0402
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC001_0402
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC001_0402
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC001_0402
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC001_0402
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC001_0402
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC001_0402
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC001_0402
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC001_0402
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC001_0402
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC001_0402
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC001_0402
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC001_0402
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC001_0402
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC001_0402
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC001_0402
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC001_0402
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC001_0402
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC001_0402
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC001_0402
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC001_0402
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC001_0402
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC001_0402
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC001_0402
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC001_0402
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC001_0402
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC001_0402
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC001_0402
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC001_0402
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC001_0402
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC001_0402
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC001_0402
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC001_0402
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC001_0402
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC001_0402
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC001_0402
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC001_0402
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC001_0402
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC001_0402
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC001_0402
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC001_0402
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC001_0402
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC001_0402
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC001_0402
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC001_0402
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC001_0402
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC001_0402
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC001_0402
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC001_0402

_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC001_0402
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC001_0402
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC001_0402
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC001_0402
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC001_0402
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC001_0402
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC001_0402
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC001_0402
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC001_0402
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC001_0402
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC001_0402
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC001_0402
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC001_0402
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC001_0402
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC001_0402
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC001_0402
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC001_0402
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC001_0402
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC001_0402
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC001_0402
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC001_0402
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC001_0402
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC001_0402
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC001_0402
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC001_0402
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC001_0402
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC001_0402
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC001_0402
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC001_0402
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC001_0402
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC001_0402
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC001_0402
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC001_0402
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC001_0402
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC001_0402
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC001_0402
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC001_0402
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC001_0402
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC001_0402
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC001_0402
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC001_0402
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC001_0402
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC001_0402
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC001_0402
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC001_0402
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC001_0402
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC001_0402
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC001_0402
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC001_0402
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC001_0402
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC001_0402
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC001_0402
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC001_0402
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC001_0402
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC001_0402
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC001_0402
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC001_0402
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC001_0402
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC001_0402
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC001_0402
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC001_0402
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC001_0402
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC001_0402
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC001_0402
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC001_0402

_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC001_0402	
Bits	Description
63:5	Reserved.
4	Sdp. Read-write. Reset: 0. SDP Read Response Parity Error
3	Hwa. Read-write. Reset: 0. Hardware Assert Error.
2	Data. Read-write. Reset: 0. L2M Data Array ECC Error.
1	Tag. Read-write. Reset: 0. L2M Tag or State Array ECC Error. MCA_STATUS_L2[CECC]=1 and MCA_STATUS_L2[UC]=1 indicates a hard error. MCA_SYND_L2[Syndrome]=17'b{1'b{tag error}, 1'b{state error}, 7'b{tag syndrome}, 1'b{reserved}, 4'b{state syndrome}, 3'b{DCValid syndrome}}.
0	MultiHit. Read-write. Reset: 0. L2M Tag Multiple-Way-Hit error.

**MSRC001_0502 [L2 Machine Check Transparent Error Syndrome Thread 0]
(MCA::L2::MCA_TRANSSYND_L2)**

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC001_0502
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC001_0502
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC001_0502
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC001_0502
 _ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC001_0502

_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC001_0502
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC001_0502
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC001_0502
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC001_0502
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC001_0502
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC001_0502
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC001_0502
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC001_0502
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC001_0502
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC001_0502
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC001_0502
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC001_0502
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC001_0502
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC001_0502
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC001_0502
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC001_0502
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC001_0502
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC001_0502
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC001_0502
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC001_0502
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC001_0502
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC001_0502
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC001_0502
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC001_0502
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC001_0502
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC001_0502
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC001_0502
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC001_0502
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC001_0502
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC001_0502
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC001_0502
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC001_0502
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC001_0502
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC001_0502
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC001_0502
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC001_0502
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC001_0502
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC001_0502
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC001_0502
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC001_0502
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC001_0502
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC001_0502
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC001_0502
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC001_0502
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC001_0502
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC001_0502
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC001_0502
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC001_0502
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC001_0502
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC001_0502
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC001_0502
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC001_0502
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC001_0502
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC001_0502
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC001_0502
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC001_0502
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC001_0502
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC001_0502
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC001_0502
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC001_0502
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC001_0502
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC001_0502
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC001_0502
_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC001_0502
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC001_0502

_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC001_0502
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC001_0502
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC001_0502
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC001_0502
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC001_0502
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC001_0502
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC001_0502
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC001_0502
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC001_0502
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC001_0502
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC001_0502
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC001_0502
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC001_0502
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC001_0502
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC001_0502
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC001_0502
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC001_0502
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC001_0502
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC001_0502
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC001_0502
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC001_0502
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC001_0502
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC001_0502
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC001_0502
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC001_0502
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC001_0502
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC001_0502
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC001_0502
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC001_0502
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC001_0502
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC001_0502
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC001_0502
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC001_0502
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC001_0502
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC001_0502
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC001_0502
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC001_0502
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC001_0502
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC001_0502
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC001_0502
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC001_0502
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC001_0502
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC001_0502
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC001_0502
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC001_0502
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC001_0502
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC001_0502
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC001_0502
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC001_0502
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC001_0502
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC001_0502
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC001_0502
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC001_0502
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC001_0502
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC001_0502
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC001_0502
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC001_0502
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC001_0502
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC001_0502
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC001_0502
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC001_0502
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC001_0502
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC001_0502
_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC001_0502
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC001_0502

_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC001_0502	
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC001_0502	
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC001_0502	
Bits	Description
63:51	Reserved.
50:32	Syndrome. Read-write, Volatile. Reset: Cold, 0_0000h. Contains the syndrome, if any, associated with the error logged in MCA::L2::MCA_STATUS_L2. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::L2::MCA_SYND_L2[Length]. The Syndrome field is only valid when MCA::L2::MCA_SYND_L2[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::L2::MCA_TRANSYND_L2. All errors writing into MCA::L2::MCA_SYND_L2 should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::L2::MCA_SYND_L2. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::L2::MCA_SYND_L2[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::L2::MCA_SYND_L2. For example, a syndrome length of 9 means that MCA::L2::MCA_SYND_L2[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

**MSRC001_0602 [L2 Machine Check Transparent Error Address Thread 0]
(MCA::L2::MCA_TRANSADDR_L2)**

Reset: Cold,0000_0000_0000_0000h.

MCA::L2::MCA_TRANSADDR_L2 stores the address associated with the error in MCA::L2::MCA_STATUS_L2. The register is only meaningful if MCA::L2::MCA_STATUS_L2[Val]=1 and MCA::L2::MCA_STATUS_L2[AddrV]=1. The lowest valid bit of the address is defined by MCA::L2::MCA_STATUS_L2[AddrLsb].

_ccd0_lthree0_core0_thread0_inst2_n0_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core0_thread1_inst2_n1_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core1_thread0_inst2_n2_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core1_thread1_inst2_n3_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core2_thread0_inst2_n4_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core2_thread1_inst2_n5_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core3_thread0_inst2_n6_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core3_thread1_inst2_n7_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core4_thread0_inst2_n8_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core4_thread1_inst2_n9_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core5_thread0_inst2_n10_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core5_thread1_inst2_n11_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core6_thread0_inst2_n12_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core6_thread1_inst2_n13_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core7_thread0_inst2_n14_aliasMSR; MSRC001_0602
 _ccd0_lthree0_core7_thread1_inst2_n15_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core0_thread0_inst2_n160_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core0_thread1_inst2_n161_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core1_thread0_inst2_n162_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core1_thread1_inst2_n163_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core2_thread0_inst2_n164_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core2_thread1_inst2_n165_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core3_thread0_inst2_n166_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core3_thread1_inst2_n167_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core4_thread0_inst2_n168_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core4_thread1_inst2_n169_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core5_thread0_inst2_n170_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core5_thread1_inst2_n171_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core6_thread0_inst2_n172_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core6_thread1_inst2_n173_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core7_thread0_inst2_n174_aliasMSR; MSRC001_0602
 _ccd10_lthree0_core7_thread1_inst2_n175_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core0_thread0_inst2_n176_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core0_thread1_inst2_n177_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core1_thread0_inst2_n178_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core1_thread1_inst2_n179_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core2_thread0_inst2_n180_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core2_thread1_inst2_n181_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core3_thread0_inst2_n182_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core3_thread1_inst2_n183_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core4_thread0_inst2_n184_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core4_thread1_inst2_n185_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core5_thread0_inst2_n186_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core5_thread1_inst2_n187_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core6_thread0_inst2_n188_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core6_thread1_inst2_n189_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core7_thread0_inst2_n190_aliasMSR; MSRC001_0602
 _ccd11_lthree0_core7_thread1_inst2_n191_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core0_thread0_inst2_n16_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core0_thread1_inst2_n17_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core1_thread0_inst2_n18_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core1_thread1_inst2_n19_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core2_thread0_inst2_n20_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core2_thread1_inst2_n21_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core3_thread0_inst2_n22_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core3_thread1_inst2_n23_aliasMSR; MSRC001_0602
 _ccd1_lthree0_core4_thread0_inst2_n24_aliasMSR; MSRC001_0602

_ccd1_lthree0_core4_thread1_inst2_n25_aliasMSR; MSRC001_0602
_ccd1_lthree0_core5_thread0_inst2_n26_aliasMSR; MSRC001_0602
_ccd1_lthree0_core5_thread1_inst2_n27_aliasMSR; MSRC001_0602
_ccd1_lthree0_core6_thread0_inst2_n28_aliasMSR; MSRC001_0602
_ccd1_lthree0_core6_thread1_inst2_n29_aliasMSR; MSRC001_0602
_ccd1_lthree0_core7_thread0_inst2_n30_aliasMSR; MSRC001_0602
_ccd1_lthree0_core7_thread1_inst2_n31_aliasMSR; MSRC001_0602
_ccd2_lthree0_core0_thread0_inst2_n32_aliasMSR; MSRC001_0602
_ccd2_lthree0_core0_thread1_inst2_n33_aliasMSR; MSRC001_0602
_ccd2_lthree0_core1_thread0_inst2_n34_aliasMSR; MSRC001_0602
_ccd2_lthree0_core1_thread1_inst2_n35_aliasMSR; MSRC001_0602
_ccd2_lthree0_core2_thread0_inst2_n36_aliasMSR; MSRC001_0602
_ccd2_lthree0_core2_thread1_inst2_n37_aliasMSR; MSRC001_0602
_ccd2_lthree0_core3_thread0_inst2_n38_aliasMSR; MSRC001_0602
_ccd2_lthree0_core3_thread1_inst2_n39_aliasMSR; MSRC001_0602
_ccd2_lthree0_core4_thread0_inst2_n40_aliasMSR; MSRC001_0602
_ccd2_lthree0_core4_thread1_inst2_n41_aliasMSR; MSRC001_0602
_ccd2_lthree0_core5_thread0_inst2_n42_aliasMSR; MSRC001_0602
_ccd2_lthree0_core5_thread1_inst2_n43_aliasMSR; MSRC001_0602
_ccd2_lthree0_core6_thread0_inst2_n44_aliasMSR; MSRC001_0602
_ccd2_lthree0_core6_thread1_inst2_n45_aliasMSR; MSRC001_0602
_ccd2_lthree0_core7_thread0_inst2_n46_aliasMSR; MSRC001_0602
_ccd2_lthree0_core7_thread1_inst2_n47_aliasMSR; MSRC001_0602
_ccd3_lthree0_core0_thread0_inst2_n48_aliasMSR; MSRC001_0602
_ccd3_lthree0_core0_thread1_inst2_n49_aliasMSR; MSRC001_0602
_ccd3_lthree0_core1_thread0_inst2_n50_aliasMSR; MSRC001_0602
_ccd3_lthree0_core1_thread1_inst2_n51_aliasMSR; MSRC001_0602
_ccd3_lthree0_core2_thread0_inst2_n52_aliasMSR; MSRC001_0602
_ccd3_lthree0_core2_thread1_inst2_n53_aliasMSR; MSRC001_0602
_ccd3_lthree0_core3_thread0_inst2_n54_aliasMSR; MSRC001_0602
_ccd3_lthree0_core3_thread1_inst2_n55_aliasMSR; MSRC001_0602
_ccd3_lthree0_core4_thread0_inst2_n56_aliasMSR; MSRC001_0602
_ccd3_lthree0_core4_thread1_inst2_n57_aliasMSR; MSRC001_0602
_ccd3_lthree0_core5_thread0_inst2_n58_aliasMSR; MSRC001_0602
_ccd3_lthree0_core5_thread1_inst2_n59_aliasMSR; MSRC001_0602
_ccd3_lthree0_core6_thread0_inst2_n60_aliasMSR; MSRC001_0602
_ccd3_lthree0_core6_thread1_inst2_n61_aliasMSR; MSRC001_0602
_ccd3_lthree0_core7_thread0_inst2_n62_aliasMSR; MSRC001_0602
_ccd3_lthree0_core7_thread1_inst2_n63_aliasMSR; MSRC001_0602
_ccd4_lthree0_core0_thread0_inst2_n64_aliasMSR; MSRC001_0602
_ccd4_lthree0_core0_thread1_inst2_n65_aliasMSR; MSRC001_0602
_ccd4_lthree0_core1_thread0_inst2_n66_aliasMSR; MSRC001_0602
_ccd4_lthree0_core1_thread1_inst2_n67_aliasMSR; MSRC001_0602
_ccd4_lthree0_core2_thread0_inst2_n68_aliasMSR; MSRC001_0602
_ccd4_lthree0_core2_thread1_inst2_n69_aliasMSR; MSRC001_0602
_ccd4_lthree0_core3_thread0_inst2_n70_aliasMSR; MSRC001_0602
_ccd4_lthree0_core3_thread1_inst2_n71_aliasMSR; MSRC001_0602
_ccd4_lthree0_core4_thread0_inst2_n72_aliasMSR; MSRC001_0602
_ccd4_lthree0_core4_thread1_inst2_n73_aliasMSR; MSRC001_0602
_ccd4_lthree0_core5_thread0_inst2_n74_aliasMSR; MSRC001_0602
_ccd4_lthree0_core5_thread1_inst2_n75_aliasMSR; MSRC001_0602
_ccd4_lthree0_core6_thread0_inst2_n76_aliasMSR; MSRC001_0602
_ccd4_lthree0_core6_thread1_inst2_n77_aliasMSR; MSRC001_0602
_ccd4_lthree0_core7_thread0_inst2_n78_aliasMSR; MSRC001_0602
_ccd4_lthree0_core7_thread1_inst2_n79_aliasMSR; MSRC001_0602
_ccd5_lthree0_core0_thread0_inst2_n80_aliasMSR; MSRC001_0602
_ccd5_lthree0_core0_thread1_inst2_n81_aliasMSR; MSRC001_0602
_ccd5_lthree0_core1_thread0_inst2_n82_aliasMSR; MSRC001_0602
_ccd5_lthree0_core1_thread1_inst2_n83_aliasMSR; MSRC001_0602
_ccd5_lthree0_core2_thread0_inst2_n84_aliasMSR; MSRC001_0602
_ccd5_lthree0_core2_thread1_inst2_n85_aliasMSR; MSRC001_0602
_ccd5_lthree0_core3_thread0_inst2_n86_aliasMSR; MSRC001_0602
_ccd5_lthree0_core3_thread1_inst2_n87_aliasMSR; MSRC001_0602
_ccd5_lthree0_core4_thread0_inst2_n88_aliasMSR; MSRC001_0602
_ccd5_lthree0_core4_thread1_inst2_n89_aliasMSR; MSRC001_0602

_ccd5_lthree0_core5_thread0_inst2_n90_aliasMSR; MSRC001_0602
_ccd5_lthree0_core5_thread1_inst2_n91_aliasMSR; MSRC001_0602
_ccd5_lthree0_core6_thread0_inst2_n92_aliasMSR; MSRC001_0602
_ccd5_lthree0_core6_thread1_inst2_n93_aliasMSR; MSRC001_0602
_ccd5_lthree0_core7_thread0_inst2_n94_aliasMSR; MSRC001_0602
_ccd5_lthree0_core7_thread1_inst2_n95_aliasMSR; MSRC001_0602
_ccd6_lthree0_core0_thread0_inst2_n96_aliasMSR; MSRC001_0602
_ccd6_lthree0_core0_thread1_inst2_n97_aliasMSR; MSRC001_0602
_ccd6_lthree0_core1_thread0_inst2_n98_aliasMSR; MSRC001_0602
_ccd6_lthree0_core1_thread1_inst2_n99_aliasMSR; MSRC001_0602
_ccd6_lthree0_core2_thread0_inst2_n100_aliasMSR; MSRC001_0602
_ccd6_lthree0_core2_thread1_inst2_n101_aliasMSR; MSRC001_0602
_ccd6_lthree0_core3_thread0_inst2_n102_aliasMSR; MSRC001_0602
_ccd6_lthree0_core3_thread1_inst2_n103_aliasMSR; MSRC001_0602
_ccd6_lthree0_core4_thread0_inst2_n104_aliasMSR; MSRC001_0602
_ccd6_lthree0_core4_thread1_inst2_n105_aliasMSR; MSRC001_0602
_ccd6_lthree0_core5_thread0_inst2_n106_aliasMSR; MSRC001_0602
_ccd6_lthree0_core5_thread1_inst2_n107_aliasMSR; MSRC001_0602
_ccd6_lthree0_core6_thread0_inst2_n108_aliasMSR; MSRC001_0602
_ccd6_lthree0_core6_thread1_inst2_n109_aliasMSR; MSRC001_0602
_ccd6_lthree0_core7_thread0_inst2_n110_aliasMSR; MSRC001_0602
_ccd6_lthree0_core7_thread1_inst2_n111_aliasMSR; MSRC001_0602
_ccd7_lthree0_core0_thread0_inst2_n112_aliasMSR; MSRC001_0602
_ccd7_lthree0_core0_thread1_inst2_n113_aliasMSR; MSRC001_0602
_ccd7_lthree0_core1_thread0_inst2_n114_aliasMSR; MSRC001_0602
_ccd7_lthree0_core1_thread1_inst2_n115_aliasMSR; MSRC001_0602
_ccd7_lthree0_core2_thread0_inst2_n116_aliasMSR; MSRC001_0602
_ccd7_lthree0_core2_thread1_inst2_n117_aliasMSR; MSRC001_0602
_ccd7_lthree0_core3_thread0_inst2_n118_aliasMSR; MSRC001_0602
_ccd7_lthree0_core3_thread1_inst2_n119_aliasMSR; MSRC001_0602
_ccd7_lthree0_core4_thread0_inst2_n120_aliasMSR; MSRC001_0602
_ccd7_lthree0_core4_thread1_inst2_n121_aliasMSR; MSRC001_0602
_ccd7_lthree0_core5_thread0_inst2_n122_aliasMSR; MSRC001_0602
_ccd7_lthree0_core5_thread1_inst2_n123_aliasMSR; MSRC001_0602
_ccd7_lthree0_core6_thread0_inst2_n124_aliasMSR; MSRC001_0602
_ccd7_lthree0_core6_thread1_inst2_n125_aliasMSR; MSRC001_0602
_ccd7_lthree0_core7_thread0_inst2_n126_aliasMSR; MSRC001_0602
_ccd7_lthree0_core7_thread1_inst2_n127_aliasMSR; MSRC001_0602
_ccd8_lthree0_core0_thread0_inst2_n128_aliasMSR; MSRC001_0602
_ccd8_lthree0_core0_thread1_inst2_n129_aliasMSR; MSRC001_0602
_ccd8_lthree0_core1_thread0_inst2_n130_aliasMSR; MSRC001_0602
_ccd8_lthree0_core1_thread1_inst2_n131_aliasMSR; MSRC001_0602
_ccd8_lthree0_core2_thread0_inst2_n132_aliasMSR; MSRC001_0602
_ccd8_lthree0_core2_thread1_inst2_n133_aliasMSR; MSRC001_0602
_ccd8_lthree0_core3_thread0_inst2_n134_aliasMSR; MSRC001_0602
_ccd8_lthree0_core3_thread1_inst2_n135_aliasMSR; MSRC001_0602
_ccd8_lthree0_core4_thread0_inst2_n136_aliasMSR; MSRC001_0602
_ccd8_lthree0_core4_thread1_inst2_n137_aliasMSR; MSRC001_0602
_ccd8_lthree0_core5_thread0_inst2_n138_aliasMSR; MSRC001_0602
_ccd8_lthree0_core5_thread1_inst2_n139_aliasMSR; MSRC001_0602
_ccd8_lthree0_core6_thread0_inst2_n140_aliasMSR; MSRC001_0602
_ccd8_lthree0_core6_thread1_inst2_n141_aliasMSR; MSRC001_0602
_ccd8_lthree0_core7_thread0_inst2_n142_aliasMSR; MSRC001_0602
_ccd8_lthree0_core7_thread1_inst2_n143_aliasMSR; MSRC001_0602
_ccd9_lthree0_core0_thread0_inst2_n144_aliasMSR; MSRC001_0602
_ccd9_lthree0_core0_thread1_inst2_n145_aliasMSR; MSRC001_0602
_ccd9_lthree0_core1_thread0_inst2_n146_aliasMSR; MSRC001_0602
_ccd9_lthree0_core1_thread1_inst2_n147_aliasMSR; MSRC001_0602
_ccd9_lthree0_core2_thread0_inst2_n148_aliasMSR; MSRC001_0602
_ccd9_lthree0_core2_thread1_inst2_n149_aliasMSR; MSRC001_0602
_ccd9_lthree0_core3_thread0_inst2_n150_aliasMSR; MSRC001_0602
_ccd9_lthree0_core3_thread1_inst2_n151_aliasMSR; MSRC001_0602
_ccd9_lthree0_core4_thread0_inst2_n152_aliasMSR; MSRC001_0602
_ccd9_lthree0_core4_thread1_inst2_n153_aliasMSR; MSRC001_0602
_ccd9_lthree0_core5_thread0_inst2_n154_aliasMSR; MSRC001_0602

_ccd9_lthree0_core5_thread1_inst2_n155_aliasMSR; MSRC001_0602	
_ccd9_lthree0_core6_thread0_inst2_n156_aliasMSR; MSRC001_0602	
_ccd9_lthree0_core6_thread1_inst2_n157_aliasMSR; MSRC001_0602	
_ccd9_lthree0_core7_thread0_inst2_n158_aliasMSR; MSRC001_0602	
_ccd9_lthree0_core7_thread1_inst2_n159_aliasMSR; MSRC001_0602	
Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::L2::MCA_STATUS_L2. The lowest-order valid bit of the address is specified in MCA::L2::MCA_STATUS_L2[AddrLsb].

3.2.5.4 DE

MSR0000_040C...MSRC000_2030 [DE Machine Check Control Thread 0] (MCA::DE::MCA_CTL_DE)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::DE::MCA_CTL_DE register must be enabled by the corresponding enable bit in Core::X86::Msrr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2030
_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2030
_ccd0_lthree0_core0_thread1_inst3_n1_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2030
_ccd0_lthree0_core1_thread0_inst3_n2_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2030
_ccd0_lthree0_core1_thread1_inst3_n3_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2030
_ccd0_lthree0_core2_thread0_inst3_n4_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2030
_ccd0_lthree0_core2_thread1_inst3_n5_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2030
_ccd0_lthree0_core3_thread0_inst3_n6_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2030
_ccd0_lthree0_core3_thread1_inst3_n7_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2030
_ccd0_lthree0_core4_thread0_inst3_n8_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2030
_ccd0_lthree0_core4_thread1_inst3_n9_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2030
_ccd0_lthree0_core5_thread0_inst3_n10_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2030
_ccd0_lthree0_core5_thread1_inst3_n11_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2030
_ccd0_lthree0_core6_thread0_inst3_n12_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2030
_ccd0_lthree0_core6_thread1_inst3_n13_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2030
_ccd0_lthree0_core7_thread0_inst3_n14_aliasMSRLEGACY; MSR0000_040C
_ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2030
_ccd0_lthree0_core7_thread1_inst3_n15_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2030
_ccd10_lthree0_core0_thread0_inst3_n160_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2030
_ccd10_lthree0_core0_thread1_inst3_n161_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2030
_ccd10_lthree0_core1_thread0_inst3_n162_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2030
_ccd10_lthree0_core1_thread1_inst3_n163_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2030
_ccd10_lthree0_core2_thread0_inst3_n164_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2030
_ccd10_lthree0_core2_thread1_inst3_n165_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2030
_ccd10_lthree0_core3_thread0_inst3_n166_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2030
_ccd10_lthree0_core3_thread1_inst3_n167_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2030
_ccd10_lthree0_core4_thread0_inst3_n168_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2030
_ccd10_lthree0_core4_thread1_inst3_n169_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2030
_ccd10_lthree0_core5_thread0_inst3_n170_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2030
_ccd10_lthree0_core5_thread1_inst3_n171_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2030
_ccd10_lthree0_core6_thread0_inst3_n172_aliasMSRLEGACY; MSR0000_040C

_ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2030
_ccd10_lthree0_core6_thread1_inst3_n173_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2030
_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSRLEGACY; MSR0000_040C
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2030
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2030
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2030
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2030
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2030
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2030
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2030
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2030
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2030
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2030
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2030
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2030
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2030
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2030
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2030
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2030
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSRLEGACY; MSR0000_040C
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2030
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2030
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2030
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2030
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2030
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2030
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2030
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2030
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2030
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2030
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2030
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2030
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2030
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2030
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2030

_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2030
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSRLEGACY; MSR0000_040C
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2030
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2030
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2030
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2030
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2030
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2030
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2030
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2030
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2030
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2030
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2030
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2030
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2030
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2030
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2030
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2030
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSRLEGACY; MSR0000_040C
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2030
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2030
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2030
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2030
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2030
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2030
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2030
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2030
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2030
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2030
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2030
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2030
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2030
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2030
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2030
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSRLEGACY; MSR0000_040C

_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2030
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSRLEGACY; MSR0000_040C
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2030
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2030
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2030
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2030
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2030
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2030
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2030
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2030
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2030
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2030
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2030
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2030
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2030
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2030
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2030
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2030
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSRLEGACY; MSR0000_040C
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2030
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2030
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2030
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2030
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2030
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2030
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2030
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2030
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2030
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2030
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2030
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2030
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2030
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2030
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2030
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2030

_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSRLEGACY; MSR0000_040C
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2030
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2030
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2030
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2030
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2030
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2030
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2030
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2030
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2030
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2030
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2030
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2030
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2030
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2030
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2030
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2030
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSRLEGACY; MSR0000_040C
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2030
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2030
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2030
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2030
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2030
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2030
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2030
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2030
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2030
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2030
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2030
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2030
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2030
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2030
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2030
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSRLEGACY; MSR0000_040C
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2030
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSRLEGACY; MSR0000_040C

_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2030
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2030
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2030
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2030
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2030
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2030
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2030
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2030
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2030
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2030
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2030
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2030
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2030
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2030
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2030
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2030
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSRLEGACY; MSR0000_040C
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2030
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2030
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2030
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2030
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2030
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2030
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2030
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2030
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2030
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2030
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2030
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2030
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2030
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2030
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2030
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2030
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSRLEGACY; MSR0000_040C
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2030

_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSRLEGACY; MSR0000_040C	
Bits	Description
63:10	Reserved.
9	HwAssertMca. Read-write. Reset: 0. Hardware Assertion MCA Error
8	OCBQ. Read-write. Reset: 0. Micro-op buffer parity error.
7	UcSeq. Read-write. Reset: 0. Patch RAM sequencer parity error
6	UcDat. Read-write. Reset: 0. Patch RAM data parity error
5	Faq. Read-write. Reset: 0. Fetch address FIFO parity error
4	Idq. Read-write. Reset: 0. Instruction dispatch queue parity error
3	UopQ. Read-write. Reset: 0. Micro-op Queue parity error
2	Ibq. Read-write. Reset: 0. IBB Register File parity error
1	OcDat. Read-write. Reset: 0. Micro-op cache DATA Array parity error
0	OcTag. Read-write. Reset: 0. Micro-op cache TAG Array parity error

MSR0000_040D...MSRC000_2031 [DE Machine Check Status Thread 0] (MCA::DE::MCA_STATUS_DE)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core0_thread0_inst3_n0_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSRLEGACY; MSR0000_040D
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2031
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2031
 _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSRLEGACY; MSR0000_040D
 _ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2031

_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSRLEGACY; MSR0000_040D
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2031
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2031
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2031
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2031
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2031
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2031
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2031
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2031
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2031
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2031
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2031
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2031
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2031
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2031
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2031
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2031
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSRLEGACY; MSR0000_040D
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2031
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2031
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2031
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2031
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2031
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2031
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2031
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2031
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2031
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2031
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2031
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2031
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2031
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2031
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2031
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSRLEGACY; MSR0000_040D
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2031
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSRLEGACY; MSR0000_040D

_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2031
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2031
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2031
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2031
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2031
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2031
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2031
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2031
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2031
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2031
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2031
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2031
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2031
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2031
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2031
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2031
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSRLEGACY; MSR0000_040D
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2031
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2031
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2031
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2031
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2031
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2031
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2031
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2031
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2031
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2031
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2031
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2031
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2031
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2031
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2031
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2031
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSRLEGACY; MSR0000_040D
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2031

_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2031
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2031
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2031
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2031
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2031
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2031
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2031
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2031
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2031
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2031
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2031
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2031
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2031
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2031
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2031
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSRLEGACY; MSR0000_040D
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2031
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2031
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2031
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2031
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2031
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2031
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2031
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2031
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2031
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2031
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2031
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2031
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2031
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2031
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2031
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2031
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSRLEGACY; MSR0000_040D
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2031
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSRLEGACY; MSR0000_040D

_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2031
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2031
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2031
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2031
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2031
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2031
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2031
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2031
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2031
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2031
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2031
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2031
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2031
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2031
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2031
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSRLEGACY; MSR0000_040D
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2031
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2031
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2031
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2031
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2031
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2031
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2031
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2031
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2031
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2031
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2031
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2031
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2031
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2031
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2031
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2031
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSRLEGACY; MSR0000_040D
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2031
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSRLEGACY; MSR0000_040D
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2031

_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSRLEGACY; MSR0000_040D	
ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSRLEGACY; MSR0000_040D	
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2031	
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSRLEGACY; MSR0000_040D	
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2031	
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSRLEGACY; MSR0000_040D	
Bits	Description

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::DE::MCA_CTL_DE. This bit is a copy of bit in MCA::DE::MCA_CTL_DE for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::DE::MCA_MISC0_DE. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::DE::MCA_ADDR_DE contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::DE::MCA_STATUS_DE[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::DE::MCA_SYND_DE. If MCA::DE::MCA_SYND_DE[ErrorPriority] is the same as the priority of the error in MCA::DE::MCA_STATUS_DE, then the information in MCA::DE::MCA_SYND_DE is associated with the error in MCA::DE::MCA_STATUS_DE. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::DE::MCA_STATUS_DE[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::DE::MCA_ADDR_DE[ErrorAddr]. A value of 0 indicates that MCA::DE::MCA_ADDR_DE[55:0] contains a valid byte address. A value of 6 indicates that MCA::DE::MCA_ADDR_DE[55:6] contains a valid cache line address and that MCA::DE::MCA_ADDR_DE[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::DE::MCA_ADDR_DE[55:12] contain a valid 4KB memory page and that MCA::DE::MCA_ADDR_DE[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::DE::MCA_CTL_DE enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 67: MCA_STATUS_DE

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
OcTag	0x0	0	0	0	0	0	0
OcDat	0x1	0	0	0	0	0	0
Ibq	0x2	1	1	1	0	0	0
UopQ	0x3	1	1	1	0	0	0
Idq	0x4	1	1	1	0	0	0
Faq	0x5	1	1	1	0	0	0
UcDat	0x6	1	1	1	0	0	0
UcSeq	0x7	1	1	1	0	0	0
OCBQ	0x8	1	1	1	0	0	0
HwAssertMc a	0x9	1	1	1	0	0	0

MSR0000_040E...MSRC000_2032 [DE Machine Check Address Thread 0] (MCA::DE::MCA_ADDR_DE)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::DE::MCA_ADDR_DE stores an address and other information associated with the error in MCA::DE::MCA_STATUS_DE. The register is only meaningful if MCA::DE::MCA_STATUS_DE[Val]=1 and MCA::DE::MCA_STATUS_DE[AddrV]=1.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core0_thread0_inst3_n0_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSRLEGACY; MSR0000_040E
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2032
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSRLEGACY; MSR0000_040E
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2032
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSRLEGACY; MSR0000_040E

_ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2032
_ccd10_lthree0_core6_thread1_inst3_n173_aliasMSRLEGACY; MSR0000_040E
_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2032
_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSRLEGACY; MSR0000_040E
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2032
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2032
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2032
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2032
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2032
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2032
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2032
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2032
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2032
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2032
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2032
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2032
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2032
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2032
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2032
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2032
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSRLEGACY; MSR0000_040E
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2032
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2032
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2032
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2032
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2032
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2032
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2032
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2032
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2032
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2032
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2032
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2032
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2032
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2032
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2032

_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2032
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSRLEGACY; MSR0000_040E
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2032
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2032
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2032
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2032
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2032
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2032
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2032
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2032
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2032
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2032
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2032
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2032
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2032
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2032
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2032
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2032
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSRLEGACY; MSR0000_040E
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2032
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2032
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2032
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2032
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2032
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2032
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2032
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2032
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2032
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2032
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2032
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2032
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2032
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2032
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2032
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSRLEGACY; MSR0000_040E

_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2032
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSRLEGACY; MSR0000_040E
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2032
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2032
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2032
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2032
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2032
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2032
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2032
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2032
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2032
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2032
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2032
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2032
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2032
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2032
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2032
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2032
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSRLEGACY; MSR0000_040E
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2032
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2032
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2032
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2032
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2032
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2032
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2032
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2032
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2032
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2032
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2032
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2032
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2032
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2032
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2032
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2032

_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSRLEGACY; MSR0000_040E
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2032
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2032
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2032
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2032
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2032
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2032
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2032
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2032
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2032
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2032
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2032
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2032
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2032
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2032
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2032
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2032
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSRLEGACY; MSR0000_040E
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2032
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2032
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2032
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2032
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2032
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2032
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2032
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2032
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2032
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2032
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2032
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2032
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2032
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2032
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2032
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSRLEGACY; MSR0000_040E
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2032
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSRLEGACY; MSR0000_040E

_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2032
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2032
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2032
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2032
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2032
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2032
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2032
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2032
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2032
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2032
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2032
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2032
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2032
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2032
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2032
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2032
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSRLEGACY; MSR0000_040E
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2032
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2032
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2032
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2032
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2032
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2032
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2032
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2032
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2032
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2032
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2032
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2032
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2032
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2032
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2032
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2032
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSRLEGACY; MSR0000_040E
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2032

_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSRLEGACY; MSR0000_040E	
Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::DE::MCA_STATUS_DE.

Table 68: MCA_ADDR_DE

Error Type	Bits	Description
OcTag	[55:0]	Reserved
OcDat	[55:0]	Reserved
Ibq	[55:0]	Reserved
UopQ	[55:0]	Reserved
Idq	[55:0]	Reserved
Faq	[55:0]	Reserved
UcDat	[55:0]	Reserved
UcSeq	[55:0]	Reserved
OCBQ	[55:0]	Reserved
HwAssertMca	[55:0]	Reserved

**MSR0000_040F...MSRC000_2033 [DE Machine Check Miscellaneous 0 Thread 0]
(MCA::DE::MCA_MISC0_DE)**

Log miscellaneous information associated with errors.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2033
_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2033
_ccd0_lthree0_core0_thread1_inst3_n1_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2033
_ccd0_lthree0_core1_thread0_inst3_n2_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2033
_ccd0_lthree0_core1_thread1_inst3_n3_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2033
_ccd0_lthree0_core2_thread0_inst3_n4_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2033
_ccd0_lthree0_core2_thread1_inst3_n5_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2033
_ccd0_lthree0_core3_thread0_inst3_n6_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2033
_ccd0_lthree0_core3_thread1_inst3_n7_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2033
_ccd0_lthree0_core4_thread0_inst3_n8_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2033
_ccd0_lthree0_core4_thread1_inst3_n9_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2033
_ccd0_lthree0_core5_thread0_inst3_n10_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2033
_ccd0_lthree0_core5_thread1_inst3_n11_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2033
_ccd0_lthree0_core6_thread0_inst3_n12_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2033
_ccd0_lthree0_core6_thread1_inst3_n13_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2033
_ccd0_lthree0_core7_thread0_inst3_n14_aliasMSRLEGACY; MSR0000_040F
_ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2033
_ccd0_lthree0_core7_thread1_inst3_n15_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2033
_ccd10_lthree0_core0_thread0_inst3_n160_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2033
_ccd10_lthree0_core0_thread1_inst3_n161_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2033
_ccd10_lthree0_core1_thread0_inst3_n162_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2033
_ccd10_lthree0_core1_thread1_inst3_n163_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2033
_ccd10_lthree0_core2_thread0_inst3_n164_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2033
_ccd10_lthree0_core2_thread1_inst3_n165_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2033
_ccd10_lthree0_core3_thread0_inst3_n166_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2033
_ccd10_lthree0_core3_thread1_inst3_n167_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2033
_ccd10_lthree0_core4_thread0_inst3_n168_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2033
_ccd10_lthree0_core4_thread1_inst3_n169_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2033
_ccd10_lthree0_core5_thread0_inst3_n170_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2033
_ccd10_lthree0_core5_thread1_inst3_n171_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2033
_ccd10_lthree0_core6_thread0_inst3_n172_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2033
_ccd10_lthree0_core6_thread1_inst3_n173_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2033

_ccd10_lthree0_core7_thread0_inst3_n174_aliasMSRLEGACY; MSR0000_040F
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2033
_ccd10_lthree0_core7_thread1_inst3_n175_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2033
_ccd11_lthree0_core0_thread0_inst3_n176_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2033
_ccd11_lthree0_core0_thread1_inst3_n177_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2033
_ccd11_lthree0_core1_thread0_inst3_n178_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2033
_ccd11_lthree0_core1_thread1_inst3_n179_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2033
_ccd11_lthree0_core2_thread0_inst3_n180_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2033
_ccd11_lthree0_core2_thread1_inst3_n181_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2033
_ccd11_lthree0_core3_thread0_inst3_n182_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2033
_ccd11_lthree0_core3_thread1_inst3_n183_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2033
_ccd11_lthree0_core4_thread0_inst3_n184_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2033
_ccd11_lthree0_core4_thread1_inst3_n185_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2033
_ccd11_lthree0_core5_thread0_inst3_n186_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2033
_ccd11_lthree0_core5_thread1_inst3_n187_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2033
_ccd11_lthree0_core6_thread0_inst3_n188_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2033
_ccd11_lthree0_core6_thread1_inst3_n189_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2033
_ccd11_lthree0_core7_thread0_inst3_n190_aliasMSRLEGACY; MSR0000_040F
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2033
_ccd11_lthree0_core7_thread1_inst3_n191_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2033
_ccd1_lthree0_core0_thread0_inst3_n16_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2033
_ccd1_lthree0_core0_thread1_inst3_n17_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2033
_ccd1_lthree0_core1_thread0_inst3_n18_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2033
_ccd1_lthree0_core1_thread1_inst3_n19_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2033
_ccd1_lthree0_core2_thread0_inst3_n20_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2033
_ccd1_lthree0_core2_thread1_inst3_n21_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2033
_ccd1_lthree0_core3_thread0_inst3_n22_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2033
_ccd1_lthree0_core3_thread1_inst3_n23_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2033
_ccd1_lthree0_core4_thread0_inst3_n24_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2033
_ccd1_lthree0_core4_thread1_inst3_n25_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2033
_ccd1_lthree0_core5_thread0_inst3_n26_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2033
_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2033
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2033
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSRLEGACY; MSR0000_040F
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2033
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSRLEGACY; MSR0000_040F

_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2033
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2033
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2033
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2033
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2033
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2033
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2033
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2033
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2033
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2033
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2033
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2033
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2033
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2033
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2033
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2033
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSRLEGACY; MSRC0000_040F
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2033
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2033
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2033
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2033
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2033
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2033
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2033
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2033
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2033
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2033
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2033
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2033
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2033
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2033
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2033
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2033
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSRLEGACY; MSRC0000_040F
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2033

_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2033
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2033
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2033
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2033
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2033
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2033
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2033
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2033
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2033
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2033
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2033
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2033
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2033
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2033
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2033
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSRLEGACY; MSR0000_040F
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2033
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2033
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2033
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2033
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2033
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2033
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2033
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2033
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2033
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2033
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2033
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2033
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2033
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2033
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2033
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2033
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSRLEGACY; MSR0000_040F
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2033
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSRLEGACY; MSR0000_040F

_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2033
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2033
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2033
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2033
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2033
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2033
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2033
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2033
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2033
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2033
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2033
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2033
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2033
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2033
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2033
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSRLEGACY; MSR0000_040F
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2033
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2033
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2033
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2033
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2033
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2033
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2033
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2033
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2033
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2033
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2033
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2033
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2033
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2033
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2033
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2033
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSRLEGACY; MSR0000_040F
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2033
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSRLEGACY; MSR0000_040F
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2033

_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSRLEGACY; MSR0000_040F	
ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSRLEGACY; MSR0000_040F	
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2033	
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSRLEGACY; MSR0000_040F	
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2033	
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSRLEGACY; MSR0000_040F	
Bits	Description

63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::DE::MCA_MISC0_DE[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::DE::MCA_MISC0_DE[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::DE::MCA_MISC0_DE[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::DE::MCA_MISC0_DE[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::DE::MCA_MISC0_DE[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::DE::MCA_MISC0_DE[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2034 [DE Machine Check Configuration Thread 0] (MCA::DE::MCA_CONFIG_DE)

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2034
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2034
 _ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2034
 _ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2034
 _ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2034

_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2034
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2034
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2034
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2034
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2034
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2034
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2034
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2034
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2034
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2034
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2034
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2034
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2034
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2034
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2034
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2034
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2034
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2034
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2034
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2034
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2034
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2034
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2034
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2034
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2034
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2034
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2034
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2034
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2034
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2034
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2034
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2034
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2034
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2034
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2034
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2034
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2034
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2034
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2034
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2034
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2034
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2034
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2034
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2034
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2034
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2034
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2034
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2034
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2034
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2034
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2034
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2034
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2034
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2034
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2034
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2034
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2034
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2034
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2034
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2034
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2034
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2034
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2034
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2034
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2034

_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2034
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2034
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2034
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2034
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2034
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2034
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2034
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2034
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2034
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2034
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2034
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2034
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2034
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2034
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2034
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2034
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2034
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2034
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2034
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2034
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2034
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2034
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2034
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2034
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2034
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2034
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2034
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2034
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2034
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2034
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2034
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2034
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2034
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2034
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2034
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2034
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2034
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2034
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2034
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2034
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2034
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2034
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2034
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2034
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2034
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2034
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2034
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2034
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2034
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2034
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2034
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2034
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2034
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2034
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2034
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2034
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2034
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2034
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2034
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2034
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2034
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2034
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2034
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2034
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2034

_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2034	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::DE::MCA_STATUS_DE, MCA::DE::MCA_ADDR_DE, MCA::DE::MCA_MISC0_DE, and MCA::DE::MCA_SYND_DE.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::DE::MCA_CONFIG_DE[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::DE::MCA_CONFIG_DE[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::DE::MCA_CONFIG_DE[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::DE::MCA_MISC0_DE[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::DE::MCA_STATUS_DE[TCC] is present.

MSRC000_2035 [DE IP Identification Thread 0] (MCA::DE::MCA_IPID_DE)

Reset: 0003_00B0_0000_0000h.

The MCA::DE::MCA_IPID_DE register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2035
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2035
 _ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2035
 _ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2035
 _ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2035

_ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2035
_ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2035
_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2035
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2035
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2035
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2035
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2035
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2035
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2035
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2035
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2035
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2035
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2035
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2035
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2035
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2035
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2035
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2035
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2035
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2035
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2035
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2035
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2035
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2035
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2035
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2035
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2035
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2035
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2035
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2035
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2035
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2035
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2035
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2035
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2035
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2035
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2035
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2035
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2035
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2035
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2035
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2035
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2035
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2035
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2035
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2035
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2035
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2035
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2035
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2035
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2035
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2035
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2035
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2035
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2035
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2035
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2035
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2035
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2035
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2035
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2035
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2035
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2035
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2035
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2035

_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2035
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2035
_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2035
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2035
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2035
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2035
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2035
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2035
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2035
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2035
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2035
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2035
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2035
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2035
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2035
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2035
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2035
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2035
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2035
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2035
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2035
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2035
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2035
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2035
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2035
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2035
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2035
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2035
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2035
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2035
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2035
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2035
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2035
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2035
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2035
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2035
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2035
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2035
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2035
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2035
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2035
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2035
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2035
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2035
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2035
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2035
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2035
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2035
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2035
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2035
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2035
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2035
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2035
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2035
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2035
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2035
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2035
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2035
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2035
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2035
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2035
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2035
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2035
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2035
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2035

_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2035	
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2035	
_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2035	
Bits	Description
63:48	McaType . Read-only. Reset: 0003h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR: 2000_9600h Init: _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR: 2000_9700h Init: _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR: 2002_9600h Init: _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR: 2002_9700h Init: _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR: 2004_9600h Init: _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR: 2004_9700h Init: _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR: 2006_9600h Init: _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR: 2006_9700h Init: _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR: 2008_9600h Init: _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR: 2008_9700h Init: _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR: 200A_9600h Init: _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR: 200A_9700h Init: _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR: 200C_9600h Init: _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR: 200C_9700h Init: _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR: 200E_9600h Init: _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR: 200E_9700h Init: _ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR: 2080_9600h Init: _ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR: 2080_9700h Init: _ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR: 2082_9600h Init: _ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR: 2082_9700h Init: _ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR: 2084_9600h Init: _ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR: 2084_9700h Init: _ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR: 2086_9600h Init: _ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR: 2086_9700h Init: _ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR: 2088_9600h Init: _ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR: 2088_9700h Init: _ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR: 208A_9600h Init: _ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR: 208A_9700h Init: _ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR: 208C_9600h Init: _ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR: 208C_9700h Init: _ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR: 208E_9600h Init: _ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR: 208E_9700h Init: _ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR: 2100_9600h Init: _ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR: 2100_9700h Init: _ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR: 2102_9600h Init: _ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR: 2102_9700h Init: _ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR: 2104_9600h Init: _ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR: 2104_9700h Init: _ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR: 2106_9600h Init: _ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR: 2106_9700h Init: _ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR: 2108_9600h Init: _ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR: 2108_9700h Init: _ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR: 210A_9600h Init: _ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR: 210A_9700h Init: _ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR: 210C_9600h Init: _ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR: 210C_9700h Init: _ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR: 210E_9600h Init: _ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR: 210E_9700h Init: _ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR: 2180_9600h Init: _ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR: 2180_9700h Init: _ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR: 2182_9600h Init: _ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR: 2182_9700h Init: _ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR: 2184_9600h

Init: _ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR: 2184_9700h
Init: _ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR: 2186_9600h
Init: _ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR: 2186_9700h
Init: _ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR: 2188_9600h
Init: _ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR: 2188_9700h
Init: _ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR: 218A_9600h
Init: _ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR: 218A_9700h
Init: _ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR: 218C_9600h
Init: _ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR: 218C_9700h
Init: _ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR: 218E_9600h
Init: _ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR: 218E_9700h
Init: _ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR: 2200_9600h
Init: _ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR: 2200_9700h
Init: _ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR: 2202_9600h
Init: _ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR: 2202_9700h
Init: _ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR: 2204_9600h
Init: _ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR: 2204_9700h
Init: _ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR: 2206_9600h
Init: _ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR: 2206_9700h
Init: _ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR: 2208_9600h
Init: _ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR: 2208_9700h
Init: _ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR: 220A_9600h
Init: _ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR: 220A_9700h
Init: _ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR: 220C_9600h
Init: _ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR: 220C_9700h
Init: _ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR: 220E_9600h
Init: _ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR: 220E_9700h
Init: _ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR: 2280_9600h
Init: _ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR: 2280_9700h
Init: _ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR: 2282_9600h
Init: _ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR: 2282_9700h
Init: _ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR: 2284_9600h
Init: _ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR: 2284_9700h
Init: _ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR: 2286_9600h
Init: _ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR: 2286_9700h
Init: _ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR: 2288_9600h
Init: _ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR: 2288_9700h
Init: _ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR: 228A_9600h
Init: _ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR: 228A_9700h
Init: _ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR: 228C_9600h
Init: _ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR: 228C_9700h
Init: _ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR: 228E_9600h
Init: _ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR: 228E_9700h
Init: _ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR: 2300_9600h
Init: _ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR: 2300_9700h
Init: _ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR: 2302_9600h
Init: _ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR: 2302_9700h
Init: _ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR: 2304_9600h
Init: _ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR: 2304_9700h
Init: _ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR: 2306_9600h
Init: _ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR: 2306_9700h
Init: _ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR: 2308_9600h
Init: _ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR: 2308_9700h
Init: _ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR: 230A_9600h
Init: _ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR: 230A_9700h
Init: _ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR: 230C_9600h
Init: _ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR: 230C_9700h
Init: _ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR: 230E_9600h
Init: _ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR: 230E_9700h
Init: _ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR: 2380_9600h
Init: _ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR: 2380_9700h
Init: _ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR: 2382_9600h
Init: _ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR: 2382_9700h
Init: _ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR: 2384_9600h
Init: _ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR: 2384_9700h

Init: _ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR: 2386_9600h
Init: _ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR: 2386_9700h
Init: _ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR: 2388_9600h
Init: _ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR: 2388_9700h
Init: _ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR: 238A_9600h
Init: _ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR: 238A_9700h
Init: _ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR: 238C_9600h
Init: _ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR: 238C_9700h
Init: _ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR: 238E_9600h
Init: _ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR: 238E_9700h
Init: _ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR: 2400_9600h
Init: _ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR: 2400_9700h
Init: _ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR: 2402_9600h
Init: _ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR: 2402_9700h
Init: _ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR: 2404_9600h
Init: _ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR: 2404_9700h
Init: _ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR: 2406_9600h
Init: _ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR: 2406_9700h
Init: _ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR: 2408_9600h
Init: _ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR: 2408_9700h
Init: _ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR: 240A_9600h
Init: _ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR: 240A_9700h
Init: _ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR: 240C_9600h
Init: _ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR: 240C_9700h
Init: _ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR: 240E_9600h
Init: _ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR: 240E_9700h
Init: _ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR: 2480_9600h
Init: _ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR: 2480_9700h
Init: _ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR: 2482_9600h
Init: _ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR: 2482_9700h
Init: _ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR: 2484_9600h
Init: _ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR: 2484_9700h
Init: _ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR: 2486_9600h
Init: _ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR: 2486_9700h
Init: _ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR: 2488_9600h
Init: _ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR: 2488_9700h
Init: _ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR: 248A_9600h
Init: _ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR: 248A_9700h
Init: _ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR: 248C_9600h
Init: _ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR: 248C_9700h
Init: _ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR: 248E_9600h
Init: _ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR: 248E_9700h
Init: _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR: 2500_9600h
Init: _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR: 2500_9700h
Init: _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR: 2502_9600h
Init: _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR: 2502_9700h
Init: _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR: 2504_9600h
Init: _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR: 2504_9700h
Init: _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR: 2506_9600h
Init: _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR: 2506_9700h
Init: _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR: 2508_9600h
Init: _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR: 2508_9700h
Init: _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR: 250A_9600h
Init: _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR: 250A_9700h
Init: _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR: 250C_9600h
Init: _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR: 250C_9700h
Init: _ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR: 250E_9600h
Init: _ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR: 250E_9700h
Init: _ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR: 2580_9600h
Init: _ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR: 2580_9700h
Init: _ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR: 2582_9600h
Init: _ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR: 2582_9700h
Init: _ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR: 2584_9600h
Init: _ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR: 2584_9700h
Init: _ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR: 2586_9600h

Init: _ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR: 2586_9700h
Init: _ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR: 2588_9600h
Init: _ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR: 2588_9700h
Init: _ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR: 258A_9600h
Init: _ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR: 258A_9700h
Init: _ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR: 258C_9600h
Init: _ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR: 258C_9700h
Init: _ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR: 258E_9600h
Init: _ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR: 258E_9700h

MSRC000_2036 [DE Machine Check Syndrome Thread 0] (MCA::DE::MCA_SYND_DE)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::DE::MCA_STATUS_DE [Thread 0](#)

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC000_2036
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC000_2036
 _ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC000_2036
 _ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC000_2036
 _ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC000_2036

_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC000_2036
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC000_2036
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC000_2036
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC000_2036
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC000_2036
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC000_2036
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC000_2036
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC000_2036
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC000_2036
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC000_2036
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC000_2036
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC000_2036
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC000_2036
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC000_2036
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC000_2036
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC000_2036
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC000_2036
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC000_2036
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC000_2036
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC000_2036
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC000_2036
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC000_2036
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC000_2036
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC000_2036
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC000_2036
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC000_2036
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC000_2036
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC000_2036
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC000_2036
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC000_2036
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC000_2036
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC000_2036
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC000_2036
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC000_2036
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC000_2036
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC000_2036
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC000_2036
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC000_2036
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC000_2036
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC000_2036
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC000_2036
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC000_2036
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC000_2036
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC000_2036
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC000_2036
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC000_2036
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC000_2036
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC000_2036
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC000_2036
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC000_2036
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC000_2036
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC000_2036
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC000_2036
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC000_2036
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC000_2036
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC000_2036
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC000_2036
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC000_2036
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC000_2036
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC000_2036
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC000_2036
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC000_2036
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC000_2036
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC000_2036
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC000_2036

_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC000_2036
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC000_2036
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC000_2036
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC000_2036
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC000_2036
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC000_2036
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC000_2036
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC000_2036
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC000_2036
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC000_2036
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC000_2036
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC000_2036
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC000_2036
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC000_2036
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC000_2036
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC000_2036
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC000_2036
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC000_2036
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC000_2036
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC000_2036
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC000_2036
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC000_2036
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC000_2036
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC000_2036
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC000_2036
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC000_2036
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC000_2036
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC000_2036
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC000_2036
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC000_2036
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC000_2036
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC000_2036
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC000_2036
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC000_2036
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC000_2036
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC000_2036
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC000_2036
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC000_2036
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC000_2036
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC000_2036
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC000_2036
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC000_2036
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC000_2036
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC000_2036
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC000_2036
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC000_2036
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC000_2036
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC000_2036
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC000_2036
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC000_2036
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC000_2036
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC000_2036
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC000_2036
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC000_2036
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC000_2036
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC000_2036
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC000_2036
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC000_2036
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC000_2036
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC000_2036
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC000_2036
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC000_2036
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC000_2036
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC000_2036
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC000_2036

_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC000_2036	
Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::DE::MCA_STATUS_DE. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::DE::MCA_SYND_DE[Length]. The Syndrome field is only valid when MCA::DE::MCA_SYND_DE[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::DE::MCA_SYND_DE. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::DE::MCA_SYND_DE[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::DE::MCA_SYND_DE. For example, a syndrome length of 9 means that MCA::DE::MCA_SYND_DE[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 69 [MCA_SYND_DE].

Table 69: MCA_SYND_DE

Error Type	Bits	Description
OcTag	[17:0]	Reserved
OcDat	[17:0]	Reserved
Ibq	[17:0]	Reserved
UopQ	[17:0]	Reserved
Idq	[17:0]	Reserved
Faq	[17:0]	Reserved
UcDat	[17:0]	Reserved
UcSeq	[17:0]	Reserved
OCBQ	[17:0]	Reserved
HwAssertMca	[17:0]	Reserved

MSRC001_0403 [DE Machine Check Control Mask Thread 0] (MCA::DE::MCA_CTL_MASK_DE)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_core0_thread0_inst3_n0_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core0_thread1_inst3_n1_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core1_thread0_inst3_n2_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core1_thread1_inst3_n3_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core2_thread0_inst3_n4_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core2_thread1_inst3_n5_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core3_thread0_inst3_n6_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core3_thread1_inst3_n7_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core4_thread0_inst3_n8_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core4_thread1_inst3_n9_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core5_thread0_inst3_n10_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core5_thread1_inst3_n11_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core6_thread0_inst3_n12_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core6_thread1_inst3_n13_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core7_thread0_inst3_n14_aliasMSR; MSRC001_0403
 _ccd0_lthree0_core7_thread1_inst3_n15_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core0_thread0_inst3_n160_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core0_thread1_inst3_n161_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core1_thread0_inst3_n162_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core1_thread1_inst3_n163_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core2_thread0_inst3_n164_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core2_thread1_inst3_n165_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core3_thread0_inst3_n166_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core3_thread1_inst3_n167_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core4_thread0_inst3_n168_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core4_thread1_inst3_n169_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core5_thread0_inst3_n170_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core5_thread1_inst3_n171_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core6_thread0_inst3_n172_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core6_thread1_inst3_n173_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core7_thread0_inst3_n174_aliasMSR; MSRC001_0403
 _ccd10_lthree0_core7_thread1_inst3_n175_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core0_thread0_inst3_n176_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core0_thread1_inst3_n177_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core1_thread0_inst3_n178_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core1_thread1_inst3_n179_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core2_thread0_inst3_n180_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core2_thread1_inst3_n181_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core3_thread0_inst3_n182_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core3_thread1_inst3_n183_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core4_thread0_inst3_n184_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core4_thread1_inst3_n185_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core5_thread0_inst3_n186_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core5_thread1_inst3_n187_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core6_thread0_inst3_n188_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core6_thread1_inst3_n189_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core7_thread0_inst3_n190_aliasMSR; MSRC001_0403
 _ccd11_lthree0_core7_thread1_inst3_n191_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core0_thread0_inst3_n16_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core0_thread1_inst3_n17_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core1_thread0_inst3_n18_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core1_thread1_inst3_n19_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core2_thread0_inst3_n20_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core2_thread1_inst3_n21_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core3_thread0_inst3_n22_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core3_thread1_inst3_n23_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core4_thread0_inst3_n24_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core4_thread1_inst3_n25_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core5_thread0_inst3_n26_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core5_thread1_inst3_n27_aliasMSR; MSRC001_0403
 _ccd1_lthree0_core6_thread0_inst3_n28_aliasMSR; MSRC001_0403

_ccd1_lthree0_core6_thread1_inst3_n29_aliasMSR; MSRC001_0403
_ccd1_lthree0_core7_thread0_inst3_n30_aliasMSR; MSRC001_0403
_ccd1_lthree0_core7_thread1_inst3_n31_aliasMSR; MSRC001_0403
_ccd2_lthree0_core0_thread0_inst3_n32_aliasMSR; MSRC001_0403
_ccd2_lthree0_core0_thread1_inst3_n33_aliasMSR; MSRC001_0403
_ccd2_lthree0_core1_thread0_inst3_n34_aliasMSR; MSRC001_0403
_ccd2_lthree0_core1_thread1_inst3_n35_aliasMSR; MSRC001_0403
_ccd2_lthree0_core2_thread0_inst3_n36_aliasMSR; MSRC001_0403
_ccd2_lthree0_core2_thread1_inst3_n37_aliasMSR; MSRC001_0403
_ccd2_lthree0_core3_thread0_inst3_n38_aliasMSR; MSRC001_0403
_ccd2_lthree0_core3_thread1_inst3_n39_aliasMSR; MSRC001_0403
_ccd2_lthree0_core4_thread0_inst3_n40_aliasMSR; MSRC001_0403
_ccd2_lthree0_core4_thread1_inst3_n41_aliasMSR; MSRC001_0403
_ccd2_lthree0_core5_thread0_inst3_n42_aliasMSR; MSRC001_0403
_ccd2_lthree0_core5_thread1_inst3_n43_aliasMSR; MSRC001_0403
_ccd2_lthree0_core6_thread0_inst3_n44_aliasMSR; MSRC001_0403
_ccd2_lthree0_core6_thread1_inst3_n45_aliasMSR; MSRC001_0403
_ccd2_lthree0_core7_thread0_inst3_n46_aliasMSR; MSRC001_0403
_ccd2_lthree0_core7_thread1_inst3_n47_aliasMSR; MSRC001_0403
_ccd3_lthree0_core0_thread0_inst3_n48_aliasMSR; MSRC001_0403
_ccd3_lthree0_core0_thread1_inst3_n49_aliasMSR; MSRC001_0403
_ccd3_lthree0_core1_thread0_inst3_n50_aliasMSR; MSRC001_0403
_ccd3_lthree0_core1_thread1_inst3_n51_aliasMSR; MSRC001_0403
_ccd3_lthree0_core2_thread0_inst3_n52_aliasMSR; MSRC001_0403
_ccd3_lthree0_core2_thread1_inst3_n53_aliasMSR; MSRC001_0403
_ccd3_lthree0_core3_thread0_inst3_n54_aliasMSR; MSRC001_0403
_ccd3_lthree0_core3_thread1_inst3_n55_aliasMSR; MSRC001_0403
_ccd3_lthree0_core4_thread0_inst3_n56_aliasMSR; MSRC001_0403
_ccd3_lthree0_core4_thread1_inst3_n57_aliasMSR; MSRC001_0403
_ccd3_lthree0_core5_thread0_inst3_n58_aliasMSR; MSRC001_0403
_ccd3_lthree0_core5_thread1_inst3_n59_aliasMSR; MSRC001_0403
_ccd3_lthree0_core6_thread0_inst3_n60_aliasMSR; MSRC001_0403
_ccd3_lthree0_core6_thread1_inst3_n61_aliasMSR; MSRC001_0403
_ccd3_lthree0_core7_thread0_inst3_n62_aliasMSR; MSRC001_0403
_ccd3_lthree0_core7_thread1_inst3_n63_aliasMSR; MSRC001_0403
_ccd4_lthree0_core0_thread0_inst3_n64_aliasMSR; MSRC001_0403
_ccd4_lthree0_core0_thread1_inst3_n65_aliasMSR; MSRC001_0403
_ccd4_lthree0_core1_thread0_inst3_n66_aliasMSR; MSRC001_0403
_ccd4_lthree0_core1_thread1_inst3_n67_aliasMSR; MSRC001_0403
_ccd4_lthree0_core2_thread0_inst3_n68_aliasMSR; MSRC001_0403
_ccd4_lthree0_core2_thread1_inst3_n69_aliasMSR; MSRC001_0403
_ccd4_lthree0_core3_thread0_inst3_n70_aliasMSR; MSRC001_0403
_ccd4_lthree0_core3_thread1_inst3_n71_aliasMSR; MSRC001_0403
_ccd4_lthree0_core4_thread0_inst3_n72_aliasMSR; MSRC001_0403
_ccd4_lthree0_core4_thread1_inst3_n73_aliasMSR; MSRC001_0403
_ccd4_lthree0_core5_thread0_inst3_n74_aliasMSR; MSRC001_0403
_ccd4_lthree0_core5_thread1_inst3_n75_aliasMSR; MSRC001_0403
_ccd4_lthree0_core6_thread0_inst3_n76_aliasMSR; MSRC001_0403
_ccd4_lthree0_core6_thread1_inst3_n77_aliasMSR; MSRC001_0403
_ccd4_lthree0_core7_thread0_inst3_n78_aliasMSR; MSRC001_0403
_ccd4_lthree0_core7_thread1_inst3_n79_aliasMSR; MSRC001_0403
_ccd5_lthree0_core0_thread0_inst3_n80_aliasMSR; MSRC001_0403
_ccd5_lthree0_core0_thread1_inst3_n81_aliasMSR; MSRC001_0403
_ccd5_lthree0_core1_thread0_inst3_n82_aliasMSR; MSRC001_0403
_ccd5_lthree0_core1_thread1_inst3_n83_aliasMSR; MSRC001_0403
_ccd5_lthree0_core2_thread0_inst3_n84_aliasMSR; MSRC001_0403
_ccd5_lthree0_core2_thread1_inst3_n85_aliasMSR; MSRC001_0403
_ccd5_lthree0_core3_thread0_inst3_n86_aliasMSR; MSRC001_0403
_ccd5_lthree0_core3_thread1_inst3_n87_aliasMSR; MSRC001_0403
_ccd5_lthree0_core4_thread0_inst3_n88_aliasMSR; MSRC001_0403
_ccd5_lthree0_core4_thread1_inst3_n89_aliasMSR; MSRC001_0403
_ccd5_lthree0_core5_thread0_inst3_n90_aliasMSR; MSRC001_0403
_ccd5_lthree0_core5_thread1_inst3_n91_aliasMSR; MSRC001_0403
_ccd5_lthree0_core6_thread0_inst3_n92_aliasMSR; MSRC001_0403
_ccd5_lthree0_core6_thread1_inst3_n93_aliasMSR; MSRC001_0403

_ccd5_lthree0_core7_thread0_inst3_n94_aliasMSR; MSRC001_0403
_ccd5_lthree0_core7_thread1_inst3_n95_aliasMSR; MSRC001_0403
_ccd6_lthree0_core0_thread0_inst3_n96_aliasMSR; MSRC001_0403
_ccd6_lthree0_core0_thread1_inst3_n97_aliasMSR; MSRC001_0403
_ccd6_lthree0_core1_thread0_inst3_n98_aliasMSR; MSRC001_0403
_ccd6_lthree0_core1_thread1_inst3_n99_aliasMSR; MSRC001_0403
_ccd6_lthree0_core2_thread0_inst3_n100_aliasMSR; MSRC001_0403
_ccd6_lthree0_core2_thread1_inst3_n101_aliasMSR; MSRC001_0403
_ccd6_lthree0_core3_thread0_inst3_n102_aliasMSR; MSRC001_0403
_ccd6_lthree0_core3_thread1_inst3_n103_aliasMSR; MSRC001_0403
_ccd6_lthree0_core4_thread0_inst3_n104_aliasMSR; MSRC001_0403
_ccd6_lthree0_core4_thread1_inst3_n105_aliasMSR; MSRC001_0403
_ccd6_lthree0_core5_thread0_inst3_n106_aliasMSR; MSRC001_0403
_ccd6_lthree0_core5_thread1_inst3_n107_aliasMSR; MSRC001_0403
_ccd6_lthree0_core6_thread0_inst3_n108_aliasMSR; MSRC001_0403
_ccd6_lthree0_core6_thread1_inst3_n109_aliasMSR; MSRC001_0403
_ccd6_lthree0_core7_thread0_inst3_n110_aliasMSR; MSRC001_0403
_ccd6_lthree0_core7_thread1_inst3_n111_aliasMSR; MSRC001_0403
_ccd7_lthree0_core0_thread0_inst3_n112_aliasMSR; MSRC001_0403
_ccd7_lthree0_core0_thread1_inst3_n113_aliasMSR; MSRC001_0403
_ccd7_lthree0_core1_thread0_inst3_n114_aliasMSR; MSRC001_0403
_ccd7_lthree0_core1_thread1_inst3_n115_aliasMSR; MSRC001_0403
_ccd7_lthree0_core2_thread0_inst3_n116_aliasMSR; MSRC001_0403
_ccd7_lthree0_core2_thread1_inst3_n117_aliasMSR; MSRC001_0403
_ccd7_lthree0_core3_thread0_inst3_n118_aliasMSR; MSRC001_0403
_ccd7_lthree0_core3_thread1_inst3_n119_aliasMSR; MSRC001_0403
_ccd7_lthree0_core4_thread0_inst3_n120_aliasMSR; MSRC001_0403
_ccd7_lthree0_core4_thread1_inst3_n121_aliasMSR; MSRC001_0403
_ccd7_lthree0_core5_thread0_inst3_n122_aliasMSR; MSRC001_0403
_ccd7_lthree0_core5_thread1_inst3_n123_aliasMSR; MSRC001_0403
_ccd7_lthree0_core6_thread0_inst3_n124_aliasMSR; MSRC001_0403
_ccd7_lthree0_core6_thread1_inst3_n125_aliasMSR; MSRC001_0403
_ccd7_lthree0_core7_thread0_inst3_n126_aliasMSR; MSRC001_0403
_ccd7_lthree0_core7_thread1_inst3_n127_aliasMSR; MSRC001_0403
_ccd8_lthree0_core0_thread0_inst3_n128_aliasMSR; MSRC001_0403
_ccd8_lthree0_core0_thread1_inst3_n129_aliasMSR; MSRC001_0403
_ccd8_lthree0_core1_thread0_inst3_n130_aliasMSR; MSRC001_0403
_ccd8_lthree0_core1_thread1_inst3_n131_aliasMSR; MSRC001_0403
_ccd8_lthree0_core2_thread0_inst3_n132_aliasMSR; MSRC001_0403
_ccd8_lthree0_core2_thread1_inst3_n133_aliasMSR; MSRC001_0403
_ccd8_lthree0_core3_thread0_inst3_n134_aliasMSR; MSRC001_0403
_ccd8_lthree0_core3_thread1_inst3_n135_aliasMSR; MSRC001_0403
_ccd8_lthree0_core4_thread0_inst3_n136_aliasMSR; MSRC001_0403
_ccd8_lthree0_core4_thread1_inst3_n137_aliasMSR; MSRC001_0403
_ccd8_lthree0_core5_thread0_inst3_n138_aliasMSR; MSRC001_0403
_ccd8_lthree0_core5_thread1_inst3_n139_aliasMSR; MSRC001_0403
_ccd8_lthree0_core6_thread0_inst3_n140_aliasMSR; MSRC001_0403
_ccd8_lthree0_core6_thread1_inst3_n141_aliasMSR; MSRC001_0403
_ccd8_lthree0_core7_thread0_inst3_n142_aliasMSR; MSRC001_0403
_ccd8_lthree0_core7_thread1_inst3_n143_aliasMSR; MSRC001_0403
_ccd9_lthree0_core0_thread0_inst3_n144_aliasMSR; MSRC001_0403
_ccd9_lthree0_core0_thread1_inst3_n145_aliasMSR; MSRC001_0403
_ccd9_lthree0_core1_thread0_inst3_n146_aliasMSR; MSRC001_0403
_ccd9_lthree0_core1_thread1_inst3_n147_aliasMSR; MSRC001_0403
_ccd9_lthree0_core2_thread0_inst3_n148_aliasMSR; MSRC001_0403
_ccd9_lthree0_core2_thread1_inst3_n149_aliasMSR; MSRC001_0403
_ccd9_lthree0_core3_thread0_inst3_n150_aliasMSR; MSRC001_0403
_ccd9_lthree0_core3_thread1_inst3_n151_aliasMSR; MSRC001_0403
_ccd9_lthree0_core4_thread0_inst3_n152_aliasMSR; MSRC001_0403
_ccd9_lthree0_core4_thread1_inst3_n153_aliasMSR; MSRC001_0403
_ccd9_lthree0_core5_thread0_inst3_n154_aliasMSR; MSRC001_0403
_ccd9_lthree0_core5_thread1_inst3_n155_aliasMSR; MSRC001_0403
_ccd9_lthree0_core6_thread0_inst3_n156_aliasMSR; MSRC001_0403
_ccd9_lthree0_core6_thread1_inst3_n157_aliasMSR; MSRC001_0403
_ccd9_lthree0_core7_thread0_inst3_n158_aliasMSR; MSRC001_0403

_ccd9_lthree0_core7_thread1_inst3_n159_aliasMSR; MSRC001_0403	
Bits	Description
63:10	Reserved.
9	HwAssertMca. Read-write. Reset: 0. Hardware Assertion MCA Error
8	OCBQ. Read-write. Reset: 0. Micro-op buffer parity error.
7	UcSeq. Read-write. Reset: 0. Patch RAM sequencer parity error
6	UcDat. Read-write. Reset: 0. Patch RAM data parity error
5	Faq. Read-write. Reset: 0. Fetch address FIFO parity error
4	Idq. Read-write. Reset: 0. Instruction dispatch queue parity error
3	UopQ. Read-write. Reset: 0. Micro-op Queue parity error
2	Ibq. Read-write. Reset: 0. IBB Register File parity error
1	OcDat. Read-write. Reset: 0. Micro-op cache DATA Array parity error
0	OcTag. Read-write. Reset: 0. Micro-op cache TAG Array parity error

3.2.5.5 EX

MSR0000_0414...MSRC000_2050 [EX Machine Check Control Thread 0] (MCA::EX::MCA_CTL_EX)

Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::EX::MCA_CTL_EX register must be enabled by the corresponding enable bit in Core::X86::Msrr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2050
_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2050
_ccd0_lthree0_core0_thread1_inst5_n1_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2050
_ccd0_lthree0_core1_thread0_inst5_n2_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2050
_ccd0_lthree0_core1_thread1_inst5_n3_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2050
_ccd0_lthree0_core2_thread0_inst5_n4_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2050
_ccd0_lthree0_core2_thread1_inst5_n5_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2050
_ccd0_lthree0_core3_thread0_inst5_n6_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2050
_ccd0_lthree0_core3_thread1_inst5_n7_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2050
_ccd0_lthree0_core4_thread0_inst5_n8_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2050
_ccd0_lthree0_core4_thread1_inst5_n9_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2050
_ccd0_lthree0_core5_thread0_inst5_n10_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2050
_ccd0_lthree0_core5_thread1_inst5_n11_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2050
_ccd0_lthree0_core6_thread0_inst5_n12_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2050
_ccd0_lthree0_core6_thread1_inst5_n13_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2050
_ccd0_lthree0_core7_thread0_inst5_n14_aliasMSRLEGACY; MSR0000_0414
_ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2050
_ccd0_lthree0_core7_thread1_inst5_n15_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2050
_ccd10_lthree0_core0_thread0_inst5_n160_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2050
_ccd10_lthree0_core0_thread1_inst5_n161_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2050
_ccd10_lthree0_core1_thread0_inst5_n162_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2050
_ccd10_lthree0_core1_thread1_inst5_n163_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2050
_ccd10_lthree0_core2_thread0_inst5_n164_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2050
_ccd10_lthree0_core2_thread1_inst5_n165_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2050
_ccd10_lthree0_core3_thread0_inst5_n166_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2050
_ccd10_lthree0_core3_thread1_inst5_n167_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2050
_ccd10_lthree0_core4_thread0_inst5_n168_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2050
_ccd10_lthree0_core4_thread1_inst5_n169_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2050
_ccd10_lthree0_core5_thread0_inst5_n170_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2050
_ccd10_lthree0_core5_thread1_inst5_n171_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2050
_ccd10_lthree0_core6_thread0_inst5_n172_aliasMSRLEGACY; MSR0000_0414

_ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2050
_ccd10_lthree0_core6_thread1_inst5_n173_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2050
_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSRLEGACY; MSR0000_0414
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2050
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2050
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2050
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2050
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2050
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2050
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2050
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2050
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2050
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2050
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2050
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2050
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2050
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2050
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2050
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2050
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSRLEGACY; MSR0000_0414
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2050
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2050
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2050
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2050
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2050
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2050
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2050
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2050
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2050
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2050
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2050
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2050
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2050
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2050
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2050

_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2050
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSRLEGACY; MSR0000_0414
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2050
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2050
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2050
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2050
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2050
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2050
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2050
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2050
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2050
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2050
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2050
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2050
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2050
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2050
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2050
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2050
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSRLEGACY; MSR0000_0414
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2050
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2050
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2050
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2050
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2050
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2050
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2050
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2050
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2050
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2050
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2050
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2050
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2050
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2050
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2050
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSRLEGACY; MSR0000_0414

_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2050
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSRLEGACY; MSR0000_0414
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2050
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2050
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2050
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2050
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2050
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2050
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2050
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2050
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2050
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2050
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2050
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2050
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2050
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2050
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2050
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2050
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSRLEGACY; MSR0000_0414
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2050
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2050
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2050
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2050
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2050
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2050
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2050
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2050
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2050
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2050
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2050
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2050
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2050
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2050
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2050
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2050

_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSRLEGACY; MSR0000_0414
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2050
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2050
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2050
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2050
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2050
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2050
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2050
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2050
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2050
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2050
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2050
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2050
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2050
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2050
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2050
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2050
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSRLEGACY; MSR0000_0414
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2050
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2050
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2050
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2050
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2050
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2050
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2050
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2050
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2050
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2050
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2050
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2050
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2050
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2050
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2050
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSRLEGACY; MSR0000_0414
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2050
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSRLEGACY; MSR0000_0414

_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2050
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2050
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2050
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2050
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2050
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2050
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2050
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2050
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2050
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2050
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2050
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2050
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2050
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2050
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2050
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2050
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSRLEGACY; MSR0000_0414
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2050
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2050
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2050
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2050
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2050
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2050
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2050
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2050
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2050
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2050
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2050
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2050
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2050
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2050
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2050
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2050
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSRLEGACY; MSR0000_0414
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2050

_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSRLEGACY; MSR0000_0414	
Bits	Description
63:14	Reserved.
13	RETMAP . Read-write. Reset: 0. Retire Map parity error.
12	SPECMAP . Read-write. Reset: 0. Spec Map parity error.
11	HWA . Read-write. Reset: 0. Hardware Assertion Error.
10	BBQ . Read-write. Reset: 0. Branch Buffer Queue (BBQ) parity error.
9	SQ . Read-write. Reset: 0. Scheduler Queue parity error.
8	STATQ . Read-write. Reset: 0. Retire status queue parity error.
7	RETDISP . Read-write. Reset: 0. Retire Dispatch Queue parity error.
6	CHKPTQ . Read-write. Reset: 0. Checkpoint Queue (AKA Map_DisbQ) parity error.
5	PLDAL . Read-write. Reset: 0. EX payload parity error.
4	PLDAG . Read-write. Reset: 0. Address generator payload parity error.
3	IDRF . Read-write. Reset: 0. Immediate displacement register file parity error.
2	FRF . Read-write. Reset: 0. Flag register file (FRF) parity error.
1	PRF . Read-write. Reset: 0. Physical register file (PRF) parity error.
0	WDT . Read-only. Reset: 0. Watchdog Timeout.

MSR0000_0415...MSRC000_2051 [EX Machine Check Status Thread 0] (MCA::EX::MCA_STATUS_EX)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core0_thread0_inst5_n0_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSRLEGACY; MSR0000_0415
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2051
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2051
 _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSRLEGACY; MSR0000_0415
 _ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2051

_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSRLEGACY; MSR0000_0415
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2051
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2051
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2051
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2051
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2051
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2051
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2051
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2051
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2051
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2051
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2051
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2051
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2051
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2051
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2051
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2051
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSRLEGACY; MSR0000_0415
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2051
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2051
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2051
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2051
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2051
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2051
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2051
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2051
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2051
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2051
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2051
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2051
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2051
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2051
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2051
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSRLEGACY; MSR0000_0415
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2051
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSRLEGACY; MSR0000_0415

_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2051
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2051
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2051
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2051
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2051
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2051
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2051
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2051
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2051
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2051
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2051
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2051
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2051
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2051
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2051
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2051
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSRLEGACY; MSR0000_0415
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2051
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2051
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2051
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2051
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2051
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2051
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2051
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2051
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2051
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2051
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2051
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2051
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2051
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2051
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2051
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2051
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSRLEGACY; MSR0000_0415
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2051

_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2051
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2051
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2051
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2051
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2051
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2051
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2051
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2051
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2051
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2051
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2051
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2051
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2051
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2051
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2051
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSRLEGACY; MSR0000_0415
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2051
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2051
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2051
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2051
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2051
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2051
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2051
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2051
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2051
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2051
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2051
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2051
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2051
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2051
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2051
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2051
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSRLEGACY; MSR0000_0415
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2051
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSRLEGACY; MSR0000_0415

_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2051
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2051
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2051
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2051
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2051
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2051
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2051
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2051
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2051
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2051
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2051
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2051
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2051
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2051
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2051
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSRLEGACY; MSR0000_0415
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2051
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2051
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2051
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2051
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2051
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2051
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2051
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2051
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2051
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2051
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2051
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2051
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2051
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2051
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2051
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2051
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSRLEGACY; MSR0000_0415
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2051
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSRLEGACY; MSR0000_0415
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2051

_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSRLEGACY; MSR0000_0415	
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2051	
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSRLEGACY; MSR0000_0415	
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2051	
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSRLEGACY; MSR0000_0415	
Bits	Description

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::EX::MCA_CTL_EX. This bit is a copy of bit in MCA::EX::MCA_CTL_EX for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::EX::MCA_MISC0_EX. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::EX::MCA_ADDR_EX contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::EX::MCA_STATUS_EX[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::EX::MCA_SYND_EX. If MCA::EX::MCA_SYND_EX[ErrorPriority] is the same as the priority of the error in MCA::EX::MCA_STATUS_EX, then the information in MCA::EX::MCA_SYND_EX is associated with the error in MCA::EX::MCA_STATUS_EX. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::EX::MCA_STATUS_EX[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::EX::MCA_ADDR_EX[ErrorAddr]. A value of 0 indicates that MCA::EX::MCA_ADDR_EX[56:0] contains a valid byte address. A value of 6 indicates that MCA::EX::MCA_ADDR_EX[56:6] contains a valid cache line address and that MCA::EX::MCA_ADDR_EX[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::EX::MCA_ADDR_EX[56:12] contain a valid 4KB memory page and that MCA::EX::MCA_ADDR_EX[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::EX::MCA_CTL_EX enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 70: MCA_STATUS_EX

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
WDT	0x0	1	1	1	0	0	1
PRF	0x1	1	1	1	0	0	0
FRF	0x2	1	1	1	0	0	0
IDRF	0x3	1	1	1	0	0	0
PLDAG	0x4	1	1	1	0	0	0
PLDAL	0x5	1	1	1	0	0	0
CHKPTQ	0x6	1	1	1	0	0	0
RETDISP	0x7	1	1	1	0	0	0
STATQ	0x8	1	1	1	0	0	0
SQ	0x9	1	1	1	0	0	0
BBQ	0xa	1	1	1	0	0	0
HWA	0xb	1	1	1	0	0	0
SPECMAP	0xc	1	1	1	0	0	0
RETMAP	0xd	1	1	1	0	0	0

MSR0000_0416...MSRC000_2052 [EX Machine Check Address Thread 0] (MCA::EX::MCA_ADDR_EX)

Reset: Cold,0000_0000_0000_0000h.

MCA::EX::MCA_ADDR_EX stores an address and other information associated with the error in MCA::EX::MCA_STATUS_EX. The register is only meaningful if MCA::EX::MCA_STATUS_EX[Val]=1 and MCA::EX::MCA_STATUS_EX[AddrV]=1.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core0_thread0_inst5_n0_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSRLEGACY; MSR0000_0416
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2052
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSRLEGACY; MSR0000_0416
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2052
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSRLEGACY; MSR0000_0416

_ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2052
_ccd10_lthree0_core6_thread1_inst5_n173_aliasMSRLEGACY; MSR0000_0416
_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2052
_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSRLEGACY; MSR0000_0416
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2052
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2052
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2052
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2052
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2052
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2052
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2052
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2052
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2052
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2052
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2052
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2052
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2052
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2052
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2052
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2052
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSRLEGACY; MSR0000_0416
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2052
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2052
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2052
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2052
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2052
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2052
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2052
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2052
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2052
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2052
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2052
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2052
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2052
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2052
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2052

_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2052
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSRLEGACY; MSR0000_0416
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2052
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2052
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2052
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2052
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2052
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2052
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2052
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2052
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2052
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2052
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2052
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2052
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2052
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2052
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2052
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2052
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSRLEGACY; MSR0000_0416
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2052
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2052
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2052
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2052
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2052
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2052
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2052
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2052
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2052
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2052
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2052
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2052
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2052
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2052
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2052
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSRLEGACY; MSR0000_0416

_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2052
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSRLEGACY; MSR0000_0416
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2052
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2052
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2052
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2052
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2052
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2052
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2052
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2052
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2052
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2052
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2052
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2052
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2052
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2052
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2052
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2052
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSRLEGACY; MSR0000_0416
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2052
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2052
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2052
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2052
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2052
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2052
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2052
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2052
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2052
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2052
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2052
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2052
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2052
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2052
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2052
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2052

_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSRLEGACY; MSR0000_0416
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2052
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2052
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2052
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2052
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2052
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2052
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2052
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2052
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2052
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2052
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2052
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2052
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2052
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2052
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2052
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2052
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSRLEGACY; MSR0000_0416
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2052
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2052
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2052
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2052
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2052
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2052
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2052
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2052
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2052
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2052
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2052
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2052
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2052
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2052
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2052
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSRLEGACY; MSR0000_0416
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2052
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSRLEGACY; MSR0000_0416

_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2052
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2052
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2052
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2052
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2052
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2052
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2052
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2052
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2052
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2052
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2052
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2052
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2052
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2052
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2052
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2052
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSRLEGACY; MSR0000_0416
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2052
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2052
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2052
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2052
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2052
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2052
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2052
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2052
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2052
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2052
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2052
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2052
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2052
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2052
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2052
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2052
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSRLEGACY; MSR0000_0416
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2052

_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSRLEGACY; MSR0000_0416	
Bits	Description
63:57	Reserved.
56:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 000_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::EX::MCA_STATUS_EX. For physical addresses, the most significant bit is given by Core::X86::CpuId::LongModeInfo[PhysAddrSize].

Table 71: MCA_ADDR_EX

Error Type	Bits	Description
WDT	[56:0]	Instruction Pointer (RIP)
PRF	[56:0]	Reserved
FRF	[56:0]	Reserved
IDRF	[56:0]	Reserved
PLDAG	[56:0]	Reserved
PLDAL	[56:0]	Reserved
CHKPTQ	[56:0]	Reserved
RETDISP	[56:0]	Reserved
STATQ	[56:0]	Reserved
SQ	[56:0]	Reserved
BBQ	[56:0]	Reserved
HWA	[56:0]	Reserved
SPECMAP	[56:0]	Reserved
RETMAP	[56:0]	Reserved

**MSR0000_0417...MSRC000_2053 [EX Machine Check Miscellaneous 0 Thread 0]
(MCA::EX::MCA_MISC0_EX)**

Log miscellaneous information associated with errors.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2053
_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2053
_ccd0_lthree0_core0_thread1_inst5_n1_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2053
_ccd0_lthree0_core1_thread0_inst5_n2_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2053
_ccd0_lthree0_core1_thread1_inst5_n3_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2053
_ccd0_lthree0_core2_thread0_inst5_n4_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2053
_ccd0_lthree0_core2_thread1_inst5_n5_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2053
_ccd0_lthree0_core3_thread0_inst5_n6_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2053
_ccd0_lthree0_core3_thread1_inst5_n7_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2053
_ccd0_lthree0_core4_thread0_inst5_n8_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2053
_ccd0_lthree0_core4_thread1_inst5_n9_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2053
_ccd0_lthree0_core5_thread0_inst5_n10_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2053
_ccd0_lthree0_core5_thread1_inst5_n11_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2053
_ccd0_lthree0_core6_thread0_inst5_n12_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2053
_ccd0_lthree0_core6_thread1_inst5_n13_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2053
_ccd0_lthree0_core7_thread0_inst5_n14_aliasMSRLEGACY; MSR0000_0417
_ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2053
_ccd0_lthree0_core7_thread1_inst5_n15_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2053
_ccd10_lthree0_core0_thread0_inst5_n160_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2053
_ccd10_lthree0_core0_thread1_inst5_n161_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2053
_ccd10_lthree0_core1_thread0_inst5_n162_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2053
_ccd10_lthree0_core1_thread1_inst5_n163_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2053
_ccd10_lthree0_core2_thread0_inst5_n164_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2053
_ccd10_lthree0_core2_thread1_inst5_n165_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2053
_ccd10_lthree0_core3_thread0_inst5_n166_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2053
_ccd10_lthree0_core3_thread1_inst5_n167_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2053
_ccd10_lthree0_core4_thread0_inst5_n168_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2053
_ccd10_lthree0_core4_thread1_inst5_n169_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2053
_ccd10_lthree0_core5_thread0_inst5_n170_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2053
_ccd10_lthree0_core5_thread1_inst5_n171_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2053
_ccd10_lthree0_core6_thread0_inst5_n172_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2053
_ccd10_lthree0_core6_thread1_inst5_n173_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2053

_ccd10_lthree0_core7_thread0_inst5_n174_aliasMSRLEGACY; MSR0000_0417
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2053
_ccd10_lthree0_core7_thread1_inst5_n175_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2053
_ccd11_lthree0_core0_thread0_inst5_n176_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2053
_ccd11_lthree0_core0_thread1_inst5_n177_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2053
_ccd11_lthree0_core1_thread0_inst5_n178_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2053
_ccd11_lthree0_core1_thread1_inst5_n179_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2053
_ccd11_lthree0_core2_thread0_inst5_n180_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2053
_ccd11_lthree0_core2_thread1_inst5_n181_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2053
_ccd11_lthree0_core3_thread0_inst5_n182_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2053
_ccd11_lthree0_core3_thread1_inst5_n183_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2053
_ccd11_lthree0_core4_thread0_inst5_n184_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2053
_ccd11_lthree0_core4_thread1_inst5_n185_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2053
_ccd11_lthree0_core5_thread0_inst5_n186_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2053
_ccd11_lthree0_core5_thread1_inst5_n187_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2053
_ccd11_lthree0_core6_thread0_inst5_n188_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2053
_ccd11_lthree0_core6_thread1_inst5_n189_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2053
_ccd11_lthree0_core7_thread0_inst5_n190_aliasMSRLEGACY; MSR0000_0417
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2053
_ccd11_lthree0_core7_thread1_inst5_n191_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2053
_ccd1_lthree0_core0_thread0_inst5_n16_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2053
_ccd1_lthree0_core0_thread1_inst5_n17_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2053
_ccd1_lthree0_core1_thread0_inst5_n18_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2053
_ccd1_lthree0_core1_thread1_inst5_n19_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2053
_ccd1_lthree0_core2_thread0_inst5_n20_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2053
_ccd1_lthree0_core2_thread1_inst5_n21_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2053
_ccd1_lthree0_core3_thread0_inst5_n22_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2053
_ccd1_lthree0_core3_thread1_inst5_n23_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2053
_ccd1_lthree0_core4_thread0_inst5_n24_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2053
_ccd1_lthree0_core4_thread1_inst5_n25_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2053
_ccd1_lthree0_core5_thread0_inst5_n26_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2053
_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2053
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2053
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSRLEGACY; MSR0000_0417
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2053
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSRLEGACY; MSR0000_0417

_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2053
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2053
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2053
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2053
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2053
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2053
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2053
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2053
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2053
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2053
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2053
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2053
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2053
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2053
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2053
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2053
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSRLEGACY; MSR0000_0417
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2053
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2053
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2053
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2053
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2053
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2053
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2053
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2053
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2053
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2053
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2053
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2053
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2053
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2053
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2053
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2053
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSRLEGACY; MSR0000_0417
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2053

_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2053
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2053
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2053
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2053
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2053
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2053
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2053
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2053
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2053
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2053
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2053
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2053
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2053
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2053
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2053
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSRLEGACY; MSR0000_0417
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2053
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2053
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2053
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2053
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2053
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2053
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2053
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2053
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2053
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2053
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2053
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2053
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2053
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2053
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2053
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2053
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSRLEGACY; MSR0000_0417
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2053
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSRLEGACY; MSR0000_0417

_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2053
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2053
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2053
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2053
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2053
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2053
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2053
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2053
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2053
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2053
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2053
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2053
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2053
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2053
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2053
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSRLEGACY; MSR0000_0417
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2053
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2053
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2053
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2053
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2053
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2053
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2053
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2053
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2053
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2053
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2053
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2053
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2053
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2053
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2053
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2053
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSRLEGACY; MSR0000_0417
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2053
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSRLEGACY; MSR0000_0417
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2053

_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSRLEGACY; MSR0000_0417	
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2053	
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSRLEGACY; MSR0000_0417	
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2053	
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSRLEGACY; MSR0000_0417	
Bits	Description

63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::EX::MCA_MISC0_EX[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::EX::MCA_MISC0_EX[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::EX::MCA_MISC0_EX[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::EX::MCA_MISC0_EX[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::EX::MCA_MISC0_EX[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::EX::MCA_MISC0_EX[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2054 [EX Machine Check Configuration Thread 0] (MCA::EX::MCA_CONFIG_EX)

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2054
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2054
 _ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2054
 _ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2054
 _ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2054

_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2054
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2054
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2054
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2054
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2054
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2054
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2054
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2054
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2054
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2054
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2054
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2054
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2054
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2054
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2054
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2054
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2054
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2054
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2054
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2054
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2054
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2054
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2054
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2054
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2054
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2054
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2054
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2054
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2054
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2054
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2054
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2054
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2054
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2054
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2054
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2054
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2054
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2054
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2054
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2054
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2054
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2054
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2054
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2054
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2054
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2054
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2054
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2054
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2054
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2054
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2054
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2054
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2054
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2054
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2054
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2054
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2054
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2054
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2054
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2054
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2054
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2054
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2054
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2054
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2054

_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2054
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2054
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2054
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2054
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2054
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2054
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2054
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2054
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2054
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2054
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2054
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2054
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2054
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2054
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2054
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2054
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2054
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2054
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2054
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2054
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2054
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2054
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2054
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2054
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2054
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2054
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2054
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2054
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2054
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2054
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2054
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2054
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2054
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2054
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2054
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2054
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2054
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2054
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2054
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2054
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2054
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2054
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2054
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2054
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2054
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2054
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2054
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2054
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2054
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2054
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2054
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2054
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2054
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2054
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2054
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2054
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2054
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2054
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2054
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2054
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2054
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2054
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2054
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2054
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2054

_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2054	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::EX::MCA_STATUS_EX, MCA::EX::MCA_ADDR_EX, MCA::EX::MCA_MISC0_EX, and MCA::EX::MCA_SYND_EX.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::EX::MCA_CONFIG_EX[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::EX::MCA_CONFIG_EX[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::EX::MCA_CONFIG_EX[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::EX::MCA_MISC0_EX[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::EX::MCA_STATUS_EX[TCC] is present.

MSRC000_2055 [EX IP Identification Thread 0] (MCA::EX::MCA_IPID_EX)

Reset: 0005_00B0_0000_0000h.

The MCA::EX::MCA_IPID_EX register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2055
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2055
 _ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2055
 _ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2055
 _ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2055

_ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2055
_ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2055
_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2055
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2055
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2055
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2055
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2055
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2055
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2055
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2055
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2055
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2055
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2055
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2055
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2055
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2055
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2055
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2055
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2055
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2055
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2055
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2055
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2055
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2055
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2055
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2055
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2055
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2055
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2055
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2055
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2055
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2055
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2055
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2055
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2055
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2055
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2055
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2055
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2055
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2055
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2055
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2055
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2055
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2055
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2055
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2055
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2055
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2055
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2055
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2055
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2055
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2055
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2055
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2055
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2055
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2055
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2055
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2055
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2055
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2055
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2055
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2055
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2055
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2055
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2055

_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2055
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2055
_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2055
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2055
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2055
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2055
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2055
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2055
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2055
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2055
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2055
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2055
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2055
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2055
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2055
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2055
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2055
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2055
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2055
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2055
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2055
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2055
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2055
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2055
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2055
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2055
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2055
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2055
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2055
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2055
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2055
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2055
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2055
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2055
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2055
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2055
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2055
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2055
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2055
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2055
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2055
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2055
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2055
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2055
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2055
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2055
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2055
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2055
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2055
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2055
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2055
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2055
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2055
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2055
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2055
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2055
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2055
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2055
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2055
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2055
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2055
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2055
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2055
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2055
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2055

_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2055	
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2055	
_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2055	
Bits	Description
63:48	McaType . Read-only. Reset: 0005h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR: 2000_9A00h Init: _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR: 2000_9B00h Init: _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR: 2002_9A00h Init: _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR: 2002_9B00h Init: _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR: 2004_9A00h Init: _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR: 2004_9B00h Init: _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR: 2006_9A00h Init: _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR: 2006_9B00h Init: _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR: 2008_9A00h Init: _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR: 2008_9B00h Init: _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR: 200A_9A00h Init: _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR: 200A_9B00h Init: _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR: 200C_9A00h Init: _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR: 200C_9B00h Init: _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR: 200E_9A00h Init: _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR: 200E_9B00h Init: _ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR: 2080_9A00h Init: _ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR: 2080_9B00h Init: _ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR: 2082_9A00h Init: _ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR: 2082_9B00h Init: _ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR: 2084_9A00h Init: _ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR: 2084_9B00h Init: _ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR: 2086_9A00h Init: _ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR: 2086_9B00h Init: _ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR: 2088_9A00h Init: _ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR: 2088_9B00h Init: _ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR: 208A_9A00h Init: _ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR: 208A_9B00h Init: _ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR: 208C_9A00h Init: _ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR: 208C_9B00h Init: _ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR: 208E_9A00h Init: _ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR: 208E_9B00h Init: _ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR: 2100_9A00h Init: _ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR: 2100_9B00h Init: _ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR: 2102_9A00h Init: _ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR: 2102_9B00h Init: _ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR: 2104_9A00h Init: _ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR: 2104_9B00h Init: _ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR: 2106_9A00h Init: _ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR: 2106_9B00h Init: _ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR: 2108_9A00h Init: _ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR: 2108_9B00h Init: _ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR: 210A_9A00h Init: _ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR: 210A_9B00h Init: _ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR: 210C_9A00h Init: _ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR: 210C_9B00h Init: _ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR: 210E_9A00h Init: _ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR: 210E_9B00h Init: _ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR: 2180_9A00h Init: _ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR: 2180_9B00h Init: _ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR: 2182_9A00h Init: _ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR: 2182_9B00h Init: _ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR: 2184_9A00h

Init: _ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR: 2184_9B00h
Init: _ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR: 2186_9A00h
Init: _ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR: 2186_9B00h
Init: _ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR: 2188_9A00h
Init: _ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR: 2188_9B00h
Init: _ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR: 218A_9A00h
Init: _ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR: 218A_9B00h
Init: _ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR: 218C_9A00h
Init: _ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR: 218C_9B00h
Init: _ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR: 218E_9A00h
Init: _ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR: 218E_9B00h
Init: _ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR: 2200_9A00h
Init: _ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR: 2200_9B00h
Init: _ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR: 2202_9A00h
Init: _ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR: 2202_9B00h
Init: _ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR: 2204_9A00h
Init: _ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR: 2204_9B00h
Init: _ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR: 2206_9A00h
Init: _ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR: 2206_9B00h
Init: _ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR: 2208_9A00h
Init: _ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR: 2208_9B00h
Init: _ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR: 220A_9A00h
Init: _ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR: 220A_9B00h
Init: _ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR: 220C_9A00h
Init: _ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR: 220C_9B00h
Init: _ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR: 220E_9A00h
Init: _ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR: 220E_9B00h
Init: _ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR: 2280_9A00h
Init: _ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR: 2280_9B00h
Init: _ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR: 2282_9A00h
Init: _ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR: 2282_9B00h
Init: _ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR: 2284_9A00h
Init: _ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR: 2284_9B00h
Init: _ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR: 2286_9A00h
Init: _ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR: 2286_9B00h
Init: _ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR: 2288_9A00h
Init: _ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR: 2288_9B00h
Init: _ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR: 228A_9A00h
Init: _ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR: 228A_9B00h
Init: _ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR: 228C_9A00h
Init: _ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR: 228C_9B00h
Init: _ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR: 228E_9A00h
Init: _ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR: 228E_9B00h
Init: _ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR: 2300_9A00h
Init: _ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR: 2300_9B00h
Init: _ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR: 2302_9A00h
Init: _ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR: 2302_9B00h
Init: _ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR: 2304_9A00h
Init: _ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR: 2304_9B00h
Init: _ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR: 2306_9A00h
Init: _ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR: 2306_9B00h
Init: _ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR: 2308_9A00h
Init: _ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR: 2308_9B00h
Init: _ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR: 230A_9A00h
Init: _ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR: 230A_9B00h
Init: _ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR: 230C_9A00h
Init: _ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR: 230C_9B00h
Init: _ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR: 230E_9A00h
Init: _ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR: 230E_9B00h
Init: _ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR: 2380_9A00h
Init: _ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR: 2380_9B00h
Init: _ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR: 2382_9A00h
Init: _ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR: 2382_9B00h
Init: _ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR: 2384_9A00h
Init: _ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR: 2384_9B00h

Init: _ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR: 2386_9A00h
Init: _ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR: 2386_9B00h
Init: _ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR: 2388_9A00h
Init: _ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR: 2388_9B00h
Init: _ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR: 238A_9A00h
Init: _ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR: 238A_9B00h
Init: _ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR: 238C_9A00h
Init: _ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR: 238C_9B00h
Init: _ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR: 238E_9A00h
Init: _ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR: 238E_9B00h
Init: _ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR: 2400_9A00h
Init: _ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR: 2400_9B00h
Init: _ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR: 2402_9A00h
Init: _ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR: 2402_9B00h
Init: _ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR: 2404_9A00h
Init: _ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR: 2404_9B00h
Init: _ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR: 2406_9A00h
Init: _ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR: 2406_9B00h
Init: _ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR: 2408_9A00h
Init: _ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR: 2408_9B00h
Init: _ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR: 240A_9A00h
Init: _ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR: 240A_9B00h
Init: _ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR: 240C_9A00h
Init: _ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR: 240C_9B00h
Init: _ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR: 240E_9A00h
Init: _ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR: 240E_9B00h
Init: _ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR: 2480_9A00h
Init: _ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR: 2480_9B00h
Init: _ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR: 2482_9A00h
Init: _ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR: 2482_9B00h
Init: _ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR: 2484_9A00h
Init: _ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR: 2484_9B00h
Init: _ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR: 2486_9A00h
Init: _ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR: 2486_9B00h
Init: _ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR: 2488_9A00h
Init: _ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR: 2488_9B00h
Init: _ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR: 248A_9A00h
Init: _ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR: 248A_9B00h
Init: _ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR: 248C_9A00h
Init: _ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR: 248C_9B00h
Init: _ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR: 248E_9A00h
Init: _ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR: 248E_9B00h
Init: _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR: 2500_9A00h
Init: _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR: 2500_9B00h
Init: _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR: 2502_9A00h
Init: _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR: 2502_9B00h
Init: _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR: 2504_9A00h
Init: _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR: 2504_9B00h
Init: _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR: 2506_9A00h
Init: _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR: 2506_9B00h
Init: _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR: 2508_9A00h
Init: _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR: 2508_9B00h
Init: _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR: 250A_9A00h
Init: _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR: 250A_9B00h
Init: _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR: 250C_9A00h
Init: _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR: 250C_9B00h
Init: _ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR: 250E_9A00h
Init: _ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR: 250E_9B00h
Init: _ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR: 2580_9A00h
Init: _ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR: 2580_9B00h
Init: _ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR: 2582_9A00h
Init: _ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR: 2582_9B00h
Init: _ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR: 2584_9A00h
Init: _ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR: 2584_9B00h
Init: _ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR: 2586_9A00h

Init: _ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR: 2586_9B00h
Init: _ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR: 2588_9A00h
Init: _ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR: 2588_9B00h
Init: _ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR: 258A_9A00h
Init: _ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR: 258A_9B00h
Init: _ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR: 258C_9A00h
Init: _ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR: 258C_9B00h
Init: _ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR: 258E_9A00h
Init: _ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR: 258E_9B00h

MSRC000_2056 [EX Machine Check Syndrome Thread 0] (MCA::EX::MCA_SYND_EX)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::EX::MCA_STATUS_EX [Thread 0](#)

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC000_2056
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC000_2056
 _ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC000_2056
 _ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC000_2056
 _ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC000_2056

_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC000_2056
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC000_2056
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC000_2056
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC000_2056
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC000_2056
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC000_2056
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC000_2056
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC000_2056
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC000_2056
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC000_2056
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC000_2056
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC000_2056
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC000_2056
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC000_2056
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC000_2056
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC000_2056
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC000_2056
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC000_2056
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC000_2056
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC000_2056
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC000_2056
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC000_2056
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC000_2056
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC000_2056
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC000_2056
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC000_2056
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC000_2056
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC000_2056
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC000_2056
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC000_2056
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC000_2056
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC000_2056
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC000_2056
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC000_2056
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC000_2056
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC000_2056
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC000_2056
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC000_2056
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC000_2056
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC000_2056
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC000_2056
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC000_2056
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC000_2056
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC000_2056
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC000_2056
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC000_2056
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC000_2056
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC000_2056
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC000_2056
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC000_2056
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC000_2056
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC000_2056
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC000_2056
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC000_2056
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC000_2056
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC000_2056
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC000_2056
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC000_2056
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC000_2056
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC000_2056
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC000_2056
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC000_2056
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC000_2056
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC000_2056
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC000_2056

_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC000_2056
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC000_2056
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC000_2056
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC000_2056
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC000_2056
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC000_2056
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC000_2056
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC000_2056
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC000_2056
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC000_2056
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC000_2056
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC000_2056
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC000_2056
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC000_2056
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC000_2056
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC000_2056
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC000_2056
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC000_2056
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC000_2056
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC000_2056
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC000_2056
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC000_2056
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC000_2056
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC000_2056
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC000_2056
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC000_2056
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC000_2056
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC000_2056
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC000_2056
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC000_2056
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC000_2056
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC000_2056
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC000_2056
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC000_2056
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC000_2056
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC000_2056
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC000_2056
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC000_2056
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC000_2056
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC000_2056
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC000_2056
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC000_2056
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC000_2056
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC000_2056
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC000_2056
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC000_2056
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC000_2056
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC000_2056
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC000_2056
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC000_2056
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC000_2056
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC000_2056
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC000_2056
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC000_2056
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC000_2056
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC000_2056
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC000_2056
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC000_2056
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC000_2056
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC000_2056
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC000_2056
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC000_2056
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC000_2056
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC000_2056
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC000_2056

_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC000_2056	
Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::EX::MCA_STATUS_EX. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::EX::MCA_SYND_EX[Length]. The Syndrome field is only valid when MCA::EX::MCA_SYND_EX[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::EX::MCA_SYND_EX. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::EX::MCA_SYND_EX[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::EX::MCA_SYND_EX. For example, a syndrome length of 9 means that MCA::EX::MCA_SYND_EX[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 72 [MCA_SYND_EX].

Table 72: MCA_SYND_EX

Error Type	Bits	Description
WDT	[17:0]	Reserved
PRF	[17:8] [7:0]	Reserved PRN
FRF	[17:5] [4] [3] [2] [1] [0]	Reserved BRU3 ALU3 ALU2 ALU1 ALU0
IDRF	[17:11] [10] [9] [8] [7] [6] [5] [4] [3] [2] [1] [0]	Reserved ALU3 BRU3 STU2 ALU2 AGU2 STU1 ALU1 AGU1 STU0 ALU0 AGU0
PLDAG	[17:11] [10] [9] [8] [7] [6] [5] [4] [3]	Reserved ALU3 BRU3 STU2 ALU2 AGU2 STU1 ALU1 AGU1

	[2] [1] [0]	STU0 ALU0 AGU0
PLDAL	[17:0]	Reserved
CHKPTQ	[17:4] [3] [2] [1] [0]	Reserved T1 on retire T0 on retire T1 on flush T0 on flush
RETDISP	[17:2] [1] [0]	Reserved T1 T0
STATQ	[17:0]	Reserved
SQ	[17:11] [10] [9] [8] [7] [6] [5] [4] [3] [2] [1] [0]	Reserved ALU3 BRU3 STU2 ALU2 AGU2 STU1 ALU1 AGU1 STU0 ALU0 AGU0
BBQ	[17:3] [2] [1] [0]	Reserved on RIP update on FIP update on Last_Branch_From update
HWA	[17:6] [5:0]	Reserved encoded error num
SPECMAP	[17:1] [0]	Reserved 1
RETMAP	[17:0]	Reserved

MSRC001_0405 [EX Machine Check Control Mask Thread 0] (MCA::EX::MCA_CTL_MASK_EX)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_core0_thread0_inst5_n0_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core0_thread1_inst5_n1_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core1_thread0_inst5_n2_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core1_thread1_inst5_n3_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core2_thread0_inst5_n4_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core2_thread1_inst5_n5_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core3_thread0_inst5_n6_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core3_thread1_inst5_n7_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core4_thread0_inst5_n8_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core4_thread1_inst5_n9_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core5_thread0_inst5_n10_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core5_thread1_inst5_n11_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core6_thread0_inst5_n12_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core6_thread1_inst5_n13_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core7_thread0_inst5_n14_aliasMSR; MSRC001_0405
 _ccd0_lthree0_core7_thread1_inst5_n15_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core0_thread0_inst5_n160_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core0_thread1_inst5_n161_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core1_thread0_inst5_n162_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core1_thread1_inst5_n163_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core2_thread0_inst5_n164_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core2_thread1_inst5_n165_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core3_thread0_inst5_n166_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core3_thread1_inst5_n167_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core4_thread0_inst5_n168_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core4_thread1_inst5_n169_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core5_thread0_inst5_n170_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core5_thread1_inst5_n171_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core6_thread0_inst5_n172_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core6_thread1_inst5_n173_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core7_thread0_inst5_n174_aliasMSR; MSRC001_0405
 _ccd10_lthree0_core7_thread1_inst5_n175_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core0_thread0_inst5_n176_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core0_thread1_inst5_n177_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core1_thread0_inst5_n178_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core1_thread1_inst5_n179_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core2_thread0_inst5_n180_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core2_thread1_inst5_n181_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core3_thread0_inst5_n182_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core3_thread1_inst5_n183_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core4_thread0_inst5_n184_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core4_thread1_inst5_n185_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core5_thread0_inst5_n186_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core5_thread1_inst5_n187_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core6_thread0_inst5_n188_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core6_thread1_inst5_n189_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core7_thread0_inst5_n190_aliasMSR; MSRC001_0405
 _ccd11_lthree0_core7_thread1_inst5_n191_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core0_thread0_inst5_n16_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core0_thread1_inst5_n17_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core1_thread0_inst5_n18_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core1_thread1_inst5_n19_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core2_thread0_inst5_n20_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core2_thread1_inst5_n21_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core3_thread0_inst5_n22_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core3_thread1_inst5_n23_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core4_thread0_inst5_n24_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core4_thread1_inst5_n25_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core5_thread0_inst5_n26_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core5_thread1_inst5_n27_aliasMSR; MSRC001_0405
 _ccd1_lthree0_core6_thread0_inst5_n28_aliasMSR; MSRC001_0405

_ccd1_lthree0_core6_thread1_inst5_n29_aliasMSR; MSRC001_0405
_ccd1_lthree0_core7_thread0_inst5_n30_aliasMSR; MSRC001_0405
_ccd1_lthree0_core7_thread1_inst5_n31_aliasMSR; MSRC001_0405
_ccd2_lthree0_core0_thread0_inst5_n32_aliasMSR; MSRC001_0405
_ccd2_lthree0_core0_thread1_inst5_n33_aliasMSR; MSRC001_0405
_ccd2_lthree0_core1_thread0_inst5_n34_aliasMSR; MSRC001_0405
_ccd2_lthree0_core1_thread1_inst5_n35_aliasMSR; MSRC001_0405
_ccd2_lthree0_core2_thread0_inst5_n36_aliasMSR; MSRC001_0405
_ccd2_lthree0_core2_thread1_inst5_n37_aliasMSR; MSRC001_0405
_ccd2_lthree0_core3_thread0_inst5_n38_aliasMSR; MSRC001_0405
_ccd2_lthree0_core3_thread1_inst5_n39_aliasMSR; MSRC001_0405
_ccd2_lthree0_core4_thread0_inst5_n40_aliasMSR; MSRC001_0405
_ccd2_lthree0_core4_thread1_inst5_n41_aliasMSR; MSRC001_0405
_ccd2_lthree0_core5_thread0_inst5_n42_aliasMSR; MSRC001_0405
_ccd2_lthree0_core5_thread1_inst5_n43_aliasMSR; MSRC001_0405
_ccd2_lthree0_core6_thread0_inst5_n44_aliasMSR; MSRC001_0405
_ccd2_lthree0_core6_thread1_inst5_n45_aliasMSR; MSRC001_0405
_ccd2_lthree0_core7_thread0_inst5_n46_aliasMSR; MSRC001_0405
_ccd2_lthree0_core7_thread1_inst5_n47_aliasMSR; MSRC001_0405
_ccd3_lthree0_core0_thread0_inst5_n48_aliasMSR; MSRC001_0405
_ccd3_lthree0_core0_thread1_inst5_n49_aliasMSR; MSRC001_0405
_ccd3_lthree0_core1_thread0_inst5_n50_aliasMSR; MSRC001_0405
_ccd3_lthree0_core1_thread1_inst5_n51_aliasMSR; MSRC001_0405
_ccd3_lthree0_core2_thread0_inst5_n52_aliasMSR; MSRC001_0405
_ccd3_lthree0_core2_thread1_inst5_n53_aliasMSR; MSRC001_0405
_ccd3_lthree0_core3_thread0_inst5_n54_aliasMSR; MSRC001_0405
_ccd3_lthree0_core3_thread1_inst5_n55_aliasMSR; MSRC001_0405
_ccd3_lthree0_core4_thread0_inst5_n56_aliasMSR; MSRC001_0405
_ccd3_lthree0_core4_thread1_inst5_n57_aliasMSR; MSRC001_0405
_ccd3_lthree0_core5_thread0_inst5_n58_aliasMSR; MSRC001_0405
_ccd3_lthree0_core5_thread1_inst5_n59_aliasMSR; MSRC001_0405
_ccd3_lthree0_core6_thread0_inst5_n60_aliasMSR; MSRC001_0405
_ccd3_lthree0_core6_thread1_inst5_n61_aliasMSR; MSRC001_0405
_ccd3_lthree0_core7_thread0_inst5_n62_aliasMSR; MSRC001_0405
_ccd3_lthree0_core7_thread1_inst5_n63_aliasMSR; MSRC001_0405
_ccd4_lthree0_core0_thread0_inst5_n64_aliasMSR; MSRC001_0405
_ccd4_lthree0_core0_thread1_inst5_n65_aliasMSR; MSRC001_0405
_ccd4_lthree0_core1_thread0_inst5_n66_aliasMSR; MSRC001_0405
_ccd4_lthree0_core1_thread1_inst5_n67_aliasMSR; MSRC001_0405
_ccd4_lthree0_core2_thread0_inst5_n68_aliasMSR; MSRC001_0405
_ccd4_lthree0_core2_thread1_inst5_n69_aliasMSR; MSRC001_0405
_ccd4_lthree0_core3_thread0_inst5_n70_aliasMSR; MSRC001_0405
_ccd4_lthree0_core3_thread1_inst5_n71_aliasMSR; MSRC001_0405
_ccd4_lthree0_core4_thread0_inst5_n72_aliasMSR; MSRC001_0405
_ccd4_lthree0_core4_thread1_inst5_n73_aliasMSR; MSRC001_0405
_ccd4_lthree0_core5_thread0_inst5_n74_aliasMSR; MSRC001_0405
_ccd4_lthree0_core5_thread1_inst5_n75_aliasMSR; MSRC001_0405
_ccd4_lthree0_core6_thread0_inst5_n76_aliasMSR; MSRC001_0405
_ccd4_lthree0_core6_thread1_inst5_n77_aliasMSR; MSRC001_0405
_ccd4_lthree0_core7_thread0_inst5_n78_aliasMSR; MSRC001_0405
_ccd4_lthree0_core7_thread1_inst5_n79_aliasMSR; MSRC001_0405
_ccd5_lthree0_core0_thread0_inst5_n80_aliasMSR; MSRC001_0405
_ccd5_lthree0_core0_thread1_inst5_n81_aliasMSR; MSRC001_0405
_ccd5_lthree0_core1_thread0_inst5_n82_aliasMSR; MSRC001_0405
_ccd5_lthree0_core1_thread1_inst5_n83_aliasMSR; MSRC001_0405
_ccd5_lthree0_core2_thread0_inst5_n84_aliasMSR; MSRC001_0405
_ccd5_lthree0_core2_thread1_inst5_n85_aliasMSR; MSRC001_0405
_ccd5_lthree0_core3_thread0_inst5_n86_aliasMSR; MSRC001_0405
_ccd5_lthree0_core3_thread1_inst5_n87_aliasMSR; MSRC001_0405
_ccd5_lthree0_core4_thread0_inst5_n88_aliasMSR; MSRC001_0405
_ccd5_lthree0_core4_thread1_inst5_n89_aliasMSR; MSRC001_0405
_ccd5_lthree0_core5_thread0_inst5_n90_aliasMSR; MSRC001_0405
_ccd5_lthree0_core5_thread1_inst5_n91_aliasMSR; MSRC001_0405
_ccd5_lthree0_core6_thread0_inst5_n92_aliasMSR; MSRC001_0405
_ccd5_lthree0_core6_thread1_inst5_n93_aliasMSR; MSRC001_0405

_ccd5_lthree0_core7_thread0_inst5_n94_aliasMSR; MSRC001_0405
_ccd5_lthree0_core7_thread1_inst5_n95_aliasMSR; MSRC001_0405
_ccd6_lthree0_core0_thread0_inst5_n96_aliasMSR; MSRC001_0405
_ccd6_lthree0_core0_thread1_inst5_n97_aliasMSR; MSRC001_0405
_ccd6_lthree0_core1_thread0_inst5_n98_aliasMSR; MSRC001_0405
_ccd6_lthree0_core1_thread1_inst5_n99_aliasMSR; MSRC001_0405
_ccd6_lthree0_core2_thread0_inst5_n100_aliasMSR; MSRC001_0405
_ccd6_lthree0_core2_thread1_inst5_n101_aliasMSR; MSRC001_0405
_ccd6_lthree0_core3_thread0_inst5_n102_aliasMSR; MSRC001_0405
_ccd6_lthree0_core3_thread1_inst5_n103_aliasMSR; MSRC001_0405
_ccd6_lthree0_core4_thread0_inst5_n104_aliasMSR; MSRC001_0405
_ccd6_lthree0_core4_thread1_inst5_n105_aliasMSR; MSRC001_0405
_ccd6_lthree0_core5_thread0_inst5_n106_aliasMSR; MSRC001_0405
_ccd6_lthree0_core5_thread1_inst5_n107_aliasMSR; MSRC001_0405
_ccd6_lthree0_core6_thread0_inst5_n108_aliasMSR; MSRC001_0405
_ccd6_lthree0_core6_thread1_inst5_n109_aliasMSR; MSRC001_0405
_ccd6_lthree0_core7_thread0_inst5_n110_aliasMSR; MSRC001_0405
_ccd6_lthree0_core7_thread1_inst5_n111_aliasMSR; MSRC001_0405
_ccd7_lthree0_core0_thread0_inst5_n112_aliasMSR; MSRC001_0405
_ccd7_lthree0_core0_thread1_inst5_n113_aliasMSR; MSRC001_0405
_ccd7_lthree0_core1_thread0_inst5_n114_aliasMSR; MSRC001_0405
_ccd7_lthree0_core1_thread1_inst5_n115_aliasMSR; MSRC001_0405
_ccd7_lthree0_core2_thread0_inst5_n116_aliasMSR; MSRC001_0405
_ccd7_lthree0_core2_thread1_inst5_n117_aliasMSR; MSRC001_0405
_ccd7_lthree0_core3_thread0_inst5_n118_aliasMSR; MSRC001_0405
_ccd7_lthree0_core3_thread1_inst5_n119_aliasMSR; MSRC001_0405
_ccd7_lthree0_core4_thread0_inst5_n120_aliasMSR; MSRC001_0405
_ccd7_lthree0_core4_thread1_inst5_n121_aliasMSR; MSRC001_0405
_ccd7_lthree0_core5_thread0_inst5_n122_aliasMSR; MSRC001_0405
_ccd7_lthree0_core5_thread1_inst5_n123_aliasMSR; MSRC001_0405
_ccd7_lthree0_core6_thread0_inst5_n124_aliasMSR; MSRC001_0405
_ccd7_lthree0_core6_thread1_inst5_n125_aliasMSR; MSRC001_0405
_ccd7_lthree0_core7_thread0_inst5_n126_aliasMSR; MSRC001_0405
_ccd7_lthree0_core7_thread1_inst5_n127_aliasMSR; MSRC001_0405
_ccd8_lthree0_core0_thread0_inst5_n128_aliasMSR; MSRC001_0405
_ccd8_lthree0_core0_thread1_inst5_n129_aliasMSR; MSRC001_0405
_ccd8_lthree0_core1_thread0_inst5_n130_aliasMSR; MSRC001_0405
_ccd8_lthree0_core1_thread1_inst5_n131_aliasMSR; MSRC001_0405
_ccd8_lthree0_core2_thread0_inst5_n132_aliasMSR; MSRC001_0405
_ccd8_lthree0_core2_thread1_inst5_n133_aliasMSR; MSRC001_0405
_ccd8_lthree0_core3_thread0_inst5_n134_aliasMSR; MSRC001_0405
_ccd8_lthree0_core3_thread1_inst5_n135_aliasMSR; MSRC001_0405
_ccd8_lthree0_core4_thread0_inst5_n136_aliasMSR; MSRC001_0405
_ccd8_lthree0_core4_thread1_inst5_n137_aliasMSR; MSRC001_0405
_ccd8_lthree0_core5_thread0_inst5_n138_aliasMSR; MSRC001_0405
_ccd8_lthree0_core5_thread1_inst5_n139_aliasMSR; MSRC001_0405
_ccd8_lthree0_core6_thread0_inst5_n140_aliasMSR; MSRC001_0405
_ccd8_lthree0_core6_thread1_inst5_n141_aliasMSR; MSRC001_0405
_ccd8_lthree0_core7_thread0_inst5_n142_aliasMSR; MSRC001_0405
_ccd8_lthree0_core7_thread1_inst5_n143_aliasMSR; MSRC001_0405
_ccd9_lthree0_core0_thread0_inst5_n144_aliasMSR; MSRC001_0405
_ccd9_lthree0_core0_thread1_inst5_n145_aliasMSR; MSRC001_0405
_ccd9_lthree0_core1_thread0_inst5_n146_aliasMSR; MSRC001_0405
_ccd9_lthree0_core1_thread1_inst5_n147_aliasMSR; MSRC001_0405
_ccd9_lthree0_core2_thread0_inst5_n148_aliasMSR; MSRC001_0405
_ccd9_lthree0_core2_thread1_inst5_n149_aliasMSR; MSRC001_0405
_ccd9_lthree0_core3_thread0_inst5_n150_aliasMSR; MSRC001_0405
_ccd9_lthree0_core3_thread1_inst5_n151_aliasMSR; MSRC001_0405
_ccd9_lthree0_core4_thread0_inst5_n152_aliasMSR; MSRC001_0405
_ccd9_lthree0_core4_thread1_inst5_n153_aliasMSR; MSRC001_0405
_ccd9_lthree0_core5_thread0_inst5_n154_aliasMSR; MSRC001_0405
_ccd9_lthree0_core5_thread1_inst5_n155_aliasMSR; MSRC001_0405
_ccd9_lthree0_core6_thread0_inst5_n156_aliasMSR; MSRC001_0405
_ccd9_lthree0_core6_thread1_inst5_n157_aliasMSR; MSRC001_0405
_ccd9_lthree0_core7_thread0_inst5_n158_aliasMSR; MSRC001_0405

_ccd9_lthree0_core7_thread1_inst5_n159_aliasMSR; MSRC001_0405	
Bits	Description
63:14	Reserved.
13	RETMAP . Read-write. Reset: 0. Retire Map parity error.
12	SPECMAP . Read-write. Reset: 0. Spec Map parity error.
11	HWA . Read-write. Reset: 0. Hardware Assertion Error.
10	BBQ . Read-write. Reset: 0. Branch Buffer Queue (BBQ) parity error.
9	SQ . Read-write. Reset: 0. Scheduler Queue parity error.
8	STATQ . Read-write. Reset: 0. Retire status queue parity error.
7	RETDISP . Read-write. Reset: 0. Retire Dispatch Queue parity error.
6	CHKPTQ . Read-write. Reset: 0. Checkpoint Queue (AKA Map_DisqQ) parity error.
5	PLDAL . Read-write. Reset: 0. EX payload parity error.
4	PLDAG . Read-write. Reset: 0. Address generator payload parity error.
3	IDRF . Read-write. Reset: 0. Immediate displacement register file parity error.
2	FRF . Read-write. Reset: 0. Flag register file (FRF) parity error.
1	PRF . Read-write. Reset: 0. Physical register file (PRF) parity error.
0	WDT . Read-write. Reset: 0. Watchdog Timeout.

3.2.5.6 FP

MSR0000_0418...MSRC000_2060 [FP Machine Check Control Thread 0] (MCA::FP::MCA_CTL_FP)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::FP::MCA_CTL_FP register must be enabled by the corresponding enable bit in Core::X86::Msrr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2060
_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2060
_ccd0_lthree0_core0_thread1_inst6_n1_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2060
_ccd0_lthree0_core1_thread0_inst6_n2_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2060
_ccd0_lthree0_core1_thread1_inst6_n3_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2060
_ccd0_lthree0_core2_thread0_inst6_n4_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2060
_ccd0_lthree0_core2_thread1_inst6_n5_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2060
_ccd0_lthree0_core3_thread0_inst6_n6_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2060
_ccd0_lthree0_core3_thread1_inst6_n7_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2060
_ccd0_lthree0_core4_thread0_inst6_n8_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2060
_ccd0_lthree0_core4_thread1_inst6_n9_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2060
_ccd0_lthree0_core5_thread0_inst6_n10_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2060
_ccd0_lthree0_core5_thread1_inst6_n11_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2060
_ccd0_lthree0_core6_thread0_inst6_n12_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2060
_ccd0_lthree0_core6_thread1_inst6_n13_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2060
_ccd0_lthree0_core7_thread0_inst6_n14_aliasMSRLEGACY; MSR0000_0418
_ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2060
_ccd0_lthree0_core7_thread1_inst6_n15_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2060
_ccd10_lthree0_core0_thread0_inst6_n160_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2060
_ccd10_lthree0_core0_thread1_inst6_n161_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2060
_ccd10_lthree0_core1_thread0_inst6_n162_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2060
_ccd10_lthree0_core1_thread1_inst6_n163_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2060
_ccd10_lthree0_core2_thread0_inst6_n164_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2060
_ccd10_lthree0_core2_thread1_inst6_n165_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2060
_ccd10_lthree0_core3_thread0_inst6_n166_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2060
_ccd10_lthree0_core3_thread1_inst6_n167_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2060
_ccd10_lthree0_core4_thread0_inst6_n168_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2060
_ccd10_lthree0_core4_thread1_inst6_n169_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2060
_ccd10_lthree0_core5_thread0_inst6_n170_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2060
_ccd10_lthree0_core5_thread1_inst6_n171_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2060
_ccd10_lthree0_core6_thread0_inst6_n172_aliasMSRLEGACY; MSR0000_0418

_ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2060
_ccd10_lthree0_core6_thread1_inst6_n173_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2060
_ccd10_lthree0_core7_thread0_inst6_n174_aliasMSRLEGACY; MSR0000_0418
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2060
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2060
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2060
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2060
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2060
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2060
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2060
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2060
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2060
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2060
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2060
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2060
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2060
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2060
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2060
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2060
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSRLEGACY; MSR0000_0418
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2060
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2060
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2060
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2060
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2060
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2060
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2060
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2060
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2060
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2060
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2060
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2060
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2060
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2060
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2060

_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2060
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSRLEGACY; MSR0000_0418
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2060
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2060
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2060
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2060
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2060
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2060
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2060
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2060
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2060
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2060
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2060
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2060
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2060
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2060
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2060
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2060
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSRLEGACY; MSR0000_0418
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2060
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2060
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2060
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2060
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2060
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2060
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2060
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2060
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2060
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2060
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2060
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2060
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2060
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2060
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2060
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSRLEGACY; MSR0000_0418

_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2060
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSRLEGACY; MSR0000_0418
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2060
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2060
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2060
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2060
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2060
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2060
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2060
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2060
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2060
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2060
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2060
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2060
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2060
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2060
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2060
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2060
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSRLEGACY; MSR0000_0418
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2060
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2060
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2060
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2060
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2060
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2060
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2060
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2060
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2060
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2060
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2060
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2060
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2060
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2060
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2060
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2060

_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSRLEGACY; MSR0000_0418
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2060
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2060
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2060
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2060
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2060
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2060
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2060
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2060
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2060
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2060
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2060
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2060
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2060
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2060
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2060
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2060
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSRLEGACY; MSR0000_0418
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2060
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2060
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2060
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2060
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2060
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2060
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2060
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2060
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2060
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2060
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2060
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2060
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2060
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2060
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2060
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSRLEGACY; MSR0000_0418
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2060
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSRLEGACY; MSR0000_0418

_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2060
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2060
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2060
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2060
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2060
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2060
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2060
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2060
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2060
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2060
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2060
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2060
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2060
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2060
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2060
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2060
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSRLEGACY; MSR0000_0418
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2060
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2060
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2060
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2060
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2060
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2060
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2060
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2060
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2060
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2060
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2060
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2060
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2060
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2060
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2060
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2060
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSRLEGACY; MSR0000_0418
_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2060

_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSRLEGACY; MSR0000_0418	
Bits	Description
63:8	Reserved.
7	KRF. Read-write. Reset: 0. Physical K mask register file (KRF) parity error.
6	HWA. Read-write. Reset: 0. Hardware assertion.
5	SRF. Read-write. Reset: 0. Status register file (SRF) parity error.
4	RQ. Read-write. Reset: 0. Retire queue (RQ) parity error.
3	NSQ. Read-write. Reset: 0. NSQ parity error.
2	SCH. Read-write. Reset: 0. Schedule queue parity error.
1	FL. Read-write. Reset: 0. Freelist (FL) parity error.
0	PRF. Read-write. Reset: 0. Physical register file (PRF) parity error.

MSR0000_0419...MSRC000_2061 [FP Machine Check Status Thread 0] (MCA::FP::MCA_STATUS_FP)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core0_thread0_inst6_n0_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSRLEGACY; MSR0000_0419
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2061
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2061
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSRLEGACY; MSR0000_0419
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2061

_ccd10_lthree0_core7_thread0_inst6_n174_aliasMSRLEGACY; MSR0000_0419
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2061
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2061
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2061
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2061
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2061
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2061
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2061
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2061
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2061
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2061
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2061
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2061
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2061
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2061
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2061
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2061
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSRLEGACY; MSR0000_0419
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2061
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2061
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2061
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2061
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2061
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2061
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2061
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2061
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2061
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2061
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2061
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2061
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2061
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2061
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2061
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSRLEGACY; MSR0000_0419
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2061
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSRLEGACY; MSR0000_0419

_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2061
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2061
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2061
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2061
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2061
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2061
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2061
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2061
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2061
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2061
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2061
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2061
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2061
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2061
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2061
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2061
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSRLEGACY; MSR0000_0419
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2061
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2061
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2061
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2061
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2061
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2061
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2061
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2061
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2061
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2061
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2061
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2061
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2061
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2061
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2061
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2061
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSRLEGACY; MSR0000_0419
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2061

_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2061
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2061
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2061
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2061
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2061
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2061
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2061
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2061
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2061
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2061
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2061
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2061
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2061
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2061
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2061
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSRLEGACY; MSR0000_0419
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2061
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2061
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2061
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2061
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2061
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2061
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2061
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2061
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2061
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2061
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2061
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2061
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2061
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2061
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2061
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2061
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSRLEGACY; MSR0000_0419
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2061
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSRLEGACY; MSR0000_0419

_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2061
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2061
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2061
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2061
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2061
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2061
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2061
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2061
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2061
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2061
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2061
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2061
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2061
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2061
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2061
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSRLEGACY; MSR0000_0419
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2061
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2061
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2061
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2061
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2061
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2061
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2061
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2061
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2061
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2061
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2061
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2061
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2061
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2061
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2061
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2061
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSRLEGACY; MSR0000_0419
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2061
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSRLEGACY; MSR0000_0419
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2061

<u>_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2061</u>	
<u>_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSRLEGACY; MSR0000_0419</u>	
<u>_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2061</u>	
<u>_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSRLEGACY; MSR0000_0419</u>	
Bits	Description

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::FP::MCA_CTL_FP. This bit is a copy of bit in MCA::FP::MCA_CTL_FP for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::FP::MCA_MISC0_FP. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::FP::MCA_ADDR_FP contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::FP::MCA_STATUS_FP[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::FP::MCA_SYND_FP. If MCA::FP::MCA_SYND_FP[ErrorPriority] is the same as the priority of the error in MCA::FP::MCA_STATUS_FP, then the information in MCA::FP::MCA_SYND_FP is associated with the error in MCA::FP::MCA_STATUS_FP. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::FP::MCA_STATUS_FP[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::FP::MCA_ADDR_FP[ErrorAddr]. A value of 0 indicates that MCA::FP::MCA_ADDR_FP[55:0] contains a valid byte address. A value of 6 indicates that MCA::FP::MCA_ADDR_FP[55:6] contains a valid cache line address and that MCA::FP::MCA_ADDR_FP[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::FP::MCA_ADDR_FP[55:12] contain a valid 4KB memory page and that MCA::FP::MCA_ADDR_FP[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::FP::MCA_CTL_FP enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 73: MCA_STATUS_FP

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
PRF	0x0	1	1	1	0	0	0
FL	0x1	1	1	1	0	0	0
SCH	0x2	1	1	1	0	0	0
NSQ	0x3	1	1	1	0	0	0
RQ	0x4	1	1	1	0	0	0
SRF	0x5	1	1	1	0	0	0
HWA	0x6	1	1	1	0	0	0
KRF	0x7	1	1	1	0	0	0

MSR0000_041A...MSRC000_2062 [FP Machine Check Address Thread 0] (MCA::FP::MCA_ADDR_FP)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::FP::MCA_ADDR_FP stores an address and other information associated with the error in MCA::FP::MCA_STATUS_FP. The register is only meaningful if MCA::FP::MCA_STATUS_FP[Val]=1 and MCA::FP::MCA_STATUS_FP[AddrV]=1.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core0_thread0_inst6_n0_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSRLEGACY; MSR0000_041A
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2062
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSRLEGACY; MSR0000_041A
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2062
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSRLEGACY; MSR0000_041A

_ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2062
_ccd10_lthree0_core6_thread1_inst6_n173_aliasMSRLEGACY; MSR0000_041A
_ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2062
_ccd10_lthree0_core7_thread0_inst6_n174_aliasMSRLEGACY; MSR0000_041A
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2062
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2062
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2062
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2062
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2062
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2062
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2062
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2062
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2062
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2062
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2062
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2062
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2062
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2062
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2062
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2062
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSRLEGACY; MSR0000_041A
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2062
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2062
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2062
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2062
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2062
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2062
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2062
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2062
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2062
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2062
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2062
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2062
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2062
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2062
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2062

_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2062
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSRLEGACY; MSR0000_041A
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2062
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2062
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2062
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2062
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2062
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2062
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2062
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2062
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2062
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2062
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2062
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2062
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2062
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2062
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2062
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2062
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSRLEGACY; MSR0000_041A
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2062
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2062
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2062
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2062
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2062
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2062
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2062
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2062
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2062
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2062
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2062
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2062
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2062
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2062
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2062
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSRLEGACY; MSR0000_041A

_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2062
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSRLEGACY; MSR0000_041A
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2062
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2062
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2062
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2062
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2062
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2062
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2062
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2062
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2062
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2062
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2062
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2062
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2062
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2062
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2062
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2062
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSRLEGACY; MSR0000_041A
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2062
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2062
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2062
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2062
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2062
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2062
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2062
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2062
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2062
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2062
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2062
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2062
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2062
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2062
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2062
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2062

_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSRLEGACY; MSR0000_041A
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2062
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2062
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2062
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2062
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2062
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2062
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2062
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2062
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2062
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2062
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2062
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2062
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2062
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2062
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2062
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2062
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSRLEGACY; MSR0000_041A
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2062
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2062
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2062
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2062
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2062
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2062
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2062
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2062
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2062
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2062
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2062
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2062
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2062
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2062
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2062
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSRLEGACY; MSR0000_041A
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2062
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSRLEGACY; MSR0000_041A

_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2062
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2062
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2062
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2062
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2062
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2062
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2062
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2062
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2062
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2062
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2062
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2062
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2062
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2062
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2062
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2062
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSRLEGACY; MSR0000_041A
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2062
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2062
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2062
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2062
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2062
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2062
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2062
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2062
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2062
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2062
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2062
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2062
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2062
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2062
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2062
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2062
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSRLEGACY; MSR0000_041A
_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2062

_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSRLEGACY; MSR0000_041A	
Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::FP::MCA_STATUS_FP.

Table 74: MCA_ADDR_FP

Error Type	Bits	Description
PRF	[55:0]	Reserved
FL	[55:0]	Reserved
SCH	[55:0]	Reserved
NSQ	[55:0]	Reserved
RQ	[55:0]	Reserved
SRF	[55:0]	Reserved
HWA	[55:0]	Reserved
KRF	[55:0]	Reserved

MSR0000_041B...MSRC000_2063 [FP Machine Check Miscellaneous 0 Thread 0] (MCA::FP::MCA_MISC0_FP)

Log miscellaneous information associated with errors.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core0_thread0_inst6_n0_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSRLEGACY; MSR0000_041B
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2063
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSRLEGACY; MSR0000_041B
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2063
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSRLEGACY; MSR0000_041B

_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2063
_ccd10_lthree0_core7_thread1_inst6_n175_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2063
_ccd11_lthree0_core0_thread0_inst6_n176_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2063
_ccd11_lthree0_core0_thread1_inst6_n177_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2063
_ccd11_lthree0_core1_thread0_inst6_n178_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2063
_ccd11_lthree0_core1_thread1_inst6_n179_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2063
_ccd11_lthree0_core2_thread0_inst6_n180_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2063
_ccd11_lthree0_core2_thread1_inst6_n181_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2063
_ccd11_lthree0_core3_thread0_inst6_n182_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2063
_ccd11_lthree0_core3_thread1_inst6_n183_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2063
_ccd11_lthree0_core4_thread0_inst6_n184_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2063
_ccd11_lthree0_core4_thread1_inst6_n185_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2063
_ccd11_lthree0_core5_thread0_inst6_n186_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2063
_ccd11_lthree0_core5_thread1_inst6_n187_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2063
_ccd11_lthree0_core6_thread0_inst6_n188_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2063
_ccd11_lthree0_core6_thread1_inst6_n189_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2063
_ccd11_lthree0_core7_thread0_inst6_n190_aliasMSRLEGACY; MSR0000_041B
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2063
_ccd11_lthree0_core7_thread1_inst6_n191_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2063
_ccd1_lthree0_core0_thread0_inst6_n16_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2063
_ccd1_lthree0_core0_thread1_inst6_n17_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2063
_ccd1_lthree0_core1_thread0_inst6_n18_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2063
_ccd1_lthree0_core1_thread1_inst6_n19_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2063
_ccd1_lthree0_core2_thread0_inst6_n20_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2063
_ccd1_lthree0_core2_thread1_inst6_n21_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2063
_ccd1_lthree0_core3_thread0_inst6_n22_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2063
_ccd1_lthree0_core3_thread1_inst6_n23_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2063
_ccd1_lthree0_core4_thread0_inst6_n24_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2063
_ccd1_lthree0_core4_thread1_inst6_n25_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2063
_ccd1_lthree0_core5_thread0_inst6_n26_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2063
_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2063
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2063
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2063
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSRLEGACY; MSR0000_041B
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2063

_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2063
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2063
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2063
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2063
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2063
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2063
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2063
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2063
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2063
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2063
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2063
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2063
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2063
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2063
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2063
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSRLEGACY; MSR0000_041B
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2063
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2063
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2063
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2063
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2063
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2063
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2063
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2063
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2063
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2063
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2063
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2063
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2063
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2063
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2063
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2063
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSRLEGACY; MSR0000_041B
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2063
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSRLEGACY; MSR0000_041B

_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2063
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2063
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2063
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2063
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2063
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2063
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2063
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2063
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2063
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2063
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2063
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2063
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2063
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2063
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2063
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSRLEGACY; MSR0000_041B
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2063
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2063
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2063
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2063
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2063
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2063
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2063
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2063
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2063
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2063
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2063
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2063
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2063
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2063
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2063
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2063
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSRLEGACY; MSR0000_041B
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2063
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2063

_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2063
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2063
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2063
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2063
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2063
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2063
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2063
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2063
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2063
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2063
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2063
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2063
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2063
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2063
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSRLEGACY; MSR0000_041B
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2063
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2063
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2063
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2063
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2063
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2063
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2063
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2063
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2063
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2063
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2063
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2063
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2063
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2063
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2063
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2063
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSRLEGACY; MSR0000_041B
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2063
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSRLEGACY; MSR0000_041B
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2063
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSRLEGACY; MSR0000_041B

<u>_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2063</u>	
<u>_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSRLEGACY; MSR0000_041B</u>	
<u>_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2063</u>	
<u>_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSRLEGACY; MSR0000_041B</u>	
Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP . Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked . Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP . Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::FP::MCA_MISC0_FP[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset . Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::FP::MCA_MISC0_FP[Locked]) ? Read-write : Read-only.
51	CntEn . Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::FP::MCA_MISC0_FP[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType . Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrlw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::FP::MCA_MISC0_FP[Locked]) ? Read-write : Read-only.
48	Ovrlw . Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::FP::MCA_MISC0_FP[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt . Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::FP::MCA_MISC0_FP[Locked]) ? Read-write : Read-only.
31:24	BlkPtr . Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2064 [FP Machine Check Configuration Thread 0] (MCA::FP::MCA_CONFIG_FP)

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2064
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2064
 _ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2064
 _ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2064
 _ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2064

_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2064
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2064
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2064
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2064
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2064
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2064
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2064
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2064
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2064
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2064
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2064
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2064
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2064
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2064
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2064
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2064
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2064
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2064
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2064
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2064
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2064
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2064
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2064
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2064
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2064
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2064
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2064
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2064
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2064
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2064
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2064
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2064
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2064
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2064
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2064
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2064
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2064
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2064
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2064
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2064
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2064
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2064
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2064
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2064
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2064
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2064
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2064
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2064
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2064
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2064
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2064
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2064
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2064
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2064
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2064
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2064
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2064
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2064
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2064
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2064
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2064
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2064
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2064
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2064
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2064

_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2064
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2064
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2064
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2064
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2064
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2064
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2064
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2064
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2064
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2064
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2064
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2064
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2064
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2064
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2064
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2064
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2064
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2064
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2064
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2064
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2064
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2064
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2064
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2064
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2064
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2064
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2064
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2064
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2064
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2064
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2064
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2064
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2064
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2064
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2064
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2064
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2064
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2064
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2064
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2064
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2064
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2064
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2064
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2064
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2064
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2064
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2064
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2064
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2064
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2064
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2064
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2064
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2064
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2064
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2064
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2064
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2064
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2064
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2064
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2064
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2064
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2064
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2064
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2064
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2064

_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2064	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::FP::MCA_STATUS_FP, MCA::FP::MCA_ADDR_FP, MCA::FP::MCA_MISC0_FP, and MCA::FP::MCA_SYND_FP.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::FP::MCA_CONFIG_FP[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::FP::MCA_CONFIG_FP[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::FP::MCA_CONFIG_FP[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::FP::MCA_MISC0_FP[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::FP::MCA_STATUS_FP[TCC] is present.

MSRC000_2065 [FP IP Identification Thread 0] (MCA::FP::MCA_IPID_FP)

Reset: 0006_00B0_0000_0000h.

The MCA::FP::MCA_IPID_FP register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2065
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2065
 _ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2065
 _ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2065
 _ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2065

_ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2065
_ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2065
_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2065
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2065
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2065
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2065
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2065
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2065
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2065
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2065
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2065
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2065
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2065
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2065
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2065
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2065
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2065
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2065
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2065
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2065
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2065
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2065
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2065
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2065
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2065
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2065
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2065
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2065
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2065
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2065
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2065
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2065
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2065
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2065
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2065
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2065
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2065
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2065
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2065
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2065
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2065
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2065
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2065
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2065
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2065
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2065
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2065
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2065
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2065
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2065
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2065
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2065
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2065
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2065
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2065
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2065
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2065
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2065
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2065
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2065
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2065
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2065
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2065
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2065
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2065

_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2065
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2065
_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2065
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2065
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2065
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2065
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2065
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2065
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2065
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2065
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2065
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2065
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2065
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2065
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2065
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2065
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2065
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2065
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2065
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2065
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2065
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2065
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2065
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2065
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2065
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2065
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2065
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2065
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2065
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2065
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2065
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2065
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2065
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2065
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2065
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2065
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2065
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2065
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2065
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2065
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2065
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2065
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2065
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2065
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2065
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2065
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2065
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2065
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2065
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2065
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2065
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2065
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2065
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2065
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2065
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2065
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2065
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2065
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2065
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2065
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2065
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2065
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2065
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2065
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2065

_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2065	
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2065	
_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2065	
Bits	Description
63:48	McaType . Read-only. Reset: 0006h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR: 2000_8600h Init: _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR: 2000_8700h Init: _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR: 2002_8600h Init: _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR: 2002_8700h Init: _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR: 2004_8600h Init: _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR: 2004_8700h Init: _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR: 2006_8600h Init: _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR: 2006_8700h Init: _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR: 2008_8600h Init: _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR: 2008_8700h Init: _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR: 200A_8600h Init: _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR: 200A_8700h Init: _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR: 200C_8600h Init: _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR: 200C_8700h Init: _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR: 200E_8600h Init: _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR: 200E_8700h Init: _ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR: 2080_8600h Init: _ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR: 2080_8700h Init: _ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR: 2082_8600h Init: _ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR: 2082_8700h Init: _ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR: 2084_8600h Init: _ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR: 2084_8700h Init: _ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR: 2086_8600h Init: _ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR: 2086_8700h Init: _ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR: 2088_8600h Init: _ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR: 2088_8700h Init: _ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR: 208A_8600h Init: _ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR: 208A_8700h Init: _ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR: 208C_8600h Init: _ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR: 208C_8700h Init: _ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR: 208E_8600h Init: _ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR: 208E_8700h Init: _ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR: 2100_8600h Init: _ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR: 2100_8700h Init: _ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR: 2102_8600h Init: _ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR: 2102_8700h Init: _ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR: 2104_8600h Init: _ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR: 2104_8700h Init: _ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR: 2106_8600h Init: _ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR: 2106_8700h Init: _ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR: 2108_8600h Init: _ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR: 2108_8700h Init: _ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR: 210A_8600h Init: _ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR: 210A_8700h Init: _ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR: 210C_8600h Init: _ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR: 210C_8700h Init: _ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR: 210E_8600h Init: _ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR: 210E_8700h Init: _ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR: 2180_8600h Init: _ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR: 2180_8700h Init: _ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR: 2182_8600h Init: _ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR: 2182_8700h Init: _ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR: 2184_8600h

Init: _ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR: 2184_8700h
Init: _ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR: 2186_8600h
Init: _ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR: 2186_8700h
Init: _ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR: 2188_8600h
Init: _ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR: 2188_8700h
Init: _ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR: 218A_8600h
Init: _ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR: 218A_8700h
Init: _ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR: 218C_8600h
Init: _ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR: 218C_8700h
Init: _ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR: 218E_8600h
Init: _ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR: 218E_8700h
Init: _ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR: 2200_8600h
Init: _ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR: 2200_8700h
Init: _ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR: 2202_8600h
Init: _ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR: 2202_8700h
Init: _ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR: 2204_8600h
Init: _ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR: 2204_8700h
Init: _ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR: 2206_8600h
Init: _ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR: 2206_8700h
Init: _ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR: 2208_8600h
Init: _ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR: 2208_8700h
Init: _ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR: 220A_8600h
Init: _ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR: 220A_8700h
Init: _ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR: 220C_8600h
Init: _ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR: 220C_8700h
Init: _ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR: 220E_8600h
Init: _ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR: 220E_8700h
Init: _ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR: 2280_8600h
Init: _ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR: 2280_8700h
Init: _ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR: 2282_8600h
Init: _ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR: 2282_8700h
Init: _ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR: 2284_8600h
Init: _ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR: 2284_8700h
Init: _ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR: 2286_8600h
Init: _ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR: 2286_8700h
Init: _ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR: 2288_8600h
Init: _ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR: 2288_8700h
Init: _ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR: 228A_8600h
Init: _ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR: 228A_8700h
Init: _ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR: 228C_8600h
Init: _ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR: 228C_8700h
Init: _ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR: 228E_8600h
Init: _ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR: 228E_8700h
Init: _ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR: 2300_8600h
Init: _ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR: 2300_8700h
Init: _ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR: 2302_8600h
Init: _ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR: 2302_8700h
Init: _ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR: 2304_8600h
Init: _ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR: 2304_8700h
Init: _ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR: 2306_8600h
Init: _ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR: 2306_8700h
Init: _ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR: 2308_8600h
Init: _ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR: 2308_8700h
Init: _ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR: 230A_8600h
Init: _ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR: 230A_8700h
Init: _ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR: 230C_8600h
Init: _ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR: 230C_8700h
Init: _ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR: 230E_8600h
Init: _ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR: 230E_8700h
Init: _ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR: 2380_8600h
Init: _ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR: 2380_8700h
Init: _ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR: 2382_8600h
Init: _ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR: 2382_8700h
Init: _ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR: 2384_8600h
Init: _ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR: 2384_8700h

Init: _ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR: 2386_8600h
Init: _ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR: 2386_8700h
Init: _ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR: 2388_8600h
Init: _ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR: 2388_8700h
Init: _ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR: 238A_8600h
Init: _ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR: 238A_8700h
Init: _ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR: 238C_8600h
Init: _ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR: 238C_8700h
Init: _ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR: 238E_8600h
Init: _ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR: 238E_8700h
Init: _ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR: 2400_8600h
Init: _ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR: 2400_8700h
Init: _ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR: 2402_8600h
Init: _ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR: 2402_8700h
Init: _ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR: 2404_8600h
Init: _ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR: 2404_8700h
Init: _ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR: 2406_8600h
Init: _ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR: 2406_8700h
Init: _ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR: 2408_8600h
Init: _ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR: 2408_8700h
Init: _ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR: 240A_8600h
Init: _ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR: 240A_8700h
Init: _ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR: 240C_8600h
Init: _ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR: 240C_8700h
Init: _ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR: 240E_8600h
Init: _ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR: 240E_8700h
Init: _ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR: 2480_8600h
Init: _ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR: 2480_8700h
Init: _ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR: 2482_8600h
Init: _ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR: 2482_8700h
Init: _ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR: 2484_8600h
Init: _ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR: 2484_8700h
Init: _ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR: 2486_8600h
Init: _ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR: 2486_8700h
Init: _ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR: 2488_8600h
Init: _ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR: 2488_8700h
Init: _ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR: 248A_8600h
Init: _ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR: 248A_8700h
Init: _ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR: 248C_8600h
Init: _ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR: 248C_8700h
Init: _ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR: 248E_8600h
Init: _ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR: 248E_8700h
Init: _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR: 2500_8600h
Init: _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR: 2500_8700h
Init: _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR: 2502_8600h
Init: _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR: 2502_8700h
Init: _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR: 2504_8600h
Init: _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR: 2504_8700h
Init: _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR: 2506_8600h
Init: _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR: 2506_8700h
Init: _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR: 2508_8600h
Init: _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR: 2508_8700h
Init: _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR: 250A_8600h
Init: _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR: 250A_8700h
Init: _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR: 250C_8600h
Init: _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR: 250C_8700h
Init: _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR: 250E_8600h
Init: _ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR: 250E_8700h
Init: _ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR: 2580_8600h
Init: _ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR: 2580_8700h
Init: _ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR: 2582_8600h
Init: _ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR: 2582_8700h
Init: _ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR: 2584_8600h
Init: _ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR: 2584_8700h
Init: _ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR: 2586_8600h

Init: _ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR: 2586_8700h
Init: _ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR: 2588_8600h
Init: _ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR: 2588_8700h
Init: _ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR: 258A_8600h
Init: _ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR: 258A_8700h
Init: _ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR: 258C_8600h
Init: _ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR: 258C_8700h
Init: _ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR: 258E_8600h
Init: _ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR: 258E_8700h

MSRC000_2066 [FP Machine Check Syndrome Thread 0] (MCA::FP::MCA_SYND_FP)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::FP::MCA_STATUS_FP [Thread 0](#)

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC000_2066
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC000_2066
 _ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC000_2066
 _ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC000_2066
 _ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC000_2066

_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC000_2066
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC000_2066
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC000_2066
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC000_2066
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC000_2066
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC000_2066
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC000_2066
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC000_2066
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC000_2066
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC000_2066
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC000_2066
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC000_2066
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC000_2066
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC000_2066
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC000_2066
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC000_2066
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC000_2066
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC000_2066
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC000_2066
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC000_2066
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC000_2066
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC000_2066
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC000_2066
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC000_2066
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC000_2066
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC000_2066
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC000_2066
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC000_2066
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC000_2066
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC000_2066
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC000_2066
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC000_2066
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC000_2066
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC000_2066
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC000_2066
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC000_2066
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC000_2066
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC000_2066
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC000_2066
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC000_2066
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC000_2066
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC000_2066
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC000_2066
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC000_2066
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC000_2066
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC000_2066
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC000_2066
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC000_2066
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC000_2066
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC000_2066
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC000_2066
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC000_2066
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC000_2066
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC000_2066
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC000_2066
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC000_2066
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC000_2066
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC000_2066
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC000_2066
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC000_2066
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC000_2066
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC000_2066
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC000_2066
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC000_2066
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC000_2066

_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC000_2066
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC000_2066
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC000_2066
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC000_2066
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC000_2066
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC000_2066
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC000_2066
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC000_2066
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC000_2066
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC000_2066
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC000_2066
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC000_2066
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC000_2066
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC000_2066
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC000_2066
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC000_2066
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC000_2066
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC000_2066
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC000_2066
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC000_2066
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC000_2066
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC000_2066
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC000_2066
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC000_2066
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC000_2066
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC000_2066
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC000_2066
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC000_2066
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC000_2066
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC000_2066
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC000_2066
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC000_2066
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC000_2066
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC000_2066
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC000_2066
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC000_2066
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC000_2066
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC000_2066
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC000_2066
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC000_2066
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC000_2066
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC000_2066
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC000_2066
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC000_2066
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC000_2066
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC000_2066
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC000_2066
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC000_2066
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC000_2066
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC000_2066
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC000_2066
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC000_2066
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC000_2066
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC000_2066
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC000_2066
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC000_2066
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC000_2066
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC000_2066
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC000_2066
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC000_2066
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC000_2066
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC000_2066
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC000_2066
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC000_2066
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC000_2066

_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC000_2066	
Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::FP::MCA_STATUS_FP. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::FP::MCA_SYND_FP[Length]. The Syndrome field is only valid when MCA::FP::MCA_SYND_FP[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::FP::MCA_SYND_FP. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::FP::MCA_SYND_FP[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::FP::MCA_SYND_FP. For example, a syndrome length of 9 means that MCA::FP::MCA_SYND_FP[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 75 [MCA_SYND_FP].

Table 75: MCA_SYND_FP

Error Type	Bits	Description
PRF	[17] [16] [15:12] [11:4] [3:0]	512-bit operation. Reserved. Data path ID. PRN. Read port ID.
FL	[17:0]	Reserved
SCH	[17:0]	Reserved
NSQ	[17:0]	Reserved
RQ	[17:0]	Reserved
SRF	[17:0]	Reserved
HWA	[17:0]	Reserved
KRF	[17:0]	Reserved

MSRC001_0406 [FP Machine Check Control Mask Thread 0] (MCA::FP::MCA_CTL_MASK_FP)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_core0_thread0_inst6_n0_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core0_thread1_inst6_n1_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core1_thread0_inst6_n2_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core1_thread1_inst6_n3_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core2_thread0_inst6_n4_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core2_thread1_inst6_n5_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core3_thread0_inst6_n6_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core3_thread1_inst6_n7_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core4_thread0_inst6_n8_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core4_thread1_inst6_n9_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core5_thread0_inst6_n10_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core5_thread1_inst6_n11_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core6_thread0_inst6_n12_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core6_thread1_inst6_n13_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core7_thread0_inst6_n14_aliasMSR; MSRC001_0406
 _ccd0_lthree0_core7_thread1_inst6_n15_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core0_thread0_inst6_n160_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core0_thread1_inst6_n161_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core1_thread0_inst6_n162_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core1_thread1_inst6_n163_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core2_thread0_inst6_n164_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core2_thread1_inst6_n165_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core3_thread0_inst6_n166_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core3_thread1_inst6_n167_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core4_thread0_inst6_n168_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core4_thread1_inst6_n169_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core5_thread0_inst6_n170_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core5_thread1_inst6_n171_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core6_thread0_inst6_n172_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core6_thread1_inst6_n173_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core7_thread0_inst6_n174_aliasMSR; MSRC001_0406
 _ccd10_lthree0_core7_thread1_inst6_n175_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core0_thread0_inst6_n176_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core0_thread1_inst6_n177_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core1_thread0_inst6_n178_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core1_thread1_inst6_n179_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core2_thread0_inst6_n180_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core2_thread1_inst6_n181_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core3_thread0_inst6_n182_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core3_thread1_inst6_n183_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core4_thread0_inst6_n184_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core4_thread1_inst6_n185_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core5_thread0_inst6_n186_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core5_thread1_inst6_n187_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core6_thread0_inst6_n188_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core6_thread1_inst6_n189_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core7_thread0_inst6_n190_aliasMSR; MSRC001_0406
 _ccd11_lthree0_core7_thread1_inst6_n191_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core0_thread0_inst6_n16_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core0_thread1_inst6_n17_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core1_thread0_inst6_n18_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core1_thread1_inst6_n19_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core2_thread0_inst6_n20_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core2_thread1_inst6_n21_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core3_thread0_inst6_n22_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core3_thread1_inst6_n23_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core4_thread0_inst6_n24_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core4_thread1_inst6_n25_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core5_thread0_inst6_n26_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core5_thread1_inst6_n27_aliasMSR; MSRC001_0406
 _ccd1_lthree0_core6_thread0_inst6_n28_aliasMSR; MSRC001_0406

_ccd1_lthree0_core6_thread1_inst6_n29_aliasMSR; MSRC001_0406
_ccd1_lthree0_core7_thread0_inst6_n30_aliasMSR; MSRC001_0406
_ccd1_lthree0_core7_thread1_inst6_n31_aliasMSR; MSRC001_0406
_ccd2_lthree0_core0_thread0_inst6_n32_aliasMSR; MSRC001_0406
_ccd2_lthree0_core0_thread1_inst6_n33_aliasMSR; MSRC001_0406
_ccd2_lthree0_core1_thread0_inst6_n34_aliasMSR; MSRC001_0406
_ccd2_lthree0_core1_thread1_inst6_n35_aliasMSR; MSRC001_0406
_ccd2_lthree0_core2_thread0_inst6_n36_aliasMSR; MSRC001_0406
_ccd2_lthree0_core2_thread1_inst6_n37_aliasMSR; MSRC001_0406
_ccd2_lthree0_core3_thread0_inst6_n38_aliasMSR; MSRC001_0406
_ccd2_lthree0_core3_thread1_inst6_n39_aliasMSR; MSRC001_0406
_ccd2_lthree0_core4_thread0_inst6_n40_aliasMSR; MSRC001_0406
_ccd2_lthree0_core4_thread1_inst6_n41_aliasMSR; MSRC001_0406
_ccd2_lthree0_core5_thread0_inst6_n42_aliasMSR; MSRC001_0406
_ccd2_lthree0_core5_thread1_inst6_n43_aliasMSR; MSRC001_0406
_ccd2_lthree0_core6_thread0_inst6_n44_aliasMSR; MSRC001_0406
_ccd2_lthree0_core6_thread1_inst6_n45_aliasMSR; MSRC001_0406
_ccd2_lthree0_core7_thread0_inst6_n46_aliasMSR; MSRC001_0406
_ccd2_lthree0_core7_thread1_inst6_n47_aliasMSR; MSRC001_0406
_ccd3_lthree0_core0_thread0_inst6_n48_aliasMSR; MSRC001_0406
_ccd3_lthree0_core0_thread1_inst6_n49_aliasMSR; MSRC001_0406
_ccd3_lthree0_core1_thread0_inst6_n50_aliasMSR; MSRC001_0406
_ccd3_lthree0_core1_thread1_inst6_n51_aliasMSR; MSRC001_0406
_ccd3_lthree0_core2_thread0_inst6_n52_aliasMSR; MSRC001_0406
_ccd3_lthree0_core2_thread1_inst6_n53_aliasMSR; MSRC001_0406
_ccd3_lthree0_core3_thread0_inst6_n54_aliasMSR; MSRC001_0406
_ccd3_lthree0_core3_thread1_inst6_n55_aliasMSR; MSRC001_0406
_ccd3_lthree0_core4_thread0_inst6_n56_aliasMSR; MSRC001_0406
_ccd3_lthree0_core4_thread1_inst6_n57_aliasMSR; MSRC001_0406
_ccd3_lthree0_core5_thread0_inst6_n58_aliasMSR; MSRC001_0406
_ccd3_lthree0_core5_thread1_inst6_n59_aliasMSR; MSRC001_0406
_ccd3_lthree0_core6_thread0_inst6_n60_aliasMSR; MSRC001_0406
_ccd3_lthree0_core6_thread1_inst6_n61_aliasMSR; MSRC001_0406
_ccd3_lthree0_core7_thread0_inst6_n62_aliasMSR; MSRC001_0406
_ccd3_lthree0_core7_thread1_inst6_n63_aliasMSR; MSRC001_0406
_ccd4_lthree0_core0_thread0_inst6_n64_aliasMSR; MSRC001_0406
_ccd4_lthree0_core0_thread1_inst6_n65_aliasMSR; MSRC001_0406
_ccd4_lthree0_core1_thread0_inst6_n66_aliasMSR; MSRC001_0406
_ccd4_lthree0_core1_thread1_inst6_n67_aliasMSR; MSRC001_0406
_ccd4_lthree0_core2_thread0_inst6_n68_aliasMSR; MSRC001_0406
_ccd4_lthree0_core2_thread1_inst6_n69_aliasMSR; MSRC001_0406
_ccd4_lthree0_core3_thread0_inst6_n70_aliasMSR; MSRC001_0406
_ccd4_lthree0_core3_thread1_inst6_n71_aliasMSR; MSRC001_0406
_ccd4_lthree0_core4_thread0_inst6_n72_aliasMSR; MSRC001_0406
_ccd4_lthree0_core4_thread1_inst6_n73_aliasMSR; MSRC001_0406
_ccd4_lthree0_core5_thread0_inst6_n74_aliasMSR; MSRC001_0406
_ccd4_lthree0_core5_thread1_inst6_n75_aliasMSR; MSRC001_0406
_ccd4_lthree0_core6_thread0_inst6_n76_aliasMSR; MSRC001_0406
_ccd4_lthree0_core6_thread1_inst6_n77_aliasMSR; MSRC001_0406
_ccd4_lthree0_core7_thread0_inst6_n78_aliasMSR; MSRC001_0406
_ccd4_lthree0_core7_thread1_inst6_n79_aliasMSR; MSRC001_0406
_ccd5_lthree0_core0_thread0_inst6_n80_aliasMSR; MSRC001_0406
_ccd5_lthree0_core0_thread1_inst6_n81_aliasMSR; MSRC001_0406
_ccd5_lthree0_core1_thread0_inst6_n82_aliasMSR; MSRC001_0406
_ccd5_lthree0_core1_thread1_inst6_n83_aliasMSR; MSRC001_0406
_ccd5_lthree0_core2_thread0_inst6_n84_aliasMSR; MSRC001_0406
_ccd5_lthree0_core2_thread1_inst6_n85_aliasMSR; MSRC001_0406
_ccd5_lthree0_core3_thread0_inst6_n86_aliasMSR; MSRC001_0406
_ccd5_lthree0_core3_thread1_inst6_n87_aliasMSR; MSRC001_0406
_ccd5_lthree0_core4_thread0_inst6_n88_aliasMSR; MSRC001_0406
_ccd5_lthree0_core4_thread1_inst6_n89_aliasMSR; MSRC001_0406
_ccd5_lthree0_core5_thread0_inst6_n90_aliasMSR; MSRC001_0406
_ccd5_lthree0_core5_thread1_inst6_n91_aliasMSR; MSRC001_0406
_ccd5_lthree0_core6_thread0_inst6_n92_aliasMSR; MSRC001_0406
_ccd5_lthree0_core6_thread1_inst6_n93_aliasMSR; MSRC001_0406

_ccd5_lthree0_core7_thread0_inst6_n94_aliasMSR; MSRC001_0406
_ccd5_lthree0_core7_thread1_inst6_n95_aliasMSR; MSRC001_0406
_ccd6_lthree0_core0_thread0_inst6_n96_aliasMSR; MSRC001_0406
_ccd6_lthree0_core0_thread1_inst6_n97_aliasMSR; MSRC001_0406
_ccd6_lthree0_core1_thread0_inst6_n98_aliasMSR; MSRC001_0406
_ccd6_lthree0_core1_thread1_inst6_n99_aliasMSR; MSRC001_0406
_ccd6_lthree0_core2_thread0_inst6_n100_aliasMSR; MSRC001_0406
_ccd6_lthree0_core2_thread1_inst6_n101_aliasMSR; MSRC001_0406
_ccd6_lthree0_core3_thread0_inst6_n102_aliasMSR; MSRC001_0406
_ccd6_lthree0_core3_thread1_inst6_n103_aliasMSR; MSRC001_0406
_ccd6_lthree0_core4_thread0_inst6_n104_aliasMSR; MSRC001_0406
_ccd6_lthree0_core4_thread1_inst6_n105_aliasMSR; MSRC001_0406
_ccd6_lthree0_core5_thread0_inst6_n106_aliasMSR; MSRC001_0406
_ccd6_lthree0_core5_thread1_inst6_n107_aliasMSR; MSRC001_0406
_ccd6_lthree0_core6_thread0_inst6_n108_aliasMSR; MSRC001_0406
_ccd6_lthree0_core6_thread1_inst6_n109_aliasMSR; MSRC001_0406
_ccd6_lthree0_core7_thread0_inst6_n110_aliasMSR; MSRC001_0406
_ccd6_lthree0_core7_thread1_inst6_n111_aliasMSR; MSRC001_0406
_ccd7_lthree0_core0_thread0_inst6_n112_aliasMSR; MSRC001_0406
_ccd7_lthree0_core0_thread1_inst6_n113_aliasMSR; MSRC001_0406
_ccd7_lthree0_core1_thread0_inst6_n114_aliasMSR; MSRC001_0406
_ccd7_lthree0_core1_thread1_inst6_n115_aliasMSR; MSRC001_0406
_ccd7_lthree0_core2_thread0_inst6_n116_aliasMSR; MSRC001_0406
_ccd7_lthree0_core2_thread1_inst6_n117_aliasMSR; MSRC001_0406
_ccd7_lthree0_core3_thread0_inst6_n118_aliasMSR; MSRC001_0406
_ccd7_lthree0_core3_thread1_inst6_n119_aliasMSR; MSRC001_0406
_ccd7_lthree0_core4_thread0_inst6_n120_aliasMSR; MSRC001_0406
_ccd7_lthree0_core4_thread1_inst6_n121_aliasMSR; MSRC001_0406
_ccd7_lthree0_core5_thread0_inst6_n122_aliasMSR; MSRC001_0406
_ccd7_lthree0_core5_thread1_inst6_n123_aliasMSR; MSRC001_0406
_ccd7_lthree0_core6_thread0_inst6_n124_aliasMSR; MSRC001_0406
_ccd7_lthree0_core6_thread1_inst6_n125_aliasMSR; MSRC001_0406
_ccd7_lthree0_core7_thread0_inst6_n126_aliasMSR; MSRC001_0406
_ccd7_lthree0_core7_thread1_inst6_n127_aliasMSR; MSRC001_0406
_ccd8_lthree0_core0_thread0_inst6_n128_aliasMSR; MSRC001_0406
_ccd8_lthree0_core0_thread1_inst6_n129_aliasMSR; MSRC001_0406
_ccd8_lthree0_core1_thread0_inst6_n130_aliasMSR; MSRC001_0406
_ccd8_lthree0_core1_thread1_inst6_n131_aliasMSR; MSRC001_0406
_ccd8_lthree0_core2_thread0_inst6_n132_aliasMSR; MSRC001_0406
_ccd8_lthree0_core2_thread1_inst6_n133_aliasMSR; MSRC001_0406
_ccd8_lthree0_core3_thread0_inst6_n134_aliasMSR; MSRC001_0406
_ccd8_lthree0_core3_thread1_inst6_n135_aliasMSR; MSRC001_0406
_ccd8_lthree0_core4_thread0_inst6_n136_aliasMSR; MSRC001_0406
_ccd8_lthree0_core4_thread1_inst6_n137_aliasMSR; MSRC001_0406
_ccd8_lthree0_core5_thread0_inst6_n138_aliasMSR; MSRC001_0406
_ccd8_lthree0_core5_thread1_inst6_n139_aliasMSR; MSRC001_0406
_ccd8_lthree0_core6_thread0_inst6_n140_aliasMSR; MSRC001_0406
_ccd8_lthree0_core6_thread1_inst6_n141_aliasMSR; MSRC001_0406
_ccd8_lthree0_core7_thread0_inst6_n142_aliasMSR; MSRC001_0406
_ccd8_lthree0_core7_thread1_inst6_n143_aliasMSR; MSRC001_0406
_ccd9_lthree0_core0_thread0_inst6_n144_aliasMSR; MSRC001_0406
_ccd9_lthree0_core0_thread1_inst6_n145_aliasMSR; MSRC001_0406
_ccd9_lthree0_core1_thread0_inst6_n146_aliasMSR; MSRC001_0406
_ccd9_lthree0_core1_thread1_inst6_n147_aliasMSR; MSRC001_0406
_ccd9_lthree0_core2_thread0_inst6_n148_aliasMSR; MSRC001_0406
_ccd9_lthree0_core2_thread1_inst6_n149_aliasMSR; MSRC001_0406
_ccd9_lthree0_core3_thread0_inst6_n150_aliasMSR; MSRC001_0406
_ccd9_lthree0_core3_thread1_inst6_n151_aliasMSR; MSRC001_0406
_ccd9_lthree0_core4_thread0_inst6_n152_aliasMSR; MSRC001_0406
_ccd9_lthree0_core4_thread1_inst6_n153_aliasMSR; MSRC001_0406
_ccd9_lthree0_core5_thread0_inst6_n154_aliasMSR; MSRC001_0406
_ccd9_lthree0_core5_thread1_inst6_n155_aliasMSR; MSRC001_0406
_ccd9_lthree0_core6_thread0_inst6_n156_aliasMSR; MSRC001_0406
_ccd9_lthree0_core6_thread1_inst6_n157_aliasMSR; MSRC001_0406
_ccd9_lthree0_core7_thread0_inst6_n158_aliasMSR; MSRC001_0406

_ccd9_lthree0_core7_thread1_inst6_n159_aliasMSR; MSRC001_0406	
Bits	Description
63:8	Reserved.
7	KRF. Read-write. Reset: 0. Physical K mask register file (KRF) parity error.
6	HWA. Read-write. Reset: 0. Hardware assertion.
5	SRF. Read-write. Reset: 0. Status register file (SRF) parity error.
4	RQ. Read-write. Reset: 0. Retire queue (RQ) parity error.
3	NSQ. Read-write. Reset: 0. NSQ parity error.
2	SCH. Read-write. Reset: 0. Schedule queue parity error.
1	FL. Read-write. Reset: 0. Freelist (FL) parity error.
0	PRF. Read-write. Reset: 0. Physical register file (PRF) parity error.

3.2.5.7 L3

MSR0000_041C...MSRC000_20E0 [L3 Machine Check Control] (MCA::L3::MCA_CTL_L3)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::L3::MCA_CTL_L3 register must be enabled by the corresponding enable bit in Core::X86::Msrr:[MCG_CTL](#). Does not affect error detection, correction, or logging.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A0
 _ccd0_lthree0_inst10_n3_aliasMSRLEGACY; MSRC000_0428
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B0
 _ccd0_lthree0_inst11_n4_aliasMSRLEGACY; MSRC000_042C
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C0
 _ccd0_lthree0_inst12_n5_aliasMSRLEGACY; MSRC000_0430
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D0
 _ccd0_lthree0_inst13_n6_aliasMSRLEGACY; MSRC000_0434
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E0
 _ccd0_lthree0_inst14_n7_aliasMSRLEGACY; MSRC000_0438
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2070
 _ccd0_lthree0_inst7_n0_aliasMSRLEGACY; MSRC000_041C
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2080
 _ccd0_lthree0_inst8_n1_aliasMSRLEGACY; MSRC000_0420
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2090
 _ccd0_lthree0_inst9_n2_aliasMSRLEGACY; MSRC000_0424
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A0
 _ccd10_lthree0_inst10_n83_aliasMSRLEGACY; MSRC000_0428
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B0
 _ccd10_lthree0_inst11_n84_aliasMSRLEGACY; MSRC000_042C
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C0
 _ccd10_lthree0_inst12_n85_aliasMSRLEGACY; MSRC000_0430
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D0
 _ccd10_lthree0_inst13_n86_aliasMSRLEGACY; MSRC000_0434
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E0
 _ccd10_lthree0_inst14_n87_aliasMSRLEGACY; MSRC000_0438
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2070
 _ccd10_lthree0_inst7_n80_aliasMSRLEGACY; MSRC000_041C
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2080
 _ccd10_lthree0_inst8_n81_aliasMSRLEGACY; MSRC000_0420
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2090
 _ccd10_lthree0_inst9_n82_aliasMSRLEGACY; MSRC000_0424
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A0
 _ccd11_lthree0_inst10_n91_aliasMSRLEGACY; MSRC000_0428
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B0
 _ccd11_lthree0_inst11_n92_aliasMSRLEGACY; MSRC000_042C
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C0
 _ccd11_lthree0_inst12_n93_aliasMSRLEGACY; MSRC000_0430
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D0
 _ccd11_lthree0_inst13_n94_aliasMSRLEGACY; MSRC000_0434
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E0
 _ccd11_lthree0_inst14_n95_aliasMSRLEGACY; MSRC000_0438
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2070
 _ccd11_lthree0_inst7_n88_aliasMSRLEGACY; MSRC000_041C
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2080
 _ccd11_lthree0_inst8_n89_aliasMSRLEGACY; MSRC000_0420
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2090
 _ccd11_lthree0_inst9_n90_aliasMSRLEGACY; MSRC000_0424
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A0
 _ccd1_lthree0_inst10_n11_aliasMSRLEGACY; MSRC000_0428
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B0
 _ccd1_lthree0_inst11_n12_aliasMSRLEGACY; MSRC000_042C
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C0
 _ccd1_lthree0_inst12_n13_aliasMSRLEGACY; MSRC000_0430
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D0
 _ccd1_lthree0_inst13_n14_aliasMSRLEGACY; MSRC000_0434
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E0
 _ccd1_lthree0_inst14_n15_aliasMSRLEGACY; MSRC000_0438

_ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2070
_ccd1_lthree0_inst7_n8_aliasMSRLEGACY; MSR0000_041C
_ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2080
_ccd1_lthree0_inst8_n9_aliasMSRLEGACY; MSR0000_0420
_ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2090
_ccd1_lthree0_inst9_n10_aliasMSRLEGACY; MSR0000_0424
_ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A0
_ccd2_lthree0_inst10_n19_aliasMSRLEGACY; MSR0000_0428
_ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B0
_ccd2_lthree0_inst11_n20_aliasMSRLEGACY; MSR0000_042C
_ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C0
_ccd2_lthree0_inst12_n21_aliasMSRLEGACY; MSR0000_0430
_ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D0
_ccd2_lthree0_inst13_n22_aliasMSRLEGACY; MSR0000_0434
_ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E0
_ccd2_lthree0_inst14_n23_aliasMSRLEGACY; MSR0000_0438
_ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2070
_ccd2_lthree0_inst7_n16_aliasMSRLEGACY; MSR0000_041C
_ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2080
_ccd2_lthree0_inst8_n17_aliasMSRLEGACY; MSR0000_0420
_ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2090
_ccd2_lthree0_inst9_n18_aliasMSRLEGACY; MSR0000_0424
_ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A0
_ccd3_lthree0_inst10_n27_aliasMSRLEGACY; MSR0000_0428
_ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B0
_ccd3_lthree0_inst11_n28_aliasMSRLEGACY; MSR0000_042C
_ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C0
_ccd3_lthree0_inst12_n29_aliasMSRLEGACY; MSR0000_0430
_ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D0
_ccd3_lthree0_inst13_n30_aliasMSRLEGACY; MSR0000_0434
_ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E0
_ccd3_lthree0_inst14_n31_aliasMSRLEGACY; MSR0000_0438
_ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2070
_ccd3_lthree0_inst7_n24_aliasMSRLEGACY; MSR0000_041C
_ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2080
_ccd3_lthree0_inst8_n25_aliasMSRLEGACY; MSR0000_0420
_ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2090
_ccd3_lthree0_inst9_n26_aliasMSRLEGACY; MSR0000_0424
_ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A0
_ccd4_lthree0_inst10_n35_aliasMSRLEGACY; MSR0000_0428
_ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B0
_ccd4_lthree0_inst11_n36_aliasMSRLEGACY; MSR0000_042C
_ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C0
_ccd4_lthree0_inst12_n37_aliasMSRLEGACY; MSR0000_0430
_ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D0
_ccd4_lthree0_inst13_n38_aliasMSRLEGACY; MSR0000_0434
_ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E0
_ccd4_lthree0_inst14_n39_aliasMSRLEGACY; MSR0000_0438
_ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2070
_ccd4_lthree0_inst7_n32_aliasMSRLEGACY; MSR0000_041C
_ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2080
_ccd4_lthree0_inst8_n33_aliasMSRLEGACY; MSR0000_0420
_ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2090
_ccd4_lthree0_inst9_n34_aliasMSRLEGACY; MSR0000_0424
_ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A0
_ccd5_lthree0_inst10_n43_aliasMSRLEGACY; MSR0000_0428
_ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B0
_ccd5_lthree0_inst11_n44_aliasMSRLEGACY; MSR0000_042C
_ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C0
_ccd5_lthree0_inst12_n45_aliasMSRLEGACY; MSR0000_0430
_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D0
_ccd5_lthree0_inst13_n46_aliasMSRLEGACY; MSR0000_0434
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E0
_ccd5_lthree0_inst14_n47_aliasMSRLEGACY; MSR0000_0438
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2070

_ccd5_lthree0_inst7_n40_aliasMSRLEGACY; MSR0000_041C
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2080
_ccd5_lthree0_inst8_n41_aliasMSRLEGACY; MSR0000_0420
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2090
_ccd5_lthree0_inst9_n42_aliasMSRLEGACY; MSR0000_0424
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A0
_ccd6_lthree0_inst10_n51_aliasMSRLEGACY; MSR0000_0428
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B0
_ccd6_lthree0_inst11_n52_aliasMSRLEGACY; MSR0000_042C
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C0
_ccd6_lthree0_inst12_n53_aliasMSRLEGACY; MSR0000_0430
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D0
_ccd6_lthree0_inst13_n54_aliasMSRLEGACY; MSR0000_0434
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E0
_ccd6_lthree0_inst14_n55_aliasMSRLEGACY; MSR0000_0438
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2070
_ccd6_lthree0_inst7_n48_aliasMSRLEGACY; MSR0000_041C
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2080
_ccd6_lthree0_inst8_n49_aliasMSRLEGACY; MSR0000_0420
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2090
_ccd6_lthree0_inst9_n50_aliasMSRLEGACY; MSR0000_0424
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A0
_ccd7_lthree0_inst10_n59_aliasMSRLEGACY; MSR0000_0428
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B0
_ccd7_lthree0_inst11_n60_aliasMSRLEGACY; MSR0000_042C
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C0
_ccd7_lthree0_inst12_n61_aliasMSRLEGACY; MSR0000_0430
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D0
_ccd7_lthree0_inst13_n62_aliasMSRLEGACY; MSR0000_0434
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E0
_ccd7_lthree0_inst14_n63_aliasMSRLEGACY; MSR0000_0438
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2070
_ccd7_lthree0_inst7_n56_aliasMSRLEGACY; MSR0000_041C
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2080
_ccd7_lthree0_inst8_n57_aliasMSRLEGACY; MSR0000_0420
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2090
_ccd7_lthree0_inst9_n58_aliasMSRLEGACY; MSR0000_0424
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A0
_ccd8_lthree0_inst10_n67_aliasMSRLEGACY; MSR0000_0428
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B0
_ccd8_lthree0_inst11_n68_aliasMSRLEGACY; MSR0000_042C
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C0
_ccd8_lthree0_inst12_n69_aliasMSRLEGACY; MSR0000_0430
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D0
_ccd8_lthree0_inst13_n70_aliasMSRLEGACY; MSR0000_0434
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E0
_ccd8_lthree0_inst14_n71_aliasMSRLEGACY; MSR0000_0438
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2070
_ccd8_lthree0_inst7_n64_aliasMSRLEGACY; MSR0000_041C
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2080
_ccd8_lthree0_inst8_n65_aliasMSRLEGACY; MSR0000_0420
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2090
_ccd8_lthree0_inst9_n66_aliasMSRLEGACY; MSR0000_0424
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A0
_ccd9_lthree0_inst10_n75_aliasMSRLEGACY; MSR0000_0428
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B0
_ccd9_lthree0_inst11_n76_aliasMSRLEGACY; MSR0000_042C
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C0
_ccd9_lthree0_inst12_n77_aliasMSRLEGACY; MSR0000_0430
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D0
_ccd9_lthree0_inst13_n78_aliasMSRLEGACY; MSR0000_0434
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E0
_ccd9_lthree0_inst14_n79_aliasMSRLEGACY; MSR0000_0438
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2070
_ccd9_lthree0_inst7_n72_aliasMSRLEGACY; MSR0000_041C

_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2080	
_ccd9_lthree0_inst8_n73_aliasMSRLEGACY; MSR0000_0420	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2090	
_ccd9_lthree0_inst9_n74_aliasMSRLEGACY; MSR0000_0424	
Bits	Description
63:9	Reserved.
8	XiWcbParityPoison. Read-write. Reset: 0. Xi Wcb Parity Poison Creation Event
7	Hwa. Read-write. Reset: 0. L3 Hardware Assertion.
6	XiVictimQueue. Read-write. Reset: 0. L3 Victim Queue Data Fabric Error.
5	SdpParity. Read-write. Reset: 0. SDP Parity Error from XI.
4	DataArray. Read-write. Reset: 0. L3M Data ECC Error.
3	MultiHitTag. Read-write. Reset: 0. L3M Tag Multi-way-hit Error.
2	Tag. Read-write. Reset: 0. L3M Tag ECC Error. MCA_STATUS_L3[CECC]=1 and MCA_STATUS[UC]=1 indicates a hard fault.
1	MultiHitShadowTag. Read-write. Reset: 0. Shadow Tag Macro Multi-way-hit Error.
0	ShadowTag. Read-write. Reset: 0. Shadow Tag Macro ECC Error. MCA_STATUS_L3[CECC]=1 and MCA_STATUS[UC]=1 indicates a hard fault.

MSR0000_041D...MSRC000_20E1 [L3 Machine Check Status] (MCA::L3::MCA_STATUS_L3)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A1
 _ccd0_lthree0_inst10_n3_aliasMSRLEGACY; MSR0000_0429
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B1
 _ccd0_lthree0_inst11_n4_aliasMSRLEGACY; MSR0000_042D
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C1
 _ccd0_lthree0_inst12_n5_aliasMSRLEGACY; MSR0000_0431
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D1
 _ccd0_lthree0_inst13_n6_aliasMSRLEGACY; MSR0000_0435
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E1
 _ccd0_lthree0_inst14_n7_aliasMSRLEGACY; MSR0000_0439
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2071
 _ccd0_lthree0_inst7_n0_aliasMSRLEGACY; MSR0000_041D
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2081
 _ccd0_lthree0_inst8_n1_aliasMSRLEGACY; MSR0000_0421
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2091
 _ccd0_lthree0_inst9_n2_aliasMSRLEGACY; MSR0000_0425
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A1
 _ccd10_lthree0_inst10_n83_aliasMSRLEGACY; MSR0000_0429
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B1
 _ccd10_lthree0_inst11_n84_aliasMSRLEGACY; MSR0000_042D
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C1
 _ccd10_lthree0_inst12_n85_aliasMSRLEGACY; MSR0000_0431
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D1
 _ccd10_lthree0_inst13_n86_aliasMSRLEGACY; MSR0000_0435
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E1
 _ccd10_lthree0_inst14_n87_aliasMSRLEGACY; MSR0000_0439
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2071
 _ccd10_lthree0_inst7_n80_aliasMSRLEGACY; MSR0000_041D
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2081
 _ccd10_lthree0_inst8_n81_aliasMSRLEGACY; MSR0000_0421
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2091
 _ccd10_lthree0_inst9_n82_aliasMSRLEGACY; MSR0000_0425
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A1
 _ccd11_lthree0_inst10_n91_aliasMSRLEGACY; MSR0000_0429
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B1
 _ccd11_lthree0_inst11_n92_aliasMSRLEGACY; MSR0000_042D
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C1
 _ccd11_lthree0_inst12_n93_aliasMSRLEGACY; MSR0000_0431
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D1
 _ccd11_lthree0_inst13_n94_aliasMSRLEGACY; MSR0000_0435
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E1
 _ccd11_lthree0_inst14_n95_aliasMSRLEGACY; MSR0000_0439
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2071
 _ccd11_lthree0_inst7_n88_aliasMSRLEGACY; MSR0000_041D
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2081
 _ccd11_lthree0_inst8_n89_aliasMSRLEGACY; MSR0000_0421
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2091
 _ccd11_lthree0_inst9_n90_aliasMSRLEGACY; MSR0000_0425
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A1
 _ccd1_lthree0_inst10_n11_aliasMSRLEGACY; MSR0000_0429
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B1
 _ccd1_lthree0_inst11_n12_aliasMSRLEGACY; MSR0000_042D
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C1
 _ccd1_lthree0_inst12_n13_aliasMSRLEGACY; MSR0000_0431
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D1
 _ccd1_lthree0_inst13_n14_aliasMSRLEGACY; MSR0000_0435
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E1
 _ccd1_lthree0_inst14_n15_aliasMSRLEGACY; MSR0000_0439
 _ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2071
 _ccd1_lthree0_inst7_n8_aliasMSRLEGACY; MSR0000_041D
 _ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2081

_ccd1_lthree0_inst8_n9_aliasMSRLEGACY; MSR0000_0421
_ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2091
_ccd1_lthree0_inst9_n10_aliasMSRLEGACY; MSR0000_0425
_ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A1
_ccd2_lthree0_inst10_n19_aliasMSRLEGACY; MSR0000_0429
_ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B1
_ccd2_lthree0_inst11_n20_aliasMSRLEGACY; MSR0000_042D
_ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C1
_ccd2_lthree0_inst12_n21_aliasMSRLEGACY; MSR0000_0431
_ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D1
_ccd2_lthree0_inst13_n22_aliasMSRLEGACY; MSR0000_0435
_ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E1
_ccd2_lthree0_inst14_n23_aliasMSRLEGACY; MSR0000_0439
_ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2071
_ccd2_lthree0_inst7_n16_aliasMSRLEGACY; MSR0000_041D
_ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2081
_ccd2_lthree0_inst8_n17_aliasMSRLEGACY; MSR0000_0421
_ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2091
_ccd2_lthree0_inst9_n18_aliasMSRLEGACY; MSR0000_0425
_ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A1
_ccd3_lthree0_inst10_n27_aliasMSRLEGACY; MSR0000_0429
_ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B1
_ccd3_lthree0_inst11_n28_aliasMSRLEGACY; MSR0000_042D
_ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C1
_ccd3_lthree0_inst12_n29_aliasMSRLEGACY; MSR0000_0431
_ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D1
_ccd3_lthree0_inst13_n30_aliasMSRLEGACY; MSR0000_0435
_ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E1
_ccd3_lthree0_inst14_n31_aliasMSRLEGACY; MSR0000_0439
_ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2071
_ccd3_lthree0_inst7_n24_aliasMSRLEGACY; MSR0000_041D
_ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2081
_ccd3_lthree0_inst8_n25_aliasMSRLEGACY; MSR0000_0421
_ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2091
_ccd3_lthree0_inst9_n26_aliasMSRLEGACY; MSR0000_0425
_ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A1
_ccd4_lthree0_inst10_n35_aliasMSRLEGACY; MSR0000_0429
_ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B1
_ccd4_lthree0_inst11_n36_aliasMSRLEGACY; MSR0000_042D
_ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C1
_ccd4_lthree0_inst12_n37_aliasMSRLEGACY; MSR0000_0431
_ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D1
_ccd4_lthree0_inst13_n38_aliasMSRLEGACY; MSR0000_0435
_ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E1
_ccd4_lthree0_inst14_n39_aliasMSRLEGACY; MSR0000_0439
_ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2071
_ccd4_lthree0_inst7_n32_aliasMSRLEGACY; MSR0000_041D
_ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2081
_ccd4_lthree0_inst8_n33_aliasMSRLEGACY; MSR0000_0421
_ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2091
_ccd4_lthree0_inst9_n34_aliasMSRLEGACY; MSR0000_0425
_ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A1
_ccd5_lthree0_inst10_n43_aliasMSRLEGACY; MSR0000_0429
_ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B1
_ccd5_lthree0_inst11_n44_aliasMSRLEGACY; MSR0000_042D
_ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C1
_ccd5_lthree0_inst12_n45_aliasMSRLEGACY; MSR0000_0431
_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D1
_ccd5_lthree0_inst13_n46_aliasMSRLEGACY; MSR0000_0435
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E1
_ccd5_lthree0_inst14_n47_aliasMSRLEGACY; MSR0000_0439
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2071
_ccd5_lthree0_inst7_n40_aliasMSRLEGACY; MSR0000_041D
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2081
_ccd5_lthree0_inst8_n41_aliasMSRLEGACY; MSR0000_0421

_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2091
_ccd5_lthree0_inst9_n42_aliasMSRLEGACY; MSR0000_0425
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A1
_ccd6_lthree0_inst10_n51_aliasMSRLEGACY; MSR0000_0429
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B1
_ccd6_lthree0_inst11_n52_aliasMSRLEGACY; MSR0000_042D
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C1
_ccd6_lthree0_inst12_n53_aliasMSRLEGACY; MSR0000_0431
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D1
_ccd6_lthree0_inst13_n54_aliasMSRLEGACY; MSR0000_0435
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E1
_ccd6_lthree0_inst14_n55_aliasMSRLEGACY; MSR0000_0439
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2071
_ccd6_lthree0_inst7_n48_aliasMSRLEGACY; MSR0000_041D
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2081
_ccd6_lthree0_inst8_n49_aliasMSRLEGACY; MSR0000_0421
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2091
_ccd6_lthree0_inst9_n50_aliasMSRLEGACY; MSR0000_0425
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A1
_ccd7_lthree0_inst10_n59_aliasMSRLEGACY; MSR0000_0429
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B1
_ccd7_lthree0_inst11_n60_aliasMSRLEGACY; MSR0000_042D
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C1
_ccd7_lthree0_inst12_n61_aliasMSRLEGACY; MSR0000_0431
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D1
_ccd7_lthree0_inst13_n62_aliasMSRLEGACY; MSR0000_0435
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E1
_ccd7_lthree0_inst14_n63_aliasMSRLEGACY; MSR0000_0439
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2071
_ccd7_lthree0_inst7_n56_aliasMSRLEGACY; MSR0000_041D
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2081
_ccd7_lthree0_inst8_n57_aliasMSRLEGACY; MSR0000_0421
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2091
_ccd7_lthree0_inst9_n58_aliasMSRLEGACY; MSR0000_0425
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A1
_ccd8_lthree0_inst10_n67_aliasMSRLEGACY; MSR0000_0429
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B1
_ccd8_lthree0_inst11_n68_aliasMSRLEGACY; MSR0000_042D
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C1
_ccd8_lthree0_inst12_n69_aliasMSRLEGACY; MSR0000_0431
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D1
_ccd8_lthree0_inst13_n70_aliasMSRLEGACY; MSR0000_0435
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E1
_ccd8_lthree0_inst14_n71_aliasMSRLEGACY; MSR0000_0439
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2071
_ccd8_lthree0_inst7_n64_aliasMSRLEGACY; MSR0000_041D
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2081
_ccd8_lthree0_inst8_n65_aliasMSRLEGACY; MSR0000_0421
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2091
_ccd8_lthree0_inst9_n66_aliasMSRLEGACY; MSR0000_0425
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A1
_ccd9_lthree0_inst10_n75_aliasMSRLEGACY; MSR0000_0429
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B1
_ccd9_lthree0_inst11_n76_aliasMSRLEGACY; MSR0000_042D
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C1
_ccd9_lthree0_inst12_n77_aliasMSRLEGACY; MSR0000_0431
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D1
_ccd9_lthree0_inst13_n78_aliasMSRLEGACY; MSR0000_0435
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E1
_ccd9_lthree0_inst14_n79_aliasMSRLEGACY; MSR0000_0439
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2071
_ccd9_lthree0_inst7_n72_aliasMSRLEGACY; MSR0000_041D
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2081
_ccd9_lthree0_inst8_n73_aliasMSRLEGACY; MSR0000_0421
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2091

_ccd9_lthree0_inst9_n74_aliasMSRLEGACY; MSR0000_0425	
Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::L3::MCA_CTL_L3. This bit is a copy of bit in MCA::L3::MCA_CTL_L3 for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::L3::MCA_MISC0_L3. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::L3::MCA_ADDR_L3 contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::L3::MCA_STATUS_L3[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::L3::MCA_SYND_L3. If MCA::L3::MCA_SYND_L3[ErrorPriority] is the same as the priority of the error in MCA::L3::MCA_STATUS_L3, then the information in MCA::L3::MCA_SYND_L3 is associated with the error in MCA::L3::MCA_STATUS_L3. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::L3::MCA_STATUS_L3[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::L3::MCA_ADDR_L3[ErrorAddr]. A value of 0 indicates that MCA::L3::MCA_ADDR_L3[55:0] contains a valid byte address. A value of 6 indicates that MCA::L3::MCA_ADDR_L3[55:6] contains a valid cache line address and that MCA::L3::MCA_ADDR_L3[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::L3::MCA_ADDR_L3[55:12] contain a valid 4KB memory page and that MCA::L3::MCA_ADDR_L3[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::L3::MCA_CTL_L3 enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 76: MCA_STATUS_L3

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
ShadowTag	0x0	0/1	0/1	0/1	0	0	1
MultiHitShadowTag	0x1	1	1	1	0	0	1
Tag	0x2	0/1	0/1	0/1	0	0	1
MultiHitTag	0x3	1	1	1	0	0	1
DataArray	0x4	0/1	0/1	0/1	0/1	0	1
SdpParity	0x5	0/1	0/1	0/1	0/1	0	0
XiVictimQueue	0x6	1	1	1	0	0	1
Hwa	0x7	1	1	1	0	0	1
XiWcbParity Poison	0x8	0	0	0	1	0	1

MSR0000_041E...MSRC000_20E2 [L3 Machine Check Address] (MCA::L3::MCA_ADDR_L3)

Reset: Cold,0000_0000_0000_0000h.

MCA::L3::MCA_ADDR_L3 stores an address and other information associated with the error in MCA::L3::MCA_STATUS_L3. The register is only meaningful if MCA::L3::MCA_STATUS_L3[Val]=1 and MCA::L3::MCA_STATUS_L3[AddrV]=1.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A2
 _ccd0_lthree0_inst10_n3_aliasMSRLEGACY; MSR0000_042A
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B2
 _ccd0_lthree0_inst11_n4_aliasMSRLEGACY; MSR0000_042E
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C2
 _ccd0_lthree0_inst12_n5_aliasMSRLEGACY; MSR0000_0432
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D2
 _ccd0_lthree0_inst13_n6_aliasMSRLEGACY; MSR0000_0436
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E2
 _ccd0_lthree0_inst14_n7_aliasMSRLEGACY; MSR0000_043A
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2072
 _ccd0_lthree0_inst7_n0_aliasMSRLEGACY; MSR0000_041E
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2082
 _ccd0_lthree0_inst8_n1_aliasMSRLEGACY; MSR0000_0422
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2092
 _ccd0_lthree0_inst9_n2_aliasMSRLEGACY; MSR0000_0426
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A2
 _ccd10_lthree0_inst10_n83_aliasMSRLEGACY; MSR0000_042A
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B2
 _ccd10_lthree0_inst11_n84_aliasMSRLEGACY; MSR0000_042E
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C2
 _ccd10_lthree0_inst12_n85_aliasMSRLEGACY; MSR0000_0432
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D2
 _ccd10_lthree0_inst13_n86_aliasMSRLEGACY; MSR0000_0436
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E2
 _ccd10_lthree0_inst14_n87_aliasMSRLEGACY; MSR0000_043A
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2072
 _ccd10_lthree0_inst7_n80_aliasMSRLEGACY; MSR0000_041E
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2082
 _ccd10_lthree0_inst8_n81_aliasMSRLEGACY; MSR0000_0422
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2092
 _ccd10_lthree0_inst9_n82_aliasMSRLEGACY; MSR0000_0426
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A2
 _ccd11_lthree0_inst10_n91_aliasMSRLEGACY; MSR0000_042A
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B2
 _ccd11_lthree0_inst11_n92_aliasMSRLEGACY; MSR0000_042E
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C2
 _ccd11_lthree0_inst12_n93_aliasMSRLEGACY; MSR0000_0432
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D2
 _ccd11_lthree0_inst13_n94_aliasMSRLEGACY; MSR0000_0436
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E2
 _ccd11_lthree0_inst14_n95_aliasMSRLEGACY; MSR0000_043A
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2072
 _ccd11_lthree0_inst7_n88_aliasMSRLEGACY; MSR0000_041E
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2082
 _ccd11_lthree0_inst8_n89_aliasMSRLEGACY; MSR0000_0422
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2092
 _ccd11_lthree0_inst9_n90_aliasMSRLEGACY; MSR0000_0426
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A2
 _ccd1_lthree0_inst10_n11_aliasMSRLEGACY; MSR0000_042A
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B2
 _ccd1_lthree0_inst11_n12_aliasMSRLEGACY; MSR0000_042E
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C2
 _ccd1_lthree0_inst12_n13_aliasMSRLEGACY; MSR0000_0432
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D2
 _ccd1_lthree0_inst13_n14_aliasMSRLEGACY; MSR0000_0436
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E2
 _ccd1_lthree0_inst14_n15_aliasMSRLEGACY; MSR0000_043A

_ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2072
_ccd1_lthree0_inst7_n8_aliasMSRLEGACY; MSR0000_041E
_ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2082
_ccd1_lthree0_inst8_n9_aliasMSRLEGACY; MSR0000_0422
_ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2092
_ccd1_lthree0_inst9_n10_aliasMSRLEGACY; MSR0000_0426
_ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A2
_ccd2_lthree0_inst10_n19_aliasMSRLEGACY; MSR0000_042A
_ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B2
_ccd2_lthree0_inst11_n20_aliasMSRLEGACY; MSR0000_042E
_ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C2
_ccd2_lthree0_inst12_n21_aliasMSRLEGACY; MSR0000_0432
_ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D2
_ccd2_lthree0_inst13_n22_aliasMSRLEGACY; MSR0000_0436
_ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E2
_ccd2_lthree0_inst14_n23_aliasMSRLEGACY; MSR0000_043A
_ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2072
_ccd2_lthree0_inst7_n16_aliasMSRLEGACY; MSR0000_041E
_ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2082
_ccd2_lthree0_inst8_n17_aliasMSRLEGACY; MSR0000_0422
_ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2092
_ccd2_lthree0_inst9_n18_aliasMSRLEGACY; MSR0000_0426
_ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A2
_ccd3_lthree0_inst10_n27_aliasMSRLEGACY; MSR0000_042A
_ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B2
_ccd3_lthree0_inst11_n28_aliasMSRLEGACY; MSR0000_042E
_ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C2
_ccd3_lthree0_inst12_n29_aliasMSRLEGACY; MSR0000_0432
_ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D2
_ccd3_lthree0_inst13_n30_aliasMSRLEGACY; MSR0000_0436
_ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E2
_ccd3_lthree0_inst14_n31_aliasMSRLEGACY; MSR0000_043A
_ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2072
_ccd3_lthree0_inst7_n24_aliasMSRLEGACY; MSR0000_041E
_ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2082
_ccd3_lthree0_inst8_n25_aliasMSRLEGACY; MSR0000_0422
_ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2092
_ccd3_lthree0_inst9_n26_aliasMSRLEGACY; MSR0000_0426
_ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A2
_ccd4_lthree0_inst10_n35_aliasMSRLEGACY; MSR0000_042A
_ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B2
_ccd4_lthree0_inst11_n36_aliasMSRLEGACY; MSR0000_042E
_ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C2
_ccd4_lthree0_inst12_n37_aliasMSRLEGACY; MSR0000_0432
_ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D2
_ccd4_lthree0_inst13_n38_aliasMSRLEGACY; MSR0000_0436
_ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E2
_ccd4_lthree0_inst14_n39_aliasMSRLEGACY; MSR0000_043A
_ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2072
_ccd4_lthree0_inst7_n32_aliasMSRLEGACY; MSR0000_041E
_ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2082
_ccd4_lthree0_inst8_n33_aliasMSRLEGACY; MSR0000_0422
_ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2092
_ccd4_lthree0_inst9_n34_aliasMSRLEGACY; MSR0000_0426
_ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A2
_ccd5_lthree0_inst10_n43_aliasMSRLEGACY; MSR0000_042A
_ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B2
_ccd5_lthree0_inst11_n44_aliasMSRLEGACY; MSR0000_042E
_ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C2
_ccd5_lthree0_inst12_n45_aliasMSRLEGACY; MSR0000_0432
_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D2
_ccd5_lthree0_inst13_n46_aliasMSRLEGACY; MSR0000_0436
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E2
_ccd5_lthree0_inst14_n47_aliasMSRLEGACY; MSR0000_043A
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2072

_ccd5_lthree0_inst7_n40_aliasMSRLEGACY; MSR0000_041E
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2082
_ccd5_lthree0_inst8_n41_aliasMSRLEGACY; MSR0000_0422
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2092
_ccd5_lthree0_inst9_n42_aliasMSRLEGACY; MSR0000_0426
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A2
_ccd6_lthree0_inst10_n51_aliasMSRLEGACY; MSR0000_042A
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B2
_ccd6_lthree0_inst11_n52_aliasMSRLEGACY; MSR0000_042E
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C2
_ccd6_lthree0_inst12_n53_aliasMSRLEGACY; MSR0000_0432
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D2
_ccd6_lthree0_inst13_n54_aliasMSRLEGACY; MSR0000_0436
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E2
_ccd6_lthree0_inst14_n55_aliasMSRLEGACY; MSR0000_043A
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2072
_ccd6_lthree0_inst7_n48_aliasMSRLEGACY; MSR0000_041E
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2082
_ccd6_lthree0_inst8_n49_aliasMSRLEGACY; MSR0000_0422
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2092
_ccd6_lthree0_inst9_n50_aliasMSRLEGACY; MSR0000_0426
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A2
_ccd7_lthree0_inst10_n59_aliasMSRLEGACY; MSR0000_042A
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B2
_ccd7_lthree0_inst11_n60_aliasMSRLEGACY; MSR0000_042E
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C2
_ccd7_lthree0_inst12_n61_aliasMSRLEGACY; MSR0000_0432
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D2
_ccd7_lthree0_inst13_n62_aliasMSRLEGACY; MSR0000_0436
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E2
_ccd7_lthree0_inst14_n63_aliasMSRLEGACY; MSR0000_043A
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2072
_ccd7_lthree0_inst7_n56_aliasMSRLEGACY; MSR0000_041E
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2082
_ccd7_lthree0_inst8_n57_aliasMSRLEGACY; MSR0000_0422
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2092
_ccd7_lthree0_inst9_n58_aliasMSRLEGACY; MSR0000_0426
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A2
_ccd8_lthree0_inst10_n67_aliasMSRLEGACY; MSR0000_042A
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B2
_ccd8_lthree0_inst11_n68_aliasMSRLEGACY; MSR0000_042E
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C2
_ccd8_lthree0_inst12_n69_aliasMSRLEGACY; MSR0000_0432
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D2
_ccd8_lthree0_inst13_n70_aliasMSRLEGACY; MSR0000_0436
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E2
_ccd8_lthree0_inst14_n71_aliasMSRLEGACY; MSR0000_043A
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2072
_ccd8_lthree0_inst7_n64_aliasMSRLEGACY; MSR0000_041E
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2082
_ccd8_lthree0_inst8_n65_aliasMSRLEGACY; MSR0000_0422
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2092
_ccd8_lthree0_inst9_n66_aliasMSRLEGACY; MSR0000_0426
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A2
_ccd9_lthree0_inst10_n75_aliasMSRLEGACY; MSR0000_042A
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B2
_ccd9_lthree0_inst11_n76_aliasMSRLEGACY; MSR0000_042E
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C2
_ccd9_lthree0_inst12_n77_aliasMSRLEGACY; MSR0000_0432
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D2
_ccd9_lthree0_inst13_n78_aliasMSRLEGACY; MSR0000_0436
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E2
_ccd9_lthree0_inst14_n79_aliasMSRLEGACY; MSR0000_043A
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2072
_ccd9_lthree0_inst7_n72_aliasMSRLEGACY; MSR0000_041E

_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2082	
_ccd9_lthree0_inst8_n73_aliasMSRLEGACY; MSR0000_0422	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2092	
_ccd9_lthree0_inst9_n74_aliasMSRLEGACY; MSR0000_0426	
Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::L3::MCA_STATUS_L3. For physical addresses, the most significant bit is given by Core::X86::CpuId::LongModeInfo[PhysAddrSize].

Table 77: MCA_ADDR_L3

Error Type	Bits	Description
ShadowTag	[55:16] [15:0]	Reserved 16'b{7'b{Index}, 3'b{Slice}, 6'b{0}}
MultiHitShadowTag	[55:16] [15:0]	Reserved 16'b{7'b{Index}, 3'b{Slice}, 6'b{0}}
Tag	[55:21] [20:0]	Reserved 21'b{1'b{Odd/Even}, 7'b{Index}, 4'b{Bank[3:0]}, 3'b{Slice}, 6'b{0}}
MultiHitTag	[55:21] [20:0]	Reserved 21'b{1'b{Odd/Even}, 7'b{Index}, 4'b{Bank[3:0]}, 3'b{Slice}, 6'b{0}}
DataArray	[55:52] [51:0]	Reserved Physical Address PA[21]: Odd/Even, PA[20:12]: Index, PA[11:9]: Bank, PA[8:6]: Slice. When index hashing is enabled and the ErrOperationType={GENERIC EVIT SNOOP}, PA[15:9] must be XOR'd with PA[27:21]) to derived the actual physical index and bank within the L3M array.
SdpParity	[55:52] [51:0]	Reserved Physical Address
XiVictimQueue	[55:52] [51:0]	Reserved Physical Address
Hwa	[55:34] [33:0]	Reserved Reserved
XiWcbParityPoison	[55:52] [51:0]	Reserved Physical Address

MSR0000_041F...MSRC000_20E3 [L3 Machine Check Miscellaneous 0] (MCA::L3::MCA_MISC0_L3)

Log miscellaneous information associated with errors.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A3
 _ccd0_lthree0_inst10_n3_aliasMSRLEGACY; MSR0000_042B
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B3
 _ccd0_lthree0_inst11_n4_aliasMSRLEGACY; MSR0000_042F
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C3
 _ccd0_lthree0_inst12_n5_aliasMSRLEGACY; MSR0000_0433
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D3
 _ccd0_lthree0_inst13_n6_aliasMSRLEGACY; MSR0000_0437
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E3
 _ccd0_lthree0_inst14_n7_aliasMSRLEGACY; MSR0000_043B
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2073
 _ccd0_lthree0_inst7_n0_aliasMSRLEGACY; MSR0000_041F
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2083
 _ccd0_lthree0_inst8_n1_aliasMSRLEGACY; MSR0000_0423
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2093
 _ccd0_lthree0_inst9_n2_aliasMSRLEGACY; MSR0000_0427
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A3
 _ccd10_lthree0_inst10_n83_aliasMSRLEGACY; MSR0000_042B
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B3
 _ccd10_lthree0_inst11_n84_aliasMSRLEGACY; MSR0000_042F
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C3
 _ccd10_lthree0_inst12_n85_aliasMSRLEGACY; MSR0000_0433
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D3
 _ccd10_lthree0_inst13_n86_aliasMSRLEGACY; MSR0000_0437
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E3
 _ccd10_lthree0_inst14_n87_aliasMSRLEGACY; MSR0000_043B
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2073
 _ccd10_lthree0_inst7_n80_aliasMSRLEGACY; MSR0000_041F
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2083
 _ccd10_lthree0_inst8_n81_aliasMSRLEGACY; MSR0000_0423
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2093
 _ccd10_lthree0_inst9_n82_aliasMSRLEGACY; MSR0000_0427
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A3
 _ccd11_lthree0_inst10_n91_aliasMSRLEGACY; MSR0000_042B
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B3
 _ccd11_lthree0_inst11_n92_aliasMSRLEGACY; MSR0000_042F
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C3
 _ccd11_lthree0_inst12_n93_aliasMSRLEGACY; MSR0000_0433
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D3
 _ccd11_lthree0_inst13_n94_aliasMSRLEGACY; MSR0000_0437
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E3
 _ccd11_lthree0_inst14_n95_aliasMSRLEGACY; MSR0000_043B
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2073
 _ccd11_lthree0_inst7_n88_aliasMSRLEGACY; MSR0000_041F
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2083
 _ccd11_lthree0_inst8_n89_aliasMSRLEGACY; MSR0000_0423
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2093
 _ccd11_lthree0_inst9_n90_aliasMSRLEGACY; MSR0000_0427
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A3
 _ccd1_lthree0_inst10_n11_aliasMSRLEGACY; MSR0000_042B
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B3
 _ccd1_lthree0_inst11_n12_aliasMSRLEGACY; MSR0000_042F
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C3
 _ccd1_lthree0_inst12_n13_aliasMSRLEGACY; MSR0000_0433
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D3
 _ccd1_lthree0_inst13_n14_aliasMSRLEGACY; MSR0000_0437
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E3
 _ccd1_lthree0_inst14_n15_aliasMSRLEGACY; MSR0000_043B
 _ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2073
 _ccd1_lthree0_inst7_n8_aliasMSRLEGACY; MSR0000_041F
 _ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2083
 _ccd1_lthree0_inst8_n9_aliasMSRLEGACY; MSR0000_0423

_ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2093
_ccd1_lthree0_inst9_n10_aliasMSRLEGACY; MSR0000_0427
_ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A3
_ccd2_lthree0_inst10_n19_aliasMSRLEGACY; MSR0000_042B
_ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B3
_ccd2_lthree0_inst11_n20_aliasMSRLEGACY; MSR0000_042F
_ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C3
_ccd2_lthree0_inst12_n21_aliasMSRLEGACY; MSR0000_0433
_ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D3
_ccd2_lthree0_inst13_n22_aliasMSRLEGACY; MSR0000_0437
_ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E3
_ccd2_lthree0_inst14_n23_aliasMSRLEGACY; MSR0000_043B
_ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2073
_ccd2_lthree0_inst7_n16_aliasMSRLEGACY; MSR0000_041F
_ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2083
_ccd2_lthree0_inst8_n17_aliasMSRLEGACY; MSR0000_0423
_ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2093
_ccd2_lthree0_inst9_n18_aliasMSRLEGACY; MSR0000_0427
_ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A3
_ccd3_lthree0_inst10_n27_aliasMSRLEGACY; MSR0000_042B
_ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B3
_ccd3_lthree0_inst11_n28_aliasMSRLEGACY; MSR0000_042F
_ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C3
_ccd3_lthree0_inst12_n29_aliasMSRLEGACY; MSR0000_0433
_ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D3
_ccd3_lthree0_inst13_n30_aliasMSRLEGACY; MSR0000_0437
_ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E3
_ccd3_lthree0_inst14_n31_aliasMSRLEGACY; MSR0000_043B
_ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2073
_ccd3_lthree0_inst7_n24_aliasMSRLEGACY; MSR0000_041F
_ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2083
_ccd3_lthree0_inst8_n25_aliasMSRLEGACY; MSR0000_0423
_ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2093
_ccd3_lthree0_inst9_n26_aliasMSRLEGACY; MSR0000_0427
_ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A3
_ccd4_lthree0_inst10_n35_aliasMSRLEGACY; MSR0000_042B
_ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B3
_ccd4_lthree0_inst11_n36_aliasMSRLEGACY; MSR0000_042F
_ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C3
_ccd4_lthree0_inst12_n37_aliasMSRLEGACY; MSR0000_0433
_ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D3
_ccd4_lthree0_inst13_n38_aliasMSRLEGACY; MSR0000_0437
_ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E3
_ccd4_lthree0_inst14_n39_aliasMSRLEGACY; MSR0000_043B
_ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2073
_ccd4_lthree0_inst7_n32_aliasMSRLEGACY; MSR0000_041F
_ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2083
_ccd4_lthree0_inst8_n33_aliasMSRLEGACY; MSR0000_0423
_ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2093
_ccd4_lthree0_inst9_n34_aliasMSRLEGACY; MSR0000_0427
_ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A3
_ccd5_lthree0_inst10_n43_aliasMSRLEGACY; MSR0000_042B
_ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B3
_ccd5_lthree0_inst11_n44_aliasMSRLEGACY; MSR0000_042F
_ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C3
_ccd5_lthree0_inst12_n45_aliasMSRLEGACY; MSR0000_0433
_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D3
_ccd5_lthree0_inst13_n46_aliasMSRLEGACY; MSR0000_0437
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E3
_ccd5_lthree0_inst14_n47_aliasMSRLEGACY; MSR0000_043B
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2073
_ccd5_lthree0_inst7_n40_aliasMSRLEGACY; MSR0000_041F
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2083
_ccd5_lthree0_inst8_n41_aliasMSRLEGACY; MSR0000_0423
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2093

_ccd5_lthree0_inst9_n42_aliasMSRLEGACY; MSR0000_0427
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A3
_ccd6_lthree0_inst10_n51_aliasMSRLEGACY; MSR0000_042B
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B3
_ccd6_lthree0_inst11_n52_aliasMSRLEGACY; MSR0000_042F
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C3
_ccd6_lthree0_inst12_n53_aliasMSRLEGACY; MSR0000_0433
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D3
_ccd6_lthree0_inst13_n54_aliasMSRLEGACY; MSR0000_0437
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E3
_ccd6_lthree0_inst14_n55_aliasMSRLEGACY; MSR0000_043B
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2073
_ccd6_lthree0_inst7_n48_aliasMSRLEGACY; MSR0000_041F
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2083
_ccd6_lthree0_inst8_n49_aliasMSRLEGACY; MSR0000_0423
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2093
_ccd6_lthree0_inst9_n50_aliasMSRLEGACY; MSR0000_0427
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A3
_ccd7_lthree0_inst10_n59_aliasMSRLEGACY; MSR0000_042B
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B3
_ccd7_lthree0_inst11_n60_aliasMSRLEGACY; MSR0000_042F
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C3
_ccd7_lthree0_inst12_n61_aliasMSRLEGACY; MSR0000_0433
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D3
_ccd7_lthree0_inst13_n62_aliasMSRLEGACY; MSR0000_0437
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E3
_ccd7_lthree0_inst14_n63_aliasMSRLEGACY; MSR0000_043B
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2073
_ccd7_lthree0_inst7_n56_aliasMSRLEGACY; MSR0000_041F
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2083
_ccd7_lthree0_inst8_n57_aliasMSRLEGACY; MSR0000_0423
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2093
_ccd7_lthree0_inst9_n58_aliasMSRLEGACY; MSR0000_0427
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A3
_ccd8_lthree0_inst10_n67_aliasMSRLEGACY; MSR0000_042B
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B3
_ccd8_lthree0_inst11_n68_aliasMSRLEGACY; MSR0000_042F
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C3
_ccd8_lthree0_inst12_n69_aliasMSRLEGACY; MSR0000_0433
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D3
_ccd8_lthree0_inst13_n70_aliasMSRLEGACY; MSR0000_0437
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E3
_ccd8_lthree0_inst14_n71_aliasMSRLEGACY; MSR0000_043B
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2073
_ccd8_lthree0_inst7_n64_aliasMSRLEGACY; MSR0000_041F
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2083
_ccd8_lthree0_inst8_n65_aliasMSRLEGACY; MSR0000_0423
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2093
_ccd8_lthree0_inst9_n66_aliasMSRLEGACY; MSR0000_0427
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A3
_ccd9_lthree0_inst10_n75_aliasMSRLEGACY; MSR0000_042B
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B3
_ccd9_lthree0_inst11_n76_aliasMSRLEGACY; MSR0000_042F
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C3
_ccd9_lthree0_inst12_n77_aliasMSRLEGACY; MSR0000_0433
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D3
_ccd9_lthree0_inst13_n78_aliasMSRLEGACY; MSR0000_0437
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E3
_ccd9_lthree0_inst14_n79_aliasMSRLEGACY; MSR0000_043B
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2073
_ccd9_lthree0_inst7_n72_aliasMSRLEGACY; MSR0000_041F
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2083
_ccd9_lthree0_inst8_n73_aliasMSRLEGACY; MSR0000_0423
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2093
_ccd9_lthree0_inst9_n74_aliasMSRLEGACY; MSR0000_0427

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::L3::MCA_MISC0_L3[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::L3::MCA_MISC0_L3[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::L3::MCA_MISC0_L3[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::L3::MCA_MISC0_L3[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::L3::MCA_MISC0_L3[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::L3::MCA_MISC0_L3[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_20[7...E]4 [L3 Machine Check Configuration] (MCA::L3::MCA_CONFIG_L3)

Reset: 0000_0002_0000_0127h.

Controls configuration of the associated machine check bank.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A4

_ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B4

_ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C4

_ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D4

_ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E4

_ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2074

_ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2084

_ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2094

_ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A4

_ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B4

_ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C4

_ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D4

_ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E4

_ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2074

_ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2084

_ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2094

_ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A4

_ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B4

_ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C4

_ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D4

_ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E4

_ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2074

_ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2084

_ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2094

_ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A4

_ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B4

_ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C4

_ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D4

_ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E4

_ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2074

_ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2084

_ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2094

_ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A4

_ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B4

_ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C4

_ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D4

_ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E4

_ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2074

_ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2084

_ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2094

_ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A4

_ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B4

_ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C4

_ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D4

_ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E4

_ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2074

_ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2084

_ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2094

_ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A4

_ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B4

_ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C4

_ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D4

_ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E4

_ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2074

_ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2084

_ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2094

_ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A4

_ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B4

_ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C4

_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D4

_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E4

_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2074	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2084	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2094	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A4	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B4	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C4	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D4	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E4	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2074	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2084	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2094	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A4	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B4	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C4	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D4	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E4	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2074	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2084	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2094	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A4	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B4	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C4	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D4	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E4	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2074	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2084	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2094	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A4	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B4	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C4	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D4	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E4	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2074	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2084	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2094	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat. Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::L3::MCA_STATUS_L3 and MCA::L3::MCA_ADDR_L3 in addition to MCA::L3::MCA_DESTAT_L3 and MCA::L3::MCA_DEADDR_L3. 0=Only log deferred errors in MCA::L3::MCA_DESTAT_L3 and MCA::L3::MCA_DEADDR_L3. This bit does not affect logging of deferred errors in MCA::L3::MCA_SYND_L3, MCA::L3::MCA_MISC0_L3.
33	TransparentErrorLoggingEnable. Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::L3::MCA_STATUS_L3, MCA::L3::MCA_ADDR_L3, MCA::L3::MCA_MISC0_L3, and MCA::L3::MCA_SYND_L3 in addition to MCA::L3::MCA_TRANSYND_L3 and MCA::L3::MCA_TRANSADDR_L3. 0=Only log transparent errors in MCA::L3::MCA_TRANSYND_L3 and MCA::L3::MCA_TRANSADDR_L3.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msrr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::L3::MCA_CONFIG_L3[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.

7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::L3::MCA_CONFIG_L3[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::L3::MCA_CONFIG_L3[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::L3::MCA_CONFIG_L3[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::L3::MCA_DESTAT_L3 and MCA::L3::MCA_DEADDR_L3 are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::L3::MCA_CONFIG_L3[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::L3::MCA_MISC0_L3[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::L3::MCA_STATUS_L3[TCC] is present.

MSRC000_20[7...E]5 [L3 IP Identification] (MCA::L3::MCA_IPID_L3)

Reset: 0007_00B0_0000_0000h.

The MCA::L3::MCA_IPID_L3 register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A5
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B5
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C5
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D5
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E5
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2075
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2085
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2095
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A5
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B5
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C5
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D5
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E5
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2075
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2085
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2095
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A5
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B5
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C5
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D5
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E5
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2075
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2085
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2095
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A5
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B5
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C5
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D5
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E5
 _ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2075
 _ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2085
 _ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2095
 _ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A5
 _ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B5
 _ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C5
 _ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D5
 _ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E5
 _ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2075
 _ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2085
 _ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2095
 _ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A5
 _ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B5
 _ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C5
 _ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D5
 _ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E5
 _ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2075
 _ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2085
 _ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2095
 _ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A5
 _ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B5
 _ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C5
 _ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D5
 _ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E5
 _ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2075
 _ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2085
 _ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2095
 _ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A5
 _ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B5
 _ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C5

_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D5	
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E5	
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2075	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2085	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2095	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A5	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B5	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C5	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D5	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E5	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2075	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2085	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2095	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A5	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B5	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C5	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D5	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E5	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2075	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2085	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2095	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A5	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B5	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C5	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D5	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E5	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2075	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2085	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2095	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A5	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B5	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C5	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D5	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E5	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2075	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2085	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2095	
Bits	Description
63:48	McaType . Read-only. Reset: 0007h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0B0h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
Init: _ccd0_lthree0_inst7_n0_aliasMSR: 2035_0000h	
Init: _ccd0_lthree0_inst8_n1_aliasMSR: 2035_0100h	
Init: _ccd0_lthree0_inst9_n2_aliasMSR: 2035_0200h	
Init: _ccd0_lthree0_inst10_n3_aliasMSR: 2035_0300h	
Init: _ccd0_lthree0_inst11_n4_aliasMSR: 2035_0400h	
Init: _ccd0_lthree0_inst12_n5_aliasMSR: 2035_0500h	
Init: _ccd0_lthree0_inst13_n6_aliasMSR: 2035_0600h	
Init: _ccd0_lthree0_inst14_n7_aliasMSR: 2035_0700h	
Init: _ccd1_lthree0_inst7_n8_aliasMSR: 20B5_0000h	
Init: _ccd1_lthree0_inst8_n9_aliasMSR: 20B5_0100h	
Init: _ccd1_lthree0_inst9_n10_aliasMSR: 20B5_0200h	
Init: _ccd1_lthree0_inst10_n11_aliasMSR: 20B5_0300h	
Init: _ccd1_lthree0_inst11_n12_aliasMSR: 20B5_0400h	
Init: _ccd1_lthree0_inst12_n13_aliasMSR: 20B5_0500h	
Init: _ccd1_lthree0_inst13_n14_aliasMSR: 20B5_0600h	
Init: _ccd1_lthree0_inst14_n15_aliasMSR: 20B5_0700h	
Init: _ccd2_lthree0_inst7_n16_aliasMSR: 2135_0000h	
Init: _ccd2_lthree0_inst8_n17_aliasMSR: 2135_0100h	
Init: _ccd2_lthree0_inst9_n18_aliasMSR: 2135_0200h	

Init: _ccd2_lthree0_inst10_n19_aliasMSR: 2135_0300h
Init: _ccd2_lthree0_inst11_n20_aliasMSR: 2135_0400h
Init: _ccd2_lthree0_inst12_n21_aliasMSR: 2135_0500h
Init: _ccd2_lthree0_inst13_n22_aliasMSR: 2135_0600h
Init: _ccd2_lthree0_inst14_n23_aliasMSR: 2135_0700h
Init: _ccd3_lthree0_inst7_n24_aliasMSR: 21B5_0000h
Init: _ccd3_lthree0_inst8_n25_aliasMSR: 21B5_0100h
Init: _ccd3_lthree0_inst9_n26_aliasMSR: 21B5_0200h
Init: _ccd3_lthree0_inst10_n27_aliasMSR: 21B5_0300h
Init: _ccd3_lthree0_inst11_n28_aliasMSR: 21B5_0400h
Init: _ccd3_lthree0_inst12_n29_aliasMSR: 21B5_0500h
Init: _ccd3_lthree0_inst13_n30_aliasMSR: 21B5_0600h
Init: _ccd3_lthree0_inst14_n31_aliasMSR: 21B5_0700h
Init: _ccd4_lthree0_inst7_n32_aliasMSR: 2235_0000h
Init: _ccd4_lthree0_inst8_n33_aliasMSR: 2235_0100h
Init: _ccd4_lthree0_inst9_n34_aliasMSR: 2235_0200h
Init: _ccd4_lthree0_inst10_n35_aliasMSR: 2235_0300h
Init: _ccd4_lthree0_inst11_n36_aliasMSR: 2235_0400h
Init: _ccd4_lthree0_inst12_n37_aliasMSR: 2235_0500h
Init: _ccd4_lthree0_inst13_n38_aliasMSR: 2235_0600h
Init: _ccd4_lthree0_inst14_n39_aliasMSR: 2235_0700h
Init: _ccd5_lthree0_inst7_n40_aliasMSR: 22B5_0000h
Init: _ccd5_lthree0_inst8_n41_aliasMSR: 22B5_0100h
Init: _ccd5_lthree0_inst9_n42_aliasMSR: 22B5_0200h
Init: _ccd5_lthree0_inst10_n43_aliasMSR: 22B5_0300h
Init: _ccd5_lthree0_inst11_n44_aliasMSR: 22B5_0400h
Init: _ccd5_lthree0_inst12_n45_aliasMSR: 22B5_0500h
Init: _ccd5_lthree0_inst13_n46_aliasMSR: 22B5_0600h
Init: _ccd5_lthree0_inst14_n47_aliasMSR: 22B5_0700h
Init: _ccd6_lthree0_inst7_n48_aliasMSR: 2335_0000h
Init: _ccd6_lthree0_inst8_n49_aliasMSR: 2335_0100h
Init: _ccd6_lthree0_inst9_n50_aliasMSR: 2335_0200h
Init: _ccd6_lthree0_inst10_n51_aliasMSR: 2335_0300h
Init: _ccd6_lthree0_inst11_n52_aliasMSR: 2335_0400h
Init: _ccd6_lthree0_inst12_n53_aliasMSR: 2335_0500h
Init: _ccd6_lthree0_inst13_n54_aliasMSR: 2335_0600h
Init: _ccd6_lthree0_inst14_n55_aliasMSR: 2335_0700h
Init: _ccd7_lthree0_inst7_n56_aliasMSR: 23B5_0000h
Init: _ccd7_lthree0_inst8_n57_aliasMSR: 23B5_0100h
Init: _ccd7_lthree0_inst9_n58_aliasMSR: 23B5_0200h
Init: _ccd7_lthree0_inst10_n59_aliasMSR: 23B5_0300h
Init: _ccd7_lthree0_inst11_n60_aliasMSR: 23B5_0400h
Init: _ccd7_lthree0_inst12_n61_aliasMSR: 23B5_0500h
Init: _ccd7_lthree0_inst13_n62_aliasMSR: 23B5_0600h
Init: _ccd7_lthree0_inst14_n63_aliasMSR: 23B5_0700h
Init: _ccd8_lthree0_inst7_n64_aliasMSR: 2435_0000h
Init: _ccd8_lthree0_inst8_n65_aliasMSR: 2435_0100h
Init: _ccd8_lthree0_inst9_n66_aliasMSR: 2435_0200h
Init: _ccd8_lthree0_inst10_n67_aliasMSR: 2435_0300h
Init: _ccd8_lthree0_inst11_n68_aliasMSR: 2435_0400h
Init: _ccd8_lthree0_inst12_n69_aliasMSR: 2435_0500h
Init: _ccd8_lthree0_inst13_n70_aliasMSR: 2435_0600h
Init: _ccd8_lthree0_inst14_n71_aliasMSR: 2435_0700h
Init: _ccd9_lthree0_inst7_n72_aliasMSR: 24B5_0000h
Init: _ccd9_lthree0_inst8_n73_aliasMSR: 24B5_0100h
Init: _ccd9_lthree0_inst9_n74_aliasMSR: 24B5_0200h
Init: _ccd9_lthree0_inst10_n75_aliasMSR: 24B5_0300h
Init: _ccd9_lthree0_inst11_n76_aliasMSR: 24B5_0400h
Init: _ccd9_lthree0_inst12_n77_aliasMSR: 24B5_0500h
Init: _ccd9_lthree0_inst13_n78_aliasMSR: 24B5_0600h
Init: _ccd9_lthree0_inst14_n79_aliasMSR: 24B5_0700h
Init: _ccd10_lthree0_inst7_n80_aliasMSR: 2535_0000h
Init: _ccd10_lthree0_inst8_n81_aliasMSR: 2535_0100h
Init: _ccd10_lthree0_inst9_n82_aliasMSR: 2535_0200h
Init: _ccd10_lthree0_inst10_n83_aliasMSR: 2535_0300h

Init: _ccd10_lthree0_inst11_n84_aliasMSR: 2535_0400h
Init: _ccd10_lthree0_inst12_n85_aliasMSR: 2535_0500h
Init: _ccd10_lthree0_inst13_n86_aliasMSR: 2535_0600h
Init: _ccd10_lthree0_inst14_n87_aliasMSR: 2535_0700h
Init: _ccd11_lthree0_inst7_n88_aliasMSR: 25B5_0000h
Init: _ccd11_lthree0_inst8_n89_aliasMSR: 25B5_0100h
Init: _ccd11_lthree0_inst9_n90_aliasMSR: 25B5_0200h
Init: _ccd11_lthree0_inst10_n91_aliasMSR: 25B5_0300h
Init: _ccd11_lthree0_inst11_n92_aliasMSR: 25B5_0400h
Init: _ccd11_lthree0_inst12_n93_aliasMSR: 25B5_0500h
Init: _ccd11_lthree0_inst13_n94_aliasMSR: 25B5_0600h
Init: _ccd11_lthree0_inst14_n95_aliasMSR: 25B5_0700h

MSRC000_20[7...E]6 [L3 Machine Check Syndrome] (MCA::L3::MCA_SYND_L3)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::L3::MCA_STATUS_L3 [Thread 0](#)

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A6
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B6
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C6
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D6
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E6
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2076
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2086
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2096
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A6
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B6
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C6
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D6
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E6
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2076
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2086
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2096
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A6
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B6
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C6
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D6
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E6
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2076
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2086
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2096
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A6
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B6
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C6
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D6
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E6
 _ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2076
 _ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2086
 _ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2096
 _ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A6
 _ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B6
 _ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C6
 _ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D6
 _ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E6
 _ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2076
 _ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2086
 _ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2096
 _ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A6
 _ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B6
 _ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C6
 _ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D6
 _ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E6
 _ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2076
 _ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2086
 _ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2096
 _ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A6
 _ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B6
 _ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C6
 _ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D6
 _ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E6
 _ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2076
 _ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2086
 _ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2096
 _ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A6
 _ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B6
 _ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C6
 _ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D6
 _ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E6

_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2076	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2086	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2096	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A6	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B6	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C6	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D6	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E6	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2076	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2086	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2096	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A6	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B6	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C6	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D6	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E6	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2076	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2086	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2096	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A6	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B6	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C6	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D6	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E6	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2076	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2086	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2096	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A6	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B6	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C6	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D6	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E6	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2076	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2086	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2096	
Bits	Description
63:51	Reserved.
50:32	Syndrome. Read-write, Volatile. Reset: Cold,0_0000h. Contains the syndrome, if any, associated with the error logged in MCA::L3::MCA_STATUS_L3. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::L3::MCA_SYND_L3[Length]. The Syndrome field is only valid when MCA::L3::MCA_SYND_L3[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold,0h. Encodes the priority of the error logged in MCA::L3::MCA_SYND_L3. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold,00h. Specifies the length in bits of the syndrome contained in MCA::L3::MCA_SYND_L3[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::L3::MCA_SYND_L3. For example, a syndrome length of 9 means that MCA::L3::MCA_SYND_L3[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold,0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 78 [MCA_SYND_L3].

Table 78: MCA_SYND_L3

Error Type	Bits	Description
ShadowTag	[17:16]	Reserved
	[15:8]	Pack
	[7:3]	Reserved
	[2:0]	Way

MultiHitShadowTag	[17:12] [11:8] [7:0]	Reserved Pack Reserved
Tag	[17:14] [13] [12] [11] [10:8] [7:4] [3:0]	4'b1100 When set to 1, indicates error is in X3D. X3DDie[1:0] identifies the X3D die. [13] ? X3DDie[1] : PA[20]. [13] ? X3DDie[0] : Bank[3] Bank[2:0] Reserved Way
MultiHitTag	[17:0]	Reserved
DataArray	[17:16] [15] [14] [13] [12] [11] [10:8] [7:4] [3:0]	2'b11 0 when error is in Ecc[9:0]/Data[127:0]. 1 when error is in Ecc[18:10]/Data[255:128]. 1'b0 <When set to 1, indicates error is in X3D. X3DDie[1:0] identifies the X3D die. [13] ? X3DDie[1] : Reserved [13] ? X3DDie[0] : Reserved Bank[2:0] Reserved Way
SdpParity	[17:0]	Reserved
XiVictimQueue	[17:0]	Reserved
Hwa	[17:0]	Reserved
XiWcbParityPoison	[17:0]	Reserved

MSRC000_20[7...E]8 [L3 Machine Check Deferred Error Status] (MCA::L3::MCA_DESTAT_L3)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A8
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B8
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C8
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D8
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E8
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2078
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2088
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2098
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A8
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B8
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C8
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D8
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E8
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2078
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2088
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2098
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A8
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B8
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C8
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D8
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E8
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2078
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2088
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2098
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A8
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B8
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C8
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D8
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E8
 _ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2078
 _ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2088
 _ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2098
 _ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A8
 _ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B8
 _ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C8
 _ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D8
 _ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E8
 _ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2078
 _ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2088
 _ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2098
 _ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A8
 _ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B8
 _ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C8
 _ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D8
 _ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E8
 _ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2078
 _ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2088
 _ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2098
 _ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A8
 _ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B8
 _ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C8
 _ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D8
 _ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E8
 _ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2078
 _ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2088
 _ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2098
 _ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A8
 _ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B8
 _ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C8
 _ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D8
 _ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E8

_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2078	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2088	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2098	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A8	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B8	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C8	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D8	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E8	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2078	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2088	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2098	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A8	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B8	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C8	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D8	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E8	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2078	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2088	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2098	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A8	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B8	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C8	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D8	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E8	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2078	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2088	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2098	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A8	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B8	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C8	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D8	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E8	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2078	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2088	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2098	
Bits	Description
63	Val. Read-write, Volatile. Reset: Cold, 0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold, 0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold, 0h.
58	AddrV. Read-write, Volatile. Reset: Cold, 0. 1=MCA::L3::MCA_DEADDR_L3 contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold, 0h.
53	SyndV. Read-write, Volatile. Reset: Cold, 0. 1=This error logged information in MCA::L3::MCA_SYND_L3. If MCA::L3::MCA_SYND_L3[ErrorPriority] is the same as the priority of the error in MCA::L3::MCA_STATUS_L3, then the information in MCA::L3::MCA_SYND_L3 is associated with the error in MCA::L3::MCA_DESTAT_L3.
52:45	RESERV2. Read-write. Reset: Cold, 00h.
44	Deferred. Read-write, Volatile. Reset: Cold, 0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold, 0000h.

29:24	AddrLsb. Read-write, Volatile. Reset: Cold, 00h. Specifies the least significant valid bit of the address contained in MCA::L3::MCA_ADDR_L3[ErrorAddr]. A value of 0 indicates that MCA::L3::MCA_ADDR_L3[55:0] contains a valid byte address. A value of 6 indicates that MCA::L3::MCA_ADDR_L3[55:6] contains a valid cache line address and that MCA::L3::MCA_ADDR_L3[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::L3::MCA_ADDR_L3[55:12] contain a valid 4KB memory page and that MCA::L3::MCA_ADDR_L3[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold, 0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold, 00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold, 0000h. Error code for this error.

MSRC000_20[7...E]9 [L3 Deferred Error Address] (MCA::L3::MCA_DEADDR_L3)

Reset: Cold,0000_0000_0000_0000h.

The MCA::L3::MCA_DEADDR_L3 register stores the address associated with the error in MCA::L3::MCA_DESTAT_L3. The register is only meaningful if MCA::L3::MCA_DESTAT_L3[Val]=1 and MCA::L3::MCA_DESTAT_L3[AddrV]=1. The lowest valid bit of the address is defined by MCA::L3::MCA_DESTAT_L3[AddrLsb].

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC000_20A9

_ccd0_lthree0_inst11_n4_aliasMSR; MSRC000_20B9

_ccd0_lthree0_inst12_n5_aliasMSR; MSRC000_20C9

_ccd0_lthree0_inst13_n6_aliasMSR; MSRC000_20D9

_ccd0_lthree0_inst14_n7_aliasMSR; MSRC000_20E9

_ccd0_lthree0_inst7_n0_aliasMSR; MSRC000_2079

_ccd0_lthree0_inst8_n1_aliasMSR; MSRC000_2089

_ccd0_lthree0_inst9_n2_aliasMSR; MSRC000_2099

_ccd10_lthree0_inst10_n83_aliasMSR; MSRC000_20A9

_ccd10_lthree0_inst11_n84_aliasMSR; MSRC000_20B9

_ccd10_lthree0_inst12_n85_aliasMSR; MSRC000_20C9

_ccd10_lthree0_inst13_n86_aliasMSR; MSRC000_20D9

_ccd10_lthree0_inst14_n87_aliasMSR; MSRC000_20E9

_ccd10_lthree0_inst7_n80_aliasMSR; MSRC000_2079

_ccd10_lthree0_inst8_n81_aliasMSR; MSRC000_2089

_ccd10_lthree0_inst9_n82_aliasMSR; MSRC000_2099

_ccd11_lthree0_inst10_n91_aliasMSR; MSRC000_20A9

_ccd11_lthree0_inst11_n92_aliasMSR; MSRC000_20B9

_ccd11_lthree0_inst12_n93_aliasMSR; MSRC000_20C9

_ccd11_lthree0_inst13_n94_aliasMSR; MSRC000_20D9

_ccd11_lthree0_inst14_n95_aliasMSR; MSRC000_20E9

_ccd11_lthree0_inst7_n88_aliasMSR; MSRC000_2079

_ccd11_lthree0_inst8_n89_aliasMSR; MSRC000_2089

_ccd11_lthree0_inst9_n90_aliasMSR; MSRC000_2099

_ccd1_lthree0_inst10_n11_aliasMSR; MSRC000_20A9

_ccd1_lthree0_inst11_n12_aliasMSR; MSRC000_20B9

_ccd1_lthree0_inst12_n13_aliasMSR; MSRC000_20C9

_ccd1_lthree0_inst13_n14_aliasMSR; MSRC000_20D9

_ccd1_lthree0_inst14_n15_aliasMSR; MSRC000_20E9

_ccd1_lthree0_inst7_n8_aliasMSR; MSRC000_2079

_ccd1_lthree0_inst8_n9_aliasMSR; MSRC000_2089

_ccd1_lthree0_inst9_n10_aliasMSR; MSRC000_2099

_ccd2_lthree0_inst10_n19_aliasMSR; MSRC000_20A9

_ccd2_lthree0_inst11_n20_aliasMSR; MSRC000_20B9

_ccd2_lthree0_inst12_n21_aliasMSR; MSRC000_20C9

_ccd2_lthree0_inst13_n22_aliasMSR; MSRC000_20D9

_ccd2_lthree0_inst14_n23_aliasMSR; MSRC000_20E9

_ccd2_lthree0_inst7_n16_aliasMSR; MSRC000_2079

_ccd2_lthree0_inst8_n17_aliasMSR; MSRC000_2089

_ccd2_lthree0_inst9_n18_aliasMSR; MSRC000_2099

_ccd3_lthree0_inst10_n27_aliasMSR; MSRC000_20A9

_ccd3_lthree0_inst11_n28_aliasMSR; MSRC000_20B9

_ccd3_lthree0_inst12_n29_aliasMSR; MSRC000_20C9

_ccd3_lthree0_inst13_n30_aliasMSR; MSRC000_20D9

_ccd3_lthree0_inst14_n31_aliasMSR; MSRC000_20E9

_ccd3_lthree0_inst7_n24_aliasMSR; MSRC000_2079

_ccd3_lthree0_inst8_n25_aliasMSR; MSRC000_2089

_ccd3_lthree0_inst9_n26_aliasMSR; MSRC000_2099

_ccd4_lthree0_inst10_n35_aliasMSR; MSRC000_20A9

_ccd4_lthree0_inst11_n36_aliasMSR; MSRC000_20B9

_ccd4_lthree0_inst12_n37_aliasMSR; MSRC000_20C9

_ccd4_lthree0_inst13_n38_aliasMSR; MSRC000_20D9

_ccd4_lthree0_inst14_n39_aliasMSR; MSRC000_20E9

_ccd4_lthree0_inst7_n32_aliasMSR; MSRC000_2079

_ccd4_lthree0_inst8_n33_aliasMSR; MSRC000_2089

_ccd4_lthree0_inst9_n34_aliasMSR; MSRC000_2099

_ccd5_lthree0_inst10_n43_aliasMSR; MSRC000_20A9

_ccd5_lthree0_inst11_n44_aliasMSR; MSRC000_20B9	
_ccd5_lthree0_inst12_n45_aliasMSR; MSRC000_20C9	
_ccd5_lthree0_inst13_n46_aliasMSR; MSRC000_20D9	
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC000_20E9	
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC000_2079	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC000_2089	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC000_2099	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC000_20A9	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC000_20B9	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC000_20C9	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC000_20D9	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC000_20E9	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC000_2079	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC000_2089	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC000_2099	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC000_20A9	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC000_20B9	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC000_20C9	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC000_20D9	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC000_20E9	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC000_2079	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC000_2089	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC000_2099	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC000_20A9	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC000_20B9	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC000_20C9	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC000_20D9	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC000_20E9	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC000_2079	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC000_2089	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC000_2099	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC000_20A9	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC000_20B9	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC000_20C9	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC000_20D9	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC000_20E9	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC000_2079	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC000_2089	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC000_2099	
Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::L3::MCA_DESTAT_L3. The lowest-order valid bit of the address is specified in MCA::L3::MCA_DESTAT_L3[AddrLsb].

MSRC001_040[7...E] [L3 Machine Check Control Mask] (MCA::L3::MCA_CTL_MASK_L3)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC001_040A_ccd0_lthree0_inst11_n4_aliasMSR; MSRC001_040B_ccd0_lthree0_inst12_n5_aliasMSR; MSRC001_040C_ccd0_lthree0_inst13_n6_aliasMSR; MSRC001_040D_ccd0_lthree0_inst14_n7_aliasMSR; MSRC001_040E_ccd0_lthree0_inst7_n0_aliasMSR; MSRC001_0407_ccd0_lthree0_inst8_n1_aliasMSR; MSRC001_0408_ccd0_lthree0_inst9_n2_aliasMSR; MSRC001_0409_ccd10_lthree0_inst10_n83_aliasMSR; MSRC001_040A_ccd10_lthree0_inst11_n84_aliasMSR; MSRC001_040B_ccd10_lthree0_inst12_n85_aliasMSR; MSRC001_040C_ccd10_lthree0_inst13_n86_aliasMSR; MSRC001_040D_ccd10_lthree0_inst14_n87_aliasMSR; MSRC001_040E_ccd10_lthree0_inst7_n80_aliasMSR; MSRC001_0407_ccd10_lthree0_inst8_n81_aliasMSR; MSRC001_0408_ccd10_lthree0_inst9_n82_aliasMSR; MSRC001_0409_ccd11_lthree0_inst10_n91_aliasMSR; MSRC001_040A_ccd11_lthree0_inst11_n92_aliasMSR; MSRC001_040B_ccd11_lthree0_inst12_n93_aliasMSR; MSRC001_040C_ccd11_lthree0_inst13_n94_aliasMSR; MSRC001_040D_ccd11_lthree0_inst14_n95_aliasMSR; MSRC001_040E_ccd11_lthree0_inst7_n88_aliasMSR; MSRC001_0407_ccd11_lthree0_inst8_n89_aliasMSR; MSRC001_0408_ccd11_lthree0_inst9_n90_aliasMSR; MSRC001_0409_ccd1_lthree0_inst10_n11_aliasMSR; MSRC001_040A_ccd1_lthree0_inst11_n12_aliasMSR; MSRC001_040B_ccd1_lthree0_inst12_n13_aliasMSR; MSRC001_040C_ccd1_lthree0_inst13_n14_aliasMSR; MSRC001_040D_ccd1_lthree0_inst14_n15_aliasMSR; MSRC001_040E_ccd1_lthree0_inst7_n8_aliasMSR; MSRC001_0407_ccd1_lthree0_inst8_n9_aliasMSR; MSRC001_0408_ccd1_lthree0_inst9_n10_aliasMSR; MSRC001_0409_ccd2_lthree0_inst10_n19_aliasMSR; MSRC001_040A_ccd2_lthree0_inst11_n20_aliasMSR; MSRC001_040B_ccd2_lthree0_inst12_n21_aliasMSR; MSRC001_040C_ccd2_lthree0_inst13_n22_aliasMSR; MSRC001_040D_ccd2_lthree0_inst14_n23_aliasMSR; MSRC001_040E_ccd2_lthree0_inst7_n16_aliasMSR; MSRC001_0407_ccd2_lthree0_inst8_n17_aliasMSR; MSRC001_0408_ccd2_lthree0_inst9_n18_aliasMSR; MSRC001_0409_ccd3_lthree0_inst10_n27_aliasMSR; MSRC001_040A_ccd3_lthree0_inst11_n28_aliasMSR; MSRC001_040B_ccd3_lthree0_inst12_n29_aliasMSR; MSRC001_040C_ccd3_lthree0_inst13_n30_aliasMSR; MSRC001_040D_ccd3_lthree0_inst14_n31_aliasMSR; MSRC001_040E_ccd3_lthree0_inst7_n24_aliasMSR; MSRC001_0407_ccd3_lthree0_inst8_n25_aliasMSR; MSRC001_0408_ccd3_lthree0_inst9_n26_aliasMSR; MSRC001_0409_ccd4_lthree0_inst10_n35_aliasMSR; MSRC001_040A_ccd4_lthree0_inst11_n36_aliasMSR; MSRC001_040B_ccd4_lthree0_inst12_n37_aliasMSR; MSRC001_040C_ccd4_lthree0_inst13_n38_aliasMSR; MSRC001_040D_ccd4_lthree0_inst14_n39_aliasMSR; MSRC001_040E_ccd4_lthree0_inst7_n32_aliasMSR; MSRC001_0407_ccd4_lthree0_inst8_n33_aliasMSR; MSRC001_0408_ccd4_lthree0_inst9_n34_aliasMSR; MSRC001_0409_ccd5_lthree0_inst10_n43_aliasMSR; MSRC001_040A_ccd5_lthree0_inst11_n44_aliasMSR; MSRC001_040B_ccd5_lthree0_inst12_n45_aliasMSR; MSRC001_040C_ccd5_lthree0_inst13_n46_aliasMSR; MSRC001_040D_ccd5_lthree0_inst14_n47_aliasMSR; MSRC001_040E

_ccd5_lthree0_inst7_n40_aliasMSR; MSRC001_0407	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC001_0408	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC001_0409	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC001_040A	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC001_040B	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC001_040C	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC001_040D	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC001_040E	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC001_0407	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC001_0408	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC001_0409	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC001_040A	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC001_040B	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC001_040C	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC001_040D	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC001_040E	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC001_0407	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC001_0408	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC001_0409	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC001_040A	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC001_040B	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC001_040C	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC001_040D	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC001_040E	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC001_0407	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC001_0408	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC001_0409	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC001_040A	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC001_040B	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC001_040C	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC001_040D	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC001_040E	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC001_0407	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC001_0408	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC001_0409	
Bits	Description
63:9	Reserved.
8	XiWcbParityPoison. Read-write. Reset: 0. Xi Wcb Parity Poison Creation Event
7	Hwa. Read-write. Reset: 0. L3 Hardware Assertion.
6	XiVictimQueue. Read-write. Reset: 0. L3 Victim Queue Data Fabric Error.
5	SdpParity. Read-write. Reset: 0. SDP Parity Error from XI.
4	DataArray. Read-write. Reset: 0. L3M Data ECC Error.
3	MultiHitTag. Read-write. Reset: 0. L3M Tag Multi-way-hit Error.
2	Tag. Read-write. Reset: 0. L3M Tag ECC Error. MCA_STATUS_L3[CECC]=1 and MCA_STATUS[UC]=1 indicates a hard fault.
1	MultiHitShadowTag. Read-write. Reset: 0. Shadow Tag Macro Multi-way-hit Error.
0	ShadowTag. Read-write. Reset: 0. Shadow Tag Macro ECC Error. MCA_STATUS_L3[CECC]=1 and MCA_STATUS[UC]=1 indicates a hard fault.

MSRC001_060[7...E] [L3 Machine Check Transparent Error Address] (MCA::L3::MCA_TRANSADDR_L3)

Reset: Cold,0000_0000_0000_0000h.

MCA::L3::MCA_TRANSADDR_L3 stores the address associated with the error in MCA::L3::MCA_STATUS_L3. The register is only meaningful if MCA::L3::MCA_STATUS_L3[Val]=1 and MCA::L3::MCA_STATUS_L3[AddrV]=1. The lowest valid bit of the address is defined by MCA::L3::MCA_STATUS_L3[AddrLsb].

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC001_060A
 _ccd0_lthree0_inst11_n4_aliasMSR; MSRC001_060B
 _ccd0_lthree0_inst12_n5_aliasMSR; MSRC001_060C
 _ccd0_lthree0_inst13_n6_aliasMSR; MSRC001_060D
 _ccd0_lthree0_inst14_n7_aliasMSR; MSRC001_060E
 _ccd0_lthree0_inst7_n0_aliasMSR; MSRC001_0607
 _ccd0_lthree0_inst8_n1_aliasMSR; MSRC001_0608
 _ccd0_lthree0_inst9_n2_aliasMSR; MSRC001_0609
 _ccd10_lthree0_inst10_n83_aliasMSR; MSRC001_060A
 _ccd10_lthree0_inst11_n84_aliasMSR; MSRC001_060B
 _ccd10_lthree0_inst12_n85_aliasMSR; MSRC001_060C
 _ccd10_lthree0_inst13_n86_aliasMSR; MSRC001_060D
 _ccd10_lthree0_inst14_n87_aliasMSR; MSRC001_060E
 _ccd10_lthree0_inst7_n80_aliasMSR; MSRC001_0607
 _ccd10_lthree0_inst8_n81_aliasMSR; MSRC001_0608
 _ccd10_lthree0_inst9_n82_aliasMSR; MSRC001_0609
 _ccd11_lthree0_inst10_n91_aliasMSR; MSRC001_060A
 _ccd11_lthree0_inst11_n92_aliasMSR; MSRC001_060B
 _ccd11_lthree0_inst12_n93_aliasMSR; MSRC001_060C
 _ccd11_lthree0_inst13_n94_aliasMSR; MSRC001_060D
 _ccd11_lthree0_inst14_n95_aliasMSR; MSRC001_060E
 _ccd11_lthree0_inst7_n88_aliasMSR; MSRC001_0607
 _ccd11_lthree0_inst8_n89_aliasMSR; MSRC001_0608
 _ccd11_lthree0_inst9_n90_aliasMSR; MSRC001_0609
 _ccd1_lthree0_inst10_n11_aliasMSR; MSRC001_060A
 _ccd1_lthree0_inst11_n12_aliasMSR; MSRC001_060B
 _ccd1_lthree0_inst12_n13_aliasMSR; MSRC001_060C
 _ccd1_lthree0_inst13_n14_aliasMSR; MSRC001_060D
 _ccd1_lthree0_inst14_n15_aliasMSR; MSRC001_060E
 _ccd1_lthree0_inst7_n8_aliasMSR; MSRC001_0607
 _ccd1_lthree0_inst8_n9_aliasMSR; MSRC001_0608
 _ccd1_lthree0_inst9_n10_aliasMSR; MSRC001_0609
 _ccd2_lthree0_inst10_n19_aliasMSR; MSRC001_060A
 _ccd2_lthree0_inst11_n20_aliasMSR; MSRC001_060B
 _ccd2_lthree0_inst12_n21_aliasMSR; MSRC001_060C
 _ccd2_lthree0_inst13_n22_aliasMSR; MSRC001_060D
 _ccd2_lthree0_inst14_n23_aliasMSR; MSRC001_060E
 _ccd2_lthree0_inst7_n16_aliasMSR; MSRC001_0607
 _ccd2_lthree0_inst8_n17_aliasMSR; MSRC001_0608
 _ccd2_lthree0_inst9_n18_aliasMSR; MSRC001_0609
 _ccd3_lthree0_inst10_n27_aliasMSR; MSRC001_060A
 _ccd3_lthree0_inst11_n28_aliasMSR; MSRC001_060B
 _ccd3_lthree0_inst12_n29_aliasMSR; MSRC001_060C
 _ccd3_lthree0_inst13_n30_aliasMSR; MSRC001_060D
 _ccd3_lthree0_inst14_n31_aliasMSR; MSRC001_060E
 _ccd3_lthree0_inst7_n24_aliasMSR; MSRC001_0607
 _ccd3_lthree0_inst8_n25_aliasMSR; MSRC001_0608
 _ccd3_lthree0_inst9_n26_aliasMSR; MSRC001_0609
 _ccd4_lthree0_inst10_n35_aliasMSR; MSRC001_060A
 _ccd4_lthree0_inst11_n36_aliasMSR; MSRC001_060B
 _ccd4_lthree0_inst12_n37_aliasMSR; MSRC001_060C
 _ccd4_lthree0_inst13_n38_aliasMSR; MSRC001_060D
 _ccd4_lthree0_inst14_n39_aliasMSR; MSRC001_060E
 _ccd4_lthree0_inst7_n32_aliasMSR; MSRC001_0607
 _ccd4_lthree0_inst8_n33_aliasMSR; MSRC001_0608
 _ccd4_lthree0_inst9_n34_aliasMSR; MSRC001_0609
 _ccd5_lthree0_inst10_n43_aliasMSR; MSRC001_060A
 _ccd5_lthree0_inst11_n44_aliasMSR; MSRC001_060B

_ccd5_lthree0_inst12_n45_aliasMSR; MSRC001_060C	
_ccd5_lthree0_inst13_n46_aliasMSR; MSRC001_060D	
_ccd5_lthree0_inst14_n47_aliasMSR; MSRC001_060E	
_ccd5_lthree0_inst7_n40_aliasMSR; MSRC001_0607	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC001_0608	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC001_0609	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC001_060A	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC001_060B	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC001_060C	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC001_060D	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC001_060E	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC001_0607	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC001_0608	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC001_0609	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC001_060A	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC001_060B	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC001_060C	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC001_060D	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC001_060E	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC001_0607	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC001_0608	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC001_0609	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC001_060A	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC001_060B	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC001_060C	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC001_060D	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC001_060E	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC001_0607	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC001_0608	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC001_0609	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC001_060A	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC001_060B	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC001_060C	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC001_060D	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC001_060E	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC001_0607	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC001_0608	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC001_0609	
Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::L3::MCA_STATUS_L3. The lowest-order valid bit of the address is specified in MCA::L3::MCA_STATUS_L3[AddrLsb].

MSRC001_050[7...E] [L3 Machine Check Transparent Error Syndrome] (MCA::L3::MCA_TRANSYND_L3)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_ccd0_lthree0_inst10_n3_aliasMSR; MSRC001_050A_ccd0_lthree0_inst11_n4_aliasMSR; MSRC001_050B_ccd0_lthree0_inst12_n5_aliasMSR; MSRC001_050C_ccd0_lthree0_inst13_n6_aliasMSR; MSRC001_050D_ccd0_lthree0_inst14_n7_aliasMSR; MSRC001_050E_ccd0_lthree0_inst7_n0_aliasMSR; MSRC001_0507_ccd0_lthree0_inst8_n1_aliasMSR; MSRC001_0508_ccd0_lthree0_inst9_n2_aliasMSR; MSRC001_0509_ccd10_lthree0_inst10_n83_aliasMSR; MSRC001_050A_ccd10_lthree0_inst11_n84_aliasMSR; MSRC001_050B_ccd10_lthree0_inst12_n85_aliasMSR; MSRC001_050C_ccd10_lthree0_inst13_n86_aliasMSR; MSRC001_050D_ccd10_lthree0_inst14_n87_aliasMSR; MSRC001_050E_ccd10_lthree0_inst7_n80_aliasMSR; MSRC001_0507_ccd10_lthree0_inst8_n81_aliasMSR; MSRC001_0508_ccd10_lthree0_inst9_n82_aliasMSR; MSRC001_0509_ccd11_lthree0_inst10_n91_aliasMSR; MSRC001_050A_ccd11_lthree0_inst11_n92_aliasMSR; MSRC001_050B_ccd11_lthree0_inst12_n93_aliasMSR; MSRC001_050C_ccd11_lthree0_inst13_n94_aliasMSR; MSRC001_050D_ccd11_lthree0_inst14_n95_aliasMSR; MSRC001_050E_ccd11_lthree0_inst7_n88_aliasMSR; MSRC001_0507_ccd11_lthree0_inst8_n89_aliasMSR; MSRC001_0508_ccd11_lthree0_inst9_n90_aliasMSR; MSRC001_0509_ccd1_lthree0_inst10_n11_aliasMSR; MSRC001_050A_ccd1_lthree0_inst11_n12_aliasMSR; MSRC001_050B_ccd1_lthree0_inst12_n13_aliasMSR; MSRC001_050C_ccd1_lthree0_inst13_n14_aliasMSR; MSRC001_050D_ccd1_lthree0_inst14_n15_aliasMSR; MSRC001_050E_ccd1_lthree0_inst7_n8_aliasMSR; MSRC001_0507_ccd1_lthree0_inst8_n9_aliasMSR; MSRC001_0508_ccd1_lthree0_inst9_n10_aliasMSR; MSRC001_0509_ccd2_lthree0_inst10_n19_aliasMSR; MSRC001_050A_ccd2_lthree0_inst11_n20_aliasMSR; MSRC001_050B_ccd2_lthree0_inst12_n21_aliasMSR; MSRC001_050C_ccd2_lthree0_inst13_n22_aliasMSR; MSRC001_050D_ccd2_lthree0_inst14_n23_aliasMSR; MSRC001_050E_ccd2_lthree0_inst7_n16_aliasMSR; MSRC001_0507_ccd2_lthree0_inst8_n17_aliasMSR; MSRC001_0508_ccd2_lthree0_inst9_n18_aliasMSR; MSRC001_0509_ccd3_lthree0_inst10_n27_aliasMSR; MSRC001_050A_ccd3_lthree0_inst11_n28_aliasMSR; MSRC001_050B_ccd3_lthree0_inst12_n29_aliasMSR; MSRC001_050C_ccd3_lthree0_inst13_n30_aliasMSR; MSRC001_050D_ccd3_lthree0_inst14_n31_aliasMSR; MSRC001_050E_ccd3_lthree0_inst7_n24_aliasMSR; MSRC001_0507_ccd3_lthree0_inst8_n25_aliasMSR; MSRC001_0508_ccd3_lthree0_inst9_n26_aliasMSR; MSRC001_0509_ccd4_lthree0_inst10_n35_aliasMSR; MSRC001_050A_ccd4_lthree0_inst11_n36_aliasMSR; MSRC001_050B_ccd4_lthree0_inst12_n37_aliasMSR; MSRC001_050C_ccd4_lthree0_inst13_n38_aliasMSR; MSRC001_050D_ccd4_lthree0_inst14_n39_aliasMSR; MSRC001_050E_ccd4_lthree0_inst7_n32_aliasMSR; MSRC001_0507_ccd4_lthree0_inst8_n33_aliasMSR; MSRC001_0508_ccd4_lthree0_inst9_n34_aliasMSR; MSRC001_0509_ccd5_lthree0_inst10_n43_aliasMSR; MSRC001_050A_ccd5_lthree0_inst11_n44_aliasMSR; MSRC001_050B_ccd5_lthree0_inst12_n45_aliasMSR; MSRC001_050C_ccd5_lthree0_inst13_n46_aliasMSR; MSRC001_050D_ccd5_lthree0_inst14_n47_aliasMSR; MSRC001_050E

_ccd5_lthree0_inst7_n40_aliasMSR; MSRC001_0507	
_ccd5_lthree0_inst8_n41_aliasMSR; MSRC001_0508	
_ccd5_lthree0_inst9_n42_aliasMSR; MSRC001_0509	
_ccd6_lthree0_inst10_n51_aliasMSR; MSRC001_050A	
_ccd6_lthree0_inst11_n52_aliasMSR; MSRC001_050B	
_ccd6_lthree0_inst12_n53_aliasMSR; MSRC001_050C	
_ccd6_lthree0_inst13_n54_aliasMSR; MSRC001_050D	
_ccd6_lthree0_inst14_n55_aliasMSR; MSRC001_050E	
_ccd6_lthree0_inst7_n48_aliasMSR; MSRC001_0507	
_ccd6_lthree0_inst8_n49_aliasMSR; MSRC001_0508	
_ccd6_lthree0_inst9_n50_aliasMSR; MSRC001_0509	
_ccd7_lthree0_inst10_n59_aliasMSR; MSRC001_050A	
_ccd7_lthree0_inst11_n60_aliasMSR; MSRC001_050B	
_ccd7_lthree0_inst12_n61_aliasMSR; MSRC001_050C	
_ccd7_lthree0_inst13_n62_aliasMSR; MSRC001_050D	
_ccd7_lthree0_inst14_n63_aliasMSR; MSRC001_050E	
_ccd7_lthree0_inst7_n56_aliasMSR; MSRC001_0507	
_ccd7_lthree0_inst8_n57_aliasMSR; MSRC001_0508	
_ccd7_lthree0_inst9_n58_aliasMSR; MSRC001_0509	
_ccd8_lthree0_inst10_n67_aliasMSR; MSRC001_050A	
_ccd8_lthree0_inst11_n68_aliasMSR; MSRC001_050B	
_ccd8_lthree0_inst12_n69_aliasMSR; MSRC001_050C	
_ccd8_lthree0_inst13_n70_aliasMSR; MSRC001_050D	
_ccd8_lthree0_inst14_n71_aliasMSR; MSRC001_050E	
_ccd8_lthree0_inst7_n64_aliasMSR; MSRC001_0507	
_ccd8_lthree0_inst8_n65_aliasMSR; MSRC001_0508	
_ccd8_lthree0_inst9_n66_aliasMSR; MSRC001_0509	
_ccd9_lthree0_inst10_n75_aliasMSR; MSRC001_050A	
_ccd9_lthree0_inst11_n76_aliasMSR; MSRC001_050B	
_ccd9_lthree0_inst12_n77_aliasMSR; MSRC001_050C	
_ccd9_lthree0_inst13_n78_aliasMSR; MSRC001_050D	
_ccd9_lthree0_inst14_n79_aliasMSR; MSRC001_050E	
_ccd9_lthree0_inst7_n72_aliasMSR; MSRC001_0507	
_ccd9_lthree0_inst8_n73_aliasMSR; MSRC001_0508	
_ccd9_lthree0_inst9_n74_aliasMSR; MSRC001_0509	
Bits	Description
63:51	Reserved.
50:32	Syndrome. Read-write, Volatile. Reset: Cold,0_0000h. Contains the syndrome, if any, associated with the error logged in MCA::L3::MCA_STATUS_L3. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::L3::MCA_SYND_L3[Length]. The Syndrome field is only valid when MCA::L3::MCA_SYND_L3[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold,0. This bit is a Valid bit in MCA::L3::MCA_TRANSSYND_L3. All errors writing into MCA::L3::MCA_SYND_L3 should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold,0h. Encodes the priority of the error logged in MCA::L3::MCA_SYND_L3. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold,00h. Specifies the length in bits of the syndrome contained in MCA::L3::MCA_SYND_L3[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::L3::MCA_SYND_L3. For example, a syndrome length of 9 means that MCA::L3::MCA_SYND_L3[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold,0_0000h. Contains error-specific information about the location of the error.

3.2.5.8 CS

MSR0000_045C...MSRC000_2180 [CS Machine Check Control] (MCA::CS::MCA_CTL_CS)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::CS::MCA_CTL_CS register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instCMP0_n12_aliasMSR; MSRC000_2170

_instCMP0_n12_aliasMSRLEGACY; MSR0000_045C

_instCMP1_n13_aliasMSR; MSRC000_2180

_instCMP1_n13_aliasMSRLEGACY; MSR0000_0460

_instCMP2_n14_aliasMSR; MSRC000_2170

_instCMP2_n14_aliasMSRLEGACY; MSR0000_045C

_instCMP3_n15_aliasMSR; MSRC000_2180

_instCMP3_n15_aliasMSRLEGACY; MSR0000_0460

_instCS0_n0_aliasMSR; MSRC000_2170

_instCS0_n0_aliasMSRLEGACY; MSR0000_045C

_instCS10_n10_aliasMSR; MSRC000_2170

_instCS10_n10_aliasMSRLEGACY; MSR0000_045C

_instCS11_n11_aliasMSR; MSRC000_2180

_instCS11_n11_aliasMSRLEGACY; MSR0000_0460

_instCS1_n1_aliasMSR; MSRC000_2180

_instCS1_n1_aliasMSRLEGACY; MSR0000_0460

_instCS2_n2_aliasMSR; MSRC000_2170

_instCS2_n2_aliasMSRLEGACY; MSR0000_045C

_instCS3_n3_aliasMSR; MSRC000_2180

_instCS3_n3_aliasMSRLEGACY; MSR0000_0460

_instCS4_n4_aliasMSR; MSRC000_2170

_instCS4_n4_aliasMSRLEGACY; MSR0000_045C

_instCS5_n5_aliasMSR; MSRC000_2180

_instCS5_n5_aliasMSRLEGACY; MSR0000_0460

_instCS6_n6_aliasMSR; MSRC000_2170

_instCS6_n6_aliasMSRLEGACY; MSR0000_045C

_instCS7_n7_aliasMSR; MSRC000_2180

_instCS7_n7_aliasMSRLEGACY; MSR0000_0460

_instCS8_n8_aliasMSR; MSRC000_2170

_instCS8_n8_aliasMSRLEGACY; MSR0000_045C

_instCS9_n9_aliasMSR; MSRC000_2180

_instCS9_n9_aliasMSRLEGACY; MSR0000_0460

Bits	Description
------	-------------

63:18	Reserved.
-------	-----------

17	HWA. Read-write. Reset: 0. Hardware Assert Error.
----	--

16	FTI_ND_SEC_VIOL. Read-write. Reset: 0. Security Violation: A security violation was detected on an incoming request from the transport layer on the request no data channel.
----	---

15	FTI_ND_ADDR_VIOL. Read-write. Reset: 0. Address Violation: An address violation was detected on an incoming request from the transport layer on the request no data channel.
----	---

14	FTI_ND_ILL_REQ. Read-write. Reset: 0. Illegal Request: An illegal request was received from the transport layer on the request no data channel.
----	--

13	SPF_ECC_ERR. Read-write. Reset: 0. Probe Filter ECC Error: An ECC error occurred on a probe filter access.
----	---

12	SPF_PRT_ERR. Read-write. Reset: 0. Probe Filter Protocol Error: Indicates a Cache Coherence Issue.
----	---

11	CNTR_UNFL. Read-write. Reset: 0. Counter underflow error.
----	--

10	CNTR_OVFL. Read-write. Reset: 0. Counter overflow error.
----	---

9	SDP_UNEXP_RETRY. Read-write. Reset: 0. SDP read response had an unexpected RETRY error.
---	--

8	SDP_RSP_NO_MTCH. Read-write. Reset: 0. SDP read response had no match in the CS queue.
---	---

7	ATM_PAR_ERR. Read-write. Reset: 0. Atomic Request Parity Error: Parity error on read of an atomic transaction.
---	---

6	SDP_PAR_ERR. Read-write. Reset: 0. Read Response Parity Error: Parity error on incoming read response data.
---	--

5	FTI_PAR_ERR. Read-write. Reset: 0. Request or Probe Parity Error: Parity error on incoming request or probe response data.
---	---

4	FTI_RSP_NO_MTCH. Read-write. Reset: 0. Unexpected Response: A response was received from the transport layer which does not match any request.
3	FTI_ILL_RSP. Read-write. Reset: 0. Illegal Response: An illegal response was received from the transport layer.
2	FTI_SEC_VIOL. Read-write. Reset: 0. Security Violation: A security violation was detected on an incoming request from the transport layer on the primary request channel.
1	FTI_ADDR_VIOL. Read-write. Reset: 0. Address Violation: An address violation was detected on an incoming request from the transport layer on the primary request channel.
0	FTI_ILL_REQ. Read-write. Reset: 0. Illegal Request: An illegal request was received from the transport layer on the primary request channel.

MSR0000_045D...MSRC000_2181 [CS Machine Check Status] (MCA::CS::MCA_STATUS_CS)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instCMP0_n12_aliasMSR; MSRC000_2171_instCMP0_n12_aliasMSRLEGACY; MSR0000_045D_instCMP1_n13_aliasMSR; MSRC000_2181_instCMP1_n13_aliasMSRLEGACY; MSR0000_0461_instCMP2_n14_aliasMSR; MSRC000_2171_instCMP2_n14_aliasMSRLEGACY; MSR0000_045D_instCMP3_n15_aliasMSR; MSRC000_2181_instCMP3_n15_aliasMSRLEGACY; MSR0000_0461_instCS0_n0_aliasMSR; MSRC000_2171_instCS0_n0_aliasMSRLEGACY; MSR0000_045D_instCS10_n10_aliasMSR; MSRC000_2171_instCS10_n10_aliasMSRLEGACY; MSR0000_045D_instCS11_n11_aliasMSR; MSRC000_2181_instCS11_n11_aliasMSRLEGACY; MSR0000_0461_instCS1_n1_aliasMSR; MSRC000_2181_instCS1_n1_aliasMSRLEGACY; MSR0000_0461_instCS2_n2_aliasMSR; MSRC000_2171_instCS2_n2_aliasMSRLEGACY; MSR0000_045D_instCS3_n3_aliasMSR; MSRC000_2181_instCS3_n3_aliasMSRLEGACY; MSR0000_0461_instCS4_n4_aliasMSR; MSRC000_2171_instCS4_n4_aliasMSRLEGACY; MSR0000_045D_instCS5_n5_aliasMSR; MSRC000_2181_instCS5_n5_aliasMSRLEGACY; MSR0000_0461_instCS6_n6_aliasMSR; MSRC000_2171_instCS6_n6_aliasMSRLEGACY; MSR0000_045D_instCS7_n7_aliasMSR; MSRC000_2181_instCS7_n7_aliasMSRLEGACY; MSR0000_0461_instCS8_n8_aliasMSR; MSRC000_2171_instCS8_n8_aliasMSRLEGACY; MSR0000_045D_instCS9_n9_aliasMSR; MSRC000_2181_instCS9_n9_aliasMSRLEGACY; MSR0000_0461**Bits Description**

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::CS::MCA_CTL_CS. This bit is a copy of bit in MCA::CS::MCA_CTL_CS for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::CS::MCA_MISC0_CS. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::CS::MCA_ADDR_CS contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::CS::MCA_STATUS_CS[PCC]=0.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::CS::MCA_SYND_CS. If MCA::CS::MCA_SYND_CS[ErrorPriority] is the same as the priority of the error in MCA::CS::MCA_STATUS_CS, then the information in MCA::CS::MCA_SYND_CS is associated with the error in MCA::CS::MCA_STATUS_CS.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::CS::MCA_STATUS_CS[UC]=0.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::CS::MCA_ADDR_CS[ErrorAddr]. A value of 0 indicates that MCA::CS::MCA_ADDR_CS[55:0] contains a valid byte address. A value of 6 indicates that MCA::CS::MCA_ADDR_CS[55:6] contains a valid cache line address and that MCA::CS::MCA_ADDR_CS[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::CS::MCA_ADDR_CS[55:12] contain a valid 4KB memory page and that MCA::CS::MCA_ADDR_CS[11:0] should be ignored by error handling software.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::CS::MCA_CTL_CS enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 79: MCA_STATUS_CS

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
FTI_ILL_REQ	0x0	0	0	0	1	0	1
FTI_ADDR_VIOL	0x1	0	0	0	1	0	1
FTI_SEC_VIOL	0x2	0	0	0	1	0	1
FTI_ILL_RSP	0x3	1	1	1	0	0	0
FTI_RSP_NO_MTCH	0x4	1	1	1	0	0	0
FTI_PAR_ERR	0x5	0	0	0	1	0	1
SDP_PAR_ERR	0x6	0	0	0	1	0	1
ATM_PAR_ERR	0x7	0	0	0	1	0	1
SDP_RSP_NO_MTCH	0x8	1	1	1	0	0	0
SDP_UNEXP_RETRY	0x9	1	1	1	0	0	1
CNTR_OVF_L	0xa	1	1	1	0	0	0
CNTR_UNF_L	0xb	1	1	1	0	0	0
SPF_PRT_ERR	0xc	1	1	1	0	0	0
SPF_ECC_ERR	0xd	0/1	0/1	0/1	0	0	1
FTI_ND_ILLEGAL_REQ	0xe	0	0	0	1	0	1
FTI_ND_ADDR_VIOL	0xf	0	0	0	1	0	1
FTI_ND_SECC_VIOL	0x10	0	0	0	1	0	1
HWA	0x11	1	1	1	0	0	0

MSR0000_045E...MSRC000_2182 [CS Machine Check Address] (MCA::CS::MCA_ADDR_CS)

Reset: Cold,0000_0000_0000_0000h.

MCA::CS::MCA_ADDR_CS stores an address and other information associated with the error in MCA::CS::MCA_STATUS_CS. The register is only meaningful if MCA::CS::MCA_STATUS_CS[Val]=1 and MCA::CS::MCA_STATUS_CS[AddrV]=1.

_instCMP0_n12_aliasMSR; MSRC000_2172_instCMP0_n12_aliasMSRLEGACY; MSR0000_045E_instCMP1_n13_aliasMSR; MSRC000_2182_instCMP1_n13_aliasMSRLEGACY; MSR0000_0462_instCMP2_n14_aliasMSR; MSRC000_2172_instCMP2_n14_aliasMSRLEGACY; MSR0000_045E_instCMP3_n15_aliasMSR; MSRC000_2182_instCMP3_n15_aliasMSRLEGACY; MSR0000_0462_instCS0_n0_aliasMSR; MSRC000_2172_instCS0_n0_aliasMSRLEGACY; MSR0000_045E_instCS10_n10_aliasMSR; MSRC000_2172_instCS10_n10_aliasMSRLEGACY; MSR0000_045E_instCS11_n11_aliasMSR; MSRC000_2182_instCS11_n11_aliasMSRLEGACY; MSR0000_0462_instCS1_n1_aliasMSR; MSRC000_2182_instCS1_n1_aliasMSRLEGACY; MSR0000_0462_instCS2_n2_aliasMSR; MSRC000_2172_instCS2_n2_aliasMSRLEGACY; MSR0000_045E_instCS3_n3_aliasMSR; MSRC000_2182_instCS3_n3_aliasMSRLEGACY; MSR0000_0462_instCS4_n4_aliasMSR; MSRC000_2172_instCS4_n4_aliasMSRLEGACY; MSR0000_045E_instCS5_n5_aliasMSR; MSRC000_2182_instCS5_n5_aliasMSRLEGACY; MSR0000_0462_instCS6_n6_aliasMSR; MSRC000_2172_instCS6_n6_aliasMSRLEGACY; MSR0000_045E_instCS7_n7_aliasMSR; MSRC000_2182_instCS7_n7_aliasMSRLEGACY; MSR0000_0462_instCS8_n8_aliasMSR; MSRC000_2172_instCS8_n8_aliasMSRLEGACY; MSR0000_045E_instCS9_n9_aliasMSR; MSRC000_2182_instCS9_n9_aliasMSRLEGACY; MSR0000_0462**Bits Description**

63:56 Reserved.

55:0 **ErrorAddr.** Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::CS::MCA_STATUS_CS. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 80: MCA_ADDR_CS

Error Type	Bits	Description
FTI_ILL_REQ	[55:2]	Address
FTI_ADDR_VIOL	[55:2]	Address
FTI_SEC_VIOL	[55:2]	Address
FTI_ILL_RSP	[56:0]	Reserved
FTI_RSP_NO_MTCH	[56:0]	Reserved
FTI_PAR_ERR	[55:2]	Address
SDP_PAR_ERR	[55:2]	Address
ATM_PAR_ERR	[55:2]	Address
SDP_RSP_NO_MTCH	[56:0]	Reserved
SDP_UNEXP_RETRY	[55:2]	Address
CNTR_OVFL	[56:0]	Reserved

CNTR_UNFL	[56:0]	Reserved
SPF_PRT_ERR	[56:0]	Reserved
SPF_ECC_ERR	[55:2]	Address
FTI_ND_ILL_REQ	[55:2]	Address
FTI_ND_ADDR_VIOL	[55:2]	Address
FTI_ND_SEC_VIOL	[55:2]	Address
HWA	[55:2]	Address

MSR0000_045F...MSRC000_2183 [CS Machine Check Miscellaneous 0] (MCA::CS::MCA_MISC0_CS)

Log miscellaneous information associated with errors.

_instCMP0_n12_aliasMSR; MSRC000_2173

_instCMP0_n12_aliasMSRLEGACY; MSR0000_045F

_instCMP1_n13_aliasMSR; MSRC000_2183

_instCMP1_n13_aliasMSRLEGACY; MSR0000_0463

_instCMP2_n14_aliasMSR; MSRC000_2173

_instCMP2_n14_aliasMSRLEGACY; MSR0000_045F

_instCMP3_n15_aliasMSR; MSRC000_2183

_instCMP3_n15_aliasMSRLEGACY; MSR0000_0463

_instCS0_n0_aliasMSR; MSRC000_2173

_instCS0_n0_aliasMSRLEGACY; MSR0000_045F

_instCS10_n10_aliasMSR; MSRC000_2173

_instCS10_n10_aliasMSRLEGACY; MSR0000_045F

_instCS11_n11_aliasMSR; MSRC000_2183

_instCS11_n11_aliasMSRLEGACY; MSR0000_0463

_instCS1_n1_aliasMSR; MSRC000_2183

_instCS1_n1_aliasMSRLEGACY; MSR0000_0463

_instCS2_n2_aliasMSR; MSRC000_2173

_instCS2_n2_aliasMSRLEGACY; MSR0000_045F

_instCS3_n3_aliasMSR; MSRC000_2183

_instCS3_n3_aliasMSRLEGACY; MSR0000_0463

_instCS4_n4_aliasMSR; MSRC000_2173

_instCS4_n4_aliasMSRLEGACY; MSR0000_045F

_instCS5_n5_aliasMSR; MSRC000_2183

_instCS5_n5_aliasMSRLEGACY; MSR0000_0463

_instCS6_n6_aliasMSR; MSRC000_2173

_instCS6_n6_aliasMSRLEGACY; MSR0000_045F

_instCS7_n7_aliasMSR; MSRC000_2183

_instCS7_n7_aliasMSRLEGACY; MSR0000_0463

_instCS8_n8_aliasMSR; MSRC000_2173

_instCS8_n8_aliasMSRLEGACY; MSR0000_045F

_instCS9_n9_aliasMSR; MSRC000_2183

_instCS9_n9_aliasMSRLEGACY; MSR0000_0463

Bits Description

63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::CS::MCA_MISC0_CS[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::CS::MCA_MISC0_CS[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::CS::MCA_MISC0_CS[Locked]) ? Read-write : Read-only.

50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::CS::MCA_MISC0_CS[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::CS::MCA_MISC0_CS[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::CS::MCA_MISC0_CS[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21[7...8]4 [CS Machine Check Configuration] (MCA::CS::MCA_CONFIG_CS)

Reset: 0000_0002_0000_0127h.

Controls configuration of the associated machine check bank.

[_instCMP0_n12_aliasMSR](#); MSRC000_2174[_instCMP1_n13_aliasMSR](#); MSRC000_2184[_instCMP2_n14_aliasMSR](#); MSRC000_2174[_instCMP3_n15_aliasMSR](#); MSRC000_2184[_instCS0_n0_aliasMSR](#); MSRC000_2174[_instCS10_n10_aliasMSR](#); MSRC000_2174[_instCS11_n11_aliasMSR](#); MSRC000_2184[_instCS1_n1_aliasMSR](#); MSRC000_2184[_instCS2_n2_aliasMSR](#); MSRC000_2174[_instCS3_n3_aliasMSR](#); MSRC000_2184[_instCS4_n4_aliasMSR](#); MSRC000_2174[_instCS5_n5_aliasMSR](#); MSRC000_2184[_instCS6_n6_aliasMSR](#); MSRC000_2174[_instCS7_n7_aliasMSR](#); MSRC000_2184[_instCS8_n8_aliasMSR](#); MSRC000_2174[_instCS9_n9_aliasMSR](#); MSRC000_2184

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::CS::MCA_STATUS_CS and MCA::CS::MCA_ADDR_CS in addition to MCA::CS::MCA_DESTAT_CS and MCA::CS::MCA_DEADDR_CS. 0=Only log deferred errors in MCA::CS::MCA_DESTAT_CS and MCA::CS::MCA_DEADDR_CS. This bit does not affect logging of deferred errors in MCA::CS::MCA_SYND_CS, MCA::CS::MCA_MISC0_CS.
33	TransparentErrorLoggingEnable . Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::CS::MCA_STATUS_CS, MCA::CS::MCA_ADDR_CS, MCA::CS::MCA_MISC0_CS, and MCA::CS::MCA_SYND_CS in addition to MCA::CS::MCA_TRANSYND_CS and MCA::CS::MCA_TRANSADDR_CS. 0=Only log transparent errors in MCA::CS::MCA_TRANSYND_CS and MCA::CS::MCA_TRANSADDR_CS.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::CS::MCA_CONFIG_CS[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::CS::MCA_CONFIG_CS[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::CS::MCA_CONFIG_CS[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::CS::MCA_CONFIG_CS[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::CS::MCA_DESTAT_CS and MCA::CS::MCA_DEADDR_CS are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::CS::MCA_CONFIG_CS[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.

0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::CS::MCA_MISC0_CS[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::CS::MCA_STATUS_CS[TCC] is present.
---	---

MSRC000_21[7...8]5 [CS IP Identification] (MCA::CS::MCA_IPID_CS)

Reset: 0002_002E_0000_0000h.

The MCA::CS::MCA_IPID_CS register is used by software to determine what IP type and revision is associated with the MCA bank.

[_instCMP0_n12_aliasMSR](#); MSRC000_2175[_instCMP1_n13_aliasMSR](#); MSRC000_2185[_instCMP2_n14_aliasMSR](#); MSRC000_2175[_instCMP3_n15_aliasMSR](#); MSRC000_2185[_instCS0_n0_aliasMSR](#); MSRC000_2175[_instCS10_n10_aliasMSR](#); MSRC000_2175[_instCS11_n11_aliasMSR](#); MSRC000_2185[_instCS1_n1_aliasMSR](#); MSRC000_2185[_instCS2_n2_aliasMSR](#); MSRC000_2175[_instCS3_n3_aliasMSR](#); MSRC000_2185[_instCS4_n4_aliasMSR](#); MSRC000_2175[_instCS5_n5_aliasMSR](#); MSRC000_2185[_instCS6_n6_aliasMSR](#); MSRC000_2175[_instCS7_n7_aliasMSR](#); MSRC000_2185[_instCS8_n8_aliasMSR](#); MSRC000_2175[_instCS9_n9_aliasMSR](#); MSRC000_2185**Bits Description**63:48 **McaType.** Read-only. Reset: 0002h. The McaType of the MCA bank within this IP.47:44 **InstanceIdHi.** Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.43:32 **HardwareID.** Read-only. Reset: 02Eh. The Hardware ID of the IP associated with this MCA bank.31:0 **InstanceId.** Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.Init: [_instCMP0_n12_aliasMSR](#): 1F00_0C00hInit: [_instCMP1_n13_aliasMSR](#): 1F00_0D00hInit: [_instCMP2_n14_aliasMSR](#): 1F00_0E00hInit: [_instCMP3_n15_aliasMSR](#): 1F00_0F00hInit: [_instCS0_n0_aliasMSR](#): 1F00_0000hInit: [_instCS1_n1_aliasMSR](#): 1F00_0100hInit: [_instCS2_n2_aliasMSR](#): 1F00_0200hInit: [_instCS3_n3_aliasMSR](#): 1F00_0300hInit: [_instCS4_n4_aliasMSR](#): 1F00_0400hInit: [_instCS5_n5_aliasMSR](#): 1F00_0500hInit: [_instCS6_n6_aliasMSR](#): 1F00_0600hInit: [_instCS7_n7_aliasMSR](#): 1F00_0700hInit: [_instCS8_n8_aliasMSR](#): 1F00_0800hInit: [_instCS9_n9_aliasMSR](#): 1F00_0900hInit: [_instCS10_n10_aliasMSR](#): 1F00_0A00hInit: [_instCS11_n11_aliasMSR](#): 1F00_0B00h

MSRC000_21[7...8]6 [CS Machine Check Syndrome] (MCA::CS::MCA_SYND_CS)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::CS::MCA_STATUS_CS [Thread 0](#)[_instCMP0_n12_aliasMSR](#); MSRC000_2176[_instCMP1_n13_aliasMSR](#); MSRC000_2186[_instCMP2_n14_aliasMSR](#); MSRC000_2176[_instCMP3_n15_aliasMSR](#); MSRC000_2186[_instCS0_n0_aliasMSR](#); MSRC000_2176[_instCS10_n10_aliasMSR](#); MSRC000_2176[_instCS11_n11_aliasMSR](#); MSRC000_2186[_instCS1_n1_aliasMSR](#); MSRC000_2186[_instCS2_n2_aliasMSR](#); MSRC000_2176[_instCS3_n3_aliasMSR](#); MSRC000_2186[_instCS4_n4_aliasMSR](#); MSRC000_2176[_instCS5_n5_aliasMSR](#); MSRC000_2186[_instCS6_n6_aliasMSR](#); MSRC000_2176[_instCS7_n7_aliasMSR](#); MSRC000_2186[_instCS8_n8_aliasMSR](#); MSRC000_2176[_instCS9_n9_aliasMSR](#); MSRC000_2186**Bits Description**

63:41 Reserved.

40:32 **Syndrone**. Read-write, Volatile. Reset: Cold, 000h. Contains the syndrome, if any, associated with the error logged in MCA::CS::MCA_STATUS_CS. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::CS::MCA_SYND_CS[Length]. The Syndrome field is only valid when MCA::CS::MCA_SYND_CS[Length] is not 0.

31:27 Reserved.

26:24 **ErrorPriority**. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::CS::MCA_SYND_CS. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.

23:18 **Length**. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::CS::MCA_SYND_CS[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::CS::MCA_SYND_CS. For example, a syndrome length of 9 means that MCA::CS::MCA_SYND_CS[Syndrome] bits [8:0] contains a valid syndrome.

17:0 **ErrorInformation**. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 81 [MCA_SYND_CS].

Table 81: MCA_SYND_CS

Error Type	Bits	Description
FTI_ILL_REQ	[17:0]	Reserved
FTI_ADDR_VIOL	[17:0]	Reserved
FTI_SEC_VIOL	[17:0]	Reserved
FTI_ILL_RSP	[17:0]	Reserved
FTI_RSP_NO_MTCH	[17:0]	Reserved
FTI_PAR_ERR	[6:0]	Reserved
SDP_PAR_ERR	[6:0]	Reserved
ATM_PAR_ERR	[6:0]	Reserved
SDP_RSP_NO_MTCH	[7:0]	Reserved
SDP_UNEXP_RETRY	[6:0]	Reserved
CNTR_OVFL	[17:0]	Reserved
CNTR_UNFL	[17:0]	Reserved
SPF_PRT_ERR	[17:0]	Reserved
SPF_ECC_ERR	[17:0]	Reserved
FTI_ND_ILL_REQ	[17:0]	Reserved

FTI_ND_ADDR_VIOL	[17:0]	Reserved
FTI_ND_SEC_VIOL	[17:0]	Reserved
HWA	[17:0]	Reserved

MSRC000_21[7...8]8 [CS Machine Check Deferred Error Status] (MCA::CS::MCA_DESTAT_CS)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instCMP0_n12_aliasMSR; MSRC000_2178_instCMP1_n13_aliasMSR; MSRC000_2188_instCMP2_n14_aliasMSR; MSRC000_2178_instCMP3_n15_aliasMSR; MSRC000_2188_instCS0_n0_aliasMSR; MSRC000_2178_instCS10_n10_aliasMSR; MSRC000_2178_instCS11_n11_aliasMSR; MSRC000_2188_instCS1_n1_aliasMSR; MSRC000_2188_instCS2_n2_aliasMSR; MSRC000_2178_instCS3_n3_aliasMSR; MSRC000_2188_instCS4_n4_aliasMSR; MSRC000_2178_instCS5_n5_aliasMSR; MSRC000_2188_instCS6_n6_aliasMSR; MSRC000_2178_instCS7_n7_aliasMSR; MSRC000_2188_instCS8_n8_aliasMSR; MSRC000_2178_instCS9_n9_aliasMSR; MSRC000_2188

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::CS::MCA_DEADDR_CS contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::CS::MCA_SYND_CS. If MCA::CS::MCA_SYND_CS[ErrorPriority] is the same as the priority of the error in MCA::CS::MCA_STATUS_CS, then the information in MCA::CS::MCA_SYND_CS is associated with the error in MCA::CS::MCA_DESTAT_CS.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::CS::MCA_ADDR_CS[ErrorAddr]. A value of 0 indicates that MCA::CS::MCA_ADDR_CS[55:0] contains a valid byte address. A value of 6 indicates that MCA::CS::MCA_ADDR_CS[55:6] contains a valid cache line address and that MCA::CS::MCA_ADDR_CS[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::CS::MCA_ADDR_CS[55:12] contain a valid 4KB memory page and that MCA::CS::MCA_ADDR_CS[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21[7...8]9 [CS Deferred Error Address] (MCA::CS::MCA_DEADDR_CS)

Reset: Cold,0000_0000_0000_0000h.

The MCA::CS::MCA_DEADDR_CS register stores the address associated with the error in MCA::CS::MCA_DESTAT_CS. The register is only meaningful if MCA::CS::MCA_DESTAT_CS[Val]=1 and MCA::CS::MCA_DESTAT_CS[AddrV]=1. The lowest valid bit of the address is defined by MCA::CS::MCA_DESTAT_CS[AddrLsb].

_instCMP0_n12_aliasMSR; MSRC000_2179

_instCMP1_n13_aliasMSR; MSRC000_2189

_instCMP2_n14_aliasMSR; MSRC000_2179

_instCMP3_n15_aliasMSR; MSRC000_2189

_instCS0_n0_aliasMSR; MSRC000_2179

_instCS10_n10_aliasMSR; MSRC000_2179

_instCS11_n11_aliasMSR; MSRC000_2189

_instCS1_n1_aliasMSR; MSRC000_2189

_instCS2_n2_aliasMSR; MSRC000_2179

_instCS3_n3_aliasMSR; MSRC000_2189

_instCS4_n4_aliasMSR; MSRC000_2179

_instCS5_n5_aliasMSR; MSRC000_2189

_instCS6_n6_aliasMSR; MSRC000_2179

_instCS7_n7_aliasMSR; MSRC000_2189

_instCS8_n8_aliasMSR; MSRC000_2179

_instCS9_n9_aliasMSR; MSRC000_2189

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::CS::MCA_DESTAT_CS. The lowest-order valid bit of the address is specified in MCA::CS::MCA_DESTAT_CS[AddrLsb].

MSRC001_041[7...8] [CS Machine Check Control Mask] (MCA::CS::MCA_CTL_MASK_CS)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instCMP0_n12_aliasMSR; MSRC001_0417_instCMP1_n13_aliasMSR; MSRC001_0418_instCMP2_n14_aliasMSR; MSRC001_0417_instCMP3_n15_aliasMSR; MSRC001_0418_instCS0_n0_aliasMSR; MSRC001_0417_instCS10_n10_aliasMSR; MSRC001_0417_instCS11_n11_aliasMSR; MSRC001_0418_instCS1_n1_aliasMSR; MSRC001_0418_instCS2_n2_aliasMSR; MSRC001_0417_instCS3_n3_aliasMSR; MSRC001_0418_instCS4_n4_aliasMSR; MSRC001_0417_instCS5_n5_aliasMSR; MSRC001_0418_instCS6_n6_aliasMSR; MSRC001_0417_instCS7_n7_aliasMSR; MSRC001_0418_instCS8_n8_aliasMSR; MSRC001_0417_instCS9_n9_aliasMSR; MSRC001_0418**Bits Description**

63:18 Reserved.

17	HWA. Read-write. Reset: 0. Hardware Assert Error.
16	FTI_ND_SEC_VIOL. Read-write. Reset: 0. Security Violation: A security violation was detected on an incoming request from the transport layer on the request no data channel.
15	FTI_ND_ADDR_VIOL. Read-write. Reset: 0. Address Violation: An address violation was detected on an incoming request from the transport layer on the request no data channel.
14	FTI_ND_ILL_REQ. Read-write. Reset: 0. Illegal Request: An illegal request was received from the transport layer on the request no data channel.
13	SPF_ECC_ERR. Read-write. Reset: 0. Probe Filter ECC Error: An ECC error occurred on a probe filter access.
12	SPF_PRT_ERR. Read-write. Reset: 0. Probe Filter Protocol Error: Indicates a Cache Coherence Issue.
11	CNTR_UNFL. Read-write. Reset: 0. Counter underflow error.
10	CNTR_OVFL. Read-write. Reset: 0. Counter overflow error.
9	SDP_UNEXP_RETRY. Read-write. Reset: 0. SDP read response had an unexpected RETRY error.
8	SDP_RSP_NO_MTCH. Read-write. Reset: 0. SDP read response had no match in the CS queue.
7	ATM_PAR_ERR. Read-write. Reset: 0. Atomic Request Parity Error: Parity error on read of an atomic transaction.
6	SDP_PAR_ERR. Read-write. Reset: 0. Read Response Parity Error: Parity error on incoming read response data.
5	FTI_PAR_ERR. Read-write. Reset: 0. Request or Probe Parity Error: Parity error on incoming request or probe response data.
4	FTI_RSP_NO_MTCH. Read-write. Reset: 0. Unexpected Response: A response was received from the transport layer which does not match any request.
3	FTI_ILL_RSP. Read-write. Reset: 0. Illegal Response: An illegal response was received from the transport layer.
2	FTI_SEC_VIOL. Read-write. Reset: 0. Security Violation: A security violation was detected on an incoming request from the transport layer on the primary request channel.
1	FTI_ADDR_VIOL. Read-write. Reset: 0. Address Violation: An address violation was detected on an incoming request from the transport layer on the primary request channel.
0	FTI_ILL_REQ. Read-write. Reset: 0. Illegal Request: An illegal request was received from the transport layer on the primary request channel.

MSRC001_051[7...8] [CS Machine Check Transparent Error Syndrome] (MCA::CS::MCA_TRANSSYND_CS)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_instCMP0_n12_aliasMSR; MSRC001_0517_instCMP1_n13_aliasMSR; MSRC001_0518_instCMP2_n14_aliasMSR; MSRC001_0517_instCMP3_n15_aliasMSR; MSRC001_0518_instCS0_n0_aliasMSR; MSRC001_0517_instCS10_n10_aliasMSR; MSRC001_0517_instCS11_n11_aliasMSR; MSRC001_0518_instCS1_n1_aliasMSR; MSRC001_0518_instCS2_n2_aliasMSR; MSRC001_0517_instCS3_n3_aliasMSR; MSRC001_0518_instCS4_n4_aliasMSR; MSRC001_0517_instCS5_n5_aliasMSR; MSRC001_0518_instCS6_n6_aliasMSR; MSRC001_0517_instCS7_n7_aliasMSR; MSRC001_0518_instCS8_n8_aliasMSR; MSRC001_0517_instCS9_n9_aliasMSR; MSRC001_0518

Bits	Description
63:41	Reserved.
40:32	Syndrome. Read-write, Volatile. Reset: Cold, 000h. Contains the syndrome, if any, associated with the error logged in MCA::CS::MCA_STATUS_CS. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::CS::MCA_SYND_CS[Length]. The Syndrome field is only valid when MCA::CS::MCA_SYND_CS[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::CS::MCA_TRANSSYND_CS. All errors writing into MCA::CS::MCA_SYND_CS should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::CS::MCA_SYND_CS. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::CS::MCA_SYND_CS[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::CS::MCA_SYND_CS. For example, a syndrome length of 9 means that MCA::CS::MCA_SYND_CS[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

MSRC001_061[7...8] [CS Machine Check Transparent Error Address] (MCA::CS::MCA_TRANSADDR_CS)

Reset: Cold,0000_0000_0000_0000h.

MCA::CS::MCA_TRANSADDR_CS stores the address associated with the error in MCA::CS::MCA_STATUS_CS. The register is only meaningful if MCA::CS::MCA_STATUS_CS[Val]=1 and MCA::CS::MCA_STATUS_CS[AddrV]=1. The lowest valid bit of the address is defined by MCA::CS::MCA_STATUS_CS[AddrLsb].

_instCMP0_n12_aliasMSR; MSRC001_0617

_instCMP1_n13_aliasMSR; MSRC001_0618

_instCMP2_n14_aliasMSR; MSRC001_0617

_instCMP3_n15_aliasMSR; MSRC001_0618

_instCS0_n0_aliasMSR; MSRC001_0617

_instCS10_n10_aliasMSR; MSRC001_0617

_instCS11_n11_aliasMSR; MSRC001_0618

_instCS1_n1_aliasMSR; MSRC001_0618

_instCS2_n2_aliasMSR; MSRC001_0617

_instCS3_n3_aliasMSR; MSRC001_0618

_instCS4_n4_aliasMSR; MSRC001_0617

_instCS5_n5_aliasMSR; MSRC001_0618

_instCS6_n6_aliasMSR; MSRC001_0617

_instCS7_n7_aliasMSR; MSRC001_0618

_instCS8_n8_aliasMSR; MSRC001_0617

_instCS9_n9_aliasMSR; MSRC001_0618

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::CS::MCA_STATUS_CS. The lowest-order valid bit of the address is specified in MCA::CS::MCA_STATUS_CS[AddrLsb].

3.2.5.9 PIE**MSR0000_0470...MSRC000_21C0 [PIE Machine Check Control] (MCA::PIE::MCA_CTL_PIE)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::PIE::MCA_CTL_PIE register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_inst[PIE0,BCST]_n0_aliasMSR; MSR0000_21C0

_inst[PIE0,BCST]_n0_aliasMSRLEGACY; MSR0000_0470

Bits	Description
63:7	Reserved.
6	CNLI. Read-write. Reset: 0. An SRAM ECC error was detected in the CNLI block.
5	WDT. Read-write. Reset: 0. Watch Dog Timer: A watch dog timer expired. A bit in the HWASStatus will also be set.
4	DEF. Read-write. Reset: 0. A data parity error was observed in the DF and converted to poisoned data.
3	FTI_DAT_STAT. Read-write. Reset: 0. Poison data consumption: Poison data was written to an internal PIE register. MCA_SYND_PIE[ErrorInformation][3:0] specifies which data channel gave the error: 4'b{Multiple, Channel, 2'b{DatAssoc}}.
2	GMI. Read-write. Reset: 0. Link Error: An error occurred on a GMI or xGMI link.
1	CSW. Read-write. Reset: 0. Register security violation: A security violation was detected on an access to an internal PIE register.
0	HW_ASSERT. Read-write. Reset: 0. Hardware Assert: A hardware assert was detected. This error includes viral messages received from a CXL™ device. The specific hardware assert is given in registers DF:: HardwareAssertStatusLow and DF:: HardwareAssertStatusHigh .

MSR0000_0471...MSRC000_21C1 [PIE Machine Check Status] (MCA::PIE::MCA_STATUS_PIE)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C1

_inst[PIE0,BCST]_n0_aliasMSRLEGACY; MSR0000_0471

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::PIE::MCA_CTL_PIE. This bit is a copy of bit in MCA::PIE::MCA_CTL_PIE for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::PIE::MCA_MISC0_PIE. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::PIE::MCA_ADDR_PIE contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::PIE::MCA_STATUS_PIE[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::PIE::MCA_SYND_PIE. If MCA::PIE::MCA_SYND_PIE[ErrorPriority] is the same as the priority of the error in MCA::PIE::MCA_STATUS_PIE, then the information in MCA::PIE::MCA_SYND_PIE is associated with the error in MCA::PIE::MCA_STATUS_PIE. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::PIE::MCA_STATUS_PIE[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PIE::MCA_ADDR_PIE[ErrorAddr]. A value of 0 indicates that MCA::PIE::MCA_ADDR_PIE[55:0] contains a valid byte address. A value of 6 indicates that MCA::PIE::MCA_ADDR_PIE[55:6] contains a valid cache line address and that MCA::PIE::MCA_ADDR_PIE[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PIE::MCA_ADDR_PIE[55:12] contain a valid 4KB memory page and that MCA::PIE::MCA_ADDR_PIE[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::PIE::MCA_CTL_PIE enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 82: MCA_STATUS_PIE

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
HW_ASSER T	0x0	1	1	1	0	0	0
CSW	0x1	0	0	0	1	0	0
GMI	0x2	0/1	0/1	0/1	0	0	0
FTI_DAT_S TAT	0x3	1	1	1	0	0	0
DEF	0x4	0	0	0	1	0	0
WDT	0x5	1	1	1	0	0	0/1
CNLI	0x6	0/1	0/1	0/1	0	0	0

SVI3FatalError	0x1F	1	1	1	0	0	0
----------------	------	---	---	---	---	---	---

MSR0000_0472...MSRC000_21C2 [PIE Machine Check Address] (MCA::PIE::MCA_ADDR_PIE)

Reset: Cold,0000_0000_0000_0000h.

MCA::PIE::MCA_ADDR_PIE stores an address and other information associated with the error in MCA::PIE::MCA_STATUS_PIE. The register is only meaningful if MCA::PIE::MCA_STATUS_PIE[Val]=1 and MCA::PIE::MCA_STATUS_PIE[AddrV]=1.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C2

_inst[PIE0,BCST]_n0_aliasMSRLEGACY; MSR0000_0472

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::PIE::MCA_STATUS_PIE. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 83: MCA_ADDR_PIE

Error Type	Bits	Description
HW_ASSERT	[56:0]	Reserved
CSW	[56:0]	Reserved
GMI	[56:0]	Reserved
FTI_DAT_STAT	[56:0]	Reserved
DEF	[56:0]	Reserved
WDT	[55:5]	Request address[MSB:5] for transaction triggering WDT, Valid when AddrV is set in MCA_STATUS
CNLI	[56:0]	Reserved

MSR0000_0473...MSRC000_21C3 [PIE Machine Check Miscellaneous 0] (MCA::PIE::MCA_MISC0_PIE)

Log miscellaneous information associated with errors.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C3

_inst[PIE0,BCST]_n0_aliasMSRLEGACY; MSR0000_0473

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PIE::MCA_MISC0_PIE[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PIE::MCA_MISC0_PIE[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PIE::MCA_MISC0_PIE[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PIE::MCA_MISC0_PIE[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PIE::MCA_MISC0_PIE[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PIE::MCA_MISC0_PIE[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21C4 [PIE Machine Check Configuration] (MCA::PIE::MCA_CONFIG_PIE)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat. Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::PIE::MCA_STATUS_PIE and MCA::PIE::MCA_ADDR_PIE in addition to MCA::PIE::MCA_DESTAT_PIE and MCA::PIE::MCA_DEADDR_PIE. 0=Only log deferred errors in MCA::PIE::MCA_DESTAT_PIE and MCA::PIE::MCA_DEADDR_PIE. This bit does not affect logging of deferred errors in MCA::PIE::MCA_SYND_PIE, MCA::PIE::MCA_MISC0_PIE.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::PIE::MCA_STATUS_PIE, MCA::PIE::MCA_ADDR_PIE, MCA::PIE::MCA_MISC0_PIE, and MCA::PIE::MCA_SYND_PIE.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::PIE::MCA_CONFIG_PIE[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::PIE::MCA_CONFIG_PIE[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::PIE::MCA_CONFIG_PIE[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::PIE::MCA_CONFIG_PIE[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::PIE::MCA_DESTAT_PIE and MCA::PIE::MCA_DEADDR_PIE are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::PIE::MCA_MISC0_PIE[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::PIE::MCA_STATUS_PIE[TCC] is present.

MSRC000_21C5 [PIE IP Identification] (MCA::PIE::MCA_IPID_PIE)

Reset: 0001_002E_0000_0000h.

The MCA::PIE::MCA_IPID_PIE register is used by software to determine what IP type and revision is associated with the MCA bank.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C5

Bits	Description
63:48	McaType. Read-only. Reset: 0001h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 02Eh. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. Init: 1F00_2C00h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.

MSRC000_21C6 [PIE Machine Check Syndrome] (MCA::PIE::MCA_SYND_PIE)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::PIE::MCA_STATUS_PIE [Thread 0](#)

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C6

Bits	Description
63:46	Reserved.
45:32	Syndrom. Read-write, Volatile. Reset: Cold, 0000h. Contains the syndrome, if any, associated with the error logged in MCA::PIE::MCA_STATUS_PIE. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PIE::MCA_SYND_PIE[Length]. The Syndrome field is only valid when MCA::PIE::MCA_SYND_PIE[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PIE::MCA_SYND_PIE. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PIE::MCA_SYND_PIE[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PIE::MCA_SYND_PIE. For example, a syndrome length of 9 means that MCA::PIE::MCA_SYND_PIE[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 84 [MCA_SYND_PIE].

Table 84: MCA_SYND_PIE

Error Type	Bits	Description
HW_ASSERT	[17:16]	Reserved
	[15:8]	Block Instance ID. For Hardware Assert ID 19, IDs 53-56 indicate CXL Link 0-3.
	[7:6]	Reserved
	[5:0]	Hardware Assert ID. ID 19 indicates a viral message received from a CXL device.
CSW	[12:0]	Register Address[14:2]
	[16:13]	Trust Level
	[17:17]	Valid
GMI	[17:17]	Link Type = {1=error was recognized on a CCD to IOD GMI link; 0=error was recognized on an IOD to IOD xGMI link}
	[16:2]	{Specifies the zero-origin link number. When GmiLinkType=1, GmiLinkNumber is the CCD number. When GmiLinkType=0, GmiLinkNumber is the Link Number.}

	[1:0]	Error type = {01b => Correctable; 10b => Uncorrectable with linkError=0; 10b => Uncorrectable with linkError=1}
FTI_DAT_STAT	[3:0]	Data channel in error: 4'b{Multiple, Channel, 2'b{DatAssoc}}.
DEF	[17:0]	Reserved
WDT	[17:5] [3:1] [0:0]	Request Destination block fabric ID or External Link ID WDT Type: 000b = CCM REQQ, 010b = PIE, 011b = Any stateless watchdogs, 100b = CCM RSPQ, 101b = IOM RSPQ, 101b = External link (CXL), Other= Reserved Set if the error address is to a NCS/IOS
CNLI	[17:8] [7:6] [5:4] [3:3] [2:0]	Specifies the zero-origin CXL link number Reserved SRAM Instance Id 00b = WKS (write AES key store), 01b = RKS (read AES key store), 10b = XKS (XTS Start key store) Other= Reserved Reserved Error Type: 111b = Correctable ECC

MSRC000_21C8 [PIE Machine Check Deferred Error Status] (MCA::PIE::MCA_DESTAT_PIE)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C8

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::PIE::MCA_DEADDR_PIE contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::PIE::MCA_SYND_PIE. If MCA::PIE::MCA_SYND_PIE[ErrorPriority] is the same as the priority of the error in MCA::PIE::MCA_STATUS_PIE, then the information in MCA::PIE::MCA_SYND_PIE is associated with the error in MCA::PIE::MCA_DESTAT_PIE.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PIE::MCA_ADDR_PIE[ErrorAddr]. A value of 0 indicates that MCA::PIE::MCA_ADDR_PIE[55:0] contains a valid byte address. A value of 6 indicates that MCA::PIE::MCA_ADDR_PIE[55:6] contains a valid cache line address and that MCA::PIE::MCA_ADDR_PIE[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PIE::MCA_ADDR_PIE[55:12] contain a valid 4KB memory page and that MCA::PIE::MCA_ADDR_PIE[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21C9 [PIE Deferred Error Address] (MCA::PIE::MCA_DEADDR_PIE)

Reset: Cold,0000_0000_0000_0000h.

The MCA::PIE::MCA_DEADDR_PIE register stores the address associated with the error in MCA::PIE::MCA_DESTAT_PIE. The register is only meaningful if MCA::PIE::MCA_DESTAT_PIE[Val]=1 and MCA::PIE::MCA_DESTAT_PIE[AddrV]=1. The lowest valid bit of the address is defined by MCA::PIE::MCA_DESTAT_PIE[AddrLsb].

_inst[PIE0,BCST]_n0_aliasMSR; MSRC000_21C9

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PIE::MCA_DESTAT_PIE. The lowest-order valid bit of the address is specified in MCA::PIE::MCA_DESTAT_PIE[AddrLsb].

MSRC001_041C [PIE Machine Check Control Mask] (MCA::PIE::MCA_CTL_MASK_PIE)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_inst[PIE0,BCST]_n0_aliasMSR; MSRC001_041C

Bits	Description
63:7	Reserved.
6	CNLI. Read-write. Reset: 0. An SRAM ECC error was detected in the CNLI block.
5	WDT. Read-write. Reset: 0. Watch Dog Timer: A watch dog timer expired. A bit in the HWASStatus will also be set.
4	DEF. Read-write. Reset: 0. A data parity error was observed in the DF and converted to poisoned data.
3	FTI_DAT_STAT. Read-write. Reset: 0. Poison data consumption: Poison data was written to an internal PIE register. MCA_SYND_PIE[ErrorInformation][3:0] specifies which data channel gave the error: 4'b{Multiple, Channel, 2'b{DatAssoc}}.
2	GMI. Read-write. Reset: 0. Link Error: An error occurred on a GMI or xGMI link.
1	CSW. Read-write. Reset: 0. Register security violation: A security violation was detected on an access to an internal PIE register.
0	HW_ASSERT. Read-write. Reset: 0. Hardware Assert: A hardware assert was detected. This error includes viral messages received from a CXL device. The specific hardware assert is given in registers DF:: HardwareAssertStatusLow and DF:: HardwareAssertStatusHigh .

3.2.5.10 UMC

MSR0000_0454...MSRC000_2160 [UMC Machine Check Control] (MCA::UMC::MCA_CTL_UMC)	
Read-write. Reset: 0000_0000_0000_0000h.	
0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::UMC::MCA_CTL_UMC register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.	
_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC000_2150	
_instUMCWPHY0UMC_n0_umc0_aliasMSRLEGACY; MSR0000_0454	
_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC000_2150	
_instUMCWPHY10UMC_n10_umc0_aliasMSRLEGACY; MSR0000_0454	
_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC000_2160	
_instUMCWPHY11UMC_n11_umc0_aliasMSRLEGACY; MSR0000_0458	
_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC000_2160	
_instUMCWPHY1UMC_n1_umc0_aliasMSRLEGACY; MSR0000_0458	
_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC000_2150	
_instUMCWPHY2UMC_n2_umc0_aliasMSRLEGACY; MSR0000_0454	
_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC000_2160	
_instUMCWPHY3UMC_n3_umc0_aliasMSRLEGACY; MSR0000_0458	
_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC000_2150	
_instUMCWPHY4UMC_n4_umc0_aliasMSRLEGACY; MSR0000_0454	
_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC000_2160	
_instUMCWPHY5UMC_n5_umc0_aliasMSRLEGACY; MSR0000_0458	
_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC000_2150	
_instUMCWPHY6UMC_n6_umc0_aliasMSRLEGACY; MSR0000_0454	
_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC000_2160	
_instUMCWPHY7UMC_n7_umc0_aliasMSRLEGACY; MSR0000_0458	
_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC000_2150	
_instUMCWPHY8UMC_n8_umc0_aliasMSRLEGACY; MSR0000_0454	
_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC000_2160	
_instUMCWPHY9UMC_n9_umc0_aliasMSRLEGACY; MSR0000_0458	
Bits	Description
63:12	Reserved.
11	RdCrcErr. Read-write. Reset: 0. Read CRC error. CRC error occurred on a DRAM read from any subchannel
10	Reserved.
9	EcsErr. Read-write. Reset: 0. ECS Error. Indicates that a device exceeded the ECS Error Threshold Count.
8	EcsRowErr. Read-write. Reset: 0. ECS Row Error. Indicates that a single device row exceeded four code word errors.
7	AesSramEccErr. Read-write. Reset: 0. AES SRAM ECC error. An ECC error occurred on a AES SRAM in the processor.
6	DcqSramEccErr. Read-write. Reset: 0. DCQ SRAM ECC error. An ECC error occurred on a DCQ SRAM in the processor.
5	WriteDataCrcErr. Read-write. Reset: 0. Write data CRC error. A write data CRC error occurred on the DRAM data bus.
4	AddressCommandParityErr. Read-write. Reset: 0. Address/Command parity error. A parity error occurred on the DRAM address/command bus.
3	ApbErr. Read-write. Reset: 0. Advanced peripheral bus error. An error occurred on the advanced peripheral bus.
2	SdpParityErr. Read-write. Reset: 0. SDP parity error. A parity error was detected on write data from the data fabric in the processor.
1	WriteDataPoisonErr. Read-write. Reset: 0. Data poison error. The system tried to write poison data to DRAM and either DRAM does not support ECC or UMC_CH.EccCtrl.WrEccEn is cleared.
0	DramEccErr. Read-write. Reset: 0. DRAM ECC error. An ECC error occurred on a DRAM read.

MSR0000_0455...MSRC000_2161 [UMC Machine Check Status] (MCA::UMC::MCA_STATUS_UMC)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC000_2151

_instUMCWPHY0UMC_n0_umc0_aliasMSRLEGACY; MSR0000_0455

_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC000_2151

_instUMCWPHY10UMC_n10_umc0_aliasMSRLEGACY; MSR0000_0455

_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC000_2161

_instUMCWPHY11UMC_n11_umc0_aliasMSRLEGACY; MSR0000_0459

_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC000_2161

_instUMCWPHY1UMC_n1_umc0_aliasMSRLEGACY; MSR0000_0459

_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC000_2151

_instUMCWPHY2UMC_n2_umc0_aliasMSRLEGACY; MSR0000_0455

_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC000_2161

_instUMCWPHY3UMC_n3_umc0_aliasMSRLEGACY; MSR0000_0459

_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC000_2151

_instUMCWPHY4UMC_n4_umc0_aliasMSRLEGACY; MSR0000_0455

_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC000_2161

_instUMCWPHY5UMC_n5_umc0_aliasMSRLEGACY; MSR0000_0459

_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC000_2151

_instUMCWPHY6UMC_n6_umc0_aliasMSRLEGACY; MSR0000_0455

_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC000_2161

_instUMCWPHY7UMC_n7_umc0_aliasMSRLEGACY; MSR0000_0459

_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC000_2151

_instUMCWPHY8UMC_n8_umc0_aliasMSRLEGACY; MSR0000_0455

_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC000_2161

_instUMCWPHY9UMC_n9_umc0_aliasMSRLEGACY; MSR0000_0459

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::UMC::MCA_CTL_UMC. This bit is a copy of bit in MCA::UMC::MCA_CTL_UMC for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::UMC::MCA_MISC0_UMC. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::UMC::MCA_ADDR_UMC contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::UMC::MCA_STATUS_UMC[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::UMC::MCA_SYND_UMC. If MCA::UMC::MCA_SYND_UMC[ErrorPriority] is the same as the priority of the error in MCA::UMC::MCA_STATUS_UMC, then the information in MCA::UMC::MCA_SYND_UMC is associated with the error in MCA::UMC::MCA_STATUS_UMC. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::UMC::MCA_STATUS_UMC[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::UMC::MCA_ADDR_UMC[ErrorAddr]. A value of 0 indicates that MCA::UMC::MCA_ADDR_UMC[55:0] contains a valid byte address. A value of 6 indicates that MCA::UMC::MCA_ADDR_UMC[55:6] contains a valid cache line address and that MCA::UMC::MCA_ADDR_UMC[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::UMC::MCA_ADDR_UMC[55:12] contain a valid 4KB memory page and that MCA::UMC::MCA_ADDR_UMC[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::UMC::MCA_CTL_UMC enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 85: MCA_STATUS_UMC

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
DramEccErr	0x0	0/1	0/1	0/1	0/1	0	1
WriteDataPoisonErr	0x1	1	1	1	0	0	0
SdpParityErr	0x2	1	1	1	0	0	0
ApbErr	0x3	1	1	1	0	0	1
AddressCommandParityErr	0x4	0/1	0/1	0/1	0	0	0/1
WriteDataCrucErr	0x5	0/1	0/1	0/1	0	0	0/1
DcqSramEccErr	0x6	0/1	0/1	0/1	0	0	0
AesSramEccErr	0x7	0/1	0/1	0/1	0	0	0
EcsRowErr	0x8	0	0	0	0	0	0
EcsErr	0x9	0	0	0	0	0	0
RdCrcErr	0xb	0	0	0	1	0	1

MSR0000_0456...MSRC000_2162 [UMC Machine Check Address] (MCA::UMC::MCA_ADDR_UMC)

Reset: Cold,0000_0000_0000_0000h.

MCA::UMC::MCA_ADDR_UMC stores an address and other information associated with the error in MCA::UMC::MCA_STATUS_UMC. The register is only meaningful if MCA::UMC::MCA_STATUS_UMC[Val]=1 and MCA::UMC::MCA_STATUS_UMC[AddrV]=1.

_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC000_2152_instUMCWPHY0UMC_n0_umc0_aliasMSRLEGACY; MSR0000_0456_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC000_2152_instUMCWPHY10UMC_n10_umc0_aliasMSRLEGACY; MSR0000_0456_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC000_2162_instUMCWPHY11UMC_n11_umc0_aliasMSRLEGACY; MSR0000_045A_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC000_2162_instUMCWPHY1UMC_n1_umc0_aliasMSRLEGACY; MSR0000_045A_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC000_2152_instUMCWPHY2UMC_n2_umc0_aliasMSRLEGACY; MSR0000_0456_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC000_2162_instUMCWPHY3UMC_n3_umc0_aliasMSRLEGACY; MSR0000_045A_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC000_2152_instUMCWPHY4UMC_n4_umc0_aliasMSRLEGACY; MSR0000_0456_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC000_2162_instUMCWPHY5UMC_n5_umc0_aliasMSRLEGACY; MSR0000_045A_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC000_2152_instUMCWPHY6UMC_n6_umc0_aliasMSRLEGACY; MSR0000_0456_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC000_2162_instUMCWPHY7UMC_n7_umc0_aliasMSRLEGACY; MSR0000_045A_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC000_2152_instUMCWPHY8UMC_n8_umc0_aliasMSRLEGACY; MSR0000_0456_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC000_2162_instUMCWPHY9UMC_n9_umc0_aliasMSRLEGACY; MSR0000_045A**Bits Description**

63:56 Reserved.

55:0 **ErrorAddr.** Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::UMC::MCA_STATUS_UMC. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 86: MCA_ADDR_UMC

Error Type	Bits	Description
DramEccErr	[55:39] [39:4]	Reserved Normalized address
WriteDataPoisonErr	[55:0]	Reserved
SdpParityErr	[55:0]	Reserved
ApbErr	[55:30] [29:0]	Reserved APB address associated with the error
AddressCommandParityErr	[55:42] [41] [40] [39:0]	Reserved 0 = SubChannel A. 1 = SubChannel B 0 = DIMM0. 1 = DIMM1. RCD error log control words 24:20
WriteDataCrcErr	[55:38] [37:36] [35:32] [31:0]	Reserved Reserved Chip Select Reserved
DcqSramEccErr	[55:0]	Reserved
AesSramEccErr	[55:0]	Reserved
EcsRowErr	[55:23] [22:20]	Reserved DDR5 BG[2:0]

	[19:18] [17:0]	DDR5 BA[1:0] DDR5 ROW The address information is derived from MR18, MR17, MR16.
EcsErr	[63:0]	Reserved
RdCrcErr	[55:39] [39:4]	Reserved Normalized address

MSR0000_0457...MSRC000_2163 [UMC Machine Check Miscellaneous 0] (MCA::UMC::MCA_MISC0_UMC)

Log miscellaneous information associated with errors.

_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC000_2153

_instUMCWPHY0UMC_n0_umc0_aliasMSRLEGACY; MSR0000_0457

_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC000_2153

_instUMCWPHY10UMC_n10_umc0_aliasMSRLEGACY; MSR0000_0457

_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC000_2163

_instUMCWPHY11UMC_n11_umc0_aliasMSRLEGACY; MSR0000_045B

_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC000_2163

_instUMCWPHY1UMC_n1_umc0_aliasMSRLEGACY; MSR0000_045B

_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC000_2153

_instUMCWPHY2UMC_n2_umc0_aliasMSRLEGACY; MSR0000_0457

_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC000_2163

_instUMCWPHY3UMC_n3_umc0_aliasMSRLEGACY; MSR0000_045B

_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC000_2153

_instUMCWPHY4UMC_n4_umc0_aliasMSRLEGACY; MSR0000_0457

_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC000_2163

_instUMCWPHY5UMC_n5_umc0_aliasMSRLEGACY; MSR0000_045B

_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC000_2153

_instUMCWPHY6UMC_n6_umc0_aliasMSRLEGACY; MSR0000_0457

_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC000_2163

_instUMCWPHY7UMC_n7_umc0_aliasMSRLEGACY; MSR0000_045B

_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC000_2153

_instUMCWPHY8UMC_n8_umc0_aliasMSRLEGACY; MSR0000_0457

_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC000_2163

_instUMCWPHY9UMC_n9_umc0_aliasMSRLEGACY; MSR0000_045B

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC0_UMC[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC0_UMC[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC0_UMC[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC0_UMC[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated.

	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC0_UMC[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported.
	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC0_UMC[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 01h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21[5...6]4 [UMC Machine Check Configuration] (MCA::UMC::MCA_CONFIG_UMC)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

[_instUMCWPHY0UMC_n0_umc0_aliasMSR](#); MSRC000_2154[_instUMCWPHY10UMC_n10_umc0_aliasMSR](#); MSRC000_2154[_instUMCWPHY11UMC_n11_umc0_aliasMSR](#); MSRC000_2164[_instUMCWPHY1UMC_n1_umc0_aliasMSR](#); MSRC000_2164[_instUMCWPHY2UMC_n2_umc0_aliasMSR](#); MSRC000_2154[_instUMCWPHY3UMC_n3_umc0_aliasMSR](#); MSRC000_2164[_instUMCWPHY4UMC_n4_umc0_aliasMSR](#); MSRC000_2154[_instUMCWPHY5UMC_n5_umc0_aliasMSR](#); MSRC000_2164[_instUMCWPHY6UMC_n6_umc0_aliasMSR](#); MSRC000_2154[_instUMCWPHY7UMC_n7_umc0_aliasMSR](#); MSRC000_2164[_instUMCWPHY8UMC_n8_umc0_aliasMSR](#); MSRC000_2154[_instUMCWPHY9UMC_n9_umc0_aliasMSR](#); MSRC000_2164

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::UMC::MCA_STATUS_UMC and MCA::UMC::MCA_ADDR_UMC in addition to MCA::UMC::MCA_DESTAT_UMC and MCA::UMC::MCA_DEADDR_UMC. 0=Only log deferred errors in MCA::UMC::MCA_DESTAT_UMC and MCA::UMC::MCA_DEADDR_UMC. This bit does not affect logging of deferred errors in MCA::UMC::MCA_SYND_UMC, MCA::UMC::MCA_MISC0_UMC.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::UMC::MCA_STATUS_UMC, MCA::UMC::MCA_ADDR_UMC, MCA::UMC::MCA_MISC0_UMC, and MCA::UMC::MCA_SYND_UMC.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:10	Reserved.
9	McaFruTextInMca . Read-write. Reset: 0. Init: BIOS,1. 1=FruText is reported McaSynd1/McaSynd2 registers
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::UMC::MCA_CONFIG_UMC[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::UMC::MCA_CONFIG_UMC[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::UMC::MCA_CONFIG_UMC[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::UMC::MCA_CONFIG_UMC[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::UMC::MCA_DESTAT_UMC and MCA::UMC::MCA_DEADDR_UMC are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.

0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::UMC::MCA_MISC0_UMC[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::UMC::MCA_STATUS_UMC[TCC] is present.
---	---

MSRC000_21[5...6]5 [UMC IP Identification] (MCA::UMC::MCA_IPID_UMC)

Reset: 0000_0096_0000_0000h.

The MCA::UMC::MCA_IPID_UMC register is used by software to determine what IP type and revision is associated with the MCA bank.

[_instUMCWPHY0UMC_n0_umc0_aliasMSR](#); MSRC000_2155[_instUMCWPHY10UMC_n10_umc0_aliasMSR](#); MSRC000_2155[_instUMCWPHY11UMC_n11_umc0_aliasMSR](#); MSRC000_2165[_instUMCWPHY1UMC_n1_umc0_aliasMSR](#); MSRC000_2165[_instUMCWPHY2UMC_n2_umc0_aliasMSR](#); MSRC000_2155[_instUMCWPHY3UMC_n3_umc0_aliasMSR](#); MSRC000_2165[_instUMCWPHY4UMC_n4_umc0_aliasMSR](#); MSRC000_2155[_instUMCWPHY5UMC_n5_umc0_aliasMSR](#); MSRC000_2165[_instUMCWPHY6UMC_n6_umc0_aliasMSR](#); MSRC000_2155[_instUMCWPHY7UMC_n7_umc0_aliasMSR](#); MSRC000_2165[_instUMCWPHY8UMC_n8_umc0_aliasMSR](#); MSRC000_2155[_instUMCWPHY9UMC_n9_umc0_aliasMSR](#); MSRC000_2165

Bits	Description
63:48	McaType. Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 096h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instUMCWPHY0UMC_n0_umc0_aliasMSR : 0005_0F00h
	Init: _instUMCWPHY10UMC_n10_umc0_aliasMSR : 00A5_0F00h
	Init: _instUMCWPHY11UMC_n11_umc0_aliasMSR : 00B5_0F00h
	Init: _instUMCWPHY1UMC_n1_umc0_aliasMSR : 0015_0F00h
	Init: _instUMCWPHY2UMC_n2_umc0_aliasMSR : 0025_0F00h
	Init: _instUMCWPHY3UMC_n3_umc0_aliasMSR : 0035_0F00h
	Init: _instUMCWPHY4UMC_n4_umc0_aliasMSR : 0045_0F00h
	Init: _instUMCWPHY5UMC_n5_umc0_aliasMSR : 0055_0F00h
	Init: _instUMCWPHY6UMC_n6_umc0_aliasMSR : 0065_0F00h
	Init: _instUMCWPHY7UMC_n7_umc0_aliasMSR : 0075_0F00h
	Init: _instUMCWPHY8UMC_n8_umc0_aliasMSR : 0085_0F00h
	Init: _instUMCWPHY9UMC_n9_umc0_aliasMSR : 0095_0F00h

MSRC000_21[5...6]6 [UMC Machine Check Syndrome] (MCA::UMC::MCA_SYND_UMC)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::UMC::MCA_STATUS_UMC [Thread 0](#)[_instUMCWPHY0UMC_n0_umc0_aliasMSR](#); MSRC000_2156[_instUMCWPHY10UMC_n10_umc0_aliasMSR](#); MSRC000_2156[_instUMCWPHY11UMC_n11_umc0_aliasMSR](#); MSRC000_2166[_instUMCWPHY1UMC_n1_umc0_aliasMSR](#); MSRC000_2166[_instUMCWPHY2UMC_n2_umc0_aliasMSR](#); MSRC000_2156[_instUMCWPHY3UMC_n3_umc0_aliasMSR](#); MSRC000_2166[_instUMCWPHY4UMC_n4_umc0_aliasMSR](#); MSRC000_2156[_instUMCWPHY5UMC_n5_umc0_aliasMSR](#); MSRC000_2166[_instUMCWPHY6UMC_n6_umc0_aliasMSR](#); MSRC000_2156[_instUMCWPHY7UMC_n7_umc0_aliasMSR](#); MSRC000_2166[_instUMCWPHY8UMC_n8_umc0_aliasMSR](#); MSRC000_2156[_instUMCWPHY9UMC_n9_umc0_aliasMSR](#); MSRC000_2166

Bits	Description
63:32	Syndromes. Read-write, Volatile. Reset: Cold, 0000_0000h. Contains the syndrome, if any, associated with the error logged in MCA::UMC::MCA_STATUS_UMC. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::UMC::MCA_SYND_UMC[Length]. The Syndrome field is only valid when MCA::UMC::MCA_SYND_UMC[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::UMC::MCA_SYND_UMC. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::UMC::MCA_SYND_UMC[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::UMC::MCA_SYND_UMC. For example, a syndrome length of 9 means that MCA::UMC::MCA_SYND_UMC[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 87 [MCA_SYND_UMC].

Table 87: MCA_SYND_UMC

Error Type	Bits	Description
DramEccErr	[17:16] [15] [14] [13:8] [7:4] [3] [2:0]	Reserved Software-Managed Bad Symbol ID Error Hardware history error Symbol. Only contains valid information on a corrected error. Cid. Specifies the rank multiply ID for supported DIMMs. Sub channel Chip Select
WriteDataPoisonErr	[17:0]	Reserved
SdpParityErr	[17:0]	Reserved
ApbErr	[17:0]	Reserved
AddressCommandParityErr	[17:0]	Reserved
WriteDataCrcErr	[17:0]	Reserved
DcqSramEccErr	[17:14] [13:0]	Reserved Reserved
AesSramEccErr	[17] [16:8] [7:4] [3:2] [1:0]	Reserved Reserved Reserved Reserved Reserved

EcsRowErr	[17:12] [11:9] [11:8] [7:4] [3] [2:0]	Reserved DevID of x8 Dram with highest Error count DevID of x4 Dram with highest Error Count CID (Specifies the rank multiply ID for supported DIMMs.) Subchannel (0=A, 1=B) Chip Select The address information is derived from MR18, MR17, MR16.
EcsErr	[17:12] [11:9] [11:8] [7:4] [3] [2:0]	Reserved DevID of x8 Dram with highest Error count DevID of x4 Dram with highest Error Count CID (Specifies the rank multiply ID for supported DIMMs.) Subchannel (0=A, 1=B) Chip Select Note: Address information is valid only if EcsRowErr is set. The DIMM latches a valid row address only when MR19 is non-zero. If EcsRowErr is not set, the SYND address is not valid.
RdCrcErr	[17:8] [7:4] [3] [2:0]	Reserved Cid. Specifies the rank multiply ID for supported DIMMs. Sub channel Chip Select

MSRC000_21[5...6]E [UMC Machine Check Syndrome Extended] (MCA::UMC::MCA_SYND1_UMC)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::UMC::MCA_STATUS_UMC [Thread 0](#)[_instUMCWPHY0UMC_n0_umc0_aliasMSR](#); MSRC000_215E[_instUMCWPHY10UMC_n10_umc0_aliasMSR](#); MSRC000_215E[_instUMCWPHY11UMC_n11_umc0_aliasMSR](#); MSRC000_216E[_instUMCWPHY1UMC_n1_umc0_aliasMSR](#); MSRC000_216E[_instUMCWPHY2UMC_n2_umc0_aliasMSR](#); MSRC000_215E[_instUMCWPHY3UMC_n3_umc0_aliasMSR](#); MSRC000_216E[_instUMCWPHY4UMC_n4_umc0_aliasMSR](#); MSRC000_215E[_instUMCWPHY5UMC_n5_umc0_aliasMSR](#); MSRC000_216E[_instUMCWPHY6UMC_n6_umc0_aliasMSR](#); MSRC000_215E[_instUMCWPHY7UMC_n7_umc0_aliasMSR](#); MSRC000_216E[_instUMCWPHY8UMC_n8_umc0_aliasMSR](#); MSRC000_215E[_instUMCWPHY9UMC_n9_umc0_aliasMSR](#); MSRC000_216E

Bits	Description
63:0	Syndrom . Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h. The MCA::UMC::MCA_SYND1_UMC register stores information associated with the error in MCA::UMC::MCA_STATUS_UMC or MCA_DESTAT. The register is meaningful if MCA::UMC::MCA_STATUS_UMC[SyndV]=1. When MCA::UMC::MCA_CONFIG_UMC[McaFruTextInMca]=1, MCA::UMC::MCA_SYND1_UMC stores ASCII FruText associated with the error.

MSRC000_21[5...6]F [UMC Machine Check Syndrome Extended] (MCA::UMC::MCA_SYND2_UMC)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::UMC::MCA_STATUS_UMC [Thread 0](#)[_instUMCWPHY0UMC_n0_umc0_aliasMSR](#); MSRC000_215F[_instUMCWPHY10UMC_n10_umc0_aliasMSR](#); MSRC000_215F[_instUMCWPHY11UMC_n11_umc0_aliasMSR](#); MSRC000_216F[_instUMCWPHY1UMC_n1_umc0_aliasMSR](#); MSRC000_216F[_instUMCWPHY2UMC_n2_umc0_aliasMSR](#); MSRC000_215F[_instUMCWPHY3UMC_n3_umc0_aliasMSR](#); MSRC000_216F[_instUMCWPHY4UMC_n4_umc0_aliasMSR](#); MSRC000_215F[_instUMCWPHY5UMC_n5_umc0_aliasMSR](#); MSRC000_216F[_instUMCWPHY6UMC_n6_umc0_aliasMSR](#); MSRC000_215F[_instUMCWPHY7UMC_n7_umc0_aliasMSR](#); MSRC000_216F[_instUMCWPHY8UMC_n8_umc0_aliasMSR](#); MSRC000_215F[_instUMCWPHY9UMC_n9_umc0_aliasMSR](#); MSRC000_216F

Bits	Description
63:0	Syndrome. Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h. The MCA::UMC::MCA_SYND2_UMC register stores information associated with the error in MCA::UMC::MCA_STATUS_UMC or MCA_DESTAT. The register is meaningful if MCA::UMC::MCA_STATUS_UMC[SyndV]=1. When MCA::UMC::MCA_CONFIG_UMC[McaFruTextInMca]=1, MCA::UMC::MCA_SYND2_UMC stores ASCII FruText associated with the error.

MSRC000_21[5...6]8 [UMC Machine Check Deferred Error Status] (MCA::UMC::MCA_DESTAT_UMC)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC000_2158_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC000_2158_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC000_2168_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC000_2168_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC000_2158_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC000_2168_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC000_2158_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC000_2168_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC000_2158_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC000_2168_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC000_2158_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC000_2168**Bits Description**

63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::UMC::MCA_DEADDR_UMC contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::UMC::MCA_SYND_UMC. If MCA::UMC::MCA_SYND_UMC[ErrorPriority] is the same as the priority of the error in MCA::UMC::MCA_STATUS_UMC, then the information in MCA::UMC::MCA_SYND_UMC is associated with the error in MCA::UMC::MCA_DESTAT_UMC.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::UMC::MCA_ADDR_UMC[ErrorAddr]. A value of 0 indicates that MCA::UMC::MCA_ADDR_UMC[55:0] contains a valid byte address. A value of 6 indicates that MCA::UMC::MCA_ADDR_UMC[55:6] contains a valid cache line address and that MCA::UMC::MCA_ADDR_UMC[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::UMC::MCA_ADDR_UMC[55:12] contain a valid 4KB memory page and that MCA::UMC::MCA_ADDR_UMC[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21[5...6]9 [UMC Deferred Error Address] (MCA::UMC::MCA_DEADDR_UMC)

Reset: Cold,0000_0000_0000_0000h.

The MCA::UMC::MCA_DEADDR_UMC register stores the address associated with the error in MCA::UMC::MCA_DESTAT_UMC. The register is only meaningful if MCA::UMC::MCA_DESTAT_UMC[Val]=1 and MCA::UMC::MCA_DESTAT_UMC[AddrV]=1. The lowest valid bit of the address is defined by MCA::UMC::MCA_DESTAT_UMC[AddrLsb].

_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC000_2159

_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC000_2159

_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC000_2169

_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC000_2169

_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC000_2159

_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC000_2169

_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC000_2159

_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC000_2169

_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC000_2159

_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC000_2169

_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC000_2159

_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC000_2169

Bits	Description
------	-------------

63:56	Reserved.
-------	-----------

55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::UMC::MCA_DESTAT_UMC. The lowest-order valid bit of the address is specified in MCA::UMC::MCA_DESTAT_UMC[AddrLsb].
------	--

MSRC000_21[5...6]A [UMC Machine Check Miscellaneous 1] (MCA::UMC::MCA_MISC1_UMC)

Log miscellaneous information associated with errors, as defined by each error type.

[_instUMCWPHY0UMC_n0_umc0_aliasMSR](#); MSRC000_215A[_instUMCWPHY10UMC_n10_umc0_aliasMSR](#); MSRC000_215A[_instUMCWPHY11UMC_n11_umc0_aliasMSR](#); MSRC000_216A[_instUMCWPHY1UMC_n1_umc0_aliasMSR](#); MSRC000_216A[_instUMCWPHY2UMC_n2_umc0_aliasMSR](#); MSRC000_215A[_instUMCWPHY3UMC_n3_umc0_aliasMSR](#); MSRC000_216A[_instUMCWPHY4UMC_n4_umc0_aliasMSR](#); MSRC000_215A[_instUMCWPHY5UMC_n5_umc0_aliasMSR](#); MSRC000_216A[_instUMCWPHY6UMC_n6_umc0_aliasMSR](#); MSRC000_215A[_instUMCWPHY7UMC_n7_umc0_aliasMSR](#); MSRC000_216A[_instUMCWPHY8UMC_n8_umc0_aliasMSR](#); MSRC000_215A[_instUMCWPHY9UMC_n9_umc0_aliasMSR](#); MSRC000_216A

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC1_UMC[Locked]) ? Read-write : Read-only.
59:52	Reserved.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC1_UMC[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]) to all cores. 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC1_UMC[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh; also set by hardware if ErrCnt is initialized to FFFh and transitions from FFFh to 000h. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC1_UMC[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::UMC::MCA_MISC1_UMC[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 01h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC001_041[5...6] [UMC Machine Check Control Mask] (MCA::UMC::MCA_CTL_MASK_UMC)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instUMCWPHY0UMC_n0_umc0_aliasMSR; MSRC001_0415_instUMCWPHY10UMC_n10_umc0_aliasMSR; MSRC001_0415_instUMCWPHY11UMC_n11_umc0_aliasMSR; MSRC001_0416_instUMCWPHY1UMC_n1_umc0_aliasMSR; MSRC001_0416_instUMCWPHY2UMC_n2_umc0_aliasMSR; MSRC001_0415_instUMCWPHY3UMC_n3_umc0_aliasMSR; MSRC001_0416_instUMCWPHY4UMC_n4_umc0_aliasMSR; MSRC001_0415_instUMCWPHY5UMC_n5_umc0_aliasMSR; MSRC001_0416_instUMCWPHY6UMC_n6_umc0_aliasMSR; MSRC001_0415_instUMCWPHY7UMC_n7_umc0_aliasMSR; MSRC001_0416_instUMCWPHY8UMC_n8_umc0_aliasMSR; MSRC001_0415_instUMCWPHY9UMC_n9_umc0_aliasMSR; MSRC001_0416**Bits Description**

63:12 Reserved.

11 **RdCrcErr.** Read-write. Reset: 0. Read CRC error. CRC error occurred on a DRAM read from any subchannel10 **ThrttlErr.** Read-write. Reset: 0. Indicates that UMC is throttling9 **EcsErr.** Read-write. Reset: 0. [ECS](#) Error. Indicates that a device exceeded the ECS Error Threshold Count.8 **EcsRowErr.** Read-write. Reset: 0. [ECS](#) Row Error. Indicates that a single device row exceeded four code word errors.7 **AesSramEccErr.** Read-write. Reset: 0. AES SRAM ECC error. An ECC error occurred on a AES SRAM in the processor.6 **DcqSramEccErr.** Read-write. Reset: 0. DCQ SRAM ECC error. An ECC error occurred on a DCQ SRAM in the processor.5 **WriteDataCrcErr.** Read-write. Reset: 0. Write data CRC error. A write data CRC error occurred on the DRAM data bus.4 **AddressCommandParityErr.** Read-write. Reset: 0. Address/Command parity error. A parity error occurred on the DRAM address/command bus.3 **ApbErr.** Read-write. Reset: 0. Advanced peripheral bus error. An error occurred on the advanced peripheral bus.2 **SdpParityErr.** Read-write. Reset: 0. SDP parity error. A parity error was detected on write data from the data fabric in the processor.1 **WriteDataPoisonErr.** Read-write. Reset: 0. Data poison error. The system tried to write poison data to DRAM and either DRAM does not support ECC or UMC_CH.EccCtrl.WrEccEn is cleared.0 **DramEccErr.** Read-write. Reset: 0. DRAM ECC error. An ECC error occurred on a DRAM read.

3.2.5.11 PB**MSR0000_0440...MSRC000_2150 [PB Machine Check Control] (MCA::PB::MCA_CTL_PB)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::PB::MCA_CTL_PB register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_instPBCCD_n0_aliasMSR; MSRC000_2100

_ccd0_instPBCCD_n0_aliasMSRLEGACY; MSR0000_0440

_ccd10_instPBCCD_n11_aliasMSR; MSRC000_2100

_ccd10_instPBCCD_n11_aliasMSRLEGACY; MSR0000_0440

_ccd11_instPBCCD_n12_aliasMSR; MSRC000_2100

_ccd11_instPBCCD_n12_aliasMSRLEGACY; MSR0000_0440

_ccd1_instPBCCD_n1_aliasMSR; MSRC000_2100

_ccd1_instPBCCD_n1_aliasMSRLEGACY; MSR0000_0440

_ccd2_instPBCCD_n2_aliasMSR; MSRC000_2100

_ccd2_instPBCCD_n2_aliasMSRLEGACY; MSR0000_0440

_ccd3_instPBCCD_n3_aliasMSR; MSRC000_2100

_ccd3_instPBCCD_n3_aliasMSRLEGACY; MSR0000_0440

_ccd4_instPBCCD_n4_aliasMSR; MSRC000_2100

_ccd4_instPBCCD_n4_aliasMSRLEGACY; MSR0000_0440

_ccd5_instPBCCD_n5_aliasMSR; MSRC000_2100

_ccd5_instPBCCD_n5_aliasMSRLEGACY; MSR0000_0440

_ccd6_instPBCCD_n6_aliasMSR; MSRC000_2100

_ccd6_instPBCCD_n6_aliasMSRLEGACY; MSR0000_0440

_ccd7_instPBCCD_n8_aliasMSR; MSRC000_2100

_ccd7_instPBCCD_n8_aliasMSRLEGACY; MSR0000_0440

_ccd8_instPBCCD_n9_aliasMSR; MSRC000_2100

_ccd8_instPBCCD_n9_aliasMSRLEGACY; MSR0000_0440

_ccd9_instPBCCD_n10_aliasMSR; MSRC000_2100

_ccd9_instPBCCD_n10_aliasMSRLEGACY; MSR0000_0440

_instPB_n7_aliasMSR; MSRC000_2150

_instPB_n7_aliasMSRLEGACY; MSR0000_0454

Bits	Description
63:1	Reserved.
0	EccError. Read-write. Reset: 0. An ECC error in the Parameter Block RAM array.

MSR0000_0441...MSRC000_2151 [PB Machine Check Status] (MCA::PB::MCA_STATUS_PB)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_instPBCCD_n0_aliasMSR; MSRC000_2101

_ccd0_instPBCCD_n0_aliasMSRLEGACY; MSR0000_0441

_ccd10_instPBCCD_n11_aliasMSR; MSRC000_2101

_ccd10_instPBCCD_n11_aliasMSRLEGACY; MSR0000_0441

_ccd11_instPBCCD_n12_aliasMSR; MSRC000_2101

_ccd11_instPBCCD_n12_aliasMSRLEGACY; MSR0000_0441

_ccd1_instPBCCD_n1_aliasMSR; MSRC000_2101

_ccd1_instPBCCD_n1_aliasMSRLEGACY; MSR0000_0441

_ccd2_instPBCCD_n2_aliasMSR; MSRC000_2101

_ccd2_instPBCCD_n2_aliasMSRLEGACY; MSR0000_0441

_ccd3_instPBCCD_n3_aliasMSR; MSRC000_2101

_ccd3_instPBCCD_n3_aliasMSRLEGACY; MSR0000_0441

_ccd4_instPBCCD_n4_aliasMSR; MSRC000_2101

_ccd4_instPBCCD_n4_aliasMSRLEGACY; MSR0000_0441

_ccd5_instPBCCD_n5_aliasMSR; MSRC000_2101

_ccd5_instPBCCD_n5_aliasMSRLEGACY; MSR0000_0441

_ccd6_instPBCCD_n6_aliasMSR; MSRC000_2101

_ccd6_instPBCCD_n6_aliasMSRLEGACY; MSR0000_0441

_ccd7_instPBCCD_n8_aliasMSR; MSRC000_2101

_ccd7_instPBCCD_n8_aliasMSRLEGACY; MSR0000_0441

_ccd8_instPBCCD_n9_aliasMSR; MSRC000_2101

_ccd8_instPBCCD_n9_aliasMSRLEGACY; MSR0000_0441

_ccd9_instPBCCD_n10_aliasMSR; MSRC000_2101

_ccd9_instPBCCD_n10_aliasMSRLEGACY; MSR0000_0441

_instPB_n7_aliasMSR; MSRC000_2151

_instPB_n7_aliasMSRLEGACY; MSR0000_0455

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::PB::MCA_CTL_PB. This bit is a copy of bit in MCA::PB::MCA_CTL_PB for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::PB::MCA_MISC0_PB. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::PB::MCA_ADDR_PB contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::PB::MCA_STATUS_PB[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::PB::MCA_SYND_PB. If MCA::PB::MCA_SYND_PB[ErrorPriority] is the same as the priority of the error in MCA::PB::MCA_STATUS_PB, then the information in MCA::PB::MCA_SYND_PB is associated with the error in MCA::PB::MCA_STATUS_PB. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::PB::MCA_STATUS_PB[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PB::MCA_ADDR_PB[ErrorAddr]. A value of 0 indicates that MCA::PB::MCA_ADDR_PB[55:0] contains a valid byte address. A value of 6 indicates that MCA::PB::MCA_ADDR_PB[55:6] contains a valid cache line address and that MCA::PB::MCA_ADDR_PB[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PB::MCA_ADDR_PB[55:12] contain a valid 4KB memory page and that MCA::PB::MCA_ADDR_PB[11:0] should be ignored by error handling software. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::PB::MCA_CTL_PB enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 88: MCA_STATUS_PB

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
EccError	0x0	0/1	0/1	0/1	0	0	0

MSR0000_0442...MSRC000_2152 [PB Machine Check Address] (MCA::PB::MCA_ADDR_PB)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::PB::MCA_ADDR_PB stores an address and other information associated with the error in MCA::PB::MCA_STATUS_PB. The register is only meaningful if MCA::PB::MCA_STATUS_PB[Val]=1 and MCA::PB::MCA_STATUS_PB[AddrV]=1.

_ccd0_instPBCCD_n0_aliasMSR; MSRC000_2102_ccd0_instPBCCD_n0_aliasMSRLEGACY; MSR0000_0442_ccd10_instPBCCD_n11_aliasMSR; MSRC000_2102_ccd10_instPBCCD_n11_aliasMSRLEGACY; MSR0000_0442_ccd11_instPBCCD_n12_aliasMSR; MSRC000_2102_ccd11_instPBCCD_n12_aliasMSRLEGACY; MSR0000_0442_ccd1_instPBCCD_n1_aliasMSR; MSRC000_2102_ccd1_instPBCCD_n1_aliasMSRLEGACY; MSR0000_0442_ccd2_instPBCCD_n2_aliasMSR; MSRC000_2102_ccd2_instPBCCD_n2_aliasMSRLEGACY; MSR0000_0442_ccd3_instPBCCD_n3_aliasMSR; MSRC000_2102_ccd3_instPBCCD_n3_aliasMSRLEGACY; MSR0000_0442_ccd4_instPBCCD_n4_aliasMSR; MSRC000_2102_ccd4_instPBCCD_n4_aliasMSRLEGACY; MSR0000_0442_ccd5_instPBCCD_n5_aliasMSR; MSRC000_2102_ccd5_instPBCCD_n5_aliasMSRLEGACY; MSR0000_0442_ccd6_instPBCCD_n6_aliasMSR; MSRC000_2102_ccd6_instPBCCD_n6_aliasMSRLEGACY; MSR0000_0442_ccd7_instPBCCD_n8_aliasMSR; MSRC000_2102_ccd7_instPBCCD_n8_aliasMSRLEGACY; MSR0000_0442_ccd8_instPBCCD_n9_aliasMSR; MSRC000_2102_ccd8_instPBCCD_n9_aliasMSRLEGACY; MSR0000_0442_ccd9_instPBCCD_n10_aliasMSR; MSRC000_2102_ccd9_instPBCCD_n10_aliasMSRLEGACY; MSR0000_0442_instPB_n7_aliasMSR; MSRC000_2152_instPB_n7_aliasMSRLEGACY; MSR0000_0456

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PB::MCA_STATUS_PB.

Table 89: MCA_ADDR_PB

Error Type	Bits	Description
EccError	[56:0]	Reserved

MSR0000_0443...MSRC000_2153 [PB Machine Check Miscellaneous 0] (MCA::PB::MCA_MISC0_PB)

Log miscellaneous information associated with errors.

[_ccd0_instPBCCD_n0_aliasMSR](#); [MSRC000_2103](#)[_ccd0_instPBCCD_n0_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd10_instPBCCD_n11_aliasMSR](#); [MSRC000_2103](#)[_ccd10_instPBCCD_n11_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd11_instPBCCD_n12_aliasMSR](#); [MSRC000_2103](#)[_ccd11_instPBCCD_n12_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd1_instPBCCD_n1_aliasMSR](#); [MSRC000_2103](#)[_ccd1_instPBCCD_n1_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd2_instPBCCD_n2_aliasMSR](#); [MSRC000_2103](#)[_ccd2_instPBCCD_n2_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd3_instPBCCD_n3_aliasMSR](#); [MSRC000_2103](#)[_ccd3_instPBCCD_n3_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd4_instPBCCD_n4_aliasMSR](#); [MSRC000_2103](#)[_ccd4_instPBCCD_n4_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd5_instPBCCD_n5_aliasMSR](#); [MSRC000_2103](#)[_ccd5_instPBCCD_n5_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd6_instPBCCD_n6_aliasMSR](#); [MSRC000_2103](#)[_ccd6_instPBCCD_n6_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd7_instPBCCD_n8_aliasMSR](#); [MSRC000_2103](#)[_ccd7_instPBCCD_n8_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd8_instPBCCD_n9_aliasMSR](#); [MSRC000_2103](#)[_ccd8_instPBCCD_n9_aliasMSRLEGACY](#); [MSR0000_0443](#)[_ccd9_instPBCCD_n10_aliasMSR](#); [MSRC000_2103](#)[_ccd9_instPBCCD_n10_aliasMSRLEGACY](#); [MSR0000_0443](#)[_instPB_n7_aliasMSR](#); [MSRC000_2153](#)[_instPB_n7_aliasMSRLEGACY](#); [MSR0000_0457](#)

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PB::MCA_MISC0_PB[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PB::MCA_MISC0_PB[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PB::MCA_MISC0_PB[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PB::MCA_MISC0_PB[Locked]) ? Read-write : Read-only.

48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PB::MCA_MISC0_PB[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PB::MCA_MISC0_PB[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21[0...5]4 [PB Machine Check Configuration] (MCA::PB::MCA_CONFIG_PB)

Reset: 0000_0002_0000_0123h.

Controls configuration of the associated machine check bank.

_ccd0_instPBCCD_n0_aliasMSR; MSRC000_2104

_ccd10_instPBCCD_n11_aliasMSR; MSRC000_2104

_ccd11_instPBCCD_n12_aliasMSR; MSRC000_2104

_ccd1_instPBCCD_n1_aliasMSR; MSRC000_2104

_ccd2_instPBCCD_n2_aliasMSR; MSRC000_2104

_ccd3_instPBCCD_n3_aliasMSR; MSRC000_2104

_ccd4_instPBCCD_n4_aliasMSR; MSRC000_2104

_ccd5_instPBCCD_n5_aliasMSR; MSRC000_2104

_ccd6_instPBCCD_n6_aliasMSR; MSRC000_2104

_ccd7_instPBCCD_n8_aliasMSR; MSRC000_2104

_ccd8_instPBCCD_n9_aliasMSR; MSRC000_2104

_ccd9_instPBCCD_n10_aliasMSR; MSRC000_2104

_instPB_n7_aliasMSR; MSRC000_2154

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable . Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::PB::MCA_STATUS_PB, MCA::PB::MCA_ADDR_PB, MCA::PB::MCA_MISC0_PB, and MCA::PB::MCA_SYND_PB in addition to MCA::PB::MCA_TRANSYND_PB and MCA::PB::MCA_TRANSADDR_PB. 0=Only log transparent errors in MCA::PB::MCA_TRANSYND_PB and MCA::PB::MCA_TRANSADDR_PB.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::PB::MCA_CONFIG_PB[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::PB::MCA_CONFIG_PB[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::PB::MCA_CONFIG_PB[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. MCA_DESTAT and MCA_DEADDR are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::PB::MCA_CONFIG_PB[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::PB::MCA_MISC0_PB[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::PB::MCA_STATUS_PB[TCC] is present.

MSRC000_21[0...5]5 [PB IP Identification] (MCA::PB::MCA_IPID_PB)

Reset: 0000_0005_0000_0000h.

The MCA::PB::MCA_IPID_PB register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_instPBCCD_n0_aliasMSR; MSRC000_2105_ccd10_instPBCCD_n11_aliasMSR; MSRC000_2105_ccd11_instPBCCD_n12_aliasMSR; MSRC000_2105_ccd1_instPBCCD_n1_aliasMSR; MSRC000_2105_ccd2_instPBCCD_n2_aliasMSR; MSRC000_2105_ccd3_instPBCCD_n3_aliasMSR; MSRC000_2105_ccd4_instPBCCD_n4_aliasMSR; MSRC000_2105_ccd5_instPBCCD_n5_aliasMSR; MSRC000_2105_ccd6_instPBCCD_n6_aliasMSR; MSRC000_2105_ccd7_instPBCCD_n8_aliasMSR; MSRC000_2105_ccd8_instPBCCD_n9_aliasMSR; MSRC000_2105_ccd9_instPBCCD_n10_aliasMSR; MSRC000_2105_instPB_n7_aliasMSR; MSRC000_2155

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 005h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: <u>_ccd0_instPBCCD_n0_aliasMSR</u> : 3008_2900h
	Init: <u>_ccd1_instPBCCD_n1_aliasMSR</u> : 3208_2900h
	Init: <u>_ccd2_instPBCCD_n2_aliasMSR</u> : 3408_2900h
	Init: <u>_ccd3_instPBCCD_n3_aliasMSR</u> : 3608_2900h
	Init: <u>_ccd4_instPBCCD_n4_aliasMSR</u> : 3808_2900h
	Init: <u>_ccd5_instPBCCD_n5_aliasMSR</u> : 3A08_2900h
	Init: <u>_ccd6_instPBCCD_n6_aliasMSR</u> : 3C08_2900h
	Init: <u>_ccd7_instPBCCD_n8_aliasMSR</u> : 3E08_2900h
	Init: <u>_ccd8_instPBCCD_n9_aliasMSR</u> : 4A08_2900h
	Init: <u>_ccd9_instPBCCD_n10_aliasMSR</u> : 4C08_2900h
	Init: <u>_ccd10_instPBCCD_n11_aliasMSR</u> : 4E08_2900h
	Init: <u>_ccd11_instPBCCD_n12_aliasMSR</u> : 5008_2900h
	Init: <u>_instPB_n7_aliasMSR</u> : 0005_EA00h

MSRC000_21[0...5]6 [PB Machine Check Syndrome] (MCA::PB::MCA_SYND_PB)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::PB::MCA_STATUS_PB [Thread 0](#)[_ccd0_instPBCCD_n0_aliasMSR](#); MSRC000_2106[_ccd10_instPBCCD_n11_aliasMSR](#); MSRC000_2106[_ccd11_instPBCCD_n12_aliasMSR](#); MSRC000_2106[_ccd1_instPBCCD_n1_aliasMSR](#); MSRC000_2106[_ccd2_instPBCCD_n2_aliasMSR](#); MSRC000_2106[_ccd3_instPBCCD_n3_aliasMSR](#); MSRC000_2106[_ccd4_instPBCCD_n4_aliasMSR](#); MSRC000_2106[_ccd5_instPBCCD_n5_aliasMSR](#); MSRC000_2106[_ccd6_instPBCCD_n6_aliasMSR](#); MSRC000_2106[_ccd7_instPBCCD_n8_aliasMSR](#); MSRC000_2106[_ccd8_instPBCCD_n9_aliasMSR](#); MSRC000_2106[_ccd9_instPBCCD_n10_aliasMSR](#); MSRC000_2106[_instPB_n7_aliasMSR](#); MSRC000_2156

Bits	Description
63:32	Syndrom . Read-write, Volatile. Reset: Cold, 0000_0000h. Contains the syndrome, if any, associated with the error logged in MCA::PB::MCA_STATUS_PB. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PB::MCA_SYND_PB[Length]. The Syndrome field is only valid when MCA::PB::MCA_SYND_PB[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PB::MCA_SYND_PB. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PB::MCA_SYND_PB[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PB::MCA_SYND_PB. For example, a syndrome length of 9 means that MCA::PB::MCA_SYND_PB[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 90 [MCA_SYND_PB].

Table 90: MCA_SYND_PB

Error Type	Bits	Description
EccError	[8]	DEDB error
	[7:4]	Reserved
	[3:0]	IntReqVal

MSRC001_041[0...5] [PB Machine Check Control Mask] (MCA::PB::MCA_CTL_MASK_PB)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_instPBCCD_n0_aliasMSR; MSRC001_0410

_ccd10_instPBCCD_n11_aliasMSR; MSRC001_0410

_ccd11_instPBCCD_n12_aliasMSR; MSRC001_0410

_ccd1_instPBCCD_n1_aliasMSR; MSRC001_0410

_ccd2_instPBCCD_n2_aliasMSR; MSRC001_0410

_ccd3_instPBCCD_n3_aliasMSR; MSRC001_0410

_ccd4_instPBCCD_n4_aliasMSR; MSRC001_0410

_ccd5_instPBCCD_n5_aliasMSR; MSRC001_0410

_ccd6_instPBCCD_n6_aliasMSR; MSRC001_0410

_ccd7_instPBCCD_n8_aliasMSR; MSRC001_0410

_ccd8_instPBCCD_n9_aliasMSR; MSRC001_0410

_ccd9_instPBCCD_n10_aliasMSR; MSRC001_0410

_instPB_n7_aliasMSR; MSRC001_0415

Bits Description

63:1 Reserved.

0 **EccError.** Read-write. Reset: 0. An ECC error in the Parameter Block RAM array.**MSRC001_051[0...5] [PB Machine Check Transparent Error Syndrome] (MCA::PB::MCA_TRANSYND_PB)**

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_ccd0_instPBCCD_n0_aliasMSR; MSRC001_0510

_ccd10_instPBCCD_n11_aliasMSR; MSRC001_0510

_ccd11_instPBCCD_n12_aliasMSR; MSRC001_0510

_ccd1_instPBCCD_n1_aliasMSR; MSRC001_0510

_ccd2_instPBCCD_n2_aliasMSR; MSRC001_0510

_ccd3_instPBCCD_n3_aliasMSR; MSRC001_0510

_ccd4_instPBCCD_n4_aliasMSR; MSRC001_0510

_ccd5_instPBCCD_n5_aliasMSR; MSRC001_0510

_ccd6_instPBCCD_n6_aliasMSR; MSRC001_0510

_ccd7_instPBCCD_n8_aliasMSR; MSRC001_0510

_ccd8_instPBCCD_n9_aliasMSR; MSRC001_0510

_ccd9_instPBCCD_n10_aliasMSR; MSRC001_0510

_instPB_n7_aliasMSR; MSRC001_0515

Bits Description63:32 **Syndrome.** Read-write, Volatile. Reset: Cold, 0000_0000h. Contains the syndrome, if any, associated with the error logged in MCA::PB::MCA_STATUS_PB. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PB::MCA_SYND_PB[Length]. The Syndrome field is only valid when MCA::PB::MCA_SYND_PB[Length] is not 0.

31:28 Reserved.

27 **Valid.** Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::PB::MCA_TRANSYND_PB. All errors writing into MCA::PB::MCA_SYND_PB should set this bit to 1.26:24 **ErrorPriority.** Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PB::MCA_SYND_PB. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.23:18 **Length.** Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PB::MCA_SYND_PB[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PB::MCA_SYND_PB. For example, a syndrome length of 9 means that MCA::PB::MCA_SYND_PB[Syndrome] bits [8:0] contains a valid syndrome.17:0 **ErrorInformation.** Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

MSRC001_061[0...5] [PB Machine Check Transparent Error Address] (MCA::PB::MCA_TRANSADDR_PB)

Read-only, Volatile. Reset: Cold, 0000_0000_0000_0000h.

MCA::PB::MCA_TRANSADDR_PB stores the address associated with the error in MCA::PB::MCA_STATUS_PB. The register is only meaningful if MCA::PB::MCA_STATUS_PB[Val]=1 and

MCA::PB::MCA_STATUS_PB[AddrV]=1. The lowest valid bit of the address is defined by

MCA::PB::MCA_STATUS_PB[AddrLsb].

_ccd0_instPBCCD_n0_aliasMSR; MSRC001_0610

_ccd10_instPBCCD_n11_aliasMSR; MSRC001_0610

_ccd11_instPBCCD_n12_aliasMSR; MSRC001_0610

_ccd1_instPBCCD_n1_aliasMSR; MSRC001_0610

_ccd2_instPBCCD_n2_aliasMSR; MSRC001_0610

_ccd3_instPBCCD_n3_aliasMSR; MSRC001_0610

_ccd4_instPBCCD_n4_aliasMSR; MSRC001_0610

_ccd5_instPBCCD_n5_aliasMSR; MSRC001_0610

_ccd6_instPBCCD_n6_aliasMSR; MSRC001_0610

_ccd7_instPBCCD_n8_aliasMSR; MSRC001_0610

_ccd8_instPBCCD_n9_aliasMSR; MSRC001_0610

_ccd9_instPBCCD_n10_aliasMSR; MSRC001_0610

_instPB_n7_aliasMSR; MSRC001_0615

Bits	Description
63:0	ErrorAddr. Read-only, Volatile. Reset: Cold, 0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PB::MCA_STATUS_PB. The lowest-order valid bit of the address is specified in MCA::PB::MCA_STATUS_PB[AddrLsb].

3.2.5.12 PSP

MSR0000_0470...MSRC000_21C0 [PSP Machine Check Control] (MCA::PSP::MCA_CTL_PSP)

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::PSP::MCA_CTL_PSP register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instMP0MP0_n0_aliasMSR; MSRC000_21C0

_instMP0MP0_n0_aliasMSRLEGACY; MSR0000_0470

Bits	Description
63	WAFL. Read-write. A corrected CRC or uncorrected error occurs on the WAFL link
62:18	Reserved.
17	Mp0SHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
16	Mp0TlbBank1Error. Read-write. Reset: 0. TLB Bank 1 parity error.
15	Mp0TlbBank0Error. Read-write. Reset: 0. TLB Bank 0 parity error.
14	Mp0DDirtyRamError. Read-write. Reset: 0. Dirty Data Ram parity error.
13	Mp0DTagBank3Error. Read-write. Reset: 0. Data Tag Bank 3 parity error.
12	Mp0DTagBank2Error. Read-write. Reset: 0. Data Tag Bank 2 parity error.
11	Mp0DTagBank1Error. Read-write. Reset: 0. Data Tag Bank 1 parity error.
10	Mp0DTagBank0Error. Read-write. Reset: 0. Data Tag Bank 0 parity error.
9	Mp0DDDataBank3Error. Read-write. Reset: 0. Data Cache Bank 3 ECC or parity error.
8	Mp0DDDataBank2Error. Read-write. Reset: 0. Data Cache Bank 2 ECC or parity error.
7	Mp0DDDataBank1Error. Read-write. Reset: 0. Data Cache Bank 1 ECC or parity error.
6	Mp0DDDataBank0Error. Read-write. Reset: 0. Data Cache Bank 0 ECC or parity error.
5	Mp0ITagRam1Error. Read-write. Reset: 0. Instruction Tag Ram 1 parity error.
4	Mp0ITagRam0Error. Read-write. Reset: 0. Instruction Tag Ram 0 parity error.
3	Mp0IDataBank1Error. Read-write. Reset: 0. Instruction Cache Bank 1 ECC or parity error.
2	Mp0IDataBank0Error. Read-write. Reset: 0. Instruction Cache Bank 0 ECC or parity error.
1	Mp0LowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	Mp0HighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

MSR0000_0471...MSRC000_21C1 [PSP Machine Check Status] (MCA::PSP::MCA_STATUS_PSP)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instMPOMP0_n0_aliasMSR; MSRC000_21C1

_instMPOMP0_n0_aliasMSRLEGACY; MSR0000_0471

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::PSP::MCA_CTL_PSP. This bit is a copy of bit in MCA::PSP::MCA_CTL_PSP for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::PSP::MCA_MISC0_PSP. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::PSP::MCA_ADDR_PSP contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::PSP::MCA_STATUS_PSP[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::PSP::MCA_SYND_PSP. If MCA::PSP::MCA_SYND_PSP[ErrorPriority] is the same as the priority of the error in MCA::PSP::MCA_STATUS_PSP, then the information in MCA::PSP::MCA_SYND_PSP is associated with the error in MCA::PSP::MCA_STATUS_PSP. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::PSP::MCA_STATUS_PSP[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PSP::MCA_ADDR_PSP[ErrorAddr]. A value of 0 indicates that MCA::PSP::MCA_ADDR_PSP[55:0] contains a valid byte address. A value of 6 indicates that MCA::PSP::MCA_ADDR_PSP[55:6] contains a valid cache line address and that MCA::PSP::MCA_ADDR_PSP[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PSP::MCA_ADDR_PSP[55:12] contain a valid 4KB memory page and that MCA::PSP::MCA_ADDR_PSP[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::PSP::MCA_CTL_PSP enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 91: MCA_STATUS_PSP

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
Mp0HighSramError	0x0	0/1	0/1	0/1	0	0	1
Mp0LowSramError	0x1	0/1	0/1	0/1	0	0	1
Mp0IDataBank0Error	0x2	0/1	0/1	0/1	0	0	1
Mp0IDataBank1Error	0x3	0/1	0/1	0/1	0	0	1
Mp0ITagRam0Error	0x4	1	1	1	0	0	1

Mp0ITagBank0Error	0x5	1	1	1	0	0	1
Mp0DDDataBank0Error	0x6	0/1	0/1	0/1	0	0	1
Mp0DDDataBank1Error	0x7	0/1	0/1	0/1	0	0	1
Mp0DDDataBank2Error	0x8	0/1	0/1	0/1	0	0	1
Mp0DDDataBank3Error	0x9	0/1	0/1	0/1	0	0	1
Mp0DTagBank0Error	0xa	1	1	1	0	0	1
Mp0DTagBank1Error	0xb	1	1	1	0	0	1
Mp0DTagBank2Error	0xc	1	1	1	0	0	1
Mp0DTagBank3Error	0xd	1	1	1	0	0	1
Mp0DDirtyRamError	0xe	1	1	1	0	0	1
Mp0TlbBank0Error	0xf	1	1	1	0	0	1
Mp0TlbBank1Error	0x10	1	1	1	0	0	1
Mp0SHubIfRdBufError	0x11	1	1	1	0	0	1
Reserved	0x3a	1	1	1	0	0	1
Reserved	0x3b	1	1	1	0	0	1
Reserved	0x3c	1	1	1	0	0	1
Reserved	0x3d	1	1	1	0	0	1
WAFI	0x3f	0/1	0/1	0/1	0	0	1

MSR0000_0472...MSRC000_21C2 [PSP Machine Check Address] (MCA::PSP::MCA_ADDR_PSP)

Reset: Cold,0000_0000_0000_0000h.

MCA::PSP::MCA_ADDR_PSP stores an address and other information associated with the error in MCA::PSP::MCA_STATUS_PSP. The register is only meaningful if MCA::PSP::MCA_STATUS_PSP[Val]=1 and MCA::PSP::MCA_STATUS_PSP[AddrV]=1.

_instMP0MP0_n0_aliasMSR; MSRC000_21C2

_instMP0MP0_n0_aliasMSRLEGACY; MSR0000_0472

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::PSP::MCA_STATUS_PSP. For physical addresses, the most significant bit is given by Core::X86::CpuId::LongModeInfo[PhysAddrSize].

Table 92: MCA_ADDR_PSP

Error Type	Bits	Description
Mp0HighSramError	[55:0]	Reserved
Mp0LowSramError	[55:0]	Reserved
Mp0IDataBank0Error	[55:0]	Reserved

Mp0IDataBank1Error	[55:0]	Reserved
Mp0ITagRam0Error	[55:0]	Reserved
Mp0ITagRam1Error	[55:0]	Reserved
Mp0DDDataBank0Error	[55:0]	Reserved
Mp0DDDataBank1Error	[55:0]	Reserved
Mp0DDDataBank2Error	[55:0]	Reserved
Mp0DDDataBank3Error	[55:0]	Reserved
Mp0DTagBank0Error	[55:0]	Reserved
Mp0DTagBank1Error	[55:0]	Reserved
Mp0DTagBank2Error	[55:0]	Reserved
Mp0DTagBank3Error	[55:0]	Reserved
Mp0DDirtyRamError	[55:0]	Reserved
Mp0TlbBank0Error	[55:0]	Reserved
Mp0TlbBank1Error	[55:0]	Reserved
Mp0SHubIfRdBufError	[55:0]	Reserved
Reserved	[55:0]	Reserved
Reserved	[55:0]	Reserved
Reserved	[55:0]	Reserved
Reserved	[55:0]	Reserved
WAFL	[55:0]	Reserved

MSR0000_0473...MSRC000_21C3 [PSP Machine Check Miscellaneous 0] (MCA::PSP::MCA_MISC0_PSP)

Log miscellaneous information associated with errors.

_instMP0MP0_n0_aliasMSR; MSRC000_21C3

_instMP0MP0_n0_aliasMSRLEGACY; MSR0000_0473

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PSP::MCA_MISC0_PSP[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PSP::MCA_MISC0_PSP[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PSP::MCA_MISC0_PSP[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PSP::MCA_MISC0_PSP[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PSP::MCA_MISC0_PSP[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PSP::MCA_MISC0_PSP[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21C4 [PSP Machine Check Configuration] (MCA::PSP::MCA_CONFIG_PSP)

Reset: 0000_0002_0000_0123h.

Controls configuration of the associated machine check bank.

_instMPOMP0_n0_aliasMSR; MSRC000_21C4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable. Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::PSP::MCA_STATUS_PSP, MCA::PSP::MCA_ADDR_PSP, MCA::PSP::MCA_MISC0_PSP, and MCA::PSP::MCA_SYND_PSP in addition to MCA::PSP::MCA_TRANSYND_PSP and MCA::PSP::MCA_TRANSADDR_PSP. 0=Only log transparent errors in MCA::PSP::MCA_TRANSYND_PSP and MCA::PSP::MCA_TRANSADDR_PSP.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::PSP::MCA_CONFIG_PSP[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::PSP::MCA_CONFIG_PSP[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::PSP::MCA_CONFIG_PSP[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. MCA_DESTAT and MCA_DEADDR are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::PSP::MCA_CONFIG_PSP[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::PSP::MCA_MISC0_PSP[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::PSP::MCA_STATUS_PSP[TCC] is present.

MSRC000_21C5 [PSP IP Identification] (MCA::PSP::MCA_IPID_PSP)

Reset: 0001_00FF_0000_0000h.

The MCA::PSP::MCA_IPID_PSP register is used by software to determine what IP type and revision is associated with the MCA bank.

_instMPOMP0_n0_aliasMSR; MSRC000_21C5

Bits	Description
63:48	McaType. Read-only. Reset: 0001h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 0FFh. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. Init: 0383_0400h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.

MSRC000_21C6 [PSP Machine Check Syndrome] (MCA::PSP::MCA_SYND_PSP)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.	
Logs physical location information associated with error in MCA::PSP::MCA_STATUS_PSP Thread 0	
_instMP0MP0_n0_aliasMSR; MSRC000_21C6	
Bits	Description
63:33	Reserved.
32	Syndromes. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::PSP::MCA_STATUS_PSP. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PSP::MCA_SYND_PSP[Length]. The Syndrome field is only valid when MCA::PSP::MCA_SYND_PSP[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PSP::MCA_SYND_PSP. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PSP::MCA_SYND_PSP[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PSP::MCA_SYND_PSP. For example, a syndrome length of 9 means that MCA::PSP::MCA_SYND_PSP[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 93 [MCA_SYND_PSP].

Table 93: MCA_SYND_PSP

Error Type	Bits	Description
Mp0HighSramError	[17:0]	Reserved
Mp0LowSramError	[17:0]	Reserved
Mp0IDataBank0Error	[17:9] [8:0]	Reserved Reserved
Mp0IDataBank1Error	[17:9] [8:0]	Reserved Reserved
Mp0ITagRam0Error	[17:7] [6:0]	Reserved Reserved
Mp0ITagRam1Error	[17:7] [6:0]	Reserved Reserved
Mp0DDataBank0Error	[17:9] [8:0]	Reserved Reserved
Mp0DDataBank1Error	[17:9] [8:0]	Reserved Reserved
Mp0DDataBank2Error	[17:9] [8:0]	Reserved Reserved
Mp0DDataBank3Error	[17:9] [8:0]	Reserved Reserved
Mp0DTagBank0Error	[17:6] [5:0]	Reserved Reserved
Mp0DTagBank1Error	[17:6] [5:0]	Reserved Reserved
Mp0DTagBank2Error	[17:6] [5:0]	Reserved Reserved
Mp0DTagBank3Error	[17:6] [5:0]	Reserved Reserved

Mp0DDirtyRamError	[17:6] [5:0]	Reserved Reserved
Mp0TlbBank0Error	[17:6] [5:0]	Reserved Reserved
Mp0TlbBank1Error	[17:6] [5:0]	Reserved Reserved
Mp0SHubIfRdBufError	[17:6] [5:0]	Reserved Reserved
Reserved	[17:0]	Reserved
Reserved	[17:0]	Reserved
Reserved	[17:0]	Reserved
Reserved	[17:0]	Reserved
WAFI	[17:0]	Reserved

MSRC001_041C [PSP Machine Check Control Mask] (MCA::PSP::MCA_CTL_MASK_PSP)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instMP0MP0_n0_aliasMSR; MSRC001_041C

Bits	Description
63:18	Reserved.
17	Mp0SHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
16	Mp0TlbBank1Error. Read-write. Reset: 0. TLB Bank 1 parity error.
15	Mp0TlbBank0Error. Read-write. Reset: 0. TLB Bank 0 parity error.
14	Mp0DDirtyRamError. Read-write. Reset: 0. Dirty Data Ram parity error.
13	Mp0DTagBank3Error. Read-write. Reset: 0. Data Tag Bank 3 parity error.
12	Mp0DTagBank2Error. Read-write. Reset: 0. Data Tag Bank 2 parity error.
11	Mp0DTagBank1Error. Read-write. Reset: 0. Data Tag Bank 1 parity error.
10	Mp0DTagBank0Error. Read-write. Reset: 0. Data Tag Bank 0 parity error.
9	Mp0DDDataBank3Error. Read-write. Reset: 0. Data Cache Bank 3 ECC or parity error.
8	Mp0DDDataBank2Error. Read-write. Reset: 0. Data Cache Bank 2 ECC or parity error.
7	Mp0DDDataBank1Error. Read-write. Reset: 0. Data Cache Bank 1 ECC or parity error.
6	Mp0DDDataBank0Error. Read-write. Reset: 0. Data Cache Bank 0 ECC or parity error.
5	Mp0ITagRam1Error. Read-write. Reset: 0. Instruction Tag Ram 1 parity error.
4	Mp0ITagRam0Error. Read-write. Reset: 0. Instruction Tag Ram 0 parity error.
3	Mp0IDataBank1Error. Read-write. Reset: 0. Instruction Cache Bank 1 ECC or parity error.
2	Mp0IDataBank0Error. Read-write. Reset: 0. Instruction Cache Bank 0 ECC or parity error.
1	Mp0LowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	Mp0HighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

MSRC001_051C [PSP Machine Check Transparent Error Syndrome] (MCA::PSP::MCA_TRANSYND_PSP)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_instMP0MP0_n0_aliasMSR; MSRC001_051C

Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::PSP::MCA_STATUS_PSP. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PSP::MCA_SYND_PSP[Length]. The Syndrome field is only valid when MCA::PSP::MCA_SYND_PSP[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::PSP::MCA_TRANSYND_PSP. All errors writing into MCA::PSP::MCA_SYND_PSP should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PSP::MCA_SYND_PSP. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PSP::MCA_SYND_PSP[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PSP::MCA_SYND_PSP. For example, a syndrome length of 9 means that MCA::PSP::MCA_SYND_PSP[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

MSRC001_061C [PSP Machine Check Transparent Error Address] (MCA::PSP::MCA_TRANSADDR_PSP)

Reset: Cold, 0000_0000_0000_0000h.

MCA::PSP::MCA_TRANSADDR_PSP stores the address associated with the error in MCA::PSP::MCA_STATUS_PSP. The register is only meaningful if MCA::PSP::MCA_STATUS_PSP[Val]=1 and MCA::PSP::MCA_STATUS_PSP[AddrV]=1. The lowest valid bit of the address is defined by MCA::PSP::MCA_STATUS_PSP[AddrLsb].

_instMP0MP0_n0_aliasMSR; MSRC001_061C

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold, 00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PSP::MCA_STATUS_PSP. The lowest-order valid bit of the address is specified in MCA::PSP::MCA_STATUS_PSP[AddrLsb].

3.2.5.13 SMU

MSR0000_0474...MSRC000_21D0 [SMU Machine Check Control] (MCA::SMU::MCA_CTL_SMU_MP1)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::SMU::MCA_CTL_SMU register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instMP1MP1_n0_aliasMSR; MSRC000_21D0_instMP1MP1_n0_aliasMSRLEGACY; MSR0000_0474

Bits	Description
63:11	Reserved.
10	Mp1SHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
9	Mp1ITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
8	Mp1ITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
7	Mp1ICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
6	Mp1ICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
5	Mp1DTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
4	Mp1DTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
3	Mp1DCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
2	Mp1DCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
1	Mp1LowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	Mp1HighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

MSR0000_0474...MSRC000_21D0 [SMU Machine Check Control] (MCA::SMU::MCA_CTL_SMU_MPIO)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::SMU::MCA_CTL_SMU register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instMPIOMPIO_n1_aliasMSR; MSRC000_21D0_instMPIOMPIO_n1_aliasMSRLEGACY; MSR0000_0474

Bits	Description
63:11	Reserved.
10	MpIOSHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
9	MpIOITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
8	MpIOITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
7	MpIOICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
6	MpIOICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
5	MpIODTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
4	MpIODTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
3	MpIODCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
2	MpIODCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
1	MpIOLowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	MpIOHighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

MSR0000_0475...MSRC000_21D1 [SMU Machine Check Status] (MCA::SMU::MCA_STATUS_SMU)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instMP1MP1_n0_aliasMSR; MSRC000_21D1

_instMP1MP1_n0_aliasMSRLEGACY; MSR0000_0475

_instMPIOMPIO_n1_aliasMSR; MSRC000_21D1

_instMPIOMPIO_n1_aliasMSRLEGACY; MSR0000_0475

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::SMU::MCA_CTL_SMU_MP1 or MCA::SMU::MCA_CTL_SMU_MPIO. This bit is a copy of bit in MCA::SMU::MCA_CTL_SMU_MP1 or MCA::SMU::MCA_CTL_SMU_MPIO for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::SMU::MCA_MISC0_SMU. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::SMU::MCA_ADDR_SMU contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::SMU::MCA_STATUS_SMU[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::SMU::MCA_SYND_SMU. If MCA::SMU::MCA_SYND_SMU[ErrorPriority] is the same as the priority of the error in MCA::SMU::MCA_STATUS_SMU, then the information in MCA::SMU::MCA_SYND_SMU is associated with the error in MCA::SMU::MCA_STATUS_SMU. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::SMU::MCA_STATUS_SMU[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::SMU::MCA_ADDR_SMU[ErrorAddr]. A value of 0 indicates that MCA::SMU::MCA_ADDR_SMU[55:0] contains a valid byte address. A value of 6 indicates that MCA::SMU::MCA_ADDR_SMU[55:6] contains a valid cache line address and that MCA::SMU::MCA_ADDR_SMU[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::SMU::MCA_ADDR_SMU[55:12] contain a valid 4KB memory page and that MCA::SMU::MCA_ADDR_SMU[11:0] should be ignored by error handling software. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::SMU::MCA_CTL_SMU_MP1 or MCA::SMU::MCA_CTL_SMU_MPIO enables error reporting for the logged error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 94: MCA_STATUS_SMU

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
MpIOHighSr amError	0x0	0/1	0/1	0/1	0	0	1
MpIOLowSr amError	0x1	0/1	0/1	0/1	0	0	1
MpIODCach eAError	0x2	0/1	0/1	0/1	0	0	1

MpIODCacheError	0x3	0/1	0/1	0/1	0	0	1
MpIODTagAError	0x4	0/1	0/1	0/1	0	0	1
MpIODTagBError	0x5	0/1	0/1	0/1	0	0	1
MpIOICacheAError	0x6	0/1	0/1	0/1	0	0	1
MpIOICacheBError	0x7	0/1	0/1	0/1	0	0	1
MpIOITagAError	0x8	0/1	0/1	0/1	0	0	1
MpIOITagBError	0x9	0/1	0/1	0/1	0	0	1
MpIOSHubIfRdBufError	0xa	0/1	0/1	0/1	0	0	1
PhyRamEccError	0xb	0	0	0	0	0	0
EdcIndicator	0x39	0	0	0	0	0	0

MSR0000_0476...MSRC000_21D2 [SMU Machine Check Address] (MCA::SMU::MCA_ADDR_SMU)

Reset: Cold,0000_0000_0000_0000h.

MCA::SMU::MCA_ADDR_SMU stores an address and other information associated with the error in MCA::SMU::MCA_STATUS_SMU. The register is only meaningful if MCA::SMU::MCA_STATUS_SMU[Val]=1 and MCA::SMU::MCA_STATUS_SMU[AddrV]=1.

_instMP1MP1_n0_aliasMSR; MSRC000_21D2_instMP1MP1_n0_aliasMSRLEGACY; MSR0000_0476_instMPIOMPIO_n1_aliasMSR; MSRC000_21D2_instMPIOMPIO_n1_aliasMSRLEGACY; MSR0000_0476

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::SMU::MCA_STATUS_SMU. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 95: MCA_ADDR_SMU

Error Type	Bits	Description
MpIOHighSramError	[55:0]	Reserved
MpIOLowSramError	[55:0]	Reserved
MpIODCacheAError	[55:0]	Reserved
MpIODCacheBError	[55:0]	Reserved
MpIODTagAError	[55:0]	Reserved
MpIODTagBError	[55:0]	Reserved
MpIOICacheAError	[55:0]	Reserved
MpIOICacheBError	[55:0]	Reserved
MpIOITagAError	[55:0]	Reserved
MpIOITagBError	[55:0]	Reserved
MpIOSHubIfRdBufError	[55:0]	Reserved
PhyRamEccError	[55:0]	Reserved
EdcIndicator	[55:0]	Reserved

MSR0000_0477...MSRC000_21D3 [SMU Machine Check Miscellaneous 0] (MCA::SMU::MCA_MISC0_SMU)

Log miscellaneous information associated with errors.

_instMP1MP1_n0_aliasMSR; MSRC000_21D3

_instMP1MP1_n0_aliasMSRLEGACY; MSR0000_0477

_instMPIOMPIO_n1_aliasMSR; MSRC000_21D3

_instMPIOMPIO_n1_aliasMSRLEGACY; MSR0000_0477

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SMU::MCA_MISC0_SMU[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SMU::MCA_MISC0_SMU[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SMU::MCA_MISC0_SMU[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SMU::MCA_MISC0_SMU[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SMU::MCA_MISC0_SMU[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SMU::MCA_MISC0_SMU[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21D4 [SMU Machine Check Configuration] (MCA::SMU::MCA_CONFIG_SMU)

Reset: 0000_0002_0000_0123h.

Controls configuration of the associated machine check bank.

_instMP1MP1_n0_aliasMSR; MSRC000_21D4_instMPIOMPIO_n1_aliasMSR; MSRC000_21D4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable . Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::SMU::MCA_STATUS_SMU, MCA::SMU::MCA_ADDR_SMU, MCA::SMU::MCA_MISC0_SMU, and MCA::SMU::MCA_SYND_SMU in addition to MCA::SMU::MCA_TRANSynd_SMU and MCA::SMU::MCA_TRANSADDR_SMU. 0=Only log transparent errors in MCA::SMU::MCA_TRANSynd_SMU and MCA::SMU::MCA_TRANSADDR_SMU.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::SMU::MCA_CONFIG_SMU[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::SMU::MCA_CONFIG_SMU[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::SMU::MCA_CONFIG_SMU[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. MCA_DESTAT and MCA_DEADDR are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::SMU::MCA_CONFIG_SMU[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::SMU::MCA_MISC0_SMU[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::SMU::MCA_STATUS_SMU[TCC] is present.

MSRC000_21D5 [SMU IP Identification] (MCA::SMU::MCA_IPID_SMU)

Reset: 0001_0001_0000_0000h.

The MCA::SMU::MCA_IPID_SMU register is used by software to determine what IP type and revision is associated with the MCA bank.

_instMP1MP1_n0_aliasMSR; MSRC000_21D5

_instMPIOMPIO_n1_aliasMSR; MSRC000_21D5

Bits	Description
63:48	McaType. Read-only. Reset: 0001h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 001h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instMP1MP1_n0_aliasMSR: 03B3_0400h
	Init: _instMPIOMPIO_n1_aliasMSR: 0C93_0400h

MSRC000_21D6 [SMU Machine Check Syndrome] (MCA::SMU::MCA_SYND_SMU)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::SMU::MCA_STATUS_SMU [Thread 0](#)

_instMP1MP1_n0_aliasMSR; MSRC000_21D6

_instMPIOMPIO_n1_aliasMSR; MSRC000_21D6

Bits	Description
63:33	Reserved.
32	Syndrom. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::SMU::MCA_STATUS_SMU. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::SMU::MCA_SYND_SMU[Length]. The Syndrome field is only valid when MCA::SMU::MCA_SYND_SMU[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::SMU::MCA_SYND_SMU. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::SMU::MCA_SYND_SMU[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::SMU::MCA_SYND_SMU. For example, a syndrome length of 9 means that MCA::SMU::MCA_SYND_SMU[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 96 [MCA_SYND_SMU].

Table 96: MCA_SYND_SMU

Error Type	Bits	Description
MpIOHighSramError	[17:15] [14:0]	Reserved Reserved
MpIOLowSramError	[17:15] [14:0]	Reserved Reserved
MpIODCacheAError	[17:8] [7:0]	Reserved Reserved
MpIODCacheBError	[17:8] [7:0]	Reserved Reserved
MpIODTagAError	[17:7] [6:0]	Reserved Reserved
MpIODTagBError	[17:7]	Reserved

	[6:0]	Reserved
MpIOICacheAError	[17:8]	Reserved
	[7:0]	Reserved
MpIOICacheBError	[17:8]	Reserved
	[7:0]	Reserved
MpIOITagAError	[17:6]	Reserved
	[5:0]	Reserved
MpIOITagBError	[17:6]	Reserved
	[5:0]	Reserved
MpIOSHubIfRdBufError	[17:6]	Reserved
	[5:0]	Reserved
PhyRamEccError	[17:0]	Reserved
EdcIndicator	[17:0]	Reserved

MSRC001_041D [SMU Machine Check Control Mask] (MCA::SMU::MCA_CTL_MASK_SMU_MP1)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instMP1MP1_n0_aliasMSR; MSRC001_041D

Bits	Description
63:11	Reserved.
10	Mp1SHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
9	Mp1ITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
8	Mp1ITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
7	Mp1ICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
6	Mp1ICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
5	Mp1DTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
4	Mp1DTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
3	Mp1DCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
2	Mp1DCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
1	Mp1LowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	Mp1HighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

MSRC001_041D [SMU Machine Check Control Mask] (MCA::SMU::MCA_CTL_MASK_SMU_MPIO)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instMPIOMPIO_n1_aliasMSR; MSRC001_041D

Bits	Description
63:11	Reserved.
10	MpIOSHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
9	MpIOITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
8	MpIOITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
7	MpIOICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
6	MpIOICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
5	MpIODTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
4	MpIODTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
3	MpIODCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
2	MpIODCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
1	MpIOLowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	MpIOHighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

**MSRC001_051D [SMU Machine Check Transparent Error Syndrome]
(MCA::SMU::MCA_TRANSynd_SMU)**

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_instMP1MP1_n0_aliasMSR; MSRC001_051D

_instMPIOMPPIO_n1_aliasMSR; MSRC001_051D

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::SMU::MCA_STATUS_SMU. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::SMU::MCA_SYND_SMU[Length]. The Syndrome field is only valid when MCA::SMU::MCA_SYND_SMU[Length] is not 0.
31:28	Reserved.
27	Valid . Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::SMU::MCA_TRANSynd_SMU. All errors writing into MCA::SMU::MCA_SYND_SMU should set this bit to 1.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::SMU::MCA_SYND_SMU. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::SMU::MCA_SYND_SMU[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::SMU::MCA_SYND_SMU. For example, a syndrome length of 9 means that MCA::SMU::MCA_SYND_SMU[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

MSRC001_061D [SMU Machine Check Transparent Error Address] (MCA::SMU::MCA_TRANSADDR_SMU)

Reset: Cold, 0000_0000_0000_0000h.

MCA::SMU::MCA_TRANSADDR_SMU stores the address associated with the error in MCA::SMU::MCA_STATUS_SMU. The register is only meaningful if MCA::SMU::MCA_STATUS_SMU[Val]=1 and MCA::SMU::MCA_STATUS_SMU[AddrV]=1. The lowest valid bit of the address is defined by MCA::SMU::MCA_STATUS_SMU[AddrLsb].

_instMP1MP1_n0_aliasMSR; MSRC001_061D

_instMPIOMPPIO_n1_aliasMSR; MSRC001_061D

Bits	Description
63:56	Reserved.
55:0	ErrorAddr . Read-write, Volatile. Reset: Cold, 00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::SMU::MCA_STATUS_SMU. The lowest-order valid bit of the address is specified in MCA::SMU::MCA_STATUS_SMU[AddrLsb].

3.2.5.14 MP5

MSR0000_043C...MSRC000_20F0 [MP5 Machine Check Control] (MCA::MP5::MCA_CTL_MP5)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::MP5::MCA_CTL_MP5 register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F0_ccd0_instMP5MP5_n0_aliasMSRLEGACY; MSR0000_043C_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F0_ccd10_instMP5MP5_n10_aliasMSRLEGACY; MSR0000_043C_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F0_ccd11_instMP5MP5_n11_aliasMSRLEGACY; MSR0000_043C_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F0_ccd1_instMP5MP5_n1_aliasMSRLEGACY; MSR0000_043C_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F0_ccd2_instMP5MP5_n2_aliasMSRLEGACY; MSR0000_043C_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F0_ccd3_instMP5MP5_n3_aliasMSRLEGACY; MSR0000_043C_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F0_ccd4_instMP5MP5_n4_aliasMSRLEGACY; MSR0000_043C_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F0_ccd5_instMP5MP5_n5_aliasMSRLEGACY; MSR0000_043C_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F0_ccd6_instMP5MP5_n6_aliasMSRLEGACY; MSR0000_043C_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F0_ccd7_instMP5MP5_n7_aliasMSRLEGACY; MSR0000_043C_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F0_ccd8_instMP5MP5_n8_aliasMSRLEGACY; MSR0000_043C_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F0_ccd9_instMP5MP5_n9_aliasMSRLEGACY; MSR0000_043C**Bits Description**

63:10 Reserved.

9 **Mp5ITagBError.** Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.8 **Mp5ITagAError.** Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.7 **Mp5ICacheBError.** Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.6 **Mp5ICacheAError.** Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.5 **Mp5DTagBError.** Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.4 **Mp5DTagAError.** Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.3 **Mp5DCacheBError.** Read-write. Reset: 0. Data Cache Bank B ECC or parity error.2 **Mp5DCacheAError.** Read-write. Reset: 0. Data Cache Bank A ECC or parity error.1 **Mp5LowSramError.** Read-write. Reset: 0. Low SRAM ECC or parity error.0 **Mp5HighSramError.** Read-write. Reset: 0. High SRAM ECC or parity error.

MSR0000_043D...MSRC000_20F1 [MP5 Machine Check Status] (MCA::MP5::MCA_STATUS_MP5)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F1

_ccd0_instMP5MP5_n0_aliasMSRLEGACY; MSR0000_043D

_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F1

_ccd10_instMP5MP5_n10_aliasMSRLEGACY; MSR0000_043D

_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F1

_ccd11_instMP5MP5_n11_aliasMSRLEGACY; MSR0000_043D

_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F1

_ccd1_instMP5MP5_n1_aliasMSRLEGACY; MSR0000_043D

_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F1

_ccd2_instMP5MP5_n2_aliasMSRLEGACY; MSR0000_043D

_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F1

_ccd3_instMP5MP5_n3_aliasMSRLEGACY; MSR0000_043D

_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F1

_ccd4_instMP5MP5_n4_aliasMSRLEGACY; MSR0000_043D

_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F1

_ccd5_instMP5MP5_n5_aliasMSRLEGACY; MSR0000_043D

_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F1

_ccd6_instMP5MP5_n6_aliasMSRLEGACY; MSR0000_043D

_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F1

_ccd7_instMP5MP5_n7_aliasMSRLEGACY; MSR0000_043D

_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F1

_ccd8_instMP5MP5_n8_aliasMSRLEGACY; MSR0000_043D

_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F1

_ccd9_instMP5MP5_n9_aliasMSRLEGACY; MSR0000_043D

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::MP5::MCA_CTL_MP5. This bit is a copy of bit in MCA::MP5::MCA_CTL_MP5 for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::MP5::MCA_MISC0_MP5. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::MP5::MCA_ADDR_MP5 contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::MP5::MCA_STATUS_MP5[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::MP5::MCA_SYND_MP5. If MCA::MP5::MCA_SYND_MP5[ErrorPriority] is the same as the priority of the error in MCA::MP5::MCA_STATUS_MP5, then the information in MCA::MP5::MCA_SYND_MP5 is associated with the error in MCA::MP5::MCA_STATUS_MP5. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::MP5::MCA_STATUS_MP5[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::MP5::MCA_ADDR_MP5[ErrorAddr]. A value of 0 indicates that MCA::MP5::MCA_ADDR_MP5[55:0] contains a valid byte address. A value of 6 indicates that MCA::MP5::MCA_ADDR_MP5[55:6] contains a valid cache line address and that MCA::MP5::MCA_ADDR_MP5[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::MP5::MCA_ADDR_MP5[55:12] contain a valid 4KB memory page and that MCA::MP5::MCA_ADDR_MP5[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::MP5::MCA_CTL_MP5 enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 97: MCA_STATUS_MP5

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
Mp5HighSramError	0x0	0/1	0/1	0/1	0	0	1
Mp5LowSramError	0x1	0/1	0/1	0/1	0	0	1
Mp5DCacheAError	0x2	0/1	0/1	0/1	0	0	1
Mp5DCacheBError	0x3	0/1	0/1	0/1	0	0	1
Mp5DTagAError	0x4	0/1	0/1	0/1	0	0	1
Mp5DTagBError	0x5	0/1	0/1	0/1	0	0	1
Mp5ICacheAError	0x6	0/1	0/1	0/1	0	0	1
Mp5ICacheBError	0x7	0/1	0/1	0/1	0	0	1
Mp5ITagAError	0x8	0/1	0/1	0/1	0	0	1
Mp5ITagBError	0x9	0/1	0/1	0/1	0	0	1

MSR0000_043E...MSRC000_20F2 [MP5 Machine Check Address] (MCA::MP5::MCA_ADDR_MP5)

Reset: Cold,0000_0000_0000_0000h.

MCA::MP5::MCA_ADDR_MP5 stores an address and other information associated with the error in MCA::MP5::MCA_STATUS_MP5. The register is only meaningful if MCA::MP5::MCA_STATUS_MP5[Val]=1 and MCA::MP5::MCA_STATUS_MP5[AddrV]=1.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F2_ccd0_instMP5MP5_n0_aliasMSRLEGACY; MSR0000_043E_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F2_ccd10_instMP5MP5_n10_aliasMSRLEGACY; MSR0000_043E_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F2_ccd11_instMP5MP5_n11_aliasMSRLEGACY; MSR0000_043E_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F2_ccd1_instMP5MP5_n1_aliasMSRLEGACY; MSR0000_043E_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F2_ccd2_instMP5MP5_n2_aliasMSRLEGACY; MSR0000_043E_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F2_ccd3_instMP5MP5_n3_aliasMSRLEGACY; MSR0000_043E_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F2_ccd4_instMP5MP5_n4_aliasMSRLEGACY; MSR0000_043E_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F2_ccd5_instMP5MP5_n5_aliasMSRLEGACY; MSR0000_043E_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F2_ccd6_instMP5MP5_n6_aliasMSRLEGACY; MSR0000_043E_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F2_ccd7_instMP5MP5_n7_aliasMSRLEGACY; MSR0000_043E_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F2_ccd8_instMP5MP5_n8_aliasMSRLEGACY; MSR0000_043E_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F2_ccd9_instMP5MP5_n9_aliasMSRLEGACY; MSR0000_043E**Bits Description**

63:56 Reserved.

55:0 **ErrorAddr.** Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::MP5::MCA_STATUS_MP5. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 98: MCA_ADDR_MP5

Error Type	Bits	Description
Mp5HighSramError	[55:0]	Reserved
Mp5LowSramError	[55:0]	Reserved
Mp5DCacheAError	[55:0]	Reserved
Mp5DCacheBError	[55:0]	Reserved
Mp5DTagAError	[55:0]	Reserved
Mp5DTagBError	[55:0]	Reserved
Mp5ICacheAError	[55:0]	Reserved
Mp5ICacheBError	[55:0]	Reserved
Mp5ITagAError	[55:0]	Reserved
Mp5ITagBError	[55:0]	Reserved

MSR0000_043F...MSRC000_20F3 [MP5 Machine Check Miscellaneous 0] (MCA::MP5::MCA_MISC0_MP5)

Log miscellaneous information associated with errors.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F3

_ccd0_instMP5MP5_n0_aliasMSRLEGACY; MSR0000_043F

_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F3

_ccd10_instMP5MP5_n10_aliasMSRLEGACY; MSR0000_043F

_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F3

_ccd11_instMP5MP5_n11_aliasMSRLEGACY; MSR0000_043F

_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F3

_ccd1_instMP5MP5_n1_aliasMSRLEGACY; MSR0000_043F

_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F3

_ccd2_instMP5MP5_n2_aliasMSRLEGACY; MSR0000_043F

_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F3

_ccd3_instMP5MP5_n3_aliasMSRLEGACY; MSR0000_043F

_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F3

_ccd4_instMP5MP5_n4_aliasMSRLEGACY; MSR0000_043F

_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F3

_ccd5_instMP5MP5_n5_aliasMSRLEGACY; MSR0000_043F

_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F3

_ccd6_instMP5MP5_n6_aliasMSRLEGACY; MSR0000_043F

_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F3

_ccd7_instMP5MP5_n7_aliasMSRLEGACY; MSR0000_043F

_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F3

_ccd8_instMP5MP5_n8_aliasMSRLEGACY; MSR0000_043F

_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F3

_ccd9_instMP5MP5_n9_aliasMSRLEGACY; MSR0000_043F

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MP5::MCA_MISC0_MP5[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MP5::MCA_MISC0_MP5[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MP5::MCA_MISC0_MP5[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MP5::MCA_MISC0_MP5[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated.

	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::MP5::MCA_MISC0_MP5[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported.
	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::MP5::MCA_MISC0_MP5[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_20F4 [MP5 Machine Check Configuration] (MCA::MP5::MCA_CONFIG_MP5)

Reset: 0000_0002_0000_0123h.

Controls configuration of the associated machine check bank.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F4

_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F4

_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F4

_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F4

_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F4

_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F4

_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F4

_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F4

_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F4

_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F4

_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F4

_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable. Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::MP5::MCA_STATUS_MP5, MCA::MP5::MCA_ADDR_MP5, MCA::MP5::MCA_MISC0_MP5, and MCA::MP5::MCA_SYND_MP5 in addition to MCA::MP5::MCA_TRANSYND_MP5 and MCA::MP5::MCA_TRANSADDR_MP5. 0=Only log transparent errors in MCA::MP5::MCA_TRANSYND_MP5 and MCA::MP5::MCA_TRANSADDR_MP5.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::MP5::MCA_CONFIG_MP5[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::MP5::MCA_CONFIG_MP5[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::MP5::MCA_CONFIG_MP5[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. MCA_DESTAT and MCA_DEADDR are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::MP5::MCA_CONFIG_MP5[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::MP5::MCA_MISC0_MP5[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::MP5::MCA_STATUS_MP5[TCC] is present.

MSRC000_20F5 [MP5 IP Identification] (MCA::MP5::MCA_IPID_MP5)

Reset: 0002_0001_0000_0000h.

The MCA::MP5::MCA_IPID_MP5 register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F5_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F5_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F5_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F5_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F5_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F5_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F5_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F5_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F5_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F5_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F5_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F5_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F5_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F5

Bits	Description
63:48	McaType . Read-only. Reset: 0002h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 001h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: <u>_ccd0_instMP5MP5_n0_aliasMSR</u> : 3043_0400h
	Init: <u>_ccd1_instMP5MP5_n1_aliasMSR</u> : 3243_0400h
	Init: <u>_ccd2_instMP5MP5_n2_aliasMSR</u> : 3443_0400h
	Init: <u>_ccd3_instMP5MP5_n3_aliasMSR</u> : 3643_0400h
	Init: <u>_ccd4_instMP5MP5_n4_aliasMSR</u> : 3843_0400h
	Init: <u>_ccd5_instMP5MP5_n5_aliasMSR</u> : 3A43_0400h
	Init: <u>_ccd6_instMP5MP5_n6_aliasMSR</u> : 3C43_0400h
	Init: <u>_ccd7_instMP5MP5_n7_aliasMSR</u> : 3E43_0400h
	Init: <u>_ccd8_instMP5MP5_n8_aliasMSR</u> : 4A43_0400h
	Init: <u>_ccd9_instMP5MP5_n9_aliasMSR</u> : 4C43_0400h
	Init: <u>_ccd10_instMP5MP5_n10_aliasMSR</u> : 4E43_0400h
	Init: <u>_ccd11_instMP5MP5_n11_aliasMSR</u> : 5043_0400h

MSRC000_20F6 [MP5 Machine Check Syndrome] (MCA::MP5::MCA_SYND_MP5)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::MP5::MCA_STATUS_MP5 [Thread 0](#)

_ccd0_instMP5MP5_n0_aliasMSR; MSRC000_20F6

_ccd10_instMP5MP5_n10_aliasMSR; MSRC000_20F6

_ccd11_instMP5MP5_n11_aliasMSR; MSRC000_20F6

_ccd1_instMP5MP5_n1_aliasMSR; MSRC000_20F6

_ccd2_instMP5MP5_n2_aliasMSR; MSRC000_20F6

_ccd3_instMP5MP5_n3_aliasMSR; MSRC000_20F6

_ccd4_instMP5MP5_n4_aliasMSR; MSRC000_20F6

_ccd5_instMP5MP5_n5_aliasMSR; MSRC000_20F6

_ccd6_instMP5MP5_n6_aliasMSR; MSRC000_20F6

_ccd7_instMP5MP5_n7_aliasMSR; MSRC000_20F6

_ccd8_instMP5MP5_n8_aliasMSR; MSRC000_20F6

_ccd9_instMP5MP5_n9_aliasMSR; MSRC000_20F6

Bits	Description
63:33	Reserved.
32	Syndromes. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::MP5::MCA_STATUS_MP5. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::MP5::MCA_SYND_MP5[Length]. The Syndrome field is only valid when MCA::MP5::MCA_SYND_MP5[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::MP5::MCA_SYND_MP5. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::MP5::MCA_SYND_MP5[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::MP5::MCA_SYND_MP5. For example, a syndrome length of 9 means that MCA::MP5::MCA_SYND_MP5[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 99 [MCA_SYND_MP5].

Table 99: MCA_SYND_MP5

Error Type	Bits	Description
Mp5HighSramError	[17:15]	Reserved
	[14:0]	Reserved
Mp5LowSramError	[17:15]	Reserved
	[14:0]	Reserved
Mp5DCacheAError	[17:8]	Reserved
	[7:0]	Reserved
Mp5DCacheBError	[17:8]	Reserved
	[7:0]	Reserved
Mp5DTagAError	[17:7]	Reserved
	[6:0]	Reserved
Mp5DTagBError	[17:7]	Reserved
	[6:0]	Reserved
Mp5ICacheAError	[17:8]	Reserved
	[7:0]	Reserved
Mp5ICacheBError	[17:8]	Reserved
	[7:0]	Reserved
Mp5ITagAError	[17:6]	Reserved
	[5:0]	Reserved

Mp5ITagBError	[17:6]	Reserved
	[5:0]	Reserved

MSRC001_040F [MP5 Machine Check Control Mask] (MCA::MP5::MCA_CTL_MASK_MP5)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC001_040F

_ccd10_instMP5MP5_n10_aliasMSR; MSRC001_040F

_ccd11_instMP5MP5_n11_aliasMSR; MSRC001_040F

_ccd1_instMP5MP5_n1_aliasMSR; MSRC001_040F

_ccd2_instMP5MP5_n2_aliasMSR; MSRC001_040F

_ccd3_instMP5MP5_n3_aliasMSR; MSRC001_040F

_ccd4_instMP5MP5_n4_aliasMSR; MSRC001_040F

_ccd5_instMP5MP5_n5_aliasMSR; MSRC001_040F

_ccd6_instMP5MP5_n6_aliasMSR; MSRC001_040F

_ccd7_instMP5MP5_n7_aliasMSR; MSRC001_040F

_ccd8_instMP5MP5_n8_aliasMSR; MSRC001_040F

_ccd9_instMP5MP5_n9_aliasMSR; MSRC001_040F

Bits	Description
63:10	Reserved.
9	Mp5ITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
8	Mp5ITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
7	Mp5ICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
6	Mp5ICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
5	Mp5DTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
4	Mp5DTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
3	Mp5DCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
2	Mp5DCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
1	Mp5LowSramError. Read-write. Reset: 0. Low SRAM ECC or parity error.
0	Mp5HighSramError. Read-write. Reset: 0. High SRAM ECC or parity error.

MSRC001_050F [MP5 Machine Check Transparent Error Syndrome] (MCA::MP5::MCA_TRANSSYND_MP5)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_ccd0_instMP5MP5_n0_aliasMSR; MSRC001_050F_ccd10_instMP5MP5_n10_aliasMSR; MSRC001_050F_ccd11_instMP5MP5_n11_aliasMSR; MSRC001_050F_ccd1_instMP5MP5_n1_aliasMSR; MSRC001_050F_ccd2_instMP5MP5_n2_aliasMSR; MSRC001_050F_ccd3_instMP5MP5_n3_aliasMSR; MSRC001_050F_ccd4_instMP5MP5_n4_aliasMSR; MSRC001_050F_ccd5_instMP5MP5_n5_aliasMSR; MSRC001_050F_ccd6_instMP5MP5_n6_aliasMSR; MSRC001_050F_ccd7_instMP5MP5_n7_aliasMSR; MSRC001_050F_ccd8_instMP5MP5_n8_aliasMSR; MSRC001_050F_ccd9_instMP5MP5_n9_aliasMSR; MSRC001_050F

Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::MP5::MCA_STATUS_MP5. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::MP5::MCA_SYND_MP5[Length]. The Syndrome field is only valid when MCA::MP5::MCA_SYND_MP5[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::MP5::MCA_TRANSSYND_MP5. All errors writing into MCA::MP5::MCA_SYND_MP5 should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::MP5::MCA_SYND_MP5. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::MP5::MCA_SYND_MP5[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::MP5::MCA_SYND_MP5. For example, a syndrome length of 9 means that MCA::MP5::MCA_SYND_MP5[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

MSRC001_060F [MP5 Machine Check Transparent Error Address] (MCA::MP5::MCA_TRANSADDR_MP5)

Reset: Cold,0000_0000_0000_0000h.

MCA::MP5::MCA_TRANSADDR_MP5 stores the address associated with the error in MCA::MP5::MCA_STATUS_MP5. The register is only meaningful if MCA::MP5::MCA_STATUS_MP5[Val]=1 and MCA::MP5::MCA_STATUS_MP5[AddrV]=1. The lowest valid bit of the address is defined by MCA::MP5::MCA_STATUS_MP5[AddrLsb].

_ccd0_instMP5MP5_n0_aliasMSR; MSRC001_060F

_ccd10_instMP5MP5_n10_aliasMSR; MSRC001_060F

_ccd11_instMP5MP5_n11_aliasMSR; MSRC001_060F

_ccd1_instMP5MP5_n1_aliasMSR; MSRC001_060F

_ccd2_instMP5MP5_n2_aliasMSR; MSRC001_060F

_ccd3_instMP5MP5_n3_aliasMSR; MSRC001_060F

_ccd4_instMP5MP5_n4_aliasMSR; MSRC001_060F

_ccd5_instMP5MP5_n5_aliasMSR; MSRC001_060F

_ccd6_instMP5MP5_n6_aliasMSR; MSRC001_060F

_ccd7_instMP5MP5_n7_aliasMSR; MSRC001_060F

_ccd8_instMP5MP5_n8_aliasMSR; MSRC001_060F

_ccd9_instMP5MP5_n9_aliasMSR; MSRC001_060F

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::MP5::MCA_STATUS_MP5. The lowest-order valid bit of the address is specified in MCA::MP5::MCA_STATUS_MP5[AddrLsb].

3.2.5.15 NBIO**MSR0000_0464...MSRC000_2190 [NBIO Machine Check Control] (MCA::NBIO::MCA_CTL_NBIO)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::NBIO::MCA_CTL_NBIO register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instIOHC_iohub0_n0_nbio0_aliasMSR; MSRC000_2190

_instIOHC_iohub0_n0_nbio0_aliasMSRLEGACY; MSR0000_0464

_instIOHC_iohub0_n2_nbio1_aliasMSR; MSRC000_2190

_instIOHC_iohub0_n2_nbio1_aliasMSRLEGACY; MSR0000_0464

_instIOHC_iohub1_n1_nbio0_aliasMSR; MSRC000_2190

_instIOHC_iohub1_n1_nbio0_aliasMSRLEGACY; MSR0000_0464

_instIOHC_iohub1_n3_nbio1_aliasMSR; MSRC000_2190

_instIOHC_iohub1_n3_nbio1_aliasMSRLEGACY; MSR0000_0464

Bits	Description
63:6	Reserved.
5	Int_ErrEvent. Read-write. Reset: 0. Internal system fatal error event was detected.
4	IOHC_Internal_Poison. Read-write. Reset: 0. IOHC Internal Poison Error. Poison data was sent to an internal IOHC client.
3	Egress_Poison. Read-write. Reset: 0. SDP Egress Poison Error. Poison was propagated to an egress port.
2	Ext_ErrEvent. Read-write. Reset: 0. External SDP ErrEvent error. A system fatal error event from an SDP interface was detected.
1	PCIE_Sideband. Read-write. Reset: 0. PCIE error. A PCIE error was logged in a PCIE root port.
0	EccParityError. Read-write. Reset: 0. ECC or Parity error. An SRAM ECC or parity error was detected.

MSR0000_0465...MSRC000_2191 [NBIO Machine Check Status] (MCA::NBIO::MCA_STATUS_NBIO)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instIOHC_iohub0_n0_nbio0_aliasMSR; MSRC000_2191

_instIOHC_iohub0_n0_nbio0_aliasMSRLEGACY; MSR0000_0465

_instIOHC_iohub0_n2_nbio1_aliasMSR; MSRC000_2191

_instIOHC_iohub0_n2_nbio1_aliasMSRLEGACY; MSR0000_0465

_instIOHC_iohub1_n1_nbio0_aliasMSR; MSRC000_2191

_instIOHC_iohub1_n1_nbio0_aliasMSRLEGACY; MSR0000_0465

_instIOHC_iohub1_n3_nbio1_aliasMSR; MSRC000_2191

_instIOHC_iohub1_n3_nbio1_aliasMSRLEGACY; MSR0000_0465

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::NBIO::MCA_CTL_NBIO. This bit is a copy of bit in MCA::NBIO::MCA_CTL_NBIO for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::NBIO::MCA_MISC0_NBIO. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::NBIO::MCA_ADDR_NBIO contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::NBIO::MCA_STATUS_NBIO[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::NBIO::MCA_SYND_NBIO. If MCA::NBIO::MCA_SYND_NBIO[ErrorPriority] is the same as the priority of the error in MCA::NBIO::MCA_STATUS_NBIO, then the information in MCA::NBIO::MCA_SYND_NBIO is associated with the error in MCA::NBIO::MCA_STATUS_NBIO. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::NBIO::MCA_STATUS_NBIO[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC . Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC . Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred . Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison . Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub . Reset: Cold,0. 1=The error was the result of a scrub operation.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId . Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb . Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::NBIO::MCA_ADDR_NBIO[ErrorAddr]. A value of 0 indicates that MCA::NBIO::MCA_ADDR_NBIO[55:0] contains a valid byte address. A value of 6 indicates that MCA::NBIO::MCA_ADDR_NBIO[55:6] contains a valid cache line address and that MCA::NBIO::MCA_ADDR_NBIO[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::NBIO::MCA_ADDR_NBIO[55:12] contain a valid 4KB memory page and that MCA::NBIO::MCA_ADDR_NBIO[11:0] should be ignored by error handling software.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt . Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::NBIO::MCA_CTL_NBIO enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode . Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 100: MCA_STATUS_NBIO

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
EccParityError	0x0	0/1	0/1	0/1	0/1	0	0
PCIE_Sideband	0x1	0/1	0/1	0/1	0/1	0	0

Ext_ErrEvent	0x2	1	1	1	0	0	0
Egress_Poison	0x3	0/1	0/1	0/1	0/1	0	0
IOHC_Internal_Poison	0x4	1	1	1	0	0	0
Int_ErrEvent	0x5	1	1	1	0	0	0

MSR0000_0466...MSRC000_2192 [NBIO Machine Check Address] (MCA::NBIO::MCA_ADDR_NBIO)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::NBIO::MCA_ADDR_NBIO stores an address and other information associated with the error in MCA::NBIO::MCA_STATUS_NBIO. The register is only meaningful if MCA::NBIO::MCA_STATUS_NBIO[Val]=1 and MCA::NBIO::MCA_STATUS_NBIO[AddrV]=1.

_instIOHC_iohub0_n0_nbio0_aliasMSR; MSRC000_2192_instIOHC_iohub0_n0_nbio0_aliasMSRLEGACY; MSR0000_0466_instIOHC_iohub0_n2_nbio1_aliasMSR; MSRC000_2192_instIOHC_iohub0_n2_nbio1_aliasMSRLEGACY; MSR0000_0466_instIOHC_iohub1_n1_nbio0_aliasMSR; MSRC000_2192_instIOHC_iohub1_n1_nbio0_aliasMSRLEGACY; MSR0000_0466_instIOHC_iohub1_n3_nbio1_aliasMSR; MSRC000_2192_instIOHC_iohub1_n3_nbio1_aliasMSRLEGACY; MSR0000_0466**Bits Description**

63:0 **ErrorAddr.** Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::NBIO::MCA_STATUS_NBIO.

Table 101: MCA_ADDR_NBIO

Error Type	Bits	Description
EccParityError	[56:0]	Reserved
PCIE_Sideband	[56:0]	Reserved
Ext_ErrEvent	[56:0]	Reserved
Egress_Poison	[56:0]	Reserved
IOHC_Internal_Poison	[56:0]	Reserved
Int_ErrEvent	[56:0]	Reserved

MSR0000_0467...MSRC000_2193 [NBIO Machine Check Miscellaneous 0] (MCA::NBIO::MCA_MISC0_NBIO)

Log miscellaneous information associated with errors.

[_instIOHC_iohub0_n0_nbio0_aliasMSR; MSRC000_2193](#)[_instIOHC_iohub0_n0_nbio0_aliasMSRLEGACY; MSR0000_0467](#)[_instIOHC_iohub0_n2_nbio1_aliasMSR; MSRC000_2193](#)[_instIOHC_iohub0_n2_nbio1_aliasMSRLEGACY; MSR0000_0467](#)[_instIOHC_iohub1_n1_nbio0_aliasMSR; MSRC000_2193](#)[_instIOHC_iohub1_n1_nbio0_aliasMSRLEGACY; MSR0000_0467](#)[_instIOHC_iohub1_n3_nbio1_aliasMSR; MSRC000_2193](#)[_instIOHC_iohub1_n3_nbio1_aliasMSRLEGACY; MSR0000_0467](#)

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIO::MCA_MISC0_NBIO[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIO::MCA_MISC0_NBIO[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIO::MCA_MISC0_NBIO[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIO::MCA_MISC0_NBIO[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIO::MCA_MISC0_NBIO[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIO::MCA_MISC0_NBIO[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2194 [NBIO Machine Check Configuration] (MCA::NBIO::MCA_CONFIG_NBIO)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

[_instIOHC_iohub0_n0_nbio0_aliasMSR](#); MSRC000_2194[_instIOHC_iohub0_n2_nbio1_aliasMSR](#); MSRC000_2194[_instIOHC_iohub1_n1_nbio0_aliasMSR](#); MSRC000_2194[_instIOHC_iohub1_n3_nbio1_aliasMSR](#); MSRC000_2194

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::NBIO::MCA_STATUS_NBIO and MCA::NBIO::MCA_ADDR_NBIO in addition to MCA::NBIO::MCA_DESTAT_NBIO. 0=Only log deferred errors in MCA::NBIO::MCA_DESTAT_NBIO. This bit does not affect logging of deferred errors in MCA::NBIO::MCA_SYND_NBIO, MCA::NBIO::MCA_MISC0_NBIO.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::NBIO::MCA_STATUS_NBIO, MCA::NBIO::MCA_ADDR_NBIO, MCA::NBIO::MCA_MISC0_NBIO, and MCA::NBIO::MCA_SYND_NBIO.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::NBIO::MCA_CONFIG_NBIO[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::NBIO::MCA_CONFIG_NBIO[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::NBIO::MCA_CONFIG_NBIO[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::NBIO::MCA_CONFIG_NBIO[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::NBIO::MCA_DESTAT_NBIO are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::NBIO::MCA_MISC0_NBIO[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::NBIO::MCA_STATUS_NBIO[TCC] is present.

MSRC000_2195 [NBIO IP Identification] (MCA::NBIO::MCA_IPID_NBIO)

Reset: 0000_0018_0000_0000h.

The MCA::NBIO::MCA_IPID_NBIO register is used by software to determine what IP type and revision is associated with the MCA bank.

[_instIOHC_iohub0_n0_nbio0_aliasMSR](#); MSRC000_2195[_instIOHC_iohub0_n2_nbio1_aliasMSR](#); MSRC000_2195[_instIOHC_iohub1_n1_nbio0_aliasMSR](#); MSRC000_2195[_instIOHC_iohub1_n3_nbio1_aliasMSR](#); MSRC000_2195

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 018h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instIOHC_iohub0_n0_nbio0_aliasMSR : 13B1_7000h
	Init: _instIOHC_iohub0_n2_nbio1_aliasMSR : 13D1_7000h
	Init: _instIOHC_iohub1_n1_nbio0_aliasMSR : 13C1_7000h
	Init: _instIOHC_iohub1_n3_nbio1_aliasMSR : 13E1_7000h

MSRC000_2196 [NBIO Machine Check Syndrome] (MCA::NBIO::MCA_SYND_NBIO)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::NBIO::MCA_STATUS_NBIO [Thread 0](#)

[_instIOHC_iohub0_n0_nbio0_aliasMSR](#); MSRC000_2196[_instIOHC_iohub0_n2_nbio1_aliasMSR](#); MSRC000_2196[_instIOHC_iohub1_n1_nbio0_aliasMSR](#); MSRC000_2196[_instIOHC_iohub1_n3_nbio1_aliasMSR](#); MSRC000_2196

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::NBIO::MCA_STATUS_NBIO. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::NBIO::MCA_SYND_NBIO[Length]. The Syndrome field is only valid when MCA::NBIO::MCA_SYND_NBIO[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::NBIO::MCA_SYND_NBIO. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::NBIO::MCA_SYND_NBIO[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::NBIO::MCA_SYND_NBIO. For example, a syndrome length of 9 means that MCA::NBIO::MCA_SYND_NBIO[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 102 [MCA_SYND_NBIO].

Table 102: MCA_SYND_NBIO

Error Type	Bits	Description
EccParityError	[17:5] [4:0]	Group ID Structure ID
PCIE_Sideband	[5:0]	EgressPortNum
Ext_ErrEvent	[3:0]	IOHC port number 0:SDF 1:PCIE0 2:PCIE1 3:NBIF0 4: IOAGR 5:L2IMU
Egress_Poison	[5:0]	Egress Port Number

IOHC_Internal_Poison	[0]	0:CfgMaster 1:TrapClient
Int_ErrEvent	[0]	0: SW syncflood (APML or Private) 1: syncflood pin

MSRC000_2198 [NBIO Machine Check Deferred Error Status] (MCA::NBIO::MCA_DESTAT_NBIO)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

[_instIOHC_iohub0_n0_nbio0_aliasMSR](#); MSRC000_2198[_instIOHC_iohub0_n2_nbio1_aliasMSR](#); MSRC000_2198[_instIOHC_iohub1_n1_nbio0_aliasMSR](#); MSRC000_2198[_instIOHC_iohub1_n3_nbio1_aliasMSR](#); MSRC000_2198

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. Reserved.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::NBIO::MCA_SYND_NBIO. If MCA::NBIO::MCA_SYND_NBIO[ErrorPriority] is the same as the priority of the error in MCA::NBIO::MCA_STATUS_NBIO, then the information in MCA::NBIO::MCA_SYND_NBIO is associated with the error in MCA::NBIO::MCA_DESTAT_NBIO.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::NBIO::MCA_ADDR_NBIO[ErrorAddr]. A value of 0 indicates that MCA::NBIO::MCA_ADDR_NBIO[55:0] contains a valid byte address. A value of 6 indicates that MCA::NBIO::MCA_ADDR_NBIO[55:6] contains a valid cache line address and that MCA::NBIO::MCA_ADDR_NBIO[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::NBIO::MCA_ADDR_NBIO[55:12] contain a valid 4KB memory page and that MCA::NBIO::MCA_ADDR_NBIO[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC001_0419 [NBIO Machine Check Control Mask] (MCA::NBIO::MCA_CTL_MASK_NBIO)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

[_instIOHC_iohub0_n0_nbio0_aliasMSR](#); MSRC001_0419[_instIOHC_iohub0_n2_nbio1_aliasMSR](#); MSRC001_0419[_instIOHC_iohub1_n1_nbio0_aliasMSR](#); MSRC001_0419[_instIOHC_iohub1_n3_nbio1_aliasMSR](#); MSRC001_0419**Bits Description**

63:6 Reserved.

5 **Int_ErrEvent**. Read-write. Reset: 0. Init: BIOS,1. Internal system fatal error event was detected.4 **IOHC_Internal_Poison**. Read-write. Reset: 0. [IOHC](#) Internal Poison Error. Poison data was sent to an internal IOHC client.3 **Egress_Poison**. Read-write. Reset: 0. SDP Egress Poison Error. Poison was propagated to an egress port.2 **Ext_ErrEvent**. Read-write. Reset: 0. Init: BIOS,1. External SDP ErrEvent error. A system fatal error event from an SDP interface was detected.1 **PCIE_Sideband**. Read-write. Reset: 0. Init: BIOS,1. PCIe error. A [PCIe](#) error was logged in a PCIe root port.0 **EccParityError**. Read-write. Reset: 0. ECC or Parity error. An SRAM ECC or parity error was detected.**3.2.5.16 PCIE****MSR0000_0458...MSRC000_21B0 [PCIE Machine Check Control] (MCA::PCIE::MCA_CTL_PCIE)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::PCIE::MCA_CTL_PCIE register must be enabled by the corresponding enable bit in Core::X86::Msrr::[MCG_CTL](#). Does not affect error detection, correction, or logging.[_instPCIE0PCIE_n0_nbio0_aliasMSR](#); MSRC000_21A0[_instPCIE0PCIE_n0_nbio0_aliasMSRLEGACY](#); MSR0000_0468[_instPCIE0PCIE_n2_nbio1_aliasMSR](#); MSRC000_21A0[_instPCIE0PCIE_n2_nbio1_aliasMSRLEGACY](#); MSR0000_0468[_instPCIE1PCIE_n4_nbio0_aliasMSR](#); MSRC000_21A0[_instPCIE1PCIE_n4_nbio0_aliasMSRLEGACY](#); MSR0000_0468[_instPCIE1PCIE_n6_nbio1_aliasMSR](#); MSRC000_21A0[_instPCIE1PCIE_n6_nbio1_aliasMSRLEGACY](#); MSR0000_0468[_instPCIE2PCIE_n5_nbio0_aliasMSR](#); MSRC000_21B0[_instPCIE2PCIE_n5_nbio0_aliasMSRLEGACY](#); MSR0000_046C[_instPCIE2PCIE_n7_nbio1_aliasMSR](#); MSRC000_21B0[_instPCIE2PCIE_n7_nbio1_aliasMSRLEGACY](#); MSR0000_046C[_instPCIE3PCIE_n1_nbio0_aliasMSR](#); MSRC000_21B0[_instPCIE3PCIE_n1_nbio0_aliasMSRLEGACY](#); MSR0000_046C[_instPCIE3PCIE_n3_nbio1_aliasMSR](#); MSRC000_21B0[_instPCIE3PCIE_n3_nbio1_aliasMSRLEGACY](#); MSR0000_046C[_instPCIE4PCIE_n8_nbio0_aliasMSR](#); MSRC000_2160[_instPCIE4PCIE_n8_nbio0_aliasMSRLEGACY](#); MSR0000_0458[_instPCIE4PCIE_n9_nbio1_aliasMSR](#); MSRC000_2160[_instPCIE4PCIE_n9_nbio1_aliasMSRLEGACY](#); MSR0000_0458**Bits Description**

63:1 Reserved.

0 **SDP_PARITY_ERR_LOG**. Read-write. Reset: 0. SDP Data Parity Error logging.

MSR0000_0459...MSRC000_21B1 [PCIE Machine Check Status] (MCA::PCIE::MCA_STATUS_PCIE)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instPCIE0PCIE_n0_nbio0_aliasMSR; MSRC000_21A1

_instPCIE0PCIE_n0_nbio0_aliasMSRLEGACY; MSR0000_0469

_instPCIE0PCIE_n2_nbio1_aliasMSR; MSRC000_21A1

_instPCIE0PCIE_n2_nbio1_aliasMSRLEGACY; MSR0000_0469

_instPCIE1PCIE_n4_nbio0_aliasMSR; MSRC000_21A1

_instPCIE1PCIE_n4_nbio0_aliasMSRLEGACY; MSR0000_0469

_instPCIE1PCIE_n6_nbio1_aliasMSR; MSRC000_21A1

_instPCIE1PCIE_n6_nbio1_aliasMSRLEGACY; MSR0000_0469

_instPCIE2PCIE_n5_nbio0_aliasMSR; MSRC000_21B1

_instPCIE2PCIE_n5_nbio0_aliasMSRLEGACY; MSR0000_046D

_instPCIE2PCIE_n7_nbio1_aliasMSR; MSRC000_21B1

_instPCIE2PCIE_n7_nbio1_aliasMSRLEGACY; MSR0000_046D

_instPCIE3PCIE_n1_nbio0_aliasMSR; MSRC000_21B1

_instPCIE3PCIE_n1_nbio0_aliasMSRLEGACY; MSR0000_046D

_instPCIE3PCIE_n3_nbio1_aliasMSR; MSRC000_21B1

_instPCIE3PCIE_n3_nbio1_aliasMSRLEGACY; MSR0000_046D

_instPCIE4PCIE_n8_nbio0_aliasMSR; MSRC000_2161

_instPCIE4PCIE_n8_nbio0_aliasMSRLEGACY; MSR0000_0459

_instPCIE4PCIE_n9_nbio1_aliasMSR; MSRC000_2161

_instPCIE4PCIE_n9_nbio1_aliasMSRLEGACY; MSR0000_0459

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::PCIE::MCA_CTL_PCIE. This bit is a copy of bit in MCA::PCIE::MCA_CTL_PCIE for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::PCIE::MCA_MISC0_PCIE. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::PCIE::MCA_ADDR_PCIE contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::PCIE::MCA_STATUS_PCIE[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::PCIE::MCA_SYND_PCIE. If MCA::PCIE::MCA_SYND_PCIE[ErrorPriority] is the same as the priority of the error in MCA::PCIE::MCA_STATUS_PCIE, then the information in MCA::PCIE::MCA_SYND_PCIE is associated with the error in MCA::PCIE::MCA_STATUS_PCIE.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::PCIE::MCA_STATUS_PCIE[UC]=0.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PCIE::MCA_ADDR_PCIE[ErrorAddr]. A value of 0 indicates that MCA::PCIE::MCA_ADDR_PCIE[55:0] contains a valid byte address. A value of 6 indicates that MCA::PCIE::MCA_ADDR_PCIE[55:6] contains a valid cache line address and that MCA::PCIE::MCA_ADDR_PCIE[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PCIE::MCA_ADDR_PCIE[55:12] contain a valid 4KB memory page and that MCA::PCIE::MCA_ADDR_PCIE[11:0] should be ignored by error handling software.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::PCIE::MCA_CTL_PCIE enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.

AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 103: MCA_STATUS_PCIE

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
SDP_PARITY_ERR_LOG	0x0	0	0	0	1	0	1

MSR0000_045A...MSRC000_21B2 [PCIE Machine Check Address] (MCA::PCIE::MCA_ADDR_PCIE)

Reset: Cold,0000_0000_0000_0000h.

MCA::PCIE::MCA_ADDR_PCIE stores an address and other information associated with the error in MCA::PCIE::MCA_STATUS_PCIE. The register is only meaningful if MCA::PCIE::MCA_STATUS_PCIE[Val]=1 and MCA::PCIE::MCA_STATUS_PCIE[AddrV]=1.

[_instPCIE0PCIE_n0_nbio0_aliasMSR; MSRC000_21A2](#)[_instPCIE0PCIE_n0_nbio0_aliasMSRLEGACY; MSR0000_046A](#)[_instPCIE0PCIE_n2_nbio1_aliasMSR; MSRC000_21A2](#)[_instPCIE0PCIE_n2_nbio1_aliasMSRLEGACY; MSR0000_046A](#)[_instPCIE1PCIE_n4_nbio0_aliasMSR; MSRC000_21A2](#)[_instPCIE1PCIE_n4_nbio0_aliasMSRLEGACY; MSR0000_046A](#)[_instPCIE1PCIE_n6_nbio1_aliasMSR; MSRC000_21A2](#)[_instPCIE1PCIE_n6_nbio1_aliasMSRLEGACY; MSR0000_046A](#)[_instPCIE2PCIE_n5_nbio0_aliasMSR; MSRC000_21B2](#)[_instPCIE2PCIE_n5_nbio0_aliasMSRLEGACY; MSR0000_046E](#)[_instPCIE2PCIE_n7_nbio1_aliasMSR; MSRC000_21B2](#)[_instPCIE2PCIE_n7_nbio1_aliasMSRLEGACY; MSR0000_046E](#)[_instPCIE3PCIE_n1_nbio0_aliasMSR; MSRC000_21B2](#)[_instPCIE3PCIE_n1_nbio0_aliasMSRLEGACY; MSR0000_046E](#)[_instPCIE3PCIE_n3_nbio1_aliasMSR; MSRC000_21B2](#)[_instPCIE3PCIE_n3_nbio1_aliasMSRLEGACY; MSR0000_046E](#)[_instPCIE4PCIE_n8_nbio0_aliasMSR; MSRC000_2162](#)[_instPCIE4PCIE_n8_nbio0_aliasMSRLEGACY; MSR0000_045A](#)[_instPCIE4PCIE_n9_nbio1_aliasMSR; MSRC000_2162](#)[_instPCIE4PCIE_n9_nbio1_aliasMSRLEGACY; MSR0000_045A](#)

Bits	Description
------	-------------

63:56 Reserved.

55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::PCIE::MCA_STATUS_PCIE. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].
------	---

Table 104: MCA_ADDR_PCIE

Error Type	Bits	Description
SDP_PARITY_ERR_LOG	[63:6]	MST_PCIE_ReqAddr[63:6]
	[5:0]	Reserved

MSR0000_045B...MSRC000_21B3 [PCIE Machine Check Miscellaneous 0] (MCA::PCIE::MCA_MISC0_PCIE)

Log miscellaneous information associated with errors.

[_instPCIE0PCIE_n0_nbio0_aliasMSR](#); [MSRC000_21A3](#)[_instPCIE0PCIE_n0_nbio0_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instPCIE0PCIE_n2_nbio1_aliasMSR](#); [MSRC000_21A3](#)[_instPCIE0PCIE_n2_nbio1_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instPCIE1PCIE_n4_nbio0_aliasMSR](#); [MSRC000_21A3](#)[_instPCIE1PCIE_n4_nbio0_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instPCIE1PCIE_n6_nbio1_aliasMSR](#); [MSRC000_21A3](#)[_instPCIE1PCIE_n6_nbio1_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instPCIE2PCIE_n5_nbio0_aliasMSR](#); [MSRC000_21B3](#)[_instPCIE2PCIE_n5_nbio0_aliasMSRLEGACY](#); [MSR0000_046F](#)[_instPCIE2PCIE_n7_nbio1_aliasMSR](#); [MSRC000_21B3](#)[_instPCIE2PCIE_n7_nbio1_aliasMSRLEGACY](#); [MSR0000_046F](#)[_instPCIE3PCIE_n1_nbio0_aliasMSR](#); [MSRC000_21B3](#)[_instPCIE3PCIE_n1_nbio0_aliasMSRLEGACY](#); [MSR0000_046F](#)[_instPCIE3PCIE_n3_nbio1_aliasMSR](#); [MSRC000_21B3](#)[_instPCIE3PCIE_n3_nbio1_aliasMSRLEGACY](#); [MSR0000_046F](#)[_instPCIE4PCIE_n8_nbio0_aliasMSR](#); [MSRC000_2163](#)[_instPCIE4PCIE_n8_nbio0_aliasMSRLEGACY](#); [MSR0000_045B](#)[_instPCIE4PCIE_n9_nbio1_aliasMSR](#); [MSRC000_2163](#)[_instPCIE4PCIE_n9_nbio1_aliasMSRLEGACY](#); [MSR0000_045B](#)

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PCIE::MCA_MISC0_PCIE[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PCIE::MCA_MISC0_PCIE[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PCIE::MCA_MISC0_PCIE[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PCIE::MCA_MISC0_PCIE[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PCIE::MCA_MISC0_PCIE[Locked]) ? Read-write : Read-only.
47:44	Reserved.

43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported.
	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::PCIE::MCA_MISC0_PCIE[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21[6...B]4 [PCIE Machine Check Configuration] (MCA::PCIE::MCA_CONFIG_PCIE)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

[_instPCIE0PCIE_n0_nbio0_aliasMSR](#); MSRC000_21A4[_instPCIE0PCIE_n2_nbio1_aliasMSR](#); MSRC000_21A4[_instPCIE1PCIE_n4_nbio0_aliasMSR](#); MSRC000_21A4[_instPCIE1PCIE_n6_nbio1_aliasMSR](#); MSRC000_21A4[_instPCIE2PCIE_n5_nbio0_aliasMSR](#); MSRC000_21B4[_instPCIE2PCIE_n7_nbio1_aliasMSR](#); MSRC000_21B4[_instPCIE3PCIE_n1_nbio0_aliasMSR](#); MSRC000_21B4[_instPCIE3PCIE_n3_nbio1_aliasMSR](#); MSRC000_21B4[_instPCIE4PCIE_n8_nbio0_aliasMSR](#); MSRC000_2164[_instPCIE4PCIE_n9_nbio1_aliasMSR](#); MSRC000_2164

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::PCIE::MCA_STATUS_PCIE and MCA::PCIE::MCA_ADDR_PCIE in addition to MCA::PCIE::MCA_DESTAT_PCIE and MCA::PCIE::MCA_DEADDR_PCIE. 0=Only log deferred errors in MCA::PCIE::MCA_DESTAT_PCIE and MCA::PCIE::MCA_DEADDR_PCIE. This bit does not affect logging of deferred errors in MCA::PCIE::MCA_SYND_PCIE, MCA::PCIE::MCA_MISC0_PCIE.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::PCIE::MCA_STATUS_PCIE, MCA::PCIE::MCA_ADDR_PCIE, MCA::PCIE::MCA_MISC0_PCIE, and MCA::PCIE::MCA_SYND_PCIE.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::PCIE::MCA_CONFIG_PCIE[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::PCIE::MCA_CONFIG_PCIE[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::PCIE::MCA_CONFIG_PCIE[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::PCIE::MCA_CONFIG_PCIE[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::PCIE::MCA_DESTAT_PCIE and MCA::PCIE::MCA_DEADDR_PCIE are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::PCIE::MCA_MISC0_PCIE[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::PCIE::MCA_STATUS_PCIE[TCC] is present.

MSRC000_21[6...B]5 [PCIE IP Identification] (MCA::PCIE::MCA_IPID_PCIE)

Reset: 0001_0046_0000_0000h.

The MCA::PCIE::MCA_IPID_PCIE register is used by software to determine what IP type and revision is associated with the MCA bank.

_instPCIE0PCIE_n0_nbio0_aliasMSR; MSRC000_21A5

_instPCIE0PCIE_n2_nbio1_aliasMSR; MSRC000_21A5

_instPCIE1PCIE_n4_nbio0_aliasMSR; MSRC000_21A5

_instPCIE1PCIE_n6_nbio1_aliasMSR; MSRC000_21A5

_instPCIE2PCIE_n5_nbio0_aliasMSR; MSRC000_21B5

_instPCIE2PCIE_n7_nbio1_aliasMSR; MSRC000_21B5

_instPCIE3PCIE_n1_nbio0_aliasMSR; MSRC000_21B5

_instPCIE3PCIE_n3_nbio1_aliasMSR; MSRC000_21B5

_instPCIE4PCIE_n8_nbio0_aliasMSR; MSRC000_2165

_instPCIE4PCIE_n9_nbio1_aliasMSR; MSRC000_2165

Bits	Description
63:48	McaType . Read-only. Reset: 0001h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 046h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instPCIE0PCIE_n0_nbio0_aliasMSR: 1A3C_0000h
	Init: _instPCIE0PCIE_n2_nbio1_aliasMSR: 1A5C_0000h
	Init: _instPCIE1PCIE_n4_nbio0_aliasMSR: 1A7C_0000h
	Init: _instPCIE1PCIE_n6_nbio1_aliasMSR: 1A9C_0000h
	Init: _instPCIE2PCIE_n5_nbio0_aliasMSR: 1A8C_0000h
	Init: _instPCIE2PCIE_n7_nbio1_aliasMSR: 1AAC_0000h
	Init: _instPCIE3PCIE_n1_nbio0_aliasMSR: 1A4C_0000h
	Init: _instPCIE3PCIE_n3_nbio1_aliasMSR: 1A6C_0000h
	Init: _instPCIE4PCIE_n8_nbio0_aliasMSR: 1ABC_0000h
	Init: _instPCIE4PCIE_n9_nbio1_aliasMSR: 1ADC_0000h

MSRC000_21[6...B]6 [PCIE Machine Check Syndrome] (MCA::PCIE::MCA_SYND_PCIE)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::PCIE::MCA_STATUS_PCIE [Thread 0](#)[_instPCIE0PCIE_n0_nbio0_aliasMSR](#); MSRC000_21A6[_instPCIE0PCIE_n2_nbio1_aliasMSR](#); MSRC000_21A6[_instPCIE1PCIE_n4_nbio0_aliasMSR](#); MSRC000_21A6[_instPCIE1PCIE_n6_nbio1_aliasMSR](#); MSRC000_21A6[_instPCIE2PCIE_n5_nbio0_aliasMSR](#); MSRC000_21B6[_instPCIE2PCIE_n7_nbio1_aliasMSR](#); MSRC000_21B6[_instPCIE3PCIE_n1_nbio0_aliasMSR](#); MSRC000_21B6[_instPCIE3PCIE_n3_nbio1_aliasMSR](#); MSRC000_21B6[_instPCIE4PCIE_n8_nbio0_aliasMSR](#); MSRC000_2166[_instPCIE4PCIE_n9_nbio1_aliasMSR](#); MSRC000_2166

Bits	Description
63:32	Syndromes. Read-write, Volatile. Reset: Cold, 0000_0000h. Contains the syndrome, if any, associated with the error logged in MCA::PCIE::MCA_STATUS_PCIE. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PCIE::MCA_SYND_PCIE[Length]. The Syndrome field is only valid when MCA::PCIE::MCA_SYND_PCIE[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PCIE::MCA_SYND_PCIE. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PCIE::MCA_SYND_PCIE[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PCIE::MCA_SYND_PCIE. For example, a syndrome length of 9 means that MCA::PCIE::MCA_SYND_PCIE[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 105 [MCA_SYND_PCIE].

Table 105: MCA_SYND_PCIE

Error Type	Bits	Description
SDP_PARITY_ERR_LOG	[17:11]	Reserved
	[10:1]	ReqTag
	[0]	MST_PCIE or PCIE_SLV interface

MSRC000_21[6...B]8 [PCIE Machine Check Deferred Error Status] (MCA::PCIE::MCA_DESTAT_PCIE)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instPCIE0PCIE_n0_nbio0_aliasMSR; MSRC000_21A8_instPCIE0PCIE_n2_nbio1_aliasMSR; MSRC000_21A8_instPCIE1PCIE_n4_nbio0_aliasMSR; MSRC000_21A8_instPCIE1PCIE_n6_nbio1_aliasMSR; MSRC000_21A8_instPCIE2PCIE_n5_nbio0_aliasMSR; MSRC000_21B8_instPCIE2PCIE_n7_nbio1_aliasMSR; MSRC000_21B8_instPCIE3PCIE_n1_nbio0_aliasMSR; MSRC000_21B8_instPCIE3PCIE_n3_nbio1_aliasMSR; MSRC000_21B8_instPCIE4PCIE_n8_nbio0_aliasMSR; MSRC000_2168_instPCIE4PCIE_n9_nbio1_aliasMSR; MSRC000_2168

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::PCIE::MCA_DEADDR_PCIE contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::PCIE::MCA_SYND_PCIE. If MCA::PCIE::MCA_SYND_PCIE[ErrorPriority] is the same as the priority of the error in MCA::PCIE::MCA_STATUS_PCIE, then the information in MCA::PCIE::MCA_SYND_PCIE is associated with the error in MCA::PCIE::MCA_DESTAT_PCIE.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PCIE::MCA_ADDR_PCIE[ErrorAddr]. A value of 0 indicates that MCA::PCIE::MCA_ADDR_PCIE[55:0] contains a valid byte address. A value of 6 indicates that MCA::PCIE::MCA_ADDR_PCIE[55:6] contains a valid cache line address and that MCA::PCIE::MCA_ADDR_PCIE[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PCIE::MCA_ADDR_PCIE[55:12] contain a valid 4KB memory page and that MCA::PCIE::MCA_ADDR_PCIE[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21[6...B]9 [PCIE Deferred Error Address] (MCA::PCIE::MCA_DEADDR_PCIE)

Reset: Cold,0000_0000_0000_0000h.

The MCA::PCIE::MCA_DEADDR_PCIE register stores the address associated with the error in MCA::PCIE::MCA_DESTAT_PCIE. The register is only meaningful if MCA::PCIE::MCA_DESTAT_PCIE[Val]=1 and MCA::PCIE::MCA_DESTAT_PCIE[AddrV]=1. The lowest valid bit of the address is defined by MCA::PCIE::MCA_DESTAT_PCIE[AddrLsb].

_instPCIE0PCIE_n0_nbio0_aliasMSR; MSRC000_21A9

_instPCIE0PCIE_n2_nbio1_aliasMSR; MSRC000_21A9

_instPCIE1PCIE_n4_nbio0_aliasMSR; MSRC000_21A9

_instPCIE1PCIE_n6_nbio1_aliasMSR; MSRC000_21A9

_instPCIE2PCIE_n5_nbio0_aliasMSR; MSRC000_21B9

_instPCIE2PCIE_n7_nbio1_aliasMSR; MSRC000_21B9

_instPCIE3PCIE_n1_nbio0_aliasMSR; MSRC000_21B9

_instPCIE3PCIE_n3_nbio1_aliasMSR; MSRC000_21B9

_instPCIE4PCIE_n8_nbio0_aliasMSR; MSRC000_2169

_instPCIE4PCIE_n9_nbio1_aliasMSR; MSRC000_2169

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PCIE::MCA_DESTAT_PCIE. The lowest-order valid bit of the address is specified in MCA::PCIE::MCA_DESTAT_PCIE[AddrLsb].

MSRC001_041[6...B] [PCIE Machine Check Control Mask] (MCA::PCIE::MCA_CTL_MASK_PCIE)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instPCIE0PCIE_n0_nbio0_aliasMSR; MSRC001_041A

_instPCIE0PCIE_n2_nbio1_aliasMSR; MSRC001_041A

_instPCIE1PCIE_n4_nbio0_aliasMSR; MSRC001_041A

_instPCIE1PCIE_n6_nbio1_aliasMSR; MSRC001_041A

_instPCIE2PCIE_n5_nbio0_aliasMSR; MSRC001_041B

_instPCIE2PCIE_n7_nbio1_aliasMSR; MSRC001_041B

_instPCIE3PCIE_n1_nbio0_aliasMSR; MSRC001_041B

_instPCIE3PCIE_n3_nbio1_aliasMSR; MSRC001_041B

_instPCIE4PCIE_n8_nbio0_aliasMSR; MSRC001_0416

_instPCIE4PCIE_n9_nbio1_aliasMSR; MSRC001_0416

Bits	Description
63:1	Reserved.
0	SDP_PARITY_ERR_LOG. Read-write. Reset: 0. SDP Data Parity Error logging.

3.2.5.17 KPX SERDES

**MSR0000_047C...MSRC000_21F0 [KPX_SERDES Machine Check Control]
(MCA::KPX::SERDES::MCA_CTL_KPX_SERDES)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::KPX::SERDES::MCA_CTL_KPX_SERDES register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F0_instSERDESAG3KPXSERDES_n7_aliasMSRLEGACY; MSR0000_047C_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F0_instSERDESAP0KPXSERDES_n0_aliasMSRLEGACY; MSR0000_047C_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F0_instSERDESBG0KPXSERDES_n4_aliasMSRLEGACY; MSR0000_047C_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F0_instSERDESBG1KPXSERDES_n5_aliasMSRLEGACY; MSR0000_047C_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F0_instSERDESBG2KPXSERDES_n6_aliasMSRLEGACY; MSR0000_047C_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F0_instSERDESBP1KPXSERDES_n1_aliasMSRLEGACY; MSR0000_047C_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F0_instSERDESBP2KPXSERDES_n2_aliasMSRLEGACY; MSR0000_047C_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F0_instSERDESBP3KPXSERDES_n3_aliasMSRLEGACY; MSR0000_047C**Bits Description**

63:4 Reserved.

3 **APB.** Read-write. Reset: 0. PHY APB error2 **ARCData.** Read-write. Reset: 0. ARC data buffer parity error1 **ARCIns.** Read-write. Reset: 0. ARC instruction buffer parity error0 **RAMECC.** Read-write. Reset: 0. RAM ECC Error.

**MSR0000_047D...MSRC000_21F1 [KPX_SERDES Machine Check Status]
(MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES)**

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F1_instSERDESAG3KPXSERDES_n7_aliasMSRLEGACY; MSR0000_047D_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F1_instSERDESAP0KPXSERDES_n0_aliasMSRLEGACY; MSR0000_047D_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F1_instSERDESBG0KPXSERDES_n4_aliasMSRLEGACY; MSR0000_047D_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F1_instSERDESBG1KPXSERDES_n5_aliasMSRLEGACY; MSR0000_047D_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F1_instSERDESBG2KPXSERDES_n6_aliasMSRLEGACY; MSR0000_047D_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F1_instSERDESBP1KPXSERDES_n1_aliasMSRLEGACY; MSR0000_047D_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F1_instSERDESBP2KPXSERDES_n2_aliasMSRLEGACY; MSR0000_047D_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F1_instSERDESBP3KPXSERDES_n3_aliasMSRLEGACY; MSR0000_047D

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::KPX::SERDES::MCA_CTL_KPX_SERDES. This bit is a copy of bit in MCA::KPX::SERDES::MCA_CTL_KPX_SERDES for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES. If MCA::KPX::SERDES::MCA_SYND_KPX_SERDES[ErrorPriority] is the same as the priority of the error in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES, then the information in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES is associated with the error in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[ErrorAddr]. A value of 0 indicates that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[55:0] contains a valid byte address. A value of 6 indicates that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[55:6] contains a valid cache line address and that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[55:12] contain a valid 4KB memory page and that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::KPX::SERDES::MCA_CTL_KPX_SERDES enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 106: MCA_STATUS_KPX_SERDES

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
RAMECC	0x0	0/1	0/1	0/1	0	-	-
ARCIns	0x1	1	1	1	0	-	-
ARCDData	0x2	1	1	1	0	-	-
APB	0x3	1	1	1	0	-	-

MSR0000_047E...MSRC000_21F2 [KPX_SERDES Machine Check Address] (MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES stores an address and other information associated with the error in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES. The register is only meaningful if MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES[Val]=1 and MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES[AddrV]=1.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F2
 _instSERDESAG3KPXSERDES_n7_aliasMSRLEGACY; MSR0000_047E
 _instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F2
 _instSERDESAP0KPXSERDES_n0_aliasMSRLEGACY; MSR0000_047E
 _instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F2
 _instSERDESBG0KPXSERDES_n4_aliasMSRLEGACY; MSR0000_047E
 _instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F2
 _instSERDESBG1KPXSERDES_n5_aliasMSRLEGACY; MSR0000_047E
 _instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F2
 _instSERDESBG2KPXSERDES_n6_aliasMSRLEGACY; MSR0000_047E
 _instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F2
 _instSERDESBP1KPXSERDES_n1_aliasMSRLEGACY; MSR0000_047E
 _instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F2
 _instSERDESBP2KPXSERDES_n2_aliasMSRLEGACY; MSR0000_047E
 _instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F2
 _instSERDESBP3KPXSERDES_n3_aliasMSRLEGACY; MSR0000_047E

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES.

Table 107: MCA_ADDR_KPX_SERDES

Error Type	Bits	Description
RAMECC	[56:0]	Reserved
ARCIns	[56:0]	Reserved
ARCDData	[56:0]	Reserved
APB	[56:0]	Reserved

**MSR0000_047F...MSRC000_21F3 [KPX_SERDES Machine Check Miscellaneous 0]
(MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES)**

Log miscellaneous information associated with errors.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F3
_instSERDESAG3KPXSERDES_n7_aliasMSRLEGACY; MSR0000_047F
_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F3
_instSERDESAP0KPXSERDES_n0_aliasMSRLEGACY; MSR0000_047F
_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F3
_instSERDESBG0KPXSERDES_n4_aliasMSRLEGACY; MSR0000_047F
_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F3
_instSERDESBG1KPXSERDES_n5_aliasMSRLEGACY; MSR0000_047F
_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F3
_instSERDESBG2KPXSERDES_n6_aliasMSRLEGACY; MSR0000_047F
_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F3
_instSERDESBP1KPXSERDES_n1_aliasMSRLEGACY; MSR0000_047F
_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F3
_instSERDESBP2KPXSERDES_n2_aliasMSRLEGACY; MSR0000_047F
_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F3
_instSERDESBP3KPXSERDES_n3_aliasMSRLEGACY; MSR0000_047F

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[Locked]) ? Read-write : Read-only.
47:44	Reserved.

43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported.
	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

**MSRC000_21F4 [KPX_SERDES Machine Check Configuration]
(MCA::KPX::SERDES::MCA_CONFIG_KPX_SERDES)**

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

[_instSERDESAG3KPXSERDES_n7_aliasMSR](#); MSRC000_21F4[_instSERDESAP0KPXSERDES_n0_aliasMSR](#); MSRC000_21F4[_instSERDESBG0KPXSERDES_n4_aliasMSR](#); MSRC000_21F4[_instSERDESBG1KPXSERDES_n5_aliasMSR](#); MSRC000_21F4[_instSERDESBG2KPXSERDES_n6_aliasMSR](#); MSRC000_21F4[_instSERDESBP1KPXSERDES_n1_aliasMSR](#); MSRC000_21F4[_instSERDESBP2KPXSERDES_n2_aliasMSR](#); MSRC000_21F4[_instSERDESBP3KPXSERDES_n3_aliasMSR](#); MSRC000_21F4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,0. 1=Log deferred errors in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES and MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES in addition to MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES and MCA::KPX::SERDES::MCA_DEADDR_KPX_SERDES. 0=Only log deferred errors in MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES and MCA::KPX::SERDES::MCA_DEADDR_KPX_SERDES. This bit does not affect logging of deferred errors in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES, MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES, MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES, MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES, and MCA::KPX::SERDES::MCA_SYND_KPX_SERDES.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::KPX::SERDES::MCA_CONFIG_KPX_SERDES[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::KPX::SERDES::MCA_CONFIG_KPX_SERDES[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::KPX::SERDES::MCA_CONFIG_KPX_SERDES[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and MCA::KPX::SERDES::MCA_CONFIG_KPX_SERDES[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES and MCA::KPX::SERDES::MCA_DEADDR_KPX_SERDES are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.

0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::KPX::SERDES::MCA_MISC0_KPX_SERDES[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES[TCC] is present.
---	---

MSRC000_21F5 [KPX_SERDES IP Identification] (MCA::KPX::SERDES::MCA_IPID_KPX_SERDES)

Reset: 0000_0259_0000_0000h.

The MCA::KPX::SERDES::MCA_IPID_KPX_SERDES register is used by software to determine what IP type and revision is associated with the MCA bank.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F5

_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F5

_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F5

_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F5

_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F5

_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F5

_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F5

_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F5

Bits	Description
63:48	McaType. Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 259h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instSERDESAG3KPXSERDES_n7_aliasMSR: 1860_0400h
	Init: _instSERDESAP0KPXSERDES_n0_aliasMSR: 17F0_0400h
	Init: _instSERDESBG0KPXSERDES_n4_aliasMSR: 1830_0400h
	Init: _instSERDESBG1KPXSERDES_n5_aliasMSR: 1840_0400h
	Init: _instSERDESBG2KPXSERDES_n6_aliasMSR: 1850_0400h
	Init: _instSERDESBP1KPXSERDES_n1_aliasMSR: 1800_0400h
	Init: _instSERDESBP2KPXSERDES_n2_aliasMSR: 1810_0400h
	Init: _instSERDESBP3KPXSERDES_n3_aliasMSR: 1820_0400h

**MSRC000_21F6 [KPX_SERDES Machine Check Syndrome]
(MCA::KPX::SERDES::MCA_SYND_KPX_SERDES)**

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES
[Thread 0](#)

[_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F6](#)

[_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F6](#)

[_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F6](#)

[_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F6](#)

[_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F6](#)

[_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F6](#)

[_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F6](#)

[_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F6](#)

Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::KPX::SERDES::MCA_SYND_KPX_SERDES[Length]. The Syndrome field is only valid when MCA::KPX::SERDES::MCA_SYND_KPX_SERDES[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES. For example, a syndrome length of 9 means that MCA::KPX::SERDES::MCA_SYND_KPX_SERDES[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 108 [MCA_SYND_KPX_SERDES].

Table 108: MCA_SYND_KPX_SERDES

Error Type	Bits	Description
RAMECC	[17:0]	Reserved
ARCIns	[17:0]	Reserved
ARCData	[17:0]	Reserved
APB	[17:0]	Reserved

**MSRC000_21F8 [KPX_SERDES Machine Check Deferred Error Status]
(MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES)**

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F8

_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F8

_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F8

_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F8

_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F8

_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F8

_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F8

_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F8

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::KPX::SERDES::MCA_DEADDR_KPX_SERDES contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES. If MCA::KPX::SERDES::MCA_SYND_KPX_SERDES[ErrorPriority] is the same as the priority of the error in MCA::KPX::SERDES::MCA_STATUS_KPX_SERDES, then the information in MCA::KPX::SERDES::MCA_SYND_KPX_SERDES is associated with the error in MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[ErrorAddr]. A value of 0 indicates that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[55:0] contains a valid byte address. A value of 6 indicates that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[55:6] contains a valid cache line address and that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[55:12] contain a valid 4KB memory page and that MCA::KPX::SERDES::MCA_ADDR_KPX_SERDES[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

**MSRC000_21F9 [KPX_SERDES Deferred Error Address]
(MCA::KPX::SERDES::MCA_DEADDR_KPX_SERDES)**

Read-only. Reset: Cold,0000_0000_0000_0000h.

The MCA::KPX::SERDES::MCA_DEADDR_KPX_SERDES register stores the address associated with the error in MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES. The register is only meaningful if MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES[Val]=1 and MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES[AddrV]=1. The lowest valid bit of the address is defined by MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES[AddrLsb].

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC000_21F9

_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC000_21F9

_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC000_21F9

_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC000_21F9

_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC000_21F9

_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC000_21F9

_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC000_21F9

_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC000_21F9

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::KPX::SERDES::MCA_DESTAT_KPX_SERDES.

**MSRC001_041F [KPX_SERDES Machine Check Control Mask]
(MCA::KPX::SERDES::MCA_CTL_MASK_KPX_SERDES)**

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instSERDESAG3KPXSERDES_n7_aliasMSR; MSRC001_041F

_instSERDESAP0KPXSERDES_n0_aliasMSR; MSRC001_041F

_instSERDESBG0KPXSERDES_n4_aliasMSR; MSRC001_041F

_instSERDESBG1KPXSERDES_n5_aliasMSR; MSRC001_041F

_instSERDESBG2KPXSERDES_n6_aliasMSR; MSRC001_041F

_instSERDESBP1KPXSERDES_n1_aliasMSR; MSRC001_041F

_instSERDESBP2KPXSERDES_n2_aliasMSR; MSRC001_041F

_instSERDESBP3KPXSERDES_n3_aliasMSR; MSRC001_041F

Bits	Description
63:4	Reserved.
3	APB. Read-write. Reset: 0. PHY APB error
2	ARCData. Read-write. Reset: 0. ARC data buffer parity error
1	ARCIns. Read-write. Reset: 0. ARC instruction buffer parity error
0	RAMECC. Read-write. Reset: 0. RAM ECC Error.

3.2.5.18 KPX GMI

**MSR0000_044C...MSRC000_2140 [KPX_GMI Machine Check Control]
(MCA::KPX::GMI::MCA_CTL_KPX_GMI)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::KPX::GMI::MCA_CTL_KPX_GMI register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2140
_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSRLEGACY; MSR0000_0450
_ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2140
_ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSRLEGACY; MSR0000_0450
_ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2140
_ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSRLEGACY; MSR0000_0450
_ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2140
_ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSRLEGACY; MSR0000_0450
_ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2140
_ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSRLEGACY; MSR0000_0450
_ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2140
_ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSRLEGACY; MSR0000_0450
_ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2140
_ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSRLEGACY; MSR0000_0450
_ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2140
_ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSRLEGACY; MSR0000_0450
_ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2140
_ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSRLEGACY; MSR0000_0450
_ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2140
_ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSRLEGACY; MSR0000_0450
_ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2140
_ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSRLEGACY; MSR0000_0450
_ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2140
_ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSRLEGACY; MSR0000_0450
_ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2140
_ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSRLEGACY; MSR0000_0450
_ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2140
_ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSRLEGACY; MSR0000_0450
_ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2140
_ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSRLEGACY; MSR0000_0450
_ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2140
_ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSRLEGACY; MSR0000_0450
_instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2130
_instGMICONTAINER0KPXGMI_n0_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2130
_instGMICONTAINER10KPXGMI_n20_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2130
_instGMICONTAINER11KPXGMI_n22_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2130
_instGMICONTAINER1KPXGMI_n2_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2130
_instGMICONTAINER2KPXGMI_n4_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2130
_instGMICONTAINER3KPXGMI_n6_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2130
_instGMICONTAINER4KPXGMI_n8_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2130
_instGMICONTAINER5KPXGMI_n10_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2130
_instGMICONTAINER6KPXGMI_n12_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2130
_instGMICONTAINER7KPXGMI_n14_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2130
_instGMICONTAINER8KPXGMI_n16_aliasMSRLEGACY; MSR0000_044C
_instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2130
_instGMICONTAINER9KPXGMI_n18_aliasMSRLEGACY; MSR0000_044C

Bits	Description
------	-------------

63:4	Reserved.
3	APB. Read-write. Reset: 0. PHY APB error
2	ARCData. Read-write. Reset: 0. ARC data buffer parity error
1	ARCIns. Read-write. Reset: 0. ARC instruction buffer parity error
0	RAMECC. Read-write. Reset: 0. RAM ECC Error.

**MSR0000_044D...MSRC000_2141 [KPX_GMI Machine Check Status]
(MCA::KPX::GMI::MCA_STATUS_KPX_GMI)**

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2141
 _ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSRLEGACY; MSR0000_0451
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2141
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSRLEGACY; MSR0000_0451
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2141
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSRLEGACY; MSR0000_0451
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2141
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSRLEGACY; MSR0000_0451
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2141
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSRLEGACY; MSR0000_0451
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2141
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSRLEGACY; MSR0000_0451
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2141
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSRLEGACY; MSR0000_0451
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2141
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSRLEGACY; MSR0000_0451
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2141
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSRLEGACY; MSR0000_0451
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2141
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSRLEGACY; MSR0000_0451
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2141
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSRLEGACY; MSR0000_0451
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2141
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSRLEGACY; MSR0000_0451
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2141
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSRLEGACY; MSR0000_0451
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2141
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSRLEGACY; MSR0000_0451
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2141
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSRLEGACY; MSR0000_0451
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2141
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSRLEGACY; MSR0000_0451
 _instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2131
 _instGMICONTAINER0KPXGMI_n0_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2131
 _instGMICONTAINER10KPXGMI_n20_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2131
 _instGMICONTAINER11KPXGMI_n22_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2131
 _instGMICONTAINER1KPXGMI_n2_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2131
 _instGMICONTAINER2KPXGMI_n4_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2131
 _instGMICONTAINER3KPXGMI_n6_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2131
 _instGMICONTAINER4KPXGMI_n8_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2131
 _instGMICONTAINER5KPXGMI_n10_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2131
 _instGMICONTAINER6KPXGMI_n12_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2131
 _instGMICONTAINER7KPXGMI_n14_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2131
 _instGMICONTAINER8KPXGMI_n16_aliasMSRLEGACY; MSR0000_044D
 _instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2131
 _instGMICONTAINER9KPXGMI_n18_aliasMSRLEGACY; MSR0000_044D

Bits	Description
------	-------------

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::KPX::GMI::MCA_CTL_KPX_GMI. This bit is a copy of bit in MCA::KPX::GMI::MCA_CTL_KPX_GMI for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::KPX::GMI::MCA_MISC0_KPX_GMI. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::KPX::GMI::MCA_ADDR_KPX_GMI contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::KPX::GMI::MCA_STATUS_KPX_GMI[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::KPX::GMI::MCA_SYND_KPX_GMI. If MCA::KPX::GMI::MCA_SYND_KPX_GMI[ErrorPriority] is the same as the priority of the error in MCA::KPX::GMI::MCA_STATUS_KPX_GMI, then the information in MCA::KPX::GMI::MCA_SYND_KPX_GMI is associated with the error in MCA::KPX::GMI::MCA_STATUS_KPX_GMI. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::KPX::GMI::MCA_STATUS_KPX_GMI[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::KPX::GMI::MCA_ADDR_KPX_GMI[ErrorAddr]. A value of 0 indicates that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[55:0] contains a valid byte address. A value of 6 indicates that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[55:6] contains a valid cache line address and that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[55:12] contain a valid 4KB memory page and that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::KPX::GMI::MCA_CTL_KPX_GMI enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 109: MCA_STATUS_KPX_GMI

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
RAMECC	0x0	0/1	0/1	0/1	0	-	-
ARCIns	0x1	1	1	1	0	-	-
ARCDData	0x2	1	1	1	0	-	-
APB	0x3	1	1	1	0	-	-

**MSR0000_044E...MSRC000_2142 [KPX_GMI Machine Check Address]
(MCA::KPX::GMI::MCA_ADDR_KPX_GMI)**

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::KPX::GMI::MCA_ADDR_KPX_GMI stores an address and other information associated with the error in MCA::KPX::GMI::MCA_STATUS_KPX_GMI. The register is only meaningful if MCA::KPX::GMI::MCA_STATUS_KPX_GMI[Val]=1 and MCA::KPX::GMI::MCA_STATUS_KPX_GMI[AddrV]=1.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2142
_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSRLEGACY; MSR0000_0452
_ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2142
_ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSRLEGACY; MSR0000_0452
_ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2142
_ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSRLEGACY; MSR0000_0452
_ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2142
_ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSRLEGACY; MSR0000_0452
_ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2142
_ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSRLEGACY; MSR0000_0452
_ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2142
_ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSRLEGACY; MSR0000_0452
_ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2142
_ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSRLEGACY; MSR0000_0452
_ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2142
_ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSRLEGACY; MSR0000_0452
_ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2142
_ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSRLEGACY; MSR0000_0452
_ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2142
_ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSRLEGACY; MSR0000_0452
_ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2142
_ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSRLEGACY; MSR0000_0452
_ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2142
_ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSRLEGACY; MSR0000_0452
_ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2142
_ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSRLEGACY; MSR0000_0452
_ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2142
_ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSRLEGACY; MSR0000_0452
_ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2142
_ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSRLEGACY; MSR0000_0452
_ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2142
_ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSRLEGACY; MSR0000_0452
_instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2132
_instGMICONTAINER0KPXGMI_n0_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2132
_instGMICONTAINER10KPXGMI_n20_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2132
_instGMICONTAINER11KPXGMI_n22_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2132
_instGMICONTAINER1KPXGMI_n2_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2132
_instGMICONTAINER2KPXGMI_n4_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2132
_instGMICONTAINER3KPXGMI_n6_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2132
_instGMICONTAINER4KPXGMI_n8_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2132
_instGMICONTAINER5KPXGMI_n10_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2132
_instGMICONTAINER6KPXGMI_n12_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2132
_instGMICONTAINER7KPXGMI_n14_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2132
_instGMICONTAINER8KPXGMI_n16_aliasMSRLEGACY; MSR0000_044E
_instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2132
_instGMICONTAINER9KPXGMI_n18_aliasMSRLEGACY; MSR0000_044E

Bits	Description
------	-------------

63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::KPX::GMI::MCA_STATUS_KPX_GMI.
------	---

Table 110: MCA_ADDR_KPX_GMI

Error Type	Bits	Description
RAMECC	[56:0]	Reserved
ARCIns	[56:0]	Reserved
ARCDData	[56:0]	Reserved
APB	[56:0]	Reserved

**MSR0000_044F...MSRC000_2143 [KPX_GMI Machine Check Miscellaneous 0]
(MCA::KPX::GMI::MCA_MISC0_KPX_GMI)**

Log miscellaneous information associated with errors.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2143
 _ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSRLEGACY; MSR0000_0453
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2143
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSRLEGACY; MSR0000_0453
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2143
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSRLEGACY; MSR0000_0453
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2143
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSRLEGACY; MSR0000_0453
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2143
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSRLEGACY; MSR0000_0453
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2143
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSRLEGACY; MSR0000_0453
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2143
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSRLEGACY; MSR0000_0453
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2143
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSRLEGACY; MSR0000_0453
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2143
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSRLEGACY; MSR0000_0453
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2143
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSRLEGACY; MSR0000_0453
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2143
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSRLEGACY; MSR0000_0453
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2143
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSRLEGACY; MSR0000_0453
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2143
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSRLEGACY; MSR0000_0453
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2143
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSRLEGACY; MSR0000_0453
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2143
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSRLEGACY; MSR0000_0453
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2143
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSRLEGACY; MSR0000_0453
 _instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2133
 _instGMICONTAINER0KPXGMI_n0_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2133
 _instGMICONTAINER10KPXGMI_n20_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2133
 _instGMICONTAINER11KPXGMI_n22_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2133
 _instGMICONTAINER1KPXGMI_n2_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2133
 _instGMICONTAINER2KPXGMI_n4_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2133
 _instGMICONTAINER3KPXGMI_n6_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2133
 _instGMICONTAINER4KPXGMI_n8_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2133
 _instGMICONTAINER5KPXGMI_n10_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2133
 _instGMICONTAINER6KPXGMI_n12_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2133
 _instGMICONTAINER7KPXGMI_n14_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2133
 _instGMICONTAINER8KPXGMI_n16_aliasMSRLEGACY; MSR0000_044F
 _instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2133
 _instGMICONTAINER9KPXGMI_n18_aliasMSRLEGACY; MSR0000_044F

Bits Description

63 **Valid.** Reset: 1. 1=A valid CntP field is present in this register.

AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.

62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::GMI::MCA_MISC0_KPX_GMI[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::GMI::MCA_MISC0_KPX_GMI[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::GMI::MCA_MISC0_KPX_GMI[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::GMI::MCA_MISC0_KPX_GMI[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::GMI::MCA_MISC0_KPX_GMI[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::KPX::GMI::MCA_MISC0_KPX_GMI[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

**MSRC000_21[3...4]4 [KPX_GMI Machine Check Configuration]
(MCA::KPX::GMI::MCA_CONFIG_KPX_GMI)**

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2144
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2144
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2144
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2144
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2144
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2144
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2144
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2144
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2144
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2144
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2144
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2144
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2144
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2144
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2144
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2144
 _instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2134
 _instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2134
 _instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2134
 _instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2134
 _instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2134
 _instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2134
 _instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2134
 _instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2134
 _instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2134
 _instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2134
 _instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2134
 _instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2134

Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat. Read-write. Reset: 0. Init: BIOS,0. 1=Log deferred errors in MCA::KPX::GMI::MCA_STATUS_KPX_GMI and MCA::KPX::GMI::MCA_ADDR_KPX_GMI in addition to MCA::KPX::GMI::MCA_DESTAT_KPX_GMI and MCA::KPX::GMI::MCA_DEADDR_KPX_GMI. 0=Only log deferred errors in MCA::KPX::GMI::MCA_DESTAT_KPX_GMI and MCA::KPX::GMI::MCA_DEADDR_KPX_GMI. This bit does not affect logging of deferred errors in MCA::KPX::GMI::MCA_SYND_KPX_GMI, MCA::KPX::GMI::MCA_MISC0_KPX_GMI.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::KPX::GMI::MCA_STATUS_KPX_GMI, MCA::KPX::GMI::MCA_ADDR_KPX_GMI, MCA::KPX::GMI::MCA_MISC0_KPX_GMI, and MCA::KPX::GMI::MCA_SYND_KPX_GMI.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msrr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::KPX::GMI::MCA_CONFIG_KPX_GMI[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.

5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::KPX::GMI::MCA_CONFIG_KPX_GMI[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::KPX::GMI::MCA_CONFIG_KPX_GMI[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and MCA::KPX::GMI::MCA_CONFIG_KPX_GMI[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::KPX::GMI::MCA_DESTAT_KPX_GMI and MCA::KPX::GMI::MCA_DEADDR_KPX_GMI are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::KPX::GMI::MCA_MISC0_KPX_GMI[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::KPX::GMI::MCA_STATUS_KPX_GMI[TCC] is present.

MSRC000_21[3...4]5 [KPX_GMI IP Identification] (MCA::KPX::GMI::MCA_IPID_KPX_GMI)

Reset: 0000_0269_0000_0000h.

The MCA::KPX::GMI::MCA_IPID_KPX_GMI register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2145
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2145
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2145
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2145
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2145
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2145
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2145
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2145
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2145
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2145
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2145
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2145
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2145
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2145
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2145
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2145
 _instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2135
 _instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2135
 _instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2135
 _instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2135
 _instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2135
 _instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2135
 _instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2135
 _instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2135
 _instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2135
 _instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2135
 _instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2135
 _instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2135

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 269h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR: 3090_0400h Init: _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR: 30A0_0400h Init: _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR: 3290_0400h Init: _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR: 32A0_0400h Init: _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR: 3490_0400h Init: _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR: 34A0_0400h Init: _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR: 3690_0400h Init: _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR: 36A0_0400h Init: _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR: 3890_0400h Init: _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR: 3A90_0400h Init: _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR: 3C90_0400h Init: _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR: 3E90_0400h Init: _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR: 4A90_0400h Init: _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR: 4C90_0400h Init: _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR: 4E90_0400h Init: _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR: 5090_0400h Init: _instGMICONTAINER0KPXGMI_n0_aliasMSR: 1890_0400h Init: _instGMICONTAINER10KPXGMI_n20_aliasMSR: 19C0_0400h Init: _instGMICONTAINER11KPXGMI_n22_aliasMSR: 19D0_0400h Init: _instGMICONTAINER1KPXGMI_n2_aliasMSR: 18A0_0400h Init: _instGMICONTAINER2KPXGMI_n4_aliasMSR: 18B0_0400h Init: _instGMICONTAINER3KPXGMI_n6_aliasMSR: 18C0_0400h

Init: _instGMICONTAINER4KPXGMI_n8_aliasMSR: 18D0_0400h
Init: _instGMICONTAINER5KPXGMI_n10_aliasMSR: 1950_0400h
Init: _instGMICONTAINER6KPXGMI_n12_aliasMSR: 1960_0400h
Init: _instGMICONTAINER7KPXGMI_n14_aliasMSR: 1970_0400h
Init: _instGMICONTAINER8KPXGMI_n16_aliasMSR: 1980_0400h
Init: _instGMICONTAINER9KPXGMI_n18_aliasMSR: 1990_0400h

MSRC000_21[3...4]6 [KPX_GMI Machine Check Syndrome] (MCA::KPX::GMI::MCA_SYND_KPX_GMI)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::KPX::GMI::MCA_STATUS_KPX_GMI [Thread 0](#)

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2146
_ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2146
_ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2146
_ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2146
_ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2146
_ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2146
_ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2146
_ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2146
_ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2146
_ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2146
_ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2146
_ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2146
_ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2146
_ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2146
_ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2146
_ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2146
_instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2136
_instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2136
_instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2136
_instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2136
_instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2136
_instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2136
_instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2136
_instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2136
_instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2136
_instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2136
_instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2136
_instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2136

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::KPX::GMI::MCA_STATUS_KPX_GMI. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::KPX::GMI::MCA_SYND_KPX_GMI[Length]. The Syndrome field is only valid when MCA::KPX::GMI::MCA_SYND_KPX_GMI[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::KPX::GMI::MCA_SYND_KPX_GMI. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::KPX::GMI::MCA_SYND_KPX_GMI[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::KPX::GMI::MCA_SYND_KPX_GMI. For example, a syndrome length of 9 means that MCA::KPX::GMI::MCA_SYND_KPX_GMI[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 111 [MCA_SYND_KPX_GMI].

Table 111: MCA_SYND_KPX_GMI

Error Type	Bits	Description
RAMECC	[17:0]	Reserved

ARCIns	[17:0]	Reserved
ARCDData	[17:0]	Reserved
APB	[17:0]	Reserved

**MSRC000_21[3...4]8 [KPX_GMI Machine Check Deferred Error Status]
(MCA::KPX::GMI::MCA_DESTAT_KPX_GMI)**

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2148
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2148
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2148
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2148
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2148
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2148
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2148
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2148
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2148
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2148
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2148
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2148
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2148
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2148
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2148
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2148
 _instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2138
 _instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2138
 _instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2138
 _instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2138
 _instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2138
 _instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2138
 _instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2138
 _instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2138
 _instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2138
 _instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2138
 _instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2138
 _instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2138

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::KPX::GMI::MCA_DEADDR_KPX_GMI contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::KPX::GMI::MCA_SYND_KPX_GMI. If MCA::KPX::GMI::MCA_SYND_KPX_GMI[ErrorPriority] is the same as the priority of the error in MCA::KPX::GMI::MCA_STATUS_KPX_GMI, then the information in MCA::KPX::GMI::MCA_SYND_KPX_GMI is associated with the error in MCA::KPX::GMI::MCA_DESTAT_KPX_GMI.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.

29:24	AddrLsb. Read-write, Volatile. Reset: Cold, 00h. Specifies the least significant valid bit of the address contained in MCA::KPX::GMI::MCA_ADDR_KPX_GMI[ErrorAddr]. A value of 0 indicates that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[55:0] contains a valid byte address. A value of 6 indicates that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[55:6] contains a valid cache line address and that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[55:12] contain a valid 4KB memory page and that MCA::KPX::GMI::MCA_ADDR_KPX_GMI[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold, 0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold, 00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold, 0000h. Error code for this error.

MSRC000_21[3...4]9 [KPX_GMI Deferred Error Address] (MCA::KPX::GMI::MCA_DEADDR_KPX_GMI)

Read-only. Reset: Cold, 0000_0000_0000_0000h.

The MCA::KPX::GMI::MCA_DEADDR_KPX_GMI register stores the address associated with the error in MCA::KPX::GMI::MCA_DESTAT_KPX_GMI. The register is only meaningful if MCA::KPX::GMI::MCA_DESTAT_KPX_GMI[Val]=1 and MCA::KPX::GMI::MCA_DESTAT_KPX_GMI[AddrV]=1. The lowest valid bit of the address is defined by MCA::KPX::GMI::MCA_DESTAT_KPX_GMI[AddrLsb].

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC000_2149
_ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC000_2149
_ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC000_2149
_ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC000_2149
_ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC000_2149
_ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC000_2149
_ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC000_2149
_ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC000_2149
_ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC000_2149
_ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC000_2149
_ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC000_2149
_ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC000_2149
_ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC000_2149
_ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC000_2149
_ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC000_2149
_ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC000_2149
_instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC000_2139
_instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC000_2139
_instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC000_2139
_instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC000_2139
_instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC000_2139
_instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC000_2139
_instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC000_2139
_instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC000_2139
_instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC000_2139
_instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC000_2139
_instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC000_2139
_instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC000_2139

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold, 0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::KPX::GMI::MCA_DESTAT_KPX_GMI.

**MSRC001_041[3...4] [KPX_GMI Machine Check Control Mask]
(MCA::KPX::GMI::MCA_CTL_MASK_KPX_GMI)**

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_instGMICONTAINER12KPXGMI_n1_aliasMSR; MSRC001_0414
 _ccd0_instGMICONTAINER13KPXGMI_n24_aliasMSR; MSRC001_0414
 _ccd10_instGMICONTAINER12KPXGMI_n21_aliasMSR; MSRC001_0414
 _ccd11_instGMICONTAINER12KPXGMI_n23_aliasMSR; MSRC001_0414
 _ccd1_instGMICONTAINER12KPXGMI_n3_aliasMSR; MSRC001_0414
 _ccd1_instGMICONTAINER13KPXGMI_n25_aliasMSR; MSRC001_0414
 _ccd2_instGMICONTAINER12KPXGMI_n5_aliasMSR; MSRC001_0414
 _ccd2_instGMICONTAINER13KPXGMI_n26_aliasMSR; MSRC001_0414
 _ccd3_instGMICONTAINER12KPXGMI_n7_aliasMSR; MSRC001_0414
 _ccd3_instGMICONTAINER13KPXGMI_n27_aliasMSR; MSRC001_0414
 _ccd4_instGMICONTAINER12KPXGMI_n9_aliasMSR; MSRC001_0414
 _ccd5_instGMICONTAINER12KPXGMI_n11_aliasMSR; MSRC001_0414
 _ccd6_instGMICONTAINER12KPXGMI_n13_aliasMSR; MSRC001_0414
 _ccd7_instGMICONTAINER12KPXGMI_n15_aliasMSR; MSRC001_0414
 _ccd8_instGMICONTAINER12KPXGMI_n17_aliasMSR; MSRC001_0414
 _ccd9_instGMICONTAINER12KPXGMI_n19_aliasMSR; MSRC001_0414
 _instGMICONTAINER0KPXGMI_n0_aliasMSR; MSRC001_0413
 _instGMICONTAINER10KPXGMI_n20_aliasMSR; MSRC001_0413
 _instGMICONTAINER11KPXGMI_n22_aliasMSR; MSRC001_0413
 _instGMICONTAINER1KPXGMI_n2_aliasMSR; MSRC001_0413
 _instGMICONTAINER2KPXGMI_n4_aliasMSR; MSRC001_0413
 _instGMICONTAINER3KPXGMI_n6_aliasMSR; MSRC001_0413
 _instGMICONTAINER4KPXGMI_n8_aliasMSR; MSRC001_0413
 _instGMICONTAINER5KPXGMI_n10_aliasMSR; MSRC001_0413
 _instGMICONTAINER6KPXGMI_n12_aliasMSR; MSRC001_0413
 _instGMICONTAINER7KPXGMI_n14_aliasMSR; MSRC001_0413
 _instGMICONTAINER8KPXGMI_n16_aliasMSR; MSRC001_0413
 _instGMICONTAINER9KPXGMI_n18_aliasMSR; MSRC001_0413

Bits	Description
63:4	Reserved.
3	APB . Read-write. Reset: 0. PHY APB error
2	ARCData . Read-write. Reset: 0. ARC data buffer parity error
1	ARCIns . Read-write. Reset: 0. ARC instruction buffer parity error
0	RAMECC . Read-write. Reset: 0. RAM ECC Error.

3.2.5.19 PCS GMI

**MSR0000_0444...MSRC000_2120 [PCS_GMI Machine Check Control]
(MCA::PCS::GMI::MCA_CTL_PCS_GMI)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::PCS::GMI::MCA_CTL_PCS_GMI register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2120
 _ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSRLEGACY; MSR0000_0448
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2120
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSRLEGACY; MSR0000_0448
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2120
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSRLEGACY; MSR0000_0448
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2120
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSRLEGACY; MSR0000_0448
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2120
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSRLEGACY; MSR0000_0448
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2120
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSRLEGACY; MSR0000_0448
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2120
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSRLEGACY; MSR0000_0448
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2120
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSRLEGACY; MSR0000_0448
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2120
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSRLEGACY; MSR0000_0448
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2120
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSRLEGACY; MSR0000_0448
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2120
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSRLEGACY; MSR0000_0448
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2120
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSRLEGACY; MSR0000_0448
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2120
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSRLEGACY; MSR0000_0448
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2120
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSRLEGACY; MSR0000_0448
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2120
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSRLEGACY; MSR0000_0448
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2120
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSRLEGACY; MSR0000_0448
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2110
 _instGMICONTAINER0PCSGMI_n0_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2110
 _instGMICONTAINER10PCSGMI_n20_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2110
 _instGMICONTAINER11PCSGMI_n22_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2110
 _instGMICONTAINER1PCSGMI_n2_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2110
 _instGMICONTAINER2PCSGMI_n4_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2110
 _instGMICONTAINER3PCSGMI_n6_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2110
 _instGMICONTAINER4PCSGMI_n8_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2110
 _instGMICONTAINER5PCSGMI_n10_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2110
 _instGMICONTAINER6PCSGMI_n12_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2110
 _instGMICONTAINER7PCSGMI_n14_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2110
 _instGMICONTAINER8PCSGMI_n16_aliasMSRLEGACY; MSR0000_0444
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2110
 _instGMICONTAINER9PCSGMI_n18_aliasMSRLEGACY; MSR0000_0444

Bits	Description
------	-------------

63:32	Reserved.
31	TwixDataLoss. Read-write. Reset: 0. TwixDataLoss
30	LFDSFcinitTimeoutErr. Read-write. Reset: 0. LFDSFcinitTimeoutErr
29	LFDSTrainingTimeoutErr. Read-write. Reset: 0. LFDSTrainingTimeoutErr
28	RxCMDPktErr. Read-write. Reset: 0. RxCMDPktErr
27	LinkSubRxTimeoutErr. Read-write. Reset: 0. LinkSubRxTimeoutErr
26	LinkSubTxTimeoutErr. Read-write. Reset: 0. LinkSubTxTimeoutErr
25	RxLfdsFifoUnderflowErr. Read-write. Reset: 0. RxLfdsFifoUnderflowErr
24	RxLfdsFifoOverflowErr. Read-write. Reset: 0. RxLfdsFifoOverflowErr
23	TwixRxBuff. Read-write. Reset: 0. TwixRxBuff
22	DeskewAbortErr. Read-write. Reset: 0. DeskewAbortErr
21	RecoveryRelockAttemptErr. Read-write. Reset: 0. RecoveryRelockAttemptErr
20	RecoveryAttemptErr. Read-write. Reset: 0. RecoveryAttemptErr
19	ReadySerialAttemptErr. Read-write. Reset: 0. ReadySerialAttemptErr
18	ReadySerialTimeoutErr. Read-write. Reset: 0. ReadySerialTimeoutErr
17	RecoveryTimeoutErr. Read-write. Reset: 0. RecoveryTimeoutErr
16	FCInitTimeoutErr. Read-write. Reset: 0. FCInitTimeoutErr
15	DataStartupLimitErr. Read-write. Reset: 0. DataStartupLimitErr
14	TwixOffline. Read-write. Reset: 0. TwixOffline
13	DeskewErr. Read-write. Reset: 0. DeskewErr
12	ElasticFifoOverflowErr. Read-write. Reset: 0. ElasticFifoOverflowErr
11	ReplayFifoUnderflowErr. Read-write. Reset: 0. ReplayFifoUnderflowErr
10	ReplayFifoOverflowErr. Read-write. Reset: 0. ReplayFifoOverflowErr
9	TxTwixOverflow. Read-write. Reset: 0. TxTwixOverflow
8	ReplayBufParityErr. Read-write. Reset: 0. ReplayBufParityErr
7	TxTwixFifoUnderflow. Read-write. Reset: 0. TxTwixFifoUnderflow
6	BERExceededErr. Read-write. Reset: 0. BERExceededErr
5	CRCErr. Read-write. Reset: 0. CRCErr
4	RxFifoOverflowErr. Read-write. Reset: 0. RxFifoOverflowErr
3	RxFifoUnderflowErr. Read-write. Reset: 0. RxFifoUnderflowErr
2	ReplayParityTwix. Read-write. Reset: 0. ReplayParityTwix
1	TrainingErr. Read-write. Reset: 0. TrainingErr.
0	DataLossErr. Read-write. Reset: 0. DataLossErr

**MSR0000_0445...MSRC000_2121 [PCS_GMI Machine Check Status]
(MCA::PCS::GMI::MCA_STATUS_PCS_GMI)**

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2121
 _ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSRLEGACY; MSR0000_0449
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2121
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSRLEGACY; MSR0000_0449
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2121
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSRLEGACY; MSR0000_0449
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2121
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSRLEGACY; MSR0000_0449
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2121
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSRLEGACY; MSR0000_0449
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2121
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSRLEGACY; MSR0000_0449
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2121
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSRLEGACY; MSR0000_0449
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2121
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSRLEGACY; MSR0000_0449
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2121
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSRLEGACY; MSR0000_0449
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2121
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSRLEGACY; MSR0000_0449
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2121
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSRLEGACY; MSR0000_0449
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2121
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSRLEGACY; MSR0000_0449
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2121
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSRLEGACY; MSR0000_0449
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2121
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSRLEGACY; MSR0000_0449
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2121
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSRLEGACY; MSR0000_0449
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2121
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSRLEGACY; MSR0000_0449
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2111
 _instGMICONTAINER0PCSGMI_n0_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2111
 _instGMICONTAINER10PCSGMI_n20_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2111
 _instGMICONTAINER11PCSGMI_n22_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2111
 _instGMICONTAINER1PCSGMI_n2_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2111
 _instGMICONTAINER2PCSGMI_n4_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2111
 _instGMICONTAINER3PCSGMI_n6_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2111
 _instGMICONTAINER4PCSGMI_n8_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2111
 _instGMICONTAINER5PCSGMI_n10_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2111
 _instGMICONTAINER6PCSGMI_n12_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2111
 _instGMICONTAINER7PCSGMI_n14_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2111
 _instGMICONTAINER8PCSGMI_n16_aliasMSRLEGACY; MSR0000_0445
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2111
 _instGMICONTAINER9PCSGMI_n18_aliasMSRLEGACY; MSR0000_0445

Bits	Description
------	-------------

63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::PCS::GMI::MCA_CTL_PCS_GMI. This bit is a copy of bit in MCA::PCS::GMI::MCA_CTL_PCS_GMI for this error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::PCS::GMI::MCA_MISC0_PCS_GMI. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::PCS::GMI::MCA_ADDR_PCS_GMI contains address information associated with the error. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::PCS::GMI::MCA_STATUS_PCS_GMI[PCC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::PCS::GMI::MCA_SYND_PCS_GMI. If MCA::PCS::GMI::MCA_SYND_PCS_GMI[ErrorPriority] is the same as the priority of the error in MCA::PCS::GMI::MCA_STATUS_PCS_GMI, then the information in MCA::PCS::GMI::MCA_SYND_PCS_GMI is associated with the error in MCA::PCS::GMI::MCA_STATUS_PCS_GMI. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::PCS::GMI::MCA_STATUS_PCS_GMI[UC]=0. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PCS::GMI::MCA_ADDR_PCS_GMI[ErrorAddr]. A value of 0 indicates that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[55:0] contains a valid byte address. A value of 6 indicates that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[55:6] contains a valid cache line address and that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[55:12] contain a valid 4KB memory page and that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::PCS::GMI::MCA_CTL_PCS_GMI enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 112: MCA_STATUS_PCS_GMI

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
DataLossErr	0x0	0/1	0/1	0/1	0	-	-
TrainingErr	0x1	0/1	0/1	0/1	0	-	-
ReplayParity Twix	0x2	0/1	0/1	0/1	0	-	-
RxFifoUnderflowErr	0x3	0/1	0/1	0/1	0	-	-
RxFifoOverflowErr	0x4	0/1	0/1	0/1	0	-	-
CRCErr	0x5	0/1	0/1	0/1	0	-	-
BERExceededErr	0x6	0/1	0/1	0/1	0	-	-

TxTwixFifo Underflow	0x7	0/1	0/1	0/1	0	-	-
ReplayBufParityErr	0x8	0/1	0/1	0/1	0	-	-
TxTwixOverflow	0x9	0/1	0/1	0/1	0	-	-
ReplayFifoOverflowErr	0xa	0/1	0/1	0/1	0	-	-
ReplayFifoUnderflowErr	0xb	0/1	0/1	0/1	0	-	-
ElasticFifoOverflowErr	0xc	0/1	0/1	0/1	0	-	-
DeskewErr	0xd	0/1	0/1	0/1	0	-	-
TwixOffline	0xe	0/1	0/1	0/1	0	-	-
DataStartupLimitErr	0xf	0/1	0/1	0/1	0	-	-
FCInitTimeoutErr	0x10	0/1	0/1	0/1	0	-	-
RecoveryTimeoutErr	0x11	0/1	0/1	0/1	0	-	-
ReadySerialTimeoutErr	0x12	0/1	0/1	0/1	0	-	-
ReadySerialAttemptErr	0x13	0/1	0/1	0/1	0	-	-
RecoveryAttemptErr	0x14	0/1	0/1	0/1	0	-	-
RecoveryRelockAttemptErr	0x15	0/1	0/1	0/1	0	-	-
DeskewAbortErr	0x16	0/1	0/1	0/1	0	-	-
TwixRxBuff	0x17	0/1	0/1	0/1	0	-	-
RxLfsFifoOverflowErr	0x18	0/1	0/1	0/1	0	-	-
RxLfsFifoUnderflowErr	0x19	0/1	0/1	0/1	0	-	-
LinkSubTxTimeoutErr	0x1a	0/1	0/1	0/1	0	-	-
LinkSubRxTimeoutErr	0x1b	0/1	0/1	0/1	0	-	-
RxCMDPktErr	0x1c	0/1	0/1	0/1	0	-	-
LFDSTrainingTimeoutErr	0x1d	0/1	0/1	0/1	0	-	-
LFDSFcinitTimeoutErr	0x1e	0/1	0/1	0/1	0	-	-
TwixDataLoss	0x1f	0/1	0/1	0/1	0	-	-

**MSR0000_0446...MSRC000_2122 [PCS_GMI Machine Check Address]
(MCA::PCS::GMI::MCA_ADDR_PCS_GMI)**

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::PCS::GMI::MCA_ADDR_PCS_GMI stores an address and other information associated with the error in MCA::PCS::GMI::MCA_STATUS_PCS_GMI. The register is only meaningful if MCA::PCS::GMI::MCA_STATUS_PCS_GMI[Val]=1 and MCA::PCS::GMI::MCA_STATUS_PCS_GMI[AddrV]=1.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2122
_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSRLEGACY; MSR0000_044A
_ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2122
_ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSRLEGACY; MSR0000_044A
_ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2122
_ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSRLEGACY; MSR0000_044A
_ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2122
_ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSRLEGACY; MSR0000_044A
_ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2122
_ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSRLEGACY; MSR0000_044A
_ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2122
_ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSRLEGACY; MSR0000_044A
_ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2122
_ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSRLEGACY; MSR0000_044A
_ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2122
_ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSRLEGACY; MSR0000_044A
_ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2122
_ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSRLEGACY; MSR0000_044A
_ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2122
_ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSRLEGACY; MSR0000_044A
_ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2122
_ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSRLEGACY; MSR0000_044A
_ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2122
_ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSRLEGACY; MSR0000_044A
_ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2122
_ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSRLEGACY; MSR0000_044A
_ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2122
_ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSRLEGACY; MSR0000_044A
_ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2122
_ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSRLEGACY; MSR0000_044A
_ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2122
_ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSRLEGACY; MSR0000_044A
_instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2112
_instGMICONTAINER0PCSGMI_n0_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2112
_instGMICONTAINER10PCSGMI_n20_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2112
_instGMICONTAINER11PCSGMI_n22_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2112
_instGMICONTAINER1PCSGMI_n2_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2112
_instGMICONTAINER2PCSGMI_n4_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2112
_instGMICONTAINER3PCSGMI_n6_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2112
_instGMICONTAINER4PCSGMI_n8_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2112
_instGMICONTAINER5PCSGMI_n10_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2112
_instGMICONTAINER6PCSGMI_n12_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2112
_instGMICONTAINER7PCSGMI_n14_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2112
_instGMICONTAINER8PCSGMI_n16_aliasMSRLEGACY; MSR0000_0446
_instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2112
_instGMICONTAINER9PCSGMI_n18_aliasMSRLEGACY; MSR0000_0446

Bits	Description
------	-------------

63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PCS::GMI::MCA_STATUS_PCS_GMI.
------	---

Table 113: MCA_ADDR_PCS_GMI

Error Type	Bits	Description
DataLossErr	[56:0]	Reserved
TrainingErr	[56:0]	Reserved
ReplayParityTwix	[56:0]	Reserved
RxFifoUnderflowErr	[56:0]	Reserved
RxFifoOverflowErr	[56:0]	Reserved
CRCErr	[56:0]	Reserved
BERExceededErr	[56:0]	Reserved
TxTwixFifoUnderflow	[56:0]	Reserved
ReplayBufParityErr	[56:0]	Reserved
TxTwixOverflow	[56:0]	Reserved
ReplayFifoOverflowErr	[56:0]	Reserved
ReplayFifoUnderflowErr	[56:0]	Reserved
ElasticFifoOverflowErr	[56:0]	Reserved
DeskewErr	[56:0]	Reserved
TwixOffline	[56:0]	Reserved
DataStartupLimitErr	[56:0]	Reserved
FCInitTimeoutErr	[56:0]	Reserved
RecoveryTimeoutErr	[56:0]	Reserved
ReadySerialTimeoutErr	[56:0]	Reserved
ReadySerialAttemptErr	[56:0]	Reserved
RecoveryAttemptErr	[56:0]	Reserved
RecoveryRelockAttemptErr	[56:0]	Reserved
DeskewAbortErr	[56:0]	Reserved
TwixRxBuff	[56:0]	Reserved
RxLfdsFifoOverflowErr	[56:0]	Reserved
RxLfdsFifoUnderflowErr	[56:0]	Reserved
LinkSubTxTimeoutErr	[56:0]	Reserved
LinkSubRxTimeoutErr	[56:0]	Reserved
RxCMDPktErr	[56:0]	Reserved
LFDSTrainingTimeoutErr	[56:0]	Reserved
LFDSFcinitTimeoutErr	[56:0]	Reserved
TwixDataLoss	[56:0]	Reserved

**MSR0000_0447...MSRC000_2123 [PCS_GMI Machine Check Miscellaneous 0]
(MCA::PCS::GMI::MCA_MISC0_PCS_GMI)**

Log miscellaneous information associated with errors.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2123
 _ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSRLEGACY; MSR0000_044B
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2123
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSRLEGACY; MSR0000_044B
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2123
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSRLEGACY; MSR0000_044B
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2123
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSRLEGACY; MSR0000_044B
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2123
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSRLEGACY; MSR0000_044B
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2123
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSRLEGACY; MSR0000_044B
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2123
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSRLEGACY; MSR0000_044B
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2123
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSRLEGACY; MSR0000_044B
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2123
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSRLEGACY; MSR0000_044B
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2123
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSRLEGACY; MSR0000_044B
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2123
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSRLEGACY; MSR0000_044B
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2123
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSRLEGACY; MSR0000_044B
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2123
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSRLEGACY; MSR0000_044B
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2123
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSRLEGACY; MSR0000_044B
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2123
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSRLEGACY; MSR0000_044B
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2123
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSRLEGACY; MSR0000_044B
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2113
 _instGMICONTAINER0PCSGMI_n0_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2113
 _instGMICONTAINER10PCSGMI_n20_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2113
 _instGMICONTAINER11PCSGMI_n22_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2113
 _instGMICONTAINER1PCSGMI_n2_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2113
 _instGMICONTAINER2PCSGMI_n4_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2113
 _instGMICONTAINER3PCSGMI_n6_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2113
 _instGMICONTAINER4PCSGMI_n8_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2113
 _instGMICONTAINER5PCSGMI_n10_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2113
 _instGMICONTAINER6PCSGMI_n12_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2113
 _instGMICONTAINER7PCSGMI_n14_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2113
 _instGMICONTAINER8PCSGMI_n16_aliasMSRLEGACY; MSR0000_0447
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2113
 _instGMICONTAINER9PCSGMI_n18_aliasMSRLEGACY; MSR0000_0447

Bits	Description
------	-------------

63	Valid. Reset: 1. 1=A valid CntP field is present in this register.
----	---

AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.	
---	--

62	CntP . Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr:: HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked . Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr:: HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP . Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr:: HWCR[McStatusWrEn] !MCA::PCS::GMI::MCA_MISC0_PCS_GMI[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset . Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic:: ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr:: HWCR[McStatusWrEn] !MCA::PCS::GMI::MCA_MISC0_PCS_GMI[Locked]) ? Read-write : Read-only.
51	CntEn . Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr:: HWCR[McStatusWrEn] !MCA::PCS::GMI::MCA_MISC0_PCS_GMI[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType . Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr:: McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr:: HWCR[McStatusWrEn] !MCA::PCS::GMI::MCA_MISC0_PCS_GMI[Locked]) ? Read-write : Read-only.
48	Ovrflw . Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr:: HWCR[McStatusWrEn] !MCA::PCS::GMI::MCA_MISC0_PCS_GMI[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt . Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr:: HWCR[McStatusWrEn] !MCA::PCS::GMI::MCA_MISC0_PCS_GMI[Locked]) ? Read-write : Read-only.
31:24	BlkPtr . Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

**MSRC000_21[1...2]4 [PCS_GMI Machine Check Configuration]
(MCA::PCS::GMI::MCA_CONFIG_PCS_GMI)**

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2124
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2124
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2124
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2124
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2124
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2124
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2124
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2124
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2124
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2124
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2124
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2124
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2124
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2124
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2124
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2124
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2114
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2114
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2114
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2114
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2114
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2114
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2114
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2114
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2114
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2114
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2114
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2114

Bits	Description
63:39	Reserved.
38:37	DeferredIntType. Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat. Read-write. Reset: 0. Init: BIOS,0. 1=Log deferred errors in MCA::PCS::GMI::MCA_STATUS_PCS_GMI and MCA::PCS::GMI::MCA_ADDR_PCS_GMI in addition to MCA::PCS::GMI::MCA_DESTAT_PCS_GMI and MCA::PCS::GMI::MCA_DEADDR_PCS_GMI. 0=Only log deferred errors in MCA::PCS::GMI::MCA_DESTAT_PCS_GMI and MCA::PCS::GMI::MCA_DEADDR_PCS_GMI. This bit does not affect logging of deferred errors in MCA::PCS::GMI::MCA_SYND_PCS_GMI, MCA::PCS::GMI::MCA_MISC0_PCS_GMI.
33	TransparentErrorLoggingEnable. Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::PCS::GMI::MCA_STATUS_PCS_GMI, MCA::PCS::GMI::MCA_ADDR_PCS_GMI, MCA::PCS::GMI::MCA_MISC0_PCS_GMI, and MCA::PCS::GMI::MCA_SYND_PCS_GMI.
32	McaXEnable. Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported. Read-only. Reset: 1. 1=MCA::PCS::GMI::MCA_CONFIG_PCS_GMI[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.

5	DeferredIntTypeSupported. Read-only. Reset: 1. 1=MCA::PCS::GMI::MCA_CONFIG_PCS_GMI[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::PCS::GMI::MCA_CONFIG_PCS_GMI[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported. Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and MCA::PCS::GMI::MCA_CONFIG_PCS_GMI[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::PCS::GMI::MCA_DESTAT_PCS_GMI and MCA::PCS::GMI::MCA_DEADDR_PCS_GMI are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported. Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::PCS::GMI::MCA_MISC0_PCS_GMI[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::PCS::GMI::MCA_STATUS_PCS_GMI[TCC] is present.

MSRC000_21[1...2]5 [PCS_GMI IP Identification] (MCA::PCS::GMI::MCA_IPID_PCS_GMI)

Reset: 0000_0241_0000_0000h.

The MCA::PCS::GMI::MCA_IPID_PCS_GMI register is used by software to determine what IP type and revision is associated with the MCA bank.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2125
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2125
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2125
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2125
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2125
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2125
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2125
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2125
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2125
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2125
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2125
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2125
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2125
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2125
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2125
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2125
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2115
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2115
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2115
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2115
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2115
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2115
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2115
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2115
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2115
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2115
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2115
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2115

Bits	Description
63:48	McaType. Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 241h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register. Init: _ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR: 3070_9200h Init: _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR: 3080_9200h Init: _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR: 3270_9200h Init: _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR: 3280_9200h Init: _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR: 3470_9200h Init: _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR: 3480_9200h Init: _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR: 3670_9200h Init: _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR: 3680_9200h Init: _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR: 3870_9200h Init: _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR: 3A70_9200h Init: _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR: 3C70_9200h Init: _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR: 3E70_9200h Init: _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR: 4A70_9200h Init: _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR: 4C70_9200h Init: _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR: 4E70_9200h Init: _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR: 5070_9200h Init: _instGMICONTAINER0PCSGMI_n0_aliasMSR: 1330_9200h Init: _instGMICONTAINER10PCSGMI_n20_aliasMSR: 19A0_9200h Init: _instGMICONTAINER11PCSGMI_n22_aliasMSR: 19B0_9200h Init: _instGMICONTAINER1PCSGMI_n2_aliasMSR: 1340_9200h Init: _instGMICONTAINER2PCSGMI_n4_aliasMSR: 1350_9200h Init: _instGMICONTAINER3PCSGMI_n6_aliasMSR: 1360_9200h

Init: _instGMICONTAINER4PCSGMI_n8_aliasMSR: 1370_9200h
Init: _instGMICONTAINER5PCSGMI_n10_aliasMSR: 1380_9200h
Init: _instGMICONTAINER6PCSGMI_n12_aliasMSR: 1390_9200h
Init: _instGMICONTAINER7PCSGMI_n14_aliasMSR: 17C0_9200h
Init: _instGMICONTAINER8PCSGMI_n16_aliasMSR: 17D0_9200h
Init: _instGMICONTAINER9PCSGMI_n18_aliasMSR: 17E0_9200h

MSRC000_21[1...2]6 [PCS_GMI Machine Check Syndrome] (MCA::PCS::GMI::MCA_SYND_PCS_GMI)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::PCS::GMI::MCA_STATUS_PCS_GMI [Thread 0](#)[_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR](#); MSRC000_2126[_ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR](#); MSRC000_2126[_ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR](#); MSRC000_2126[_ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR](#); MSRC000_2126[_ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR](#); MSRC000_2126[_ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR](#); MSRC000_2126[_ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR](#); MSRC000_2126[_ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR](#); MSRC000_2126[_ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR](#); MSRC000_2126[_ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR](#); MSRC000_2126[_ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR](#); MSRC000_2126[_ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR](#); MSRC000_2126[_ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR](#); MSRC000_2126[_ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR](#); MSRC000_2126[_ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR](#); MSRC000_2126[_ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR](#); MSRC000_2126[_instGMICONTAINER0PCSGMI_n0_aliasMSR](#); MSRC000_2116[_instGMICONTAINER10PCSGMI_n20_aliasMSR](#); MSRC000_2116[_instGMICONTAINER11PCSGMI_n22_aliasMSR](#); MSRC000_2116[_instGMICONTAINER1PCSGMI_n2_aliasMSR](#); MSRC000_2116[_instGMICONTAINER2PCSGMI_n4_aliasMSR](#); MSRC000_2116[_instGMICONTAINER3PCSGMI_n6_aliasMSR](#); MSRC000_2116[_instGMICONTAINER4PCSGMI_n8_aliasMSR](#); MSRC000_2116[_instGMICONTAINER5PCSGMI_n10_aliasMSR](#); MSRC000_2116[_instGMICONTAINER6PCSGMI_n12_aliasMSR](#); MSRC000_2116[_instGMICONTAINER7PCSGMI_n14_aliasMSR](#); MSRC000_2116[_instGMICONTAINER8PCSGMI_n16_aliasMSR](#); MSRC000_2116[_instGMICONTAINER9PCSGMI_n18_aliasMSR](#); MSRC000_2116

Bits	Description
63:32	Syndrome. Read-write, Volatile. Reset: Cold, 0000_0000h. Contains the syndrome, if any, associated with the error logged in MCA::PCS::GMI::MCA_STATUS_PCS_GMI. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PCS::GMI::MCA_SYND_PCS_GMI[Length]. The Syndrome field is only valid when MCA::PCS::GMI::MCA_SYND_PCS_GMI[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PCS::GMI::MCA_SYND_PCS_GMI. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PCS::GMI::MCA_SYND_PCS_GMI[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PCS::GMI::MCA_SYND_PCS_GMI. For example, a syndrome length of 9 means that MCA::PCS::GMI::MCA_SYND_PCS_GMI[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 114 [MCA_SYND_PCS_GMI].

Table 114: MCA_SYND_PCS_GMI

Error Type	Bits	Description
DataLossErr	[17:0]	Reserved
TrainingErr	[17:0]	Reserved

ReplayParityTwix	[17:0]	Reserved
RxFifoUnderflowErr	[17:0]	Reserved
RxFifoOverflowErr	[17:0]	Reserved
CRCErr	[17:0]	Reserved
BERExceededErr	[17:0]	Reserved
TxTwixFifoUnderflow	[17:0]	Reserved
ReplayBufParityErr	[17:0]	Reserved
TxTwixOverflow	[17:0]	Reserved
ReplayFifoOverflowErr	[17:0]	Reserved
ReplayFifoUnderflowErr	[17:0]	Reserved
ElasticFifoOverflowErr	[17:0]	Reserved
DeskewErr	[17:0]	Reserved
TwixOffline	[17:0]	Reserved
DataStartupLimitErr	[17:0]	Reserved
FCInitTimeoutErr	[17:0]	Reserved
RecoveryTimeoutErr	[17:0]	Reserved
ReadySerialTimeoutErr	[17:0]	Reserved
ReadySerialAttemptErr	[17:0]	Reserved
RecoveryAttemptErr	[17:0]	Reserved
RecoveryRelockAttemptErr	[17:0]	Reserved
DeskewAbortErr	[17:0]	Reserved
TwixRxBuff	[17:0]	Reserved
RxLfdsFifoOverflowErr	[17:0]	Reserved
RxLfdsFifoUnderflowErr	[17:0]	Reserved
LinkSubTxTimeoutErr	[17:0]	Reserved
LinkSubRxTimeoutErr	[17:0]	Reserved
RxCMDPktErr	[17:0]	Reserved
LFDSTrainingTimeoutErr	[17:0]	Reserved
LFDSFcinitTimeoutErr	[17:0]	Reserved
TwixDataLoss	[17:0]	Reserved

**MSRC000_21[1...2]8 [PCS_GMI Machine Check Deferred Error Status]
(MCA::PCS::GMI::MCA_DESTAT_PCS_GMI)**

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2128
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2128
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2128
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2128
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2128
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2128
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2128
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2128
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2128
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2128
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2128
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2128
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2128
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2128
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2128
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2128
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2118
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2118
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2118
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2118
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2118
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2118
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2118
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2118
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2118
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2118
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2118
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2118

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::PCS::GMI::MCA_DEADDR_PCS_GMI contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::PCS::GMI::MCA_SYND_PCS_GMI. If MCA::PCS::GMI::MCA_SYND_PCS_GMI[ErrorPriority] is the same as the priority of the error in MCA::PCS::GMI::MCA_STATUS_PCS_GMI, then the information in MCA::PCS::GMI::MCA_SYND_PCS_GMI is associated with the error in MCA::PCS::GMI::MCA_DESTAT_PCS_GMI.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.

29:24	AddrLsb. Read-write, Volatile. Reset: Cold, 00h. Specifies the least significant valid bit of the address contained in MCA::PCS::GMI::MCA_ADDR_PCS_GMI[ErrorAddr]. A value of 0 indicates that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[55:0] contains a valid byte address. A value of 6 indicates that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[55:6] contains a valid cache line address and that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[55:12] contain a valid 4KB memory page and that MCA::PCS::GMI::MCA_ADDR_PCS_GMI[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold, 0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold, 00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold, 0000h. Error code for this error.

MSRC000_21[1...2]9 [PCS_GMI Deferred Error Address] (MCA::PCS::GMI::MCA_DEADDR_PCS_GMI)

Read-only. Reset: Cold, 0000_0000_0000_0000h.

The MCA::PCS::GMI::MCA_DEADDR_PCS_GMI register stores the address associated with the error in MCA::PCS::GMI::MCA_DESTAT_PCS_GMI. The register is only meaningful if MCA::PCS::GMI::MCA_DESTAT_PCS_GMI[Val]=1 and MCA::PCS::GMI::MCA_DESTAT_PCS_GMI[AddrV]=1. The lowest valid bit of the address is defined by MCA::PCS::GMI::MCA_DESTAT_PCS_GMI[AddrLsb].

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC000_2129

_ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC000_2129

_ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC000_2129

_ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC000_2129

_ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC000_2129

_ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC000_2129

_ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC000_2129

_ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC000_2129

_ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC000_2129

_ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC000_2129

_ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC000_2129

_ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC000_2129

_ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC000_2129

_ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC000_2129

_ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC000_2129

_ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC000_2129

_instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC000_2119

_instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC000_2119

_instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC000_2119

_instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC000_2119

_instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC000_2119

_instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC000_2119

_instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC000_2119

_instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC000_2119

_instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC000_2119

_instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC000_2119

_instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC000_2119

_instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC000_2119

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold, 0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PCS::GMI::MCA_DESTAT_PCS_GMI.

**MSRC001_041[1...2] [PCS_GMI Machine Check Control Mask]
(MCA::PCS::GMI::MCA_CTL_MASK_PCS_GMI)**

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ccd0_instGMICONTAINER12PCSGMI_n1_aliasMSR; MSRC001_0412
 _ccd0_instGMICONTAINER13PCSGMI_n24_aliasMSR; MSRC001_0412
 _ccd10_instGMICONTAINER12PCSGMI_n21_aliasMSR; MSRC001_0412
 _ccd11_instGMICONTAINER12PCSGMI_n23_aliasMSR; MSRC001_0412
 _ccd1_instGMICONTAINER12PCSGMI_n3_aliasMSR; MSRC001_0412
 _ccd1_instGMICONTAINER13PCSGMI_n25_aliasMSR; MSRC001_0412
 _ccd2_instGMICONTAINER12PCSGMI_n5_aliasMSR; MSRC001_0412
 _ccd2_instGMICONTAINER13PCSGMI_n26_aliasMSR; MSRC001_0412
 _ccd3_instGMICONTAINER12PCSGMI_n7_aliasMSR; MSRC001_0412
 _ccd3_instGMICONTAINER13PCSGMI_n27_aliasMSR; MSRC001_0412
 _ccd4_instGMICONTAINER12PCSGMI_n9_aliasMSR; MSRC001_0412
 _ccd5_instGMICONTAINER12PCSGMI_n11_aliasMSR; MSRC001_0412
 _ccd6_instGMICONTAINER12PCSGMI_n13_aliasMSR; MSRC001_0412
 _ccd7_instGMICONTAINER12PCSGMI_n15_aliasMSR; MSRC001_0412
 _ccd8_instGMICONTAINER12PCSGMI_n17_aliasMSR; MSRC001_0412
 _ccd9_instGMICONTAINER12PCSGMI_n19_aliasMSR; MSRC001_0412
 _instGMICONTAINER0PCSGMI_n0_aliasMSR; MSRC001_0411
 _instGMICONTAINER10PCSGMI_n20_aliasMSR; MSRC001_0411
 _instGMICONTAINER11PCSGMI_n22_aliasMSR; MSRC001_0411
 _instGMICONTAINER1PCSGMI_n2_aliasMSR; MSRC001_0411
 _instGMICONTAINER2PCSGMI_n4_aliasMSR; MSRC001_0411
 _instGMICONTAINER3PCSGMI_n6_aliasMSR; MSRC001_0411
 _instGMICONTAINER4PCSGMI_n8_aliasMSR; MSRC001_0411
 _instGMICONTAINER5PCSGMI_n10_aliasMSR; MSRC001_0411
 _instGMICONTAINER6PCSGMI_n12_aliasMSR; MSRC001_0411
 _instGMICONTAINER7PCSGMI_n14_aliasMSR; MSRC001_0411
 _instGMICONTAINER8PCSGMI_n16_aliasMSR; MSRC001_0411
 _instGMICONTAINER9PCSGMI_n18_aliasMSR; MSRC001_0411

Bits	Description
------	-------------

63:32	Reserved.
-------	-----------

31	TwixDataLoss. Read-write. Reset: 0. TwixDataLoss
30	LFDSFcinitTimeoutErr. Read-write. Reset: 0. LFDSFcinitTimeoutErr
29	LFDSTrainingTimeoutErr. Read-write. Reset: 0. LFDSTrainingTimeoutErr
28	RxCMDPktErr. Read-write. Reset: 0. RxCMDPktErr
27	LinkSubRxTimeoutErr. Read-write. Reset: 0. LinkSubRxTimeoutErr
26	LinkSubTxTimeoutErr. Read-write. Reset: 0. LinkSubTxTimeoutErr
25	RxLfdsFifoUnderflowErr. Read-write. Reset: 0. RxLfdsFifoUnderflowErr
24	RxLfdsFifoOverflowErr. Read-write. Reset: 0. RxLfdsFifoOverflowErr
23	TwixRxBuff. Read-write. Reset: 0. TwixRxBuff
22	DeskewAbortErr. Read-write. Reset: 0. DeskewAbortErr
21	RecoveryRelockAttemptErr. Read-write. Reset: 0. RecoveryRelockAttemptErr
20	RecoveryAttemptErr. Read-write. Reset: 0. RecoveryAttemptErr
19	ReadySerialAttemptErr. Read-write. Reset: 0. ReadySerialAttemptErr
18	ReadySerialTimeoutErr. Read-write. Reset: 0. ReadySerialTimeoutErr
17	RecoveryTimeoutErr. Read-write. Reset: 0. RecoveryTimeoutErr
16	FCInitTimeoutErr. Read-write. Reset: 0. FCInitTimeoutErr
15	DataStartupLimitErr. Read-write. Reset: 0. DataStartupLimitErr
14	TwixOffline. Read-write. Reset: 0. TwixOffline
13	DeskewErr. Read-write. Reset: 0. DeskewErr
12	ElasticFifoOverflowErr. Read-write. Reset: 0. ElasticFifoOverflowErr
11	ReplayFifoUnderflowErr. Read-write. Reset: 0. ReplayFifoUnderflowErr

10	ReplayFifoOverflowErr. Read-write. Reset: 0. ReplayFifoOverflowErr
9	TxTwixOverflow. Read-write. Reset: 0. TxTwixOverflow
8	ReplayBufParityErr. Read-write. Reset: 0. ReplayBufParityErr
7	TxTwixFifoUnderflow. Read-write. Reset: 0. TxTwixFifoUnderflow
6	BERExceededErr. Read-write. Reset: 0. BERExceededErr
5	CRCErr. Read-write. Reset: 0. CRCErr
4	RxFifoOverflowErr. Read-write. Reset: 0. RxFifoOverflowErr
3	RxFifoUnderflowErr. Read-write. Reset: 0. RxFifoUnderflowErr
2	ReplayParityTwix. Read-write. Reset: 0. ReplayParityTwix
1	TrainingErr. Read-write. Reset: 0. TrainingErr.
0	DataLossErr. Read-write. Reset: 0. DataLossErr

3.2.5.20 KPX WAFL

MSR0000_0470...MSRC000_21C0 [KPX_WAFL Machine Check Control] (MCA::KPX::WAFL::MCA_CTL_KPX_WAFL)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::KPX::WAFL::MCA_CTL_KPX_WAFL register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C0

_instKPXWAFL_n[1:0]_aliasMSRLEGACY; MSR0000_0470

Bits	Description
63:4	Reserved.
3	APB. Read-write. Reset: 0. PHY APB error
2	ARCData. Read-write. Reset: 0. ARC data buffer parity error
1	ARCIns. Read-write. Reset: 0. ARC instruction buffer parity error
0	RAMECC. Read-write. Reset: 0. RAM ECC Error.

**MSR0000_0471...MSRC000_21C1 [KPX_WAFL Machine Check Status]
(MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL)**

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C1

_instKPXWAFL_n[1:0]_aliasMSRLEGACY; MSR0000_0471

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::KPX::WAFL::MCA_CTL_KPX_WAFL. This bit is a copy of bit in MCA::KPX::WAFL::MCA_CTL_KPX_WAFL for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL. If MCA::KPX::WAFL::MCA_SYND_KPX_WAFL[ErrorPriority] is the same as the priority of the error in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL, then the information in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL is associated with the error in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

46	CECC . Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC . Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred . Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison . Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41 . Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub . Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38 . Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId . Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30 . Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb . Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[ErrorAddr]. A value of 0 indicates that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[55:0] contains a valid byte address. A value of 6 indicates that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[55:6] contains a valid cache line address and that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[55:12] contain a valid 4KB memory page and that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22 . Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt . Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::KPX::WAFL::MCA_CTL_KPX_WAFL enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode . Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 115: MCA_STATUS_KPX_WAFL

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
RAMECC	0x0	0/1	0/1	0/1	0	-	-
ARCIns	0x1	1	1	1	0	-	-
ARCDData	0x2	1	1	1	0	-	-
APB	0x3	1	1	1	0	-	-

**MSR0000_0472...MSRC000_21C2 [KPX_WAFL Machine Check Address]
(MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL)**

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL stores an address and other information associated with the error in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL. The register is only meaningful if MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL[Val]=1 and MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL[AddrV]=1.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C2

_instKPXWAFL_n[1:0]_aliasMSRLEGACY; MSR0000_0472

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL.

Table 116: MCA_ADDR_KPX_WAFL

Error Type	Bits	Description
RAMECC	[56:0]	Reserved
ARCIns	[56:0]	Reserved
ARCData	[56:0]	Reserved
APB	[56:0]	Reserved

**MSR0000_0473...MSRC000_21C3 [KPX_WAFL Machine Check Miscellaneous 0]
(MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL)**

Log miscellaneous information associated with errors.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C3

_instKPXWAFL_n[1:0]_aliasMSRLEGACY; MSR0000_0473

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] ! MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] ! MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] ! MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] ! MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] ! MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] ! MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

**MSRC000_21C4 [KPX_WAFL Machine Check Configuration]
(MCA::KPX::WAFL::MCA_CONFIG_KPX_WAFL)**

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL and MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL in addition to MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL and MCA::KPX::WAFL::MCA_DEADDR_KPX_WAFL. 0=Only log deferred errors in MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL and MCA::KPX::WAFL::MCA_DEADDR_KPX_WAFL. This bit does not affect logging of deferred errors in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL, MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL, MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL, MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL, and MCA::KPX::WAFL::MCA_SYND_KPX_WAFL.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::KPX::WAFL::MCA_CONFIG_KPX_WAFL[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::KPX::WAFL::MCA_CONFIG_KPX_WAFL[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::KPX::WAFL::MCA_CONFIG_KPX_WAFL[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::KPX::WAFL::MCA_CONFIG_KPX_WAFL[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL and MCA::KPX::WAFL::MCA_DEADDR_KPX_WAFL are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::KPX::WAFL::MCA_MISC0_KPX_WAFL[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL[TCC] is present.

MSRC000_21C5 [KPX_WAFL IP Identification] (MCA::KPX::WAFL::MCA_IPID_KPX_WAFL)

Reset: 0000_0267_0000_0000h.

The MCA::KPX::WAFL::MCA_IPID_KPX_WAFL register is used by software to determine what IP type and revision is associated with the MCA bank.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C5

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 267h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instKPXWAFL_n0_aliasMSR: 1870_0400h
	Init: _instKPXWAFL_n1_aliasMSR: 1880_0400h

MSRC000_21C6 [KPX_WAFL Machine Check Syndrome] (MCA::KPX::WAFL::MCA_SYND_KPX_WAFL)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL [Thread 0](#)

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C6

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::KPX::WAFL::MCA_SYND_KPX_WAFL[Length]. The Syndrome field is only valid when MCA::KPX::WAFL::MCA_SYND_KPX_WAFL[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL. For example, a syndrome length of 9 means that MCA::KPX::WAFL::MCA_SYND_KPX_WAFL[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 117 [MCA_SYND_KPX_WAFL].

Table 117: MCA_SYND_KPX_WAFL

Error Type	Bits	Description
RAMECC	[17:0]	Reserved
ARCIns	[17:0]	Reserved
ARCDData	[17:0]	Reserved
APB	[17:0]	Reserved

MSRC000_21C8 [KPX_WAFL Machine Check Deferred Error Status] (MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C8

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::KPX::WAFL::MCA_DEADDR_KPX_WAFL contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL. If MCA::KPX::WAFL::MCA_SYND_KPX_WAFL[ErrorPriority] is the same as the priority of the error in MCA::KPX::WAFL::MCA_STATUS_KPX_WAFL, then the information in MCA::KPX::WAFL::MCA_SYND_KPX_WAFL is associated with the error in MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[ErrorAddr]. A value of 0 indicates that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[55:0] contains a valid byte address. A value of 6 indicates that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[55:6] contains a valid cache line address and that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[55:12] contain a valid 4KB memory page and that MCA::KPX::WAFL::MCA_ADDR_KPX_WAFL[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21C9 [KPX_WAFL Deferred Error Address] (MCA::KPX::WAFL::MCA_DEADDR_KPX_WAFL)

Read-only. Reset: Cold,0000_0000_0000_0000h.

The MCA::KPX::WAFL::MCA_DEADDR_KPX_WAFL register stores the address associated with the error in MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL. The register is only meaningful if MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL[Val]=1 and MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL[AddrV]=1. The lowest valid bit of the address is defined by MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL[AddrLsb].

_instKPXWAFL_n[1:0]_aliasMSR; MSRC000_21C9

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::KPX::WAFL::MCA_DESTAT_KPX_WAFL.

**MSRC001_041C [KPX_WAFL Machine Check Control Mask]
(MCA::KPX::WAFL::MCA_CTL_MASK_KPX_WAFL)**

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instKPXWAFL_n[1:0]_aliasMSR; MSRC001_041C

Bits	Description
63:4	Reserved.
3	APB . Read-write. Reset: 0. PHY APB error
2	ARCData . Read-write. Reset: 0. ARC data buffer parity error
1	ARCIns . Read-write. Reset: 0. ARC instruction buffer parity error
0	RAMECC . Read-write. Reset: 0. RAM ECC Error.

3.2.5.21 PCS XGMI

**MSR0000_0478...MSRC000_21E0 [PCS_XGMI Machine Check Control]
(MCA::PCS::XGMI::MCA_CTL_PCS_XGMI)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::PCS::XGMI::MCA_CTL_PCS_XGMI register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E0

_instSERDESAG3PCSXGMI_n7_aliasMSRLEGACY; MSR0000_0478

_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E0

_instSERDESAP0PCSXGMI_n0_aliasMSRLEGACY; MSR0000_0478

_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E0

_instSERDESBG0PCSXGMI_n4_aliasMSRLEGACY; MSR0000_0478

_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E0

_instSERDESBG1PCSXGMI_n5_aliasMSRLEGACY; MSR0000_0478

_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E0

_instSERDESBG2PCSXGMI_n6_aliasMSRLEGACY; MSR0000_0478

_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E0

_instSERDESBP1PCSXGMI_n1_aliasMSRLEGACY; MSR0000_0478

_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E0

_instSERDESBP2PCSXGMI_n2_aliasMSRLEGACY; MSR0000_0478

_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E0

_instSERDESBP3PCSXGMI_n3_aliasMSRLEGACY; MSR0000_0478

Bits	Description
------	-------------

63:29	Reserved.
-------	-----------

28	RxCMDPktErr. Read-write. Reset: 0. RxCMDPktErr for xGMI3
----	---

27	LinkSubRxTimeoutErr. Read-write. Reset: 0. LinkSubRxTimeoutErr for xGMI3
----	---

26	LinkSubTxTimeoutErr. Read-write. Reset: 0. LinkSubTxTimeoutErr for xGMI3
----	---

25	RxReplayTimeoutErr. Read-write. Reset: 0. RxReplayTimeoutErr for xGMI3
----	---

24	TxReplayTimeoutErr. Read-write. Reset: 0. TxReplayTimeoutErr for xGMI3
----	---

23	SyncHdrErr. Read-write. Reset: 0. SyncHdrErr for xGMI3
----	---

22	ReplayAttemptErr. Read-write. Reset: 0. ReplayAttemptErr for xGMI3
----	---

21	RecoveryRelockAttemptErr. Read-write. Reset: 0. RecoveryRelockAttemptErr
----	---

20	RecoveryAttemptErr. Read-write. Reset: 0. RecoveryAttemptErr
----	---

19	ReadySerialAttemptErr. Read-write. Reset: 0. ReadySerialAttemptErr
----	---

18	ReadySerialTimeoutErr. Read-write. Reset: 0. ReadySerialTimeoutErr
----	---

17	RecoveryTimeoutErr. Read-write. Reset: 0. RecoveryTimeoutErr
----	---

16	FCInitTimeoutErr. Read-write. Reset: 0. FCInitTimeoutErr
----	---

15	DataStartupLimitErr. Read-write. Reset: 0. DataStartupLimitErr
----	---

14	FlowCtrlCRCErr. Read-write. Reset: 0. FlowCtrlCRCErr for xGMI3
----	---

13	DeskewErr. Read-write. Reset: 0. DeskewErr
----	---

12	ElasticFifoOverflowErr. Read-write. Reset: 0. ElasticFifoOverflowErr
----	---

11	ReplayFifoUnderflowErr. Read-write. Reset: 0. ReplayFifoUnderflowErr
----	---

10	ReplayFifoOverflowErr. Read-write. Reset: 0. ReplayFifoOverflowErr
----	---

9	DataParityErr. Read-write. Reset: 0. DataParityErr
---	---

8	ReplayBufParityErr. Read-write. Reset: 0. ReplayBufParityErr
---	---

7	TxMetaDataErr_TxVcidDataErr. Read-write. Reset: 0. TxVcidDataErr for xGMI3.
---	--

6	BERExceededErr. Read-write. Reset: 0. BERExceededErr
---	---

5	CRCErr. Read-write. Reset: 0. CRCErr
---	---

4	RxFifoOverflowErr. Read-write. Reset: 0. RxFifoOverflowErr for xGMI3
---	---

3	RxFifoUnderflowErr. Read-write. Reset: 0. RxFifoUnderflowErr for xGMI3
---	---

2	FlowCtrlAckErr. Read-write. Reset: 0. FlowCtrlAckErr for xGMI3.
---	--

1	TrainingErr. Read-write. Reset: 0. TrainingErr.
---	--

0	DataLossErr. Read-write. Reset: 0. DataLossErr
---	---

**MSR0000_0479...MSRC000_21E1 [PCS_XGMI Machine Check Status]
(MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI)**

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

[_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E1](#)[_instSERDESAG3PCSXGMI_n7_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E1](#)[_instSERDESAP0PCSXGMI_n0_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E1](#)[_instSERDESBG0PCSXGMI_n4_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E1](#)[_instSERDESBG1PCSXGMI_n5_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E1](#)[_instSERDESBG2PCSXGMI_n6_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E1](#)[_instSERDESBP1PCSXGMI_n1_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E1](#)[_instSERDESBP2PCSXGMI_n2_aliasMSRLEGACY; MSR0000_0479](#)[_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E1](#)[_instSERDESBP3PCSXGMI_n3_aliasMSRLEGACY; MSR0000_0479](#)

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::PCS::XGMI::MCA_CTL_PCS_XGMI. This bit is a copy of bit in MCA::PCS::XGMI::MCA_CTL_PCS_XGMI for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI. If MCA::PCS::XGMI::MCA_SYND_PCS_XGMI[ErrorPriority] is the same as the priority of the error in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI, then the information in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI is associated with the error in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI[UC]=0. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[ErrorAddr]. A value of 0 indicates that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[55:0] contains a valid byte address. A value of 6 indicates that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[55:6] contains a valid cache line address and that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[55:12] contain a valid 4KB memory page and that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[11:0] should be ignored by error handling software. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::PCS::XGMI::MCA_CTL_PCS_XGMI enables error reporting for the logged error. AccessType: Core::X86::Msrr: HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 118: MCA_STATUS_PCS_XGMI

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
DataLossErr	0x0	0/1	0/1	0/1	0	-	-
TrainingErr	0x1	0/1	0/1	0/1	0	-	-
FlowCtrlAck Err	0x2	0/1	0/1	0/1	0	-	-
RxFifoUnderflowErr	0x3	0/1	0/1	0/1	0	-	-
RxFifoOverflowErr	0x4	0/1	0/1	0/1	0	-	-
CRCErr	0x5	0/1	0/1	0/1	0	-	-
BERExceededErr	0x6	0/1	0/1	0/1	0	-	-
TxMetaDataErr_TxVcidDataErr	0x7	0/1	0/1	0/1	0	-	-
ReplayBufParityErr	0x8	0/1	0/1	0/1	0	-	-
DataParityErr	0x9	0/1	0/1	0/1	0	-	-
ReplayFifoOverflowErr	0xa	0/1	0/1	0/1	0	-	-
ReplayFifoUnderflowErr	0xb	0/1	0/1	0/1	0	-	-
ElasticFifoOverflowErr	0xc	0/1	0/1	0/1	0	-	-
DeskewErr	0xd	0/1	0/1	0/1	0	-	-
FlowCtrlCRC Err	0xe	0/1	0/1	0/1	0	-	-
DataStartupLimitErr	0xf	0/1	0/1	0/1	0	-	-
FCInitTimeoutErr	0x10	0/1	0/1	0/1	0	-	-
RecoveryTimeoutErr	0x11	0/1	0/1	0/1	0	-	-
ReadySerialTimeoutErr	0x12	0/1	0/1	0/1	0	-	-
ReadySerialAttemptErr	0x13	0/1	0/1	0/1	0	-	-
RecoveryAttemptErr	0x14	0/1	0/1	0/1	0	-	-
RecoveryRelockAttemptErr	0x15	0/1	0/1	0/1	0	-	-
ReplayAttem	0x16	0/1	0/1	0/1	0	-	-

ptErr							
SyncHdrErr	0x17	0/1	0/1	0/1	0	-	-
TxReplayTimeoutErr	0x18	0/1	0/1	0/1	0	-	-
RxReplayTimeoutErr	0x19	0/1	0/1	0/1	0	-	-
LinkSubTxTimeoutErr	0x1a	0/1	0/1	0/1	0	-	-
LinkSubRxTimeoutErr	0x1b	0/1	0/1	0/1	0	-	-
RxCMDPktErr	0x1c	0/1	0/1	0/1	0	-	-
Reserved_29	0x1d	0/1	0/1	0/1	0	-	-
Reserved_30	0x1e	0/1	0/1	0/1	0	-	-
Reserved_31	0x1f	0/1	0/1	0/1	0	-	-

MSR0000_047A...MSRC000_21E2 [PCS_XGMI Machine Check Address] (MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI stores an address and other information associated with the error in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI. The register is only meaningful if MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI[Val]=1 and MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI[AddrV]=1.

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E2
_instSERDESAG3PCSXGMI_n7_aliasMSRLEGACY; MSR0000_047A
_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E2
_instSERDESAP0PCSXGMI_n0_aliasMSRLEGACY; MSR0000_047A
_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E2
_instSERDESBG0PCSXGMI_n4_aliasMSRLEGACY; MSR0000_047A
_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E2
_instSERDESBG1PCSXGMI_n5_aliasMSRLEGACY; MSR0000_047A
_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E2
_instSERDESBG2PCSXGMI_n6_aliasMSRLEGACY; MSR0000_047A
_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E2
_instSERDESBP1PCSXGMI_n1_aliasMSRLEGACY; MSR0000_047A
_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E2
_instSERDESBP2PCSXGMI_n2_aliasMSRLEGACY; MSR0000_047A
_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E2
_instSERDESBP3PCSXGMI_n3_aliasMSRLEGACY; MSR0000_047A

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI.

Table 119: MCA_ADDR_PCS_XGMI

Error Type	Bits	Description
DataLossErr	[56:0]	Reserved
TrainingErr	[56:0]	Reserved
FlowCtrlAckErr	[56:0]	Reserved
RxFifoUnderflowErr	[56:0]	Reserved
RxFifoOverflowErr	[56:0]	Reserved
CRCErr	[56:0]	Reserved
BERExceededErr	[56:0]	Reserved
TxMetaDataErr_TxVcidDataErr	[56:0]	Reserved

ReplayBufParityErr	[56:0]	Reserved
DataParityErr	[56:0]	Reserved
ReplayFifoOverflowErr	[56:0]	Reserved
ReplayFifoUnderflowErr	[56:0]	Reserved
ElasticFifoOverflowErr	[56:0]	Reserved
DeskewErr	[56:0]	Reserved
FlowCtrlCRCErr	[56:0]	Reserved
DataStartupLimitErr	[56:0]	Reserved
FCInitTimeoutErr	[56:0]	Reserved
RecoveryTimeoutErr	[56:0]	Reserved
ReadySerialTimeoutErr	[56:0]	Reserved
ReadySerialAttemptErr	[56:0]	Reserved
RecoveryAttemptErr	[56:0]	Reserved
RecoveryRelockAttemptErr	[56:0]	Reserved
ReplayAttemptErr	[56:0]	Reserved
SyncHdrErr	[56:0]	Reserved
TxReplayTimeoutErr	[56:0]	Reserved
RxReplayTimeoutErr	[56:0]	Reserved
LinkSubTxTimeoutErr	[56:0]	Reserved
LinkSubRxTimeoutErr	[56:0]	Reserved
RxCMDPktErr	[56:0]	Reserved
Reserved_29	[56:0]	Reserved
Reserved_30	[56:0]	Reserved
Reserved_31	[56:0]	Reserved

**MSR0000_047B...MSRC000_21E3 [PCS_XGMI Machine Check Miscellaneous 0]
(MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI)**

Log miscellaneous information associated with errors.

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E3
_instSERDESAG3PCSXGMI_n7_aliasMSRLEGACY; MSR0000_047B
_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E3
_instSERDESAP0PCSXGMI_n0_aliasMSRLEGACY; MSR0000_047B
_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E3
_instSERDESBG0PCSXGMI_n4_aliasMSRLEGACY; MSR0000_047B
_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E3
_instSERDESBG1PCSXGMI_n5_aliasMSRLEGACY; MSR0000_047B
_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E3
_instSERDESBG2PCSXGMI_n6_aliasMSRLEGACY; MSR0000_047B
_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E3
_instSERDESBP1PCSXGMI_n1_aliasMSRLEGACY; MSR0000_047B
_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E3
_instSERDESBP2PCSXGMI_n2_aliasMSRLEGACY; MSR0000_047B
_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E3
_instSERDESBP3PCSXGMI_n3_aliasMSRLEGACY; MSR0000_047B

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[Locked]) ? Read-write : Read-only.
47:44	Reserved.

43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported.
	AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] ! MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

**MSRC000_21E4 [PCS_XGMI Machine Check Configuration]
(MCA::PCS::XGMI::MCA_CONFIG_PCS_XGMI)**

Reset: 0000_0000_0000_0121h.

Controls configuration of the associated machine check bank.

[_instSERDESAG3PCSXGMI_n7_aliasMSR](#); MSRC000_21E4[_instSERDESAP0PCSXGMI_n0_aliasMSR](#); MSRC000_21E4[_instSERDESBG0PCSXGMI_n4_aliasMSR](#); MSRC000_21E4[_instSERDESBG1PCSXGMI_n5_aliasMSR](#); MSRC000_21E4[_instSERDESBG2PCSXGMI_n6_aliasMSR](#); MSRC000_21E4[_instSERDESBP1PCSXGMI_n1_aliasMSR](#); MSRC000_21E4[_instSERDESBP2PCSXGMI_n2_aliasMSR](#); MSRC000_21E4[_instSERDESBP3PCSXGMI_n3_aliasMSR](#); MSRC000_21E4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,0. 1=Log deferred errors in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI and MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI in addition to MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI and MCA::PCS::XGMI::MCA_DEADDR_PCS_XGMI. 0=Only log deferred errors in MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI and MCA::PCS::XGMI::MCA_DEADDR_PCS_XGMI. This bit does not affect logging of deferred errors in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI, MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI, MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI, MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI, and MCA::PCS::XGMI::MCA_SYND_PCS_XGMI.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::PCS::XGMI::MCA_CONFIG_PCS_XGMI[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::PCS::XGMI::MCA_CONFIG_PCS_XGMI[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::PCS::XGMI::MCA_CONFIG_PCS_XGMI[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and MCA::PCS::XGMI::MCA_CONFIG_PCS_XGMI[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI and MCA::PCS::XGMI::MCA_DEADDR_PCS_XGMI are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.

0	McaX. Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::PCS::XGMI::MCA_MISC0_PCS_XGMI[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI[TCC] is present.
---	---

MSRC000_21E5 [PCS_XGMI IP Identification] (MCA::PCS::XGMI::MCA_IPID_PCS_XGMI)

Reset: 0000_0050_0000_0000h.

The MCA::PCS::XGMI::MCA_IPID_PCS_XGMI register is used by software to determine what IP type and revision is associated with the MCA bank.

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E5

_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E5

_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E5

_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E5

_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E5

_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E5

_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E5

_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E5

Bits	Description
63:48	McaType. Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi. Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID. Read-only. Reset: 050h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId. Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instSERDESAG3PCSXGMI_n7_aliasMSR: 1210_9200h
	Init: _instSERDESAP0PCSXGMI_n0_aliasMSR: 11A0_9200h
	Init: _instSERDESBG0PCSXGMI_n4_aliasMSR: 11E0_9200h
	Init: _instSERDESBG1PCSXGMI_n5_aliasMSR: 11F0_9200h
	Init: _instSERDESBG2PCSXGMI_n6_aliasMSR: 1200_9200h
	Init: _instSERDESBP1PCSXGMI_n1_aliasMSR: 11B0_9200h
	Init: _instSERDESBP2PCSXGMI_n2_aliasMSR: 11C0_9200h
	Init: _instSERDESBP3PCSXGMI_n3_aliasMSR: 11D0_9200h

MSRC000_21E6 [PCS_XGMI Machine Check Syndrome] (MCA::PCS::XGMI::MCA_SYND_PCS_XGMI)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI [Thread 0](#)_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E6_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E6_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E6_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E6_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E6_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E6_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E6_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E6

Bits	Description
63:32	Syndrone. Read-write, Volatile. Reset: Cold, 0000_0000h. Contains the syndrome, if any, associated with the error logged in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::PCS::XGMI::MCA_SYND_PCS_XGMI[Length]. The Syndrome field is only valid when MCA::PCS::XGMI::MCA_SYND_PCS_XGMI[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI. For example, a syndrome length of 9 means that MCA::PCS::XGMI::MCA_SYND_PCS_XGMI[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 120 [MCA_SYND_PCS_XGMI].

Table 120: MCA_SYND_PCS_XGMI

Error Type	Bits	Description
DataLossErr	[17:0]	Reserved
TrainingErr	[17:0]	Reserved
FlowCtrlAckErr	[17:0]	Reserved
RxFifoUnderflowErr	[17:0]	Reserved
RxFifoOverflowErr	[17:0]	Reserved
CRCErr	[17:0]	Reserved
BERExceededErr	[17:0]	Reserved
TxMetaDataErr_TxVcidDataErr	[17:0]	Reserved
ReplayBufParityErr	[17:0]	Reserved
DataParityErr	[17:0]	Reserved
ReplayFifoOverflowErr	[17:0]	Reserved
ReplayFifoUnderflowErr	[17:0]	Reserved
ElasticFifoOverflowErr	[17:0]	Reserved
DeskewErr	[17:0]	Reserved
FlowCtrlCRCErr	[17:0]	Reserved
DataStartupLimitErr	[17:0]	Reserved
FCInitTimeoutErr	[17:0]	Reserved
RecoveryTimeoutErr	[17:0]	Reserved
ReadySerialTimeoutErr	[17:0]	Reserved
ReadySerialAttemptErr	[17:0]	Reserved
RecoveryAttemptErr	[17:0]	Reserved
RecoveryRelockAttemptErr	[17:0]	Reserved

ReplayAttemptErr	[17:0]	Reserved
SyncHdrErr	[17:0]	Reserved
TxReplayTimeoutErr	[17:0]	Reserved
RxReplayTimeoutErr	[17:0]	Reserved
LinkSubTxTimeoutErr	[17:0]	Reserved
LinkSubRxTimeoutErr	[17:0]	Reserved
RxCMDPktErr	[17:0]	Reserved
Reserved_29	[17:0]	Reserved
Reserved_30	[17:0]	Reserved
Reserved_31	[17:0]	Reserved

**MSRC000_21E8 [PCS_XGMI Machine Check Deferred Error Status]
(MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI)**

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E8

_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E8

_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E8

_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E8

_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E8

_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E8

_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E8

_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E8

Bits	Description
------	-------------

63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::PCS::XGMI::MCA_DEADDR_PCS_XGMI contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI. If MCA::PCS::XGMI::MCA_SYND_PCS_XGMI[ErrorPriority] is the same as the priority of the error in MCA::PCS::XGMI::MCA_STATUS_PCS_XGMI, then the information in MCA::PCS::XGMI::MCA_SYND_PCS_XGMI is associated with the error in MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[ErrorAddr]. A value of 0 indicates that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[55:0] contains a valid byte address. A value of 6 indicates that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[55:6] contains a valid cache line address and that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[55:12] contain a valid 4KB memory page and that MCA::PCS::XGMI::MCA_ADDR_PCS_XGMI[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21E9 [PCS_XGMI Deferred Error Address] (MCA::PCS::XGMI::MCA_DEADDR_PCS_XGMI)

Read-only. Reset: Cold,0000_0000_0000_0000h.

The MCA::PCS::XGMI::MCA_DEADDR_PCS_XGMI register stores the address associated with the error in MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI. The register is only meaningful if MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI[Val]=1 and MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI[AddrV]=1. The lowest valid bit of the address is defined by MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI[AddrLsb].

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC000_21E9

_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC000_21E9

_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC000_21E9

_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC000_21E9

_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC000_21E9

_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC000_21E9

_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC000_21E9

_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC000_21E9

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::PCS::XGMI::MCA_DESTAT_PCS_XGMI.

**MSRC001_041E [PCS_XGMI Machine Check Control Mask]
(MCA::PCS::XGMI::MCA_CTL_MASK_PCS_XGMI)**

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instSERDESAG3PCSXGMI_n7_aliasMSR; MSRC001_041E

_instSERDESAP0PCSXGMI_n0_aliasMSR; MSRC001_041E

_instSERDESBG0PCSXGMI_n4_aliasMSR; MSRC001_041E

_instSERDESBG1PCSXGMI_n5_aliasMSR; MSRC001_041E

_instSERDESBG2PCSXGMI_n6_aliasMSR; MSRC001_041E

_instSERDESBP1PCSXGMI_n1_aliasMSR; MSRC001_041E

_instSERDESBP2PCSXGMI_n2_aliasMSR; MSRC001_041E

_instSERDESBP3PCSXGMI_n3_aliasMSR; MSRC001_041E

Bits	Description
------	-------------

63:29 Reserved.

28 **RxCMDPktErr**. Read-write. Reset: 0. RxCMDPktErr for xGMI327 **LinkSubRxTimeoutErr**. Read-write. Reset: 0. LinkSubRxTimeoutErr for xGMI326 **LinkSubTxTimeoutErr**. Read-write. Reset: 0. LinkSubTxTimeoutErr for xGMI325 **RxReplayTimeoutErr**. Read-write. Reset: 0. RxReplayTimeoutErr for xGMI324 **TxReplayTimeoutErr**. Read-write. Reset: 0. TxReplayTimeoutErr for xGMI323 **SyncHdrErr**. Read-write. Reset: 0. SyncHdrErr for xGMI322 **ReplayAttemptErr**. Read-write. Reset: 0. ReplayAttemptErr for xGMI321 **RecoveryRelockAttemptErr**. Read-write. Reset: 0. RecoveryRelockAttemptErr20 **RecoveryAttemptErr**. Read-write. Reset: 0. RecoveryAttemptErr19 **ReadySerialAttemptErr**. Read-write. Reset: 0. ReadySerialAttemptErr18 **ReadySerialTimeoutErr**. Read-write. Reset: 0. ReadySerialTimeoutErr17 **RecoveryTimeoutErr**. Read-write. Reset: 0. RecoveryTimeoutErr16 **FCInitTimeoutErr**. Read-write. Reset: 0. FCInitTimeoutErr15 **DataStartupLimitErr**. Read-write. Reset: 0. DataStartupLimitErr14 **FlowCtrlCRCErr**. Read-write. Reset: 0. FlowCtrlCRCErr for xGMI313 **DeskewErr**. Read-write. Reset: 0. DeskewErr12 **ElasticFifoOverflowErr**. Read-write. Reset: 0. ElasticFifoOverflowErr11 **ReplayFifoUnderflowErr**. Read-write. Reset: 0. ReplayFifoUnderflowErr10 **ReplayFifoOverflowErr**. Read-write. Reset: 0. ReplayFifoOverflowErr9 **DataParityErr**. Read-write. Reset: 0. DataParityErr8 **ReplayBufParityErr**. Read-write. Reset: 0. ReplayBufParityErr7 **TxMetaDataErr_TxVcidDataErr**. Read-write. Reset: 0. TxVcidDataErr for xGMI3.6 **BERExceededErr**. Read-write. Reset: 0. BERExceededErr5 **CRCErr**. Read-write. Reset: 0. CRCErr4 **RxFifoOverflowErr**. Read-write. Reset: 0. RxFifoOverflowErr for xGMI33 **RxFifoUnderflowErr**. Read-write. Reset: 0. RxFifoUnderflowErr for xGMI32 **FlowCtrlAckErr**. Read-write. Reset: 0. FlowCtrlAckErr for xGMI3.1 **TrainingErr**. Read-write. Reset: 0. TrainingErr.0 **DataLossErr**. Read-write. Reset: 0. DataLossErr

3.2.5.22 NBIF

MSR0000_046C...MSRC000_21C0 [NBIF Machine Check Control] (MCA::NBIF::MCA_CTL_NBIF)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::NBIF::MCA_CTL_NBIF register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC000_21B0_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSRLEGACY; MSR0000_046C_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC000_21B0_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSRLEGACY; MSR0000_046C_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC000_21B0_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSRLEGACY; MSR0000_046C_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC000_21B0_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSRLEGACY; MSR0000_046C_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC000_21C0_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSRLEGACY; MSR0000_0470_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC000_21C0_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSRLEGACY; MSR0000_0470

Bits	Description
63:4	Reserved.
3	SDP_PARITY_ERR. Read-write. Reset: 0. SDP parity error was detected.
2	NTB_ERR_EVENT. Read-write. Reset: 0. NTB ERROR EVENT was detected.
1	SRAM_ECC_ERR. Read-write. Reset: 0. SRAM ECC error in the NBIF was detected.
0	TIMEOUT_ERR. Read-write. Reset: 0. A GMI block DMA transaction timeout was detected.

MSR0000_046D...MSRC000_21C1 [NBIF Machine Check Status] (MCA::NBIF::MCA_STATUS_NBIF)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC000_21B1

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSRLEGACY; MSR0000_046D

_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC000_21B1

_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSRLEGACY; MSR0000_046D

_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC000_21B1

_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSRLEGACY; MSR0000_046D

_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC000_21B1

_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSRLEGACY; MSR0000_046D

_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC000_21C1

_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSRLEGACY; MSR0000_0471

_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC000_21C1

_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSRLEGACY; MSR0000_0471

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::NBIF::MCA_CTL_NBIF. This bit is a copy of bit in MCA::NBIF::MCA_CTL_NBIF for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::NBIF::MCA_MISC0_NBIF. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::NBIF::MCA_ADDR_NBIF contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::NBIF::MCA_STATUS_NBIF[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::NBIF::MCA_SYND_NBIF. If MCA::NBIF::MCA_SYND_NBIF[ErrorPriority] is the same as the priority of the error in MCA::NBIF::MCA_STATUS_NBIF, then the information in MCA::NBIF::MCA_SYND_NBIF is associated with the error in MCA::NBIF::MCA_STATUS_NBIF. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::NBIF::MCA_STATUS_NBIF[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::NBIF::MCA_ADDR_NBIF[ErrorAddr]. A value of 0 indicates that MCA::NBIF::MCA_ADDR_NBIF[55:0] contains a valid byte address. A value of 6 indicates that MCA::NBIF::MCA_ADDR_NBIF[55:6] contains a valid cache line address and that MCA::NBIF::MCA_ADDR_NBIF[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::NBIF::MCA_ADDR_NBIF[55:12] contain a valid 4KB memory page and that MCA::NBIF::MCA_ADDR_NBIF[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::NBIF::MCA_CTL_NBIF enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 121: MCA_STATUS_NBIF

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
------------	------------------	----	-----	-----	----------	--------	-------

TIMEOUT_ERR	0x0	1	1	1	0	0	0
SRAM_ECC_ERR	0x1	0/1	0/1	0/1	0/1	0	0
NTB_ERR_EVENT	0x2	1	1	1	0	0	0
SDP_PARITY_ERR	0x3	0/1	0/1	0/1	0/1	0	0

MSR0000_046E...MSRC000_21C2 [NBIF Machine Check Address] (MCA::NBIF::MCA_ADDR_NBIF)

Reset: Cold,0000_0000_0000_0000h.

MCA::NBIF::MCA_ADDR_NBIF stores an address and other information associated with the error in MCA::NBIF::MCA_STATUS_NBIF. The register is only meaningful if MCA::NBIF::MCA_STATUS_NBIF[Val]=1 and MCA::NBIF::MCA_STATUS_NBIF[AddrV]=1.

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC000_21B2_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSRLEGACY; MSR0000_046E_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC000_21B2_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSRLEGACY; MSR0000_046E_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC000_21B2_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSRLEGACY; MSR0000_046E_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC000_21B2_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSRLEGACY; MSR0000_046E_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC000_21C2_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSRLEGACY; MSR0000_0472_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC000_21C2_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSRLEGACY; MSR0000_0472

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::NBIF::MCA_STATUS_NBIF. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 122: MCA_ADDR_NBIF

Error Type	Bits	Description
TIMEOUT_ERR	[56:0]	Reserved
SRAM_ECC_ERR	[56:0]	Reserved
NTB_ERR_EVENT	[56:0]	Reserved
SDP_PARITY_ERR	[56:0]	Reserved

MSR0000_046F...MSRC000_21C3 [NBIF Machine Check Miscellaneous 0] (MCA::NBIF::MCA_MISC0_NBIF)

Log miscellaneous information associated with errors.

[_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR](#); MSRC000_21B3[_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSRLEGACY](#); MSR0000_046F[_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR](#); MSRC000_21B3[_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSRLEGACY](#); MSR0000_046F[_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR](#); MSRC000_21B3[_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSRLEGACY](#); MSR0000_046F[_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR](#); MSRC000_21B3[_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSRLEGACY](#); MSR0000_046F[_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR](#); MSRC000_21C3[_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSRLEGACY](#); MSR0000_0473[_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR](#); MSRC000_21C3[_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSRLEGACY](#); MSR0000_0473

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIF::MCA_MISC0_NBIF[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIF::MCA_MISC0_NBIF[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIF::MCA_MISC0_NBIF[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIF::MCA_MISC0_NBIF[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIF::MCA_MISC0_NBIF[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::NBIF::MCA_MISC0_NBIF[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.

23:0	Reserved.
------	-----------

MSRC000_21[B...C]4 [NBIF Machine Check Configuration] (MCA::NBIF::MCA_CONFIG_NBIF)	
Reset: 0000_0000_0000_0125h.	
Controls configuration of the associated machine check bank.	
_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC000_21B4	
_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC000_21B4	
_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC000_21B4	
_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC000_21B4	
_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC000_21C4	
_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC000_21C4	
Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::NBIF::MCA_STATUS_NBIF and MCA::NBIF::MCA_ADDR_NBIF in addition to MCA::NBIF::MCA_DESTAT_NBIF and MCA::NBIF::MCA_DEADDR_NBIF. 0=Only log deferred errors in MCA::NBIF::MCA_DESTAT_NBIF and MCA::NBIF::MCA_DEADDR_NBIF. This bit does not affect logging of deferred errors in MCA::NBIF::MCA_SYND_NBIF, MCA::NBIF::MCA_MISC0_NBIF.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::NBIF::MCA_STATUS_NBIF, MCA::NBIF::MCA_ADDR_NBIF, MCA::NBIF::MCA_MISC0_NBIF, and MCA::NBIF::MCA_SYND_NBIF.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::NBIF::MCA_CONFIG_NBIF[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::NBIF::MCA_CONFIG_NBIF[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::NBIF::MCA_CONFIG_NBIF[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::NBIF::MCA_CONFIG_NBIF[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::NBIF::MCA_DESTAT_NBIF and MCA::NBIF::MCA_DEADDR_NBIF are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::NBIF::MCA_MISC0_NBIF[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::NBIF::MCA_STATUS_NBIF[TCC] is present.

MSRC000_21[B...C]5 [NBIF IP Identification] (MCA::NBIF::MCA_IPID_NBIF)

Reset: 0000_006C_0000_0000h.

The MCA::NBIF::MCA_IPID_NBIF register is used by software to determine what IP type and revision is associated with the MCA bank.

[_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR](#); MSRC000_21B5[_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR](#); MSRC000_21B5[_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR](#); MSRC000_21B5[_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR](#); MSRC000_21B5[_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR](#); MSRC000_21C5[_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR](#); MSRC000_21C5

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 06Ch. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR : 1013_C000h
	Init: _ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR : 1033_C000h
	Init: _ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR : 1023_C000h
	Init: _ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR : 1043_C000h
	Init: _ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR : 1053_C000h
	Init: _ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR : 1073_C000h

MSRC000_21[B...C]6 [NBIF Machine Check Syndrome] (MCA::NBIF::MCA_SYND_NBIF)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::NBIF::MCA_STATUS_NBIF [Thread 0](#)

[_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR](#); MSRC000_21B6[_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR](#); MSRC000_21B6[_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR](#); MSRC000_21B6[_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR](#); MSRC000_21B6[_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR](#); MSRC000_21C6[_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR](#); MSRC000_21C6

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::NBIF::MCA_STATUS_NBIF. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::NBIF::MCA_SYND_NBIF[Length]. The Syndrome field is only valid when MCA::NBIF::MCA_SYND_NBIF[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::NBIF::MCA_SYND_NBIF. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::NBIF::MCA_SYND_NBIF[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::NBIF::MCA_SYND_NBIF. For example, a syndrome length of 9 means that MCA::NBIF::MCA_SYND_NBIF[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 123 [MCA_SYND_NBIF].

Table 123: MCA_SYND_NBIF

Error Type	Bits	Description
TIMEOUT_ERR	[17:0]	[10:0] means SDP transaction Req_unitID [13:11] means leaf ID/number of RAS leaf (which detected this

		error) [16:14] means RAS central ID (previous RAS leaf is under the RAS Central) [17] when acting as a SDP completer and a parity error is detected in the completer's responses, this bit will be asserted.
SRAM_ECC_ERR	[17:0]	[10:0] means SDP transaction Req_unitID [13:11] means leaf ID/number of RAS leaf (which detected this error) [16:14] means RAS central ID (previous RAS leaf is under the RAS Central) [17] when acting as a SDP completer and a parity error is detected in the completer's responses, this bit will be asserted.
NTB_ERR_EVENT	[14:0]	[10:0] means SDP transaction Req_unitID [14:11] means SDP transaction's ReqUser bits
SDP_PARITY_ERR	[17:0]	[10:0] means SDP transaction Req_unitID [13:11] means leaf ID/number of RAS leaf (which detected this error) [16:14] means RAS central ID (previous RAS leaf is under the RAS Central) [17] when acting as a SDP completer and a parity error is detected in the completer's responses, this bit will be asserted.

MSRC000_21[B...C]8 [NBIF Machine Check Deferred Error Status] (MCA::NBIF::MCA_DESTAT_NBIF)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC000_21B8_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC000_21B8_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC000_21B8_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC000_21B8_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC000_21C8_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC000_21C8

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::NBIF::MCA_DEADDR_NBIF contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::NBIF::MCA_SYND_NBIF. If MCA::NBIF::MCA_SYND_NBIF[ErrorPriority] is the same as the priority of the error in MCA::NBIF::MCA_STATUS_NBIF, then the information in MCA::NBIF::MCA_SYND_NBIF is associated with the error in MCA::NBIF::MCA_DESTAT_NBIF.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::NBIF::MCA_ADDR_NBIF[ErrorAddr]. A value of 0 indicates that MCA::NBIF::MCA_ADDR_NBIF[55:0] contains a valid byte address. A value of 6 indicates that MCA::NBIF::MCA_ADDR_NBIF[55:6] contains a valid cache line address and that MCA::NBIF::MCA_ADDR_NBIF[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::NBIF::MCA_ADDR_NBIF[55:12] contain a valid 4KB memory page and that MCA::NBIF::MCA_ADDR_NBIF[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21[B...C]9 [NBIF Deferred Error Address] (MCA::NBIF::MCA_DEADDR_NBIF)

Reset: Cold,0000_0000_0000_0000h.

The MCA::NBIF::MCA_DEADDR_NBIF register stores the address associated with the error in MCA::NBIF::MCA_DESTAT_NBIF. The register is only meaningful if MCA::NBIF::MCA_DESTAT_NBIF[Val]=1 and MCA::NBIF::MCA_DESTAT_NBIF[AddrV]=1. The lowest valid bit of the address is defined by MCA::NBIF::MCA_DESTAT_NBIF[AddrLsb].

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC000_21B9

_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC000_21B9

_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC000_21B9

_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC000_21B9

_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC000_21C9

_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC000_21C9

Bits	Description
------	-------------

63:56	Reserved.
-------	-----------

55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::NBIF::MCA_DESTAT_NBIF. The lowest-order valid bit of the address is specified in MCA::NBIF::MCA_DESTAT_NBIF[AddrLsb].
------	--

MSRC001_041[B...C] [NBIF Machine Check Control Mask] (MCA::NBIF::MCA_CTL_MASK_NBIF)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ch0_instNBIF0NBIF0_n0_nbio0_aliasMSR; MSRC001_041B

_ch0_instNBIF0NBIF0_n4_nbio1_aliasMSR; MSRC001_041B

_ch0_instNBIF1NBIF1_n2_nbio0_aliasMSR; MSRC001_041B

_ch0_instNBIF1NBIF1_n5_nbio1_aliasMSR; MSRC001_041B

_ch0_instNBIF2NBIF2_n1_nbio0_aliasMSR; MSRC001_041C

_ch0_instNBIF2NBIF2_n3_nbio1_aliasMSR; MSRC001_041C

Bits	Description
------	-------------

63:4	Reserved.
------	-----------

3	SDP_PARITY_ERR. Read-write. Reset: 0. SDP parity error was detected.
---	---

2	NTB_ERR_EVENT. Read-write. Reset: 0. NTB ERROR EVENT was detected.
---	---

1	SRAM_ECC_ERR. Read-write. Reset: 0. SRAM ECC error in the NBIF was detected.
---	---

0	TIMEOUT_ERR. Read-write. Reset: 0. A GMI block DMA transaction timeout was detected.
---	---

3.2.5.23 SHUB

MSR0000_0464...MSRC000_2190 [SHUB Machine Check Control] (MCA::SHUB::MCA_CTL_SHUB)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::SHUB::MCA_CTL_SHUB register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2190

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSRLEGACY; MSR0000_0464

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2190

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSRLEGACY; MSR0000_0464

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2190

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSRLEGACY; MSR0000_0464

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2190

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSRLEGACY; MSR0000_0464

Bits	Description
63:4	Reserved.
3	SDP_PARITY_ERR. Read-write. Reset: 0. SDP parity error was detected.
2	NTB_ERR_EVENT. Read-write. Reset: 0. NTB ERROR EVENT was detected.
1	SRAM_ECC_ERR. Read-write. Reset: 0. SRAM ECC error in the NBIF was detected.
0	TIMEOUT_ERR. Read-write. Reset: 0. A GMI block DMA transaction timeout was detected.

MSR0000_0465...MSRC000_2191 [SHUB Machine Check Status] (MCA::SHUB::MCA_STATUS_SHUB)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2191

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSRLEGACY; MSR0000_0465

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2191

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSRLEGACY; MSR0000_0465

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2191

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSRLEGACY; MSR0000_0465

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2191

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSRLEGACY; MSR0000_0465

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::SHUB::MCA_CTL_SHUB. This bit is a copy of bit in MCA::SHUB::MCA_CTL_SHUB for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::SHUB::MCA_MISC0_SHUB. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::SHUB::MCA_ADDR_SHUB contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::SHUB::MCA_STATUS_SHUB[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::SHUB::MCA_SYND_SHUB. If MCA::SHUB::MCA_SYND_SHUB[ErrorPriority] is the same as the priority of the error in MCA::SHUB::MCA_STATUS_SHUB, then the information in MCA::SHUB::MCA_SYND_SHUB is associated with the error in MCA::SHUB::MCA_STATUS_SHUB. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::SHUB::MCA_STATUS_SHUB[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC . Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC . Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred . Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison . Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub . Reset: Cold,0. 1=The error was the result of a scrub operation.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId . Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb . Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::SHUB::MCA_ADDR_SHUB[ErrorAddr]. A value of 0 indicates that MCA::SHUB::MCA_ADDR_SHUB[55:0] contains a valid byte address. A value of 6 indicates that MCA::SHUB::MCA_ADDR_SHUB[55:6] contains a valid cache line address and that MCA::SHUB::MCA_ADDR_SHUB[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::SHUB::MCA_ADDR_SHUB[55:12] contain a valid 4KB memory page and that MCA::SHUB::MCA_ADDR_SHUB[11:0] should be ignored by error handling software.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt . Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::SHUB::MCA_CTL_SHUB enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode . Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 124: MCA_STATUS_SHUB

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
TIMEOUT_ERR	0x0	1	1	1	0	0	0
SRAM_ECC_ERR	0x1	0/1	0/1	0/1	0/1	0	0

NTB_ERR_EVENT	0x2	1	1	1	0	0	0
SDP_PARITY_ERR	0x3	0/1	0/1	0/1	0/1	0	0

MSR0000_0466...MSRC000_2192 [SHUB Machine Check Address] (MCA::SHUB::MCA_ADDR_SHUB)

Reset: Cold,0000_0000_0000_0000h.

MCA::SHUB::MCA_ADDR_SHUB stores an address and other information associated with the error in MCA::SHUB::MCA_STATUS_SHUB. The register is only meaningful if MCA::SHUB::MCA_STATUS_SHUB[Val]=1 and MCA::SHUB::MCA_STATUS_SHUB[AddrV]=1.

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2192_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSRLEGACY; MSR0000_0466_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2192_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSRLEGACY; MSR0000_0466_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2192_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSRLEGACY; MSR0000_0466_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2192_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSRLEGACY; MSR0000_0466

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::SHUB::MCA_STATUS_SHUB. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 125: MCA_ADDR_SHUB

Error Type	Bits	Description
TIMEOUT_ERR	[56:0]	Reserved
SRAM_ECC_ERR	[56:0]	Reserved
NTB_ERR_EVENT	[56:0]	Reserved
SDP_PARITY_ERR	[56:0]	Reserved

**MSR0000_0467...MSRC000_2193 [SHUB Machine Check Miscellaneous 0]
(MCA::SHUB::MCA_MISC0_SHUB)**

Log miscellaneous information associated with errors.

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2193

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSRLEGACY; MSR0000_0467

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2193

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSRLEGACY; MSR0000_0467

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2193

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSRLEGACY; MSR0000_0467

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2193

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSRLEGACY; MSR0000_0467

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SHUB::MCA_MISC0_SHUB[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SHUB::MCA_MISC0_SHUB[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SHUB::MCA_MISC0_SHUB[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SHUB::MCA_MISC0_SHUB[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SHUB::MCA_MISC0_SHUB[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SHUB::MCA_MISC0_SHUB[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_2194 [SHUB Machine Check Configuration] (MCA::SHUB::MCA_CONFIG_SHUB)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2194_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2194_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2194_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2194

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::SHUB::MCA_STATUS_SHUB and MCA::SHUB::MCA_ADDR_SHUB in addition to MCA::SHUB::MCA_DESTAT_SHUB and MCA::SHUB::MCA_DEADDR_SHUB. 0=Only log deferred errors in MCA::SHUB::MCA_DESTAT_SHUB and MCA::SHUB::MCA_DEADDR_SHUB. This bit does not affect logging of deferred errors in MCA::SHUB::MCA_SYND_SHUB, MCA::SHUB::MCA_MISC0_SHUB.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::SHUB::MCA_STATUS_SHUB, MCA::SHUB::MCA_ADDR_SHUB, MCA::SHUB::MCA_MISC0_SHUB, and MCA::SHUB::MCA_SYND_SHUB.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::SHUB::MCA_CONFIG_SHUB[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::SHUB::MCA_CONFIG_SHUB[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::SHUB::MCA_CONFIG_SHUB[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::SHUB::MCA_CONFIG_SHUB[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::SHUB::MCA_DESTAT_SHUB and MCA::SHUB::MCA_DEADDR_SHUB are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::SHUB::MCA_MISC0_SHUB[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::SHUB::MCA_STATUS_SHUB[TCC] is present.

MSRC000_2195 [SHUB IP Identification] (MCA::SHUB::MCA_IPID_SHUB)

Reset: 0000_0080_0000_0000h.

The MCA::SHUB::MCA_IPID_SHUB register is used by software to determine what IP type and revision is associated with the MCA bank.

[_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR](#); MSRC000_2195[_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR](#); MSRC000_2195[_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR](#); MSRC000_2195[_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR](#); MSRC000_2195

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 080h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR : 0141_C000h
	Init: _ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR : 0161_C000h
	Init: _ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR : 0151_C000h
	Init: _ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR : 0171_C000h

MSRC000_2196 [SHUB Machine Check Syndrome] (MCA::SHUB::MCA_SYND_SHUB)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::SHUB::MCA_STATUS_SHUB [Thread 0](#)

[_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR](#); MSRC000_2196[_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR](#); MSRC000_2196[_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR](#); MSRC000_2196[_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR](#); MSRC000_2196

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::SHUB::MCA_STATUS_SHUB. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::SHUB::MCA_SYND_SHUB[Length]. The Syndrome field is only valid when MCA::SHUB::MCA_SYND_SHUB[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::SHUB::MCA_SYND_SHUB. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::SHUB::MCA_SYND_SHUB[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::SHUB::MCA_SYND_SHUB. For example, a syndrome length of 9 means that MCA::SHUB::MCA_SYND_SHUB[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 126 [MCA_SYND_SHUB].

Table 126: MCA_SYND_SHUB

Error Type	Bits	Description
TIMEOUT_ERR	[17:0]	[10:0] => SDP transaction Req_unitID [13:11] => leaf ID/number of RAS leaf (which detected this error) [16:14] => RAS central ID (previous RAS leaf is under the RAS Central) [17] => when acting as a SDP completer and a parity error is detected in the completer's responses, this bit will be asserted.

SRAM_ECC_ERR	[17:0]	[10:0] => SDP transaction Req_unitID [13:11] => leaf ID/number of RAS leaf (which detected this error) [16:14] => RAS central ID (previous RAS leaf is under the RAS Central) [17] => when acting as a SDP completer and a parity error is detected in the completer's responses, this bit will be asserted.
NTB_ERR_EVENT	[14:11]	[10:0] => SDP transaction Req_unitID [14:11] => SDP transaction's ReqUser bits
SDP_PARITY_ERR	[17:0]	[10:0] => SDP transaction Req_unitID [13:11] => leaf ID/number of RAS leaf (which detected this error) [16:14] => RAS central ID (previous RAS leaf is under the RAS Central) [17] => when acting as a SDP completer and a parity error is detected in the completer's responses, this bit will be asserted.

MSRC000_2198 [SHUB Machine Check Deferred Error Status] (MCA::SHUB::MCA_DESTAT_SHUB)

Reset: Cold,0000_0000_0000_0000h.	
Holds status information for the first deferred error seen in this bank.	
_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2198	
_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2198	
_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2198	
_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2198	
Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::SHUB::MCA_DEADDR_SHUB contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::SHUB::MCA_SYND_SHUB. If MCA::SHUB::MCA_SYND_SHUB[ErrorPriority] is the same as the priority of the error in MCA::SHUB::MCA_STATUS_SHUB, then the information in MCA::SHUB::MCA_SYND_SHUB is associated with the error in MCA::SHUB::MCA_DESTAT_SHUB.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::SHUB::MCA_ADDR_SHUB[ErrorAddr]. A value of 0 indicates that MCA::SHUB::MCA_ADDR_SHUB[55:0] contains a valid byte address. A value of 6 indicates that MCA::SHUB::MCA_ADDR_SHUB[55:6] contains a valid cache line address and that MCA::SHUB::MCA_ADDR_SHUB[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::SHUB::MCA_ADDR_SHUB[55:12] contain a valid 4KB memory page and that MCA::SHUB::MCA_ADDR_SHUB[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_2199 [SHUB Deferred Error Address] (MCA::SHUB::MCA_DEADDR_SHUB)

Reset: Cold,0000_0000_0000_0000h.

The MCA::SHUB::MCA_DEADDR_SHUB register stores the address associated with the error in MCA::SHUB::MCA_DESTAT_SHUB. The register is only meaningful if MCA::SHUB::MCA_DESTAT_SHUB[Val]=1 and MCA::SHUB::MCA_DESTAT_SHUB[AddrV]=1. The lowest valid bit of the address is defined by MCA::SHUB::MCA_DESTAT_SHUB[AddrLsb].

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC000_2199

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC000_2199

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC000_2199

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC000_2199

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::SHUB::MCA_DESTAT_SHUB. The lowest-order valid bit of the address is specified in MCA::SHUB::MCA_DESTAT_SHUB[AddrLsb].

MSRC001_0419 [SHUB Machine Check Control Mask] (MCA::SHUB::MCA_CTL_MASK_SHUB)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_ch1_instNBIF0SHUB0_n0_nbio0_aliasMSR; MSRC001_0419

_ch1_instNBIF0SHUB0_n2_nbio1_aliasMSR; MSRC001_0419

_ch1_instNBIF1SHUB1_n1_nbio0_aliasMSR; MSRC001_0419

_ch1_instNBIF1SHUB1_n3_nbio1_aliasMSR; MSRC001_0419

Bits	Description
63:4	Reserved.
3	SDP_PARITY_ERR. Read-write. Reset: 0. SDP parity error was detected.
2	NTB_ERR_EVENT. Read-write. Reset: 0. NTB ERROR EVENT was detected.
1	SRAM_ECC_ERR. Read-write. Reset: 0. SRAM ECC error in the NBIF was detected.
0	TIMEOUT_ERR. Read-write. Reset: 0. A GMI block DMA transaction timeout was detected.

3.2.5.24 USB**MSR0000_0470...MSRC000_21C0 [USB Machine Check Control] (MCA::USB::MCA_CTL_USB)**

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::USB::MCA_CTL_USB register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instUSB0USB_n0_aliasMSR; MSRC000_21C0

_instUSB0USB_n0_aliasMSRLEGACY; MSR0000_0470

_instUSB1USB_n1_aliasMSR; MSRC000_21C0

_instUSB1USB_n1_aliasMSRLEGACY; MSR0000_0470

Bits	Description
63:6	Reserved.
5	AXI_slv. Read-write. Reset: 0. axi slave response error
4	PHY_RAM1. Read-write. Reset: 0. parity error for phy ram1
3	PHY_RAM0. Read-write. Reset: 0. parity error for phy ram0
2	S0_RAM2. Read-write. Reset: 0. parity error or ecc error for s0 ram2
1	S0_RAM1. Read-write. Reset: 0. parity error or ecc error for s0 ram1
0	S0_RAM0. Read-write. Reset: 0. parity error or ecc error for s0 ram0

MSR0000_0471...MSRC000_21C1 [USB Machine Check Status] (MCA::USB::MCA_STATUS_USB)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instUSB0USB_n0_aliasMSR; MSRC000_21C1

_instUSB0USB_n0_aliasMSRLEGACY; MSR0000_0471

_instUSB1USB_n1_aliasMSR; MSRC000_21C1

_instUSB1USB_n1_aliasMSRLEGACY; MSR0000_0471

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::USB::MCA_CTL_USB. This bit is a copy of bit in MCA::USB::MCA_CTL_USB for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::USB::MCA_MISC0_USB. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::USB::MCA_ADDR_USB contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::USB::MCA_STATUS_USB[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::USB::MCA_SYND_USB. If MCA::USB::MCA_SYND_USB[ErrorPriority] is the same as the priority of the error in MCA::USB::MCA_STATUS_USB, then the information in MCA::USB::MCA_SYND_USB is associated with the error in MCA::USB::MCA_STATUS_USB. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::USB::MCA_STATUS_USB[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC . Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred . Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison . Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub . Reset: Cold,0. 1=The error was the result of a scrub operation.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId . Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb . Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::USB::MCA_ADDR_USB[ErrorAddr]. A value of 0 indicates that MCA::USB::MCA_ADDR_USB[54:0] contains a valid byte address. A value of 6 indicates that MCA::USB::MCA_ADDR_USB[54:6] contains a valid cache line address and that MCA::USB::MCA_ADDR_USB[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::USB::MCA_ADDR_USB[54:12] contain a valid 4KB memory page and that MCA::USB::MCA_ADDR_USB[11:0] should be ignored by error handling software.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt . Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::USB::MCA_CTL_USB enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode . Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 127: MCA_STATUS_USB

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
S0_RAM0	0x0	0/1	0/1	0/1	0/1	-	-
S0_RAM1	0x1	0/1	0/1	0/1	0/1	-	-
S0_RAM2	0x2	0/1	0/1	0/1	0/1	-	-
PHY_RAM0	0x3	0/1	0/1	0/1	0/1	-	-
PHY_RAM1	0x4	0/1	0/1	0/1	0/1	-	-
AXI_slv	0x5	0/1	0/1	0/1	0/1	-	-

MSR0000_0472...MSRC000_21C2 [USB Machine Check Address] (MCA::USB::MCA_ADDR_USB)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::USB::MCA_ADDR_USB stores an address and other information associated with the error in MCA::USB::MCA_STATUS_USB. The register is only meaningful if MCA::USB::MCA_STATUS_USB[Val]=1 and MCA::USB::MCA_STATUS_USB[AddrV]=1.

_instUSB0USB_n0_aliasMSR; MSRC000_21C2

_instUSB0USB_n0_aliasMSRLEGACY; MSR0000_0472

_instUSB1USB_n1_aliasMSR; MSRC000_21C2

_instUSB1USB_n1_aliasMSRLEGACY; MSR0000_0472

Bits	Description
63:62	Reserved.
61:56	LSB. Read-only. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::USB::MCA_ADDR_USB[ErrorAddr]. For example, a value of 0 indicates that MCA::USB::MCA_ADDR_USB[55:0] contains a valid byte address. A value of 6 indicates that MCA::USB::MCA_ADDR_USB[55:6] contains a valid cache line address and that MCA::USB::MCA_ADDR_USB[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::USB::MCA_ADDR_USB[55:12] contain a valid 4KB memory page and that MCA::USB::MCA_ADDR_USB[11:0] should be ignored by error handling software.
55:0	ErrorAddr. Read-only. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::USB::MCA_STATUS_USB.

Table 128: MCA_ADDR_USB

Error Type	Bits	Description
S0_RAM0	[55:0]	Reserved
S0_RAM1	[55:0]	Reserved
S0_RAM2	[55:0]	Reserved
PHY_RAM0	[55:0]	Reserved
PHY_RAM1	[55:0]	Reserved
AXI_slv	[55:0]	Reserved

MSR0000_0473...MSRC000_21C3 [USB Machine Check Miscellaneous 0] (MCA::USB::MCA_MISC0_USB)

Log miscellaneous information associated with errors.

_instUSB0USB_n0_aliasMSR; MSRC000_21C3

_instUSB0USB_n0_aliasMSRLEGACY; MSR0000_0473

_instUSB1USB_n1_aliasMSR; MSRC000_21C3

_instUSB1USB_n1_aliasMSRLEGACY; MSR0000_0473

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::USB::MCA_MISC0_USB[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::USB::MCA_MISC0_USB[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::USB::MCA_MISC0_USB[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::USB::MCA_MISC0_USB[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::USB::MCA_MISC0_USB[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::USB::MCA_MISC0_USB[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21C4 [USB Machine Check Configuration] (MCA::USB::MCA_CONFIG_USB)

Reset: 0000_0002_0000_0023h.

Controls configuration of the associated machine check bank.

_instUSB0USB_n0_aliasMSR; MSRC000_21C4_instUSB1USB_n1_aliasMSR; MSRC000_21C4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable . Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::USB::MCA_STATUS_USB, MCA::USB::MCA_ADDR_USB, MCA::USB::MCA_MISC0_USB, and MCA::USB::MCA_SYND_USB in addition to MCA::USB::MCA_TRANSADDR_USB and MCA::USB::MCA_TRANSADDR_USB. 0=Only log transparent errors in MCA::USB::MCA_TRANSADDR_USB and MCA::USB::MCA_TRANSADDR_USB.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::USB::MCA_CONFIG_USB[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::USB::MCA_CONFIG_USB[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. MCA_DESTAT and MCA_DEADDR are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::USB::MCA_CONFIG_USB[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::USB::MCA_MISC0_USB[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::USB::MCA_STATUS_USB[TCC] is present.

MSRC000_21C5 [USB IP Identification] (MCA::USB::MCA_IPID_USB)

Reset: 0000_00AA_0000_0000h.

The MCA::USB::MCA_IPID_USB register is used by software to determine what IP type and revision is associated with the MCA bank.

_instUSB0USB_n0_aliasMSR; MSRC000_21C5_instUSB1USB_n1_aliasMSR; MSRC000_21C5

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0AAh. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: <u>_instUSB0USB_n0_aliasMSR; 16DF_E000h</u>
	Init: <u>_instUSB1USB_n1_aliasMSR; 16FF_E000h</u>

MSRC000_21C6 [USB Machine Check Syndrome] (MCA::USB::MCA_SYND_USB)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::USB::MCA_STATUS_USB [Thread 0](#)

_instUSB0USB_n0_aliasMSR; MSRC000_21C6

_instUSB1USB_n1_aliasMSR; MSRC000_21C6

Bits	Description
63:33	Reserved.
32	Syndrone. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::USB::MCA_STATUS_USB. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::USB::MCA_SYND_USB[Length]. The Syndrome field is only valid when MCA::USB::MCA_SYND_USB[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::USB::MCA_SYND_USB. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::USB::MCA_SYND_USB[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::USB::MCA_SYND_USB. For example, a syndrome length of 9 means that MCA::USB::MCA_SYND_USB[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 129 [MCA_SYND_USB].

Table 129: MCA_SYND_USB

Error Type	Bits	Description
S0_RAM0	[17:0]	Reserved
S0_RAM1	[17:0]	Reserved
S0_RAM2	[17:0]	Reserved
PHY_RAM0	[17:0]	Reserved
PHY_RAM1	[17:0]	Reserved
AXI_slv	[17:0]	Reserved

MSRC001_041C [USB Machine Check Control Mask] (MCA::USB::MCA_CTL_MASK_USB)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instUSB0USB_n0_aliasMSR; MSRC001_041C

_instUSB1USB_n1_aliasMSR; MSRC001_041C

Bits	Description
63:6	Reserved.
5	AXI_slv. Read-write. Reset: 0. axi slave response error
4	PHY_RAM1. Read-write. Reset: 0. parity error for phy ram1
3	PHY_RAM0. Read-write. Reset: 0. parity error for phy ram0
2	S0_RAM2. Read-write. Reset: 0. parity error or ecc error for s0 ram2
1	S0_RAM1. Read-write. Reset: 0. parity error or ecc error for s0 ram1
0	S0_RAM0. Read-write. Reset: 0. parity error or ecc error for s0 ram0

MSRC001_051C [USB Machine Check Transparent Error Syndrome] (MCA::USB::MCA_TRANSYND_USB)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_instUSB0USB_n0_aliasMSR; MSRC001_051C

_instUSB1USB_n1_aliasMSR; MSRC001_051C

Bits	Description
63:33	Reserved.
32	Syndrome. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::USB::MCA_STATUS_USB. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::USB::MCA_SYND_USB[Length]. The Syndrome field is only valid when MCA::USB::MCA_SYND_USB[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::USB::MCA_TRANSYND_USB. All errors writing into MCA::USB::MCA_SYND_USB should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::USB::MCA_SYND_USB. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::USB::MCA_SYND_USB[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::USB::MCA_SYND_USB. For example, a syndrome length of 9 means that MCA::USB::MCA_SYND_USB[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

MSRC001_061C [USB Machine Check Transparent Error Address] (MCA::USB::MCA_TRANSADDR_USB)

Read-only, Volatile. Reset: Cold, 0000_0000_0000_0000h.

MCA::USB::MCA_TRANSADDR_USB stores the address associated with the error in MCA::USB::MCA_STATUS_USB. The register is only meaningful if MCA::USB::MCA_STATUS_USB[Val]=1 and MCA::USB::MCA_STATUS_USB[AddrV]=1. The lowest valid bit of the address is defined by MCA::USB::MCA_ADDR_USB[LSB].

_instUSB0USB_n0_aliasMSR; MSRC001_061C

_instUSB1USB_n1_aliasMSR; MSRC001_061C

Bits	Description
63:0	ErrorAddr. Read-only, Volatile. Reset: Cold, 0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::USB::MCA_STATUS_USB. The lowest-order valid bit of the address is specified in MCA::USB::MCA_ADDR_USB[LSB].

3.2.5.25 SATA

MSR0000_0468...MSRC000_21A0 [SATA Machine Check Control] (MCA::SATA::MCA_CTL_SATA)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::SATA::MCA_CTL_SATA register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instSATA0SATA_n0_aliasMSR; MSRC000_21A0_instSATA0SATA_n0_aliasMSRLEGACY; MSR0000_0468_instSATA1SATA_n1_aliasMSR; MSRC000_21A0_instSATA1SATA_n1_aliasMSRLEGACY; MSR0000_0468_instSATA2SATA_n2_aliasMSR; MSRC000_21A0_instSATA2SATA_n2_aliasMSRLEGACY; MSR0000_0468_instSATA3SATA_n3_aliasMSR; MSRC000_21A0_instSATA3SATA_n3_aliasMSRLEGACY; MSR0000_0468

Bits	Description
63:8	Reserved.
7	perr_p7 . Read-write. Reset: 0. parity error for port 7
6	perr_p6 . Read-write. Reset: 0. parity error for port 6
5	perr_p5 . Read-write. Reset: 0. parity error for port 5
4	perr_p4 . Read-write. Reset: 0. parity error for port 4
3	perr_p3 . Read-write. Reset: 0. parity error for port 3
2	perr_p2 . Read-write. Reset: 0. parity error for port 2
1	perr_p1 . Read-write. Reset: 0. parity error for port 1
0	perr_p0 . Read-write. Reset: 0. parity error for port 0

MSR0000_0469...MSRC000_21A1 [SATA Machine Check Status] (MCA::SATA::MCA_STATUS_SATA)

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instSATA0SATA_n0_aliasMSR; MSRC000_21A1_instSATA0SATA_n0_aliasMSRLEGACY; MSR0000_0469_instSATA1SATA_n1_aliasMSR; MSRC000_21A1_instSATA1SATA_n1_aliasMSRLEGACY; MSR0000_0469_instSATA2SATA_n2_aliasMSR; MSRC000_21A1_instSATA2SATA_n2_aliasMSRLEGACY; MSR0000_0469_instSATA3SATA_n3_aliasMSR; MSRC000_21A1_instSATA3SATA_n3_aliasMSRLEGACY; MSR0000_0469

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::SATA::MCA_CTL_SATA. This bit is a copy of bit in MCA::SATA::MCA_CTL_SATA for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::SATA::MCA_MISC0_SATA. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::SATA::MCA_ADDR_SATA contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::SATA::MCA_STATUS_SATA[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::SATA::MCA_SYND_SATA. If MCA::SATA::MCA_SYND_SATA[ErrorPriority] is the same as the priority of the error in MCA::SATA::MCA_STATUS_SATA, then the information in MCA::SATA::MCA_SYND_SATA is associated with the error in MCA::SATA::MCA_STATUS_SATA. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::SATA::MCA_STATUS_SATA[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits.

	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
46	CECC . Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC . Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred . Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison . Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub . Reset: Cold,0. 1=The error was the result of a scrub operation.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId . Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb . Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::SATA::MCA_ADDR_SATA[ErrorAddr]. A value of 0 indicates that MCA::SATA::MCA_ADDR_SATA[55:0] contains a valid byte address. A value of 6 indicates that MCA::SATA::MCA_ADDR_SATA[55:6] contains a valid cache line address and that MCA::SATA::MCA_ADDR_SATA[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::SATA::MCA_ADDR_SATA[55:12] contain a valid 4KB memory page and that MCA::SATA::MCA_ADDR_SATA[11:0] should be ignored by error handling software.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22 . Reset: Cold,0h. MCA_STATUS Register Reserved bits.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt . Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::SATA::MCA_CTL_SATA enables error reporting for the logged error.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode . Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field.
	AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 130: MCA_STATUS_SATA

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
perr_p0	0x0	1	1	1	0	0	0
perr_p1	0x1	1	1	1	0	0	0
perr_p2	0x2	1	1	1	0	0	0
perr_p3	0x3	1	1	1	0	0	0

perr_p4	0x4	1	1	1	0	0	0
perr_p5	0x5	1	1	1	0	0	0
perr_p6	0x6	1	1	1	0	0	0
perr_p7	0x7	1	1	1	0	0	0

MSR0000_046A...MSRC000_21A2 [SATA Machine Check Address] (MCA::SATA::MCA_ADDR_SATA)

Read-only. Reset: Cold,0000_0000_0000_0000h.

MCA::SATA::MCA_ADDR_SATA stores an address and other information associated with the error in MCA::SATA::MCA_STATUS_SATA. The register is only meaningful if MCA::SATA::MCA_STATUS_SATA[Val]=1 and MCA::SATA::MCA_STATUS_SATA[AddrV]=1.

_instSATA0SATA_n0_aliasMSR; MSRC000_21A2_instSATA0SATA_n0_aliasMSRLEGACY; MSR0000_046A_instSATA1SATA_n1_aliasMSR; MSRC000_21A2_instSATA1SATA_n1_aliasMSRLEGACY; MSR0000_046A_instSATA2SATA_n2_aliasMSR; MSRC000_21A2_instSATA2SATA_n2_aliasMSRLEGACY; MSR0000_046A_instSATA3SATA_n3_aliasMSR; MSRC000_21A2_instSATA3SATA_n3_aliasMSRLEGACY; MSR0000_046A

Bits	Description
63:0	ErrorAddr. Read-only. Reset: Cold,0000_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::SATA::MCA_STATUS_SATA.

Table 131: MCA_ADDR_SATA

Error Type	Bits	Description
perr_p0	[56:0]	Reserved
perr_p1	[56:0]	Reserved
perr_p2	[56:0]	Reserved
perr_p3	[56:0]	Reserved
perr_p4	[56:0]	Reserved
perr_p5	[56:0]	Reserved
perr_p6	[56:0]	Reserved
perr_p7	[56:0]	Reserved

MSR0000_046B...MSRC000_21A3 [SATA Machine Check Miscellaneous 0] (MCA::SATA::MCA_MISC0_SATA)

Log miscellaneous information associated with errors.

[_instSATA0SATA_n0_aliasMSR](#); [MSRC000_21A3](#)[_instSATA0SATA_n0_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instSATA1SATA_n1_aliasMSR](#); [MSRC000_21A3](#)[_instSATA1SATA_n1_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instSATA2SATA_n2_aliasMSR](#); [MSRC000_21A3](#)[_instSATA2SATA_n2_aliasMSRLEGACY](#); [MSR0000_046B](#)[_instSATA3SATA_n3_aliasMSR](#); [MSRC000_21A3](#)[_instSATA3SATA_n3_aliasMSRLEGACY](#); [MSR0000_046B](#)

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SATA::MCA_MISC0_SATA[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SATA::MCA_MISC0_SATA[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SATA::MCA_MISC0_SATA[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SATA::MCA_MISC0_SATA[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SATA::MCA_MISC0_SATA[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msr::HWCR[McStatusWrEn] !MCA::SATA::MCA_MISC0_SATA[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21A4 [SATA Machine Check Configuration] (MCA::SATA::MCA_CONFIG_SATA)

Reset: 0000_0000_0000_0125h.

Controls configuration of the associated machine check bank.

_instSATA0SATA_n0_aliasMSR; MSRC000_21A4_instSATA1SATA_n1_aliasMSR; MSRC000_21A4_instSATA2SATA_n2_aliasMSR; MSRC000_21A4_instSATA3SATA_n3_aliasMSR; MSRC000_21A4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:35	Reserved.
34	LogDeferredInMcaStat . Read-write. Reset: 0. Init: BIOS,1. 1=Log deferred errors in MCA::SATA::MCA_STATUS_SATA and MCA::SATA::MCA_ADDR_SATA in addition to MCA::SATA::MCA_DESTAT_SATA and MCA::SATA::MCA_DEADDR_SATA. 0=Only log deferred errors in MCA::SATA::MCA_DESTAT_SATA and MCA::SATA::MCA_DEADDR_SATA. This bit does not affect logging of deferred errors in MCA::SATA::MCA_SYND_SATA, MCA::SATA::MCA_MISC0_SATA.
33	TransparentErrorLoggingEnable . Read-write. Reset: 0. 1=When set, Log transparent errors in MCA::SATA::MCA_STATUS_SATA, MCA::SATA::MCA_ADDR_SATA, MCA::SATA::MCA_MISC0_SATA, and MCA::SATA::MCA_SYND_SATA.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::SATA::MCA_CONFIG_SATA[McaLsbInStatusSupported] indicates that AddrLbc is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::SATA::MCA_CONFIG_SATA[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::SATA::MCA_CONFIG_SATA[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 1. 1=Deferred errors are supported in this MCA bank, and MCA::SATA::MCA_CONFIG_SATA[LogDeferredInMcaStat] controls the logging behavior of these errors. MCA::SATA::MCA_DESTAT_SATA and MCA::SATA::MCA_DEADDR_SATA are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 0. 1=Transparent errors are supported in this MCA bank, and the TransparentErrorLoggingEnable field in this register controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::SATA::MCA_MISC0_SATA[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::SATA::MCA_STATUS_SATA[TCC] is present.

MSRC000_21A5 [SATA IP Identification] (MCA::SATA::MCA_IPID_SATA)

Reset: 0000_00A8_0000_0000h.

The MCA::SATA::MCA_IPID_SATA register is used by software to determine what IP type and revision is associated with the MCA bank.

_instSATA0SATA_n0_aliasMSR; MSRC000_21A5_instSATA1SATA_n1_aliasMSR; MSRC000_21A5_instSATA2SATA_n2_aliasMSR; MSRC000_21A5_instSATA3SATA_n3_aliasMSR; MSRC000_21A5

Bits	Description
63:48	McaType . Read-only. Reset: 0000h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 0A8h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: <u>_instSATA0SATA_n0_aliasMSR; 0310_2000h</u>
	Init: <u>_instSATA1SATA_n1_aliasMSR; 0320_2000h</u>
	Init: <u>_instSATA2SATA_n2_aliasMSR; 0330_2000h</u>
	Init: <u>_instSATA3SATA_n3_aliasMSR; 0340_2000h</u>

MSRC000_21A6 [SATA Machine Check Syndrome] (MCA::SATA::MCA_SYND_SATA)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::SATA::MCA_STATUS_SATA [Thread 0](#)

_instSATA0SATA_n0_aliasMSR; MSRC000_21A6_instSATA1SATA_n1_aliasMSR; MSRC000_21A6_instSATA2SATA_n2_aliasMSR; MSRC000_21A6_instSATA3SATA_n3_aliasMSR; MSRC000_21A6

Bits	Description
63:33	Reserved.
32	Syndrom . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::SATA::MCA_STATUS_SATA. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::SATA::MCA_SYND_SATA[Length]. The Syndrome field is only valid when MCA::SATA::MCA_SYND_SATA[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::SATA::MCA_SYND_SATA. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::SATA::MCA_SYND_SATA[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::SATA::MCA_SYND_SATA. For example, a syndrome length of 9 means that MCA::SATA::MCA_SYND_SATA[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 132 [MCA_SYND_SATA].

Table 132: MCA_SYND_SATA

Error Type	Bits	Description
perr_p0	[17:0]	Reserved
perr_p1	[17:0]	Reserved
perr_p2	[17:0]	Reserved
perr_p3	[17:0]	Reserved
perr_p4	[17:0]	Reserved
perr_p5	[17:0]	Reserved

perr_p6	[17:0]	Reserved
perr_p7	[17:0]	Reserved

MSRC000_21A8 [SATA Machine Check Deferred Error Status] (MCA::SATA::MCA_DESTAT_SATA)

Reset: Cold,0000_0000_0000_0000h.

Holds status information for the first deferred error seen in this bank.

_instSATA0SATA_n0_aliasMSR; MSRC000_21A8_instSATA1SATA_n1_aliasMSR; MSRC000_21A8_instSATA2SATA_n2_aliasMSR; MSRC000_21A8_instSATA3SATA_n3_aliasMSR; MSRC000_21A8

Bits	Description
63	Val. Read-write, Volatile. Reset: Cold,0. 1=A valid error has been detected (whether it is enabled or not).
62	Overflow. Read-write, Volatile. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. (See the section on overwrite priorities.)
61:59	RESERV4. Read-write. Reset: Cold,0h.
58	AddrV. Read-write, Volatile. Reset: Cold,0. 1=MCA::SATA::MCA_DEADDR_SATA contains address information associated with the error.
57:54	RESERV3. Read-write. Reset: Cold,0h.
53	SyndV. Read-write, Volatile. Reset: Cold,0. 1=This error logged information in MCA::SATA::MCA_SYND_SATA. If MCA::SATA::MCA_SYND_SATA[ErrorPriority] is the same as the priority of the error in MCA::SATA::MCA_STATUS_SATA, then the information in MCA::SATA::MCA_SYND_SATA is associated with the error in MCA::SATA::MCA_DESTAT_SATA.
52:45	RESERV2. Read-write. Reset: Cold,00h.
44	Deferred. Read-write, Volatile. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; poison is created and an exception is deferred until the poison data is consumed.
43:30	RESERV1. Read-write. Reset: Cold,0000h.
29:24	AddrLsb. Read-write, Volatile. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::SATA::MCA_ADDR_SATA[ErrorAddr]. A value of 0 indicates that MCA::SATA::MCA_ADDR_SATA[55:0] contains a valid byte address. A value of 6 indicates that MCA::SATA::MCA_ADDR_SATA[55:6] contains a valid cache line address and that MCA::SATA::MCA_ADDR_SATA[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::SATA::MCA_ADDR_SATA[55:12] contain a valid 4KB memory page and that MCA::SATA::MCA_ADDR_SATA[11:0] should be ignored by error handling software.
23:22	RESERV0. Read-write. Reset: Cold,0h.
21:16	ErrorCodeExt. Read-write, Volatile. Reset: Cold,00h. Logs an extended error code when an error is detected. This model-specific field is used in conjunction with ErrorCode? to identify the error sub-type for root cause analysis.
15:0	ErrorCode. Read-write, Volatile. Reset: Cold,0000h. Error code for this error.

MSRC000_21A9 [SATA Deferred Error Address] (MCA::SATA::MCA_DEADDR_SATA)

Reset: Cold,0000_0000_0000_0000h.

The MCA::SATA::MCA_DEADDR_SATA register stores the address associated with the error in MCA::SATA::MCA_DESTAT_SATA. The register is only meaningful if MCA::SATA::MCA_DESTAT_SATA[Val]=1 and MCA::SATA::MCA_DESTAT_SATA[AddrV]=1. The lowest valid bit of the address is defined by MCA::SATA::MCA_DESTAT_SATA[AddrLsb].

_instSATA0SATA_n0_aliasMSR; MSRC000_21A9

_instSATA1SATA_n1_aliasMSR; MSRC000_21A9

_instSATA2SATA_n2_aliasMSR; MSRC000_21A9

_instSATA3SATA_n3_aliasMSR; MSRC000_21A9

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::SATA::MCA_DESTAT_SATA. The lowest-order valid bit of the address is specified in MCA::SATA::MCA_DESTAT_SATA[AddrLsb].

MSRC001_041A [SATA Machine Check Control Mask] (MCA::SATA::MCA_CTL_MASK_SATA)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instSATA0SATA_n0_aliasMSR; MSRC001_041A

_instSATA1SATA_n1_aliasMSR; MSRC001_041A

_instSATA2SATA_n2_aliasMSR; MSRC001_041A

_instSATA3SATA_n3_aliasMSR; MSRC001_041A

Bits	Description
63:8	Reserved.
7	perr_p7. Read-write. Reset: 0. parity error for port 7
6	perr_p6. Read-write. Reset: 0. parity error for port 6
5	perr_p5. Read-write. Reset: 0. parity error for port 5
4	perr_p4. Read-write. Reset: 0. parity error for port 4
3	perr_p3. Read-write. Reset: 0. parity error for port 3
2	perr_p2. Read-write. Reset: 0. parity error for port 2
1	perr_p1. Read-write. Reset: 0. parity error for port 1
0	perr_p0. Read-write. Reset: 0. parity error for port 0

3.2.5.26 MPDMA

MSR0000_0474...MSRC000_21D0 [MPDMA Machine Check Control] (MCA::MPDMA::MCA_CTL_MPDMA)

Read-write. Reset: 0000_0000_0000_0000h.

0=Disables error reporting for the corresponding error. 1=Enables error reporting via machine check exception for the corresponding error. The MCA::MPDMA::MCA_CTL_MPDMA register must be enabled by the corresponding enable bit in Core::X86::Msr::MCG_CTL. Does not affect error detection, correction, or logging.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D0

_instMPDMAMPDMA1_n[5:0]_aliasMSRLEGACY; MSR0000_0474

Bits	Description
63:49	Reserved.
48	MpDMAPteCmdmemInternalFifoMemError. Read-write. Reset: 0. MPDMA PTE Command Memory Internal ECC or parity error
47	MpDMAPteCmdmemDMAFifoMemError. Read-write. Reset: 0. MPDMA PTE Command Memory DMA ECC or parity error
46	MpDMAPteInternalDataFifoMemError. Read-write. Reset: 0. MPDMA PTE Internal Data FIFO ECC or parity error
45	MpDMAPteHubDataFifoMemError. Read-write. Reset: 0. MPDMA PTE Hub Data FIFO ECC or parity error
44	MpDMAPteCmdfifoMemError. Read-write. Reset: 0. MPDMA PTE Command FIFO ECC or parity error
43	sdp_watchdog_timeout_error. Read-write. Reset: 0. sdp watchdog timer expired
42	MpDMATvfSdpMasterMem6Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 6 ECC or parity error
41	MpDMATvfSdpMasterMem5Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 5 ECC or parity error
40	MpDMATvfSdpMasterMem4Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 4 ECC or parity error
39	MpDMATvfSdpMasterMem3Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 3 ECC or parity error
38	MpDMATvfSdpMasterMem2Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 2 ECC or parity error
37	MpDMATvfSdpMasterMem1Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 1 ECC or parity error
36	MpDMATvfSdpMasterMem0Error. Read-write. Reset: 0. MPDMA TVF SDP Master Memory 0 ECC or parity error
35	MpDMATvfSdpSlaveMem2Error. Read-write. Reset: 0. MPDMA TVF SDP Slave Memory 2 ECC or parity error
34	MpDMATvfSdpSlaveMem1Error. Read-write. Reset: 0. MPDMA TVF SDP Slave Memory 1 ECC or parity error
33	MpDMATvfSdpSlaveMem0Error. Read-write. Reset: 0. MPDMA TVF SDP Slave Memory 0 ECC or parity error
32	MpDMATvfDoorbellMemError. Read-write. Reset: 0. MPDMA TVF Doorbell Memory ECC or parity error
31	MpDMATvfMmioMbox1Error. Read-write. Reset: 0. MPDMA TVF MMIO Mailbox1 ECC or parity error
30	MpDMATvfMmioMbox0Error. Read-write. Reset: 0. MPDMA TVF MMIO Mailbox0 ECC or parity error
29	MpDMATvfDVSecError. Read-write. Reset: 0. MPDMA TVF DVSEC Memory ECC or Parity Error
28	MpDMASHubIfRdBufError. Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
27	MpDMAITagBError_2. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
26	MpDMAITagAError_2. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
25	MpDMAICacheBError_2. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
24	MpDMAICacheAError_2. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
23	MpDMADTagBError_2. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
22	MpDMADTagAError_2. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
21	MpDMADCacheBError_2. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.

20	MpDMADCacheAError_2. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
19	MpDMAITagBError_1. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
18	MpDMAITagAError_1. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
17	MpDMAICacheBError_1. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
16	MpDMAICacheAError_1. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
15	MpDMADTagBError_1. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
14	MpDMADTagAError_1. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
13	MpDMADCacheBError_1. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
12	MpDMADCacheAError_1. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
11	MpDMAITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
10	MpDMAITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
9	MpDMAICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
8	MpDMAICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
7	MpDMADTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
6	MpDMADTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
5	MpDMADCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
4	MpDMADCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
3	MpDMASram3Error. Read-write. Reset: 0. Main SRAM [127:96] bank ECC or parity error.
2	MpDMASram2Error. Read-write. Reset: 0. Main SRAM [95:64] bank ECC or parity error.
1	MpDMASram1Error. Read-write. Reset: 0. Main SRAM [63:32] bank ECC or parity error.
0	MpDMASram0Error. Read-write. Reset: 0. Main SRAM [31:0] bank ECC or parity error.

**MSR0000_0475...MSRC000_21D1 [MPDMA Machine Check Status]
(MCA::MPDMA::MCA_STATUS_MPDMA)**

Reset: Cold,0000_0000_0000_0000h.

Logs information associated with errors.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D1

_instMPDMAMPDMA1_n[5:0]_aliasMSRLEGACY; MSR0000_0475

Bits	Description
63	Val. Reset: Cold,0. 1=A valid error has been detected. This bit should be cleared by software after the register has been read. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
62	Overflow. Reset: Cold,0. 1=An error was detected while the valid bit (Val) was set; at least one error was not logged. Overflow is set independently of whether the existing error is overwritten. See 3.1.3 [Machine Check Errors]. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
61	UC. Reset: Cold,0. 1=The error was not corrected by hardware. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
60	En. Reset: Cold,0. 1=MCA error reporting is enabled for this error, as indicated by the corresponding bit in MCA::MPDMA::MCA_CTL_MPDMA. This bit is a copy of bit in MCA::MPDMA::MCA_CTL_MPDMA for this error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
59	MiscV. Reset: Cold,0. 1=Valid thresholding in MCA::MPDMA::MCA_MISC0_MPDMA. In certain modes, MISC registers are owned by platform firmware and will RAZ when read by non-SMM code. Therefore, it is possible for MiscV=1 and the MISC register to read as all zeros. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
58	AddrV. Reset: Cold,0. 1=MCA::MPDMA::MCA_ADDR_MPDMA contains address information associated with the error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
57	PCC. Reset: Cold,0. 1=Hardware context held by the processor may have been corrupted. Continued operation of the system may have unpredictable results. The error is not recoverable or survivable, and the system should be reinitialized. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
56	ErrCoreIdVal. Reset: Cold,0. 1=The ErrCoreId field is valid. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
55	TCC. Reset: Cold,0. 1=Hardware context of the process thread to which the error was reported may have been corrupted. Continued operation of the thread may have unpredictable results. The thread must be terminated. Only meaningful when MCA::MPDMA::MCA_STATUS_MPDMA[PCC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
54	RESERV54. Reset: Cold,0. MCA_STATUS Register Reserved bit. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
53	SyndV. Reset: Cold,0. 1=This error logged information in MCA::MPDMA::MCA_SYND_MPDMA. If MCA::MPDMA::MCA_SYND_MPDMA[ErrorPriority] is the same as the priority of the error in MCA::MPDMA::MCA_STATUS_MPDMA, then the information in MCA::MPDMA::MCA_SYND_MPDMA is associated with the error in MCA::MPDMA::MCA_STATUS_MPDMA. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
52	Transparent. Reset: Cold,0. 1=This error is a transparent error. This bit is only meaningful when MCA::MPDMA::MCA_STATUS_MPDMA[UC]=0. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
51:47	RESERV47. Reset: Cold,00h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

46	CECC. Reset: Cold,0. 1=The error was a correctable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
45	UECC. Reset: Cold,0. 1=The error was an uncorrectable ECC error according to the restrictions of the ECC algorithm. UC indicates whether the error was actually corrected by the processor. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
44	Deferred. Reset: Cold,0. 1=A deferred error was created. A deferred error is the result of an uncorrectable data error which did not immediately cause a processor exception; an exception is deferred until the erroneous data is consumed. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
43	Poison. Reset: Cold,0. 1=The error was the result of attempting to consume poisoned data. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
42:41	RESERV41. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
40	Scrub. Reset: Cold,0. 1=The error was the result of a scrub operation. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
39:38	RESERV38. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
37:32	ErrCoreId. Reset: Cold,00h. When ErrCoreIdVal=1 this field indicates which core within the processor is associated with the error; Otherwise this field is reserved. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
31:30	RESERV30. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
29:24	AddrLsb. Reset: Cold,00h. Specifies the least significant valid bit of the address contained in MCA::MPDMA::MCA_ADDR_MPDMA[ErrorAddr]. A value of 0 indicates that MCA::MPDMA::MCA_ADDR_MPDMA[55:0] contains a valid byte address. A value of 6 indicates that MCA::MPDMA::MCA_ADDR_MPDMA[55:6] contains a valid cache line address and that MCA::MPDMA::MCA_ADDR_MPDMA[5:0] are not part of the address and should be ignored by error handling software. A value of 12 indicates that MCA::MPDMA::MCA_ADDR_MPDMA[55:12] contain a valid 4KB memory page and that MCA::MPDMA::MCA_ADDR_MPDMA[11:0] should be ignored by error handling software. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
23:22	RESERV22. Reset: Cold,0h. MCA_STATUS Register Reserved bits. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
21:16	ErrorCodeExt. Reset: Cold,00h. Extended Error Code. This field is used to identify the error type for root cause analysis. This field indicates which bit position in MCA::MPDMA::MCA_CTL_MPDMA enables error reporting for the logged error. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.
15:0	ErrorCode. Reset: Cold,0000h. Error code for this error. See 3.1.3.3 [Error Codes] for details on decoding this field. AccessType: Core::X86::Msr::HWCR[McStatusWrEn] ? Read-write : Read,Write-0-only,Error-on-write-1.

Table 133: MCA_STATUS_MPDMA

Error Type	ErrorCode Ext	UC	PCC	TCC	Deferred	Poison	AddrV
MpDMASra m0Error	0x0	0/1	0/1	0/1	0	0	1
MpDMASra m1Error	0x1	0/1	0/1	0/1	0	0	1
MpDMASra	0x2	0/1	0/1	0/1	0	0	1

m2Error							
MpDMASra m3Error	0x3	0/1	0/1	0/1	0	0	1
MpDMADC acheAError	0x4	0/1	0/1	0/1	0	0	1
MpDMADC acheBError	0x5	0/1	0/1	0/1	0	0	1
MpDMADT agAError	0x6	0/1	0/1	0/1	0	0	1
MpDMADT agBError	0x7	0/1	0/1	0/1	0	0	1
MpDMAICa cheAError	0x8	0/1	0/1	0/1	0	0	1
MpDMAICa cheBError	0x9	0/1	0/1	0/1	0	0	1
MpDMAITa gAError	0xa	0/1	0/1	0/1	0	0	1
MpDMAITa gBError	0xb	0/1	0/1	0/1	0	0	1
MpDMADC acheAError_ 1	0xc	0/1	0/1	0/1	0	0	1
MpDMADC acheBError_ 1	0xd	0/1	0/1	0/1	0	0	1
MpDMADT agAError_1	0xe	0/1	0/1	0/1	0	0	1
MpDMADT agBError_1	0xf	0/1	0/1	0/1	0	0	1
MpDMAICa cheAError_1	0x10	0/1	0/1	0/1	0	0	1
MpDMAICa cheBError_1	0x11	0/1	0/1	0/1	0	0	1
MpDMAITa gAError_1	0x12	0/1	0/1	0/1	0	0	1
MpDMAITa gBError_1	0x13	0/1	0/1	0/1	0	0	1
MpDMADC acheAError_ 2	0x14	0/1	0/1	0/1	0	0	1
MpDMADC acheBError_ 2	0x15	0/1	0/1	0/1	0	0	1
MpDMADT agAError_2	0x16	0/1	0/1	0/1	0	0	1
MpDMADT agBError_2	0x17	0/1	0/1	0/1	0	0	1
MpDMAICa cheAError_2	0x18	0/1	0/1	0/1	0	0	1
MpDMAICa cheBError_2	0x19	0/1	0/1	0/1	0	0	1

MpDMAITa gAError_2	0x1a	0/1	0/1	0/1	0	0	1
MpDMAITa gBError_2	0x1b	0/1	0/1	0/1	0	0	1
MpDMASH ubIfRdBufEr ror	0x1c	0/1	0/1	0/1	0	0	1
MpDMATvf DVSecError	0x1d	0/1	0/1	0/1	0	0	1
MpDMATvf MMioMbox 0Error	0x1e	0/1	0/1	0/1	0	0	1
MpDMATvf MmioMbox1 Error	0x1f	0/1	0/1	0/1	0	0	1
MpDMATvf DoorbellMe mError	0x20	0/1	0/1	0/1	0	0	1
MpDMATvf SdpSlaveMe m0Error	0x21	0/1	0/1	0/1	0	0	1
MpDMATvf SdpSlaveMe m1Error	0x22	0/1	0/1	0/1	0	0	1
MpDMATvf SdpSlaveMe m2Error	0x23	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em0Error	0x24	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em1Error	0x25	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em2Error	0x26	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em3Error	0x27	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em4Error	0x28	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em5Error	0x29	0/1	0/1	0/1	0	0	1
MpDMATvf SdpMasterM em6Error	0x2a	0/1	0/1	0/1	0	0	1
MpDMApte CmdfifoMe mError	0x2b	0/1	0/1	0/1	0	0	1

MpDMA Pte HubDataFifo MemError	0x2c	0/1	0/1	0/1	0	0	1
MpDMA Pte InternalDataFifo MemError	0x2d	0/1	0/1	0/1	0	0	1
MpDMA Pte CmdmemDMAFifo MemError	0x2e	0/1	0/1	0/1	0	0	1
MpDMA Pte CmdmemInternalFifo MemError	0x2f	0/1	0/1	0/1	0	0	1
sdp_watchdog_timeout_error	0x30	1	1	1	0	0	1

MSR0000_0476...MSRC000_21D2 [MPDMA Machine Check Address] (MCA::MPDMA::MCA_ADDR_MPDMA)

Reset: Cold,0000_0000_0000_0000h.

MCA::MPDMA::MCA_ADDR_MPDMA stores an address and other information associated with the error in MCA::MPDMA::MCA_STATUS_MPDMA. The register is only meaningful if MCA::MPDMA::MCA_STATUS_MPDMA[Val]=1 and MCA::MPDMA::MCA_STATUS_MPDMA[AddrV]=1.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D2

_instMPDMAMPDMA1_n[5:0]_aliasMSRLEGACY; MSR0000_0476

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Unless otherwise specified by an error, contains the address associated with the error logged in MCA::MPDMA::MCA_STATUS_MPDMA. For physical addresses, the most significant bit is given by Core::X86::Cpuid::LongModeInfo[PhysAddrSize].

Table 134: MCA_ADDR_MPDMA

Error Type	Bits	Description
MpDMASram0Error	[55:0]	Reserved
MpDMASram1Error	[55:0]	Reserved
MpDMASram2Error	[55:0]	Reserved
MpDMASram3Error	[55:0]	Reserved
MpDMADCacheAError	[55:0]	Reserved
MpDMADCacheBError	[55:0]	Reserved
MpDMADTagAError	[55:0]	Reserved
MpDMADTagBError	[55:0]	Reserved
MpDMAICacheAError	[55:0]	Reserved
MpDMAICacheBError	[55:0]	Reserved
MpDMAITagAError	[55:0]	Reserved
MpDMAITagBError	[55:0]	Reserved
MpDMADCacheAError_1	[55:0]	Reserved
MpDMADCacheBError_1	[55:0]	Reserved
MpDMADTagAError_1	[55:0]	Reserved
MpDMADTagBError_1	[55:0]	Reserved

MpDMAICacheAError_1	[55:0]	Reserved
MpDMAICacheBError_1	[55:0]	Reserved
MpDMAITagAError_1	[55:0]	Reserved
MpDMAITagBError_1	[55:0]	Reserved
MpDMADCacheAError_2	[55:0]	Reserved
MpDMADCacheBError_2	[55:0]	Reserved
MpDMADTagAError_2	[55:0]	Reserved
MpDMADTagBError_2	[55:0]	Reserved
MpDMAICacheAError_2	[55:0]	Reserved
MpDMAICacheBError_2	[55:0]	Reserved
MpDMAITagAError_2	[55:0]	Reserved
MpDMAITagBError_2	[55:0]	Reserved
MpDMASHubIfRdBufError	[55:0]	Reserved
MpDMATvfDVSecError	[55:0]	Reserved
MpDMATvfMMioMbox0Error	[55:0]	Reserved
MpDMATvfMmioMbox1Error	[55:0]	Reserved
MpDMATvfDoorbellMemError	[55:0]	Reserved
MpDMATvfSdpSlaveMem0Error	[55:0]	Reserved
MpDMATvfSdpSlaveMem1Error	[55:0]	Reserved
MpDMATvfSdpSlaveMem2Error	[55:0]	Reserved
MpDMATvfSdpMasterMem0Error	[55:0]	Reserved
MpDMATvfSdpMasterMem1Error	[55:0]	Reserved
MpDMATvfSdpMasterMem2Error	[55:0]	Reserved
MpDMATvfSdpMasterMem3Error	[55:0]	Reserved
MpDMATvfSdpMasterMem4Error	[55:0]	Reserved
MpDMATvfSdpMasterMem5Error	[55:0]	Reserved
MpDMATvfSdpMasterMem6Error	[55:0]	Reserved
MpDMA PteCmdfifoMemError	[55:0]	Reserved
MpDMA PteHubDataFifoMemError	[55:0]	Reserved
MpDMA PteInternalDataFifoMemError	[55:0]	Reserved
MpDMA PteCmdmemDMAFifoMemError	[55:0]	Reserved
MpDMA PteCmdmemInternalFifoMemError	[55:0]	Reserved
sdp_watchdog_timeout_error	[55:0]	Reserved

**MSR0000_0477...MSRC000_21D3 [MPDMA Machine Check Miscellaneous 0]
(MCA::MPDMA::MCA_MISC0_MPDMA)**

Log miscellaneous information associated with errors.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D3

_instMPDMAMPDMA1_n[5:0]_aliasMSRLEGACY; MSR0000_0477

Bits	Description
63	Valid. Reset: 1. 1=A valid CntP field is present in this register. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
62	CntP. Reset: 1. 1=A valid threshold counter is present. AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
61	Locked. Reset: 0. 1=Writes to this register are ignored. This bit is set by BIOS to indicate that this register is not available for OS use. BIOS should set this bit if ThresholdIntType is set to SMI . AccessType: Core::X86::Msrr::HWCR[McStatusWrEn] ? Read-write : Read-only.
60	IntP. Reset: 1. 1=ThresholdIntType can be used to generate interrupts. 0=ThresholdIntType and interrupt generation are not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MPDMA::MCA_MISC0_MPDMA[Locked]) ? Read-write : Read-only.
59:56	Reserved.
55:52	LvtOffset. Reset: 0h. One per die. For error thresholding interrupts, specifies the address of the LVT entry in the APIC registers as follows: LVT address = (LvtOffset shifted left 4 bits) + 500h (see Core::X86::Apic::ExtendedInterruptLvtEntries). AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MPDMA::MCA_MISC0_MPDMA[Locked]) ? Read-write : Read-only.
51	CntEn. Reset: 0. 1=Count thresholding errors. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MPDMA::MCA_MISC0_MPDMA[Locked]) ? Read-write : Read-only.
50:49	ThresholdIntType. Reset: Cold,0h. Specifies the type of interrupt signaled when Ovrflw is set and IntP==1. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msrr::McaIntrCfg[ThresholdLvtOffset]). 10b = SMI trigger event. 11b = Reserved. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MPDMA::MCA_MISC0_MPDMA[Locked]) ? Read-write : Read-only.
48	Ovrflw. Reset: Cold,0. Set by hardware when ErrCnt transitions from FFEh to FFFh. When this field is set, ErrCnt no longer increments. When this bit is set, the interrupt selected by the ThresholdIntType field is generated. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MPDMA::MCA_MISC0_MPDMA[Locked]) ? Read-write : Read-only.
47:44	Reserved.
43:32	ErrCnt. Reset: Cold,000h. This is written by software to set the starting value of the error counter. This is incremented by hardware when errors are logged. When this counter overflows, it stays at FFFh (no rollover). The threshold value, written by software, is (FFFh - the desired error count (the number of errors necessary in order for an interrupt to be taken)); the desired error count of 0 (a write value of FFFh) is not supported. AccessType: (Core::X86::Msrr::HWCR[McStatusWrEn] !MCA::MPDMA::MCA_MISC0_MPDMA[Locked]) ? Read-write : Read-only.
31:24	BlkPtr. Read-write. Reset: 00h. 00h=Extended MISC MSR block is not valid. 01h=Extended MSR block is valid.
23:0	Reserved.

MSRC000_21D4 [MPDMA Machine Check Configuration] (MCA::MPDMA::MCA_CONFIG_MPDMA)

Reset: 0000_0002_0000_0123h.

Controls configuration of the associated machine check bank.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D4

Bits	Description
63:39	Reserved.
38:37	DeferredIntType . Read-write. Reset: 0h. Specifies the type of interrupt signaled when a deferred error is logged. 00b = No interrupt. 01b = APIC based interrupt (see Core::X86::Msr::McaIntrCfg[DeferredLvtOffset]). 10b = SMI trigger event. 11b = Reserved.
36:34	Reserved.
33	TransparentErrorLoggingEnable . Read-write. Reset: 1. Init: BIOS,0. 1=Log transparent errors in MCA::MPDMA::MCA_STATUS_MPDMA, MCA::MPDMA::MCA_ADDR_MPDMA, MCA::MPDMA::MCA_MISC0_MPDMA, and MCA::MPDMA::MCA_SYND_MPDMA in addition to MCA::MPDMA::MCA_TRANSYND_MPDMA and MCA::MPDMA::MCA_TRANSADDR_MPDMA. 0=Only log transparent errors in MCA::MPDMA::MCA_TRANSYND_MPDMA and MCA::MPDMA::MCA_TRANSADDR_MPDMA.
32	McaXEnable . Read-write. Reset: 0. Init: BIOS,1. Check: 1. 1=Software has acknowledged support for the MCAX feature set. 0=Software has not acknowledged support for the MCAX feature set. All uncorrected and fatal errors will cause an ErrorEvent packet to be generated. Deferred error interrupts are configured via Core::X86::Msr::McaIntrCfg.
31:9	Reserved.
8	McaLsbInStatusSupported . Read-only. Reset: 1. 1=MCA::MPDMA::MCA_CONFIG_MPDMA[McaLsbInStatusSupported] indicates that AddrLsb is located in McaStatus registers.
7:6	Reserved.
5	DeferredIntTypeSupported . Read-only. Reset: 1. 1=MCA::MPDMA::MCA_CONFIG_MPDMA[DeferredIntType] controls the type of interrupt generated on a deferred error. Deferred errors are supported in this bank only if MCA::MPDMA::MCA_CONFIG_MPDMA[DeferredErrorLoggingSupported]=1.
4:3	Reserved.
2	DeferredErrorLoggingSupported . Read-only. Reset: 0. 1=Deferred errors are supported in this MCA bank, and the LogDeferredInMcaStat field in this register controls the logging behavior of these errors. MCA_DESTAT and MCA_DEADDR are supported in this MCA bank. 0=Deferred errors are not supported in this bank.
1	TransparentErrorLoggingSupported . Read-only. Reset: 1. 1=Transparent errors are supported in this MCA bank, and MCA::MPDMA::MCA_CONFIG_MPDMA[TransparentErrorLoggingEnable] controls the logging behavior of these errors. 0=Transparent errors are not supported in this bank.
0	McaX . Read-only. Reset: 1. 1=This bank provides Machine Check Architecture Extensions. Up to 4 additional MISC registers (MISC1-MISC4) are supported. MCA::MPDMA::MCA_MISC0_MPDMA[BlkPtr] indicates the presence of the additional MISC registers, but is not used to determine their MSR numbers. Deferred error interrupt type is specifiable by MCA bank. MCA::MPDMA::MCA_STATUS_MPDMA[TCC] is present.

MSRC000_21D5 [MPDMA IP Identification] (MCA::MPDMA::MCA_IPID_MPDMA)

Reset: 0003_0001_0000_0000h.

The MCA::MPDMA::MCA_IPID_MPDMA register is used by software to determine what IP type and revision is associated with the MCA bank.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D5

Bits	Description
63:48	McaType . Read-only. Reset: 0003h. The McaType of the MCA bank within this IP.
47:44	InstanceIdHi . Read-write. Reset: 0h. The HI value instance ID of this IP. This is initialized to a unique ID per instance of this register.
43:32	HardwareID . Read-only. Reset: 001h. The Hardware ID of the IP associated with this MCA bank.
31:0	InstanceId . Read-write. Reset: 0000_0000h. The instance ID of this IP. This is initialized to a unique ID per instance of this register.
	Init: _instMPDMAMPDMA1_n0_aliasMSR: 0CF3_0400h
	Init: _instMPDMAMPDMA1_n1_aliasMSR: 0E63_0400h
	Init: _instMPDMAMPDMA1_n2_aliasMSR: 0ED3_0400h
	Init: _instMPDMAMPDMA1_n3_aliasMSR: 0F43_0400h
	Init: _instMPDMAMPDMA1_n4_aliasMSR: 0953_0400h
	Init: _instMPDMAMPDMA1_n5_aliasMSR: 0983_0400h

MSRC000_21D6 [MPDMA Machine Check Syndrome] (MCA::MPDMA::MCA_SYND_MPDMA)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Logs physical location information associated with error in MCA::MPDMA::MCA_STATUS_MPDMA [Thread 0](#)

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC000_21D6

Bits	Description
63:33	Reserved.
32	Syndrome . Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::MPDMA::MCA_STATUS_MPDMA. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::MPDMA::MCA_SYND_MPDMA[Length]. The Syndrome field is only valid when MCA::MPDMA::MCA_SYND_MPDMA[Length] is not 0.
31:27	Reserved.
26:24	ErrorPriority . Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::MPDMA::MCA_SYND_MPDMA. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length . Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::MPDMA::MCA_SYND_MPDMA[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::MPDMA::MCA_SYND_MPDMA. For example, a syndrome length of 9 means that MCA::MPDMA::MCA_SYND_MPDMA[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation . Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error. Decoding is available in Table 135 [MCA_SYND_MPDMA].

Table 135: MCA_SYND_MPDMA

Error Type	Bits	Description
MpDMASram0Error	[17:8] [7:0]	Reserved Reserved
MpDMASram1Error	[17:8] [7:0]	Reserved Reserved
MpDMASram2Error	[17:8] [7:0]	Reserved Reserved
MpDMASram3Error	[17:8] [7:0]	Reserved Reserved
MpDMADCacheAError	[17:8]	Reserved

	[7:0]	Reserved
MpDMADCacheBError	[17:8] [7:0]	Reserved Reserved
MpDMADTagAError	[17:7] [6:0]	Reserved Reserved
MpDMADTagBError	[17:7] [6:0]	Reserved Reserved
MpDMAICacheAError	[17:8] [7:0]	Reserved Reserved
MpDMAICacheBError	[17:8] [7:0]	Reserved Reserved
MpDMAITagAError	[17:6] [5:0]	Reserved Reserved
MpDMAITagBError	[17:6] [5:0]	Reserved Reserved
MpDMADCacheAError_1	[17:8] [7:0]	Reserved Reserved
MpDMADCacheBError_1	[17:8] [7:0]	Reserved Reserved
MpDMADTagAError_1	[17:7] [6:0]	Reserved Reserved
MpDMADTagBError_1	[17:7] [6:0]	Reserved Reserved
MpDMAICacheAError_1	[17:8] [7:0]	Reserved Reserved
MpDMAICacheBError_1	[17:8] [7:0]	Reserved Reserved
MpDMAITagAError_1	[17:6] [5:0]	Reserved Reserved
MpDMAITagBError_1	[17:6] [5:0]	Reserved Reserved
MpDMADCacheAError_2	[17:8] [7:0]	Reserved Reserved
MpDMADCacheBError_2	[17:8] [7:0]	Reserved Reserved
MpDMADTagAError_2	[17:7] [6:0]	Reserved Reserved
MpDMADTagBError_2	[17:7] [6:0]	Reserved Reserved
MpDMAICacheAError_2	[17:8] [7:0]	Reserved Reserved
MpDMAICacheBError_2	[17:8] [7:0]	Reserved Reserved
MpDMAITagAError_2	[17:6] [5:0]	Reserved Reserved
MpDMAITagBError_2	[17:6] [5:0]	Reserved Reserved
MpDMASHubIfRdBufError	[17:6] [5:0]	Reserved Reserved

MpDMATvfDVSecError	[17:8] [7:0]	Reserved Reserved
MpDMATvfMMioMbox0Error	[17:9] [8:0]	Reserved Reserved
MpDMATvfMmioMbox1Error	[17:9] [8:0]	Reserved Reserved
MpDMATvfDoorbellMemError	[17:9] [8:0]	Reserved Reserved
MpDMATvfSdpSlaveMem0Error	[17:4] [3:0]	Reserved Reserved
MpDMATvfSdpSlaveMem1Error	[17:4] [3:0]	Reserved Reserved
MpDMATvfSdpSlaveMem2Error	[17:5] [4:0]	Reserved Reserved
MpDMATvfSdpMasterMem0Error	[17:7] [6:0]	Reserved Reserved
MpDMATvfSdpMasterMem1Error	[17:6] [5:0]	Reserved Reserved
MpDMATvfSdpMasterMem2Error	[17:5] [4:0]	Reserved Reserved
MpDMATvfSdpMasterMem3Error	[17:4] [3:0]	Reserved Reserved
MpDMATvfSdpMasterMem4Error	[17:6] [5:0]	Reserved Reserved
MpDMATvfSdpMasterMem5Error	[17:4] [3:0]	Reserved Reserved
MpDMATvfSdpMasterMem6Error	[17:6] [5:0]	Reserved Reserved
MpDMA PteCmdfifoMemError	[17:7] [6:0]	Reserved Reserved
MpDMA PteHubDataFifoMemError	[17:10] [9:0]	Reserved Reserved
MpDMA PteInternalDataFifoMemError	[17:9] [8:0]	Reserved Reserved
MpDMA PteCmdmemDMAFifoMemError	[17:5] [4:0]	Reserved Reserved
MpDMA PteCmdmemInternalFifoMemError	[17:7] [6:0]	Reserved Reserved
sdp_watchdog_timeout_error	[17:7] [6:0]	Reserved Reserved

MSRC001_041D [MPDMA Machine Check Control Mask] (MCA::MPDMA::MCA_CTL_MASK_MPDMA)

Read-write. Reset: 0000_0000_0000_0000h.

Inhibit detection of an error source.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC001_041D

Bits	Description
63:49	Reserved.
48	MpDMAPteCmdmemInternalFifoMemError . Read-write. Reset: 0. MPDMA PTE Command Memory Internal ECC or parity error
47	MpDMAPteCmdmemDMAFifoMemError . Read-write. Reset: 0. MPDMA PTE Command Memory DMA ECC or parity error
46	MpDMAPteInternalDataFifoMemError . Read-write. Reset: 0. MPDMA PTE Internal Data FIFO ECC or parity error
45	MpDMAPteHubDataFifoMemError . Read-write. Reset: 0. MPDMA PTE Hub Data FIFO ECC or parity error
44	MpDMAPteCmdfifoMemError . Read-write. Reset: 0. MPDMA PTE Command FIFO ECC or parity error
43	sdp_watchdog_timeout_error . Read-write. Reset: 0. sdp watchdog timer expired
42	MpDMATvfSdpMasterMem6Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 6 ECC or parity error
41	MpDMATvfSdpMasterMem5Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 5 ECC or parity error
40	MpDMATvfSdpMasterMem4Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 4 ECC or parity error
39	MpDMATvfSdpMasterMem3Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 3 ECC or parity error
38	MpDMATvfSdpMasterMem2Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 2 ECC or parity error
37	MpDMATvfSdpMasterMem1Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 1 ECC or parity error
36	MpDMATvfSdpMasterMem0Error . Read-write. Reset: 0. MPDMA TVF SDP Master Memory 0 ECC or parity error
35	MpDMATvfSdpSlaveMem2Error . Read-write. Reset: 0. MPDMA TVF SDP Slave Memory 2 ECC or parity error
34	MpDMATvfSdpSlaveMem1Error . Read-write. Reset: 0. MPDMA TVF SDP Slave Memory 1 ECC or parity error
33	MpDMATvfSdpSlaveMem0Error . Read-write. Reset: 0. MPDMA TVF SDP Slave Memory 0 ECC or parity error
32	MpDMATvfDoorbellMemError . Read-write. Reset: 0. MPDMA TVF Doorbell Memory ECC or parity error
31	MpDMATvfMmioMbox1Error . Read-write. Reset: 0. MPDMA TVF MMIO Mailbox1 ECC or parity error
30	MpDMATvfMmioMbox0Error . Read-write. Reset: 0. MPDMA TVF MMIO Mailbox0 ECC or parity error
29	MpDMATvfDVSecError . Read-write. Reset: 0. MPDMA TVF DVSEC Memory ECC or Parity Error
28	MpDMASHubIfRdBufError . Read-write. Reset: 0. System Hub Read Buffer ECC or parity error.
27	MpDMAITagBError_2 . Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
26	MpDMAITagAError_2 . Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
25	MpDMAICacheBError_2 . Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
24	MpDMAICacheAError_2 . Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
23	MpDMADTagBError_2 . Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
22	MpDMADTagAError_2 . Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
21	MpDMADCacheBError_2 . Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
20	MpDMADCacheAError_2 . Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
19	MpDMAITagBError_1 . Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
18	MpDMAITagAError_1 . Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.

17	MpDMAICacheBError_1. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
16	MpDMAICacheAError_1. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
15	MpDMADTagBError_1. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
14	MpDMADTagAError_1. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
13	MpDMADCacheBError_1. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
12	MpDMADCacheAError_1. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
11	MpDMAITagBError. Read-write. Reset: 0. Instruction Tag Cache Bank B ECC or parity error.
10	MpDMAITagAError. Read-write. Reset: 0. Instruction Tag Cache Bank A ECC or parity error.
9	MpDMAICacheBError. Read-write. Reset: 0. Instruction Cache Bank B ECC or parity error.
8	MpDMAICacheAError. Read-write. Reset: 0. Instruction Cache Bank A ECC or parity error.
7	MpDMADTagBError. Read-write. Reset: 0. Data Tag Cache Bank B ECC or parity error.
6	MpDMADTagAError. Read-write. Reset: 0. Data Tag Cache Bank A ECC or parity error.
5	MpDMADCacheBError. Read-write. Reset: 0. Data Cache Bank B ECC or parity error.
4	MpDMADCacheAError. Read-write. Reset: 0. Data Cache Bank A ECC or parity error.
3	MpDMASram3Error. Read-write. Reset: 0. Main SRAM [127:96] bank ECC or parity error.
2	MpDMASram2Error. Read-write. Reset: 0. Main SRAM [95:64] bank ECC or parity error.
1	MpDMASram1Error. Read-write. Reset: 0. Main SRAM [63:32] bank ECC or parity error.
0	MpDMASram0Error. Read-write. Reset: 0. Main SRAM [31:0] bank ECC or parity error.

MSRC001_051D [MPDMA Machine Check Transparent Error Syndrome] (MCA::MPDMA::MCA_TRANSynd_MPDMA)

Read-write, Volatile. Reset: Cold, 0000_0000_0000_0000h.

Contains physical location information for the first transparent error logged in the associated machine check bank.

_instMPDMAMPDMA1_n[5:0]_aliasMSR; MSRC001_051D

Bits	Description
63:33	Reserved.
32	Syndrom. Read-write, Volatile. Reset: Cold, 0. Contains the syndrome, if any, associated with the error logged in MCA::MPDMA::MCA_STATUS_MPDMA. The low-order bit of the syndrome is stored in bit 0, and the syndrome has a length specified by MCA::MPDMA::MCA_SYND_MPDMA[Length]. The Syndrome field is only valid when MCA::MPDMA::MCA_SYND_MPDMA[Length] is not 0.
31:28	Reserved.
27	Valid. Read-write, Volatile. Reset: Cold, 0. This bit is a Valid bit in MCA::MPDMA::MCA_TRANSynd_MPDMA. All errors writing into MCA::MPDMA::MCA_SYND_MPDMA should set this bit to 1.
26:24	ErrorPriority. Read-write, Volatile. Reset: Cold, 0h. Encodes the priority of the error logged in MCA::MPDMA::MCA_SYND_MPDMA. 3'b000 = No error; 3'b001 = Reserved; 3'b010 = Corrected Error; 3'b011 = Deferred Error; 3'b100 = Uncorrected Error; 3'b101 = Fatal Error; all others reserved.
23:18	Length. Read-write, Volatile. Reset: Cold, 00h. Specifies the length in bits of the syndrome contained in MCA::MPDMA::MCA_SYND_MPDMA[Syndrome]. A value of 0 indicates that there is no valid syndrome in MCA::MPDMA::MCA_SYND_MPDMA. For example, a syndrome length of 9 means that MCA::MPDMA::MCA_SYND_MPDMA[Syndrome] bits [8:0] contains a valid syndrome.
17:0	ErrorInformation. Read-write, Volatile. Reset: Cold, 0_0000h. Contains error-specific information about the location of the error.

**MSRC001_061D [MPDMA Machine Check Transparent Error Address]
(MCA::MPDMA::MCA_TRANSADDR_MPDMA)**

Reset: Cold,0000_0000_0000_0000h.

MCA::MPDMA::MCA_TRANSADDR_MPDMA stores the address associated with the error in MCA::MPDMA::MCA_STATUS_MPDMA. The register is only meaningful if MCA::MPDMA::MCA_STATUS_MPDMA[Val]=1 and MCA::MPDMA::MCA_STATUS_MPDMA[AddrV]=1. The lowest valid bit of the address is defined by MCA::MPDMA::MCA_STATUS_MPDMA[AddrLsb].

_instMPDMMPDMA1_n[5:0]_aliasMSR; MSRC001_061D

Bits	Description
63:56	Reserved.
55:0	ErrorAddr. Read-write, Volatile. Reset: Cold,00_0000_0000_0000h. Contains the address, if any, associated with the error logged in MCA::MPDMA::MCA_STATUS_MPDMA. The lowest-order valid bit of the address is specified in MCA::MPDMA::MCA_STATUS_MPDMA[AddrLsb].

3.3 Data Poisoning

Data poisoning is a [RAS](#) mechanism for enabling MCA Recovery (see 3.1.3.7 [MCA Recovery]) by turning system fatal errors into localized errors, reducing the number of system outages. Data poisoning involves marking and tracking corrupt data at a cache line (64Byte) granularity. Poison data propagates through the system with the poison status preserved in memory, caches, and links so that corrupt data is not used.

Creating poison is the process of recognizing that an uncorrected error exists in the cache line and marking the cache line with poison status. When creating poison, blocks with an MCA bank log a deferred error. Poison created in the NBIO propagates to its destination and logged as an Uncorrected Converted to Poison (UCP) error in the NBIO block.

Once created, poison propagates through all blocks until it reaches a point of consumption. The [PCIe](#) root complex can be configured on a link-by-link basis to propagate poison to a link or to halt and interrupt on receiving poison.

When data that has been marked as poison is consumed, a machine check exception is generated. This machine check sets MCA_STATUS[UC] = 1, MCA_STATUS[PCC] = 0. This indicates a context-synchronous uncorrectable error. The OS can terminate the affected process but leave the rest of the system operational.

A poison consumption error is not adequate in determining the physical resource which contains the fault, including in cases where data poisoning is disabled. This is the reason that a deferred error is created when poison status is created. Poison that was transmitted to the consuming block (rather than generated in the consuming block) sets MCA_STATUS[Poison] = 1. This can trigger software to look for a deferred error log. Errors that set MCA_STATUS[Poison] = 1 do not log information in MCA_SYND, as they do not have information about the physical resource in error. This ensures that a poison consumption error does not overwrite physical resource information in MCA_SYND from a previous deferred error in the same block.

3.3.1 Deferred Errors

Deferred errors indicate error conditions which could not be corrected, but have not been consumed. A deferred error is a log indicating the physical resource which encountered an uncorrected error and converted the error to poison. Deferred errors can be used by SEER code to consider fault mitigation such as hardware reconfiguration or retiring a memory page.

The MCA_DESTAT and MCA_DEADDR registers log deferred errors. These registers log only deferred errors; they do not log any other error. In the event that more than one deferred error occurs, the registers preserve the oldest deferred error.

When MCA_CONFIG[LogDeferredErrorsInMcaStat] == 1, deferred errors are also logged in the MCA_STATUS and

MCA_ADDR registers. A deferred error logged in these registers may be overwritten by a later uncorrected or fatal error.

Deferred errors also log information in MCA_SYND when MCA_DESTAT[SyndV] is set. This information may be overwritten by a later uncorrected or fatal error, or may not be logged if MCA_SYND already holds information for an uncorrected or fatal error. The MCA_SYND register contains an indicator of the associated error severity.

Deferred errors can be configured to generate an interrupt upon occurrence. When MCA_CONFIG[DeferredIntTypeSupported] == 1, MCA_CONFIG[DeferredIntType] determines the type of interrupt to be generated for deferred errors in that bank. This can be set to None, [LVT](#), or [SMI](#). If this is set to LVT, Core::X86::Msrr::McaIntrCfg[DeferredLvtOffset] specifies the LVT offset to use for deferred errors. Only one LVT offset is used per socket.

The deferred error is logged independently of any associated poison data machine check and may not be visible to the core that receives the machine check. For example, in a multiprocessor system, it is possible for a memory read to result in a deferred error on the processor to which the memory is attached, and a corresponding machine check exception to a core on the processor which loads the data. In this example, the core receiving the machine check does not have access to the MCA bank in which the deferred error is logged. Therefore, a deferred error handler must check all banks on all cores for a deferred error.

It is also possible to log a deferred error with no corresponding machine check, for example due to speculative operations that are never consumed.

It is possible for a single poisoned cache line to generate multiple deferred errors, for instance if the poison is due to a bad memory location, or if the cache line is overwritten, clearing the poison indicator, and the line is read again.

3.3.2 Clearing Poison

Deferred errors and poison consumption machine checks indicate that one or more cache lines in the system is marked as poison. These errors log the associated address in MCA_DEADDR and MCA_ADDR, respectively.

In order to remove the poison indicator from a cache line, the system can issue a CLZERO instruction to the address associated with the error. Support for CLZERO is published in Core::X86::Cpuid::FeatureExtIdEbx[CLZERO]. CLZERO is the only mechanism that ensures removal of the poison indicator from a cache line, though poison may sometimes be cleared by other system activity (e.g., if an entire cache line is overwritten).

In certain blocks, MCA_DEADDR may not log a system physical address. The system must convert this to a physical address before using this address. See the documentation for each block's registers to determine whether a system physical address is logged for each error. DRAM ECC errors logged in MCA::UMC::MCA_DEADDR_UMC and MCA::UMC::MCA_ADDR_UMC log a truncated address known as a normalized address. AMD provides sample code to convert normalized addresses to system physical addresses.

It is illegal to issue a CLZERO instruction to either APIC or PCI configuration space. Uncorrected errors on APIC or PCI configuration addresses results in a System Fatal Error Event.

When using AMD Secure Encrypted Virtualization (SEV), a hypervisor that wishes to issue a CLZERO instruction to a guest address must first flush the corresponding guest page from all CPU caches using Core::X86::Msrr::VMPAGE_FLUSH.

3.3.3 Enabling and Disabling

To use data poisoning, DRAM ECC must be enabled.

Data poisoning is enabled by default. [AGESA™](#) provides an input parameter for platform firmware to enable and disable

data poisoning.

If UMC::[EccCtrl](#)[[UCFatalEn](#)] = 1b, then data poisoning has been disabled. Programming UMC::[EccCtrl](#)[[UCFatalEn](#)] to 1b is not sufficient to disable data poisoning.

3.4 Platform First Error Handling

Platform First Error Handling (PFEH) allows platform firmware ([SMM](#) code) to see all error conditions, including corrected errors and uncorrected errors, before the OS (non-SMM code). This includes error status stored in register state and active notification such as machine check exceptions, deferred error interrupts, and error thresholding interrupts.

PFEH provides firmware control of two major functions:

- Visibility of MCA error information by non-SMM code ("cloaking"); and
- Invocation of non-SMM error handlers after initial processing by SMM error handlers ("percolation").

Platform firmware can dynamically cloak and uncloak other MCA registers from non-SMM (e.g., OS and hypervisor) accesses. Clearing of an unclocked MCA_STATUS register traps into SMM so that the bank may be cloaked again. If non-SMM error handlers are to be invoked upon exit from the SMM code, an interface is provided to percolate the error. Alternatively, other interfaces such as APEI may be used for firmware to communicate the error condition to the OS.

3.4.1 Overview

Platform First Error Handling consists of the following basic parts:

- Feature indicator: Hardware indicates the presence of the feature.
- Error cloaking: Platform firmware can "cloak" error status logged in the MCA so that non-[SMM](#) code only sees error conditions when platform firmware allows it.
- Interrupt Redirection: Hardware redirects all error notifications to platform firmware, including deferred error and threshold interrupts and machine check exceptions.
- Error Percolation: After handling by platform firmware, select error notifications can be propagated to the OS for further handling upon SMM exit.

3.4.2 Feature Indicator

Support for Platform First Error Handling is indicated in Core::[X86::Cpuid::RasCap](#)[[PfehSupportPresent](#)].

3.4.3 Error Cloaking

Error cloaking is controlled via Core::[X86::Msr::PFEH_CLOAK_CFG](#).

Platform-First Error Handling defines three classes of MCA register:

- "State" registers, which are MCA_STATUS, MCA_ADDR, MCA_DESTAT, MCA_DEADDR, and MCA_SYND
- "Thresholding" registers, which are all MCA_MISCx registers.
- "Control" registers, which are MCA_CTL, MCA_CTL_MASK, MCA_IPID, and MCA_CONFIG

PFEH does not affect reads and writes to MCA "control" registers.

When PFEH is enabled, all MCA "thresholding" registers are RAZ/WRIG except from [SMM](#).

When PFEH is enabled and an MCA bank is cloaked, all "state" registers in that bank are RAZ/WRIG except from SMM.

When PFEH is enabled and an MCA bank is uncloaked, a RDMSR to a "state" register in that MCA bank returns the value of the register. A WRMSR to any "state" registers in that bank triggers an immediate [SMI](#). Writes to MCA_STATUS

with non-zero data values, which trigger a #GP when PFEH is disabled and Core::X86::Msrb::HWCR[McStatusWrEn] is clear, trigger an SMI instead of a #GP. The SMI includes a dedicated reason code in the SMM save state (see Core::X86::Smm::LocalSmiStatus[WRMSR]).

On an SMI generated due to a WRMSR, SMM code can interrogate and take action:

- Read ECX to determine which MCA bank and register is being queried.
- Read EAX:EDX to determine the value being written.
- Perform the update (or not) as desired.
- Re-cloak the MCA bank as desired.

3.4.4 Interrupt Redirection

Once PFEH and MCE/interrupt redirection are enabled, all MCA events – machine check errors, deferred error interrupts, and threshold interrupts – cause the machine to enter SMM. Once in SMM:

- The Local SMI Status register (Core::X86::Smm::LocalSmiStatus) is set to "SmiSrcMca" for deferred and threshold interrupts or "MceRedirSts" for machine check errors.
- The MCA bank corresponding to the error contains valid information about the error.
- For MCE events, Core::X86::Msrb::MCG_STAT contains valid information about the machine check error (e.g., the expected values of RIPV and EIPV for this error).

When Core::X86::Msrb::PFEH_CFG[PfehEnable]=1, deferred error interrupts can be masked using the Core::X86::Msrb::PFEH_DEF_INT_MASK.

When an MCE or interrupt is redirected to SMI, all threads should scan through all MCA banks to look for an error. Any thread that sees an error should handle it. These threads can operate independently of one another; no inter-thread communication is needed.

3.4.5 Event Percolation

When SMM code decides to percolate an error to the OS:

- SMM code uncloaks banks by clearing Core::X86::Msrb::PFEH_CLOAK_CFG[i] for each bank i that it wishes the OS to see.
- SMM code percolates machine check exceptions by setting Core::X86::Msrb::PFEH_CFG[GenerateMceOnExit]. After exiting SMM, hardware generates an MCE and the state in uncloaked MCA Banks is visible to the operating system.
- SMM code percolates deferred error interrupts by setting Core::X86::Msrb::PFEH_CFG[GenerateDeferredLvtOnExit]. After exiting SMM, hardware generates a deferred error interrupt and the state in uncloaked MCA banks is visible to the operating system.
- If SMM code sets both Core::X86::Msrb::PFEH_CFG[GenerateMceOnExit] and Core::X86::Msrb::PFEH_CFG[GenerateDeferredLvtOnExit], both an MCE and a deferred error interrupt is generated.

3.4.6 Enabling and Disabling

Platform First Error Handling requires MCA_CONFIG[McaEn] to be 1 in all MCA banks.

Platform-First error handling requires Core::X86::Msrb::HWCR[SmmLock] to be 1 on all processors.

Platform-First Error Handling must be enabled prior to enabling the MCA in Core::X86::Msrb::MCG_CTL.

At reset time, SMM code enables Platform-First Error Handling as follows:

- Detect support for Platform First Error Handling.

- Enable MCE redirection to [SMI](#) (see below for examples)
- Enable Platform-First Error Handling via Core::X86::Msrr::[PFEH_CFG\[PfehEnable\]](#).
- Mask deferred error interrupts from individual MCA banks as desired via Core::X86::Msrr::[PFEH_DEF_INT_MASK\[Mask\]](#).
- Cloak any banks as desired via Core::X86::Msrr::[PFEH_CLOAK_CFG](#).
- Configure error thresholding in MCA_MISCx registers as desired. If threshold interrupts are desired, SMM must set MCA_MISCx[ThresholdIntType] to SMI.

Examples of how to enable MCE redirection to SMI:

- Method 1
 - Set Core::X86::Msrr::[McExcepRedir\[RedirSmiEn\]](#)
 - Configure Core::X86::Msrr::[SmiTrigIoCycle](#) with the desired I/O cycle
 - Configure FCH::MISC::[IOTRAPPING0](#) to trap the specified I/O address
 - Configure FCH::MISC::[SMITRAPPINGRWRDOVR\[iotrappingrw0\]](#) to trap the specified I/O access type
 - Set FCH::SMI::[SMICONTROL9\[smicontrol_148\]](#) to SMI
- Method 2
 - Set Core::X86::Msrr::[McExcepRedir\[RedirSmiEn\]](#)
 - Get Software SMI I/O port from FCH::PM::[ACPISMICMD](#)
 - Configure Core::X86::Msrr::[SmiTrigIoCycle](#) to Software SMI port with desired IoData value

If SMM code decides to disable Platform First Error Handling:

- SMM code disables the MCA by clearing Core::X86::Msrr::[MCG_CTL](#).
- SMM code disables Platform First Error Handling by clearing Core::X86::Msrr::[PFEH_CFG\[PfehEnable\]](#).
- SMM code disables interrupt redirection by clearing Core::X86::Msrr::[McExcepRedir\[RedirSmiEn\]](#).
- SMM code re-enables the MCA by setting Core::X86::Msrr::[MCG_CTL](#).

3.5 CPU Core RAS Features

The CPU core has a number of [RAS](#) features. Correctness-critical arrays in the CPU are protected against errors by parity or ECC, depending on the size of the array.

3.5.1 L2 and L3 Data Array DEC-TED ECC

The L2 and L3 caches support a double error correct-triple error detect error correcting code (DEC-TED ECC) on the data arrays. This provides robust protection against multi-bit faults in these large SRAM arrays.

Logging of single-bit errors in the MCA can be enabled by setting MCA::L2::MCA_CONFIG_L2[TransparentErrorLoggingEnable] and MCA::L3::MCA_CONFIG_L3[TransparentErrorLoggingEnable]. If this bit is cleared, single-bit errors are only logged in debug registers in L2 and L3 (MCA::L2::MCA_TRANSYND_L2, MCA::L3::MCA_TRANSYND_L3, MCA::L2::MCA_TRANSADDR_L2 and MCA::L3::MCA_TRANSADDR_L3).

The L2 and L3 caches log enough information in MCA_ADDR and MCA_SYND or MCA_TRANSADDR and MCA_TRANSYND to isolate physical failing bits in the cache.

Table 136 [L2 and L3 DEC-TED ECC Syndromes] provides single-bit error syndromes for the DEC-TED ECC. A syndrome for a double-bit error is derived by XORing the syndromes for the individual single-bit errors.

Given an error syndrome E_s , the bits in error can be identified by performing the following sequence of operations:

- If E_s is found in the table entry corresponding to bit[A], the error is a single-bit error in bit[A].
- Otherwise, repeat the following for each bit[B] (with syndrome B_s) in the table:
 - If $E_s \text{ XOR } B_s$ is found in table entry corresponding to bit[A], the error is a double-bit error in bits[A] and

B

- Otherwise, the error is an uncorrectable error.

For example, if $E_s == 0x5152C$, working through the table we try ECC bit[2] with syndrome $0x5AF7C$ and notice that $5152C \text{ XOR } 5AF7C = 0BA50$ which is the syndrome for data bit[51]. So the 2 bits in error are data ECC bit[2] and data bit[51].

Table 136: L2 and L3 DEC-TED ECC Syndromes

	Syndrome	Bit	Syndrome	Bit	Syndrome	Bit	Syndrome
Data[0]	6ce03	Data[70]	13cc5	Data[140]	75157	Data[210]	26f03
Data[1]	59e06	Data[71]	7539f	Data[141]	2798a	Data[211]	51b7a
Data[2]	3e767	Data[72]	248db	Data[142]	0dbe6	Data[212]	3009e
Data[3]	73c0c	Data[73]	1490b	Data[143]	6a73f	Data[213]	40b68
Data[4]	59a01	Data[74]	542d5	Data[144]	254fd	Data[214]	41a0b
Data[5]	3cecf	Data[75]	159c3	Data[145]	093b6	Data[215]	1c3a9
Data[6]	75cf9	Data[76]	09b84	Data[146]	12661	Data[216]	1963a
Data[7]	67a18	Data[77]	50d2e	Data[147]	29017	Data[217]	3cce0
Data[8]	4ebf3	Data[78]	6c06e	Data[148]	7dff4	Data[218]	36d24
Data[9]	73402	Data[79]	5ac10	Data[149]	685aa	Data[219]	7e908
Data[10]	5b0c9	Data[80]	2445c	Data[150]	22e3c	Data[220]	70ed3
Data[11]	39f9e	Data[81]	14b5f	Data[151]	2b187	Data[221]	438a4
Data[12]	12e14	Data[82]	57225	Data[152]	79528	Data[222]	5b988
Data[13]	6bbf2	Data[83]	4cabb	Data[153]	13509	Data[223]	7b52f
Data[14]	38ecd	Data[84]	77fd3	Data[154]	3781c	Data[224]	57ea1
Data[15]	4f630	Data[85]	601d8	Data[155]	6185d	Data[225]	4c8e9
Data[16]	2ef86	Data[86]	6c871	Data[156]	33b90	Data[226]	74e44
Data[17]	5d5e7	Data[87]	5864b	Data[157]	582dc	Data[227]	392ac
Data[18]	057f0	Data[88]	614ab	Data[158]	23ca3	Data[228]	52c27
Data[19]	66a04	Data[89]	3398f	Data[159]	75820	Data[229]	68a72
Data[20]	533ae	Data[90]	1b287	Data[160]	653da	Data[230]	61c4d
Data[21]	76192	Data[91]	7ae54	Data[161]	08ab8	Data[231]	313cc
Data[22]	7ea15	Data[92]	0bcf0	Data[162]	4d5d5	Data[232]	1e383
Data[23]	33f3d	Data[93]	15f78	Data[163]	294bf	Data[233]	1dc86
Data[24]	5aa34	Data[94]	08257	Data[164]	7f67d	Data[234]	47c9b
Data[25]	25c28	Data[95]	0fef6	Data[165]	6e44a	Data[235]	23a09
Data[26]	0b875	Data[96]	2c41f	Data[166]	1ce64	Data[236]	74160
Data[27]	577e5	Data[97]	5845b	Data[167]	59576	Data[237]	7a1a4
Data[28]	0e339	Data[98]	624f7	Data[168]	7fc9f	Data[238]	08ef4
Data[29]	31f9a	Data[99]	6aad0	Data[169]	6ffa7	Data[239]	0fcd6
Data[30]	5e1ba	Data[100]	656a2	Data[170]	00cf2	Data[240]	2f5c5
Data[31]	5ec60	Data[101]	4a2ef	Data[171]	403b1	Data[241]	416e3
Data[32]	1eed8	Data[102]	4b523	Data[172]	031c9	Data[242]	5c122
Data[33]	1df0d	Data[103]	174a0	Data[173]	592e2	Data[243]	1a66f
Data[34]	44db2	Data[104]	0d226	Data[174]	3e5ed	Data[244]	25edf
Data[35]	7a9cf	Data[105]	68cbd	Data[175]	70c96	Data[245]	488a5
Data[36]	0a585	Data[106]	604b4	Data[176]	408c7	Data[246]	0e547
Data[37]	0ade1	Data[107]	2e1d4	Data[177]	42b56	Data[247]	707d6
Data[38]	48697	Data[108]	1e670	Data[178]	45145	Data[248]	012b9
Data[39]	4d608	Data[109]	5f484	Data[179]	2731f	Data[249]	1e91a

Data[40]	2a5af	Data[110]	41c52	Data[180]	0c127	Data[250]	5c646
Data[41]	6655d	Data[111]	5db97	Data[181]	3650e	Data[251]	1be65
Data[42]	500ec	Data[112]	46574	Data[182]	7c3b4	Data[252]	7adb9
Data[43]	6c325	Data[113]	3c856	Data[183]	75ea8	Data[253]	08c66
Data[44]	19dc7	Data[114]	74439	Data[184]	64b89	Data[254]	4cdbf
Data[45]	7d62a	Data[115]	388e6	Data[185]	179e0	Data[255]	76701
Data[46]	2aebc	Data[116]	0ee43	Data[186]	4d058	Ecc[0]	3f3b3
Data[47]	27e7b	Data[117]	31d04	Data[187]	2bcf1	Ecc[1]	6cd00
Data[48]	4c32d	Data[118]	5d0d2	Data[188]	397b7	Ecc[2]	5af7c
Data[49]	75468	Data[119]	07e6b	Data[189]	104ae	Ecc[3]	675f9
Data[50]	65177	Data[120]	60b71	Data[190]	6eae6	Ecc[4]	4d964
Data[51]	0ba50	Data[121]	2d337	Data[191]	1fdec	Ecc[5]	2960a
Data[52]	5475e	Data[122]	44552	Data[192]	43659	Ecc[6]	3c766
Data[53]	170ea	Data[123]	782eb	Data[193]	18a3e	Ecc[7]	376c3
Data[54]	4fa42	Data[124]	0f48d	Data[194]	2174e	Ecc[8]	22af8
Data[55]	6edcb	Data[125]	2df32	Data[195]	708b6	Ecc[9]	698d7
Data[56]	1e42b	Data[126]	04633	Data[196]	4211d	Ecc[10]	7f50a
Data[57]	1c473	Data[127]	7b380	Data[197]	44bee	Ecc[11]	6d41a
Data[58]	18e82	Data[128]	56680	Data[198]	3b090	Ecc[12]	05d3a
Data[59]	23f35	Data[129]	53bfc	Data[199]	557a0	Ecc[13]	2719c
Data[60]	3699b	Data[130]	34a05	Data[200]	4a95a	Ecc[14]	4f0dd
Data[61]	7c175	Data[131]	3bb61	Data[201]	4af44	Ecc[15]	2f66c
Data[62]	36e99	Data[132]	54d6b	Data[202]	0b7d0	Ecc[16]	426d9
Data[63]	7d8c0	Data[133]	56a0d	Data[203]	545de	Ecc[17]	053c2
Data[64]	69cfe	Data[134]	138ce	Data[204]	54116	Ecc[18]	6434b
Data[65]	3ddb0	Data[135]	37a36	Data[205]	56847		
Data[66]	6b506	Data[136]	228e1	Data[206]	508aa		
Data[67]	3bc1b	Data[137]	2a9eb	Data[207]	2e940		
Data[68]	155f5	Data[138]	267f1	Data[208]	1cc7e		
Data[69]	49965	Data[139]	530cb	Data[209]	1a44c		

3.5.2 L2 Tag and State Array SEC-DED ECC

The [L2 cache](#) tag and state arrays support a single-error correct, double-error detect error correcting code (SEC-DED ECC). The L2 tag array ECC code protects 42 tag bits with 7 check bits.

3.5.3 L3 Tag Array SEC-DED ECC

The L3 cache tag array supports a single-error correct, double-error detect error correcting code (SEC-DED ECC).

3.5.4 Shadow Tag Array SEC-DED ECC

The shadow tag array supports a single-error correct, double-error detect error correcting code (SEC-DED ECC).

3.5.5 CPU Watchdog Timer (CPU WDT)

The CPU watchdog timer ([WDT](#)) is implemented as a counter that counts out a configurable time period. The counter starts counting when CpuWdtEn is set. The counter does not count during Halt or stop-grant. It restarts the count each

time an operation of an instruction completes. If no operation completes by the specified time period, then a watchdog timeout is logged in MCA::EX::MCA_STATUS_EX. If a watchdog timeout occurs, a System Fatal Error Event is generated and the system resets.

There is one watchdog timer per thread. The WDT can be configured using register Core::X86::Msr::CpuWdtCfg.

The CPU/SMU/PSP watchdog timers should be set to a smaller interval than the Data Fabric watchdog timers. The CPU/SMU/PSP watchdog timers log more debug information than the Data Fabric watchdog timers.

3.6 Data Fabric RAS Features

3.6.1 Data Fabric Watchdog Timers (DF WDT)

Each data fabric master block has a watchdog timer. These timers are implemented as counters that count out a configurable time period. When these watchdog timers expire, a hardware assertion is logged in MCA::PIE::MCA_STATUS_PIE.

The data fabric watchdog timers can be configured in the DF::DfGlobalCtrl registers.

3.6.2 Bad Data Containment

In certain cases, when an error cannot be contained, the processor signals a System Fatal Error Event packet on the data fabric and links. This is used to quickly stop the transmission of potentially bad data when there are no other means to do so and causes a reset of the system.

3.6.3 Data Fabric Links

In multi-node processors, a link is a group of signals connecting two sockets or two dies. All die-to-die and socket-to-socket links are protected by a cyclic redundancy check (CRC). CRC errors are automatically retried by the links; errors are reported when the rate of corrected errors exceeds a configurable threshold, or when an unrecoverable link error occurs.

Link corrected errors (which indicate that the bit error rate of the link has exceeded a configured threshold) and unrecoverable errors are reported in MCA::PIE::MCA_STATUS_PIE.

3.6.4 Probe Filter ECC

In multi-node processors, the data fabric may implement a probe filter. If so, the probe filter is protected by SEC-DED ECC.

3.7 DRAM RAS Features

3.7.1 Error Correcting Codes

DRAM is protected by an error correcting code (ECC). The memory controller supports multiple error correcting codes. Each error correcting code features an ECC word formed by a symbol based code. The primary difference between the codes is the symbol size:

- The x4 code uses thirty-six 4-bit symbols to make a 144-bit ECC word made up of 128 data bits and 16 check bits

- The x16 code uses up to eighteen 16-bit symbols to make up to a 288-bit ECC word made up of up to 256 data bits and 32 check bits.

The x4 code is a single symbol correcting (SSC) and a double symbol detecting (DSD) code. This means the x4 code is able to correct 100% of single symbol errors (any bit error combination within one symbol), and detect 100% of double symbol errors (any bit error combination within two symbols).

The x16 code is a single symbol correcting and 100% double symbol detecting code. The x16 code can detect 99.97% of all double symbol errors greater than two bits. The code is enhanced by an automated hardware history mechanism. The combination of x16 ECC and the hardware history mechanism detects over 99.99% of two symbol errors based on a statistical model of expected DRAM faults.

In certain configurations, the ECC provides AMDC or chipkill functionality; all errors from a single DRAM device are corrected. AMDC is only possible when indicated by UMC::UmcCapHi[EccChipKillCap]. For AMDC functionality, with SSC (Single Symbol Correction) ECC support and each code word covering 4 beats, the symbol size must be at least 4 times of the DRAM device width and bit interleaving must be enabled via UMC::EccCtrl. For example, for a system configured with x4 devices, x16 symbols are necessary so that the failure of one device can be corrected, since the device contributes 16 total bits of data confined to one symbol. x4 symbols cannot support x4 device AMDC.

In other configurations, the ECC provides bounded fault correction; all errors from two adjacent DQ pins of a single DRAM device are corrected.

AMD recommends using the x16 ECC to ensure optimal error correction and data integrity.

DRAM ECC features are enabled and configured via UMC::EccCtrl.

3.7.1.1 ECC Mapping

To understand the effects of configuration on DRAM device errors, the ECC must be mapped to the physical memory configuration, including device width and bit interleaving.

- DRAM device width refers to the number of bits sourced simultaneously from a single memory chip. For example, a x4 device contributes 4-bits to the ECC word on each beat of data. Either nine or ten devices, representing one rank, contribute 36- or 40-bits on each beat.
- Bit interleaving refers to the way bits from the two memory beats are organized to form an ECC word. When specified, even and odd bits from multiple beats can be interleaved to create an ECC word. When bit interleaving is enabled, a partially failing device (e.g., pin failure) contributes two incorrect bits to the same symbol. When bit interleaving is not enabled, a partially failing device contributes failures to two different symbols.

In DDR5, the x16 ECC mapping is as follows:

- On a 40-bit channel, with x4 or x8 DRAM devices, there are 4 160-bit ECC words per cache line comprised of 128 data bits and 32 check bits each (data bits 255:128 of the ECC are tied to 0). Each ECC word covers 4 consecutive beats of data. When bit interleaving is enabled, data from 4 adjacent DQ pins is mapped to a single ECC symbol. When using x4 DRAM devices, this mapping provides AMDC correction (device 0 maps to symbol 0, device 1 maps to symbol 1, etc; ECC devices map to symbols 10h and 11h). When using x8 DRAM devices, this mapping provides "bounded fault" correction.
- On a 36-bit channel with x4 DRAM devices, there are 2 288-bit ECC words per cache line comprised of 256 data bits and 32 check bits each. Each ECC word covers 8 consecutive beats of data. When bit interleaving is enabled, data from 2 adjacent DQ pins is mapped to a single ECC symbol (DQ[1:0] map to symbol 0, DQ[3:2] map to symbol 1, etc; ECC DQs map to symbols 10h and 11h). This provides "bounded fault" correction.

In DDR5, the x4 ECC mapping is as follows:

- On a 40-bit channel, with x4 or x8 DRAM devices, there are 8 80-bit ECC words per cache line comprised of 64 data bits and 16 check bits each (data bits 127:64 of the ECC are tied to 0). Each ECC word covers 2 consecutive beats of data. When bit interleaving is enabled, data from 2 adjacent DQ pins is mapped to a single ECC symbol. When using x4 DRAM devices, this mapping provides "bounded fault" correction.
- On a 36-bit channel with x4 DRAM devices, there are 4 144-bit ECC words per cache line comprised of 128 data bits and 16 check bits each. Each ECC word covers 4 consecutive beats of data. When bit interleaving is enabled, data from a single DQ pin is mapped to a single ECC symbol.

Upon detecting an error, the memory controller logs the chip select and symbol of the error in the MCA::UMC::MCA_SYND_UMC register. This can be used to trace the error back to a specific DRAM device. The symbol logged in MCA::UMC::MCA_SYND_UMC corresponds to the symbol number from Table 137 [x16 ECC Single-Bit Correctable Syndrome Bits[31:15]] plus one (e.g., symbol 00h in Table 137 [x16 ECC Single-Bit Correctable Syndrome Bits[31:15]] is logged as 01h).

When an error occurs in DRAM, the MCA::UMC::MCA_ADDR_UMC register logs the address associated with the error. This address is a UMC-normalized address. This address can be translated to a system physical address or to a DRAM bank/row/column address using functions made available by AMD.

3.7.1.2 DRAM Device to Symbol Mapping

A symbol logged in MCA::UMC::MCA_SYND_UMC can be mapped to a DRAM device or devices using the following algorithm when using DIMMs with x4 or x8 DRAM devices.

```
void mapSymbolToDevice(uint32 &DeviceStart, uint32 &DeviceEnd) {

    uint32 Symbol = MCA_SYND_UMC[ErrorInformation[13:8]] - 1 // Subtract one from logged symbol

    if (UMC::DimmCfg[X4Dram]==1) { // x4 DIMM
        if (UMC::EccCtrl[EccSymbolSize]==0 && UMC::EccCtrl[EccSymbolSize16]==0) { // x4 ECC
            if (UMC::EccCtrl[EccBitInterleaving]==1) { // Bit Interleaving enabled
                // Symbol 0, 1 -> device 0; symbol 2, 3 -> device 1; etc
                DeviceStart = Symbol / 2h
                DeviceEnd = Symbol / 2h
            }
            else { // UMC::EccCtrl[EccBitInterleaving]==0
                // Symbol 0 -> device 0; symbol 1 -> device 1; symbol 16 -> device 0; etc.
                // Symbol 32,34 -> device 16; symbol 33,35 -> device 16
                if (Symbol < 20h) {
                    DeviceStart = Symbol % 10h
                    DeviceEnd = Symbol % 10h
                }
                else { // 20h <= Symbol <= 23h
                    DeviceStart = 10h + (Symbol % 2h)
                    DeviceEnd = 10h + (Symbol % 2h)
                }
            }
        }
    }
    else if (UMC::EccCtrl[EccSymbolSize16]==1) { // x16 ECC
        if (UMC::EccCtrl[EccBitInterleaving]==1) {
            // Symbol 0 -> device 0; symbol 1 -> device 1; symbol 17 -> device 17
            DeviceStart = Symbol
            DeviceEnd = Symbol
        }
    }
}
```

```

    }
    else { // UMC::EccCtrl[EccBitInterleaving]==0
        // Symbol 0 -> device 0,1,2,3; symbol 1 -> device 4,5,6,7;
        // Symbol 4 -> device 0,1,2,3; symbol 5 -> device 4,5,6,7;
        // Symbol 16 -> device 16, 17; Symbol 17 -> device 16, 17
        if (Symbol < 10h) {
            DeviceStart = (4h * Symbol) % 10h
            DeviceEnd = ((4h * Symbol) + 3) % 10h
        }
        else { // Symbol == 16 or 17
            DeviceStart = 10h
            DeviceEnd = 11h
        }
    }
}

}
else if (UMC::DimmCfg[X4Drain]==0 && UMC::DimmCfg[X16Drain]==0) { // x8 DIMM
    if (UMC::EccCtrl[EccSymbolSize]==0 && UMC::EccCtrl[EccSymbolSize16]==0) { // x4 ECC
        if (UMC::EccCtrl[EccBitInterleaving]==1) {
            // Symbol 0, 1 -> device 0; symbol 2, 3 -> device 0; symbol 4, 5 -> device 1
            DeviceStart = Symbol / 4h
            DeviceEnd = Symbol / 4h
        }
        else if (UMC::EccCtrl[EccBitInterleaving]==0) {
            // Symbol 0 -> device 0; symbol 1 -> device 0; symbol 2 -> device 1
            // Symbol 16 -> device 0; symbol 17 -> device 0; symbol 18 -> device 1
            // Symbol 32,33,34,35 -> device 8
            if (Symbol < 20h) {
                DeviceStart = (Symbol / 2h) % 8h
                DeviceEnd = (Symbol / 2h) % 8h
            }
            else { // 32 <= Symbol <= 35
                DeviceStart = 8h
                DeviceEnd = 8h
            }
        }
    }
}
else if (UMC::EccCtrl[EccSymbolSize16]==1) { // x16 ECC
    if (UMC::EccCtrl[EccBitInterleaving]==1) {
        // Symbol 0 -> device 0; symbol 1 -> device 0; symbol 2 -> device 1
        DeviceStart = Symbol / 2h
        DeviceEnd = Symbol / 2h
    }
    else { // UMC::EccCtrl[EccBitInterleaving]==0
        // Symbol 0 -> device 0, 1; symbol 1 -> device 2,3;
        // Symbol 4 -> device 0, 1; symbol 5 -> device 2,3;
        // Symbol 16 -> device 16, 17; Symbol 17 -> device 16, 17
        if (Symbol < 10h) {
            DeviceStart = 2h * (Symbol % 4h)
            DeviceEnd = 2h * (Symbol % 4h) + 1
        }
        else { // Symbol == 16 or 17
            DeviceStart = 10h

```

```

        DeviceEnd = 11h
    }
}
}
}
}

```

3.7.1.3 Hardware Managed ECC History Mechanism

The x16 ECC is enhanced by the following automated hardware history mechanism to provide additional coverage against two symbol errors that result from one or more defective DRAM devices:

- The hardware error history is cleared at the beginning of each cache line received from DRAM.
- For each ECC word in the cache line:
 - If the ECC indicates that there is no error, then the data can safely be forwarded.
 - If the ECC indicates that there is an uncorrectable error, then the ECC word is considered uncorrectable.
 - If the ECC indicates a correctable error, then:
 - If it is the first error in the cache line, the symbol position in error is saved for the remainder of the cache line (only the first error of the cache line is saved), and the data is corrected but not yet forwarded.
 - If the ECC indicates a subsequent correctable error in the same symbol position, then the data is corrected, but not yet forwarded.
 - If the ECC indicates subsequent correctable errors in symbols other than the saved symbol position, then all ECC words in the cache line are considered uncorrectable.
 - Uncorrectable errors detected by this mechanism are indicated in MCA_SYND.
- At the end of the cache line, all data is forwarded.

3.7.1.4 ECC Syndromes

3.7.1.4.1 X16 ECC

The syndrome field uniquely identifies the failing bit positions of a correctable ECC error. A full syndrome table is not provided in this document. Instead, to decode the location of a x16 ECC error, refer to data captured in MCA_SYND_UMC.

MCA_SYND_UMC[13:8] contains the symbol in error plus one.

- 72-bit Channel: Symbols 07h-00h map to data bits[127:0]; symbols 0Fh-08h map to data bits[255:128]; symbol 12h maps to special purpose non-data bits; symbols 11h-10h map to ECC check bits for data bits[255:0] and symbol 12h. If MCA_SYND_UMC[13:8] is between 12h-01h, the error is correctable. Otherwise the error is uncorrectable.
- 80-bit Channel: Symbols 07h-00h map to data bits[127:0]; bits[255:128] are unused; symbol 12h maps to special purpose non-data bits; symbols 11h-10h map to ECC check bits for data bits[127:0] and symbol 12h. If MCA_SYND_UMC[13:8] is between 08h-01h or 12h-11h, the error is correctable. Otherwise the error is uncorrectable.

MCA_SYND_UMC[47:32] contains the low-order bits of the syndrome, which contain the bitmask of the error within the symbol. MCA_SYND_UMC[63:48] can be ignored for the purposes of mapping errors to failing bit positions.

For example, if MCA_SYND_UMC[63:32] == 7c760001h and MCA_SYND_UMC[13:8] == 1h, the error maps to bit[0] of symbol 0h. Similarly, if MCA_SYND_UMC[63:32] == de9c0013h and MCA_SYND_UMC[13:8] == 6h, the error maps to bits[0], [1], and [4] of symbol 5h.

An alternate means of constructing the syndrome values uses Table 137 [x16 ECC Single-Bit Correctable Syndrome Bits[31:15]]. The table provides the most significant 16 bits of the syndrome for all possible single-bit errors. The lower sixteen bits of the syndrome for each single-bit error are the bitmask value in the left-most column of the table. The full syndrome for any error can be constructed by XORing the syndromes for the set of single-bit errors that make up the error.

For example, the syndrome for the two bit error consisting of an error in symbol 0 bit[0], and symbol 2 bit[0] is 0x7C76_0001 XOR 0xB32E_0001, or 0xCF58_0000. If that two bit error occurs, 0xCF58_0000 will appear in MCA_SYND_UMC[63:32].

Table 137: x16 ECC Single-Bit Correctable Syndrome Bits[31:15]

0001	7C76	D22B	B32E	0C76	0CB4	3153	0C25	B3ED	4DE7	E3E9	3DB5	FE59	1157	A2E8	2C20	6156	0000	0001
0002	F8EC	0447	C64D	18EC	1968	62A6	184A	C7CB	9BCE	67C3	7B6A	5CA3	22AE	E5C1	5840	C2AC	0000	0002
0004	51C9	088E	2C8B	31D8	32D0	C54C	3094	2F87	978D	CF86	F6D4	B946	455C	6B93	B080	2549	0000	0004
0008	A392	111C	5916	63B0	65A0	2A89	6128	5F0E	8F0B	3F1D	4DB9	D29D	8AB8	D726	C111	4A92	0000	0008
0010	E735	2238	B22C	C760	CB40	5512	C250	BE1C	BE07	7E3A	9B72	052B	B561	0E5D	2233	9524	0000	0010
0020	6E7B	4470	C449	2ED1	3691	AA24	24B1	DC29	DC1F	FC74	96F5	0A56	CAD3	1CBA	4466	8A59	0000	0020
0040	DCF6	88E0	2883	5DA2	6D22	F459	4962	1843	182F	58F9	8DFB	14AC	35B7	3974	88CC	B4A3	0000	0040
0080	19FD	B1D1	5106	BB44	DA44	48A3	92C4	3086	305E	B1F2	BBE7	2958	6B6E	72E8	B189	C957	0000	0080
0100	33FA	C3B3	A20C	D699	1499	9146	8599	610C	60BC	C3F5	D7DF	52B0	D6DC	E5D0	C303	32BF	0000	0100
0200	67F4	2777	E409	0D23	2932	829D	AB23	C218	C178	27FB	0FAF	A560	0DA9	6BB1	2617	657E	0000	0200
0400	CFE8	4EEE	6803	1A46	5264	A52B	F657	2421	22E1	4FF6	1F5E	EAD1	1B52	D762	4C2E	CAFC	0000	0400
0800	3FC1	9DDC	D006	348C	A4C8	EA47	4CBF	4842	45C2	9FEC	3EBC	75B3	36A4	0ED5	985C	35E9	0000	0800
1000	7F82	9BA9	001D	6918	E981	749F	997E	9084	8B84	9FC9	7D78	EB66	6D48	1DAA	90A9	6BD2	0000	1000
2000	FF04	9743	003A	D230	7313	E93E	92ED	8119	B719	9F83	FAF0	76DD	DA90	3B54	8143	D7A4	0000	2000
4000	5E19	8E97	0074	0471	E626	726D	85CB	A223	CE23	9F17	55F1	EDB A	1531	76A8	A297	0F59	0000	4000
8000	BC32	BD3F	00E8	08E2	6C5D	E4DA	AB87	E457	3C57	9E3F	ABE2	7B65	2A62	ED50	E53F	1EB2	0000	8000
Bitmask	0h	1h	2h	3h	4h	5h	6h	7h	8h	9h	Ah	Bh	Ch	Dh	Eh	Fh	10h	11h
Symbol in Error																		

3.7.1.4.2 X4 ECC

The syndrome field uniquely identifies the failing bit positions of a correctable ECC error. Only syndromes identified by Table 138 [x4 ECC Correctable Syndromes] are correctable by the error correcting code.

Symbols 0Fh-00h map to data bits[63:0]; symbols 1Fh-10h map to data bits[127:64]; symbols 21h-20h map to ECC check bits for data bits[63:0]; symbols 23h-22h map to ECC check bits for data bits[127:64].

To use table Table 138 [x4 ECC Correctable Syndromes], first find the 16-bit syndrome value in the table. This is performed by using low order 4 bits of the syndrome to select the appropriate error bitmask column. The entire four digit syndrome should then be in one of the rows of that column. The Symbol In Error row indicates which symbol, and therefore which DIMM has the error, and the column indicates which bits within the symbol.

For example, if the ECC syndrome is 6913h, then symbol 05h has the error, and bits[0] and [1] within that symbol are corrupted, since the syndrome is in column 3h (0011b). Symbol 05h maps to bits[23:20], so the corrupted bits are [20] and [21].

Table 138: x4 ECC Correctable Syndromes

Symbol	Error Bitmask
--------	---------------

In Error	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Data 0	e821	7c32	9413	bb44	5365	c776	2f57	dd88	35a9	a1ba	499b	66cc	8eed	1afe	f2df
Data 1	5d31	a612	fb23	9584	c8b5	3396	6ea7	eac8	b7f9	4cda	11eb	7f4c	227d	d95e	846f
Data 2	0001	0002	0003	0004	0005	0006	0007	0008	0009	000a	000b	000c	000d	000e	000f
Data 3	2021	3032	1013	4044	6065	7076	5057	8088	a0a9	b0ba	909b	c0cc	e0ed	f0fe	d0df
Data 4	5041	a082	f0c3	9054	c015	30d6	6097	e0a8	b0e9	402a	106b	70fc	20bd	d07e	803f
Data 5	be21	d732	6913	2144	9f65	f676	4857	3288	8ca9	e5ba	5b9b	13cc	aded	c4fe	7adf
Data 6	4951	8ea2	c7f3	5394	1ac5	dd36	9467	a1e8	e8b9	2f4a	661b	f27c	bb2d	7cde	358f
Data 7	74e1	9872	ec93	d6b4	a255	4ec6	3a27	6bd8	1f39	f3aa	874b	bd6c	c98d	251e	51ff
Data 8	15c1	2a42	3f83	cef4	db35	e4b6	f177	4758	5299	6d1a	78db	89ac	9c6d	a3ee	b62f
Data 9	3d01	1602	2b03	8504	b805	9306	ae07	ca08	f709	dc0a	e10b	4f0c	720d	590e	640f
Data 10	9801	ec02	7403	6b04	f305	8706	1f07	bd08	2509	510a	c90b	d60c	4e0d	3a0e	a20f
Data 11	d131	6212	b323	3884	e9b5	5a96	8ba7	1cc8	cdf9	7eda	afeb	244c	f57d	465e	976f
Data 12	e1d1	7262	93b3	b834	59e5	ca56	2b87	dc18	3dc9	ae7a	4fab	642c	85fd	164e	f79f
Data 13	6051	b0a2	d0f3	1094	70c5	a036	c067	20e8	40b9	904a	f01b	307c	502d	80de	e08f
Data 14	a4c1	f842	5c83	e6f4	4235	1eb6	ba77	7b58	df99	831a	27db	9dac	396d	65ee	c12f
Data 15	11c1	2242	3383	c8f4	d935	eab6	fb77	4c58	5d99	6e1a	7fdb	84ac	956d	a6ee	b72f
Data 16	45d1	8a62	cfb3	5e34	1be5	d456	9187	a718	e2c9	2d7a	68ab	f92c	bcfd	734e	369f
Data 17	63e1	b172	d293	14b4	7755	a5c6	c627	28d8	4b39	99aa	fa4b	3c6c	5f8d	8d1e	eeff
Data 18	b741	d982	6ec3	2254	9515	fbdb	4c97	33a8	84e9	ea2a	5d6b	11fc	a6bd	c87e	7f3f
Data 19	dd41	6682	bbc3	3554	e815	53d6	8e97	1aa8	c7e9	7c2a	a16b	2ffc	f2bd	497e	943f
Data 20	2bd1	3d62	16b3	4f34	64e5	7256	5987	8518	aec9	b87a	93ab	ca2c	e1fd	f74e	dc9f
Data 21	83c1	c142	4283	a4f4	2735	65b6	e677	f858	7b99	391a	badb	5cac	df6d	9dee	1e2f
Data 22	8fd1	c562	4ab3	a934	26e5	6c56	e387	fe18	71c9	3b7a	b4ab	572c	d8fd	924e	1d9f
Data 23	4791	89e2	ce73	5264	15f5	db86	9c17	a3b8	e429	2a5a	6dcb	f1dc	b64d	783e	3faf
Data 24	5781	a9c2	fe43	92a4	c525	3b66	6ce7	e3f8	b479	4a3a	1dbb	715c	26dd	d89e	8f1f
Data 25	bf41	d582	6ac3	2954	9615	fcdb	4397	3ea8	81e9	eb2a	546b	17fc	a8bd	c27e	7d3f
Data 26	9391	e1e2	7273	6464	f7f5	8586	1617	b8b8	2b29	595a	cacb	dcdb	4f4d	3d3e	aeaf
Data 27	cce1	4472	8893	fdb4	3155	b9c6	7527	56d8	9a39	12aa	de4b	ab6c	678d	ef1e	23ff
Data 28	a761	f9b2	5ed3	e214	4575	1ba6	bcc7	7328	d449	8a9a	2dfb	913c	365d	688e	cfef
Data 29	ff61	55b2	aad3	7914	8675	2ca6	d3c7	9e28	6149	cb9a	34fb	e73c	185d	b28e	4def
Data 30	5451	a8a2	fcf3	9694	c2c5	3e36	6a67	ebe8	bfb9	434a	171b	7d7c	292d	d5de	818f
Data 31	6fc1	b542	da83	19f4	7635	acb6	c377	2e58	4199	9b1a	f4db	37ac	586d	82ee	ed2f
Check0	be01	d702	6903	2104	9f05	f606	4807	3208	8c09	e50a	5b0b	130c	ad0d	c40e	7a0f
Check1	4101	8202	c303	5804	1905	da06	9b07	ac08	ed09	2e0a	6f0b	f40c	b50d	760e	370f
Check2	c441	4882	8cc3	f654	3215	bed6	7a97	5ba8	9fe9	132a	d76b	adfc	69bd	e57e	213f
Check3	7621	9b32	ed13	da44	ac65	4176	3757	6f88	19a9	f4ba	829b	b5cc	c3ed	2efe	58df

3.7.2 Bad Symbol ID

The Bad Symbol ID feature can be used with the x16 ECC to provide additional coverage against double symbol errors.

When ECC is enabled, Bad Symbol ID should be enabled at boot using UMC::[EccCtrl\[EccBadDramSymEn\]](#).

Hardware provides one Bad Symbol ID per DRAM subchannel, rank-multiplier, and rank. Software can program the bad symbol by using UMC::[EccBadSymbolIndex](#) and UMC::[EccBadSymbolData](#).

Hardware uses the information that a symbol is bad as follows:

- If an ECC word has a correctable error only in a rank and symbol identified by this register, then the error is a correctable error.
- If an ECC word has an error in a rank identified by this register, but a different symbol, then the word is presumed to have at least two symbol errors and the error is treated as uncorrectable.

3.7.3 DRAM Scrubbers

Scrubbers are used to periodically read memory locations. There are two primary benefits to scrubbing. First, scrubbing

corrects any correctable errors which are discovered before they can possibly migrate into uncorrectable errors. This is particularly important for soft errors, which are caused by external sources such as radiation and which are temporary conditions which do not indicate malfunctioning hardware. Second, scrubbers help identify marginal or failed hardware by finding and logging repeated errors at the same locations.

The scrubbers can be enabled independently or together.

3.7.3.1 Periodic Scrubber

Periodic scrubber ensures that all cachelines in DRAM memory are periodically read and, if correctable errors are discovered, they are corrected. The periodic scrubber is configured by the following registers:

- UMC::[DramScrubBaseAddr](#)
- UMC::[DramScrubLimitAddr](#)
- UMC::[DramScrubAddrLo](#)
- UMC::[DramScrubAddrHi](#)

Periodic scrubbing must be disabled during address-based error injection.

There is one periodic scrubber per memory controller. Unless the memory is die-interleaved or socket-interleaved, each periodic scrubber should be configured to scrub all memory attached to a given die. When the memory is interleaved, use the periodic scrubber on the lowest numbered die to scrub all dies in the interleave region.

The scrub rate is specified as the time between successive scrub events. A scrub event occurs when a line of memory is checked for errors; the amount of memory that is checked varies based on the memory block (see field descriptions).

The time required to fully scrub the memory in a controller is determined as:

- $\text{Time} = ((\text{memory size in bytes})/64) * (\text{DramScrubRate})$.
- E.g., a die with 1 GB (2^{30}) of system memory and a Dram Scrub Rate of 5.24 ms is fully scrubbed about once every 24.4 hours.

AMD recommends that the DRAM periodic scrubber should be set to scrub all of memory every 24 hours (approximately), unless other guidelines are given by the DRAM vendor. Adjustments may be necessary due to special circumstances.

The periodic scrubber can be programmed to start at a specific address using UMC::[DramScrubAddrLo](#) and UMC::[DramScrubAddrHi](#). To program a start address, use the following programming sequence:

- Disable the periodic scrubber via UMC::[DramScrubCtrl](#)
- Program UMC::[DramScrubAddrHi](#) to the desired location
- Program UMC::[DramScrubAddrLo](#) to the desired location
- Re-enable the periodic scrubber via UMC::[DramScrubCtrl](#)

Changing the scrub address via any other sequence is invalid and may result in unpredictable behavior.

Changing the scrub address is allowed on production parts only after a secure-unlock.

3.7.3.2 Redirect and Poison Scrubbers

The redirect scrubber scrubs DRAM by issuing a writeback to a DRAM address accessed during normal operation. These scrubs are used to correct single bit correctable errors in a cache line.

The redirect scrubber is enabled via UMC::[DramScrubCtrl](#). When enabled, the redirect scrubber is invoked any time a memory read detects a corrected ECC error. UMC::[DramScrubCtrl](#) can be used to limit the number of redirect scrubs in flight at any one time (e.g., to limit the performance impact of redirect scrubs). AMD recommends that the number of in-flight redirect scrubs be limited to at most 8.

The poison scrubber issues a writeback to a DRAM address accessed during normal operation. When an uncorrectable error is found in DRAM, poison data is created and sent to the consumer and a deferred error is logged. The poison scrubber is used to mark the location as poison in DRAM to avoid subsequent accesses from logging another deferred error. AMD recommends the poison scrubber be enabled.

3.7.4 Corrected ECC Error Counters

Registers UMC::[EccErrCntSel](#) and UMC::[EccErrCnt](#) provide DRAM corrected error counters. DRAM error counters increment on detection of a corrected ECC error. One error counter per chip select is implemented. Each counter is 16 bits. Each counter stops and latches at a count of 0xFFFF. The individual counters must be written to a non-0xFFFF value by a handler to clear the event.

Each counter can be configured to generate an interrupt when transitioning from 0xFFFF => 0xFFFF. Each counter is capable of generating: no interrupt, an APIC-based interrupt, or [SMI](#). When an SMI is generated, Core::X86::Smm::[LocalSmiStatus\[SmiSrcMca\]](#) is set.

3.7.5 DRAM ECC Error Injection

Types of DRAM ECC error injection supported:

- Explicitly addressed ("coherent") Injection. When enabled by software, corrupts a single physical address with a known error pattern. This is done coherently (e.g., all cached copies are invalidated) so that there is no chance the injection is lost because a more up-to-date cached copy of the address exists. This can corrupt selected or all beats of a cache line. Error injection to the CC6 save is not supported.
- Implicitly addressed ("persistent") Injection. When enabled by software, corrupts all future writes within a given memory controller with a known error pattern until injection is disabled.
- Implicitly addressed ("one-shot") Injection. When enabled by software, corrupts the next write within a given memory controller with a known error pattern.

When performing error injection on multi-node systems, the registers must be programmed on the die and memory controller to which the memory is attached. Both periodic and redirect scrubbing must be disabled during the error injection process. Errors are detected and logged when the system reads a memory line that has been corrupted by a write.

Error injection can be disabled by setting UMC::[MiscCfg\[DisErrInj\]](#). Address-based error injection can also be disabled by using the data fabric register security settings.

When injecting errors, consider the ECC scheme being used, including symbol size. Injecting multi-symbol errors may exceed the limits of the ECC to detect errors; see 3.7.1 [Error Correcting Codes]. A multi-symbol error is composed of two or more single-symbol errors. If the exclusive-or of the syndromes of each single-symbol error (from the Correctable Syndromes table for the selected ECC) results in a syndrome listed in the Correctable Syndromes table, the multi-symbol error should not be used for error injection.

Error injection requires ECC to be enabled.

DDR5 memory writes are performed over 16 beats of 2-Data-bytes per-beat.

Data scrambling, encryption and DQ swizzling is applied after the error injection during writes, and after unscrambling, de-encryption, and de-swizzling on reads.

3.7.5.1 Explicitly Addressed Error Injection

Explicitly addressed error injection allows diagnostic firmware to inject errors into DRAM and test the ECC logic. Explicitly addressed error injection only works on addresses mapped as "WB" memory. The injection process ensures that the error is injected into the most recent copy of the cacheline (i.e., the injection is coherent).

Explicitly addressed error injection is configured by the following registers:

- DF::[DramErrInjAddrLo](#)
- DF::[DramErrInjAddrHi](#)
- UMC::[EccErrInj](#)
- UMC::[EccErrInjCtrl](#)

The steps to enable explicitly addressed error injection are as follows:

- Program the desired error pattern into UMC::[EccErrInj](#) for each beat of data.
- Enable address-based injection in UMC::[EccErrInjCtrl\[EccErrAddrEn\]](#).
- Program DF::[DramErrInjAddrHi](#) with the upper bits of the address to be injected.
- Program DF::[DramErrInjAddrLo](#) with the lower bits of the address to be injected and DF::[DramErrInjAddrLo\[ErrInjEn\]](#) set to 1. The address must be 64-byte aligned.
- The error injection is complete when DF::[DramErrInjAddrLo\[ErrInjDone\]](#) is set.

3.7.5.2 Persistent Error Injection

Persistent error injection allows diagnostic firmware to inject errors into future DRAM writes in a memory controller. To enable persistent error injection:

- Program the desired 16-bit error pattern vector into UMC::[EccErrInj](#) for each beat of data. Select one of either 8 Data words or an ECC words to be corrupted for correctable error injection, and more than one for uncorrectable error type.
- Enable persistent error injection using UMC::[EccErrInjCtrl\[EccErrPersistentEn\]](#).

3.7.5.3 One-shot Error Injection

One-shot error injection allows diagnostic firmware to inject errors into the next DRAM write in a memory controller. To enable persistent error injection:

- Program the desired 16-bit error pattern vector into UMC::[EccErrInj](#) for each beat of data. Select one of either 8 Data words or an ECC words to be corrupted for correctable error injection, and more than one for uncorrectable error type.
- Enable one-shot error injection using UMC::[EccErrInjCtrl\[EccErrOneShotEn\]](#).

3.7.6 Uncorrected ECC Error Retry

On server products, the memory controller supports retry of transactions with an uncorrected ECC error. A programmable number of retries will be attempted. If a retried transaction returns with no error, or with a corrected ECC error, a corrected error is logged with MCA::UMC::MCA_STATUS_UMC[UECC] = 1 to indicate that the error was due to a retried uncorrected ECC error. If all retries return with an uncorrected ECC error, the memory controller treats the error as uncorrectable and takes appropriate action (e.g., creating poison and logging a deferred error).

3.7.6.1 Uncorrected ECC Error Injection

See 3.7.8.3 [CRC and UECC Error Injection].

3.7.7 Address/Command Parity Errors (CAPEs)

On server products, the memory controller supports address/command parity to protect data sent on the DIMM command bus. When enabled, the memory controller calculates parity on DIMM address and sends this during one selected cycle of the command. When a parity error is detected, the DIMM notifies the memory controller and incoming commands are blocked momentarily. The controller ceases sending new commands and waits until the DIMM can receive new commands. The controller is usually configured to read all DIMM Error Log registers and decodes the Error Status to determine which DIMM and sub-channel detected the error. When configured, the memory controller supports retry of the affected transaction(s) up to a programmable limit to recover the transactions.

When a retry is successful, a corrected error is logged. If the retry limit is exceeded, a system fatal error event is created. The controller logs the lowest chip select number of the DIMM reporting the error.

To enable parity when registered DIMMs (RDIMMs) are installed, program UMC::[RecCtrl\[RcvrMode\[1\]\]](#) to 0 and UMC::[BeqCtrl0\[RDimmParEn\]](#) to 1.

To enable retry of parity errors, program UMC::[RecCtrl\[RecEn\[0\]\]](#) to 1. The retry limit can be configured using UMC::[RecCtrl\[MaxParRply\]](#).

3.7.7.1 Address/Command Parity Error Injection

Each UMC allows periodic or one-shot CAPE error injection into only one selected DDR5 sub-channel at a time using either a DCA0 or DPAR signal inversion to signal a parity error. DDR5 commands are sent on multiple cycles of the clock. CAPE error injection is configured to select the cycle number, the sub-channel, and the signal bit. Periodicity may be on a count of memclks or commands. Command types may be masked.

To enable CAPE error injection, program the following:

UMC::[BeqErrInj3\[InjTxCmdChn\]](#) – Encoded channel select. 1=Channel A. 2 = Channel B. Must select only one channel.
UMC::[BeqErrInj3\[InjTxCmdCyc\]](#) – Encoded command cycle. 1 2 4 or 8 selects cycle 1,2,3 or 4 respectively. Must select only one cycle.

UMC::[BeqErrInj2\[InjPeriodMult\]](#) – [Valid range 3:0] Multiplies injection UMC::[BeqErrInj\[InjTxCmdPeriod\]](#) by $2^{\text{InjPeriodMult}}$. Recommended when using UMC::[BeqErrInj\[InjTxCmdByClk\]](#) = 1.

UMC::[BeqErrInj2\[InjTxCmdMsk\]](#) - 11 bit mask to exclude DDR5 command types when using

UMC::[BeqErrInj\[InjTxCmdByClk\]](#) = 0.

UMC::[BeqErrInj\[InjTxCmdEn\]](#) – 1: Enables Parity Error injection mode . 0: Parity error injection mode is disabled.

UMC::[BeqErrInj\[InjTxCmdByClk\]](#) – 1: Enables periodicity by count of Memclk. 0: Enables periodicity by count of commands sent.

UMC::[BeqErrInj\[InjTxCmdParBit\]](#) – 1: Inverts the DPAR signal on address bus. 0: Inverts the DCA0 signal on address bus.

UMC::[BeqErrInj\[InjTxCmdPeriod\]](#) – [255:8 valid range] Injection frequency by clock or command multiplied by $32 * (2^{\text{InjPeriodMult}})$. Setting period too small can cause next error before current recover completes.

Finally, chose one from below list to begin error injection when DIMM traffic is active. Error injection will randomly inject errors on unmasked command types to either DIMM if 2 DIMMs are used on the memory channel.

UMC::[BeqErrInj\[InjErrOneShotEn\]](#) – 1 : Injects one CAPE, then stops. Cleared by hardware. Use exclusively to InjErrEn

UMC::[BeqErrInj\[InjErrEn\]](#) – 1: Persistently injects errors at the frequency specified in InjTxCmdPeriod until cleared by software or a fatal error occurs.

To inject a fatal parity error, set UMC::[RecCtrl\[MaxParRply\]](#) = 1

3.7.8 Read and Write Data CRC Errors

On server products, the memory controller supports cyclic redundancy check (CRC) to protect data read and/or written to DRAM, enabled individually or concurrently.

3.7.8.1 Write Data CRC (WRCRC)

WRCRC is only supported on DIMMs that are ECC-Capable when ECC is enabled. WRCRC and ECC-Disable is supported only on Non-ECC Capable DIMMs. ECC Capable and non-ECC capable DIMMs may not be mixed.

When enabled, the memory controller calculates CRC on write data for all 16-beats on a per-nibble basis and send this to DRAM using two additional data beats. The Dram device calculates CRC on the receive data and compares it to the CRC in the last 2 data beats. If a CRC compare error is detected, the DRAM device(s) notify the memory controller concurrently of simultaneous CRC errors and stores the bad data. The controller is responsible for re-writing the data to correct it.

When configured, the memory controller halts new commands to all DIMMs and sub-channels. The controller retries the affected transaction(s) up to a programmable retry limit to recover the failing transaction(s).

When a retry is successful, a corrected error is logged. If the retry limit is exceeded, a system fatal error event is created.

To enable write data CRC, program UMC::[RecCtrl\[RcvrMode\[2\]\]](#) = 0 and UMC::[BeqCtrl1\[WrcrcEn\]](#) = 1. To enable retry of CRC errors, program UMC::[RecCtrl\[RecEn\[1\]\]](#) = 1. The retry limit can be configured using UMC::[RecCtrl\[MaxCrcRply\]](#). See [DDR5 [RAS](#) Recovery].

To inject a fatal error, set UMC::[RecCtrl\[MaxCrcRply\]](#) = 1

3.7.8.2 Read Data CRC (RDCRC)

When enabled, each DRAM device calculates CRC on all 16-beats of read data on a per-nibble basis and sends this to the controller using two additional data beats. If a CRC error is detected, the memory controller halts new Dram traffic on all DIMMs and sub-channels, waits for commands in progress to complete, then retries the affected transaction(s) up to a programmable retry limit to recover the transactions. If ECC is enabled also, and the error occurs in the first 16 data beats, an ECC error also occurs. The ECC error is highest priority. If the ECC error is correctable, it will be corrected via ECC.

When a RdCRC retry is successful, a corrected error is logged. If the retry limit is exceeded, a system fatal error event is created.

To enable read data CRC, program UMC::[BeqCtrl1\[RdCrcEn\]](#) = 1. To enable retry of CRC errors, program UMC::[RecCtrl\[RecEn\[3\]\]](#) = 1. The retry limit can be configured using and [RecCtrl2\[MaxRdCrcRply\]](#). See [DDR5 [RAS](#) Recovery].

To inject a fatal error with Rdrcrc, manually clear UMC::[RecCtrl\[RecEn\[3\]\]](#) before injecting the error.

3.7.8.3 CRC and UECC Error Injection

Each UMC allows periodic or one-shot Data path error injection into only one selected DDR5 sub-channel at a time using a pattern chosen by user to choose which symbols corrupted on the read data buffer. Periodicity is on a count of data beats or Single-shot. To Data Path error injection, program the following:

UMC::[BeqErrInj3\[InjTxCmdChn\]](#) – Encoded channel select. 1=Channel A. 2 = Channel B. Must select only one channel.
UMC::[BeqErrInj2\[InjPeriodMult\]](#) – [Valid range 3:0] Multiplies injection

UMC::[BeqErrInj\[InjTxDatPeriod,InjRxDatPeriod\]](#) by $2^{\wedge}\text{InjperiodMult}$. Recommended when using CRC or UECC error injection.

UMC::[BeqErrInj2\[InjTxDatVec\]](#) - 18 bit mask to select nibbles 0-7, and Ecc nibbles 0,1 for inversion on Beat0 of data. TxDatVec bits 15:8 are not used on DDR5. Bits 7:0 are for nibbles 7:0. Bits 17,16 are for Ecc nibbles 1,0. Select at least 1 for CRC Error injection. At least 2 for UECC error injection. UECC Retries must be disabled when injecting errors on 2 or more nibbles doing RdCRC errors. RdCrc must be disabled to inject UECC errors or only inject errors on the CRC nibbles

UMC::[BeqErrInj\[InjTxDatEn\]](#) – 1: Enables Error injection on Write data path for Wrcrc testing. 0: WRCRC error injection mode is disabled.

UMC::[BeqErrInj\[InjRxDatEn\]](#) – 1: Enables Error injection on the Read data buffer for either RdCRC or UECC testing.

UMC::[BeqErrInj\[InjTxDatPeriod\]](#) – [255:8 valid range] Injection frequency for WRCRC errors data beat multiplied by $32 * (2^{\wedge}\text{InjPeriodMult})$. Setting period too small can cause next error before current recover completes.

UMC::[BeqErrInj\[InjRxDatPeriod\]](#) – [255:8 valid range] Injection frequency for read data errors by data beat multiplied by $32 * (2^{\wedge}\text{InjPeriodMult})$. Setting period too small can cause next error before current recover completes.

Finally, chose one from below list to begin error injection when DIMM traffic is active. Error injection will randomly inject errors on read or write data beats to either DIMM if 2 DIMMs are used on the memory channel.

UMC::[BeqErrInj\[InjErrOneShotEn\]](#) – 1 : Injects one data error, then stops. Cleared by hardware. Use exclusively with InjErrEn.

UMC::[BeqErrInj\[InjErrEn\]](#) – 1: Persistently injects errors at the frequency specified in InjRxDatPeriod or InjTxDatPeriod until cleared by software or a fatal error occurs.

3.7.9 DDR5 Error Check and Scrub (ECS)

See 12.2.20 [[DDR5 DRAM On-die ECC Error Check and Scrub \(ECS\)](#)].

3.7.10 Post Package Repair

The processor supports post-package repair when using DDR5 DRAMs. This feature allows spare DRAM rows to replace malfunctioning rows via an in-field repair mechanism. Policy decisions on repairs are the responsibility of platform firmware. Refer to the [AGESA](#) specification for details on the platform firmware interface.

3.8 PCI Express® RAS Features

3.8.1 PCI Express® Advanced Error Reporting (AER)

Each of the [PCIe](#) root ports support AER functionality. This applies to both the physical root ports used to connect to external devices, as well as the internal root ports used to connect internal controllers.

The physical root ports do not support the following optional AER features:

- Receiver Buffer Overflow
- Flow Control Protocol Error
- Surprise Down

The internal root ports do not support the following optional AER features:

- Poisoned TLP Egress Blocking
- Receiver Buffer Overflow
- Flow Control Protocol Error
- Surprise Down

The processor additionally supports reporting PCIe errors via standard MCA registers. If MCA reporting is desired, the steps in the Enablement and Severity sections should be followed, the steps in the Actions and Status sections should be omitted, and the corresponding error in the MCA_CTL_MASK register should be unmasked.

For poisoned TLP error injection:

RX:

1. Write 1 to PCIE_ERR_CNTL.RX_GENERATE_POIS_TLP[6]
2. This will simulate receiving a TLP with the poisoned bit (EP bit) set to 1 on the next incoming TLP that has data-payload.

TX:

1. Write 1 to PCIE_TX_ERR_CTRL.TX_GENERATE_POIS_TLP[2]
2. This will force the TX to set the the poisoned bit (EP bit) 1 on the next out-bound TLP that has data-payload.

3.8.1.1 Recommended PCI Express® AER Severity

It is recommended that SBIOS set the initial [PCIe](#) AER severity throughout the PCIe topology as per the following tables. These may be subsequently modified by the operating system. The processor root ports and internal endpoints do not support reporting all of the error types listed below, but those errors may still be reported by platform level switches and endpoints.

Table 139: AER Severity for Root Complex and Switch Downstream Ports

AER Feature	Recommended Severity	Mask
Data Link Protocol Error	Fatal	0
Surprise Down Error	Fatal	0
Poisoned TLP Received	Advisory Non-Fatal	0
Flow Control Protocol Error	Fatal	0
Completion Timeout Error	Fatal	0
Completer Abort	Advisory Non-Fatal	0
Unexpected Completion Error	Fatal	0
Receiver Overflow Status	Fatal	0
Malformed TLP	Fatal	0
ECRC Error	Fatal	0
Unsupported Request Error	Advisory Non-Fatal	0
ACS Violation	Fatal	0
Uncorrectable Internal Error	Fatal	0
MC Blocked TLP	Advisory Non-Fatal	0
AtomicOp Egress Blocked	Fatal	0
TLP Prefix Blocked Error	Fatal	0
Poisoned TLP Egress Blocked	Fatal	0
Receiver Error	Correctable	0
Bad TLP	Correctable	0
Bad DLLP	Correctable	0
REPLAY_NUM Rollover	Correctable	0
Replay Timer Timeout	Correctable	0
Advisory Non-Fatal Error	Correctable	0
Corrected Internal Error	Correctable	0
Header Log Overflow	Correctable	0

Table 140: AER Severity for Endpoint Devices and Switch Upstream Ports

AER Feature	Recommended Severity	Mask
Data Link Protocol Error	Fatal	0
Surprise Down Error	Fatal	0
Poisoned TLP Received	Advisory Non-Fatal	0
Flow Control Protocol Error	Fatal	0
Completion Timeout Error	Fatal	0
Completer Abort	Advisory Non-Fatal	0
Unexpected Completion Error	Fatal	0
Receiver Overflow Status	Fatal	0
Malformed TLP	Fatal	0
ECRC Error	Fatal	0
Unsupported Request Error	Advisory Non-Fatal	1
ACS Violation	Fatal	0
MC Blocked TLP	Fatal	0
Uncorrectable Internal Error	Fatal	0
AtomicOp Egress Blocked	Fatal	0
TLP Prefix Blocked Error	Fatal	0
Poisoned TLP Egress Blocked	Fatal	0
Receiver Error	Correctable	0
Bad TLP	Correctable	0
Bad DLLP	Correctable	0
REPLAY_NUM Rollover	Correctable	0
Replay Timer Timeout	Correctable	0
Advisory Non-Fatal Error	Correctable	0
Corrected Internal Error	Correctable	0
Header Log Overflow	Correctable	0

3.8.2 PCIe® Error Actions

Errors may be reported to an external management controller using the [APML](#) interface by setting APMI_ERR_En. Upon notification of the error, the management controller may need to warm reset the processor to successfully extract error status. [SMI](#), SCI or NMI interrupts may also be generated to the CPU using the IntrGenSel field.

For [PCIe](#) AER errors, standard controls within each PCIe root port PCIe capability and AER capability are used to enable interrupts to the Hypervisor or Operating System. These interrupts are generated in parallel to those enabled through the IntrGenSel field of the PCIE*_ACTION_CONTROL registers. Coordination between multiple software entities may be needed to avoid configuring multiple interrupts for the same event. There are separate action control registers for errors detected by the root port itself and errors detected by external devices and reported through ERR_FATAL/ERR_NONFATAL/ERR_CORR messages over the PCIe link.

The PCIE*_ACTION_CONTROL registers work as follows:

PCIE*_Int*_ACTION_CONTROL registers are for legacy PCIe error reporting, which are not necessarily logged with more detail in AER registers. When an error is logged in a legacy PCIe error register, it directly triggers the action specified in the PCIE*_Int*_ACTION_CONTROL registers.

PCIE*_Ext*_ACTION_CONTROL registers are for AER error reporting. If AER is enabled, an error first is logged in a

legacy PCIe error registers (which will trigger the PCIe*Int*_ACTION_CONTROL registers) and then the error will be logged by the root port, if the error is unmasked. This last part triggers the action specified in the PCIe*Ext*_ACTION_CONTROL registers.

If AER is enabled, an error may trigger duplicate actions from both PCIe*Int*_ACTION_CONTROL and PCIe*Ext*_ACTION_CONTROL if overlap exists. To avoid duplicate actions, the following programming is recommended:

```
If (AER enabled) {
    Set PCIe*Port*ExtCorr_ACTION_CONTROL
    Set PCIe*Port*ExtFatal_ACTION_CONTROL
    Set PCIe*Port*ExtNonFatal_ACTION_CONTROL
    Clear PCIe*Port*IntFatal_ACTION_CONTROL
    Clear PCIe*Port*IntFatal_ACTION_CONTROL
    Clear PCIe*Port*IntNonFatal_ACTION_CONTROL}
Else {
    Clear PCIe*Port*ExtCorr_ACTION_CONTROL
    Clear PCIe*Port*ExtFatal_ACTION_CONTROL
    Clear PCIe*Port*ExtNonFatal_ACTION_CONTROL
    Set PCIe*Port*IntCorr_ACTION_CONTROL
    Set PCIe*Port*IntFatal_ACTION_CONTROL
    Set PCIe*Port*IntNonFatal_ACTION_CONTROL}
```

The PCIe<0,1><Port>PARITYERR_ACTION_CONTROL registers trigger actions when the corresponding PCIe root port sets its PARITY_ERROR_DETECTED status bit located in its primary side STATUS register. This status corresponds to the transmission of poisoned TLPs onto the PCIe link. Received poison TLPs are logged in the standard AER poisoned TLP received register. These trigger either the fatal or correctable PCIe internal action for the receiving port depending on the severity level set in the standard AER error severity register.

The processor may be halted to contain potentially corrupted data by setting the LinkDis_En and SyncFlood_En fields. If this is done, interrupt based error reporting may not be used as the processor is halted and unable to service the interrupt. APLM-based error reporting may still be used.

3.8.3 PCIe® Error Status

[PCIe](#) errors are logged in standard AER status registers.

3.8.4 PCI Express® eDPC

eDPC is supported on the externally facing [PCIe](#) root ports used to connect external PCIe devices.

3.8.5 PCI Express® ECRC

The processor supports PCI Express® ECRC generation and checking on all [PCIe](#) root ports. ECRC is not preserved for peer-to-peer requests that cross through the processor. In general, ECRC generation capabilities should only be enabled in root ports and endpoints if there is a switch between them.

Note that it is legal for the processor to send requests with ECRC to endpoints that do not support ECRC. This may occur if a root port connects to a switch with multiple downstream ports where some ports connect to endpoint devices supporting ECRC and others connect to endpoints that do not support ECRC. Endpoints that do not support ECRC should ignore the ECRC information.

The processor implements a standard PCI Express ECRC control as part of AER located at offset 0x168 within each PCI

Express bridge. Please refer to the PCI Express Base Specification section 7.10.7 for more details on the registers used to control ECRC.

The following algorithm may be used to determine when and where to enable ECRC. System bios should enable ECRC before handing off control to the OS.

For each PCIE root port, check the device immediately behind the bridge.

If this is an endpoint device, do not enable ECRC on the root port and go to the next root port

Else if this is a PCI Express bridge and there are additional PCI Express bridges located on the bus behind it, the root port is connected to a PCI Express switch (as opposed to a different type of bridge such as a PCI Express to PCI-X bridge).

Search all PCI Express endpoint devices below the switch (which may be below additional switches).

If any endpoint device contains an AER capability in the extended capability list (type 0x1) and sets the ECRC Check Capable bit in Advanced Error Capabilities and Control Register (offset 0x18 bit[8] inside the device's AER capability) and the root port has ECRC_GEN_CAP = 0x1.

Set ECRC Check Enable in the endpoint device (offset 0x18 bit[7] inside the device's AER capability).

Set ECRC Generation Enable in the root port (root port offset 0x168 bit[6]).

If any endpoint device contains an AER capability in the extended capability list (type 0x1) and sets the ECRC Generation Capable bit in Advanced Error Capabilities and Control Register (offset 0x18 bit[5] inside the AER cap) and the root port has ECRC_CHECK_CAP = 0x1.

Set ECRC Check Enable in the root port (root port offset 0x168 bit[8]).

Set ECRC Generation Enable in the endpoint device (offset 0x18 bit[6] inside the device's AER capability).

3.8.6 PCI Express® Error Injection

The [PCIe](#) controller is capable of injecting LCRC and ECRC errors in both the transmit and receive directions of each PCI Express port by writing to the bits in the NBIO_PCIE_ERR_CNTL register of the appropriate port.

When injecting LCRC errors, a TLP packet must be sent across the PCI Express link in the direction of the error injection. For example, a configuration register Read of the device on the far side of the link triggers TLP packets in both directions and cause an injected LCRC error to occur. Errors cannot be injected across disabled links.

When injecting ECRC errors, ECRC must be enabled on both sides of the PCI Express link in the direction of the error injection. Additionally, a TLP packet must be sent across the link in the direction of the error injection. Errors cannot be injected across disabled links.

Before initiating PCIe error injection, LCLK Clock gating feature and ASPM within PCIe must be disabled:

PCIEPORT::[PCIE_TX_ERR_CTRL\[TX_GENERATE_ECRC_ERR\]](#) = 1

PCIEPORT::[PCIE_TX_ERR_CTRL\[TX_GENERATE_LCRC_ERR\]](#) = 1

3.9 NBIO RAS Features

The section describes the programming sequences needed to enable and configure various [RAS](#) related features in the NBIO. By default, all RAS features are disabled after reset. Several different methods of reporting and handling errors within the system are supported and it is up to the platform designer to choose the method that is appropriate for them. Specifically, the processor supports error detection for the following general classes of errors:

- Internal SRAM ECC and Parity Errors
- Datapath Parity Errors
- System Fatal Error Event
- Data Poisoning
- ErrEvent/NMI# pin

The following modes of error handling are supported:

- Error handling through System BIOS/Firmware.
- System halt on error via ErrEvent
- PCI Express link disable
- Error handling through Operating System/Hypervisor ([PCIe](#) errors only)
- Error handling through MCA

It is possible to configure a system to use different error handling methods for different classes of errors.

3.9.1 SRAM ECC and Parity

Common control logic is used to support SRAM ECC, SRAM parity, and datapath data parity (DDP) errors within NBIO. All SRAM arrays inside the NBIO use either ECC or parity protection.

Each protected structure is assigned a group ID based on its associated functional block, and structure ID.

Parity protected SRAM arrays may generate either correctable or uncorrectable errors depending on the specific structure. For example, [IOMMU](#) TLB arrays are parity protected but correct errors by reloading data from DRAM.

ECC protected SRAM arrays may report both correctable and uncorrectable errors. Selected ECC protected arrays support the conversion of uncorrectable errors into poisoned data. Datapath data parity errors are normally reported as uncorrectable but these errors may also be converted into poisoned data. NBIO reports a data error converted to poisoned data as a Uncorrectable Conversion to Poisoned data (UCP) error.

Error severity is controlled on a per-group basis. All internal errors of the same severity map to the same error actions.

Error reporting in each structure is disabled by default. System BIOS software must enable error reporting within each structure individually. The IOHC::[PARITY CONTROL 1](#) register is used to enable error reporting, correction and conversion to poisoned data where applicable.

The processor supports reporting SRAM ECC and Parity errors via standard MCA registers. If MCA reporting is desired, the steps in the Enablement and Severity sections should be followed, the steps in the Actions, Status, and Thresholding sections should be omitted, and the corresponding error in the MCA_CTL_MASK register should be unmasked.

3.9.1.1 SRAM ECC and Parity Enablement

For each row in the following table, execute this sequence using the Group ID and Structure ID from the table row. This enables correctable parity structures within NBIO.

Write IOHC::[PARITY CONTROL 1](#) = 0x0

Poll IOHC::[PARITY CONTROL 1](#)[ParityErrGenInjectAllow] until it reads back 0x1

Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenGroupTypeSel] = 0x1 (CORR)

Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenGroupSel] = Group ID

Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenIdSel] = Structure ID

Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenCmd] = 0x5 - enable error reporting

Write IOHC::[PARITY CONTROL 1](#)[[ParityErrGenTrigger](#)] = 0x1

Table 141: NBIO Correctable Parity Structures

Group ID	Structure ID	IP (NBIO Left/ Right)
19	0	L1IMUPCIE0 (L1PCIE3)
19	1	L1IMUPCIE0 (L1PCIE3)
20	0	L1IMUPCIE1 (L1PCIE2)
20	1	L1IMUPCIE1 (L1PCIE2)
21	0	L1IMUIOAGR
21	1	L1IMUIOAGR
22	0	L2IMU
22	1	L2IMU
22	2	L2IMU
22	3	L2IMU
22	4	L2IMU
22	5	L2IMU
22	6	L2IMU
22	7	L2IMU
22	8	L2IMU
22	9	L2IMU
22	10	L2IMU
22	11	L2IMU
22	12	L2IMU
22	13	L2IMU
22	14	L2IMU
22	15	L2IMU
22	16	L2IMU
22	17	L2IMU
22	18	L2IMU
22	19	L2IMU
23	0	L2IMU
23	1	L2IMU
23	2	L2IMU
23	3	L2IMU
23	4	L2IMU
23	5	L2IMU
23	6	L2IMU
23	7	L2IMU

For each row in the following table, execute this sequence using the Group ID and Structure ID from the table row. This enables uncorrectable parity structures within NBIO.

Write IOHC::[PARITY CONTROL 1](#) = 0x0

Poll IOHC::[PARITY CONTROL 1](#)[[ParityErrGenInjectAllow](#)] until it reads back 0x1

Write IOHC::[PARITY CONTROL 1](#)[[ParityErrGenGroupTypeSel](#)] = 0x0 (UNCORR)

Write IOHC::[PARITY CONTROL 1](#)[[ParityErrGenGroupSel](#)] = Group ID

Write IOHC::[PARITY CONTROL 1](#)[[ParityErrGenIdSel](#)] = Structure ID

Write IOHC::[PARITY CONTROL 1](#)[[ParityErrGenCmd](#)] = 0x5 - enable error reporting

Write IOHC::[PARITY CONTROL 1](#)[[ParityErrGenTrigger](#)] = 0x1

Table 142: NBIO Uncorrectable Parity Structures

Group ID	Structure ID	IP
19	0	PCIE1/ PCIE2
19	1	PCIE1/ PCIE2
19	2	PCIE1/ PCIE2
19	3	PCIE1/ PCIE2
19	4	PCIE1/ PCIE2
19	5	PCIE1/ PCIE2
19	6	PCIE1/ PCIE2
19	7	PCIE1/ PCIE2
19	8	PCIE1/ PCIE2
19	9	PCIE1/ PCIE2
19	10	PCIE1/ PCIE2
19	11	PCIE1/ PCIE2
19	12	PCIE1/ PCIE2
19	13	PCIE1/ PCIE2
20	0	PCIE1/ PCIE2
21	0	PCIE0/ PCIE3
21	1	PCIE0/ PCIE3
21	2	PCIE0/ PCIE3
21	3	PCIE0/ PCIE3
21	4	PCIE0/ PCIE3
21	5	PCIE0/ PCIE3
21	6	PCIE0/ PCIE3
21	7	PCIE0/ PCIE3
21	8	PCIE0/ PCIE3
21	9	PCIE0/ PCIE3
21	10	PCIE0/ PCIE3
21	11	PCIE0/ PCIE3
21	12	PCIE0/ PCIE3
21	13	PCIE0/ PCIE3
22	0	PCIE0/ PCIE3
23	0	PCIE4/ NA
23	1	PCIE4/ NA
23	2	PCIE4/ NA
23	3	PCIE4/ NA
23	4	PCIE4/ NA
23	5	PCIE4/ NA
23	6	PCIE4/ NA
23	7	PCIE4/ NA
24	0	PCIE4/ NA

For each row in the following table, execute this sequence using the Group ID and Structure ID from the table row. This enables ECC structures within NBIO, and turns on SEC and DED error reporting.

Write IOHC::[PARITY_CONTROL_1](#) = 0x0

Poll IOHC::[PARITY_CONTROL_1](#)[[ParityErrGenInjectAllow](#)] until it reads back 0x1

Write IOHC::[PARITY_CONTROL_1](#)[[ParityErrGenGroupTypeSel](#)] = 0x1 (CORR)

Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenGroupSel] = Group ID
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenIdSel] = Structure ID
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenCmd] = 0x1 - enable ECC correction
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenTrigger] = 0x1
 Poll IOHC::[PARITY CONTROL 1](#)[ParityErrGenInjectAllow] until it reads back 0x1
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenCmd] = 0x4 - enable SEC reporting
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenTrigger] = 0x1
 Poll IOHC::[PARITY CONTROL 1](#)[ParityErrGenInjectAllow] until it reads back 0x1
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenCmd] = 0x7 - enable DED reporting
 Write IOHC::[PARITY CONTROL 1](#)[ParityErrGenTrigger] = 0x1

Table 143: NBIO ECC Structures

Group ID	Structure ID	IP (NBIO Left/ Right)
0	0	PCIE1/ PCIE2
0	1	PCIE1/ PCIE2
0	2	PCIE1/ PCIE2
0	3	PCIE1/ PCIE2
0	4	PCIE1/ PCIE2
0	5	PCIE1/ PCIE2
0	6	PCIE1/ PCIE2
0	7	PCIE1/ PCIE2
0	8	PCIE1/ PCIE2
0	9	PCIE1/ PCIE2
0	10	PCIE1/ PCIE2
0	11	PCIE1/ PCIE2
0	12	PCIE1/ PCIE2
0	13	PCIE1/ PCIE2
0	14	PCIE1/ PCIE2
0	15	PCIE1/ PCIE2
0	16	PCIE1/ PCIE2
0	17	PCIE1/ PCIE2
0	18	PCIE1/ PCIE2
0	19	PCIE1/ PCIE2
0	20	PCIE1/ PCIE2
1	0	PCIE1/ PCIE2
1	1	PCIE1/ PCIE2
1	2	PCIE1/ PCIE2
1	3	PCIE1/ PCIE2
1	4	PCIE1/ PCIE2
1	5	PCIE1/ PCIE2
1	6	PCIE1/ PCIE2
1	7	PCIE1/ PCIE2
1	8	PCIE1/ PCIE2
1	9	PCIE1/ PCIE2
1	10	PCIE1/ PCIE2
1	11	PCIE1/ PCIE2
1	12	PCIE1/ PCIE2
1	13	PCIE1/ PCIE2
1	14	PCIE1/ PCIE2

1	15	PCIE1/ PCIE2
1	16	PCIE1/ PCIE2
1	17	PCIE1/ PCIE2
1	18	PCIE1/ PCIE2
1	19	PCIE1/ PCIE2
1	20	PCIE1/ PCIE2
1	21	PCIE1/ PCIE2
2	0	PCIE0/ PCIE3
2	1	PCIE0/ PCIE3
2	2	PCIE0/ PCIE3
2	3	PCIE0/ PCIE3
2	4	PCIE0/ PCIE3
2	5	PCIE0/ PCIE3
2	6	PCIE0/ PCIE3
2	7	PCIE0/ PCIE3
2	8	PCIE0/ PCIE3
2	9	PCIE0/ PCIE3
2	10	PCIE0/ PCIE3
2	11	PCIE0/ PCIE3
2	12	PCIE0/ PCIE3
2	13	PCIE0/ PCIE3
2	14	PCIE0/ PCIE3
2	15	PCIE0/ PCIE3
2	16	PCIE0/ PCIE3
2	17	PCIE0/ PCIE3
2	18	PCIE0/ PCIE3
2	19	PCIE0/ PCIE3
2	20	PCIE0/ PCIE3
3	0	PCIE0/ PCIE3
3	1	PCIE0/ PCIE3
3	2	PCIE0/ PCIE3
3	3	PCIE0/ PCIE3
3	4	PCIE0/ PCIE3
3	5	PCIE0/ PCIE3
3	6	PCIE0/ PCIE3
3	7	PCIE0/ PCIE3
3	8	PCIE0/ PCIE3
3	9	PCIE0/ PCIE3
3	10	PCIE0/ PCIE3
3	11	PCIE0/ PCIE3
3	12	PCIE0/ PCIE3
3	13	PCIE0/ PCIE3
3	14	PCIE0/ PCIE3
3	15	PCIE0/ PCIE3
3	16	PCIE0/ PCIE3
3	17	PCIE0/ PCIE3
3	18	PCIE0/ PCIE3
3	19	PCIE0/ PCIE3

3	20	PCIE0/ PCIE3
3	21	PCIE0/ PCIE3
4	0	PCIE4/ NA
4	1	PCIE4/ NA
4	2	PCIE4/ NA
4	3	PCIE4/ NA
4	4	PCIE4/ NA
4	5	PCIE4/ NA
4	6	PCIE4/ NA
4	7	PCIE4/ NA
4	8	PCIE4/ NA
4	9	PCIE4/ NA
4	10	PCIE4/ NA
4	11	PCIE4/ NA
4	12	PCIE4/ NA
4	13	PCIE4/ NA
4	14	PCIE4/ NA
5	0	PCIE4/ NA
5	1	PCIE4/ NA
5	2	PCIE4/ NA
5	3	PCIE4/ NA
5	4	PCIE4/ NA
5	5	PCIE4/ NA
5	6	PCIE4/ NA
5	7	PCIE4/ NA
5	8	PCIE4/ NA
5	9	PCIE4/ NA
5	10	PCIE4/ NA
5	11	PCIE4/ NA
6	0	L1PCIE1/ L1PCIE3
6	1	L1PCIE0/ L1PCIE3
6	2	L1PCIE0/ L1PCIE3
6	3	L1PCIE0/ L1PCIE3
6	4	L1PCIE0/ L1PCIE3
7	0	L1PCIE1/ L1PCIE2
7	1	L1PCIE1/ L1PCIE2
7	2	L1PCIE1/ L1PCIE2
7	3	L1PCIE1/ L1PCIE2
7	4	L1PCIE1/ L1PCIE2
8	0	L1IOAGR
8	1	L1IOAGR
8	2	L1IOAGR
8	3	L1IOAGR
8	4	L1IOAGR
9	0	L2IMU
9	1	L2IMU
9	2	L2IMU
9	4	L2IMU

9	5	L2IMU
9	6	L2IMU
9	7	L2IMU
9	8	L2IMU
9	9	L2IMU
9	10	L2IMU
9	11	L2IMU
10	0	IOHC
10	1	IOHC
10	5	IOHC
10	6	IOHC
10	7	IOHC
10	8	IOHC
10	9	IOHC
10	10	IOHC
10	11	IOHC
10	12	IOHC
11	4	NBIF0/ NBIF1
11	5	NBIF0/ NBIF1
12	0	NBIF0/ NBIF1
12	1	NBIF0/ NBIF1
12	2	NBIF0/ NBIF1
15	5	NBIF2/ NA
15	6	NBIF2/ NA
16	0	PCIE1/ PCIE2
16	1	PCIE1/ PCIE2
16	2	PCIE1/ PCIE2
16	3	PCIE1/ PCIE2
16	4	PCIE1/ PCIE2
16	5	PCIE1/ PCIE2
16	6	PCIE1/ PCIE2
16	7	PCIE1/ PCIE2
16	8	PCIE1/ PCIE2
17	0	PCIE0/ PCIE3
17	1	PCIE0/ PCIE3
17	2	PCIE0/ PCIE3
17	3	PCIE0/ PCIE3
17	4	PCIE0/ PCIE3
17	5	PCIE0/ PCIE3
17	6	PCIE0/ PCIE3
17	7	PCIE0/ PCIE3
17	8	PCIE0/ PCIE3
18	0	PCIE4/ NA
18	1	PCIE4/ NA
18	2	PCIE4/ NA
18	3	PCIE4/ NA

For each row in the following table, execute this sequence using the Group ID and Structure ID from the table row. This

enables datapath data parity checks with error conversion to poisoned data within NBIO.

Write IOHC::[PARITY_CONTROL_1](#) = 0x0

Poll IOHC::[PARITY_CONTROL_1\[ParityErrGenInjectAllow\]](#) until it reads back 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenGroupSel\]](#) = Group ID

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenGroupTypeSel\]](#) = 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenIdSel\]](#) = Structure ID

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x9 - enable UCP conversion

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenTrigger\]](#) = 0x1

Poll IOHC::[PARITY_CONTROL_1\[ParityErrGenInjectAllow\]](#) until it reads back 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x4 - enable SEC reporting

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenTrigger\]](#) = 0x1

Poll IOHC::[PARITY_CONTROL_1\[ParityErrGenInjectAllow\]](#) until it reads back 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0xC - enable UCP reporting

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenTrigger\]](#) = 0x1

Alternatively, to disable error conversion to poison data (UCP) and instead report uncorrectable errors, use the following sequence.

Write IOHC::[PARITY_CONTROL_1](#) = 0x0

Poll IOHC::[PARITY_CONTROL_1\[ParityErrGenInjectAllow\]](#) until it reads back 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenGroupSel\]](#) = Group ID

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenGroupTypeSel\]](#) = 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenIdSel\]](#) = Structure ID

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x4 - enable SEC reporting

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenTrigger\]](#) = 0x1

Poll IOHC::[PARITY_CONTROL_1\[ParityErrGenInjectAllow\]](#) until it reads back 0x1

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x7 - enable DED reporting

Write IOHC::[PARITY_CONTROL_1\[ParityErrGenTrigger\]](#) = 0x1

Table 144: NBIO DDP+UCP Structures

Group ID	Structure ID	IP (NBIO Left/ Right)
6	5	L1PCIE0/ L1PCIE3
7	5	L1PCIE1/ L1PCIE2
8	5	L1IOAGR
9	3	L2IMU
10	2	IOHC
10	3	IOHC
10	4	IOHC
11	0	NBIF0/ NBIF1
11	1	NBIF0/ NBIF1
11	2	NBIF0/ NBIF1
11	3	NBIF0/ NBIF1
12	3	NBIF0/ NBIF1
12	4	NBIF0/ NBIF1
12	5	NBIF0/ NBIF1
12	6	NBIF0/ NA
13	0	NBIF0/ NBIF1
13	1	NBIF0/ NBIF1
13	2	NBIF0/ NBIF1
13	3	NBIF0/ NA

14	0	NBIF0/ NA
14	1	NBIF0/ NA
14	2	NBIF0/ NA
15	0	NBIF2/ NA
15	1	NBIF2/ NA
15	2	NBIF2/ NA
15	3	NBIF2/ NA
15	4	NBIF2/ NA

3.9.1.2 SRAM ECC and Parity Error Severity

Error severity for internal errors may be set to 1 of 4 levels on a per-group basis. The recommended severity configuration is shown below.

Set correctable parity and ECC errors to correctable severity:

IOHC::[PARITY_SEVERITY_CONTROL_CORR_0](#)=0x0

IOHC::[PARITY_SEVERITY_CONTROL_CORR_1](#)=0x0

Set uncorrectable parity and ECC errors to fatal severity:

IOHC::[PARITY_SEVERITY_CONTROL_UNCORR_0](#)=0xAAAA_AAAA

IOHC::[PARITY_SEVERITY_CONTROL_UNCORR_1](#)=0xAAAA_AAAA

Set uncorrectable data errors converted to poisoned data to correctable severity. This is used for notification only and is separate from consuming poisoned data:

IOHC::[PARITY_SEVERITY_CONTROL_UCP_0](#)=0x0

IOHC::[PARITY_SEVERITY_CONTROL_UCP_1](#)=0x0

Set ErrEvent to fatal severity:

IOHC::[MISC_SEVERITY_CONTROL](#)[[ErrEventErrSev](#)]=0x2

3.9.1.3 SRAM ECC and Parity Error Actions

NBIO can be programmed to take certain actions when it logs an error of a given severity. Errors may be reported to an external management controller using the [APML](#) interface by setting APML_ERR_En. Upon notification of the error, the management controller may need to warm reset the processor to successfully extract error status. [SMI](#), SCI or NMI interrupts may also be generated to the CPU using the IntrGenSel field. The recommended configuration is shown below.

3.9.1.4 SRAM ECC and Parity Error Status

SRAM ECC and parity errors are logged in standard MCA registers.

Internal parity, ECC, DDP and UCP errors are logged in PARITY_ERROR_STATUS_* registers. Errors are logged into status registers based on structure prior to severity classification. For each structure enabled in 3.9.1.1 [SRAM ECC and Parity Enablement] the corresponding error status location is:

PARITY_ERROR_STATUS_<TypeSel>_GRP<Group ID>[<Structure ID>]

The <Group ID> and <Structure ID> correspond to the values in each row of the tables in the previously referenced section. The <TypeSel> field is determined by the structure type and error type as described in the following table.

Table 145: Error Status TypeSel Selection

Structure Type	Error Type	<TypeSel> value	Notes
Correctable Parity	Correctable error detected and corrected	CORR	
Uncorrectable Parity	Uncorrectable error detected	UNCORR	
ECC	Correctable error detected and corrected	CORR	
ECC	Uncorrectable error detected	UNCORR	
ECC+UCP	Correctable error detected and corrected	CORR	
ECC+UCP	Uncorrectable error detected and converted to poisoned data	UCP	Data was poisoned
DDP+UCP	Uncorrectable error detected and converted to poisoned data	UCP	Data was poisoned

3.9.1.5 SRAM ECC and Parity Correctable Error Thresholding

Error thresholding can be performed through standard MCA mechanisms.

Error thresholding may be enabled with correctable parity groups, ECC groups and UCP groups. The feature allows low-level error rates to be recorded without triggering an action such as a notification to software. Only error rates that exceed the threshold set correctable error status and trigger [RAS](#) actions.

One threshold is provided for correctable parity and correctable ECC errors. This is programmed via IOHC::[PARITY_CONTROL_0\[ParityCorrThreshold\]](#). A separate threshold is provided for UCP errors. This is programmed via IOHC::[PARITY_CONTROL_0\[ParityUCPThreshold\]](#).

Error counters are maintained for each internal error group which supports thresholding. Software is responsible for periodically polling and clearing the counters in IOHC::[PARITY_COUNTER_CORR_GRP](#) and IOHC::[PARITY_COUNTER_UCP_GRP](#). The software polling rate and the threshold level determine the rate at which errors may accumulate without actions being taken which includes notifying software.

3.9.1.6 SRAM ECC and Parity Error Injection

Software may artificially inject internal parity errors into the system in order to test error handling capabilities. For parity protected arrays, the NBIO can inject parity errors into either the read or write side of the array. Write-side injection may not result in a detected error if the memory location is never read or is overwritten before being read, which may occur in structures such as caches and replay buffers. For both ECC and parity protected arrays, real data is not corrupted. Additionally, the target array must be enabled for error reporting.

The procedure to inject an error is similar to the procedure used to enable internal error reporting as described in 3.9.1.1 [SRAM ECC and Parity Enablement]. For parity protected arrays, instead of setting IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x5, software can set IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x1 or IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x2 for Write or Read side injection. For all other array types, software can set IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x2 to inject a correctable ECC error, IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0x5 to inject an uncorrectable ECC or DDP error and IOHC::[PARITY_CONTROL_1\[ParityErrGenCmd\]](#) = 0xA to inject a UCP error.

3.9.2 System Fatal Error Event

The processor supports reporting System Fatal Error Event errors via standard MCA registers. If MCA reporting is desired, the steps in the Action section should be followed, but Status should be ignored and the corresponding error in the MCA_CTL_MASK register should be unmasked.

3.9.2.1 System Fatal Error Event Severity

No severity programming is provided for System Fatal Error Events.

3.9.2.2 System Fatal Error Event Actions

Set syncflood errors to trigger [PCIe](#) link disable:
IOHC::[ErrEvent_ACTION_CONTROL](#) = 0x0000_0018

3.9.2.3 System Fatal Error Event Status

If a system fatal error event occurs, status information may be recovered after a warm reset. The IOHC::[SYNCFLOOD_STATUS](#) register contains information about the source of the system fatal error event relative to the [IOHC](#). The IOHC port numbers correspond to the IOHC port numbering listed in the following table.

Table 146: IOHC Port Numbering and System Fatal Error Event Status

IP Connected to IOHC Port	IOHC Port Number	System fatal error event Status
Data Fabric (IOM)	0	IOHC:: SYNCFLOOD_STATUS[SyncfloodFromIOHCPortN_8]
Data Fabric (NCM)	1	IOHC:: SYNCFLOOD_STATUS[SyncfloodFromIOHCPortN_9]
PCIe0	2	IOHC:: SYNCFLOOD_STATUS[SyncfloodFromIOHCPortN_10]
PCIe1	3	IOHC:: SYNCFLOOD_STATUS[SyncfloodFromIOHCPortN_11]
PCIe2/nBIF1/ FCH /IOAPIC	4	IOHC:: SYNCFLOOD_STATUS[SyncfloodFromIOHCPortN_12]
IOMMU L2	5	IOHC:: SYNCFLOOD_STATUS[SyncfloodFromIOHCPortN_13]

3.9.3 Data Poisoning

The processor supports reporting egress and internal data poison errors via standard MCA registers. If MCA reporting is desired, the steps in the Severity sections should be followed, the steps in the Actions section should be omitted, error status should not be polled, and the corresponding error in the MCA_CTL_MASK register should be cleared.

3.9.3.1 Egress Port Poisoned Data Severity

[IOHC](#) is capable of triggering [RAS](#) actions when propagating poisoned data to out one of its egress ports. Poisoned data going through each port may be classified as high or low severity ([IOHC::EGRESS_POISON_SEVERITY_LO](#), [IOHC::EGRESS_POISON_SEVERITY_HI](#)), or may be masked off ([IOHC::EGRESS_POISON_MASK_LO](#), [IOHC::EGRESS_POISON_MASK_HI](#)) to prevent triggering actions. This may be used if the system knows that a specific [PCIe](#) device cannot properly handle poisoned data. If data is sent out the IOHC logical port towards that device, a sync flood may be triggered to halt the system. The separate severity levels may be used to trigger interrupt notification on some ports and sync flood on others.

It is recommended that the processor be halted using a system fatal error event when poisoned data is sent to the [FCH](#). The FCH is associated with egress port 0x30.

[IOHC::EGRESS_POISON_SEVERITY_HI](#)[16] = 0x1 (FCH)
[IOHC::EGRESS_POISON_SEVERITY_HI](#)[17] = 0x1 (IOAPIC)
[IOHC::EGRESS_POISON_SEVERITY_HI](#)[0] = 0x1 (NBIF0PortA)
[IOHC::EGRESS_POISON_SEVERITY_HI](#)[1] = 0x1 (NBIF0PortB)
[IOHC::EGRESS_POISON_SEVERITY_HI](#)[2] = 0x1 (NBIF0PortC)
[IOHC::EGRESS_POISON_SEVERITY_LO](#)[2] = 1 (IOHC internal)

All other bits in [IOHC::EGRESS_POISON_SEVERITY_HI](#) and [IOHC::EGRESS_POISON_SEVERITY_LO](#) should be programmed to 0x0.

3.9.3.2 Egress Port Poisoned Data Actions

Set high-severity poison propagation to external devices to trigger syncflood and disable [PCIe](#) links:

[IOHC::POISON_ACTION_CONTROL](#)[[EgressPoisonHSAPMLErrEn](#)] = 0x1
[IOHC::POISON_ACTION_CONTROL](#)[[EgressPoisonHSLinkDisEn](#)] = 0x1
[IOHC::POISON_ACTION_CONTROL](#)[[EgressPoisonHSSyncFloodEn](#)] = 0x1

Set low-severity poison propagation to external devices to propagate poison:

[IOHC::POISON_ACTION_CONTROL](#)[[EgressPoisonLSAPMLErrEn](#)] = 0x0
[IOHC::POISON_ACTION_CONTROL](#)[[EgressPoisonLSLinkDisEn](#)] = 0x0
[IOHC::POISON_ACTION_CONTROL](#)[[EgressPoisonLSSyncFloodEn](#)] = 0x0

Set egress poison mask register to ignore low-severity errors:

[IOHC::EGRESS_POISON_MASK_LO](#) = inverse of [IOHC::EGRESS_POISON_SEVERITY_LO](#)
[IOHC::EGRESS_POISON_MASK_HI](#) = inverse of [IOHC::EGRESS_POISON_SEVERITY_HI](#)

3.9.3.3 Egress Port Poisoned Data Status

Poison status is logged in [IOHC::EGRESS_POISON_STATUS_LO](#) and [IOHC::EGRESS_POISON_STATUS_HI](#). When data poisoning is disabled, firmware should check these registers at boot to determine which port poison was sent to.

3.9.3.4 Internal Port Poisoned Data Severity

No severity programming is provided for Internal Port Poisoned Data.

3.9.3.5 Internal Port Poisoned Data Actions

The [IOHC](#) may consume poisoned data when propagating poisoned transactions to targets that do not handle poisoned

data.

If the poisoned transaction is targeting the register bus or an internal IOHC resource, IOHC logs status in IOHC::[INTERNAL_POISON_STATUS](#). The status may be masked to prevent triggering actions using IOHC::[INTERNAL_POISON_MASK](#). The IntPoison fields of IOHC::[POISON_ACTION_CONTROL](#) control the actions of triggered by unmasked status.

Set internal consumption of poison data to trigger a syncflood, disable the [PCIe](#) links, and generate a notification to the [APML](#):

IOHC::[POISON_ACTION_CONTROL](#)[IntPoisonSyncFloodEn] = 0x1

IOHC::[POISON_ACTION_CONTROL](#)[IntPoisonLinkDisEn] = 0x1

IOHC::[POISON_ACTION_CONTROL](#)[IntPoisonAPMLErrEn] = 0x1

3.9.3.6 Internal Port Poisoned Data Status

Poison status is logged in IOHC::[INTERNAL_POISON_STATUS](#). When data poisoning is disabled, firmware should check these registers at boot to determine to which port poison was sent.

3.9.4 Global Error Status

Error status information is distributed throughout the NBIO. Each NBIO instance however, contains a set of global status registers (IOHC::[RAS_GLOBAL_STATUS_HI](#), IOHC::[RAS_GLOBAL_STATUS_LO](#)) intended to help software quickly identify whether any blocks or functions have errors that require additional processing within that NBIO instance.

Within the global status registers, most of the status bits are read-only and reflect status that is located in multiple locations. Status must be cleared in the original location before it is reflected in the global status.

The ParityErr* status bits in the global status register are Read, [Write-1-to-clear](#). These indicate internal parity, ECC, DDP and UCP errors and are set based on the programmed severity of the underlying error. Software must clear these status bits by writing them to 1 before clearing any further underlying status.

Global status registers also indicate detected [PCIe](#) errors but depending on the configuration of the PCIe root port, these indications may not persist over warm reset. Therefore it is recommended that firmware and software always check PCIe AER registers for errors after each boot.

3.9.5 Multifunction NBIO RAS pin

The NMI_SYNCFLOOD_L pin is an input pin to the processor which may be used by an external management controller to either generate an NMI to the CPU or trigger a sync flood which halts the processor.

3.10 SATA, USB, and FCH RAS Features

The SATA, USB, and [FCH](#) blocks may support features to detect, correct, and report errors. If so, the requirements for enabling and configuring these features are documented in the SATA, USB, and FCH sections of the PPR.

4 System Management Network (SMN)

4.1 Definitions

Table 147: Definitions

Term	Description
SCF	Scalable Control Fabric, synonymous with SMN .
SMN	System Management Network.

4.2 SMN Network Overview

System Management Network ([SMN](#)) is a packetized SOC network with a minimum 8-bit packet width in each direction with either synchronous or source synchronous clocking. It is expected that the packetized width of SMN may be adjusted to provide increased bandwidth over a SMN segment as appropriate to meet SOC and IP requirements. Network and IP interface partitioning is such that the IP interface is immune to changes in packet widths or other characteristics of SMN. SMN Network interface Units (NIU), routers and Source Routing requirements are mapped at build time.

SMN transactions may originate from the AMD Secure [Processor](#) ([ASP](#), formerly [PSP](#)) or Power Management microprocessor ([MP1](#)) firmware or any IP which requires SMN initiator behavior. SMN provides a standard interface which is the aggregation of SMN and CPL request into an IP.